PHP — Linguagem

Manual oficial do PHP (php.net) — Português do Brasil

Fonte oficial: https://www.php.net/manual/
Gerado para o projeto doc-frog

Começando

Introdução

O que é o PHP?

O PHP (um acrônimo recursivo para *PHP: Hypertext Preprocessor*) é uma linguagem de script open source de uso geral, muito utilizada, e especialmente adequada para o desenvolvimento web e que pode ser embutida dentro do HTML.

Ótimo, mas o que isso significa? Por exemplo:

Exemplo #1 Um exemplo introdutório

```
<!DOCTYPE html>
<html>
    <head>
        <title>Exemplo</title>
    </head>
    <body>

        <?php
            echo "Olá, eu sou um script PHP!";
        ?>

    </body>
</html>
```

Em vez de muitos comandos para mostrar HTML (como acontece com C ou Perl), as páginas PHP contém HTML em código mesclado que faz alguma coisa (neste caso, mostra Olá, eu sou um script PHP!). O código PHP é delimitado pelas instruções de processamento (tags) de início e fim `<?php` e `?>` que permitem entrar e sair do "modo PHP".

O que distingue o PHP de algo como o JavaScript no lado do cliente é que o código é executado no servidor, gerando o HTML que é então enviado para o navegador. O navegador recebe os resultados da execução desse script, mas não sabe qual era o código fonte. Um servidor web pode até mesmo ser configurado para processar todos os seus arquivos HTML com o PHP, e assim não haverá como os usuários dizerem que o PHP está sendo usado.

A melhor coisa em usar o PHP é que ele é extremamente simples para um iniciante, mas oferece muitos recursos avançados para um programador profissional. Não tenha medo de ler a longa lista de recursos do PHP. Com o PHP, praticamente qualquer pessoa pode começar a trabalhar e escrever scripts simples imediatamente.

Apesar do desenvolvimento do PHP ser focado nos scripts do lado do servidor, muito mais pode ser feito com ele. Leia mais sobre isso na seção O que o PHP pode fazer?, ou vá diretamente para o tutorial introdutório para começar a aprender sobre programação para a web.

O que o PHP pode fazer?

Qualquer coisa. O PHP é focado principalmente nos scripts do lado do servidor, portanto ele pode fazer qualquer coisa que qualquer outro programa CGI pode fazer, como coletar dados de formulários, gerar páginas com conteúdo dinâmico ou enviar e receber cookies. Mas o PHP pode fazer muito mais.

Existem duas áreas principais onde os scripts PHP são usados:

- Execução de script no servidor. Este é o mais amplamente usado e o principal campo de atuação do PHP. Três coisas são necessárias para fazer isso funcionar: o interpretador do PHP (CGI ou módulo do servidor), um servidor web e um navegador web. Todos esses podem ser executados em uma máquina local para apenas experimentar a programação com o PHP. Consulte a seção sobre instruções de instalação para mais informações.

- Execução de scripts na linha de comando. Um script PHP pode ser executado sem qualquer servidor ou navegador, apenas o interpretador PHP é necessário para ser usado desta forma. Esse tipo de uso é ideal para scripts normalmente executados usando o **cron** (em Unix ou macOS) ou o Agendador de Tarefas (no Windows). Esses scripts podem ser usados também para rotinas de processamento de texto simples. Veja a seção Utilizando o PHP em linha de comando para mais informações.

O PHP pode ser utilizado na maioria dos sistemas operacionais, incluindo Linux, várias variantes do Unix (como HP-UX, Solaris e OpenBSD), Microsoft Windows, macOS, RISC OS e provavelmente outros. O PHP também tem suporte à maioria dos servidores web atualmente. Isso inclui o Apache, o IIS e muitos outros. E isso inclui qualquer servidor web que possa utilizar o binário FastCGI do PHP, como o lighttpd e o nginx. O PHP trabalha tanto como módulo quanto como um processador CGI.

Com o PHP, portanto, os desenvolvedores têm liberdade para escolher um sistema operacional e um servidor web. Além disso, podem escolher entre utilizar programação estruturada ou programação orientada a objeto (OOP), ou ainda uma mistura das duas.

O PHP não é limitado apenas a gerar HTML. As habilidades do PHP incluem gerar tipos de arquivos complexos, como imagens ou arquivos PDF, criptografar dados e enviar emails. Ele também pode gerar facilmente qualquer texto, como JSON ou XML. O PHP consegue gerar automaticamente esses arquivos, e gravá-los no sistema de arquivos, ao invés de apenas imprimir, possibilitando criar um cache no lado do servidor para conteúdo dinâmico.

Uma das características mais fortes e mais significativas do PHP é seu suporte a uma ampla variedade de banco de dados. Escrever uma página web consultando um banco de dados é incrivelmente simples usando uma das extensões específicas de banco de dados (por exemplo, mysql), ou usando uma camada de abstração como o PDO ou conectar a qualquer banco de dados que suporte o padrão "Open Database Connection" usando a extensão ODBC. Outros bancos de dados podem utilizar cURL ou sockets, como o CouchDB.

O PHP também tem suporte para comunicação com outros serviços utilizando protocolos como LDAP, IMAP, SNMP, NNTP, POP3, HTTP, COM (no Windows) e incontáveis outros. Ele pode também abrir soquetes brutos de rede e interagir usando qualquer outro protocolo.

O PHP tem recursos úteis para processamento de texto, incluindo expressões regulares compatíveis com Perl (PCRE), e muitas outras extensões e ferramentas para analisar e acessar documentos XML. O PHP padroniza todas as extensões XML a partir da base sólida da libxml2, além de estender o conjunto de recursos adicionando suporte a SimpleXML, XMLReader e XMLWriter.

E existem muitas outras extensões interessantes, que são categorizadas tanto alfabeticamente quanto por categoria. E existem também as extensões PECL adicionais que podem, ou não, estar documentadas dentro

do próprio manual do PHP, como a » XDebug.

Esta página não é suficiente para listar todos os recursos e benefícios que o PHP pode oferecer. Leia as seções sobre a instalação do PHP, e veja a parte de referência das funções para detalhes sobre as extensões mencionadas aqui.

Um simples tutorial

Índice

Aqui nós iremos mostrar o básico do básico do PHP em um tutorial simples e curto. Este texto fala somente sobre a criação de páginas web dinâmicas com PHP, embora o PHP não seja apenas capaz de criar páginas web. Veja a seção intitulada O que o PHP pode fazer para mais informações.

Fazer páginas com PHP é o mesmo que criar páginas HTML e você pode criá-las e editá-las da mesma maneira que as páginas HTML comuns são criadas.

Sua primeira página PHP

Este tutorial assume que o PHP já está instalado. Instruções de instalação podem ser encontradas na » página de download.

Crie um novo arquivo chamado *hello.php* com o seguinte conteúdo:

Exemplo #1 Nosso primeiro script PHP: *hello.php*

```
<?php

echo "Olá, Mundo!";

?>
```

Usando o terminal, navegue até o diretório contendo este arquivo e inicie o servidor de desenvolvimento com o seguinte comando:

```
php -S localhost:8000
```

Use um navegador para acessar o arquivo com a URL do servidor web, terminando com a referência ao arquivo `/hello.php`. De acordo com o último comando executado, a URL será `http://localhost:8000/hello.php`. Se tudo estiver configurado corretamente, este arquivo será interpretado pelo PHP e a mensagem "Olá, Mundo!" será mostrada no navegador.

O PHP pode ser incorporado em uma página HTML normal. Isso significa que dentro do documento HTML podem ser escritas as instruções PHP, conforme demonstrado no exemplo a seguir:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Teste PHP</title>
    </head>
    <body>
        <?php echo '<p>Olá, Mundo!</p>'; ?>
```

```
    </body>
</html>
```

Isto resultará na seguinte saída:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Teste PHP</title>
    </head>
    <body>
        <p>Olá, Mundo!</p>
    </body>
</html>
```

Este programa é realmente simples e você não precisa do PHP para criar uma página assim (fixa). Tudo o que ela faz é mostrar: `Olá, Mundo!` utilizando a instrução echo. Note que o arquivo *não precisa ser executável* ou especial. O servidor web descobre que este arquivo precisa ser interpretado pelo PHP por causa da extensão ".php", que o servidor é configurado para repassar ao PHP. Pense nisso como um arquivo HTML normal que por acaso possui um conjunto de etiquetas especiais disponíveis para você fazer muitas coisas interessantes.

O objetivo do exemplo é mostrar o formato especial das etiquetas do PHP. Neste exemplo nós usamos `<?php` para indicar que a partir daquele ponto entramos no modo PHP. Então nós colocamos a instrução do PHP e saímos do modo PHP adicionando a etiqueta de fechamento, `?>`. Você pode entrar e sair do modo PHP num arquivo HTML desta maneira em qualquer lugar que você queira. Para mais detalhes, leia a seção do manual que fala da sintaxe básica do PHP.

> **Nota** : **Uma Nota sobre Fins de linha**
>
> Fins de linha têm pouco significado no HTML, entretanto ainda é uma boa idéia fazer o seu HTML ter uma boa aparência e organização através da divisão em linhas. Um fim de linha imediatamente após a etiqueta de fechamento `?>` será removido pelo PHP. Isso pode ser extremamente útil quando você está juntando vários blocos de PHP ou arquivos incluídos contendo apenas PHP que não deveriam imprimir nada no processo. Ao mesmo tempo isso pode ser um pouco confuso. Você pode colocar um espaço depois da tag de fechamento `?>` para forçar a impressão de um espaço e fim de linha ou você pode explicitamente imprimir o fim de linha com um último echo/print de dentro de seu bloco PHP.

> **Nota** : **Uma Nota sobre os Editores de Texto**
>
> Há muitos editores de textos e Ambientes de Desenvolvimento Integrado (IDEs - Integrated Development Enviroments) que você pode usar para criar, editar e gerenciar arquivos PHP. Uma lista parcial destas ferramentas pode ser vista na » Lista de Editores para PHP. Se você gostaria de recomendar um editor, por favor visite a página acima e peça ao administrador da página para adicionar o editor à lista. Ter um editor que destaca a sintaxe das etiquetas pode ser muito útil.

> **Nota** : **Uma Nota sobre os Processadores de Texto**
>
> Processadores de texto como o StarOffice Writer, Microsoft Word e Abiword não são boas escolhas para editar arquivos PHP. Se você deseja usar um desses para este script de teste, você precisa verificar se você está salvando o arquivo como *texto puro* ou o PHP não será capaz de ler e executar o script.

Agora que você criou com sucesso um script simples em PHP, é hora de criar o mais famoso dos scripts PHP! Uma chamada à função phpinfo() e você verá muitas informações úteis sobre seu sistema e configurações como as Variáveis Predefinidas disponíveis, módulos carregados pelo PHP, e as opções de configuração. Tire algum tempo para ver e revisar estas informações importantes.

Exemplo #2 Como obter informações sobre o PHP

```
<?php

phpinfo();

?>
```

Algo Útil

Vamos fazer alguma coisa um pouco mais útil agora. Nós iremos checar qual é o tipo de navegador que o visitante está utilizando para ver a nossa página. Para fazer isto nós iremos checar qual é o valor da string agente que o navegador envia como parte de sua requisição HTTP. Esta informação é armazenada em uma variável. Variáveis sempre começam com um símbolo de cifrão no PHP. A variável que nos interessa no momento é *$_SERVER['HTTP_USER_AGENT']*.

> **Nota** :
>
> *$_SERVER* é uma variável especial reservada do PHP que contém todas as informações sobre o servidor web. Ela é conhecida como uma superglobal. Veja a página do manual relacionada às superglobais para mais informações.

Para exibir esta variável, nós podemos fazer isto:

Exemplo #1 Imprimindo uma variável (elemento do array)

```
<?php

echo $_SERVER['HTTP_USER_AGENT'];

?>
```

Um exemplo de saída desse script poderia ser:

```
Mozilla/5.0 (Linux) Firefox/112.0
```

Há muitos tipos de variáveis disponíveis no PHP. No exemplo acima nós imprimimos um elemento do array. Arrays podem ser muito úteis.

$_SERVER é somente uma das variáveis que o PHP automaticamente disponibiliza para você. A lista destas variáveis pode ser vista na seção Variáveis Reservadas do manual ou você pode obter uma lista completa delas olhando o resultado da função phpinfo() utilizada no exemplo da seção anterior.

Você pode colocar múltiplas instruções PHP dentro da etiqueta PHP e criar pequenos blocos de códigos que fazem muito mais do que um simples 'echo'. Por exemplo, se você quiser verificar se o navegador é o Firefox você pode fazer isso:

Exemplo #2 Exemplo utilizando estruturas de controle e funções

```php
<?php

if (str_contains($_SERVER['HTTP_USER_AGENT'], 'Firefox')) {
    echo 'Você está usando o Firefox.';
}

?>
```

Um exemplo de saída desse script seria:

```
Você está usando o Firefox.
```

Aqui nós introduzimos alguns conceitos novos. Nós temos a declaração if. Se você está familiarizado com a sintaxe básica usada pela linguagem C isto parecerá ser lógico para você. Se não, você provavelmente deve ver um livro introdutório de PHP e ler os capítulos iniciais, ou ainda ler a seção Referência da Linguagem do manual.

O segundo conceito que iremos abordar é a chamada à função str_contains(). A função str_contains() vem junto com o PHP e busca uma palavra em outra palavra. Neste caso nós estamos procurando pelo texto `'Firefox'` dentro de *$_SERVER['HTTP_USER_AGENT']*. Se o dado pesquisado for encontrado na coleção, a função retorna verdadeiro. Se não, ela retorna **`false`**. Se ela retornar **`true`**, o if avalia para **`true`** e o código dentro dos {colchetes} é executado. Caso contrário o código não roda. Tente criar exemplos similares com if, else, e outras funções como strtoupper() e strlen(). Cada página relacionada contém exemplos também. Se você estiver incerto em como utilizar funções você deve ler as páginas sobre como ler uma definição de função e a seção sobre funções do PHP.

Nós podemos avançar agora e mostrar a você como alternar entre os modos PHP mesmo que você esteja executando blocos de códigos:

Exemplo #3 Mesclando entre os modos PHP e HTML

```php
<?php
if (str_contains($_SERVER['HTTP_USER_AGENT'], 'Firefox') !== false) {
    ?>
```

```
    <h3>str_contains() retornou não falso</h3>
    <p>Você está usando Firefox</p>
    <?php
} else {
    ?>
    <h3>str_contains() retornou false</h3>
    <p>Você não está usando Firefox</p>
    <?php
}
?>
```

Um exemplo de saída desse script poderia ser:

```
<h3>str_contains() retornou não falso</h3>
<p>Você está usando Firefox</p>
```

Ao invés de usar a declaração echo do PHP para imprimir a saída dos dados, nós saímos do modo do PHP e usamos o HTML normal. O importante a notar aqui é que a lógica do script continua intacta. Somente um dos blocos HTML será enviado dependendo do resultado da chamada à função str_contains(). Em outras palavras, depende se a palavra `Firefox` for encontrada ou não.

Tratando Formulários

Uma das características mais fortes do PHP é a forma como ele trata formulários HTML. O conceito básico que é importante entender é que qualquer elemento de formulário irá automaticamente ficar disponível para seus scripts PHP. Por favor leia a seção Variáveis externas do PHP para mais informações e exemplos de como usar formulários com PHP. Aqui vai um exemplo de formulário HTML:

Exemplo #1 Um simples formulário HTML

```
<form action="action.php" method="post">
    <label for="nome">Seu nome:</label>
    <input name="nome" id="nome" type="text">

    <label for="idade">Sua idade:</label>
    <input name="idade" id="idade" type="number">

    <button type="submit">Enviar</button>
</form>
```

Não há nada de especial neste formulário. É um formulário HTML comum sem nenhuma etiqueta especial de qualquer tipo. Quando o usuário preencher este formulário e clicar no botão "Enviar", a página *action.php* é chamada. Neste arquivo nós teremos algo como isto:

Exemplo #2 Imprimindo dados de nosso formulário

```
Olá, <?php echo htmlspecialchars($_POST['nome']); ?>.
Você tem <?php echo (int) $_POST['idade']; ?> anos de idade.
```

Um exemplo de saída deste script seria:

```
Olá, Maria. Você tem 22 anos de idade.
```

Para além de htmlspecialchars() e `(int)`, deve ser óbvio o que o script faz. htmlspecialchars() transforma caracteres que sejam especiais no HTML na sua forma codificada, de forma que não seja possível injetar etiquetas HTML ou JavaScript em sua página. O campo "idade", por ser um número, pode ser simplesmente convertido para um int que automaticamente eliminará qualquer caractere estranho. Você também pode fazer o PHP automaticamente fazer isso utilizando a extensão filter. As variáveis *$_POST['nome']* e *$_POST['idade']* são criadas automaticamente pelo PHP. Anteriormente utilizamos a superglobal *$_SERVER*; acima mostramos que a superglobal *$_POST* contém todos os dados POST. Perceba como o *método* (method) do formulário é POST. Se fosse utilizado o método *GET*, os dados do formulário acabariam na superglobal *$_GET*. Você também pode utilizar a superglobal *$_REQUEST*, se não se importar com a origem do dado enviado. Ela conterá os dados mesclados das origens GET, POST e COOKIE.

O que vem a seguir?

Com este novo conhecimento fica fácil entender a maior parte do manual.

Em particular, você pode explorar os seguintes recursos:

- Ler e gravar arquivos com as funções de sistema de arquivos
- Lidar com envio de arquivos
- Obter páginas e arquivos remotos com o Curl
- Armazenar e analisar dados em um banco de dados com o PDO (SQLite pode ser usado sem executar um servidor de banco de dados)
- Manter dados persistentes através das Sessões

Existem várias bibliotecas e » frameworks para cada ocasião no » repositório Packagist, todas instaláveis pelo gerenciador de pacotes Composer.

Para ajuda e conselhos da comunidade, consulte a » página de Ajuda.

Para uma variedade de podcasts, apresentações e outros vídeos, confira a » comunidade PeerTube.

Outros recursos da comunidade que podem te ajudar incluem "listas incríveis" (diretórios selecionados de links) e "roteiros para desenvolvedores" (listas de tópicos relacionados).

Quando você estiver sem saber por onde começar, tente dividir seu projeto ou problema em partes menores, o que permitirá que você veja mais facilmente o que já sabe fazer e o que precisa aprender. A lista pode ser tão extensa/detalhada quanto você precisar. Por exemplo, a criação de um blog pode ser dividida nas seguintes partes:

- Listando e visualizando páginas
    - Lendo registros (páginas) de um banco de dados
- Criando páginas

    - Lidando com submissão de formulários
    - Gravando registros (páginas) em um banco de dados
- Login do administrador
    - Lendo registros (usuários) de um banco de dados
    - Lidando com senhas
    - Persistência de dados (login do usuário) entre requisições/páginas (sessões)

Se não houver nada específico que você queira construir, pode tentar procurar exercícios de programação como katas, desafios e "golfe de código". Mesmo que não sejam especificamente voltados para PHP, a maioria deve ser possível de concluir e provavelmente desafiará seu conhecimento e raciocínio.

Referência da Linguagem

Sintaxe Básica

Índice

Tags PHP

Quando o PHP processa um arquivo, ele reconhece as etiquetas de abertura e fechamento, `<?php` e `?>`, para definir os limites da execução de código PHP. O conteúdo fora destas etiquetas é ignorado pelo interpretador PHP, permitindo que o PHP seja incluído em vários tipos de documentos.

Um caractere de espaço em branco (espaço, tabulação ou nova linha) precisa estar presente logo após `<?php` para assegurar uma separação correta de tokens. Omitir este espaço resultará em erro de sintaxe.

O PHP também inclui a etiqueta `<?=` que é um encurtamento para `<?php echo`.

Exemplo #1 Tags de Abertura e Fechamento do PHP

```
1.  <?php echo 'Se quiser servir código PHP em documentos XHTML ou XML,
                use essas tags'; ?>

2.  A tag echo curta <?= 'imprima essa string' ?> também pode ser usada.
    Ela é equivalente a <?php echo 'imprima essa string' ?>.

3.  <? echo 'Este código está entre tags curtas, mas só funcionará '.
            'se a diretiva short_open_tag estiver habilitada'; ?>
```

As tags curtas (a terceira linha no exemplo acima) estão disponíveis por padrão mas podem ser desabilitadas através da diretiva short_open_tag no arquivo de configuração *php.ini*, ou estarão desabilitadas por padrão se o PHP for compilado com a configuração **--disable-short-tags**.

> **Nota** :
>
> Como as tags curtas podem ser desabilitadas, é recomendado usar apenas as tags normais (`<?php ?>` e `<?= ?>`) para maximizar a compatibilidade.

Se um arquivo terminar com código PHP, é preferível omitir a tag de fechamento do PHP no final do arquivo. Isso evita que espaços em branco ou novas linhas sejam adicionados acidentalmente após a tag de fechamento do PHP, o que pode causar efeitos indesejados, porque o PHP iniciará o buffer de saída quando não houver intenção de enviar qualquer saída naquele ponto do script.

Exemplo #2 Arquivo apenas com código PHP

```
<?php
echo "Olá Mundo\n";

// ... mais código
```

```php
echo "Última instrução\n";

// o script termina aqui sem a tag de fechamento do PHP
```

Escapando o HTML

Tudo fora de um par de tags de abertura e fechamento é ignorado pelo interpretador PHP, o que permite que arquivos PHP tenham conteúdo misto. Isso permite que o PHP seja incluído em documentos HTML, por exemplo, para criar templates.

Exemplo #1 Incorporando o PHP no HTML

```php
<p>Isso será ignorado pelo PHP e exibido pelo navegador.</p>
<?php echo 'Enquanto isto vai ser interpretado.'; ?>
<p>Isso também será ignorado pelo PHP e exibido no navegador.</p>
```

Isso funcionará como esperado, porque quando o interpretador PHP encontra a tag de fechamento ?>, ele simplesmente começa a repassar o que quer que encontre (exceto a nova linha imediata, veja a seção sobre separação de instruções), até encontrar outra tag de abertura, a menos que esteja no meio de uma declaração condicional, caso em que o interpretador determinará o resultado da condicional antes de decidir qual caminho tomar. Veja o próximo exemplo.

Usando estruturas com condições

Exemplo #2 Escape avançado usando condições

```php
<?php if (true): ?>
  Isso irá aparecer se a expressão for verdadeira.
<?php else: ?>
  Senão isso irá aparecer.
<?php endif; ?>
```

Neste exemplo, o PHP pulará os blocos onde a condição não for satisfeita, mesmo que os trechos de código estejam fora das tags de abertura e fechamento do PHP, pois o interpretador PHP pulará os blocos contidos em uma condição que não foi satisfeita.

Para imprimir grandes blocos de texto, sair do modo de interpretação do PHP é geralmente mais eficiente do que enviar todo o texto usando echo ou print.

> **Nota** :
>
> Se o PHP for incluído no XML ou XHTML, as tags normais `<?php ?>` devem ser usadas para manter a conformidade com os padrões.

Separação de instruções

Como em C ou Perl, o PHP requer que as instruções sejam terminadas com um ponto e vírgula no final de cada declaração. A tag de fechamento de um bloco de código PHP implica automaticamente em um ponto e vírgula; a última linha de um bloco PHP não precisa ser terminada com um ponto e vírgula. A tag de fechamento do bloco incluirá uma nova linha em seguida, se estiver presente.

Exemplo #1 Exemplo mostrando a tag de fechamento incluindo uma nova linha final

```
<?php echo "Algum texto"; ?>
Sem nova linha
<?= "Mas agora há uma nova linha" ?>
```

O exemplo acima produzirá:

```
Algum textoSem nova linha
Mas agora há uma nova linha
```

Exemplo #2 Exemplos de entrada e saída do interpretador PHP

```
<?php
    echo "Isto é um teste\n";
?>

<?php echo "Isto é um teste\n" ?>

<?php echo "A última tag de fechamento foi omitida\n";
```

> **Nota** :
>
> A tag de fechamento de um bloco PHP no final de um arquivo é opcional e, em alguns casos, omiti-la é útil ao usar include ou require, assim espaços em branco indesejados não ocorrerão no final dos arquivos, e ainda será possível adicionar cabeçalhos à resposta posteriormente. Isso também é útil ao usar o buffer de saída, para evitar a inclusão de espaços em branco indesejados no final das partes geradas pelos arquivos incluídos.

Comentários

O PHP suporta comentários no estilo C, C++ e Unix shell (estilo Perl). Por exemplo:

Exemplo #1 Comentários

```
<?php
    echo "Isto é um teste\n"; // Este é um comentário de uma linha no estilo C++
    /* Este é um comentário de várias linhas
       mais uma linha de comentário */
    echo "Este é mais um teste\n";
    echo "Um teste final\n"; # Este é um comentário de uma linha no estilo shell
?>
```

Os estilos de comentário de uma linha comentam apenas até o final da linha ou do bloco atual de código PHP, o que ocorrer primeiro. Isso significa que o código HTML após `// ... ?>` ou `# ... ?>` SERÁ impresso: `?>` sai do modo PHP e retorna ao modo HTML, e `//` ou `#` não podem influenciar isso.

Exemplo #2 Comentários de uma linha

```
<h1>Este é um <?php # echo 'simples';?> exemplo</h1>
<p>O cabeçalho acima dirá 'Este é um exemplo'.</p>
```

Os comentários no estilo C terminam no primeiro `*/` encontrado. Certifique-se de não aninhar comentários no estilo C. É fácil cometer esse erro ao tentar comentar um grande bloco de código.

```
<?php
 /*
    echo 'Isto é um teste'; /* Este comentário causará um problema */
 */
?>
```

Tipos

Índice

Introdução

Cada expressão no PHP tem um dos seguintes tipos ou valores internos:

- null
- bool
- int
- float (número de ponto flutuante)
- string
- array
- object
- callable
- resource

O PHP é uma linguagem com tipagem dinâmica, o que significa que por padrão não existe a necessidade de especificar o tipo de uma variável, já que isso será determinado em tempo de execução. Entretanto, é possível restringir estaticamente os tipos através do uso de declarações de tipo. Os diferentes tipos suportados pelo PHP podem ser encontrados na página de sistema de tipos.

Os tipos restringem quais operações podem ser realizadas nos valores. Entretanto, se uma expressão ou variável for utilizada numa operação que o tipo não suporte, o PHP tentará transformar o valor em um tipo que suporte a operação. Esse processo depende do contexto de onde o valor é utilizado. Para mais informações, veja a seção sobre Transformações de tipos.

> **Dica**

> A tabela de comparação de tipos também pode ser útil, pelos vários exemplos de comparação de valores demonstrados.

> **Nota** :
>
> É possível forçar a reavaliação de uma expressão para um tipo específico através do uso de cast. Uma variável pode ser modificada para um tipo através da função settype().

Para verificar o valor e tipo de uma expressão, utilize a funçãovar_dump(). Para extrair o tipo de uma expressão, utilize a função get_debug_type(). Para apenas validar se uma expressão é de um certo tipo, utilize as funções `is_type`.

Exemplo #1 Tipos diferentes

```php
<?php
$a_bool = true;   // um valor booleano
$a_str  = "foo";  // um texto
$a_str2 = 'foo';  // um texto
$an_int = 12;     // um inteiro

echo get_debug_type($a_bool), "\n";
echo get_debug_type($a_str), "\n";

// Se essa variável contiver um inteiro, aumenta o número em quatro
if (is_int($an_int)) {
    $an_int += 4;
}
var_dump($an_int);

// Se $a_bool for um texto, exibe
if (is_string($a_bool)) {
    echo "String: $a_bool";
}
?>
```

Saída do exemplo acima no PHP 8:

```
bool
string
int(16)
```

> **Nota** :
>
> get_debug_type() tornou-se disponível no PHP 8.0.0. Antes dessa versão, a função gettype() podia ser utilizada. Entretanto, a função antiga não retorna nomes bem definidos.

Sistema de tipagem

O PHP usa um sistema de tipagem nominal com uma forte relação de subtipagem comportamental. A relação de subtipagem é verificada em tempo de compilação, enquanto a verificação de tipos é verificada dinamicamente em tempo de execução.

O sistema de tipos do PHP suporta vários tipos atômicos que podem ser compostos juntos para criar tipos mais complexos. Alguns desses tipos podem ser escritos como declarações de tipo

Tipos atômicos

Tipos atômicos são tipos internos que são fortemente integrados com a linguagem e não podem ser reproduzidos com tipos definidos pelo usuário.

A lista de tipos básicos é:

- Tipos integrados
    - Tipos escalares:
        - tipo bool
        - tipo int
        - tipo float
        - tipo string
    - tipo array
    - tipo object
    - tipo resource
    - tipo never
    - tipo void
    - Tipos de classes relativas: self, parent, e static
    - Tipos únicos
        - false
        - true
    - Tipos de unidade:
        - null
- Tipos definidos pelo usuário (geralmente chamados de tipos de classe)
    - Interfaces
    - Classes
    - Enumerações

- tipo callable

Tipos escalares

Um valor é considerado escalar se for do tipo int, float, string ou bool.

Tipos definidos pelo usuário

É possível definir tipos personalizados com interfaces, classes e enumerações. Essas são consideradas tipos definidos pelo usuário, ou tipos-classe. Por exemplo, uma classe chamada `Elephant` pode ser definida, em seguida objetos do tipo `Elephant` podem ser instanciados, e uma função pode requerer um parâmetro do tipo `Elephant`.

Tipos compostos

É possível combinar vários tipos atômicos em tipos compostos. O PHP permite que os tipos sejam combinados das seguintes maneiras:

- Interseção de tipos de classes (interfaces e nomes de classes).
- União de tipos.

Tipos de interseção

Um tipo de interseção aceita valores que satisfazem várias declarações de tipo de classe, em vez de uma única. Os tipos individuais que formam o tipo de interseção são unidos pelo símbolo `&`. Portanto, um tipo de interseção composto pelos tipos `T`, `U`, e `V` será escrito como `T&U&V`.

Tipos de união

Um tipo de união aceita valores de vários tipos diferentes, em vez de um único. Tipos individuais que formam o tipo de união são unidos pelo `|` símbolo. Portanto, um tipo de união composto pelos tipos `T`, `U`, e `V` será escrito como `T|U|V`. Se um dos tipos for um tipo de interseção, ele precisa ser colocado entre parênteses para que seja escrito em DNF: `T|(X&Y)`.

Alias de tipo

O PHP suporta dois aliases de tipo: mixed e iterable que corresponde ao tipo de união de `object|resource|array|string|float|int|bool|null` e `Traversable|array`, respectivamente.

> **Nota** :
>
> O PHP não suporta aliases de tipo definidos pelo usuário.

NULL

O tipo null é um tipo unitário do PHP, ou seja, possui apenas um valor: **`null`**.

Variáveis indefinidas e com definição removida por unset() serão interpretadas como **`null`**.

Sintaxe

Há apenas um valor do tipo null, que é a a constante **`null`**, insensível a maiúsculas/minúsculas.

```php
<?php
$var = NULL;
?>
```

Convertendo para `null`

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 7.2.0 e foi *REMOVIDO* a partir do PHP 8.0.0. O uso deste recurso é fortemente desencorajado.

Converter uma variável para null utilizando `(unset) $var` *não* removerá nem a variável nem o seu valor. Apenas retornará um valor **`null`**.

Veja também

- is_null()
- unset()

Booleanos

O tipo bool tem apenas dois valores e é usado para expressar um valor de verdade. Pode ser **`true`** ou **`false`**.

Sintaxe

Para especificar um bool literal, use as palavras-chave **`true`** ou **`false`**. Ambas são case-insensitive.

```php
<?php

$foo = True; // atribui o valor True para $foo

var_dump($foo); // bool(true)
?>
```

Tipicamente, o resultado de um operador que retorne um valor bool, é passado para uma estrutura de controle.

```php
<?php

$acao = "mostrar_versao";
$exibir_separadores = true;

// == É um operador que testa
// igualdade e retorna um booleano.
if ($acao == "mostrar_versao") {
    echo "A versão é 1.23";
}

// isto não é necessário ...
if ($exibir_separadores == TRUE) {
    echo "<hr>\n";
}

// ... porque pode simplesmente ser escrito assim:
```

```php
if ($exibir_separadores) {
    echo "<hr>\n";
}
?>
```

Convertendo para booleano

Para converter explicitamente um valor em bool, use a conversão `(bool)`. Geralmente isso não é necessário porque quando um valor é usado em um contexto lógico ele será automaticamente interpretado como um valor do tipo bool. Para obter mais informações, consulte a página Manipulação de tipos.

Ao converter para bool, os seguintes valores são considerados **`false`**:

- o próprio booleano **`false`**
- o inteiro `0` (zero)
- os pontos flutuantes `0.0` e `-0.0` (zero)
- uma string vazia `""`, e a string `"0"`
- um array sem elementos
- o tipo especial NULL (incluindo variáveis não definidas)
- Objetos internos que sobrecarregam seu comportamento de conversão para booleano. Por exemplo, objetos GMP que representam o valor `0`.

Todos os outros valores são considerados **`true`** (incluindo resource e **`NAN`**).

> **Aviso**
>
> `-1` é considerado **`true`**, como qualquer número que não seja zero (negativos ou positivos)!

Exemplo #1 Convertendo para booleano

```php
<?php
var_dump((bool) "");        // bool(false)
var_dump((bool) "0");       // bool(false)
var_dump((bool) 1);         // bool(true)
var_dump((bool) -2);        // bool(true)
var_dump((bool) "foo");     // bool(true)
var_dump((bool) 2.3e5);     // bool(true)
var_dump((bool) array(12)); // bool(true)
var_dump((bool) array());   // bool(false)
var_dump((bool) "false");   // bool(true)
?>
```

Inteiros

Um int é um número do conjunto $\mathbb{Z}$ = {..., -2, -1, 0, 1, 2, ...}.

Veja também

- Números de ponto flutuante

- Precisão arbitrária / BCMath
- Número inteiro de comprimento arbitrário / GMP

Sintaxe

Int pode ser especificado em notação decimal (base 10), hexadecimal (base 16), octal (base 8) ou binária (base 2). O sinal de menos pode ser usado para indicar um int negativo.

Para usar a notação octal, preceda o número com um `0` (zero). A partir do PHP 8.1.0, a notação octal também pode ser precedida por `0o` ou `0O`. Para utilizar a notação hexadecimal, preceda o número com `0x`. Para utilizar a notação binária, preceda o número com `0b`.

A partir do PHP 7.4.0, inteiros literais podem conter sublinhados (_) entre os dígitos, para melhorar a sua leitura. Estes sublinhados serão removidos pelo analisador do PHP.

Exemplo #1 Inteiros literais

```
<?php
$a = 1234; // número decimal
$a = 0123; // número octal (equivalente a 83 em decimal)
$a = 0o123; // número octal (a partir do PHP 8.1.0)
$a = 0x1A; // número hexadecimal (equivalente a 26 em decimal)
$a = 0b11111111; // número binário (equivalente a 255 em decimal)
$a = 1_234_567; // número decimal (a partir do PHP 7.4.0)
?>
```

Formalmente, a estrutura para int literais é esta a partir do PHP 8.1.0 (anteriormente, os prefixos octal `0o` ou `0O` não eram permitidos, e antes do PHP 7.4.0 os sublinhados não eram permitidos):

```
decimal     : [1-9][0-9]*(_[0-9]+)*
            | 0

hexadecimal : 0[xX][0-9a-fA-F]+(_[0-9a-fA-F]+)*

octal       : 0[oO]?[0-7]+(_[0-7]+)*

binário     : 0[bB][01]+(_[01]+)*

inteiro     : decimal
            | hexadecimal
            | octal
            | binário
```

O tamanho de um int depende da plataforma, embora um valor máximo de cerca de dois bilhões seja o valor usual (são 32 bits com sinal). As plataformas de 64 bits geralmente têm um valor máximo de cerca de 9E18. O PHP não suporta int sem sinal. O tamanho int pode ser determinado usando a constante **`PHP_INT_SIZE`**, o valor máximo usando a constante **`PHP_INT_MAX`** e o valor mínimo usando a constante **`PHP_INT_MIN`**.

Overflow de inteiros

Se o PHP encontrar um número além dos limites do tipo int, ele será interpretado como um float. Assim, uma operação que resulte em um número além dos limites do tipo int, retornará um float.

Exemplo #2 Estouro de inteiros

```php
<?php
$large_number = 50000000000000000000;
var_dump($large_number);         // float(5.0E+19)

var_dump(PHP_INT_MAX + 1);       // sistemas 32 bits: float(2147483648)
                                 // sistemas 64 bits: float(9.2233720368548E+18)
?>
```

Divisão de inteiros

Não existe operador de divisão int no PHP, para isso deve ser usada a função intdiv(). `1/2` produz o float `0.5`. O valor pode ser convertido em um int para arredondar para zero ou pode ser usada a função round() para um controle mais preciso sobre o arredondamento.

Exemplo #3 Divisões

```php
<?php
var_dump(25/7);         // float(3.5714285714286)
var_dump((int) (25/7)); // int(3)
var_dump(round(25/7));  // float(4)
?>
```

Convertendo para inteiro

Para converter explicitamente um valor para int, use a conversão `(int)`. Entretanto, na maioria dos casos, o modificador não é necessário, já que o valor será automaticamente convertido se um operador, função ou estrutura de controle requerer um int como argumento. Um valor também pode ser convertido para int utilizando a função intval().

Se um resource for convertido para um int, resultará no número único do recurso, atribuído ao resource pelo PHP em tempo de execução.

Veja também Manipulação de Tipos.

Convertendo a partir de booleanos

`false` será retornado como `0` (zero), e `true` retornará `1` (um).

Convertendo a partir de números de ponto flutuante

Ao converter de float para int, o número será arredondado *em direção ao zero*. A partir do PHP 8.1.0, um aviso de descontinuação é emitido quando ocorre a conversão implícita de um float não integral para int que perca precisão.

Exemplo #4 Convertendo a partir de float

```php
<?php

function foo($value): int {
  return $value;
}

var_dump(foo(8.1)); // Descontinuado: conversão implícita de float 8.1 para int perdendo precisão a
partir do PHP 8.1.0
var_dump(foo(8.1)); // 8 antes do PHP 8.1.0
```

```php
var_dump(foo(8.0)); // 8 em ambos os casos

var_dump((int) 8.1); // 8 em ambos os casos
var_dump(intval(8.1)); // 8 em ambos os casos
?>
```

Se o float estiver além dos limites de int (geralmente `+/- 2.15e+9 = 2^31` em plataformas de 32 bits e `+/- 9.22e+18 = 2^63` em plataformas de 64 bits), o resultado é indefinido, já que o float não tem precisão suficiente para fornecer um resultado int exato. Nenhum alerta, nem mesmo uma nota, será emitido quando isso acontecer!

> **Nota** :
>
> `NaN`, `Inf` e `-Inf` sempre serão zero quando convertidos para int.

> **Aviso**
>
> Nunca converta uma fração desconhecida para int porque isto pode, algumas vezes, levar a resultados inesperados.
>
> ```php
> <?php
> echo (int) ( (0.1+0.7) * 10 ); // mostra 7!
> ?>
> ```
>
> Veja também o alerta sobre a precisão de número flutuante.

Convertendo a partir de strings

Se a string for numérica ou inicialmente numérica, ela será convertida para o valor inteiro correspondente, caso contrário, será convertida para zero (`0`).

Convertendo a partir de NULL

`null` é sempre convertido para zero (`0`).

Convertendo a partir de outros tipos

> **Cuidado**
>
> O comportamento da conversão de um int é desconhecido para outros tipos. *Não* se deve depender de nenhum comportamento observado, pois ele pode ser alterado sem aviso.

Números de ponto flutuante

Números de ponto flutuante (também conhecidos como "floats", "doubles" ou "números reais"), podem ser especificados utilizando qualquer uma das seguintes sintaxes:

```php
<?php
$a = 1.234;
$b = 1.2e3;
$c = 7E-10;
```

```php
$d = 1_234.567; // a partir do PHP 7.4.0
?>
```

Formalmente a partir do PHP 7.4.0 (anteriormente, sublinhados não eram permitidos):

```
LNUM          [0-9]+(_[0-9]+)*
DNUM          ({LNUM}?"."{LNUM}) | ({LNUM}"."{LNUM}?)
EXPONENT_DNUM (({LNUM} | {DNUM}) [eE][+-]? {LNUM})
```

O tamanho de um número de ponto flutuante depende da plataforma, sendo o máximo de ~1.8e308 com precisão de 14 dígitos decimais um valor comum (número de 64 bits no formato IEEE).

> **Aviso**
>
> ### Precisão de números de ponto flutuante
>
> Números de ponto flutuante tem precisão limitada. Embora dependa do sistema, o PHP geralmente utiliza o formato de precisão dupla do IEEE 754, que trará uma precisão máxima devida a arredondamentos da ordem de 1.11e-16. Operações matemáticas incomuns poderão ocasionar erros maiores, e, claro, a propagação de erros deve ser considerada quando várias operações forem realizadas.
>
> Além disso, números racionais que têm representação exata em números em base 10, como `0.1` ou `0.7`, não possuem representação exata em ponto flutuante na base 2, o formato utilizado internamente, não importando o tamanho da mantissa. Portanto, não existe conversão para o formato interno sem uma pequena perda de precisão. Isso pode ocasionar resultados confusos: por exemplo, `floor((0.1+0.7)*10)` normalmente retornará `7`, em vez do resultado esperado `8`, porque a representação interna final será algo como `7.9999999999999991118...`.
>
> Então, nunca confie em resultados com números de ponto flutuante até a última casa, e nunca compare números de ponto flutuante em igualdades. Se você realmente precisar de alta precisão, você pode utilizar as funções matemáticas de precisão arbitrária e as funções gmp estão disponíveis.
>
> Para uma explicação "simples" dessa questão, veja o » guia sobre ponto flutuante, que também tem o título alternativo de "Porque meus números não somam direito?".

Convertendo para float

A partir de strings

Se a string é numérica ou começa numérica então ela irá resolver para o valor float correspondente, do contrário ela é convertida em zero (`0`).

A partir de outros tipos

Para valores de outros tipos, a conversão é realizada convertendo o valor para int primeiro e em seguida para float. Veja Convertendo para inteiro para mais informações.

> **Nota** :
>
> Como certos tipos têm comportamento indefinido ao converter para int, este também é o caso ao converter para float.

Comparando floats

Como notado acima, testar números de ponto flutuante com igualdade é problemático, por causa da maneira como são representados internamente. Entretanto existem maneiras de fazer comparações com números de ponto flutuante que contornam essas limitações.

Para testar números de ponto flutuante, utilize um "valor de erro máximo" na comparação utilizada. Esse valor é também chamado de epsilon, ou unidade de erro, e deve ser a diferença mínima aceitável no resultado dos cálculos.

$a e *$b* serão consideradas iguais até o 5º dígito de precisão.

Exemplo #1 Comparando números de ponto flutuante

```php
<?php
$a = 1.23456789;
$b = 1.23456780;
$epsilon = 0.00001;

if (abs($a - $b) < $epsilon) {
    echo "iguais";
}
?>
```

NaN

Algumas operações numéricas podem resultar em valores representados pela constante `NAN`. Esse resultado representa um valor desconhecido ou não representável nos cálculos de ponto flutuante. Qualquer comparação frouxa ou restrita deste valor com qualquer outro, inclusive ele mesmo, com exceção de `true`, terá como resultado `false`.

Como o `NAN` representa um resultado irrepresentável, `NAN` não deve ser comparado com outros valores, incluindo ele mesmo, em vez disso, deve-se checá-lo utilizando a função is_nan().

Strings

Uma string é uma série de caracteres, onde um caractere é o mesmo que um byte. Isso significa que o PHP possui suporte a um conjunto de apenas 256 caracteres, e, portanto, não possui suporte nativo a Unicode. Veja mais detalhes do tipo string.

> **Nota** :
>
> Em sistemas de 32 bits, uma string pode ter até 2GB (2147483647 bytes) de comprimento.

Sintaxe

Uma string literal pode ser especificada de quatro formas diferentes.

- aspas simples
- aspas duplas
- sintaxe heredoc

- sintaxe nowdoc

Aspas simples

A maneira mais simples de se especificar uma string é delimitá-la entre aspas simples (o caractere `'`).

Para especificar um apóstrofo, escape-o com uma contrabarra (`\`). Para especificar uma contrabarra literal, duplique-a (`\\`). Todas as outras ocorrências da contrabarra serão tratadas como uma contrabarra literal: isso significa que outras sequências de escape que se esteja acostumado a utilizar, como `\r` ou `\n`, serão literalmente impressas em vez de ter qualquer significado especial.

> **Nota** :
>
> Diferentemente das sintaxes com aspas duplas e heredoc, variáveis e sequências de escape para caracteres especiais *não* serão expandidas quando ocorrerem dentro de uma string delimitada por aspas simples.

Exemplo #1 Variantes de sintaxe

```php
<?php
echo 'isto é uma string comum', PHP_EOL;

echo 'Você pode incluir novas linhas em strings
dessa maneira que estará
tudo bem', PHP_EOL;

// Exibe: Arnold disse uma vez: "I'll be back"
echo 'Arnold disse uma vez: "I\'ll be back"', PHP_EOL;

// Exibe: Você excluiu C:\*.*?
echo 'Você excluiu C:\\*.*?', PHP_EOL;

// Exibe: Você excluiu C:\*.*?
echo 'Você excluiu C:\*.*?', PHP_EOL;

// Exibe: Isto não será substituído por \n uma nova linha
echo 'Isto não será substituído por \n uma nova linha', PHP_EOL;

// Exibe: Variáveis $também não $expandem
echo 'Variáveis $também não $expandem', PHP_EOL;
?>
```

Aspas duplas

Se a string for delimitada entre aspas duplas (`"`), o PHP interpretará a seguinte sequência de escape como caracteres especiais:

Sequências de escape

Sequências	Significado
`\n`	Nova linha (LF ou 0x0A (10) em ASCII)
`\r`	Retorno de carro (CR ou 0x0D (13) em ASCII)

Sequências	Significado
`\t`	Tabulação horizontal (HT ou 0x09 (9) em ASCII)
`\v`	Tabulação vertical (VT ou 0x0B (11) em ASCII)
`\e`	Escape (ESC ou 0x1B (27) em ASCII)
`\f`	Form feed (FF ou 0x0C (12) em ASCII)
`\\`	contrabarra ou barra invertida
`\$`	Sinal de cifrão
`\"`	aspas duplas
`\[0-7]{1,3}`	Octal: uma sequência de caracteres que coincide com a expressão regular `[0-7]{1,3}` é um caractere em notação octal (por exemplo, `"\101" === "A"`), o qual é silenciosamente turcado para um byte (por exemplo `"\400" === "\000"`)
`\x[0-9A-Fa-f]{1,2}`	Hexadecimal: uma sequência de caracteres que coincide com a expressão regular `[0-9A-Fa-f]{1,2}` é um caractere em notação decimal (por exemplo, `"\x41" === "A"`)
`\u{[0-9A-Fa-f]+}`	Unicode: uma sequência de caracteres coincidindo com a expressão regular `[0-9A-Fa-f]+` é um codepoint Unicode, o qual terá como resultado uma string contendo a representação UTF-8 desses codepoints. Os colchetes são requeridos. Por exemplo, `"\u{41}" === "A"`

Como com as strings entre aspas simples, escapar qualquer outro caractere resultará em uma contrabarra sendo impressa.

O recurso mais importante de strings delimitadas por aspas duplas é o fato de que nomes de variáveis serão expandidos. Veja a interpolação de strings para detalhes.

Heredoc

Uma terceira maneira de delimitar strings é a sintaxe heredoc: <<<. Após este operador, um marcador é fornecido seguido de uma nova linha. A própria string é colocada em seguida e a seguir o mesmo marcador novamente para fechar a string.

O marcador de fechamento pode ser indentado com espaços ou tabulações. Espaços iguais à indentação do marcador de fechamento são removidos do início de cada linha da string heredoc. Anteriormente ao PHP 7.3.0, o marcador *precisava* estar no começo da linha.

Além disso, o marcador de fechamento precisa seguir as mesmas convenções de nomes de outros identificadores do PHP: ele precisa conter apenas caracteres alfanuméricos e sublinhados, e precisa começar com uma letra ou sublinhado.

Exemplo #2 Exemplo Heredoc simples no PHP 7.3.0

```
<?php
// Marcador de fechamento não indentado
echo <<<END
      a
     b
    c
\n
END;

// Marcador de fechamento indentado com 3 espaços; 3 espaços removidos de cada linha
```

```
echo <<<END
      a
     b
    c
   END;
```

Saída do exemplo acima no PHP 7.3:

```
      a
     b
    c

   a
  b
 c
```

Se o marcador de fechamento estiver identado diferentemente das linhas do corpo, um ParseError será lançado:

Exemplo #3 Marcador de fechamento indentado diferente das linhas do corpo

```
<?php
echo <<<END
  a
 b
c
   END;
```

Saída do exemplo acima no PHP 7.3:

```
Parse error: Invalid body indentation level (expecting an indentation level of at least 3) in
example.php on line 4
```

Se o marcador de fechamento é identado, tabulações podem ser utilizadas. No entanto, tabulações e espaços *não podem* estar misturados. A identação do corpo e o marcador de fechamento precisam usar um ou outro consistentemente. Se espaços e tabulações forem misturados, um ParseError será lançado. Essas restrições de whitespace estão incluídas dados que a mistura de tabulações e espaços são prejudiciais à legibilidade.

Exemplo #4 Indentação diferente para corpo e marcador de fechamento

```
<?php
// Todos esses códidos falharão.

// Indentação diferente para corpo (espaços) e marcador de fechamento (tabulações)
{
echo <<<END
```

```
a
END;
}

// Misturar tabulações e espaços no corpo
{
    echo <<<END
    a
     END;
}

// Misturar espaços e tabulações no identificador de fechamento
{
echo <<<END
a
END;
}
```

Saída do exemplo acima no PHP 7.3:

```
Parse error: Invalid indentation - tabs and spaces cannot be mixed in example.php line 8
```

O identificador de fechamento do corpo não necessita de um ponto e vírgula ou nova linha. Por exemplo, o seguinte código é permitido a partir do PHP7.3.0:

Exemplo #5 Continuando uma expressão após o identificador de fechamento

```
<?php
$values = [<<<END
a
  b
    c
END, 'd e f'];
var_dump($values);
```

Saída do exemplo acima no PHP 7.3:

```
array(2) {
  [0] =>
  string(11) "a
  b
    c"
  [1] =>
  string(5) "d e f"
}
```

> **Aviso**
>
> Se o marcador de fechamento for encontrado no começo da linha, existe não importando se for parte de outra palavra, será considerado como marcador de fechamento e causará um ParseError.
>
> **Exemplo #6 Marcador de fechamento no corpo da string tende a causar um ParseError**
>
> ```
> <?php
> $values = [<<<END
> a
> b
> END ING
> END, 'd e f'];
> ```
>
> Saída do exemplo acima no PHP 7.3:
>
> ```
> Parse error: syntax error, unexpected identifier "ING", expecting "]" in example.php on line
> 5
> ```
>
> Para evitar esse problema, basta seguir a seguinte regra: *escolha identificadores que não aparecem no corpo do texto*.

> **Aviso**
>
> Anteriormente ao PHP 7.3.0, era muito importante notar que a linha com o identificador de fechamento não poderia conter outros caracteres, exceto um ponto e vírgula (`;`). Isso significava que o identificador de fechamento *não poderia ser identado*, e que não poderia haver espaços ou tabulações antes ou depois do ponto e vírgula. É importante notar também que o primeiro caractere antes do fechamento deveria ser uma newline, como definido pelo sistema operacional. Ou seja, `\n` em sistemas Unix, incluindo o macOS. O identificador de fechamento também deveria ser seguido de um newline.
>
> Se essa regra não for seguida, e o identificador de fechamento não estiver "limpo", ele não será considerado como fechamento, e o PHP continuará lendo o script como se fosse texto. Se um identificador de fechamento correto não for encontrado até o final do arquivo, um erro de interpretação será lançado na última linha.
>
> **Exemplo #7 Exemplo inválido, antes do PHP 7.3.0**
>
> ```
> <?php
> class foo {
>     public $bar = <<<EOT
> bar
>     EOT;
> }
> // O identificador não deve ser indentado
> ?>
> ```
>
> **Exemplo #8 Exemplo válido, mesmo antes do PHP 7.3.0**

```php
<?php
class foo {
    public $bar = <<<EOT
bar
EOT;
}
?>
```

Heredocs não podem ser usados para inicializar propriedades de classe.

Textos heredoc se comportam como strings delimitadas por aspas duplas, sem as aspas duplas. Isso significa que aspas simples em heredocs não precisam ser escapadas, apesar de que os códigos de escape listados acima podem continuar sendo utilizados. Variáveis são expandidas, mas o mesmo cuidado deve ser tomado ao expressar variáveis complexas dentro do heredoc assim como nas strings.

Exemplo #9 Exemplo de delimitação de strings heredoc

```php
<?php
$str = <<<EOD
Exemplo de uma string
distribuída em várias linhas
utilizando a sintaxe heredoc.
EOD;

/* Exemplo mais complexo, com variáveis */
class foo
{
    var $foo;
    var $bar;

    function __construct()
    {
        $this->foo = 'Foo';
        $this->bar = array('Bar1', 'Bar2', 'Bar3');
    }
}

$foo = new foo();
$name = 'Meu nome';

echo <<<EOT
Meu nome é "$name". Eu estou imprimindo $foo->foo.
Agora, eu estou imprimindo {$foo->bar[1]}.
Isto deve imprimir um 'A' maiúsculo: \x41
EOT;
?>
```

O exemplo acima produzirá:

```
Meu nome é "Meu nome". Eu estou imprimindo Foo.
Agora, eu estou imprimindo Bar2.
Isto deve imprimir um 'A' maiúsculo: A
```

É possível também utilizar a sintaxe Heredoc para passar dados para argumentos de funções:

Exemplo #10 Exemplo de Heredoc em argumentos

```
<?php
var_dump(array(<<<EOD
foobar!
EOD
));
?>
```

É possível inicializar variáveis estáticas e propriedades/constantes de classe utilizando a sintaxe heredoc:

Exemplo #11 Utilizando o Heredoc na inicialização de valores estáticos

```
<?php
// Variáveis estáticas
function foo()
{
    static $bar = <<<LABEL
Nada aqui...
LABEL;
}

// Classe propriedades/constantes
class foo
{
    const BAR = <<<FOOBAR
Exemplo de constante
FOOBAR;

    public $baz = <<<FOOBAR
Exemplo de propriedade
FOOBAR;
}
?>
```

O identificador de abertura do Heredoc pode ser opcionalmente delimitado por aspas duplas:

Exemplo #12 Usando aspas duplas no Heredoc

```
<?php
echo <<<"FOOBAR"
Olá, mundo!
FOOBAR;
?>
```

Nowdoc

Nowdocs estão para aspas simples assim como os heredocs estão para aspas duplas em strings. Um nowdoc é especificado similarmente a um heredoc, mas *nenhuma interpolação de string é feita* dentro de um nowdoc. A construção é ideal para colocar códigos PHP ou outros grandes blocos de texto sem a necessidade de usar

escapes. Compartilha algumas características em comum com a construção SGML <![CDATA[ ]]>, assim é declarado um bloco de texto onde nada será analisado.

Um nowdoc é identificado com a mesma sequência <<< usada para heredocs, mas o identificador precisa ficar entre aspas simples, por exemplo. <<<'EOT'. Todas as regras para identificadores heredoc também se aplicam para identificadores nowdoc, especialmente aquelas referentes a aparência do identificador de fechamento.

Exemplo #13 Exemplo de string em Nowdoc

```php
<?php
echo <<<'EOD'
Exemplo de string abrangendo várias linhas
usando a sintaxe nowdoc. As barras invertidas são sempre tratadas literalmente,
por exemplo. \\ e \'.
EOD;
```

O exemplo acima produzirá:

```
Exemplo de string abrangendo várias linhas
usando a sintaxe nowdoc. As barras invertidas são sempre tratadas literalmente,
por exemplo. \\ e \'.
```

Exemplo #14 Exemplo de Nowdoc, com variáveis

```php
<?php
class foo
{
    public $foo;
    public $bar;

    function __construct()
    {
        $this->foo = 'Foo';
        $this->bar = array('Bar1', 'Bar2', 'Bar3');
    }
}

$foo = new foo();
$name = 'MyName';

echo <<<'EOT'
Meu nome é "$nome". Estou imprimindo alguns $foo->foo.
Agora, estou imprimindo alguns {$foo->bar[1]}.
Isso não deve imprimir um 'A' maiúsculo: \x41
EOT;
?>
```

O exemplo acima produzirá:

```
Meu nome é "$nome". Estou imprimindo alguns $foo->foo.
Agora, estou imprimindo alguns {$foo->bar[1]}.
Isso não deve imprimir um 'A' maiúsculo: \x41
```

Exemplo #15 Exemplo de dado estático

```php
<?php
class foo {
    public $bar = <<<'EOT'
bar
EOT;
}
?>
```

Interpolação de strings

Quando uma string é especificada dentro de aspas duplas ou heredoc, as variáveis podem ser substituídas dentro dela.

Há dois tipos de sintaxe: uma básica e uma avançada. A sintaxe básica é a mais comum e conveniente. Provê uma maneira de incorporar uma variável, o valor de um array ou uma propriedade de object em uma string com o mínimo de esforço.

Sintaxe básica

Se um sinal de cifrão (`$`) for encontrado, os caracteres que o seguem, e que podem ser usados em nome de variável, serão interpretados como tal e substituídos.

Exemplo #16 Interpolação de string

```php
<?php
$suco = "maçã";

echo "Ele bebeu um pouco de suco de $suco." . PHP_EOL;

?>
```

O exemplo acima produzirá:

```
Ele bebeu um pouco de suco de maçã.
```

Formalmente, a estrutura para a sintaxe básica de substituição de variável é o seguinte:

```
string-variável::
     nome-de-variável   (posição-ou-propriedade)?
   | ${   expressão   }
```

```
posição-ou-propriedade::
     posição-na-string
   | propriedade-na-string

posição-na-string::
     [   nome   ]
   | [   nome-de-variável   ]
   | [   inteiro-literal   ]

propriedade-na-string::
     ->  nome

nome-de-variável::
     $   nome

nome::
     [a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*
```

> **Aviso**
>
> A sintaxe `${ expressão }` foi descontinuada a partir do PHP 8.2.0, já que pode ser interpretada como variáveis variáveis:
>
> ```php
> <?php
> const foo = 'bar';
> $foo = 'foo';
> $bar = 'bar';
> var_dump("${foo}");
> var_dump("${(foo)}");
> ?>
> ```
>
> Saída do exemplo acima no PHP 8.2:
>
> ```
> Deprecated: Using ${var} in strings is deprecated, use {$var} instead in file on line 6
>
> Deprecated: Using ${expr} (variable variables) in strings is deprecated, use {${expr}}
> instead in file on line 9
> string(3) "foo"
> string(3) "bar"
> ```
>
> O exemplo acima produzirá:
>
> ```
> string(3) "foo"
> string(3) "bar"
> ```
>
> A sintaxe avançada de interpolação de string deve ser usada em seu lugar.

> **Nota** :
>
> Se não for possível formar um nome válido, o símbolo de cifrão permanece literal na string:

```php
<?php
echo "Nenhuma interpolação $ aconteceu\n";
echo "Nenhuma interpolação $\n aconteceu\n";
echo "Nenhuma interpolação $2 aconteceu\n";
?>
```

O exemplo acima produzirá:

```
Nenhuma interpolação $ aconteceu
Nenhuma interpolação $
 aconteceu
Nenhuma interpolação $2 aconteceu
```

Exemplo #17 Interpolando o valor da primeira dimensão de um array ou propriedade

```php
<?php
$sucos = array("maça", "laranja", "chave_string" => "roxo");

echo "Ele bebeu um pouco de suco de $sucos[0].";
echo PHP_EOL;
echo "Ele bebeu um pouco de suco de $sucos[1].";
echo PHP_EOL;
echo "Ele bebeu um pouco de suco de $sucos[chave_string].";
echo PHP_EOL;

class A {
    public $s = "string";
}

$o = new A();

echo "Valor do objeto: $o->s.";
?>
```

O exemplo acima produzirá:

```
Ele bebeu um pouco de suco de maçã.
Ele bebeu um pouco de suco de laranja.
Ele bebeu um pouco de suco de roxo.
Valor do objeto: string.
```

> **Nota** :
>
> A chave do array deve ser sem aspas, e por isso não é possível referenciar uma constante como chave com a sintaxe básica. Use a sintaxe avançada em seu lugar.

A partir do PHP 7.1.0, índices numéricos *negativos* também são suportados.

Exemplo #18 Índices numéricos negativos

```php
<?php
$string = 'string';
echo "O caractere no índice -2 é $string[-2].", PHP_EOL;
$string[-3] = 'o';
echo "Alterar o caractere no índice -3 para 'o' dá $string.", PHP_EOL;
?>
```

O exemplo acima produzirá:

```
O caractere no índice -2 é n.
Alterar o caractere no índice -3 para o dá strong.
```

Para qualquer coisa mais complexa, a sintaxe avançada deve ser usada.

Sintaxe avançada (com chaves)

A sintaxe avançada permite a interpolação de *variáveis* com acessadores arbitrários.

Qualquer variável escalar, elemento de array ou propriedade de objeto (static ou não) com uma representação de uma string pode ser incluída com essa sintaxe. A expressão é escrita da mesma forma como apareceria fora da string e então coloque-o entre `{` e `}`. Já que que `{` não pode escapado, esta sintaxe será somente reconhecida quando o `$` seguir, imediatamente, o `{`. Use `{\$` para obter um literal `{$`. Alguns exemplos que tornam isto claro:

Exemplo #19 Sintaxe com chaves

```php
<?php
const DATA_KEY = 'chave-const';
$fantastico = 'fantástico';
$arr = [
    '1',
    '2',
    '3',
    [41, 42, 43],
    'chave' => 'Valor indexado',
    'chave-const' => 'Chave com sinal de menos',
    'foo' => ['foo1', 'foo2', 'foo3']
];

// Não vai funcionar, resultado: Isso é { fantástico}
```

```php
echo "Isso é { $fantastico}" . PHP_EOL;

// Funciona, resultado: Isso é fantástico
echo "Isso é {$fantastico}" . PHP_EOL;

// Para mostrar chaves na saída:
echo "Isso é {{$fantastico}}" . PHP_EOL;

class Square {
    public $width;

    public function __construct(int $width) { $this->width = $width; }
}

$square = new Square(5);

// Funciona
echo "Este quadrado tem {$square->width}00 centímetros de largura." . PHP_EOL;


// Funciona, as chaves entre aspas funcionam apenas usando a sintaxe de chaves
echo "Isso funciona: {$arr['chave']}" . PHP_EOL;


// Funciona
echo "Isso funciona: {$arr[3][2]}" . PHP_EOL;

echo "Isso funciona: {$arr[DATA_KEY]}" . PHP_EOL;

// Ao usar arrays multidimensionais, sempre use chaves em torno dos arrays
// quando dentro de strings
echo "Funciona: {$arr['foo'][2]}" . PHP_EOL;

echo "Funciona: {$obj->values[3]->name}" . PHP_EOL;

echo "Funciona: {$obj->$staticProp}" . PHP_EOL;

// Não funciona, imprime: C:\diretório\{fantástico}.txt
echo "C:\diretório\{$fantastico}.txt" . PHP_EOL;

// Funciona, imprime: C:\diretório\fantástico.txt
echo "C:\\diretório\\{$fantastico}.txt" . PHP_EOL;
?>
```

> **Nota** :
>
> Como esta sintaxe permite expressões arbitrárias, é possível usar variáveis variáveis dentro da sintaxe avançada.

Acesso e modificação da string por caractere

É possível acessar e modificar caracteres dentro de strings especificando a posição, baseada em zero, do caractere desejado na string usando colchetes, parecido com arrays, por exemplo *$str[42]*. Imagine uma string como um array de caracteres. As funções substr() e substr_replace(), podem ser utilizadas quando se deseja extrair ou substituir mais que 1 caractere.

> **Nota** :
>
> A partir do PHP 7.1.0, posições negativas para strings são suportadas. Estas especificam a posição do final da string. Antes, posições negativas emitiam um `E_NOTICE` para leitura (gerando um string vazia) e um `E_WARNING` para escrita (deixando a string intocada).

> **Nota** :
>
> Anteriormente ao PHP 8.0.0, strings também poderiam ser usados utilizando chaves, por exemplo *$str{42}*. Essa sintaxe por chaves foi descontinuada desde o PHP 7.4.0 e não é mais suportada a partir do PHP 8.0.0.

> **Aviso**
>
> Escrever em uma posição fora do intervalo preenche a string com espaços. Tipos diferentes de inteiro são convertidos para inteiro. Tipo ilegal de deslocamento emite um `E_WARNING`. Apenas o primeiro caractere de uma string atribuída é usado. A partir de PHP 7.1.0, a atribuição de uma string vazia lança um erro fatal. Antes, era atribuído o byte NULL.

> **Aviso**
>
> Internamente, as strings do PHP são arrays de bytes. Como resultado, acessar ou modificar uma string usando colchetes não é seguro com multi-bytes e deve ser feito somente em strings que utilizem apenas um byte em sua codificação, como a ISO-8859-1.

> **Nota** :
>
> A partir do PHP 7.1.0, aplicar um operador de índice vazio em uma string lança um erro fatal Antes, a string vazia era convertida em um array.

Exemplo #20 Alguns exemplos com strings

```php
<?php
// Obtém o primeiro caractere de uma string
$str = 'Isto é um teste.';
$first = $str[0];
var_dump($first);

// Obtém o terceiro caractere de uma string
$third = $str[2];
var_dump($third);

// Obtém o último caractere de uma string
$str = 'Isto ainda é um teste.';
$last = $str[strlen($str)-1];
var_dump($last);

// Modifica o último caractere de uma string
$str = 'Olhe para a onda';
$str[strlen($str)-1] = 'e';
```

```php
var_dump($str);
?>
```

Índices de strings devem ser inteiros ou strings parecidos com inteiros, de outra forma um aviso é lançado.

Exemplo #21 Exemplo de índices inválidos em strings

```php
<?php
$str = 'abc';

$keys = [ '1', '1.0', 'x', '1x' ];

foreach ($keys as $keyToTry) {
    var_dump(isset($str[$keyToTry]));

    try {
        var_dump($str[$keyToTry]);
    } catch (TypeError $e) {
        echo $e->getMessage(), PHP_EOL;
    }

    echo PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
bool(true)
string(1) "b"

bool(false)
Cannot access offset of type string on string

bool(false)
Cannot access offset of type string on string

bool(false)

Warning: Illegal string offset "1x" in Standard input code on line 10
string(1) "b"
```

> **Nota** :
>
> Acessar variáveis de outros tipos (excluindo-se arrays ou objetos que implementem certas interfaces) usando `[]` ou `{}` silenciosamente retornará `null`.

> **Nota** :

> Caracteres dentro de strings literais podem ser acessados utilizando `[]` ou `{}`.

> **Nota** :
>
> Acessar caracteres em strings utilizando a sintax por `{}` foi descontinuada desde o PHP 7.4, e não existe no PHP 8.0.

Funções e operadores úteis

Strings podem ser concatenadas utilizando o operador '.' (ponto). Note que o operador '+' (adição) *não* funciona para isso. Veja operadores de string para mais informações.

Há bastantes funções úteis para manipulação de strings.

Veja a seção de funções de string para funções gerais e funções de expressões regulares compatíveis com Perl para funcionalidades avançadas de localização & e substituição.

Há também funções para strings URL e funções para criptografia/descriptografia de strings (Sodium e Hash).

Finalmente, veja também as funções de tipos de caracteres.

Conversão em strings

Um valor pode ser convertido em uma string utilizando o modificador `(string)` ou a função strval(). A conversão em string é automaticamente realizada no escopo de uma expressão onde uma string é necessária. Isto acontece no uso das funções echo ou print ou quando o valor de uma variável é comparado com uma string. As seções Tipos e Conversão de Tipos tornarão um pouco mais claro o assunto. Veja também a função settype().

O valor bool **`true`** é convertido para a string `"1"`. O bool **`false`** é convertido para `""` (uma string vazia). Isso permite converter nos dois sentidos entre os valores bool e string.

Um int ou um float é convertido para uma string representando o número textualmente (incluindo a parte do expoente nos floats). Números de ponto flutuante podem ser convertidos usando a notação exponencial (`4.1E+6`).

> **Nota** :
>
> A partir do PHP 8.0.0, o caractere de separação de decimais sempre é um ponto (`"."`). Anteriormente ao PHP 8.0.0, o separador de decimais era definido pela localidade do script (LC_NUMERIC). Veja mais detalhes na função setlocale().

Arrays são sempre convertidos para uma string `"Array"`; por isso echo e print não podem, por si só, mostrar o conteúdo de um array. Para visualizar um único elemento, use uma construção como `echo $arr['foo']`. Veja abaixo dicas de como visualizar todo seu conteúdo.

Para converter objects em strings, o método mágico __toString deve ser usado.

Resources são sempre convertidos para strings na estrutura `"Resource id #1"` onde `1` é o número único do resource atribuído pelo PHP em tempo de execução. Apesar de que a estrutura exata desta string não seja

confiável e esteja sujeita a modificações, será sempre única a um dado recurso dentro do ciclo de vida de um script sendo executado (por exemplo, em uma requisição Web ou em um processo CLI) e não será reutilizada. Para obter o tipo de um resource, utilize a função get_resource_type().

`null` é sempre convertido para uma string vazia.

Como declarado acima, converter diretamente um array, object ou resource para uma string não fornece nenhuma informação útil sobre o valor, a não ser seu tipo. Veja as funções print_r() e var_dump() para maneiras mais efetivas de inspecionar o conteúdo destes tipos.

A maioria dos valores no PHP também podem ser convertidos em strings para armazenamento permanente. Este método é chamado de serialização e pode ser feito com a função serialize().

Detalhes do tipo string

A string no PHP é implementada como um array de bytes e um inteiro indicando seu tamanho no buffer. Não existem informações de como esses bytes são traduzidos em caracteres, ficando essa tarefa com o programador. Não existem limitações sobre a composição dos valores da string; em particular, bytes com valor `0` ("bytes NUL") são permitidos em qualquer lugar da string. Entretanto, algumas funções mencionadas neste manual como não seguras em termos binários podem repassar as strings para bibliotecas que ignoram os dados após o byte nulo.

Esta natureza do tipo string explica o motivo de não haver o tipo "byte" disponível no PHP – as strings assumem este papel. Funções que não retornam nenhuma informação textual – por exemplo, dados arbitrários lidos de um socket de rede – continuarão retornando strings.

Dado que o PHP não dita uma codificação específica para strings, pode-se questionar como strings literais seriam codificadas. Por exemplo, a string `"á"` é equivalente a `"\xE1"` (ISO-8859-1), `"\xC3\xA1"` (UTF-8, C form), `"\x61\xCC\x81"` (UTF-8, D form) ou qualquer outra representação possível? A resposta é que a string poderá ser codificada em qualquer forma que o arquivo de script estiver codificado. Assim, se o script for escrito na codificação ISO-8859-1, a string será codificada como ISO-8859-1 e assim por diante. Entretanto, isso não se aplica se o Zend Multibyte estiver ativado; neste caso, o script pode ser escrito em uma codificação arbitrária (que é declarada explicitamente ou é detectada) e então convertida em um codificação interna, a qual será a codificação utilizada nas strings literais. Note que há algumas restrições na codificação do script (ou na codificação interna caso o Zend Multibyte esteja ativo) – isso quase sempre significa que essa codificação deve ser um conjunto que contenha dentro de si a ASCII, como UTF-8 ou ISO-8859-1. Note, entretanto, que codificações que são dependentes de estado, onde os mesmos valores de bytes possam ser utilizados tanto em estados iniciais quanto não iniciais, podem ser problemáticos.

Claro que, para serem úteis, funções que operam em textos podem ter que fazer certas suposições sobre como as strings foram codificadas. Infelizmente, existem muitas variações sobre este assunto através das funções do PHP:

- Algumas funções assumem que a string foi codificada em alguma (qualquer) codificação de byte único, mas elas não precisam interpretar estes bytes como caracteres específicos. Este é o caso, por exemplo, das funções substr(), strpos(), strlen() ou strcmp(). Outra forma de refletir sobre essas funções é que elas operam em buffers de memória, isto é, elas trabalham com bytes e deslocamento de bytes.

- Outras funções recebem a codificação da string como um parâmetro ou assumem um padrão se nenhum for informado. Este é o caso da função htmlentities() e da maioria das funções da extensão mbstring.

- Outras utilizam o idioma atual (veja setlocale()), mas elas operam byte-a-byte.

- Finalmente, elas podem simplesmente assumir que a string esteja utilizando uma codificação específica, geralmente UTF-8. Este é o caso da maioria das funções das extensões intl e PCRE (na segunda, somente quando o modificador `u` é utilizado).

Por fim, isso significa que escrever programas corretos utilizando Unicode depende de evitar cuidadosamente funções que não funcionam e que provavelmente irão corromper os dados e usar, em substituição, funções que se comportem corretamente, geralmente das extensões intl e mbstring. Entretanto, usar funções que manipulam codificação Unicode é somente o começo. Não importa as funções que a linguagem provê, é essencial saber a especificação Unicode. Por exemplo, um programa que supõe que só existem maiúsculas e mínusculas está fazendo uma suposição errada.

Strings numéricas

Uma string PHP é considerada numérica se puder ser interpretada como int ou float.

Formalmente a partir do PHP 8.0.0:

```
WHITESPACES      \s*
LNUM             [0-9]+
DNUM             ([0-9]*[\.]{LNUM}) | ({LNUM}[\.][0-9]*)
EXPONENT_DNUM    (({LNUM} | {DNUM}) [eE][+-]? {LNUM})
INT_NUM_STRING   {WHITESPACES} [+-]? {LNUM} {WHITESPACES}
FLOAT_NUM_STRING {WHITESPACES} [+-]? ({DNUM} | {EXPONENT_DNUM}) {WHITESPACES}
NUM_STRING       ({INT_NUM_STRING} | {FLOAT_NUM_STRING})
```

O PHP também tem um conceito de strings numéricas *principais*. Isso é simplesmente uma string que começa como uma string numérica seguida por qualquer caractere.

> **Nota** :
>
> Qualquer string que contenha a letra `E` (sem distinção entre maiúsculas e minúsculas) delimitada por números será vista como um número expresso em notação científica. Isso pode produzir resultados inesperados.
>
> **Exemplo #1 Comparações de notações científicas**
>
> ```php
> <?php
> var_dump("0D1" == "000"); // falso, "0D1" não é notação científica
> var_dump("0E1" == "000"); // verdadeiro, "0E1" é 0 * (10 ^ 1), ou 0
> var_dump("2E1" == "020"); // verdadeiro, "2E1" é 2 * (10 ^ 1), ou 20
> ?>
> ```

Strings usadas em contextos numéricos

Quando uma string precisa ser avaliada como um número (por exemplo, operações aritméticas, declaração de tipo int, etc.), as seguintes etapas são executadas para determinar o resultado:

1. Se a string for numérica, resolva para um int se a string for uma string numérica inteira e se encaixar nos limites do tipo int (conforme definido por **`PHP_INT_MAX`**), caso contrário resolva para um float.

2. Se o contexto permitir strings numéricas iniciais e a string for uma, resolva para um int se a parte inicial da string for uma string numérica inteira e se encaixar nos limites do tipo int (conforme definido por

PHP_INT_MAX), caso contrário resolva para um float. Além disso, um erro de nível E_WARNING é gerado.

3. Se a string não for numérica, lança um TypeError.

Comportamento antes do PHP 8.0.0

Antes do PHP 8.0.0, uma string era considerada numérica se tivesse espaços em branco somente à *esquerda*, se tivesse espaços em branco à *direita*, a string era considerada numérica à esquerda.

Antes do PHP 8.0.0, quando uma string era usada em um contexto numérico, ela executava as mesmas etapas acima com as seguintes diferenças:

- O uso de uma sequência numérica inicial geraria um E_NOTICE em vez de um E_WARNING.
- Se a string não fosse numérica, um E_WARNING era emitido e o valor 0 seria retornado.

Antes do PHP 7.1.0, nem E_NOTICE nem E_WARNING eram gerados.

```
<?php
$foo = 1 + "10.5";                // $foo é float (11.5)
$foo = 1 + "-1.3e3";              // $foo é float (-1299)
$foo = 1 + "bob-1.3e3";           // TypeError a partir do PHP 8.0.0, $foo é inteiro (1)
anteriormente
$foo = 1 + "bob3";                // TypeError a partir do PHP 8.0.0, $foo é inteiro (1)
anteriormente
$foo = 1 + "10 Small Pigs";       // $foo é inteiro (11) e um E_WARNING é emitido no PHP 8.0.0,
E_NOTICE anteriormente
$foo = 4 + "10.2 Little Piggies"; // $foo é float (14.2) e um E_WARNING é emitido no PHP 8.0.0,
E_NOTICE anteriormente
$foo = "10.0 pigs " + 1;          // $foo é float (11) e um E_WARNING é emitido no PHP 8.0.0,
E_NOTICE anteriormente
$foo = "10.0 pigs " + 1.0;        // $foo é float (11) e um E_WARNING é emitido no PHP 8.0.0,
E_NOTICE anteriormente
?>
```

Arrays

Para uma lista de todas as funções de array, consulte o capítulo sobre funções de array na documentação.

Um array no PHP é na verdade um mapa ordenado. Um mapa é um tipo que relaciona *valores* a *chaves*. Este tipo é otimizado para vários usos diferentes: ele pode ser tratado como um array, uma lista (vetor), tabela de hashes (que é uma implementação de mapa), dicionário, coleção, pilha, fila e provavelmente mais. Assim como existe a possibilidade dos valores do array serem outros arrays, árvores e arrays multidimensionais.

A explicação dessas estruturas está além do escopo deste manual, mas pelo menos um exemplo é dado para cada uma delas. Para mais informações, busque a considerável literatura que existe sobre esse extenso tópico.

Sintaxe

Especificando com array()

Um array pode ser criado com o construtor de linguagem array(). Ele leva qualquer quantidade de pares separados por vírgula `chave => valor` como argumentos.

```
array(
    chave  => valor,
```

```
    chave2 => valor2,
    chave3 => valor3,
    ...
)
```

A vírgula após o último elemento do array é opcional e pode ser omitida. Isso geralmente é feito para arrays em uma única linha, por exemplo, `array(1, 2)` é preferível em vez de `array(1, 2, )`. Para arrays escritos em várias linhas, por outro lado, a vírgula à direita do último elemento é usada frequentemente, para permitir a adição de forma simples de novos elementos ao final.

> **Nota** :
>
> Existe uma sintaxe curta que substitui `array()` por `[]`.

Exemplo #1 Um array simples

```php
<?php
$array1 = array(
    "foo" => "bar",
    "bar" => "foo",
);

// Utilizando a sintaxe curta
$array2 = [
    "foo" => "bar",
    "bar" => "foo",
];

var_dump($array1, $array2);
?>
```

A chave pode ser um int ou uma string. O valor pode ser de qualquer tipo.

Adicionalmente, as seguintes coerções ocorrerão a chave:

- strings contendo quaisquer ints válidos, que não sejam precedidos do sinal de `+`, serão convertidos para o tipo int. Por exemplo, a chave`"8"` será, na verdade, armazenada como `8`. Entretanto, `"08"` não será convertido, por não ser um inteiro decimal válido.
- floats também são convertidos para ints, isso significa que a parte fracionária será removida. Por exemplo, a chave `8.7` será na verdade armazenada como `8`.
- bools são convertidos para ints, igualmente, por exemplo, a chave **`true`**, será na verdade armazenada como `1` e a chave **`false`** como `0`.
- null será convertido para uma string vazia, por exemplo, a chave **`null`** na verdade será armazenada como `""`.
- arrays e objects *não podem* ser usados como chaves. Fazer isso resultará em um aviso: `Illegal offset type`.

Se vários elementos na declaração do array utilizam a mesma chave, apenas o último será utilizado, enquanto todos os outros serão sobrescritos.

Exemplo #2 Exemplo de conversão de tipo e sobrescrita

```php
<?php
$array = array(
    1    => "a",
    "1"  => "b",
    1.5  => "c",
    true => "d",
);
var_dump($array);
?>
```

O exemplo acima produzirá:

```
array(1) {
  [1]=>
  string(1) "d"
}
```

Como todas as chaves do exemplo acima foram convertidas para `1`, o valor será sobrescrito a cada novo elemento e o valor final atribuído `"d"`, será o único que restará.

As chaves dos arrays no PHP podem conter, ao mesmo tempo, int e string, porque o PHP não faz distinção entre arrays indexados e associativos.

Exemplo #3 Misturando int e string nas chaves

```php
<?php
$array = array(
    "foo" => "bar",
    "bar" => "foo",
    100   => -100,
    -100  => 100,
);
var_dump($array);
?>
```

O exemplo acima produzirá:

```
array(4) {
  ["foo"]=>
  string(3) "bar"
  ["bar"]=>
  string(3) "foo"
  [100]=>
  int(-100)
  [-100]=>
```

```
  int(100)
}
```

A key é opcional. Se não for especificada, o PHP utilizará o incremento da chave do tipo int com maior valor utilizado.

Exemplo #4 Arrays indexados sem chaves

```php
<?php
$array = array("foo", "bar", "hello", "world");
var_dump($array);
?>
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  string(3) "foo"
  [1]=>
  string(3) "bar"
  [2]=>
  string(5) "hello"
  [3]=>
  string(5) "world"
}
```

É possível especificar a chave somente para alguns elementos e omití-las para outros:

Exemplo #5 Chaves em alguns elementos

```php
<?php
$array = array(
         "a",
         "b",
    6 => "c",
         "d",
);
var_dump($array);
?>
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  string(1) "a"
  [1]=>
```

```
  string(1) "b"
  [6]=>
  string(1) "c"
  [7]=>
  string(1) "d"
}
```

Como pode ver, o último valor `"d"` foi atribuído à chave `7`. Isso acontece porque a chave com maior inteiro antes dela era `6`.

Exemplo #6 Exemplo de sobrescrita e cast de tipo complexo

Este exemplo inclui todas as variações de cast de tipo de chaves e sobrescrita de elementos.

```php
<?php
$array = array(
    1    => 'a',
    '1'  => 'b', // o valor "a" será sobrescrito por "b"
    1.5  => 'c', // o valor "b" será sobrescrito por "c"
    -1 => 'd',
    '01'  => 'e', // não é uma string de inteiro e NÃO sobrescreve a chave 1
    '1.5' => 'f', // não é uma string de inteiro e NÃO sobrescreve a chave 1
    true => 'g', // o valor "c" será sobrescrito por "g"
    false => 'h',
    '' => 'i',
    null => 'j', // o valor "i" será sobrescrito por "j"
    'k',      // o valor "k" será alocado na chave 2. Isto porque é o maior inteiro após a chave 1
    2 => 'l', // o valor "k" será sobrescrito por "l"
);

var_dump($array);
?>
```

O exemplo acima produzirá:

```
array(7) {
  [1]=>
  string(1) "g"
  [-1]=>
  string(1) "d"
  ["01"]=>
  string(1) "e"
  ["1.5"]=>
  string(1) "f"
  [0]=>
  string(1) "h"
  [""]=>
  string(1) "j"
  [2]=>
  string(1) "l"
}
```

Exemplo #7 Exemplo de índice negativo

Ao atribuir uma chave inteira negativa `n`, o PHP irá atribuir `n+1` à chave seguinte.

```php
<?php
$array = [];

$array[-5] = 1;
$array[] = 2;

var_dump($array);
?>
```

O exemplo acima produzirá:

```
array(2) {
  [-5]=>
  int(1)
  [-4]=>
  int(2)
}
```

> **Aviso**
>
> Antes do PHP 8.3.0, atribuir uma chave inteira negativa `n` causaria a atribuição de `0` à chave seguinte, portando o exemplo anterior teria este resultado:
>
> ```
> array(2) {
>   [-5]=>
>   int(1)
>   [0]=>
>   int(2)
> }
> ```

Acessando elementos do array com colchetes

Elementos do array podem ser acessados utilizando a sintaxe `array[chave]`.

Exemplo #8 Acessando elementos do array

```php
<?php
$array = array(
    "foo" => "bar",
    42    => 24,
```

```
    "multi" => array(
         "dimensional" => array(
             "array" => "foo"
         )
    )
);

var_dump($array["foo"]);
var_dump($array[42]);
var_dump($array["multi"]["dimensional"]["array"]);
?>
```

O exemplo acima produzirá:

```
string(3) "bar"
int(24)
string(3) "foo"
```

> **Nota** :
>
> Anteriormente ao PHP 8.0.0, chaves e colchetes podiam ser utilizados para acessar elementos de um array (por exemplo, `$array[42]` e `$array{42}` irão fazer a mesma coisa que o exemplo anterior). O acesso por chaves foi descontinuado a partir do PHP 7.4.0 e não é mais suportado a partir do PHP 8.0.0.

Exemplo #9 Referenciando elemento de um array

```
<?php
function getArray() {
    return array(1, 2, 3);
}

$secondElement = getArray()[1];

var_dump($secondElement);
?>
```

> **Nota** :
>
> Tentativas de acesso a uma chave de array que não foi definida é o mesmo que acessar qualquer outra variável indefinida: uma mensagem de erro de nível **`E_WARNING`** (**`E_NOTICE`** anteriormente ao PHP 8.0.0) será emitida, e o resultado será **`null`**.

> **Nota** :
>
> Acessar um array através de uma chave que não é string retorna **`null`**. Anteriormente ao PHP 7.4.0 não era emitida uma mensagem. A partir do PHP 7.4.0 emite um **`E_NOTICE`**; e a partir do PHP 8.0.0 é emitido um

> **E_WARNING**.

Criando/modificando com a sintaxe de colchetes

Você pode também modificar um array existente explicitamente assimilando valores a ele.

Isto é feito apenas assimilando valores para o array enquanto especifica a chave em colchetes. Você pode omitir a chave, colocando um par vazio de colchetes (`[]`).

```
$arr[chave] = valor;
$arr[] = valor;
// chave pode ser tanto um int quanto uma string
// valor pode ser qualquer valor de qualquer tipo
```

Se *$arr* ainda não existir ou estiver definido como **null** ou **false**, ele será criado, por isso também é uma alternativa para criação de um array. Entretanto, essa prática é desencorajada por que se *$arr* conter algum valor (por exemplo, uma string de uma variável de requisição), então este valor permanecerá e o `[]` funcionará como um operador de acesso a string. Sempre será recomendado a inicialização de uma variável por atribuição direta.

> **Nota** :
>
> A partir do PHP 7.1.0, aplicar um operador de índice vazio a uma string lança um erro fatal. Antes, a string era silenciosamente convertida em um array.

> **Nota** :
>
> A partir do PHP 8.1.0, criar um novo array a partir do valor **false** foi descontinuado. Criar um array a partir de **null** e valores indefinidos ainda é permitido.

Para mudar um certo valor, apenas atribua um novo valor a um elemento especificado por sua chave. Se a intenção for remover um par de chave/valor, é necessário aplicar a função unset() na chave.

Exemplo #10 Usando colchetes com arrays

```
<?php
$arr = array(5 => 1, 12 => 2);

$arr[] = 56;    // Isto é o mesmo que $arr[13] = 56;
                // nesse ponto do script

$arr["x"] = 42; // Isto acrescenta um novo elemento
                // para o array com a chave "x"

unset($arr[5]); // Isto remove um elemento do array

var_dump($arr);

unset($arr);    // E isto apaga todo o array
```

```
var_dump($arr);
?>
```

> **Nota** :
>
> Se nenhuma chave for especificada, ela será igual ao maior índice inteiro existente + `1`. Se o array não tiver índices inteiros positivos, a chave será `0`. A partir do PHP 8.3.0, também pode ser um inteiro negativo.
>
> Note que a chave inteira de maior valor utilizada *não precisa necessariamente existir no array*. Ele precisa ter existido no array em algum momento desde a última reindexação do array. Veja o exemplo a seguir:

```
<?php
// Criando um array normal
$array = array(1, 2, 3, 4, 5);
print_r($array);

// Agora apagando todos os itens, mas deixando o array intacto:
foreach ($array as $i => $value) {
    unset($array[$i]);
}
print_r($array);

// Acrescentando um item (note que a chave é 5, em vez de zero).
$array[] = 6;
print_r($array);

// Reindexando:
$array = array_values($array);
$array[] = 7;
print_r($array);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => 1
    [1] => 2
    [2] => 3
    [3] => 4
    [4] => 5
)
Array
(
)
Array
(
    [5] => 6
)
Array
(
    [0] => 6
```

```
    [1] => 7
)
```

Desconstruindo arrays

Arrays podem ser desconstruídos utilizando `[]` (a partir do PHP 7.1.0) ou list(). Esses constructos podem ser utilizados para desconstruir o array em variáveis distintas.

Exemplo #11 Desconstruindo arrays

```php
<?php
$source_array = ['foo', 'bar', 'baz'];

[$foo, $bar, $baz] = $source_array;

echo $foo, PHP_EOL;    // exibe "foo"
echo $bar, PHP_EOL;    // exibe "bar"
echo $baz, PHP_EOL;    // exibe "baz"
?>
```

A desconstrução de arrays pode ser utilizada no `foreach` para desconstruir um array multidimensional enquanto o percorre.

Exemplo #12 Desconstruindo arrays em Foreach

```php
<?php
$source_array = [
    [1, 'John'],
    [2, 'Jane'],
];

foreach ($source_array as [$id, $name]) {
    echo "{$id}: '{$name}'\n";
}
?>
```

Elementos de arrays serão ignorados se a variável não for providenciada. A desconstrução de arrays sempre inicia no índice `0`.

Exemplo #13 Ignorando Elementos

```php
<?php
$source_array = ['foo', 'bar', 'baz'];

// Atribui o elemento de índice 2 na variável $baz
[, , $baz] = $source_array;

echo $baz;    // exibe "baz"
?>
```

A partir do PHP 7.1.0, é possível desconstruir arrays associativos. Isto permite fácil seleção do elemento correto de uma forma indexada numericamente dado que os índices podem ser especificados.

Exemplo #14 Desconstruindo arrays associativos

```php
<?php
$source_array = ['foo' => 1, 'bar' => 2, 'baz' => 3];

// Atribui o elemento de índice 'baz' na variável $three
['baz' => $three] = $source_array;

echo $three, PHP_EOL;    // exibe 3

$source_array = ['foo', 'bar', 'baz'];

// Atribui o elemento de índice 2 na variável $baz
[2 => $baz] = $source_array;

echo $baz, PHP_EOL;    // exibe "baz"
?>
```

A desconstrução de arrays pode ser utilizada para trocar duas variáveis.

Exemplo #15 Trocando duas variáveis

```php
<?php
$a = 1;
$b = 2;

[$b, $a] = [$a, $b];

echo $a, PHP_EOL;    // exibe 2
echo $b, PHP_EOL;    // exibe 1
?>
```

> **Nota** :
>
> O operador de espalhamento (`...`) não é suportado em atribuições.

> **Nota** :
>
> Tentar acessar uma chave indefinida têm o mesmo efeito de acessar uma variável indefinida: um aviso `E_WARNING` (`E_NOTICE` anteriormente ao PHP 8.0.0) será emitido, e o resultado será `null`.

> **Nota** :
>
> A desconstrução de um valor escalar atribui `null` a todas as variáveis.

Funções úteis

Há uma série de funções úteis para trabalhar com arrays. Veja a seção sobre funções para arrays.

> **Nota** :

> A função unset() permite apagar chaves de um array. Esteja avisado que o array *não* será reindexado. Se um comportamento como "remover e deslocar" é o desejado, o array poderá ser reindexado utilizando a função array_values().
>
> **Exemplo #16 Eliminando elementos intermediários**

```php
<?php
$a = array( 1 => 'um', 2 => 'dois', 3 => 'três' );

/* irá produzir um array que pode ser definido como
   $a = array( 1=>'um', 3=>'três');
   e NÃO
   $a = array( 1 => 'um', 2 => 'três');
*/
unset($a[2]);
var_dump($a);

$b = array_values($a);
// Agora $b é o  array(0 => 'um', 1 =>'três')
var_dump($b);
?>
```

A estrutura de controle `foreach` existe especificamente para lidar com arrays. Ele provê uma maneira fácil de percorrer um array.

Array: o que fazer e o que não fazer

Porque `$foo[bar]` está errado?

Sempre utilize delimitadores em torno dos índices do array. Por exemplo, `$foo['bar']` está correto, enquanto que `$foo[bar]`, não está. Mas porque? É comum encontrar essa sintaxe em scripts antigos:

```php
<?php
$foo[bar] = 'inimigo';
echo $foo[bar];
// etc
?>
```

Isto está errado, mas funciona. A razão é que este código possui uma constante indefinida (`bar`) em vez de uma string (`'bar'` - repare nas aspas simples). Isto funciona, porque o PHP converte automaticamente uma *string base* (uma string não delimitada que não corresponde a nenhum símbolo conhecido) em uma string que contém a string base. Por exemplo, se não existir uma constante definida com o nome **`bar`**, então o PHP irá substituí-la pela string `'bar'` e usá-la.

> **Aviso**
>
> O fallback que trata uma constante indefinida como uma string lança um erro do nível **`E_NOTICE`**. Isso foi descontinuado desde o PHP 7.2.0, e lança um erro do nível **`E_WARNING`**. A partir do PHP 8.0.0, esse fallback foi removido e agora lança uma exceção Error.

Isto não significa que deve-se *sempre* delimitar as chaves. Não delimite chaves que sejam constantes ou variáveis, porque isso impedirá o PHP de interpretá-las.

Exemplo #17 Delimitando chaves

```php
<?php
error_reporting(E_ALL);
ini_set('display_errors', true);
ini_set('html_errors', false);

// Arrays simples:
$array = array(1, 2);
$count = count($array);

for ($i = 0; $i < $count; $i++) {
    echo "\nVerificando $i: \n";
    echo "Ruim: " . $array['$i'] . "\n";
    echo "Bom: " . $array[$i] . "\n";
    echo "Ruim: {$array['$i']}\n";
    echo "Bom: {$array[$i]}\n";
}
?>
```

O exemplo acima produzirá:

```
Verificando 0:
Notice: Undefined index:  $i in /path/to/script.html on line 9
Ruim:
Bom: 1
Notice: Undefined index:  $i in /path/to/script.html on line 11
Ruim:
Bom: 1

Verificando 1:
Notice: Undefined index:  $i in /path/to/script.html on line 9
Ruim:
Bom: 2
Notice: Undefined index:  $i in /path/to/script.html on line 11
Ruim:
Bom: 2
```

Mais exemplos para demonstrar esse comportamento:

Exemplo #18 Mais exemplos

```php
<?php
// Mostrando todos os erros
error_reporting(E_ALL);

$arr = array('fruta' => 'maçã', 'legume' => 'cenoura');

// Correto
echo $arr['fruta'], PHP_EOL;  // maçã
echo $arr['legume'], PHP_EOL; // cenoura
```

```php
// Errado. Isto não funciona e lança um erro PHP por causa
// de uma constante indefinida de nome 'fruta'
//
// Error: Undefined constant "fruta"
try {
    echo $arr[fruta];
} catch (Error $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
}

// Esta instrução define uma constante para demonstrar o que acontece. O valor 'legume'
// é atribuído a uma constante denominada 'fruta'
define('fruta', 'legume');

// Observe a diferença agora
echo $arr['fruta'], PHP_EOL;  // maçã
echo $arr[fruta], PHP_EOL;    // cenoura

// O exemplo seguinte está ok, já que está dentro de uma string. Constantes não são
// observadas dentro de strings e por isso nenhum erro acontece aqui
echo "Olá $arr[fruta]", PHP_EOL;      // Olá maçã

// Com uma exceção: chaves envolvendo arrays dentro de strings
// ativam a checagem de constantes, como em
echo "Olá {$arr[fruta]}", PHP_EOL;    // Olá cenoura
echo "Olá {$arr['fruta']}", PHP_EOL;  // Olá maçã

// Concatenação é uma outra opção
echo "Olá " . $arr['fruta'], PHP_EOL; // Olá maçã
?>
```

```php
<?php
// E isso não funciona, resultando em um erro de interpretação do tipo:
// Parse error: parse error, expecting T_STRING' or T_VARIABLE' or T_NUM_STRING'
// Isso também se aplica ao uso de superglobais em strings
print "Olá $arr['fruta']";
print "Olá $_GET['foo']";
?>
```

Como especificado na seção sobre sintaxe, o que estiver entre os colchetes ('[' e ']'), precisa ser uma expressão. Isso significa que códigos como estes funcionam:

```php
<?php
echo $arr[algumafuncao($bar)];
?>
```

Isto é um exemplo de utilização de um valor de retorno de função como um índice de array. O PHP também aceita constantes:

```php
<?php
$error_descriptions[E_ERROR] = "Um erro fatal ocorreu";
$error_descriptions[E_WARNING] = "O PHP emitiu um alarme";
$error_descriptions[E_NOTICE] = "Apenas um aviso informal";
?>
```

Observe que `E_ERROR` também é um identificador válido, assim como `bar` no primeiro exemplo. Mas no último exemplo seria, na verdade, o mesmo que escrever:

```
<?php
$error_descriptions[1] = "Um erro fatal ocorreu";
$error_descriptions[2] = "O PHP emitiu um alarme";
$error_descriptions[8] = "Apenas um aviso informal";
?>
```

porque `E_ERROR` é igual a `1`, e assim por diante.

Então porque isso é tão ruim?

Se em algum ponto do futuro, o time do PHP quiser acrescentar outra constante ou palavra chave, ou você mesmo introduzir outra constante na sua aplicação, você terá problemas. Por exemplo, já não se pode utilizar as palavras `empty` e `default` dessa maneira, desde que elas são palavras reservadas especiais.

> **Nota** :
>
> Para reforçar, dentro de uma string delimitada por aspas duplas, é válido não englobar o índice do array com aspas simples, então `"$foo[bar]"` é válido. Veja os exemplos anteriores para detalhes assim como a seção sobre interpretação de variáveis em strings.

Convertendo para array

Para qualquer dos tipos: int, float, string, bool e resource, converter um valor para um array, resultará em um array com um único elemento com índice zero e valor escalar convertido. Em outras palavras, `(array) $scalarValue` é exatamente o mesmo que `array($scalarValue)`.

Se um object é convertido para um array, o resultado será um array em que seus elementos serão as propriedades do object. As chaves serão os nomes das propriedades com pequenas notáveis exceções: propriedades inteiras serão inacessíveis; variáveis privadas tem o nome da classe prefixando o nome da variável; variáveis protegidas tem um '*' prefixando o nome da variável. Esses valores prefixados têm um byte `NUL` em cada lado. Propriedades tipadas não inicializadas são ignoradas.

Exemplo #19 Convertendo para um array

```
<?php

class A {
    private $B;
    protected $C;
    public $D;
    function __construct()
    {
        $this->{1} = null;
    }
}

var_export((array) new A());
?>
```

O exemplo acima produzirá:

```
array (
  '' . "\0" . 'A' . "\0" . 'B' => NULL,
  '' . "\0" . '*' . "\0" . 'C' => NULL,
  'D' => NULL,
  1 => NULL,
)
```

Esses NUL podem ocasionar comportamentos inesperados:

Exemplo #20 Convertendo um objeto em um Array

```php
<?php

class A {
    private $A; // Transformado na chave '\0A\0A'
}

class B extends A {
    private $A; // Transformado na chave '\0B\0A'
    public $AA; // Transformado na chave 'AA'
}

var_dump((array) new B());
?>
```

Saída do exemplo acima no PHP 8.0:

```
array(3) {
  ["BA"]=>
  NULL
  ["AA"]=>
  NULL
  ["AA"]=>
  NULL
}
```

Saída do exemplo acima no PHP 8.1:

```
array(3) {
  ["AA"]=>
  NULL
  ["BA"]=>
```

```
  NULL
  ["AA"]=>
  NULL
}
```

Acima aparecem duas chaves chamadas 'AA', embora uma delas é, na verdade, nomeada '\0A\0A'.

Converter um valor `null` para um array resultará em um array vazio.

Comparando

É possível comparar arrays utilizando a função array_diff() e com a utilização dos operadores de array.

Expansão de arrays

Um array prefixado com `...` será expandido no local desse operador. Somente arrays e objetos que implementem Traversable podem ser expandidos. A expansão de arrays com `...` está disponível desde o PHP 7.4.0. Ele também é chamado de operador de expansão.

É possível realizar a expansão várias vezes, e adicionar elementos antes ou depois do operador `...`:

Exemplo #21 Expansão de arrays

```php
<?php
// Utilizando a sintaxe curta.
// Também funciona com a sintax array().
$arr1 = [1, 2, 3];
$arr2 = [...$arr1]; // [1, 2, 3]
$arr3 = [0, ...$arr1]; // [0, 1, 2, 3]
$arr4 = [...$arr1, ...$arr2, 111]; // [1, 2, 3, 1, 2, 3, 111]
$arr5 = [...$arr1, ...$arr1]; // [1, 2, 3, 1, 2, 3]

function getArr() {
  return ['a', 'b'];
}
$arr6 = [...getArr(), 'c' => 'd']; // ['a', 'b', 'c' => 'd']

var_dump($arr1, $arr2, $arr3, $arr4, $arr5, $arr6);
?>
```

Expandir um array com `...` segue a semântica da função array_merge(). Ou seja, chaves strings sobrescrevem valores antigo e chaves inteiros são renumerados:

Exemplo #22 Expansão de arrays com chaves duplicadas

```php
<?php
// chave string
$arr1 = ["a" => 1];
$arr2 = ["a" => 2];
$arr3 = ["a" => 0, ...$arr1, ...$arr2];
var_dump($arr3); // ["a" => 2]

// chave inteira
$arr4 = [1, 2, 3];
```

```php
$arr5 = [4, 5, 6];
$arr6 = [...$arr4, ...$arr5];
var_dump($arr6); // [1, 2, 3, 4, 5, 6]
// Que resulta em [0 => 1, 1 => 2, 2 => 3, 3 => 4, 4 => 5, 5 => 6]
// pois as chaves inteiras anteriores não são preservadas.
?>
```

> **Nota** :
>
> Chaves que não sejam inteiros ou strings lançam um TypeError. Essas chaves podem ser geradas por um objeto Traversable.

> **Nota** :
>
> Anteriormente ao PHP 8.1, expansão de arrays com chave string não era suportado:
>
> ```php
> <?php
>
> $arr1 = [1, 2, 3];
> $arr2 = ['a' => 4];
> $arr3 = [...$arr1, ...$arr2];
> // Fatal error: Uncaught Error: Cannot unpack array with string keys in example.php:5
>
> $arr4 = [1, 2, 3];
> $arr5 = [4, 5];
> $arr6 = [...$arr4, ...$arr5]; // works. [1, 2, 3, 4, 5]
> ?>
> ```

Exemplos

O tipo array do PHP é muito versátil. Temos aqui alguns exemplos:

Exemplo #23 Versatilidade do array

```php
<?php
// Isso:
$a = array( 'color' => 'red',
            'taste' => 'sweet',
            'shape' => 'round',
            'name'  => 'apple',
            4        // a chave será 0
          );

$b = array('a', 'b', 'c');

var_dump($a, $b);

// ...é totalmente equivalente a isso:
$a = array();
$a['color'] = 'red';
$a['taste'] = 'sweet';
$a['shape'] = 'round';
$a['name']  = 'apple';
$a[]        = 4;        // a chave será 0
```

```php
$b = array();
$b[] = 'a';
$b[] = 'b';
$b[] = 'c';

// Após a execução do código acima, $ será o array
// array('color' => 'red', 'taste' => 'sweet', 'shape' => 'round',
// 'name' => 'apple', 0 => 4), e $b será o array
// array(0 => 'a', 1 => 'b', 2 => 'c'), ou simplesmente array('a', 'b', 'c').

var_dump($a, $b);
?>
```

Exemplo #24 Utilizando array()

```php
<?php
// Array como (propriedade-)mapa
$map = array( 'versao'     => 4,
              'OS'         => 'Linux',
              'lang'       => 'inglês',
              'short_tags' => true
            );
var_dump($map);

// apenas chaves numéricas
// isto é o mesmo que array(0 => 7, 1 => 8, ...)
$array = array( 7,
                8,
                0,
                156,
                -10
              );
var_dump($array);

$switching = array(         10, // chave = 0
                    5    =>  6,
                    3    =>  7,
                    'a'  =>  4,
                             11, // chave = 6 (o índice máximo era 5)
                    '8'  =>  2, // chave = 8 (inteiro!)
                    '02' => 77, // chave = '02'
                    0    => 12  // o valor 10 será sobrescrito por 12
                  );
var_dump($switching);

// array vazio
$empty = array();
var_dump($empty);
?>
```

Exemplo #25 Coleção

```php
<?php
$cores = array('vermelho', 'azul', 'verde', 'amarelo');
```

```php
foreach ($cores as $cor) {
    echo "Você gosta de $cor?\n";
}

?>
```

O exemplo acima produzirá:

```
Você gosta de vermelho?
Você gosta de azul?
Você gosta de verde?
Você gosta de amarelo?
```

É possível alterar diretamente os valores de um array passando-os como referência.

Exemplo #26 Alterando valores em um laço

```php
<?php
$cores = array('vermelho', 'azul', 'verde', 'amarelo');

foreach ($cores as &$cor) {
    $cor = mb_strtoupper($cor);
}
unset($cor); /* garante que as próximas alterações em
$cor não modificam o último elemento do array */

print_r($cores);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => VERMELHO
    [1] => AZUL
    [2] => VERDE
    [3] => AMARELO
)
```

Este exemplo cria um array na base 1.

Exemplo #27 Array baseado em 1

```php
<?php
$primeiroquarto = array(1 => 'Janeiro', 'Fevereiro', 'Março');
```

```
print_r($primeiroquarto);
?>
```

O exemplo acima produzirá:

```
Array
(
    [1] => Janeiro
    [2] => Fevereiro
    [3] => Março
)
```

Exemplo #28 Preenchendo um array

```
<?php
// preenchendo um array com todos os itens de um diretório
$handle = opendir('.');
while (false !== ($file = readdir($handle))) {
    $files[] = $file;
}
closedir($handle);

var_dump($files);
?>
```

arrays são ordenados. A ordem pode ser modificada utilizando várias funções de ordenação. Veja a seção de funções de arrays para mais informações. A função count() pode ser utilizada para contar o número de itens em um array.

Exemplo #29 Ordenando arrays

```
<?php
sort($files);
print_r($files);
?>
```

Como o valor de um array pode ser qualquer coisa, isso significa que também poderá ser outro array. Isso habilita a criação de arrays recursivos e multidimensionais.

Exemplo #30 Arrays recursivos e multidimensionais

```
<?php
$fruits = array ( "frutas"  => array ( "a" => "laranja",
                                       "b" => "banana",
                                       "c" => "maçã"
                                     ),
                  "numeros" => array ( 1,
                                       2,
                                       3,
                                       4,
```

```
                                    5,
                                    6
                                ),
                "buracos" => array (      "primeiro",
                                    5 => "segundo",
                                         "terceiro",
                                )
            );
var_dump($fruits);

// Alguns exemplos de endereços dos valores do array acima
echo $fruits["buracos"][5];   // imprime "segundo"
echo $fruits["frutas"]["a"];  // imprime "laranja"
unset($fruits["buracos"][0]); // remove  "primeiro"

// Criando um novo array multidimensional
$sucos["maca"]["verde"] = "bom";
var_dump($juices);
?>
```

Atribuições de array sempre envolvem cópias de valores. Use o operador de referência para copiar um array por referência.

Exemplo #31 Copiando arrays

```
<?php
$arr1 = array(2, 3);
$arr2 = $arr1;
$arr2[] = 4; // $arr2 é modificado,
             // $arr1 continua sendo apenas array(2, 3)

$arr3 = &$arr1;
$arr3[] = 4; // agora $arr1 e $arr3 são os mesmos

var_dump($arr1, $arr2, $arr3);
?>
```

Objetos

Inicialização de objetos

Para criar um novo object, utilize a instrução `new` para instanciar uma classe:

Exemplo #1 Construção de objeto

```
<?php
class foo
{
    function do_foo()
    {
        echo "Fazendo foo.";
    }
}

$bar = new foo;
```

```php
$bar->do_foo();
?>
```

Para uma discussão completa, veja o capítulo Classes e Objetos.

Convertendo para objetos

Se um object é convertido para um object, ele não é modificado. Se um valor de qualquer outro tipo é convertido para um object, uma nova instância da classe interna stdClass é criada. Se o valor for `null`, a nova instância será vazia. Um array é convertido para um object com as propriedades nomeadas pelas chaves e com seus valores correspondentes. Antes do PHP 7.1.0, chaves numéricas ficavam inacessíveis a menos que fossem iteradas.

Exemplo #2 Convertendo para um objeto

```php
<?php
$obj = (object) array('1' => 'foo');
var_dump(isset($obj->{'1'})); // exibe 'bool(true)'

// Descontinuado a partir do PHP 8.1
var_dump(key($obj)); // exibe 'string(1) "1"'
?>
```

Para qualquer outro valor, uma propriedade chamada `scalar` conterá o valor.

Exemplo #3 Conversão com `(object)`

```php
<?php
$obj = (object) 'ciao';
echo $obj->scalar;  // exibe 'ciao'
?>
```

Enumerações

(PHP 8 >= 8.1.0)

Enumerações Básicas

Enumerações são uma camada de restrição sobre classes e constantes de classe, destinadas a fornecer uma maneira de definir um conjunto fechado de valores possíveis para um tipo.

```php
<?php
enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

function do_stuff(Suit $s)
{
    // ...
}
```

```php
do_stuff(Suit::Spades);
?>
```

Para uma discussão completa, consulte o capítulo sobre enumerações.

Casting

Se um enum for convertido em um object, ele não será modificado. Se um enum for convertido em um array, será criado um array com uma única chave de `name` (para enumerações puras) ou um array com chaves `name` e `value` (para enumerações com backup). Todos os outros tipos de conversão resultarão em erro.

Recursos

Um resource é uma variável especial, que mantém uma referência a um recurso externo. Recursos são criados e usados por funções especiais. Veja o apêndice para uma lista de todas essas funções e seus tipos resource correspondentes.

Veja também a função get_resource_type().

Convertendo para recurso

Como as variáveis resource mantém manipuladores especiais para arquivos abertos, conexões de bancos de dados, canvas de imagens e coisas do tipo, converter para resource não faz sentido algum.

Liberando recursos

Graças ao sistema de contagem de referência introduzido com o mecanismo Zend, um resource que não tem mais nenhuma referência é detectado automaticamente, e liberado pelo coletor de lixo. Por esta razão, é rara a necessidade de liberação manual de memória.

> **Nota** :
>
> Conexões persistentes de bancos são exceções à regra. Elas *não são* destruídas pelo coletor de lixo. Veja também a seção conexões persistentes para mais informações.

Chamáveis

Um chamável é uma referência a uma função ou a um método que é passada a outra função como um argumento. Chamáveis são representados com a declaração de tipo callable.

```php
<?php
function foo(callable $callback) {
    $callback();
}
?>
```

Algumas funções aceitam funções como um parâmetro, por exemplo, array_map(), usort() ou preg_replace_callback().

Criação de chamáveis

Um objeto chamável é um tipo que representa algo que pode ser invocado. Os objetos invocáveis podem ser passados como argumentos para funções ou métodos que esperam um parâmetro de função de retorno (callback) ou podem ser invocados diretamente. O tipo callable não pode ser usado como uma declaração de tipo para propriedades de classe. Em vez disso, use uma declaração de tipo Closure.

Chamáveis podem ser criados de diferentes formas:

- Objeto Closure
- string contendo o nome de uma função ou método
- array contendo um nome de classe ou um object no índice 0 e o nome do método no índice 1
- object implementando a método mágico __invoke()

Um objeto Closure pode ser criado usando-se a sintaxe de função anônima, de função seta, de chamável de primeira classe, ou com o médoto Closure::fromCallable().

> **Nota** :
>
> A sintaxe de chamável de primeira classe está disponível somente a partir do PHP 8.1.0.

Exemplo #1 Exemplo de função de retorno usando Closure

```php
<?php
// Usando a sintaxe de função anônima
$double1 = function ($a) {
    return $a * 2;
};

// Usando a sintaxe de chamável de primeira classe
function double_function($a) {
    return $a * 2;
}
$double2 = double_function(...);

// Usando a sintaxe de função seta
$double3 = fn($a) => $a * 2;

// Usando Closure::fromCallable
$double4 = Closure::fromCallable('double_function');

// Usando o closure como uma função de retorno aqui para
// duplicar o tamanho de cada elemento no intervalo
$new_numbers = array_map($double1, range(1, 5));
print implode(' ', $new_numbers) . PHP_EOL;

$new_numbers = array_map($double2, range(1, 5));
print implode(' ', $new_numbers) . PHP_EOL;

$new_numbers = array_map($double3, range(1, 5));
print implode(' ', $new_numbers) . PHP_EOL;

$new_numbers = array_map($double4, range(1, 5));
print implode(' ', $new_numbers);
```

```
?>
```

Saída do exemplo acima no PHP 8.1:

```
2 4 6 8 10
2 4 6 8 10
2 4 6 8 10
2 4 6 8 10
```

Um chamável também pode ser uma string contendo o nome de uma função ou de um método estático. Qualquer função interna ou definida pelo usuário pode ser utilizada, com exceção dos construtores de linguagem como: array(), echo, empty(), eval(), isset(), list(), print ou unset().

Métodos estáticos também podem ser passados sem a instância de um object dessa classe criando um array com o nome da classe no índice 0 e o nome do método no índice 1 ou usando a sintaxe especial com operador de resolução de escopo `::` como em `'ClassName::methodName'`.

Um método de um object instanciado pode ser um chamável quando fornecido como um array com o object no índice 0 e o nome do método no índice 1.

A principal diferença entre um objeto Closure e o tipo callable é que um objeto Closure é independente de escopo e sempre pode ser invocado, enquanto um tipo 'callable' pode ser dependente de escopo e pode não ser invocado diretamente. Closure é a maneira preferida de criar objetos chamáveis.

> **Nota** :
>
> Embora os objetos Closure estejam vinculados ao escopo onde são criados, os objetos chamáveis que referenciam métodos de classe como strings ou arrays são resolvidos no escopo onde são chamados. Para criar um objeto chamável a partir de um método privado ou protegido, que pode então ser invocado de fora do escopo da classe, use Closure::fromCallable() ou a sintaxe de chamável de primeira classe.

O PHP permite a criação de chamáveis que podem ser usados como argumento de função de retorno mas não podem ser chamadas diretamente. Essas são funções dependentes do contexto que fazem referência a um método de classe na hierarquia de herança de uma classe, por exemplo: `'parent::method'` ou `["static", "method"]`.

> **Nota** :
>
> A partir do PHP 8.2.0, chamáveis dependentes de contexto foram descontinuados. Remova a dependência de contexto substituindo `'parent::method'` por `parent::class . '::method'` ou use a sintaxe de chamável de primeira classe.

Exemplo #2 Invocando vários tipos de chamáveis com call_user_func()

```php
<?php

// Exemplo de função de retorno
function minha_funcao_retorno() {
    echo 'olá mundo!', PHP_EOL;
}

// Exemplo de método de retorno
class MinhaClasse {
    static function meuMetodoRetorno() {
        echo 'Olá Mundo!', PHP_EOL;
    }
}

// Tipo 1: Função de Retorno Simples
call_user_func('minha_funcao_retorno');

// Tipo 2: Chamada a métodos estáticos
call_user_func(['MinhaClasse', 'meuMetodoRetorno']);

// Tipo 3: Chamada a métodos de objetos
$obj = new MinhaClasse();
call_user_func([$obj, 'meuMetodoRetorno']);

// Tipo 4: Chamada a métodos estáticos
call_user_func('MinhaClasse::meuMetodoRetorno');

// Tipo 5: Chamada a métodos estáticos usando a palavra-chave ::class
call_user_func([MyClass::class, 'myCallbackMethod']);

// Type 6: Relative static class method call
class A {
    public static function quem() {
        echo "A\n", PHP_EOL;
    }
}

class B extends A {
    public static function quem() {
        echo "B\n", PHP_EOL;
    }
}

call_user_func(['B', 'parent::quem']); // descontinuado a partir do PHP 8.2.0

// Type 7: Objetos que implementam __invoke podem ser utilizados como chamáveis
class C {
    public function __invoke($nome) {
        echo 'Olá ', $nome;
    }
}

$c = new C();
call_user_func($c, 'PHP!');
?>
```

O exemplo acima produzirá:

```
olá mundo!
Olá Mundo!
Olá Mundo!
Olá Mundo!
Olá Mundo!

Deprecated: Callables of the form ["B", "parent::who"] are deprecated in script on line 44
A
Olá, PHP!
```

> **Nota** :
>
> Callbacks registrados com funções como call_user_func() e call_user_func_array() não serão chamados se houver uma exceção não capturada que foi lançada em um callback anterior.

Misto

O tipo mixed aceita todos os valores. É equivalente ao tipo de união `object|resource|array|string|float|int|bool|null`. Disponível a partir do PHP 8.0.0.

mixed é, na linguagem da teoria dos tipos, o tipo universal, que significa que todos os outros tipos são um subtipo dele.

Void

void é uma declaração de tipo de retorno indicando que a função não retorna um valor, mas a função ainda pode terminar. Portanto, não pode fazer parte de uma declaração de tipo de união. Disponível a partir do PHP 7.1.0.

> **Nota** :
>
> Mesmo que uma função tenha um tipo de retorno void, ela ainda retornará um valor, esse valor é sempre `null`.

Descartando um valor com `(void)`

A sintaxe `(void)` pode ser usada para descartar explicitamente o resultado de uma expressão. Isso é útil para indicar que ignorar um valor de retorno é intencional, especialmente ao chamar uma função ou método marcado com o atributo NoDiscard.

Diferentemente de outras conversões, `(void)` não converte o valor para outro tipo e não produz um valor. É uma instrução e não pode ser usada como parte de uma expressão.

Exemplo #1 Descartando um valor de retorno

```php
<?php
#[\NoDiscard]
function process(): bool {
    return true;
}

(void) process(); // Descarta explicitamente o valor de retorno
?>
```

Never

never é um tipo de retorno que indica que a função não termina. Isso significa que a função chama exit(), lança uma exceção ou é um loop infinito. Portanto, não pode fazer parte de uma declaração de tipo de união. Disponível a partir do PHP 8.1.0.

never é, na linguagem da teoria dos tipos, o tipo inferior. Isso significa que é o subtipo de todos os outros tipos e pode substituir qualquer outro tipo de retorno durante a herança.

Tipos de classes relativas

Estas declarações de tipo só podem ser usadas dentro de classes.

self

O valor deve ser uma `instanceof` da mesma classe daquela em que a declaração de tipo é usada.

parent

O valor deve ser uma `instanceof` de um pai da classe na qual a declaração de tipo é usada.

static

static é um tipo somente de retorno que requer que o valor retornado seja uma `instanceof` da mesma classe em que o método é chamado. Disponível a partir do PHP 8.0.0.

Tipos únicos

Tipos únicos são aqueles que permitem apenas um valor. O PHP tem suporte para dois tipos únicos: false a partir do PHP 8.0.0, e true a partir do PHP 8.2.0.

> **Aviso**
>
> Antes do PHP 8.2.0, o tipo false só podia ser usado como parte de um tipo união.

> **Nota** :
>
> Não é possível definir tipos únicos customizados. Considere usar enumerações como alternativa.

Iterável

Iterable é um sinônimo de tipo integrado em tempo de compilação para `array|Traversable`. Desde sua introdução no PHP 7.1.0 e antes do PHP 8.2.0, iterable era um pseudotipo integrado que atuava como o sinônimo de tipo mencionado anteriormente e podia ser usado como uma declaração de tipo. Um tipo iterável pode ser usado em `foreach` e com **yield from** de dentro de um gerador.

> **Nota** :
>
> As funções que declaram iterável como um tipo de retorno também podem ser geradoras.
>
> **Exemplo #1 Exemplo de tipo de retorno do gerador iterável**

```php
<?php

function gen(): iterable {
    yield 1;
    yield 2;
    yield 3;
}

foreach(gen() as $value) {
    echo $value, "\n";
}
?>
```

Declarações de tipo

As declarações de tipo podem ser adicionadas a argumentos de função, valores de retorno, a partir do PHP 7.4.0, propriedades de classe e, a partir do PHP 8.3.0, constantes de classe. Elas garantem que o valor seja do tipo especificado no momento da chamada, caso contrário, um TypeError é lançado.

Cada tipo que o PHP suporta, com exceção de resource, pode ser usado dentro de uma declaração de tipo de usuário. Esta página contém um changelog de disponibilidade dos diferentes tipos e documentação sobre o uso deles em declarações de tipo.

> **Nota** :
>
> Quando uma classe implementa um método de interface ou reimplementa um método já definido por uma classe pai, ela deve ser compatível com a definição acima. Um método é compatível se seguir as regras de variância.

Registro de Alterações

Versão	Descrição
8.3.0	Suporte à constantes em classes, interfaces, traits e enums foi adicionado.
8.2.0	O suporte para tipos DNF foi adicionado.
8.2.0	O suporte para o tipo literal true foi adicionado.
8.2.0	Os tipos null e false agora podem ser usados de forma independente.
8.1.0	O suporte para tipos de interseção foi adicionado.

Versão	Descrição
8.1.0	Retornar por referência de uma função void agora foi descontinuado.
8.1.0	O suporte para o tipo somente retorno never foi adicionado.
8.0.0	O suporte para mixed foi adicionado.
8.0.0	O suporte para o tipo de retorno apenas static foi adicionado.
8.0.0	O suporte para tipos de união foi adicionado.
7.4.0	Suporte para propriedades tipadas foi adicionado.
7.2.0	Suporte para object foi adicionado.
7.1.0	Suporte para iterable foi adicionado.
7.1.0	Suporte para void foi adicionado.
7.1.0	O suporte para tipos anuláveis foi adicionado.

Notas de uso em tipos atômicos

Os tipos atômicos têm um comportamento direto com algumas ressalvas menores que são descritas nesta seção.

Tipos escalares

> **Aviso**
>
> Aliases de nome para tipos escalares (bool, int, float, string) não são suportados. Em vez disso, eles são tratados como nomes de classe ou interface. Por exemplo, usar `boolean` como uma declaração de tipo exigirá que o valor seja uma `instanceof` da classe ou interface `boolean`, em vez do tipo bool:

```php
<?php
    function test(boolean $param) {}
    test(true);
?>
```

Saída do exemplo acima no PHP 8:

```
Warning: "boolean" will be interpreted as a class name. Did you mean "bool"? Write "\boolean"
to suppress this warning in /in/9YrUX on line 2

Fatal error: Uncaught TypeError: test(): Argument #1 ($param) must be of type boolean, bool
given, called in - on line 3 and defined in -:2
Stack trace:
#0 -(3): test(true)
#1 {main}
  thrown in - on line 2
```

void

> **Nota** :
>
> Retornar por referência de uma função void foi descontinuado a partir do PHP 8.1.0, porque tal função é contraditória. Anteriormente, ele já emitia o seguinte `E_NOTICE` quando chamado: Only variable references should be returned by reference.
>
> ```
> <?php
> function &test(): void {}
> ?>
> ```

Tipos que podem ser chamados

Este tipo não pode ser usado como uma declaração de tipo de propriedade de classe.

> **Nota** :
>
> Não é possível especificar a assinatura da função.

Declarações de tipo em parâmetros de passagem por referência

Se um parâmetro de passagem por referência tiver uma declaração de tipo, o tipo da variável é *somente* verificado na entrada da função, no início da chamada, mas não quando a função retorna. Isso significa que uma função pode alterar o tipo da variável passada por referência.

Exemplo #1 Parâmetros de passagem por referência digitados

```
<?php
function array_baz(array &$param)
{
    $param = 1;
}
$var = [];
array_baz($var);
var_dump($var);
array_baz($var);
?>
```

O exemplo acima produzirá algo semelhante a:

```
int(1)

Fatal error: Uncaught TypeError: array_baz(): Argument #1 ($param) must be of type array, int
given, called in - on line 9 and defined in -:2
Stack trace:
#0 -(9): array_baz(1)
#1 {main}
  thrown in - on line 2
```

Notas de Uso de Tipos Compostos

As declarações de tipo composto estão sujeitas a algumas restrições e realizarão uma verificação de redundância no tempo de compilação para evitar erros simples.

> **Cuidado**
>
> Antes do PHP 8.2.0 e da introdução dos tipos DNF, não era possível combinar tipos de interseção com tipos de união.

União de tipos

> **Aviso**
>
> Não é possível combinar os dois tipos únicos false e true juntos em um tipo de união. Em vez disso, use bool.

> **Cuidado**
>
> Antes do PHP 8.2.0, como false e null não podiam ser usados como tipos autônomos, um tipo de união composto apenas por esses tipos não era permitido. Isso inclui os seguintes tipos: false, `false|null`, e `?false`.

Açúcar sintático do tipo anulável

Uma única declaração de tipo base pode ser marcada como anulável prefixando o tipo com um ponto de interrogação (`?`). Assim, `?T` e `T|null` são idênticos.

> **Nota** :
>
> Esta sintaxe é suportada a partir do PHP 7.1.0, e é anterior ao suporte generalizado de tipos de união.

> **Nota** :
>
> Também é possível obter argumentos anuláveis tornando `null` o valor padrão. Isso não é recomendado porque se o valor padrão for alterado em uma classe filha, uma violação de compatibilidade de tipo será gerada, pois o tipo null precisará ser adicionado à declaração de tipo. Este comportamento também foi descontinuado desde o PHP 8.4.
>
> **Exemplo #2 Maneira antiga de tornar os argumentos anuláveis**
>
> ```php
> <?php
> class C {}
>
> function f(C $c = null) {
>     var_dump($c);
> }
>
> f(new C);
> ```

```php
f(null);
?>
```

O exemplo acima produzirá:

```
object(C)#1 (0) {
}
NULL
```

Tipos duplicados e redundantes

Para capturar bugs simples em declarações de tipo composto, tipos redundantes que podem ser detectados sem executar o carregamento de classe resultarão em um erro de tempo de compilação. Isso inclui:

- Cada tipo resolvido por nome pode ocorrer apenas uma vez. Tipos como `int|string|INT` ou `Countable&Traversable&COUNTABLE` resultam em erro.
- O uso de mixed ou never resulta em um erro.
- Para tipos de união:
  - Se bool for usado, false ou true não pode ser usado adicionalmente.
  - Se object for usado, os tipos de classe não podem ser usados adicionalmente.
  - Se iterable for usado, array e Traversable não podem ser usados adicionalmente.
- Para interseção de tipos:
  - Usar um tipo que não é um tipo de classe resulta em erro.
  - O uso de self, parent ou static resulta em erro.
- Para tipos DNF:
  - Se um tipo mais genérico for usado, o mais restritivo será redundante.
  - Usando dois tipos de interseção idênticos.

> **Nota** :
>
> Isso não garante que o tipo seja "mínimo", porque isso exigiria o carregamento de todos os tipos de classe usados.

Por exemplo, se `A` e `B` são aliases de classe, `A|B` continua sendo um tipo de união legal, mesmo que possa ser reduzido a `A` ou `B`. Da mesma forma, se a classe `B extends A {}`, `A|B` também é um tipo de união legal, embora pudesse ser reduzida a apenas `A`.

```php
<?php
function foo(): int|INT {} // Proibido
function foo(): bool|false {} // Proibido
function foo(): int&Traversable {} // Proibido
function foo(): self&Traversable {} // Proibido

use A as B;
function foo(): A|B {} // Não permitido ("use" faz parte da resolução de nomes)
function foo(): A&B {} // Não permitido ("use" faz parte da resolução de nomes)

class_alias('X', 'Y');
function foo(): X|Y {} // Permitido (a redundância só é conhecida em tempo de execução)
function foo(): X&Y {} // Permitido (a redundância só é conhecida em tempo de execução)
?>
```

Examples

Exemplo #3 Declaração de tipo de classe básica

```php
<?php
class C {}
class D extends C {}

// Isso não estende C.
class E {}

function f(C $c) {
    echo get_class($c)."\n";
}

f(new C);
f(new D);
f(new E);
?>
```

Saída do exemplo acima no PHP 8:

```
C
D

Fatal error: Uncaught TypeError: f(): Argument #1 ($c) must be of type C, E given, called in
/in/gLonb on line 14 and defined in /in/gLonb:8
Stack trace:
#0 -(14): f(Object(E))
#1 {main}
  thrown in - on line 8
```

Exemplo #4 Declaração de tipo de interface básica

```php
<?php
interface I { public function f(); }
```

```php
class C implements I { public function f() {} }

// Isso não implementa I.
class E {}

function f(I $i) {
    echo get_class($i)."\n";
}

f(new C);
f(new E);
?>
```

Saída do exemplo acima no PHP 8:

```
C

Fatal error: Uncaught TypeError: f(): Argument #1 ($i) must be of type I, E given, called in - on line 13 and defined in -:8
Stack trace:
#0 -(13): f(Object(E))
#1 {main}
  thrown in - on line 8
```

Exemplo #5 Declaração de tipo de retorno básico

```php
<?php
function sum($a, $b): float {
    return $a + $b;
}

// Observe que um float será retornado.
var_dump(sum(1, 2));
?>
```

O exemplo acima produzirá:

```
float(3)
```

Exemplo #6 Retornando um objeto

```php
<?php
class C {}

function getC(): C {
    return new C;
```

```php
}

var_dump(getC());
?>
```

O exemplo acima produzirá:

```
object(C)#1 (0) {
}
```

Exemplo #7 Declaração de tipo de argumento anulável

```php
<?php
class C {}

function f(?C $c) {
    var_dump($c);
}

f(new C);
f(null);
?>
```

O exemplo acima produzirá:

```
object(C)#1 (0) {
}
NULL
```

Exemplo #8 Declaração de tipo de retorno anulável

```php
<?php
function get_item(): ?string {
    if (isset($_GET['item'])) {
        return $_GET['item'];
    } else {
        return null;
    }
}
?>
```

Exemplo #9 Declaração de tipo de propriedade

```php
<?php
class User {
    public static string $foo = 'foo';
```

```
    public int $id;
    public string $username;

    public function __construct(int $id, string $username) {
        $this->id = $id;
        $this->username = $username;
    }
}
?>
```

Tipagem estrita

Por padrão, o PHP forçará valores do tipo errado na declaração de tipo escalar esperada, se possível. Por exemplo, uma função que recebe um int para um parâmetro que espera uma string obterá uma variável do tipo string.

É possível ativar o modo estrito por arquivo. No modo estrito, apenas um valor correspondente exatamente à declaração do tipo será aceito, caso contrário, um TypeError será lançado. A única exceção a essa regra é que um valor int passará por uma declaração do tipo float.

> **Aviso**
>
> Chamadas de função dentro de funções internas não serão afetadas pela declaração `strict_types`.

Para habilitar o modo estrito, a `declare` é usada com a declaração `strict_types`:

> **Nota** :
>
> A digitação estrita se aplica a chamadas de função feitas de *dentro* do arquivo com tipagem estrita habilitada, não para as funções declaradas dentro desse arquivo. Se um arquivo sem tipagem estrita habilitada fizer uma chamada para uma função que foi definida em um arquivo com tipagem estrita, a preferência do chamador (tipagem coercitiva) será respeitada e o valor será convertido.

> **Nota** :
>
> A tipagem estrita é definida apenas para declarações de tipo escalar.

Exemplo #10 Tipagem estrita para valores de argumentos

```
<?php
declare(strict_types=1);

function sum(int $a, int $b) {
    return $a + $b;
}

var_dump(sum(1, 2));
var_dump(sum(1.5, 2.5));
?>
```

Saída do exemplo acima no PHP 8:

```
int(3)

Fatal error: Uncaught TypeError: sum(): Argument #1 ($a) must be of type int, float given, called
in - on line 9 and defined in -:4
Stack trace:
#0 -(9): sum(1.5, 2.5)
#1 {main}
  thrown in - on line 4
```

Exemplo #11 Tipagem coercitiva para valores de argumento

```php
<?php
function sum(int $a, int $b) {
    return $a + $b;
}

var_dump(sum(1, 2));

// Estes serão convertidos em números inteiros: observe a saída abaixo!
var_dump(sum(1.5, 2.5));
?>
```

O exemplo acima produzirá:

```
int(3)
int(3)
```

Exemplo #12 Tipagem estrita para valores de retorno

```php
<?php
declare(strict_types=1);

function sum($a, $b): int {
    return $a + $b;
}

var_dump(sum(1, 2));
var_dump(sum(1, 2.5));
?>
```

O exemplo acima produzirá:

```
int(3)

Fatal error: Uncaught TypeError: sum(): Return value must be of type int, float returned in -:5
Stack trace:
#0 -(9): sum(1, 2.5)
#1 {main}
  thrown in - on line 5
```

Conversão automática de tipos

O PHP não requer a tipagem explícita em declarações de variáveis. Nesse caso, o tipo de uma variável é determinado pelo valor armazenado. Ou seja, se uma string é armazenada numa variável *$var*, então *$var* terá o tipo string. Se após isso um valor int for armazenado em *$var*, ela será do tipo int.

O PHP pode tentar converter o tipo de um valor em outro automaticamente em alguns contextos. Esses contextos são:

- Numérico
- String
- Lógico
- Inteiro e string
- Comparação
- Função

> **Nota** :
>
> Quando um valor precisa ser interpretado como um tipo diferente, o valor em si *não* se altera, apenas uma "cópia" é convertida.

Para forçar uma variável a ser avaliada como um certo tipo, consulte a seção sobre conversão de tipo. Se for necessário mudar o tipo de uma variável, consulte a função settype().

Contexto numérico

Este contexto é utilizado em operadores aritméticos.

Neste contexto, se um dos operandos for um float (ou não interpretável como int), ambos os operandos serão interpretados como float, e o resultado será um float. Caso contrário, os operandos serão interpretados como int, e o resultado também será um int. A partir do PHP 8.0.0, se um dos operandos não puser ser interpretado, um TypeError é lançado.

Contextos string

Este contexto ocorre ao se utilizar echo, print, interpolação de strings ou o operador de concatenação.

Neste contexto, o valor será interpretado como string. Se o valor não puder ser interpretado, um TypeError é lançado. Anteriormente ao 7.4.0, um `E_RECOVERABLE_ERROR` era emitido.

Contextos lógicos

Este contexto ocorre em instruções condicionais, no uso do operador ternário, ou em uma operação lógica.

Neste contexto, o valor será interpretado como bool.

Contextos inteiros e string

Este contexto ocorre ao utilizar operadores binários.

Neste contexto, se todos os operandos forem do tipo string, o resultado também será uma string. Caso contrário, os operados serão interpretados como int, e o resultado também será um int. A partir do PHP 8.0.0, se um dos operandos não puder ser interpretado, um TypeError é lançado.

Contextos de comparação

Este contexto ocorre ao utilizar operadores de comparação.

As conversões de tipo que ocorrem neste contexto são explicadas na tabela de comparações entre vários tipos.

Contextos de função

Este contexto ocorre quando um valor é um parâmetro tipado ou quando uma propriedade ou valor retornado de uma função declara um tipo.

Nesse contexto, o valor precisa ser do tipo informado. Existem duas exceções. A primeira é: se o valor for do tipo int e o tipo declarado for um float, o inteiro será convertido num número de ponto flutuante. A segunda é: se o tipo declarado for um tipo *escalar*, ou ser o valor for conversível a um tipo escalar e o modo de tipagem coercitiva estiver ativo (o padrão), o valor pode ser convertido para um valor escalar aceito. Veja abaixo uma descrição deste comportamento.

> **Aviso**
>
> Funções internas convertem automaticamente `null` para tipos escalares. Este comportamento foi *DESCONTINUADO* a partir do PHP 8.1.0.

Coerção de tipos com tipos declarados

- Declaração de tipo bool: o valor é convertido para bool.
- Declaração de tipo int: o valor é convertido para int quando a conversão for bem definida. Por exemplo, quando uma string é numérica.
- Declaração de tipo float: o valor é convertido para float quando a conversão for bem definida. Por exemplo, quando uma string é numérica.
- Declaração de tipo string: o valor é convertido para string.

Conversão de tipos em tipos union

Quando `strict_types` está desativado, declarações de tipos escalares estão sujeitas a conversões implícitas e limitadas. Se o tipo exato do valor não é parte da união, então o tipo final da conversão é escolhido na seguinte ordem de preferência:

1. int
2. float
3. string
4. bool

Se o tipo existe na união e o valor puder ser convertido pela mesma semântica do PHP, então esse tipo é escolhido. Senão, um tipo seguinte é tentado.

> **Cuidado**
>
> Existe uma exceção, no caso do valor ser uma string, e ambos int e float serem partes da união, o tipo preferido será determinado pela semântica de strings numéricas. Por exemplo, para `"42"` o tipo int é escolhido, enquanto que para `"42.0"` o tipo escolhido é float.

> **Nota** :
>
> Tipos que não fazem parte da lista acima não são elegíveis para destinos da conversão implícita. Em particular, não ocorrem conversões implícitas para os tipos null, false e true.

Exemplo #1 Exemplo de tipos sendo convertidos em um tipo parte da união

```
<?php
// int|string
42    --> 42          // tipo exato
"42"  --> "42"        // tipo exato
new ObjectWithToString --> "Resultado de __toString()"
                      // e objero não compatível com int, padrão para string
42.0  --> 42          // float compatível com int
42.1  --> 42          // float compatível com int
1e100 --> "1.0E+100"  // float muito grande para int, padrão para string
INF   --> "INF"       // float muito grande para int, padrão para string
true  --> 1           // bool compatível com int
[]    --> TypeError   // array não é compatível com int ou string

// int|float|bool
"45"    --> 45        // string numérica int
"45.0"  --> 45.0      // string numérica float

"45X"   --> true      // string não numérica, padrão para bool
""      --> false     // string não numérica, padrão para bool
"X"     --> true      // string não numérica, padrão para bool
[]      --> TypeError // array não é compatível com int, float ou bool
?>
```

Conversão explícita

Converte o valor do tipo para o tipo escolhido através da escrita do tipo entre parênteses, antes do valor a ser convertido.

Exemplo #2 Conversão de tipo

```php
<?php
$foo = 10;           // $foo é um inteiro
$bar = (bool) $foo;  // $bar é um booleano

var_dump($bar);
?>
```

As conversões previstas são:

- `(int)` - converte para int
- `(bool)` - converte para bool
- `(float)` - converte para float
- `(string)` - converte para string
- `(array)` - converte para array
- `(object)` - converte para object
- `(unset)` - converte para NULL

A conversão `(void)` também está disponível a partir do PHP 8.5.0, mas não é uma conversão de valor. Ela é usada como uma instrução para descartar explicitamente o resultado de uma expressão.

> **Aviso**
>
> `(integer)` é um sinônimo de `(int)`. `(boolean)` é um sinônimo de `(bool)`. `(binary)` é um sinônimo de `(string)`. `(double)` e `(real)` são sinônimos de `(float)`. Essas conversões alternativas não usam o nome canônico de tipo e foram descontinuadas a partir do PHP 8.5.0.

> **Aviso**
>
> A conversão `(real)` foi descontinuada a partir do PHP 7.4.0 e removida a partir do PHP 8.0.0.

> **Aviso**
>
> A conversão `(unset)` foi descontinuada a partir do PHP 7.2.0. Observe que a conversão `(unset)` tem o mesmo efeito que informar o valor NULL para a variável ou chamada de função. A conversão `(unset)` foi removida no PHP 8.0.0.

> **Cuidado**
>
> A conversão `(binary)` e o prefixo `b` existem para compatibilidade futura. Atualmente, `(binary)` e `(string)` são idênticos. No entanto, isso pode mudar e não deve ser considerado estável.

> **Nota** :
>
> Espaços são ignorados dentro dos parênteses de uma conversão. Portanto, as linhas seguintes são equivalentes:

```php
<?php
$foo = (int) $bar;
$foo = ( int ) $bar;
?>
```

Convertendo string e variáveis para strings binárias:

```php
<?php
$binary = (binary) $string;
$binary = b"binary string";
?>
```

Em vez de converter uma variável para string, é possível delimitar a variável entre aspas.

Exemplo #3 Diferentes mecanismos de conversão

```php
<?php
$foo = 10;            // $foo é int
$str = "$foo";        // $str é string
$fst = (string) $foo; // $fst também é string

// Isto exibe "são iguais"
if ($fst === $str) {
    echo "são iguais", PHP_EOL;
}
?>
```

Não é tão óbvio o que acontece ao converter entre certos tipos. Para mais informações, veja as seções:

- Convertendo para boolean
- Convertendo para integer
- Convertendo para float
- Convertendo para string
- Convertendo para array
- Convertendo para object
- Convertendo para resource
- Convertendo para NULL
- Descartando um valor com `(void)`
- Tabela de conversão de tipos

> **Nota** :
>
> O PHP suporta a indexação de string utilizando a mesma sintaxe de arrays, de forma que o exemplo seguinte é válido em todas as versões do PHP:
>
> **Exemplo #4 Usando índice de array com uma string**
>
> ```php
> <?php
> $a    = 'car'; // $a é uma string
> $a[0] = 'b';   // $a ainda é uma string
> echo $a;       // bar
> ?>
> ```

> Veja a seção sobre Acesso a caracteres da string para mais informações.

Variáveis

Índice

Básico

As variáveis no PHP são representadas por um cifrão ($) seguido pelo nome da variável. Os nomes de variável são sensíveis a maiúsculas e minúsculas.

Um nome válido de variável inicia com uma letra (`A-Z`, `a-z`, ou os bytes de 128 a 255) ou sublinhado, seguido de qualquer número de letras, números ou sublinhados. Em uma expressão regular, poderia ser representado assim: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`

> **Nota** :
>
> O PHP não suporta nomes de variáveis em Unicode, porém, algumas codificações de caracteres (tal como UTF-8) codificam os caracteres de uma forma que todos os bytes de um caractere multibyte fiquem dentro da faixa permitida, fazendo com que o nome da variável seja válido.

> **Nota** :
>
> `$this` é uma variável especial que não pode ser atribuída. Antes do PHP 7.1.0, atribuição indireta (por exemplo, usando variáveis variáveis) era possível.

> **Dica**
>
> Veja também o Guia de nomenclatura em espaço de usuário.

Exemplo #1 Nomes de variáveis válidos

```php
<?php
$var = 'Bob';
$Var = 'Joe';
echo "$var, $Var";      // exibe "Bob, Joe"

$_4site = 'not yet';    // válido; começa com um sublinhado
$täyte = 'mansikka';    // válido; 'ä' é um caracter ASCII (estendido) 228
```

Exemplo #2 Nomes de variáveis inválidos

```php
<?php
$4site = 'not yet';     // inválido; começa com um número
```

O PHP aceita uma sequência de quaisquer bytes como um nome de variável. Nomes de variáveis que não seguem as regras de nomenclatura mencionadas acima só podem ser acessadas dinamicamente no momento da execução. Consulte a seção sobre variáveis variáveis para informação sobre como acessá-las.

Exemplo #3 Acessando nomes obscuros de variáveis

```php
<?php
${'invalid-name'} = 'bar';
$name = 'invalid-name';
echo ${'invalid-name'}, " ", $$name;
```

O exemplo acima produzirá:

```
bar bar
```

Por padrão, as variáveis são sempre atribuídas por valor. Isto significa que ao atribuir uma expressão a uma variável, o valor da expressão original é copiado integralmente para a variável de destino. Isto significa também que, após atribuir o valor de uma variável a outra, a alteração de uma destas variáveis não afetará a outra. Para maiores informações sobre este tipo de atribuição, veja o capítulo em Expressões.

O PHP também oferece um outro meio de atribuir valores a variáveis: atribuição por referência. Isto significa que a nova variável simplesmente referencia (em outras palavras, "torna-se um apelido para" ou "aponta para") a variável original. Alterações na nova variável afetam a original, e vice-versa.

Para atribuir por referência, simplesmente adicione um e-comercial (&) na frente do nome da variável que estiver sendo atribuída (variável de origem). Por exemplo, o trecho de código abaixo exibe '`My name is Bob`' duas vezes:

```php
<?php
$foo = 'Bob';              // Atribui o valor 'Bob' a variável $foo
$bar = &$foo;              // Referencia $foo através de $bar.
$bar = "My name is $bar";  // Altera $bar...
echo $bar . PHP_EOL;
echo $foo . PHP_EOL;       // $foo é alterada também.
```

Uma observação importante a se fazer, é que somente variáveis podem ser atribuídas por referência.

```php
<?php
$original = 25;
$ref1 = &$original;      // Esta atribuição é válida.
$ref2 = &(24 * 7);       // Inválida; referencia uma expressão sem nome.
```

```php
<?php
// Observe a falta de & indicando uma referência retornada na declaração da função,
// ou seja, a função não retorna uma referência, portanto o resultado não pode ser atribuído por
referência
function test()
{
   $original = 25;
   return $original;
}

// Nota: Um aviso é emitido mas o valor de não-referência é atribuído
$result1 = &test();   // Inválido porque test() não retorna uma variável por referência.
```

```php
var_dump($result1);

// A definição desta função mostra que ela retorna uma referência mas não retorna uma variável
function &test2()
{
    return 26;        // Inválido porque o valor de retorno não é uma referência a uma variável.
}

// Nota: o valor de não-referência é atribuído
$result2 = &test2();
var_dump($result2);
```

Não é necessário declarar variáveis no PHP, contudo é uma ótima prática. Acessar uma variável indefinida resultará em um `E_WARNING` (antes do PHP 8.0.0, `E_NOTICE`). Uma variável indefinida tem um valor padrão de `null`. O construtor de linguagem isset() pode ser usado para detectar se uma variável já foi inicializada.

Exemplo #4 Valor padrão de uma variável não inicializada

```php
<?php
// Variável não definida E não referenciada (sem contexto de uso);
var_dump($unset_var);
```

O exemplo acima produzirá:

```
Warning: Undefined variable $unset_var in ...
NULL
```

O PHP permite autovivificação de array (criação automática de novos arrays) a partir de uma variável indefinida. Anexar um elemento a uma variável indefinida criará um novo array e não gerará um alerta.

Exemplo #5 Autovivificação de um array a partir de uma variável indefinida

```php
<?php
$unset_array[] = 'value'; // Não gera um alerta.
var_dump($unset_array);
```

> **Aviso**
>
> Depender do valor padrão de uma variável não inicializada é problemático ao incluir um arquivo em outro que usa o mesmo nome de variável.

Uma variável pode ser destruída usando o construtor de linguagem unset().

Para obter informações sobre funções relacionadas a variáveis, consulte a referência de funções variáveis.

Variáveis Predefinidas

O PHP fornece um número de variáveis predefinidas. O PHP também fornece um conjunto adicional de arrays predefinidos contendo as variáveis do servidor web (se aplicável), as variáveis de ambiente e as entradas do

usuário. Esses estão automaticamente disponíveis em qualquer escopo. Por essa razão, também são conhecidos como "superglobais" (não há um mecanismo no PHP para superglobais definidos pelo usuário). Consulte a lista de superglobais para mais detalhes.

> **Nota** : **Variáveis variáveis**
>
> Superglobais não podem ser utilizadas como variáveis variáveis dentro de funções ou métodos de classe.

Se determinadas variáveis não estiverem definidas na diretiva variables_order, seus arrays predefinidos também estarão vazios.

Escopo de variáveis

O escopo de uma variável é o contexto onde ela está definida. O PHP tem um escopo de função e um escopo global. Qualquer variável definida fora de uma função está limitada ao escopo global. Quando um arquivo é incluído, o código que ele contém herda o escopo da variável da linha na qual a inclusão ocorre.

Exemplo #1 Exemplo de escopo global de variável

```
<?php
$a = 1;
include 'b.inc'; // Variável $a estará disponível dentro de b.inc
```

Qualquer variável criada dentro de uma função nomeada ou de uma função anônima é limitada ao escopo do corpo da função. Entretanto, funções de seta vinculam variáveis do escopo pai para disponibilizá-las dentro do corpo. Se uma inclusão de arquivo ocorrer dentro de uma função dentro do arquivo chamador, as variáveis contidas no arquivo chamado estarão disponíveis como se tivessem sido definidas dentro da função chamadora.

Exemplo #2 Exemplo de escopo local de variável

```
<?php
$a = 1; // escopo global

function test()
{
    var_dump($a); // Variável $a é indefinida já que refere-se a uma versão local de $a
}

test();
```

O exemplo acima irá gerar um `E_WARNING` de variável indefinida (ou um `E_NOTICE` antes do PHP 8.0.0). Isto acontece porque a instrução echo refere-se a uma versão local da variável *$a*, e não possui nenhum valor atribuído neste escopo. Pode-se perceber que esta é uma pequena diferença em relação à linguagem C, em que variáveis globais estão automaticamente disponíveis para funções a menos que tenham sido substituídas por uma definição local. Isto pode causar problemas quando uma variável global for inadvertidamente modificada. No PHP, as variáveis globais precisam ser declaradas como globais dentro de uma função, se elas precisarem ser utilizadas na função.

A palavra-chave `global`

A palavra-chave `global` é usada para vincular uma variável de um escopo global a um escopo local. A palavra-chave pode ser usada com uma lista de variáveis ou com uma única variável. Uma variável local será criada referenciando a variável global de mesmo nome. Se a variável global não existir, a variável será criada no escopo global e receberá o valor `null`.

Exemplo #3 Usando `global`

```
<?php
$a = 1;
$b = 2;

function Soma()
{
    global $a, $b;

    $b = $a + $b;
}

Soma();
echo $b;
```

O exemplo acima produzirá:

```
3
```

Declarar *$a* e *$b* como globais na função fará com que todas as referências a essas variáveis refiram-se às versões globais. Não há um limite para o número de variáveis globais que podem ser manipuladas por uma função.

Uma segunda maneira de acessar variáveis do escopo global é utilizando o array especial *$GLOBALS* definido pelo PHP. O exemplo anterior poderia ser reescrito como:

Exemplo #4 Usando *$GLOBALS* no lugar de global

```
<?php
$a = 1;
$b = 2;

function Soma()
{
    $GLOBALS['b'] = $GLOBALS['a'] + $GLOBALS['b'];
}

Soma();
echo $b;
```

O array *$GLOBALS* é um array associativo, com o nome da variável global na chave e o conteúdo dessa variável no valor do elemento do array. Veja que *$GLOBALS* existe em qualquer escopo, isto porque *$GLOBALS* é uma superglobal. Eis um exemplo demonstrando o poder das superglobais:

Exemplo #5 Exemplo demonstrando superglobals e escopos

```php
<?php
function test_superglobal()
{
    echo $_POST['name'];
}
```

> **Nota** :
>
> Utilizar a instrução `global` fora de uma função não é um erro. Pode ser utilizado se o arquivo for incluído de dentro de uma função.

Utilizando variáveis `static`

Outro recurso importante do escopo de variáveis é a variável *estática*. Uma variável estática existe somente no escopo local da função, mas não perde seu valor quando a execução do programa deixa o escopo. Considere o seguinte exemplo:

Exemplo #6 Exemplo demonstrando a necessidade de variáveis estáticas

```php
<?php
function Teste()
{
    $a = 0;
    echo $a . PHP_EOL;
    $a++;
}

test();
test();
test();
```

Essa função é inútil, já que cada vez que é chamada, define *$a* com o valor `0`, e exibe `0`. A instrução *$a++* , que aumenta o valor da variável, não tem sentido já que assim que a função termina, a variável *$a* desaparece. Para fazer um função de contagem útil, que não perderá a contagem atual, a variável *$a* será declarada como estática:

Exemplo #7 Exemplo de uso de variáveis estáticas

```php
<?php
function Teste()
{
    static $a = 0;
    echo $a . PHP_EOL;
    $a++;
}

test();
test();
test();
```

Agora, a variável *$a* é inicializada apenas na primeira chamada da função e cada vez que a função `test()` for chamada, exibirá o valor de *$a* e depois o incrementará.

Variáveis estáticas fornecem uma solução para lidar com funções recursivas. A seguinte função recursiva conta até 10, utilizando a variável estática *$count* para saber quando parar:

Exemplo #8 Variáveis estáticas em funções recursivas

```php
<?php
function Teste()
{
    static $count = 0;

    $count++;
    echo $count . PHP_EOL;
    if ($count < 10) {
        Teste();
    }
    $count--;
}

test();
```

Antes do PHP 8.3.0, variáveis estáticas somente poderiam ser inicializadas usando uma expressão constante. A partir do PHP 8.3.0, expressões dinâmicas (por exemplos, chamadas de funções) também são permitidas:

Exemplo #9 Declarando variáveis estáticas

```php
<?php
function foo(){
    static $int = 0;          // correto
    static $int = 1+2;        // correto
    static $int = sqrt(121);  // correto a partir do PHP 8.3

    $int++;
    echo $int;
}
```

Variáveis estáticas dentro de funções anônimas persistem apenas dentro daquela instância de função específica. Se a função anônima for recriada em cada chamada, a variável estática será reinicializada.

Exemplo #10 Variáveis estáticas em funções anônimas

```php
<?php
function funcaoExemplo($input) {
    $result = (static function () use ($input) {
        static $counter = 0;
        $counter++;
        return "Entrada: $input, Contador: $counter\n";
    });

    return $result();
}

// Chamadas a funcaoExemplo irão recriar a função anônima, de forma
```

```
// que a variável estática não reterá seu valor.
echo funcaoExemplo('A'); // Exibe: Entrada: A, Contador: 1
echo funcaoExemplo('B'); // Exibe: Entrada: B, Contador: 1
```

A partir do PHP 8.1.0, quando um método com variáveis estáticas é herdado (mas não sobrescrito), o método herdado não compartilhará as variáveis estáticas do método acima. Isso significa que variáveis estáticas nos métodos agora se comportam da mesma forma que propriedades estáticas.

A partir do PHP 8.3.0, variáveis estáticas podem ser inicializadas com expressões arbitrárias. Isto significa que chamadas de métodos, por exemplo, podem ser usadas para iniciá-las.

Exemplo #11 Uso de variáveis estáticas em métodos herdados

```
<?php
class Foo {
    public static function counter() {
        static $counter = 0;
        $counter++;
        return $counter;
    }
}
class Bar extends Foo {}
var_dump(Foo::counter()); // int(1)
var_dump(Foo::counter()); // int(2)
var_dump(Bar::counter()); // int(3), e antes do PHP 8.1.0 int(1)
var_dump(Bar::counter()); // int(4), e antes do PHP 8.1.0 int(2)
```

Referências em variáveis `global` e `static`

O PHP implementa os modificadores static e global para variáveis, em termo de referências. Por exemplo, uma variável global verdadeira, importada dentro do escopo de uma função com a instrução `global`, na verdade, cria uma referência para a variável global. Isto pode levar a comportamentos imprevisíveis nos seguintes casos:

```
<?php
function test_global_ref() {
    global $obj;
    $new = new stdClass;
    $obj = &$new;
}

function test_global_noref() {
    global $obj;
    $new = new stdClass;
    $obj = $new;
}

test_global_ref();
var_dump($obj);
test_global_noref();
var_dump($obj);
```

O exemplo acima produzirá:

```
```

```
NULL
object(stdClass)#1 (0) {
}
```

Um comportamento similar se aplica à declaração `static`. Referências não são armazenadas estaticamente:

```php
<?php
function &get_instance_ref() {
    static $obj;

    echo 'Objeto estático: ';
    var_dump($obj);
    if (!isset($obj)) {
        $new = new stdClass;
        // Atribui uma referência à variável estática
        $obj = &$new;
    }
    if (!isset($obj->property)) {
        $obj->property = 1;
    } else {
        $obj->property++;
    }
    return $obj;
}

function &get_instance_noref() {
    static $obj;

    echo "Objeto estático: ";
    var_dump($obj);
    if (!isset($obj)) {
        $new = new stdClass;
        // Atribui o objeto à variável estática
        $obj = $new;
    }
    if (!isset($obj->property)) {
        $obj->property = 1;
    } else {
        $obj->property++;
    }
    return $obj;
}

$obj1 = get_instance_ref();
$still_obj1 = get_instance_ref();
echo "\n";
$obj2 = get_instance_noref();
$still_obj2 = get_instance_noref();
```

O exemplo acima produzirá:

```
Objeto estático: NULL
Objeto estático: NULL

Objeto estático: NULL
Objeto estático: object(stdClass)#3 (1) {
  ["property"]=>
  int(1)
}
```

Este exemplo demonstra que ao atribuir uma referência a uma variável estática, ela não será *lembrada* quando a função `&get_instance_ref()` for chamada uma segunda vez.

Variáveis variáveis

Às vezes, é conveniente possuir nome variáveis para variáveis. Isto é, o nome de uma variável que pode ser definido e utilizado dinamicamente. Uma variável normal é definida com uma instrução como:

```php
<?php
$a = 'hello';
var_dump($a);
```

Uma variável variável obtém o valor de uma variável e a trata como o nome de uma variável. No exemplo acima, *hello* pode ser utilizado como o nome de uma variável utilizando dois sinais de cifrão:

```php
<?php
$a = 'hello';
$$a = "world";
var_dump($hello);
```

Neste ponto, duas variáveis foram definidas e armazenadas na árvore de símbolos do PHP: *$a* que contém "hello" e *$hello* que contém "world". Assim, esta instrução:

```php
<?php
$a = 'hello';
$$a = 'world';
echo "$a {$$a}";
```

produz a mesma saída que:

```php
<?php
$a = 'hello';
$$a = 'world';
echo "$a $hello";
```

assim sendo, as duas produzem: hello world.

Para poder utilizar variáveis variáveis com arrays, é necessário resolver um problema de ambiguidade. Isso é, se for escrito *$$a[1]*, o interpretador precisa saber se a intenção é utilizar *$a[1]* como uma variável ou se a intenção é usar *$$a* como uma variável e `[1]` como o índice dessa variável. A sintaxe para resolver essa ambiguidade é *${$a[1]}* para o primeiro caso e *${$a}[1]* para o segundo.

Propriedades de classes também podem ser acessadas utilizando-se nomes de propriedades variáveis. O nome de propriedade variável será resolvido dentro do escopo em que a chamada foi feita. Por exemplo, se houver uma expressão como *$foo->$bar*, o escopo local será examinado procurando por *$bar* e seu valor será utilizado como o nome da propriedade *$foo*. Isso também funciona se *$bar* for um array.

Chaves também podem ser utilizadas para delimitar claramente os nomes de propriedade. São muito úteis ao acessar valores de uma propriedade que contenham um array, quando o nome da propriedade tem várias partes, ou quando o nome da propriedade contém caracteres que não são válidos (por exemplo, em saídas da função json_decode() ou SimpleXML).

Exemplo #1 Exemplo de propriedade variável

```php
<?php
class Foo {
    public $bar = 'Eu sou bar.';
    public $arr = ['Eu sou A.', 'Eu sou B.', 'Eu sou C.'];
    public $r   = 'Eu sou r.';
}

$foo = new Foo();
$bar = 'bar';
$baz = ['foo', 'bar', 'baz', 'quux'];
echo $foo->$bar . "\n";
echo $foo->{$baz[1]} . "\n";

$start = 'b';
$end   = 'ar';
echo $foo->{$start . $end} . "\n";

$arr = 'arr';
echo $foo->{$arr[1]} . "\n";
echo $foo->{$arr}[1] . "\n";
```

O exemplo acima produzirá:

```
Eu sou bar.
Eu sou bar.
Eu sou bar.
Eu sou r.
Eu sou B.
```

> **Aviso**
>
> Observe que variáveis variáveis não podem ser utilizadas nos arrays superglobais dentro de funções ou métodos de classe. A variável `$this` também é uma variável especial que não pode ser referenciada dinamicamente.

Variáveis de fontes externas

Formulários HTML (GET e POST)

Quando um formulário é submetido para um script PHP, a informação deste formulário estará automaticamente disponível para o script. Há algumas maneiras de acessar estas informações, por exemplo:

Exemplo #1 Um formulário HTML simples

```
<form action="foo.php" method="post">
    Nome:  <input type="text" name="username" /><br />
    Email: <input type="text" name="email" /><br />
    <input type="submit" name="submit" value="Me aperte!" />
</form>
```

Existem apenas duas formas de acessar dados de formulários HTML. Os métodos disponíveis atualmente são listadas a seguir:

Exemplo #2 Acessando dados de um formulário HTML via POST

```
<?php
echo $_POST['username'];
echo $_REQUEST['username'];
```

Utilizar um formulário GET é similar, exceto que será utilizada a variável predefinida GET em seu lugar. GET também se aplica à variável `QUERY_STRING` (a informação depois do '?' numa URL). Então, por exemplo, `http://www.example.com/test.php?id=3` contém os dados GET que estarão acessíveis com *$_GET['id']*. Veja também *$_REQUEST*.

> **Nota** :
>
> Pontos e espaços em nomes de variáveis são convertidos para sublinhados. Por exemplo `<input name="a.b" />` se tornará `$_REQUEST["a_b"]`.

O PHP também entende arrays no contexto de variáveis de formulário (veja essa FAQ). Pode-se, por exemplo, agrupar variáveis relacionadas, ou utilizar este recurso para obter valores de uma entrada de seleção múltipla. No exemplo abaixo, o formulário é enviado para si mesmo, e depois de submetido, os dados são mostrados:

Exemplo #3 Variáveis de formulários mais complexos

```
<?php
if ($_POST) {
    echo '<pre>';
    echo htmlspecialchars(print_r($_POST, true));
    echo '</pre>';
}
?>
<form action="" method="post">
    Nome:  <input type="text" name="personal[name]" /><br />
    Email: <input type="text" name="personal[email]" /><br />
    Cerveja: <br />
    <select multiple name="beer[]">
        <option value="heineken">Heineken</option>
        <option value="stella">Stella Artois</option>
        <option value="av42">Double Rye Ipa</option>
```

```
    </select><br />
    <input type="submit" value="Enviar dados!" />
</form>
```

> **Nota** :
>
> Se um nome de variável externa se parece com sintaxe de array válida, as letras finais são silenciosamente ignoradas. Por exemplo, `<input name="foo[bar]baz">` se torna `$_REQUEST['foo']['bar']`.

Nomes de variáveis SUBMIT IMAGE

Ao submeter um formulário, é possível utilizar imagens em vez do botão de submissão padrão com uma tag do tipo:

```
<input type="image" src="image.gif" name="sub" />
```

Quando o usuário clicar em algum lugar da imagem, o formulário será transmitido para o servidor com duas variáveis adicionais, *sub_x* e *sub_y*. Elas contém as coordenadas do clique do usuário na imagem. Os mais experientes percebem que os nomes reais dessas variáveis enviadas ao navegador contêm um ponto ao invés de um sublinhado, mas o PHP converte o ponto para um sublinhado automaticamente.

Cookies HTTP

O PHP suporta transparentemente cookies HTTP como os definidos pela » RFC 6265. Cookies são um mecanismo o armazenamento de dados no navegador, rastreando e identificando o retorno de usuários. Você pode definir cookies com a função setcookie(). Cookies são parte do cabeçalho HTTP, então a função SetCookie precisa ser chamada antes de qualquer saída ser enviada ao navegador. Esta é a mesma restrição da função header(). Dados de cookies estão disponíveis nos arrays de dados de cookies apropriados, como *$_COOKIE* e também em *$_REQUEST*. Veja o manual de setcookie() para mais detalhes e exemplos.

> **Nota** :
>
> A partir dos PHP 7.2.34, 7.3.23 e 7.4.11, respectivamente, os *nomes* dos cookies recebidos não são decodificados (a partir do formato URL) por razões de segurança.

Se a intenção for atribuir vários valores a uma única variável do cookie, deve-se atribuí-la como um array:

```
<?php
setcookie("MyCookie[foo]", 'Testing 1', time()+3600);
setcookie("MyCookie[bar]", 'Testing 2', time()+3600);
```

Isso criará dois cookies separados embora *MyCookie* será agora um único array no script. Se a intenção for definir apenas um cookie com vários valores, pode-se utilizar serialize() ou explode() nos valores primeiro.

Note que um cookie substituirá um anterior com o mesmo nome no navegador, a menos que o caminho ou o domínio seja diferente. Portanto, para uma aplicação de carrinho de compras, pode-se manter um contador e repassá-lo:

Exemplo #4 Um exemplo de setcookie()

```php
<?php
if (isset($_COOKIE['count'])) {
    $count = $_COOKIE['count'] + 1;
} else {
    $count = 1;
}
setcookie('count', $count, time()+3600);
setcookie("Cart[$count]", $item, time()+3600);
```

Pontos em nomes de variáveis recebidas

Normalmente o PHP não altera o nome de variáveis quando elas são passadas para um script. Entretanto, deve ser notado que o ponto (ponto final) não é um caractere válido em nome de variável do PHP. Para ilustrar o motivo, veja o seguinte exemplo:

```php
<?php
$varname.ext;  /* nome de variável inválido */
```

Dessa forma, o interpretador entende isso como uma variável com nome de *$varname*, seguida do operador de concatenação de strings, seguida de uma string crua (uma string não delimitada que não bate com nenhuma palavra-chave ou reservada) 'ext'. Obviamente, isso não tem os resultados pretendidos.

Nessa situação, é importante notar que o PHP substituirá automaticamente qualquer ponto nos nomes de variáveis recebidas por sublinhados.

Determinando o tipo das variáveis

Como o PHP determina os tipos de variáveis e faz conversões (geralmente) quando necessário, nem sempre é óbvio o tipo de uma variável em cada momento. O PHP inclui várias funções que permitem determinar qual o tipo de uma variável, por exemplo: gettype(), is_array(), is_float(), is_int(), is_object(),e is_string(). Veja também o capítulo sobre Tipos.

Pelo fato do HTTP ser um protocolo de texto, a maioria, se não todo o conteúdo presente nos arrays superglobais como *$_POST* e *$_GET* permanecerão como strings. O PHP não tentará converter valores para um tipo específico. No exemplo abaixo, *$_GET["var1"]* conterá a string "null" e *$_GET["var2"]* conterá a string "123".

```
/index.php?var1=null&var2=123
```

Registro de Alterações

Versão	Descrição
7.2.34, 7.3.23, 7.4.11	Os *nomes* dos cookies recebidos não são mais decodificados (a partir do formato URL) por razões de segurança.

Constantes

Índice

Uma constante é um identificador (nome) para um valor simples. Como o nome sugere, esse valor não pode mudar durante a execução do script (exceto para as constantes mágicas, que na verdade não são constantes).

As constantes diferenciam maiúsculas de minúsculas. Por convenção, identificadores de constantes são sempre em maiúsculas.

> **Nota** :
>
> Antes do PHP 8.0.0, as constantes definidas usando a função define() podem não diferenciar maiúsculas de minúsculas.

O nome de uma constante segue as mesmas regras de qualquer rótulo no PHP. Um nome de constante válido começa com uma letra ou sublinhado, seguido por qualquer número de letras, números ou sublinhados. Como uma expressão regular, seria expresso assim: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`

É possível usar define() para definir constantes com nomes reservados ou mesmo inválidos, cujo valor só pode ser recuperado com a função constant(). No entanto, isso não é recomendado.

> **Dica**
>
> Veja também o Guia de nomenclatura em espaço de usuário.

Exemplo #1 Nomes de constantes válidos e inválidos

```php
<?php

// Nomes de constantes válidos
define("FOO",     "alguma coisa");
define("FOO2",    "alguma outra coisa");
define("FOO_BAR", "alguma coisa mais");

// Nomes de constantes inválidos
define("2FOO",    "alguma coisa");

// Isso é válido, mas deve ser evitado:
// O PHP pode um dia fornecer uma constante mágica
// que quebrará seu script
define("__FOO__", "alguma coisa");
```

> **Nota** :
>
> Para os exemplos aqui, uma letra é a-z, A-Z e os caracteres ASCII de 128 a 255 (0x80-0xff).

Como as superglobals, o escopo de uma constante é global. As constantes podem ser acessadas de qualquer lugar em um script, independentemente do escopo. Para mais informações sobre escopo, leia a seção do manual escopo de variáveis.

> **Nota** :
>
> A partir do PHP 7.1.0, as constantes de classe podem ser declaradas com visibilidade protected ou private, tornando-as disponíveis apenas no escopo hierárquico da classe em que foram definidas.

Sintaxe

As constantes podem ser definidas usando a palavra-chave `const` ou usando a função define(). Enquanto define() permite que uma constante seja definida usando uma expressão arbitrária, a palavra-chave `const` possui restrições, conforme descrito no próximo parágrafo. Uma vez definida, a constante nunca poderá ser modificada ou anulada.

Ao usar a palavra-chave `const`, somente expressões escalares (bool, int, float e string) e arrays constantes contendo apenas expressões escalares são aceitos. É possível definir constantes como um resource, mas isso deve ser evitado, pois pode causar resultados inesperados.

O valor de uma constante é acessado simplesmente informando seu nome. Ao contrário das variáveis, uma constante *não* é precedida por um `$`. Também é possível usar a função constant() para ler o valor de uma constante se o nome da constante for obtido dinamicamente. Use get_defined_constants() para obter uma lista de todas as constantes definidas.

> **Nota** :
>
> As constantes e variáveis (globais) estão em espaços de nomes diferentes. Isso implica que, por exemplo, `true` e *$TRUE* são geralmente diferentes.

Se uma constante indefinida for usada, um erro Error será lançado. Antes do PHP 8.0.0, constantes indefinidas eram interpretadas como uma simples string, ou seja, CONSTANTE vs. "CONSTANTE". Esse mecanismo alternativo foi descontinuado a partir do PHP 7.2.0 e um alerta de nível `E_WARNING` será lançado quando isso acontecer. Antes do PHP 7.2.0, um erro de nível E_NOTICE era lançado. Veja também por que $foo[bar] é errado (a não ser que `bar` seja uma constante). Isso não se aplica às constantes (totalmente) qualificadas, que sempre lançarão um Error se forem indefinidas.

> **Nota** :
>
> Para verificar se uma constante está definida, use a função defined().

Estas são as diferenças entre constantes e variáveis:

- Constantes não têm um cifrão ($) antes delas;
- Constantes podem ser definidas e acessadas em qualquer lugar sem considerar as regras de escopo de variáveis;
- Constantes não podem ser redefinidas ou anuladas depois de definidas; e
- Constantes só podem ter valores escalares ou arrays.

Exemplo #1 Definindo Constantes

```php
<?php
define("CONSTANTE", "Olá Mundo.");
echo CONSTANTE; // imprime "Olá Mundo."
echo Constante; // Lança um erro Error: Undefined constant "Constante"
                // Antes do PHP 8.0.0, imprime "Constante" e emite um aviso.
```

Exemplo #2 Definindo Constantes usando a palavra-chave `const`

```php
<?php
// Valor escalar simples
const CONSTANTE = 'Olá Mundo';

echo CONSTANTE . "\n";

// Expressão escalar
const OUTRA_CONSTANTE = CONSTANTE . '; Adeus Mundo';
echo OUTRA_CONSTANTE . "\n";

const ANIMAIS = array('cachorro', 'gato', 'pássaro');
echo ANIMAIS[1] . "\n"; // imprime "gato"

// Array constante
define('ANIMAIS_DEF', array(
    'cachorro',
    'gato',
    'pássaro'
));
echo ANIMAIS_DEF[1] . "\n"; // imprime "gato"
```

> **Nota** :
>
> Ao contrário das constantes definidas usando define(), as constantes definidas usando a palavra-chave `const` devem ser declaradas no escopo mais externo porque são definidas em tempo de compilação. Isto significa que constantes não podem ser declaradas dentro de funções, loops, declarações `if` ou blocos `try/catch`.

Veja também

- Constantes de Classe

Constantes Predefinidas

O PHP fornece um grande número de constantes predefinidas para qualquer script executado. Muitas dessas constantes, no entanto, são criadas por várias extensões e só estarão presentes quando essas extensões estiverem disponíveis, seja por carregamento dinâmico ou porque foram compiladas.

Constantes Mágicas

Existem algumas constantes mágicas que mudam dependendo de onde são usadas. Por exemplo, o valor de `__LINE__` depende da linha em que é usada no script. Todas essas constantes "mágicas" são resolvidas em tempo de compilação, ao contrário das constantes regulares, que são resolvidas em tempo de execução. Essas constantes especiais não diferenciam maiúsculas de minúsculas e são as seguintes:

Constantes Mágicas do PHP

Nome	Descrição
`__LINE__`	O número da linha atual do arquivo.

Nome	Descrição
`__FILE__`	O caminho completo e o nome do arquivo com links simbólicos resolvidos. Se usada dentro de um include, o nome do arquivo incluído será retornado.
`__DIR__`	O diretório do arquivo. Se usada dentro de um include, o diretório do arquivo incluído será retornado. Isso é equivalente a `dirname(__FILE__)`. O nome do diretório não possui barra no final, a menos que seja o diretório raiz.
`__FUNCTION__`	O nome da função ou `{closure}` para funções anônimas.
`__CLASS__`	O nome da classe. O nome da classe inclui o namespace em que foi declarada (por exemplo, `Foo\Bar`). Quando usada dentro de um método de trait, `__CLASS__` é o nome da classe em que o trait for usado.
`__TRAIT__`	O nome do trait. O nome do trait inclui o namespace em que foi declarado (por exemplo, `Foo\Bar`).
`__METHOD__`	O nome do método da classe.
`__PROPERTY__`	Válido somente dentro de um gancho de propriedade. É igual ao nome da propriedade.
`__NAMESPACE__`	O nome do namespace atual.
`ClassName::class`	O nome totalmente qualificado da classe.

Veja também

- ::class
- get_class()
- get_object_vars()
- file_exists()
- function_exists()

Expressões

Expressões são os blocos de construção mais importantes do PHP. No PHP, quase tudo o que você escreve são expressões. A maneira mais simples e ainda mais precisa de definir uma expressão é "tudo o que tem um valor".

As formas mais básicas de expressões são constantes e variáveis. Quando você digita `$a = 5`, você está atribuindo `5` dentro de *$a*. `5` obviamente tem o valor 5, ou em outras palavras `5` é uma expressão com o valor de 5 (nesse caso `5` é uma constante inteira).

Depois desta atribuição, você pode esperar que o valor de *$a* seja 5 também, assim se você escrever `$b = $a`, você pode esperar que ele se comporte da mesma forma que se você escrevesse `$b = 5`. Em outras palavras, *$a* é uma expressão com valor 5 também. Se tudo funcionou bem isto é exatamente o que aconteceu.

Exemplos ligeiramente mais complexos para expressões são as funções. Por exemplo, considere a seguinte função:

```
<?php
function foo ()
{
    return 5;
```

```
}
?>
```

Assumindo que você está familiarizado com o conceito de funções (se não estiver, dê uma olhada no capítulo sobre funções), você pode assumir que digitar `$c = foo()` é essencialmente a mesma coisa que escrever `$c = 5`, e você está certo. Funções são expressões com o valor igual ao seu valor de retorno. Como `foo()` retorna 5, o valor da expressão '`foo()`' é 5. Geralmente, as funções não retornam apenas um valor estático, mas computam algo.

Claro, valores em PHP não tem que ser inteiros, e muito comumente eles não são. O PHP suporta quatro tipos de valores escalares: valores int (inteiros), valores de ponto flutuante (float), valores string (caracteres) e valores bool (booleano) (valores escalares são valores que você não pode partir em peças menores, diferentemente de arrays, por exemplo). O PHP também suporta dois tipos compostos (não escalar): arrays e objetos. Cada um desses valores podem ser definidos em uma variável ou retornados de uma função.

O PHP leva as expressões muito além, da mesma maneira que muitas outras linguagens fazem. O PHP é uma linguagem orientada a expressões, no sentido de que quase tudo são expressões. Considere o exemplo com o qual já lidamos, `$a = 5`. É fácil ver que há dois valores envolvidos aqui, o valor da constante inteira `5`, e o valor de *$a* que está sendo atualizado para 5 também. Mas a verdade é que há um valor adicional envolvido aqui, e que é o próprio valor da atribuição. A própria atribuição é avaliada com o valor atribuído, neste caso 5. Na prática, significa que `$a = 5`, independente do que faça, é uma expressão com o valor 5. Portanto, escrever algo como `$b = ($a = 5)` é como escrever `$a = 5; $b = 5;` (um ponto-e-vírgula marca o fim do comando). Como as atribuições são analisadas da direita para a esquerda, você também pode escrever `$b = $a = 5`.

Outro bom exemplo de orientação de expressão é o pré e o pós-incremento e decremento. Usuários de PHP 2 e muitas outras linguagens podem estar familiarizados com a notação de `variable++` e `variable--`. Estes são operadores de incremento e decremento. Em PHP, como em C, há dois tipos de incremento - pré-incremento e pós-incremento. Tanto o pré-incremento quanto o pós-incremento, essencialmente, incrementam as variáveis, e o efeito sobre a variável é idêntico. A diferença é com o valor da expressão de incremento. O pré-incremento, que é escrito `++$variable`', é avaliado como o valor de incremento (o PHP incrementa a variável antes de ler seu valor, por isso o nome pré-incremento). O pós-incremento, que é escrito `$variable++` é avaliado como o valor original de *$variable*, antes de ser incrementada (o PHP incrementa a variável depois de ler seu valor, por isso o nome 'pós-incremento').

Um tipo muito comum de expressões são expressões de comparação. Estas expressões avaliam como **`false`** ou **`true`**. O PHP suporta > (maior que), >= (maior ou igual a), == (igual), != (diferente), < (menor que) e <= (menor ou igual a). A linguagem também suporta um conjunto de operador de equivalência estrita: `===` (igual a e do mesmo tipo) e !== (diferente de ou não do mesmo tipo). Estas expressões são mais comumente usadas dentro de execução condicional como comandos `if`.

O último exemplo de expressões com que nós vamos lidar aqui são as expressões combinadas operador-atribuição. Você já sabe que se você quer incrementar *$a* de 1, você só precisa escrever `$a++` ou `++$a`. Mas e se você quiser somar mais que um a ele, por exemplo 3? Você poderia escrever `$a++` várias vezes, mas esta obviamente não é uma forma muito eficiente ou confortável. Uma prática muito mais comum é escrever `$a = $a + 3`. `$a + 3` é avaliada como o valor de *$a* mais 3, e é atribuído de volta a *$a*, que resulta em incrementar *$a* em 3. Em PHP, como em várias outras linguagens como o C, você pode escrever isto de uma forma mais curta, que com o tempo se torna mais limpa e rápida de se entender. Somar 3 ao valor corrente de *$a* pode ser escrito `$a += 3`. Isto significa exatamente "pegue o valor de *$a*, some 3 a ele, e atribua-o de volta a *$a*." Além de ser mais curto e mais limpo, isto também resulta em execução mais rápida. O valor de `$a += 3`, como o valor de uma atribuição regular, é o valor atribuído. Note que NÃO é 3, mas o valor combinado de *$a* mais 3 (este é o valor que é atribuído a *$a*). Qualquer operador de dois parâmetros pode ser usado neste modo

operador-atribuição, por exemplo `$a -= 5` (subtrai 5 do valor de *$a*), `$b *= 7` (multiplica o valor de *$b* por 7), etc.

Há mais uma expressão que podem parecer estranha se você não a viu em outras linguagens, o operador condicional ternário:

```
<?php
$primeira ? $segunda : $terceira
?>
```

Se o valor da primeira sub-expressão é verdadeiro (**true**, não-zero), então a segunda sub-expressão é avaliada, e este é o resultado da expressão condicional. Caso contrário, a terceira sub-expressão é avaliada e este é o valor.

O seguinte exemplo deve ajudá-lo a entender um pouco melhor pré e pós-incremento e expressões em geral:

```
<?php
function double($i)
{
    return $i * 2;
}

$b = $a = 5;        /* atribui o valor cinco às variáveis $a e $b */
$c = $a++;          /* pós-incremento, atribui o valor original de $a
                       (5) para $c */
print "a = {$a}; b = {$b}; c = {$c}" . PHP_EOL;

$e = $d = ++$b;     /* pré-incremento, atribui o valor incrementado de
                       $b (6) a $d e $e */
print "b = {$b}; d = {$d}; e = {$e}" . PHP_EOL;

/* neste ponto, tanto $d quanto $e são iguais a 6 */

$f = double($d++);  /* atribui o dobro do valor de $d antes
                       do incremento, 2*6 = 12 a $f */
print "d = {$d}; f = {$f}" . PHP_EOL;

$g = double(++$e);  /* atribui o dobro do valor de $e depois
                       do incremento, 2*7 = 14 a $g */
print "e = {$e}; g = {$g}" . PHP_EOL;

$h = $g += 10;      /* primeiro, $g é incrementado de 10 e termina com o
                       valor 24. o valor da atribuição (24) é
                       então atribuído a $h, e $h termina com o valor
                       24 também. */
print "g = {$g}; h = {$h}" . PHP_EOL;
?>
```

Algumas expressões podem ser consideradas instruções. Neste caso, uma instrução na forma '`expr ;`' ou seja, uma expressão seguida de um ponto e vírgula. Em `$b = $a = 5;`, `$a = 5` é uma expressão válida, mas não é um comando por si só. `$b = $a = 5;` porém é um comando válido.

Uma última coisa que vale mencionar é o valor-verdade de expressões. Em muitos eventos, principalmente em instruções condicionais e loops, você não está interessado no valor específico da expressão, mas somente se ela significa **true** ou **false** (o PHP não tem um tipo booleano dedicado). As constantes **true** e **false**

(insensitivas ao caso) são seus dois valores booleanos possíveis. Às vezes uma expressão é automaticamente convertida para um booleano. Veja a seção sobre type-casting para detalhes de como isso é feito.

O PHP fornece uma implementação completa e poderosa de expressões, e a completa documentação dela vai além do escopo deste manual. Os exemplos acima devem dar a você uma boa ideia sobre o que são as expressões e como você pode construir expressões úteis. Através do restante do manual nós escreveremos *expr* (ou expressão) para indicar qualquer expressão PHP válida.

Operadores

Índice

Um operador é algo que recebe um ou mais valores (ou expressões, no jargão de programação) e que devolve outro valor (e por isso os operadores são expressões também).

Operadores podem ser agrupados segundo o número de valores que aceitam. Operadores unários recebem um único valor, por exemplo `!` (o operador lógico de negação) ou `++` (o operador de incremento). Operadores binários aceitam dois valores, como os operadores aritméticos `+` (soma) e `-` (subtração), além da maioria dos operadores PHP dessa categoria. Finalmente há um único operador ternário, `? :`, que aceita três valores; normalmente conhecido simplesmente como "o operador ternário" (embora um nome melhor fosse operador condicional).

A lista completa dos operadores no PHP está na seção sobre Precedência de Operadores. Essa seção também explica precedência e combinações, que governam exatamente como expressões contendo vários operadores são avaliados.

Existe uma extensão PECL que permite a sobrecarga de alguns operadores para objetos. Para mais informações, consulte a seção sobre Sobrecarga de Operadores para Objetos.

Precedência de Operadores

A precedência de um operador especifica quem tem mais prioridade quando há duas delas juntas. Por exemplo, na expressão `1 + 5 * 3`, a resposta é `16` e não `18` porque o operador de multiplicação ("*") tem prioridade de precedência que o operador de adição ("+"). Parênteses podem ser utilizados para forçar a precedência, se necessário. Assim, `(1 + 5) * 3` é avaliado como `18`.

Quando operadores têm precedência igual a associatividade decide como os operadores são agrupados. Por exemplo "-" é associado à esquerda, de forma que `1 - 2 - 3` é agrupado como `(1 - 2) - 3` e resulta em `-4`. "=" por outro lado associa para a direita, de forma que `$a = $b = $c` é agrupado como `$a = ($b = $c)`.

Operadores de igual precedência e associatividade não podem ser utilizados uns próximos aos outros. Por exemplo `1 < 2 > 1` é ilegal no PHP. A expressão `1 <= 1 == 1` por outro lado é válida, porque o operador `==` tem menor precedência que o operador `<=`.

A associatividade somente faz sentido para operadores binários e ternários. Operadores unários são precedentes ou seguintes, de forma que essa outra noção não se aplica. Por exemplo `!!$a` somente pode ser agrupado como `!(!$a)`.

O uso de parênteses, embora não estritamente necessário, pode melhorar a leitura do código ao deixar o agrupamento explícito em vez de depender da associatividade e precedências implícitos.

A tabela seguinte mostra a precedência dos operadores, com a maior precedência no começo. Operadores com a mesma precedência estão na mesma linha, nesses casos a associatividade deles decidirá qual ordem eles serão avaliados.

Precedência dos operadores

Associação	Operadores	Informação Adicional
(não aplicável)	`clone new`	clone e new
direita	`**`	aritmético
(não aplicável)	`+ - ++ -- ~ (int) (float) (string) (array) (object) (bool) @`	aritméticos (unário + e -), incremento/decremento, binário, conversão de tipo e controle de erro
esquerda	`instanceof`	tipo
(não aplicável)	`!`	lógico
esquerda	`* / %`	aritméticos
esquerda	`+ - .`	aritméticos (binário + e -), array e string (`.` anteriormente ao PHP 8.0.0)
esquerda	`<< >>`	binários
esquerda	`.`	string (a partir do PHP 8.0.0)
esquerda	`|>`	pipe
não associativo	`< <= > >=`	comparação
não associativo	`== != === !== <> <=>`	comparação
esquerda	`&`	binário e referências
esquerda	`^`	binário
esquerda	`|`	binário
esquerda	`&&`	lógico
esquerda	`||`	lógico
direita	`??`	coalescência nula
não associativo	`? :`	ternário (esquerda anteriormente ao PHP 8.0.0)
direita	`= += -= *= **= /= .= %= &= |= ^= <<= >>= ??=`	atribuição
(não aplicável)	`yield from`	yield from
(não aplicável)	`yield`	yield
(não aplicável)	`print`	print
esquerda	`and`	lógicos

Associação	Operadores	Informação Adicional
esquerda	`xor`	lógicos
esquerda	`or`	lógicos
(n/a)	`throw`	throw

> **Nota** :
>
> Os operadores unários que compartilham a linha com os operadores de conversão (`~`, `@`, e os unários `+`/`-`) se ligam com mais força que `instanceof`, enquanto que `!` se liga com menos força. Consequentemente, `(int) $x instanceof Foo` é agrupado como `((int) $x) instanceof Foo`, enquanto que `!$x instanceof Foo` é agrupado como `!($x instanceof Foo)`.

Exemplo #1 Associação

```php
<?php
$a = 3 * 3 % 5; // (3 * 3) % 5 = 4
var_dump($a);

$a = 1;
$b = 2;
$a = $b += 3; // $a = ($b += 3) -> $a = 5, $b = 5
var_dump($a, $b);
?>
```

O operador ternário requer especificamente o uso de parênteses para remover a ambiguidade de precedência.

Exemplo #2 Precedência explícita

```php
<?php
$a = true ? 0 : (true ? 1 : 2);
var_dump($a);

// isto não é permitido desde o PHP 8
// $a = true ? 0 : true ? 1 : 2;
?>
```

A precedência e a associatividade apenas determinam como as expressões são agrupadas, mas não especificam a ordem de avaliação. O PHP (geralmente) não especifica em que ordem as expressões são avaliadas, então códigos que assumem ordens específicas de avaliação devem ser evitados porque o comportamento pode ser alterado entre versões do PHP ou dependendo do código em volta.

Exemplo #3 Ordem de avaliação não definida

```php
<?php
$a = 1;
echo $a + $a++; // pode exibir 2 ou 3

$i = 1;
$array[$i] = $i++; // pode definir o índice 1 ou 2
?>
```

Exemplo #4 Precedência de +, - e .

```php
<?php
$x = 4;
// esta linha pode resultar em uma saída inesperada:
echo "x menos um é igual a " . $x-1 . ", ou assim eu espero\n";

// a precedência desejada pode ser aplicada utilizando parênteses:
echo "x menos um é igual a " . ($x-1) . ", ou assim eu espero\n";

// isto não é permitido e lança um TypeError:
echo (("x menos um é igual a " . $x) - 1) . ", ou assim eu espero\n";
?>
```

O exemplo acima produzirá:

```
-1, ou assim eu espero
-1, ou assim eu espero
Fatal error: Uncaught TypeError: Unsupported operand types: string - int
```

Exemplo #5 Antes do PHP 8, +, - e . tinham a mesma precedência

```php
<?php
$x = 4;
// this line might result in unexpected output:
echo "x menos um é igual a " . $x-1 . ", ou assim eu espero\n";

// porque é avaliada como esta linha (antes do PHP  8.0.0):
echo (("x menos um é igual a " . $x) - 1) . ", ou assim eu espero\n";

// a precedência desejada pode ser aplicada utilizando parênteses:
echo "x menos um é igual a " . ($x-1) . ", ou assim eu espero\n";
?>
```

O exemplo acima produzirá:

```
-1, ou assim eu espero
-1, ou assim eu espero
x menos um é igual a 3, ou assim eu espero
```

> **Nota** :
>
> Embora o = tenha uma precedência menor que a maioria dos operadores, o PHP ainda permite expressões similares a `if (!$a = foo())`, onde o valor retornado de `foo()` é colocado em *$a*.

> Isso é possível porque o lado esquerdo de uma atribuição deve ser uma variável, portanto a atribuição é agrupada com essa variável em vez de com o operador de prefixo de precedência superior circundante. O mesmo se aplica aos outros operadores de prefixo que recebem um operando de expressão, tal como `clone`, os operadores de conversão, `@` e ~: por exemplo, `clone $a = $b` é agrupado como `clone ($a = $b)`, e não `(clone $a) = $b`.

Registro de Alterações

Versão	Descrição
8.0.0	Concatenação de strings (`.`) tem uma precedência menor que adição/subtração (`+` e `-`) e shift de bits à esquerda/direita (`<<` e `>>`); anteriormente ele tinha a mesma precedência que `+` e `-` uma precedência maior que `<<` e `>>`.
8.0.0	O operador ternário (`? :`) agora é não associativo; anteriormente associava a esquerda.
7.4.0	Depender da precedência da concatenação de strings (`.`) em relação à adição/subtração (`+` ou `-`) ou shift de bits a esquerda/direta (`<<` ou `>>`), ou seja, os usar numa mesma expressão, foi descontinuado.
7.4.0	Depender da associatividade à esquerda do operador ternário (`? :`), ou seja, o utilizar em várias expressões sem parênteses, foi descontinuado.

Aritméticos

Lembra-se da aritmética básica da escola? Estes operadores funcionam exatamente como aqueles.

Operadores Aritméticos

Exemplo	Nome	Resultado
`+$a`	Identidade	Conversão de *$a* para int ou float conforme apropriado.
`-$a`	Negativação	Oposto de *$a*.
`$a + $b`	Adição	Soma de *$a* e *$b*.
`$a - $b`	Subtração	Diferença entre *$a* e *$b*.
`$a * $b`	Multiplicação	Produto de *$a* e *$b*.
`$a / $b`	Divisão	Quociente de *$a* e *$b*.
`$a % $b`	Resto	Resto de *$a* dividido por *$b*.
`$a ** $b`	Exponenciação	Resultado de *$a* elevado à potência de *$b*.

O operador de divisão `/` retorna um valor float, a não ser que os dois operandos sejam int (ou strings numéricas que são convertidas para int) e o numerador seja um múltiplo do divisor, neste caso será retornado um valor inteiro. Para divisão de inteiros, consulte intdiv().

Operandos da operação de resto de divisão são convertidos para int antes do processamento. Para restos de divisão de números em ponto flutuante, veja fmod().

O resultado do operador de resto `%` tem o mesmo sinal do dividendo — ou seja, o resultado de `$a % $b` terá o mesmo sinal de *$a*. Exemplos:

Exemplo #1 O Operador de Resto

```php
<?php
var_dump(5 % 3);
var_dump(5 % -3);
var_dump(-5 % 3);
var_dump(-5 % -3);
?>
```

O exemplo acima produzirá:

```
int(2)
int(2)
int(-2)
int(-2)
```

Veja também

- Funções matemáticas

Incremento e Decremento

O PHP suporta operadores de pré e pós-incremento operadores de decremento. Esses operadores unários permitem aumentar o diminuir o valor por um.

Operadores de Incremento/Decremento

Exemplo	Nome	Efeito
++$a	Pré-incremento	Incrementa *$a* em um, e então retorna *$a*.
$a++	Pós-incremento	Retorna *$a*, e então incrementa *$a* em um.
--$a	Pré-decremento	Diminuiu *$a* em um, e então retorna *$a*.
$a--	Pós-decremento	Retorna *$a*, e então diminui *$a* em um.

Exemplo #1 Exemplos de incremento/decremento

```php
<?php
echo 'Pós-incremento:', PHP_EOL;
$a = 5;
var_dump($a++);
var_dump($a);

echo 'Pré-incremento:', PHP_EOL;
$a = 5;
var_dump(++$a);
var_dump($a);

echo 'Pós-incremento:', PHP_EOL;
$a = 5;
var_dump($a--);
var_dump($a);
```

```php
echo 'Pré-decremento:', PHP_EOL;
$a = 5;
var_dump(--$a);
var_dump($a);
?>
```

O exemplo acima produzirá:

```
Pós-incremento:
int(5)
int(6)
Pré-incremento:
int(6)
int(6)
Pós-incremento:
int(5)
int(4)
Pós-decremento:
int(4)
int(4)
```

> **Aviso**
>
> Os operadores de incremento e decremento não tem efeitos em valores do tipo bool. Um **E_WARNING** é emitido desde o PHP 8.3.0, porque isso causará uma conversão implícita para um tipo int no futuro.
>
> O operador de decremento não tem efeito em valores do tipo null. Um **E_WARNING** é emitido desde o PHP 8.3.0, porque isso causará uma conversão implícita para um tipo int no futuro.
>
> O operador de decremento não tem efeito em strings não numéricas. Um **E_WARNING** é emitido desde o PHP 8.3.0, porque isso lançará um TypeError no futuro.

> **Nota** :
>
> Objetos internos que suportam a sobrecarga de adição ou subtração também podem ser incrementados ou decrementados. Um exemplo de objeto assim é o GMP.

Incremento de strings no estilo PERL

> **Aviso**
>
> Esse recurso está desencorajado desde o PHP 8.3.0. A função str_increment() deve ser utilizada.

É possível incremente strings não numéricas no PHP. A string precisa ser um texto ASCII. O efeito é incrementar a última letra até a próxima, e quando é alcançada a letra z o incremento é carregado para a letra à esquerda. Por exemplo, `$a = 'Z'; $a++;` transforma *$a* em `'AA'`.

Exemplo #2 Exemplo de incremento de string estilo PERL

```php
<?php
echo '== String alfabética ==' . PHP_EOL;
$s = 'W';
for ($n=0; $n<6; $n++) {
    echo ++$s . PHP_EOL;
}
// String alfanuméricas tem outro comportamento
echo '== Strings alfanuméricas ==' . PHP_EOL;
$d = 'A8';
for ($n=0; $n<6; $n++) {
    echo ++$d . PHP_EOL;
}
$d = 'A08';
for ($n=0; $n<6; $n++) {
    echo ++$d . PHP_EOL;
}
?>
```

O exemplo acima produzirá:

```
== String alfabética ==
X
Y
Z
AA
AB
AC
== Strings alfanuméricas ==
A9
B0
B1
B2
B3
B4
A09
A10
A11
A12
A13
A14
```

> **Aviso**
>
> Se uma string alfanumérica pode ser interpretada como uma string numérica ela será convertida para um int ou float. Isso pode ser um problema para strings que parecem como números de ponto flutuante escritos em notação exponencial. A função str_increment() não sofre desse problema de conversão implícita.
>
> **Exemplo #3 Strings alfanuméricas convertida para float**

```php
<?php
$s = "5d9";
var_dump(++$s);
var_dump(++$s);
?>
```

O exemplo acima produzirá:

```
string(3) "5e0"
float(6)
```

Isto ocorre porque o valor `"5e0"` é interpretado como float e é convertido para o valor de `5.0` antes de ser incrementado.

Atribuição

O operador básico de atribuição é "=". A primeira inclinação seria pensar nisto como "é igual". Não é verdade. Isto quer dizer que o operando da esquerda recebe o valor da expressão da direita (ou seja, "é definido para").

O valor de uma expressão de atribuição é o valor atribuído. Ou seja, o valor de "`$a = 3`" é 3. Isto permite alguns truques:

Exemplo #1 Atribuições aninhadas

```php
<?php
$a = ($b = 4) + 5; // $a é igual a 9 agora e $b foi definido como 4.
var_dump($a);
?>
```

Além do operador básico de atribuição, há "operadores combinados" para todos os operadores aritméticos, de array e string que permitem a você pegar um valor de uma expressão e então usar seu próprio valor para o resultado daquela expressão. Por exemplo:

Exemplo #2 Atribuições combinadas

```php
<?php
$a = 3;
$a += 5; // define $a para 8, como se disséssemos: $a = $a + 5;
$b = "Bom ";
$b .= "Dia!"; // define $b para "Bom Dia!", como em $b = $b . "Dia!";

var_dump($a, $b);
?>
```

Note que a atribuição copia a variável original para a nova (atribuição por valor), assim a mudança de uma não afeta a outra. Isto pode ser relevante se você precisa copiar algo como um grande array dentro de um loop apertado.

Uma exceção ao comportamento geral de atribuição ocorre com objects, que são atribuídos por referência. Objetos podem ser explicitamente copiados através da instrução clone.

Atribuição por referência

Atribuição por referência também é possível, utilizando-se a sintaxe "$var = &$othervar;". A atribuição por referência significa que ambas as variáveis apontam para o mesmo dado, e nada é copiado.

Exemplo #3 Atribuição por referência

```php
<?php
$a = 3;
$b = &$a; // $b é uma referência de $a

print "$a\n"; // imprime 3
print "$b\n"; // imprime 3

$a = 4; // modificamos $a

print "$a\n"; // imprime 4
print "$b\n"; // imprime 4 também, pois $b é uma referência de $a, que foi
              // modificada
?>
```

O operador new retorna uma referência automaticamente, de forma que a atribuição de um resultado de new por referência é um erro.

Exemplo #4 Operador new por referência

```php
<?php
class C {}

$o = &new C;
?>
```

O exemplo acima produzirá:

```
Parse error: syntax error, unexpected token ";", expecting "("
```

Mais informações sobre referências e seus potenciais usos podem ser encontrados na seção Explicando Referências do manual.

Arithmetic Assignment Operators

Exemplo	Equivalente	Operação
$a += $b	$a = $a + $b	Adição
$a -= $b	$a = $a - $b	Subtração
$a *= $b	$a = $a * $b	Multiplicação

Exemplo	Equivalente	Operação
$a /= $b	$a = $a / $b	Divisão
$a %= $b	$a = $a % $b	Módulo
$a **= $b	$a = $a ** $b	Exponentiation

Operadores de bits

Exemplo	Equivalente	Operação
$a &= $b	$a = $a & $b	Bitwise E
$a |= $b	$a = $a | $b	Bitwise Ou
$a ^= $b	$a = $a ^ $b	Bitwise Xor
$a <<= $b	$a = $a << $b	Desloca para esquerda
$a >>= $b	$a = $a >> $b	Desloca para direita

Outras operações de assimilação

Exemplo	Equivalente	Operação
$a .= $b	$a = $a . $b	Concatenação de strings
$a ??= $b	$a = $a ?? $b	Aglutinação de nulls (null coalesce)

Veja também

- operadores aritméticos
- operadores em bits
- operador de aglutinação de null

Binários

Operadores binários permitem a avaliação e modificação de bits específicos em um tipo inteiro.

Operadores binários

Exemplo	Nome	Resultado
`$a & $b`	E (AND)	Os bits que estão ativos tanto em *$a* quanto em *$b* são ativados.
`$a | $b`	OU (OR inclusivo)	Os bits que estão ativos em *$a* ou em *$b* são ativados.
`$a ^ $b`	XOR (OR exclusivo)	Os bits que estão ativos em *$a* ou em *$b*, mas não em ambos, são ativados.
`~ $a`	NÃO (NOT)	Os bits que estão ativos em *$a* não são ativados, e vice-versa.
`$a << $b`	Deslocamento à esquerda	Desloca os bits de *$a* em *$b* passos para a esquerda (cada passo significa "multiplica por dois")
`$a >> $b`	Deslocamento à direita	Desloca os bits de *$a* em *$b* passos para a direita (cada passo significa "divide por dois")

O deslocamento de bits no PHP é aritmético. Bits deslocados no lado final são descartados. Deslocamentos à esquerda tem zeros inseridos à direita e o bit de sinal é perdido, e assim o sinal do operando não é

preservado. Deslocamentos à direita têm cópias do bit de sinal à esquerda, e assim o sinal do operando é preservado.

Utilize parênteses para garantir a precedência desejada. Por exemplo `$a & $b == true` avalia primeiro a equivalência e depois a operação de bits, enquanto que `($a & $b) == true` avalia primeiro a operação de bits e só depois a equivalência.

Se ambos os operandos de `&`, `|` e `^` forem strings então a operação será realizada nos valores ASCII dos caracteres das strings, e o resultado final será uma string. Em todos os outros casos ambos os operandos serão convertidos para inteiros e o resultado será um inteiro.

Se o operando de `~` for uma string então a operação será realizada nos valores ASCII dos caracteres da string, e o resultado será uma string. Nos demais casos o operando e o resultado serão tratados como inteiros.

Ambos os operandos e o resultado para `<<` e `>>` sempre são tratados como inteiros.

A configuração error_reporting ini utiliza valores posicionados em bits, fornecendo um exemplo real de ligar ou desligar bits. Para mostrar todos os erros mas não os alertas coloque no php.ini: **`E_ALL & ~E_NOTICE`**

```
    Isto funciona pois começa com E_ALL:
    00000000000000000111011111111111
    Depois pegando o valor de E_NOTICE...
    00000000000000000000000000001000
    ... que invertido através de ~:
    11111111111111111111111111110111
    ... é finalmente mesclado via E (&) e assim o valor final
    se torna:
    00000000000000000111011111110111
```

Outra forma de conseguir esse efeito seria através do XOR (^) para encontrar os bits que estão ativos em apenas um ou no outro: **`E_ALL ^ E_NOTICE`**

O error_reporting pode também ser utilizado para demonstrar a ativação de bits. Para mostrar apenas erros e erros recuperáveis: **`E_ERROR | E_RECOVERABLE_ERROR`**

```
    Este processo combina E_ERROR
    00000000000000000000000000000001
    e
    00000000000000000001000000000000
    utilizando OU (|)
    para pegar os bits ativos de ambos os valores:
    00000000000000000001000000000001
```

Exemplo #1 Operações bitwise AND, OR e XOR em inteiros

```php
<?php
/*
 * Ignore essa primeira parte,
 * que é apenas para deixar a impressão mais limpa.
 */

$format = '(%1$2d = %1$04b) = (%2$2d = %2$04b)'
        . ' %3$s (%4$2d = %4$04b)' . "\n";
```

```
echo <<<EOH
 ---------     ---------  -- ---------
 resultado     valor      op teste
 ---------     ---------  -- ---------
EOH;


/*
 * Agora os exemplos
 */

$values = array(0, 1, 2, 4, 8);
$test = 1 + 4;

echo "\n Operador de bit E \n";
foreach ($values as $value) {
    $result = $value & $test;
    printf($format, $result, $value, '&', $test);
}

echo "\n Operador de bit OU inclusivo \n";
foreach ($values as $value) {
    $result = $value | $test;
    printf($format, $result, $value, '|', $test);
}

echo "\n Operador de bit OU Exclusivo (XOR) \n";
foreach ($values as $value) {
    $result = $value ^ $test;
    printf($format, $result, $value, '^', $test);
}
?>
```

O exemplo acima produzirá:

```
 ---------     ---------  -- ---------
 resultado     valor      op teste
 ---------     ---------  -- ---------
 Operador de bit E
( 0 = 0000) = ( 0 = 0000) & ( 5 = 0101)
( 1 = 0001) = ( 1 = 0001) & ( 5 = 0101)
( 0 = 0000) = ( 2 = 0010) & ( 5 = 0101)
( 4 = 0100) = ( 4 = 0100) & ( 5 = 0101)
( 0 = 0000) = ( 8 = 1000) & ( 5 = 0101)

 Operador de bit OU inclusivo
( 5 = 0101) = ( 0 = 0000) | ( 5 = 0101)
( 5 = 0101) = ( 1 = 0001) | ( 5 = 0101)
( 7 = 0111) = ( 2 = 0010) | ( 5 = 0101)
( 5 = 0101) = ( 4 = 0100) | ( 5 = 0101)
(13 = 1101) = ( 8 = 1000) | ( 5 = 0101)

 Operador de bit OU exclusivo (XOR)
( 5 = 0101) = ( 0 = 0000) ^ ( 5 = 0101)
```

```
( 4 = 0100) = ( 1 = 0001) ^ ( 5 = 0101)
( 7 = 0111) = ( 2 = 0010) ^ ( 5 = 0101)
( 1 = 0001) = ( 4 = 0100) ^ ( 5 = 0101)
(13 = 1101) = ( 8 = 1000) ^ ( 5 = 0101)
```

Exemplo #2 Operações de bit XOR em strings

```php
<?php
echo 12 ^ 9, PHP_EOL; // Exibe '5'

echo "12" ^ "9", PHP_EOL; // Exibe caractere backspace (ASCII 8)
                          // ('1' (ASCII 49)) ^ ('9' (ASCII 57)) = #8

echo "hallo" ^ "hello";, PHP_EOL // Exibe os valores ASCII #0 #4 #0 #0 #0
                                 // 'a' ^ 'e' = #4

echo 2 ^ "3", PHP_EOL; // Exibe 1
                       // 2 ^ ((int) "3") == 1

echo "2" ^ 3, PHP_EOL; // Exibe 1
                       // ((int) "2") ^ 3 == 1
?>
```

Exemplo #3 Deslocamento de bits em inteiros

```php
<?php
/*
 * Aqui estão os exemplos.
 */

echo "\n--- MOVENDO BITS A DIREITA EM INTEIROS POSITIVOS ---\n";

$val = 4;
$places = 1;
$res = $val >> $places;
p($res, $val, '>>', $places, 'cópia do bit de sinal trocada para a esquerda');

$val = 4;
$places = 2;
$res = $val >> $places;
p($res, $val, '>>', $places);

$val = 4;
$places = 3;
$res = $val >> $places;
p($res, $val, '>>', $places, 'move os bits para fora da direita');

$val = 4;
$places = 4;
$res = $val >> $places;
p($res, $val, '>>', $places, 'mesmo resultado que acima; não se pode mover além do 0');


echo "\n--- MOVENDO BITS A DIREITA EM INTEIROS NEGATIVOS ---\n";
```

```php
$val = -4;
$places = 1;
$res = $val >> $places;
p($res, $val, '>>', $places, 'cópia do bit de sinal trocada para a esquerda');

$val = -4;
$places = 2;
$res = $val >> $places;
p($res, $val, '>>', $places, 'move os bits para fora da direita');

$val = -4;
$places = 3;
$res = $val >> $places;
p($res, $val, '>>', $places, 'mesmo resultado que acima; não se pode mover além do -1');


echo "\n--- MOVENDO BITS A ESQUERDA EM INTEIROS POSITIVOS ---\n";

$val = 4;
$places = 1;
$res = $val << $places;
p($res, $val, '<<', $places, 'preenche com zeros o lado direito');

$val = 4;
$places = (PHP_INT_SIZE * 8) - 4;
$res = $val << $places;
p($res, $val, '<<', $places);

$val = 4;
$places = (PHP_INT_SIZE * 8) - 3;
$res = $val << $places;
p($res, $val, '<<', $places, 'bit de sinal movido para fora');

$val = 4;
$places = (PHP_INT_SIZE * 8) - 2;
$res = $val << $places;
p($res, $val, '<<', $places, 'bits movidos para fora da esquerda');


echo "\n--- MOVENDO BITS A ESQUERDA EM INTEIROS NEGATIVOS ---\n";

$val = -4;
$places = 1;
$res = $val << $places;
p($res, $val, '<<', $places, 'preenche com zeros o lado direito');

$val = -4;
$places = (PHP_INT_SIZE * 8) - 3;
$res = $val << $places;
p($res, $val, '<<', $places);

$val = -4;
$places = (PHP_INT_SIZE * 8) - 2;
$res = $val << $places;
p($res, $val, '<<', $places, 'bits movidos para fora pela esquerda, incluindo o bit de sinal');
```

```php
/*
 * Ignore essa seção abaixo,
 * é apenas para formatar a saída e deixar mais clara a apresentação.
 */

function p($res, $val, $op, $places, $note = '') {
    $format = '%0' . (PHP_INT_SIZE * 8) . "b\n";

    printf("Expressão: %d = %d %s %d\n", $res, $val, $op, $places);

    echo " Decimal:\n";
    printf("  val=%d\n", $val);
    printf("  res=%d\n", $res);

    echo " Binário:\n";
    printf('  val=' . $format, $val);
    printf('  res=' . $format, $res);

    if ($note) {
        echo " NOTA: $note\n";
    }

    echo "\n\n";
}
?>
```

Saída do exemplo acima em máquinas de 32 bits:

```
--- MOVENDO BITS A DIREITA EM INTEIROS POSITIVOS ---
Expressão: 2 = 4 >> 1
 Decimal:
  val=4
  res=2
 Binário:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000010
 NOTA: cópia do bit de sinal trocada para a esquerda

Expressão: 1 = 4 >> 2
 Decimal:
  val=4
  res=1
 Binário:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000001

Expressão: 0 = 4 >> 3
 Decimal:
  val=4
  res=0
 Binário:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000000
 NOTA: move os bits para fora da direita
```

```
Expressão: 0 = 4 >> 4
 Decimal:
  val=4
  res=0
 Binário:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000000
 NOTA: mesmo resultado que acima; não se pode mover além do 0


--- MOVENDO BITS A DIREITA EM INTEIROS NEGATIVOS ---
Expressão: -2 = -4 >> 1
 Decimal:
  val=-4
  res=-2
 Binário:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111110
 NOTA: cópia do bit de sinal trocada para a esquerda

Expressão: -1 = -4 >> 2
 Decimal:
  val=-4
  res=-1
 Binário:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111111
 NOTA: move os bits para fora da direita

Expressão: -1 = -4 >> 3
 Decimal:
  val=-4
  res=-1
 Binário:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111111
 NOTA: mesmo resultado que acima; não se pode mover além do -1


--- MOVENDO BITS A ESQUERDA EM INTEIROS POSITIVOS ---
Expressão: 8 = 4 << 1
 Decimal:
  val=4
  res=8
 Binário:
  val=00000000000000000000000000000100
  res=00000000000000000000000000001000
 NOTA: preenche com zeros o lado direito

Expressão: 1073741824 = 4 << 28
 Decimal:
  val=4
  res=1073741824
 Binário:
  val=00000000000000000000000000000100
  res=01000000000000000000000000000000
```

```
Expressão: -2147483648 = 4 << 29
 Decimal:
  val=4
  res=-2147483648
 Binário:
  val=00000000000000000000000000000100
  res=10000000000000000000000000000000
 NOTA: bit de sinal movido para fora

Expressão: 0 = 4 << 30
 Decimal:
  val=4
  res=0
 Binário:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000000
 NOTA: bits movidos para fora da esquerda


--- MOVENDO BITS A ESQUERDA EM INTEIROS NEGATIVOS ---
Expressão: -8 = -4 << 1
 Decimal:
  val=-4
  res=-8
 Binário:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111000
 NOTA: preenche com zeros o lado direito

Expressão: -2147483648 = -4 << 29
 Decimal:
  val=-4
  res=-2147483648
 Binário:
  val=11111111111111111111111111111100
  res=10000000000000000000000000000000

Expressão: 0 = -4 << 30
 Decimal:
  val=-4
  res=0
 Binário:
  val=11111111111111111111111111111100
  res=00000000000000000000000000000000
 NOTA: bits movidos para fora pela esquerda, incluindo o bit de sinal
```

Saída do exemplo acima em máquinas de 64 bits:

```
--- MOVENDO BITS A DIREITA EM INTEIROS POSITIVOS ---
Expressão: 2 = 4 >> 1
 Decimal:
```

```
  val=4
  res=2
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000010
 NOTA: cópia do bit de sinal trocada para a esquerda

Expressão: 1 = 4 >> 2
 Decimal:
  val=4
  res=1
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000001

Expressão: 0 = 4 >> 3
 Decimal:
  val=4
  res=0
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000000
 NOTA: move os bits para fora da direita

Expressão: 0 = 4 >> 4
 Decimal:
  val=4
  res=0
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000000
 NOTA: mesmo resultado que acima; não se pode mover além do 0


--- MOVENDO BITS A DIREITA EM INTEIROS NEGATIVOS ---
Expressão: -2 = -4 >> 1
 Decimal:
  val=-4
  res=-2
 Binário:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111110
 NOTA: cópia do bit de sinal trocada para a esquerda

Expressão: -1 = -4 >> 2
 Decimal:
  val=-4
  res=-1
 Binário:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111111
 NOTA: move os bits para fora da direita

Expressão: -1 = -4 >> 3
 Decimal:
  val=-4
  res=-1
 Binário:
```

```
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111111
 NOTA: mesmo resultado que acima; não se pode mover além do -1


--- MOVENDO BITS A ESQUERDA EM INTEIROS POSITIVOS ---
Expressão: 8 = 4 << 1
 Decimal:
  val=4
  res=8
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000001000
 NOTA: preenche com zeros o lado direito

Expressão: 4611686018427387904 = 4 << 60
 Decimal:
  val=4
  res=4611686018427387904
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=01000000000000000000000000000000000000000000000000000000000000000

Expressão: -9223372036854775808 = 4 << 61
 Decimal:
  val=4
  res=-9223372036854775808
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=10000000000000000000000000000000000000000000000000000000000000000
 NOTA: bit de sinal movido para fora

Expressão: 0 = 4 << 62
 Decimal:
  val=4
  res=0
 Binário:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000000
 NOTA: bits movidos para fora da esquerda


--- MOVENDO BITS A ESQUERDA EM INTEIROS NEGATIVOS ---
Expressão: -8 = -4 << 1
 Decimal:
  val=-4
  res=-8
 Binário:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111000
 NOTA: preenche com zeros o lado direito

Expressão: -9223372036854775808 = -4 << 61
 Decimal:
  val=-4
  res=-9223372036854775808
 Binário:
  val=11111111111111111111111111111111111111111111111111111111111111100
```

```
  res=1000000000000000000000000000000000000000000000000000000000000000

Expressão: 0 = -4 << 62
 Decimal:
  val=-4
  res=0
 Binário:
  val=1111111111111111111111111111111111111111111111111111111111111100
  res=0000000000000000000000000000000000000000000000000000000000000000
 NOTA: bits movidos para fora pela esquerda, incluindo o bit de sinal
```

> **Aviso**
>
> Utilize as funções da extensão gmp para manipulações de troca de bit em números maiores que `PHP_INT_MAX`.

Veja também

- pack()
- unpack()
- gmp_and()
- gmp_or()
- gmp_xor()
- gmp_testbit()
- gmp_clrbit()

Comparação

Operadores de comparação, como os seus nomes implicam, permitem que você compare dois valores. Você pode se interessar em ver as tabelas de comparação de tipos, que tem exemplo das várias comparações entre tipos relacionadas.

Operadores de comparação

Exemplo	Nome	Resultado
$a == $b	Igual	`true` se *$a* é igual a *$b* após equalização de tipos.
$a === $b	Idêntico	`true` se *$a* é igual a *$b*, e eles são do mesmo tipo.
$a != $b	Diferente	`true` se *$a* não é igual a *$b* depois de equalização de ativos.
$a <> $b	Diferente	`true` se *$a* não é igual a *$b* depois de equalização de ativos.
$a !== $b	Não idêntico	`true` se *$a* não é igual a *$b*, ou eles não são do mesmo tipo.
$a < $b	Menor que	`true` se *$a* é estritamente menor que *$b*.
$a > $b	Maior que	`true` se *$a* é estritamente maior que *$b*.
$a <= $b	Menor ou igual	`true` se *$a* é menor ou igual a *$b*.
$a >= $b	Maior ou igual	`true` se *$a* é maior ou igual a *$b*.

Exemplo	Nome	Resultado
$a <=> $b	Spaceship (nave espacial)	Um int menor que, igual a ou maior que zero quando *$a* é, respectivamente, menor que, igual a ou maior que *$b*.

Se ambos os operadores são strings numéricas, ou um operando é um número e o outro é uma string numérica, então a comparação é realizada numericamente. Estas regras se aplicam à instrução switch. A conversão de tipos não é realizada quando a comparação é realizada através `===` ou `!==` porque aqui é realizada a comparação de tipos, além de valores.

> **Aviso**
>
> Anteriormente ao PHP 8.0.0, se uma string era comparada a um número ou uma string numérica, então a string era convertida para número antes de realizar a comparação. Isto pode levar a resultados inesperados, como observado no exemplo a seguir:

```php
<?php
var_dump(0 == "a");
var_dump("1" == "01");
var_dump("10" == "1e1");
var_dump(100 == "1e2");

switch ("a") {
case 0:
    echo "0";
    break;
case "a":
    echo "a";
    break;
}
?>
```

Saída do exemplo acima no PHP 7:

```
bool(true)
bool(true)
bool(true)
bool(true)
0
```

Saída do exemplo acima no PHP 8:

```
bool(false)
bool(true)
bool(true)
bool(true)
a
```

Exemplo #1 Operadores de Comparação

```php
<?php
// Inteiros
echo 1 <=> 1, ' '; // 0
echo 1 <=> 2, ' '; // -1
echo 2 <=> 1, ' '; // 1

// Floats
echo 1.5 <=> 1.5, ' '; // 0
echo 1.5 <=> 2.5, ' '; // -1
echo 2.5 <=> 1.5, ' '; // 1

// Strings
echo "a" <=> "a", ' '; // 0
echo "a" <=> "b", ' '; // -1
echo "b" <=> "a", ' '; // 1

echo "a" <=> "aa", ' ';  // -1
echo "zz" <=> "aa", ' '; // 1

// Arrays
echo [] <=> [], ' ';               // 0
echo [1, 2, 3] <=> [1, 2, 3], ' '; // 0
echo [1, 2, 3] <=> [], ' ';        // 1
echo [1, 2, 3] <=> [1, 2, 1], ' '; // 1
echo [1, 2, 3] <=> [1, 2, 4], ' '; // -1

// Objetos
$a = (object) ["a" => "b"];
$b = (object) ["a" => "b"];
echo $a <=> $b, ' '; // 0

$a = (object) ["a" => "b"];
$b = (object) ["a" => "c"];
echo $a <=> $b, ' '; // -1

$a = (object) ["a" => "c"];
$b = (object) ["a" => "b"];
echo $a <=> $b, ' '; // 1

// valores e tipos precisam coincidir
$a = (object) ["a" => "b"];
$b = (object) ["b" => "b"];
echo $a <=> $b, ' '; // 1

?>
```

Para vários tipos, comparações são feitas de acordo com a seguinte tabela (em ordem).

Comparação com vários tipos

Tipo do 1º operando	Tipo do 2º operando	Resultado

Tipo do 1º operando	Tipo do 2º operando	Resultado
null ou string	string	Converte `null` para "", numérico ou comparação léxica
bool ou null	qualquer	Converte para bool, `false` < `true`
object	object	Classes nativas podem definir suas próprias comparações, classes diferentes são incomparáveis, same class see Comparação de Objetos
string, resource, int ou float	string, resource, int ou float	Transforma strings e resources para números
array	array	Array com menos membros é menor, se a chave do operando 1 não é encontrada no operando 2, então os arrays são incomparáveis, caso contrário compara valor por valor (veja o seguinte exemplo)
object	qualquer	object é sempre maior
array	qualquer	array é sempre maior

Exemplo #2 Comparações de boolean e null

```php
<?php
// bool e null são sempre comparados como booleanos
var_dump(1 == TRUE);  // TRUE - o mesmo que (bool) 1 == TRUE
var_dump(0 == FALSE); // TRUE - o mesmo que (bool) 0 == FALSE
var_dump(100 < TRUE); // FALSE - o mesmo que (bool) 100 < TRUE
var_dump(-10 < FALSE);// FALSE - o mesmo que (bool) -10 < FALSE
var_dump(min(-100, -10, NULL, 10, 100)); // NULL - (bool) NULL < (bool) -100 é FALSE < TRUE
?>
```

Exemplo #3 Transcrição do padrão de comparação de array

```php
<?php
// Arrays são comparados assim quando utilizando-se os operadores padrão e operador nave-espacial
function standard_array_compare($op1, $op2)
{
    if (count($op1) < count($op2)) {
        return -1; // $op1 < $op2
    } elseif (count($op1) > count($op2)) {
        return 1; // $op1 > $op2
    }
    foreach ($op1 as $key => $val) {
        if (!array_key_exists($key, $op2)) {
            return 1;
        } elseif ($val < $op2[$key]) {
            return -1;
        } elseif ($val > $op2[$key]) {
            return 1;
        }
    }
    return 0; // $op1 == $op2
}
?>
```

> **Aviso**

> **Comparison of floating point numbers**
>
> Por conta da forma que floats são representados internamente não se deve testar dois floats com o comparador de igualdade.
>
> Veja a documentação de float para mais detalhes.

> **Nota** :
>
> Cuidado. A conversão automática de tipos do PHP não é sempre tão óbvia, quando comparando valores de tipos diferentes, particularmente comparando ints para bools ou ints para strings. No geral é recomendado utilizar as comparações `===` e `!==` em vez de utilizar `==` e `!=` na maioria dos casos.

> **Nota** :
>
> A ocorrência de recursão durante uma comparação — por exemplo, ao comparar um array ou objeto que contenha uma referência a si mesmo — gera um Error a partir do PHP 8.4.0; anteriormente, isso resultava em um erro fatal.

Valores não comparáveis

Embora a comparação de identidade (`===` e `!==`) possa ser aplicada para valores arbitrários, os outros operadores de comparação somente devem ser aplicados a valores comparáveis. O resultado de comparação de valores não comparáveis não é definido, e não pode ser garantido.

Veja também

- strcasecmp()
- strcmp()
- Array operators
- Types

Operador Ternário

Outro operador condicional é o operador "?:" (ou ternário).

Exemplo #4 Atribuindo um valor padrão

```php
<?php
// Example usage for: Ternary Operator
$action = (empty($_POST['action'])) ? 'default' : $_POST['action'];

// O código acima é idêntico a esta instrução if/else
if (empty($_POST['action'])) {
    $action = 'default';
} else {
    $action = $_POST['action'];
}
?>
```

A expressão `(expr1) ? (expr2) : (expr3)` é avaliada para expr2 se expr1 é avaliada como `true`, ou expr3 se expr1 é avaliada como `false`.

É possível deixar vazia a parte central do operador ternário. A expressão `expr1 ?: expr3` retorna o resultado de expr1 se expr1 avaliar para `true`, e expr3 se não. expr1 somente é avaliada nessa única situação.

> **Nota** :
>
> Note que o operador ternário é uma expressão, e ele não é avaliado para uma variável, mas para o resultado de uma expressão. Isto é importante saber se você quer retornar uma variável por referência. A declaração `return $var == 42 ? $a : $b;` em uma função que retorna por referência conseqüêntemente não irá funcionar e será avisado.

> **Nota** :
>
> É recomendado evitar empilhar operadores ternários. O comportamento do PHP quando utilizando mais de um operador ternário sem parênteses numa única expressão é menos óbvia, comparada a outras linguagens. Antes do PHP 8.0.0, os operadores ternários eram avaliados com associatividade à esquerda, em vez de associatividade à direita, como na maioria de outras linguagens. Depender da associatividade à esquerda foi descontinuado a partir do PHP 7.4.0. A partir do PHP 8.0.0, o operador ternário é não associativo.
>
> **Exemplo #5 Comportamento não óbvio do ternário**
>
> ```php
> <?php
> // Pode parecer que a expressão a seguir imprime 'true'
> echo (true ? 'true' : false ? 't' : 'f');
>
> // No entanto, a saída é na verdade 't' antes do PHP 8.0.0
> // Isto ocorre porque operadores ternários são associativos à esquerda
>
> // O seguinte é a versão mais intuitiva do código acima
> echo ((true ? 'true' : false) ? 't' : 'f');
>
> // Aqui, é possível ver que a primeira expressão avalia para 'true', e então
> // é avaliada para (bool) true, e portanto retorna o variante true da
> // segunda expressão ternária
> ?>
> ```

> **Nota** :
>
> Encadeamento de ternários curtos (`?:`), é estável, e comporta-se intuitivamente. Ele avaliará como o primeiro argumento que avalia para uma valor não falso. Observe que valores não definidos irão emitir um alerta.
>
> **Exemplo #6 Encadeamento de ternários curtos**
>
> ```php
> <?php
> echo 0 ?: 1 ?: 2 ?: 3, PHP_EOL; //1
> echo 0 ?: 0 ?: 2 ?: 3, PHP_EOL; //2
> ```

```php
echo 0 ?: 0 ?: 0 ?: 3, PHP_EOL; //3
?>
```

Operador de aglutinação null (Null Coalescing)

Existe ainda o operador "??" (ou null coalescing).

Exemplo #7 Atribuindo um valor padrão

```php
<?php
// Exemplo do operador Null Coalesce
$action = $_POST['action'] ?? 'default';

// O conteúdo acima é idêntico à essa declaração if/else
if (isset($_POST['action'])) {
    $action = $_POST['action'];
} else {
    $action = 'default';
}
?>
```

A expressão `(expr1) ?? (expr2)` é avaliada para expr2 se expr1 for `null`, e expr1 do contrário.

Esse operador em particular não emite aviso caso o valor da esquerda não exista, assim como isset(). Sendo especialmente útil em chaves de arrays.

> **Nota** :
>
> Note que o operador null coalescing é uma expressão, e ele não é avaliado para uma variável, mas para o resultado de uma expressão. Isto é importante saber se você quer retornar uma variável por referência. A declaração `return $foo ?? $bar;` em uma função que retorna por referência conseqüêntemente não irá funcionar e será avisado.

> **Nota** :
>
> O operador de aglutinação null tem baixa precedência. Isso significa que misturá-lo com outros operadores (como por exemplo concatenação ou aritmética) irá provavelmente exigir parênteses.
>
> ```php
> <?php
> // Emite um aviso que $name não está definido.
> print 'Mr. ' . $name ?? 'Anonymous';
>
> // Exibe "Mr. Anonymous"
> print 'Mr. ' . ($name ?? 'Anonymous');
> ?>
> ```

> **Nota** :
>
> Note que o operador null coalescing permite a criação de aninhamentos simples:
>
> **Exemplo #8 Aninhando o operador null coalescing**

```php
<?php

$foo = null;
$bar = null;
$baz = 1;
$qux = 2;

echo $foo ?? $bar ?? $baz ?? $qux; // exibe 1

?>
```

Controle de Erro

O PHP suporta um operador de controle de erro: o sinal 'arroba' (@). Quando ele precede uma expressão em PHP, qualquer mensagem de erro que possa ser gerada por aquela expressão será ignorada.

Se um manipulador de erro personalizado estiver configurado com set_error_handler(), ele ainda assim será chamado, mesmo com a mensagem de diagnóstico suprimida.

> **Aviso**
>
> Antes do PHP 8.0.0, error_reporting() chamada dentro de um manipulador de erro sempre retornava `0` se a mensagem de erro era suprimida pelo operador `@`. Desde o PHP 8.0.0, ele retorna o valor da expressão bitwise dos valores: `E_ERROR | E_CORE_ERROR | E_COMPILE_ERROR | E_USER_ERROR | E_RECOVERABLE_ERROR | E_PARSE`.

Quaisquer mensagens de erro geradas na expressão ficam disponíveis no elemento `"message"` do array retornado por error_get_last(). O resultado daquela função irá se alterar em cada erro, então é necessário fazer a verificação imediatamente.

Exemplo #1 Erro de arquivo intencional

```php
<?php
$my_file = @file ('arquivo_nao_existente') or
    die ("Falha abrindo arquivo: erro foi '" . error_get_last()['message'] . "'");
?>
```

Exemplo #2 Suprimindo expressões

```php
<?php
// Isto funciona para qualquer expressão, não apenas para funções:
$value = @$cache[$key];
// não haverá nenhum aviso se a chave $key não existir.
?>
```

> **Nota** :
>
> O operador `@` funciona somente em expressões. Uma regra simples para lembrar disso: se você pode pegar o valor de alguma coisa, você pode prefixar isso com o operador `@`. Por exemplo, ele pode prefixar variáveis, chamadas de funções, alguns constructos de linguagem (include), e assim por diante. O

operador não pode ser prefixado em definições de classes ou funções, estruturas condicionais como `if`, `foreach`, e semelhantes.

> **Aviso**
>
> Antes do PHP 8.0.0, o operador `@` conseguia inibir erros críticos que iriam encerrar a execução do script. Por exemplo, prefixar `@` numa chamada de uma função inexistente, por estar indisponível ou por erro de digitação, iria encerrar o script sem nenhuma indicação da causa.

Veja também

- error_reporting()
- Manipulação de erros e funções de log

Execução

O PHP suporta um operador de execução: acentos graves (``). Note que não são aspas simples! O PHP tentará executar o conteúdo dentro dos acentos graves como um comando do shell; a saída será retornada (isto é, ela não será simplesmente mostrada na tela; ela pode ser atribuída a uma variável). A utilização do operador de execução é idêntica a da função shell_exec().

Exemplo #1 Operador de execução

```php
<?php
$output = `ls -al`;
echo "<pre>$output</pre>";
?>
```

> **Nota** :
>
> O operador de execução fica desabilitado quando shell_exec() está desabilitado.

> **Nota** :
>
> Ao contrário de outras linguagens, acentos graves não têm sentido especial dentro de strings delimitadas por aspas.

Registro de Alterações

Versão	Descrição
8.5.0	O operador de execução como um sinônimo para shell_exec() foi descontinuado.

Veja também

- Funções de execução de programas
- popen()
- proc_open()
- Utilizando o PHP em linha de comando

Lógica

Operadores Lógicos

Exemplo	Nome	Resultado
$a and $b	E	`true` se ambos *$a* e *$b* são `true`.
$a or $b	OU	`true` se *$a* ou *$b* é `true`.
$a xor $b	XOR	`true` se *$a* ou *$b* é `true`, mas não ambos ao mesmo tempo.
! $a	Não	`true` se *$a* não é `true`.
$a && $b	E	`true` se ambos *$a* e *$b* são `true`.
$a || $b	OU	`true` se *$a* ou *$b* é `true`.

A razão para as duas variantes dos operandos "and" e "or" é que eles operam com precedências diferentes. (Veja Precedência de Operadores.)

Exemplo #1 Ilustrando operadores lógicos

```php
<?php

// --------------------
// foo() nunca será chamada porque toda a expressão sofre curto circuito

$a = (false && foo());
$b = (true  || foo());
$c = (false and foo());
$d = (true  or  foo());

// --------------------
// "||" tem maior precedência que "or"

// O resultado da expressão (false || true) é atribuído em $e
// Funciona como: ($e = (false || true))
$e = false || true;

// A constante false é atribuída a $f antes que o "or" ocorra
// Funciona como: (($f = false) or true)
$f = false or true;

var_dump($e, $f);

// --------------------
// "&&" tem maior precedência que "and"

// O resultado da expressão (true && false) é atribuído em $g
// Funciona como: ($g = (true && false))
$g = true && false;

// A constante true é atribuída em $h antes que o "and" ocorra
// Funciona como: (($h = true) and false)
$h = true and false;

var_dump($g, $h);
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
bool(false)
bool(false)
bool(true)
```

String

Há dois operadores de string. O primeiro é o operador de concatenação ('.'), que retorna a concatenação dos seus argumentos direito e esquerdo. O segundo é o operador de atribuição de concatenação ('.='), que acrescenta o argumento do lado direito no argumento do lado esquerdo. Veja em Operadores de Atribuição para mais informações.

Exemplo #1 Concatenação de strings

```php
<?php
$a = "Olá ";
$b = $a . "mundo!"; // agora $b contém "Olá mundo!"
var_dump($b);

$a = "Olá ";
$a .= "mundo!";     // agora $a contém "Olá mundo!"
var_dump($a);
?>
```

Veja também

- Tipo string
- Funções string

Arrays

Operadores de array

Exemplo	Nome	Resultado
$a + $b	União	União de *$a* e *$b*.
$a == $b	Igualdade	`true` se *$a* e *$b* têm as mesmas chaves e valores.
$a === $b	Identidade	`true` se *$a* e *$b* têm as mesmas chaves e valores, na mesma ordem e com os mesmos tipos.
$a != $b	Desigualdade	`true` se *$a* não é igual *$b*.
$a <> $b	Desigualdade	`true` se *$a* não é igual *$b*.
$a !== $b	Não identidade	`true` se *$a* não é idêntico a *$b*.

O operador + retorna o array à direita anexado ao array da esquerda. Para chaves que existam nos dois arrays os elementos do array à esquerda serão mantidos, os valores de mesma chave no array da direita são

ignorados.

Exemplo #1 Operador de união de array

```php
<?php
$a = array("a" => "apple", "b" => "banana");
$b = array("a" => "pear", "b" => "strawberry", "c" => "cherry");

$c = $a + $b; // União de $a e $b
echo "União de \$a e \$b: \n";
var_dump($c);

$c = $b + $a; // União de $b e $a
echo "União de \$b e \$a: \n";
var_dump($c);

$a += $b; // União de $a += $b é $a e $b
echo "União de \$a += \$b: \n";
var_dump($a);
?>
```

O exemplo acima produzirá:

```
União de $a e $b:
array(3) {
  ["a"]=>
  string(5) "apple"
  ["b"]=>
  string(6) "banana"
  ["c"]=>
  string(6) "cherry"
}
União de $b e $a:
array(3) {
  ["a"]=>
  string(4) "pear"
  ["b"]=>
  string(10) "strawberry"
  ["c"]=>
  string(6) "cherry"
}
União de $a += $b:
array(3) {
  ["a"]=>
  string(5) "apple"
  ["b"]=>
  string(6) "banana"
  ["c"]=>
  string(6) "cherry"
}
```

Elementos do array são iguais para efeitos de comparação se eles possuem o mesmo valor e chave.

Exemplo #2 Comparando arrays

```php
<?php
$a = array("maçã", "banana");
$b = array(1 => "banana", "0" => "maçã");

var_dump($a == $b); // bool(true)
var_dump($a === $b); // bool(false)
?>
```

Veja também

- Tipo array
- Funções em arrays

Tipo

`instanceof` é usado para determinar se um variável do PHP é uma objeto instanciado de uma certa classe:

Exemplo #1 Usando `instanceof` com classes

```php
<?php
class MyClass
{
}

class NotMyClass
{
}
$a = new MyClass;

var_dump($a instanceof MyClass);
var_dump($a instanceof NotMyClass);
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(false)
```

`instanceof` pode também ser usado para determinar se uma variável é um objeto instanciado de uma classe que herda de uma classe pai:

Exemplo #2 Usando `instanceof` com herança

```php
<?php
class ParentClass
{
}
```

```php
class MyClass extends ParentClass
{
}

$a = new MyClass;

var_dump($a instanceof MyClass);
var_dump($a instanceof ParentClass);
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
```

Para verificar se um objeto *não* é uma instância de uma classe, o operador lógico `not` pode ser usado.

Exemplo #3 Usando `instanceof` para verificar se o objeto *não* é uma instância da classe

```php
<?php
class MyClass
{
}

$a = new MyClass;
var_dump(!($a instanceof stdClass));
?>
```

O exemplo acima produzirá:

```
bool(true)
```

Por fim, `instanceof` pode também ser usado para determinar se uma variável é um objeto instanciado de uma classe que implementa uma interface:

Exemplo #4 Usando `instanceof` para classe

```php
<?php
interface MyInterface
{
}

class MyClass implements MyInterface
{
}
```

```php
$a = new MyClass;

var_dump($a instanceof MyClass);
var_dump($a instanceof MyInterface);
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
```

Embora `instanceof` é usualmente usado com um nome de classe literal, ele pode também ser usado com outro objeto ou uma variável string:

Exemplo #5 Usando `instanceof` com outras variáveis

```php
<?php
interface MyInterface
{
}

class MyClass implements MyInterface
{
}

$a = new MyClass;
$b = new MyClass;
$c = 'MyClass';
$d = 'NotMyClass';

var_dump($a instanceof $b); // $b é um objeto da classe MyClass
var_dump($a instanceof $c); // $c é uma string 'MyClass'
var_dump($a instanceof $d); // $d é uma string 'NotMyClass'
?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
bool(false)
```

instanceof não lança erros se a variável testada não for um objeto, apenas retorna `false`. Constantes não são permitidas anteriormente ao PHP 7.3.0.

Exemplo #6 Utilizando `instanceof` para testar outras variáveis

```php
<?php
$a = 1;
$b = NULL;
$c = fopen('/tmp/', 'r');
var_dump($a instanceof stdClass); // $a é um inteiro
var_dump($b instanceof stdClass); // $b é NULL
var_dump($c instanceof stdClass); // $c é um recurso
var_dump(FALSE instanceof stdClass);
?>
```

O exemplo acima produzirá:

```
bool(false)
bool(false)
bool(false)
PHP Fatal error:  instanceof expects an object instance, constant given
```

Desde o PHP 7.3.0, constantes são permitidas à esquerda do operador `instanceof`.

Exemplo #7 Utilizando `instanceof` em constantes

```php
<?php
var_dump(FALSE instanceof stdClass);
?>
```

Saída do exemplo acima no PHP 7.3:

```
bool(false)
```

Desde o PHP 8.0.0, `instanceof` pode ser utilizado em expressões arbitrárias. A expressão precisa estar dentro de parênteses e produzir uma string.

Exemplo #8 Usando `instanceof` em expressões arbitrárias

```php
<?php

class ClassA extends \stdClass {}
class ClassB extends \stdClass {}
class ClassC extends ClassB {}
class ClassD extends ClassA {}

function getSomeClass(): string
{
    return ClassA::class;
}
```

```php
var_dump(new ClassA instanceof ('std' . 'Class'));
var_dump(new ClassB instanceof ('Class' . 'B'));
var_dump(new ClassC instanceof ('Class' . 'A'));
var_dump(new ClassD instanceof (getSomeClass()));
?>
```

Saída do exemplo acima no PHP 8:

```
bool(true)
bool(true)
bool(false)
bool(true)
```

O operador `instanceof` tem uma variação funcional chamada is_a().

Veja também

- get_class()
- is_a()

Funcional

O PHP 8.5 e versões posteriores suportam um operador que funciona diretamente em chamáveis. O operador `|>`, ou "pipe", aceita um chamável de parâmetro único à direita e passa o valor do lado esquerdo para ele, avaliando o resultado do chamável. O chamável à direita pode ser qualquer chamável PHP válido: um Closure, um chamável de primeira classe, um objeto que implementa __invoke(), etc.

Isso significa que as duas linhas seguintes são logicamente equivalentes.

Exemplo #1 Usando `|>`

```php
<?php
$result = "Hello World" |> strlen(...);
echo $result, PHP_EOL;

$result = strlen("Hello World");
echo $result, PHP_EOL;
?>
```

O exemplo acima produzirá:

```
11
11
```

Para uma única chamada, isso não é muito útil. Torna-se útil quando várias chamadas são encadeadas. Ou seja, os dois fragmentos de código a seguir são logicamente equivalentes:

Exemplo #2 Encadeando chamadas com |>

```php
<?php
$result = "PHP Rocks"
    |> htmlentities(...)
    |> str_split(...)
    |> (fn($x) => array_map(strtoupper(...), $x))
    |> (fn($x) => array_filter($x, fn($v) => $v != 'O'))
;
print_r($result);

$temp = "PHP Rocks";
$temp = htmlentities($temp);
$temp = str_split($temp);
$temp = array_map(strtoupper(...), $temp);
$temp = array_filter($temp, fn($v) => $v != 'O');
$result = $temp;
print_r($result);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => P
    [1] => H
    [2] => P
    [3] =>
    [4] => R
    [6] => C
    [7] => K
    [8] => S
)
Array
(
    [0] => P
    [1] => H
    [2] => P
    [3] =>
    [4] => R
    [6] => C
    [7] => K
    [8] => S
)
```

O lado esquerdo do pipe pode ser qualquer valor ou expressão. O lado direito pode ser qualquer chamável PHP válido que receba um único parâmetro, ou qualquer expressão que seja avaliado como tal chamável. Funções com mais de um parâmetro obrigatório não são permitidas e falharão como se fossem chamadas

normalmente com argumentos insuficientes. Funções que recebem uma variável por referência não são permitidas. Se o lado direito não for avaliado como um chamável válido, será gerado um erro.

> **Nota** :
>
> Esteja ciente de que, para evitar ambiguidade de sintaxe, funções de seta PRECISAM ser colocadas entre parênteses quando usadas com um operador de pipe, como nos exemplos acima. Caso contrário, ocorrerá um erro fatal.

Veja também

- Closure

Estruturas de Controle

Índice

Introdução

Qualquer script PHP é construído por uma série de instruções. Uma instrução pode ser uma atribuição, uma chamada de função, um laço de repetição, uma instrução condicional, ou mesmo uma instrução que não faz nada (um comando vazio). Instruções geralmente terminam com um ponto e vírgula. Além disso, as instruções podem ser agrupadas em um bloco de comandos através do encapsulamento de um grupo de comandos com chaves. Um grupo de comandos é uma instrução também. Os vários tipos de instruções são descritos neste capítulo.

Veja também

Os itens a seguir são considerados como construtos da linguagem, mesmo que eles sejam referenciados como funções neste manual.

- list()
- array()
- echo
- eval()
- print

if

(PHP 4, PHP 5, PHP 7, PHP 8)

O construtor `if` é um dos recursos mais importantes em muitas linguagens, inclusive no PHP. Permite a execução condicional de fragmentos de código. O PHP apresenta uma estrutura `if` semelhante a do C:

```
if (expr)
  statement
```

Como descrito na seção sobre expressões, expressões são avaliadas por seus valores Booleanos. Se uma expressão for avaliada como `true`, o PHP executará a declaração, e se avaliá-la como `false` - ignorá-la. Mais

informações sobre quais valores são avaliados como `false` podem ser encontradas na seção 'Conversão para booleano' .

O exemplo a seguir exibirá a é maior que b se *$a* for maior que *$b*:

```
<?php
$a = 3;
$b = 1;
if ($a > $b)
  echo "a é maior que b\n";
```

O exemplo acima produzirá:

```
a é maior que b
```

Muitas vezes deseja-se que mais de uma declaração seja condicionalmente executada. É claro que não é necessário envolver cada declaração em uma cláusula `if`. Em vez disso, pode-se agrupar várias declarações em grupos. Por exemplo, este código exibirá a é maior que b se *$a* for maior que *$b*, e atribuirá o valor de *$a* em *$b*:

```
<?php
$a = 3;
$b = 1;
if ($a > $b) {
  echo "a é maior que b\n";
  $b = $a;
}
var_dump($b);
```

O exemplo acima produzirá:

```
a é maior que b
int(3)
```

A declaração `If` pode ser aninhada indefinidamente dentro de outras declarações `if`, fornecendo completa flexibilidade para execução condicional de várias partes do programa.

else

(PHP 4, PHP 5, PHP 7, PHP 8)

Muitas vezes deseja-se executar uma instrução se uma certa condição for válida, e uma instrução diferente se a mesma condição não for válida. Para isso que o `else` serve. O `else` estende a instrução `if` para executar

outras caso a expressão no `if` retornar `false`. Por exemplo, o código a seguir exibirá a é maior que b se *$a* for maior que *$b*, e a NÃO é maior que b caso contrário:

```
<?php
$a = 3;
$b = 5;
if ($a > $b) {
  echo "a é maior que b\n";
} else {
  echo "a NÃO é maior que b\n";
}
```

O exemplo acima produzirá:

```
a NÃO é maior que b
```

A instrução `else` só é executada se a expressão de avaliação do `if` for avaliada como `false`, e se tiver qualquer expressão `elseif` - somente se também retornarem `false` (veja elseif).

> **Nota** : **Else pendurado**
>
> Em caso de declarações `if-else` aninhadas, um `else` é sempre associado com o `if` mais próximo.
>
> ```
> <?php
> $a = false;
> $b = true;
> if ($a)
>     if ($b)
>         echo "b";
> else
>     echo "c";
> ```
>
> O exemplo acima produzirá:
>
> ```
> 
> ```
>
> Apesar da indentação (o que não importa para o PHP), o `else` é associado com o `if ($b)`, então o exemplo não produz nenhuma saída. Embora seja válido confiar nesse comportamento, é recomendado evitá-lo usando chaves para resolver ambiguidades em potencial.

elseif/else if

(PHP 4, PHP 5, PHP 7, PHP 8)

`elseif`, como o nome sugere, é uma combinação do `if` e `else`. Como o `else`, estende um `if` para executar instruções diferentes no caso da expressão `if` original ser avaliada como `false`. Entretanto, diferentemente do

`else`, executará uma expressão alternativa somente se a expressão condicional do `elseif` for avaliada como `true`. Por exemplo, o código a seguir exibirá a é maior que b, a é igual a b ou a é menor que b:

```
<?php
$a = 1;
$b = 1;
if ($a > $b) {
    echo "a é maior que b\n";
} elseif ($a == $b) {
    echo "a é igual a b\n";
} else {
    echo "a é menor que b\n";
}
```

O exemplo acima produzirá:

```
a é igual a b
```

Pode haver vários `elseif`s dentro da mesma instrução `if`. A primeira expressão `elseif` (se houver) que retornar `true` será executada. No PHP, pode-se escrever `else if` (em duas palavras), e o comportamento será idêntico ao do `elseif` (em uma única palavra). O significado sintático é um pouco diferente (se você está familiarizado com C, mas, no final, ambos terão exatamente o mesmo comportamento.

O `elseif` só é executado se o `if` precedente ou qualquer `elseif` for avaliado como `false`, e o `elseif` corrente for avaliado como `true`.

> **Nota** :
>
> Note que o `elseif` e `else if` só serão considerados exatamente iguais se usados com chaves como no exemplo abaixo. Ao utilizar os dois pontos (:) para definir as condições de `if`/`elseif`, então o uso de `elseif` como uma única palavras se torna necessário, ou o PHP falhará com um erro de interpretação se `else if` estiver separado em duas palavras.

```
<?php
$a = 3;
$b = 1;

/* Método Incorreto: */
if ($a > $b):
    echo $a." é maior que ".$b;
else if ($a == $b): // Não irá compilar.
    echo "A linha acima causa um erro de análise.";
endif;
```

O exemplo acima produzirá:

```
Parse error: syntax error, unexpected token "if", expecting ":" in script on line 8
```

```php
<?php
$a = 3;
$b = 1;

/* Método Correto: */
if ($a > $b):
    echo $a." é maior que ".$b;
elseif ($a == $b): // Note a combinação das palavras.
    echo $a." é igual a ".$b;
else:
    echo $a." não é maior nem igual a ".$b;
endif;
```

O exemplo acima produzirá:

```
3 é maior que 1
```

Sintaxe Alternativa para estruturas de controle

(PHP 4, PHP 5, PHP 7, PHP 8)

O PHP oferece uma sintaxe alternativa para algumas estruturas de controle; a saber, `if`, `while`, `for`, `foreach`, e `switch`. Em cada caso, basicamente a sintaxe alternativa é trocar a chave de abertura por dois pontos (:) e a chave de fechamento por `endif;`, `endwhile;`, `endfor;`, `endforeach;`, ou `endswitch;`, respectivamente.

```php
<?php
$a = 5;
if ($a == 5): ?>
A é igual a 5
<?php endif; ?>
```

O exemplo acima produzirá:

```
A é igual a 5
```

No exemplo acima, o bloco HTML "A é igual a 5" está dentro de um `if` escrito na sintaxe alternativa. O bloco HTML será exibido somente se *$a* for igual a 5.

A sintaxe alternativa também se aplica ao `else` e `elseif`. A seguir um exemplo de um `if` com `elseif` e `else` no formato alternativo:

```php
<?php
$a = 5;
if ($a == 5):
    echo "a é igual a 5";
    echo "...";
elseif ($a == 6):
    echo "a é igual a 6";
    echo "!!!";
else:
    echo "a não é 5 nem 6";
endif;
```

O exemplo acima produzirá:

```
a é igual a 5...
```

> **Nota** :
>
> Misturar a sintaxe no mesmo bloco de controle não é suportado.

> **Aviso**
>
> Qualquer saída (inclusive espaços em branco) entre um `switch` e o primeiro `case` irá resultar em um erro de sintaxe. O exemplo abaixo é inválido:
>
> ```php
> <?php
> $foo = 1;
> switch ($foo): ?>
>     <?php case 1: ?>
>     Foo é igual a 1
> <?php endswitch; ?>
> ```
>
> O exemplo acima produzirá:
>
> ```
> Parse error: syntax error, unexpected T_INLINE_HTML "    ", expecting "endswitch" or "case"
> or "default" in script on line 4
> ```
>
> Enquanto que este é válido, já que a quebra de linha depois do `switch` é considerado parte da tag de fechamento `?>` e, portanto, nenhuma saída será enviada entre o `switch` e `case`:
>
> ```php
> <?php
> $foo = 1;
> switch ($foo): ?>
> <?php case 1: ?>
> ```

```
    Foo é igual a 1
<?php endswitch; ?>
```

O exemplo acima produzirá:

```
    Foo é igual a 1
```

Veja mais sobre while, for, e if para outros exemplos.

while

(PHP 4, PHP 5, PHP 7, PHP 8)

Laços `while` são os mais simples tipos de laços do PHP. Possui comportamento semelhante ao C. O formato básico de uma declaração `while` é:

```
while (expr)
    statement
```

O propósito da declaração `while` é simples. Ele dirá ao PHP para executar as declarações aninhadas repetidamente, enquanto a expressão do `while` for avaliada como **`true`**. O valor da expressão é checado a cada vez que o laço é iniciado, então, mesmo seu valor mude durante a execução das declarações aninhadas, a execução não será interrompida até o final da iteração (cada vez que o PHP executa as declarações dentro do laço é uma iteração). Se a expressão do `while` for avaliada como **`false`** desde o início, as declarações aninhadas não serão executadas nenhuma vez.

Similar a declaração `if`, pode-se agrupar múltiplas declarações no mesmo laço `while` delimitando o grupo de declarações por chaves, ou utilizando a sintaxe alternativa:

```
while (expr):
    statement
    ...
endwhile;
```

Os exemplos a seguir são idênticos, e ambos imprimem os números de 1 a 10.

```
<?php
$i = 1;
while ($i <= 10) {
    echo $i++ . "\n";  /* o valor exibido seria
                          $i antes do incremento
                          (incrementa depois) */
}
```

O exemplo acima produzirá:

```
```

```
1
2
3
4
5
6
7
8
9
10
```

```php
<?php
$i = 1;
while ($i <= 10):
    echo $i . "\n";
    $i++;
endwhile;
```

O exemplo acima produzirá:

```
1
2
3
4
5
6
7
8
9
10
```

do-while

(PHP 4, PHP 5, PHP 7, PHP 8)

Os laços `do-while` são muito similares aos laços `while`, com exceção que a expressão de avaliação é verificada ao final de cada iteração em vez de no começo. A maior diferença para o laço `while` é que a primeira iteração do laço `do-while` sempre é executada (a expressão de avaliação é executada somente no final da iteração), considerando que no laço `while` não é necessariamente executada (a expressão de avaliação é executada no começo de cada iteração; se avaliada como `false` logo no começo, a execução do laço será abortada imediatamente).

Só há uma sintaxe para o laço `do-while`:

```php
<?php
$i = 0;
do {
```

```
    echo $i;
} while ($i > 0);
```

O exemplo acima produzirá:

```
0
```

O laço acima será executado somente uma vez, pois após a primeira iteração, quando a expressão verdade for verificada, resultará em `false` (*$i* não é maior que 0) e a execução do laço será encerrada.

Usuários avançados de C devem estar familiarizados com um uso diferente do laço `do-while`, que permite parar a execução no meio do bloco de códigos, encapsulando-os em um `do-while` (0), e usando o `break`. O código a seguir demonstra isso:

```
<?php
$i = 50;
$factor = 0.5;
$minimum_limit = 20;
do {
    if ($i < 5) {
        echo "i não é grande o suficiente\n";
        break;
    }
    $i *= $factor;
    if ($i < $minimum_limit) {
        echo "i é menor que o limite mínimo após fatoração\n";
        break;
    }
    echo $i . " está ok\n";

    /* processa i */

} while (0);
```

O exemplo acima produzirá:

```
25 está ok
```

É possível usar o `goto` ao invés desse truque.

for

(PHP 4, PHP 5, PHP 7, PHP 8)

Os laços `for` são os mais complexos no PHP. Possui comportamento semelhante ao C. A sintaxe do laço `for` é:

```
for (expr1; expr2; expr3)
    statement
```

A primeira expressão (*expr1*) é avaliada (executada), uma vez, incondicionalmente, no início do laço.

No começo de cada iteração a *expr2* é avaliada. Se a avaliada como `true`, o laço continuará e as instruções aninhadas serão executadas. Se avaliada como `false`, a execução do laço terminará.

No final de cada iteração, a *expr3* é avaliada (executada).

Cada uma das expressões podem ser vazias ou conter múltiplas expressões separadas por vírgulas. Na *expr2*, todas as expressões separadas por vírgula são avaliadas mas o resultado é obtido da última parte. Se a *expr2* estiver vazia significa que o laço deve ser executado indefinidamente (O PHP considera implicitamente como `true`, igual ao C). Isto não é inútil como se imagina, pois muitas vezes deseja-se interromper o laço utilizando a instrução `break` ao invés de usar a expressão verdade do `for`.

Analise os seguintes exemplos. Todos exibem números de 1 até 10:

```php
<?php
for ($i = 1; $i <= 10; $i++) {
    echo $i . " ";
}
```

O exemplo acima produzirá:

```
1 2 3 4 5 6 7 8 9 10
```

```php
<?php
for ($i = 1; ; $i++) {
    if ($i > 10) {
        break;
    }
    echo $i . " ";
}
```

O exemplo acima produzirá:

```
1 2 3 4 5 6 7 8 9 10
```

```php
<?php
$i = 1;
for (; ; ) {
    if ($i > 10) {
        break;
    }
    echo $i . " ";
```

```
    $i++;
}
```

O exemplo acima produzirá:

```
1 2 3 4 5 6 7 8 9 10
```

```
<?php
for ($i = 1, $j = 0; $i <= 10; $j += $i, print $i . " ", $i++);
```

O exemplo acima produzirá:

```
1 2 3 4 5 6 7 8 9 10
```

É claro que o primeiro exemplo aparenta ser o mais simpático (ou talvez o quarto), mas pode-se achar que o uso de expressões vazias no laço `for`, seja vantajoso em algumas ocasiões.

O PHP também suporta a sintaxe alternativa "dois pontos" para o laço `for`.

```
for (expr1; expr2; expr3):
    statement
    ...
endfor;
```

É comum, para muitos usuários, iterar em arrays como no exemplo abaixo.

```
<?php
/*
 * Este é um array com alguns dados que devem ser modificados
 * durante a execução do loop for.
 */
$pessoas = array(
    array('nome' => 'Lucas', 'salt' => 856412),
    array('nome' => 'Marina', 'salt' => 215863)
);

for($i = 0; $i < count($pessoas); ++$i) {
    $pessoas[$i]['salt'] = random_int(100000, 999999);
}
var_dump($people);
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {
  [0]=>
  array(2) {
    ["name"]=>
    string(5) "Lucas"
    ["salt"]=>
    int(454478)
  }
  [1]=>
  array(2) {
    ["name"]=>
    string(6) "Marina"
    ["salt"]=>
    int(776978)
  }
}
```

O código acima pode se tornar lento, pois o tamanho do array será calculado a cada iteração. Desde que o tamanho nunca mude, o laço pode ser facilmente otimizado usando uma variável intermediária para armazenar o tamanho em vez de executar repetidamente o count():

```php
<?php
$pessoas = array(
    array('nome' => 'Lucas', 'salt' => 856412),
    array('nome' => 'Marina', 'salt' => 215863)
);

for($i = 0, $tamanho = count($pessoas); $i < $tamanho; ++$i) {
    $pessoas[$i]['salt'] = random_int(100000, 999999);
}
var_dump($people);
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {
  [0]=>
  array(2) {
    ["name"]=>
    string(5) "Lucas"
    ["salt"]=>
    int(454478)
  }
  [1]=>
  array(2) {
    ["name"]=>
    string(6) "Marina"
    ["salt"]=>
    int(776978)
  }
}
```

foreach

(PHP 4, PHP 5, PHP 7, PHP 8)

O construtor `foreach` fornece uma maneira fácil de iterar sobre arrays e objetos Traversable. O `foreach` emitirá um erro quando usado com uma variável contendo um tipo de dado diferente ou com uma variável não inicializada.

`foreach` pode, opcionalmente, obter a chave (`key`) de cada elemento:

```
foreach (expressão_iterável as $valor) {
    lista_de_instruções
}

foreach (expressão_iterável as $chave => $valor) {
    lista_de_instruções
}
```

A primeira forma percorre o iterável dado por `iterable_expression`. A cada iteração, o valor do elemento atual é atribuído à variável `$valor`.

A segunda forma irá, adicionalmente, atribuir a chave do elemento corrente à variável `$chave` a cada iteração.

Note que o `foreach` não modifica o ponteiro interno do array, que é usado por funções como current() e key().

É possível customizar a iteração em objetos.

Exemplo #1 Usos comuns de `foreach`

```php
<?php

echo "Somente valor\n";
$array = [1, 2, 3, 17];

foreach ($array as $value) {
    echo "Elemento atual de \$array: $value.\n";
}

echo "\n\nChave e valor:\n";
$array = [
    "um" => 1,
    "dois" => 2,
    "três" => 3,
    "dezessete" => 17
];

foreach ($array as $key => $value) {
    echo "Chave: $key => Valor: $value\n";
}

echo "\n\nArrays chave-valor multidimensionais:\n";
$grid = [];
$grid[0][0] = "a";
```

```php
$grid[0][1] = "b";
$grid[1][0] = "y";
$grid[1][1] = "z";

foreach ($grid as $y => $row) {
    foreach ($row as $x => $value) {
        echo "Valor na posição x=$x e y=$y: $value\n";
    }
}

echo "\n\nArrays dinâmicos:\n";
foreach (range(1, 5) as $value) {
    echo "$value\n";
}
```

O exemplo acima produzirá:

```
Somente valor
Elemento atual de $array: 1.
Elemento atual de $array: 2.
Elemento atual de $array: 3.
Elemento atual de $array: 17.


Chave e valor:
Chave: um => Valor: 1
Chave: dois => Valor: 2
Chave: três => Valor: 3
Chave: dezessete => Valor: 17


Arrays chave-valor multidimensionais:
Valor na posição x=0 e y=0: a
Valor na posição x=1 e y=0: b
Valor na posição x=0 e y=1: y
Valor na posição x=1 e y=1: z


Arrays dinâmicos:
1
2
3
4
5
```

> **Nota** :
>
> `foreach` não suporta a capacidade de supressão de mensagens de erro usando o operador `@`.

Desempacotando arrays aninhados

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

É possível iterar sobre um array de arrays e desempacotar o array aninhado em variáveis do laço usando desconstrução de array através de `[]` ou usando a construção de linguagem list() como o valor.

> **Nota** :
>
> Observe que a desconstrução de array através de `[]` só é possível a partir do PHP 7.1.0.

Em ambos os exemplos a seguir, `$a` será definida para o primeiro elemento do array aninhado e `$b` conterá o segundo elemento:

```php
<?php
$array = [
    [1, 2],
    [3, 4],
];

foreach ($array as [$a, $b]) {
    echo "A: $a; B: $b\n";
}

foreach ($array as list($a, $b)) {
    echo "A: $a; B: $b\n";
}
```

O exemplo acima produzirá:

```
A: 1; B: 2
A: 3; B: 4
A: 1; B: 2
A: 3; B: 4
```

Ao fornecer menos variáveis que o número de elementos no array, os elementos restantes serão ignorados. Da mesma forma, elementos podem ser pulados usando uma vírgula:

```php
<?php
$array = [
    [1, 2, 5],
    [3, 4, 6],
];

foreach ($array as [$a, $b]) {
    // Observe que não existe $c aqui.
    echo "$a $b\n";
}

foreach ($array as [, , $c]) {
    // Pulando $a e $b
```

```
    echo "$c\n";
}
```

O exemplo acima produzirá:

```
1 2
3 4
5
6
```

Um aviso será gerado se não houver elementos suficientes no array para o preenchimento do construtor list():

```php
<?php
$array = [
    [1, 2],
    [3, 4],
];

foreach ($array as [$a, $b, $c]) {
    echo "A: $a; B: $b; C: $c\n";
}
```

O exemplo acima produzirá:

```
Notice: Undefined offset: 2 in example.php on line 7
A: 1; B: 2; C:

Notice: Undefined offset: 2 in example.php on line 7
A: 3; B: 4; C:
```

foreach e referências

É possível modificar elementos de array diretamente dentro de um laço, precedendo `$value` com `&`. Neste caso, o valor será atribuído por referência.

```php
<?php
$arr = [1, 2, 3, 4];
foreach ($arr as &$value) {
    $value = $value * 2;
}
// $arr agora é [2, 4, 6, 8]
var_dump($arr);
unset($value); // quebra a referência com o último elemento
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  int(2)
  [1]=>
  int(4)
  [2]=>
  int(6)
  [3]=>
  &int(8)
}
```

> **Aviso**
>
> A referência ao `$value` do último elemento do array permanece mesmo depois do laço `foreach`. É recomendado destruir essas referências usando unset(). Caso contrário, o comportamento a seguir irá ocorrer:

```php
<?php
$arr = [1, 2, 3, 4];
foreach ($arr as &$value) {
    $value = $value * 2;
}
// $arr agora é [2, 4, 6, 8]
var_dump($arr);

// sem um unset($value), $value ainda é uma referência ao último item: $arr[3]

echo "\nOutra repetição:\n";
foreach ($arr as $key => $value) {
    // $arr[3] será atualizado com cada valor de $arr...
    echo "{$key} => {$value}\n";
}
// ...até finalmente o penúltimo valor ser copiado no último valor
var_dump($arr);
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  int(2)
  [1]=>
  int(4)
  [2]=>
  int(6)
  [3]=>
  &int(8)
}

Another loop:
```

```
0 => 2
1 => 4
2 => 6
3 => 6
array(4) {
  [0]=>
  int(2)
  [1]=>
  int(4)
  [2]=>
  int(6)
  [3]=>
  &int(6)
}
```

Exemplo #2 Iterando valores de um array constante por referência

```php
<?php
foreach ([1, 2, 3, 4] as &$value) {
    $value = $value * 2;
    echo $value . "\n";
}
```

O exemplo acima produzirá:

```
2
4
6
8
```

Veja também

- array
- Traversable
- iterable
- list()

break

(PHP 4, PHP 5, PHP 7, PHP 8)

`break` finaliza a execução da estrutura `for`, `foreach`, `while`, `do-while` ou `switch` atual.

`break` aceita um argumento numérico opcional que diz quantas estruturas aninhadas deverão ser interrompidas. O valor padrão é `1`, somente a estrutura imediata é interrompida.

```php
<?php
$arr = array('um', 'dois', 'três', 'quatro', 'pare', 'cinco');
```

```php
foreach ($arr as $val) {
    if ($val == 'pare') {
        break;    /* Pode-se também usar 'break 1;' aqui. */
    }
    echo "$val\n";
}
```

O exemplo acima produzirá:

```
um
dois
três
quatro
```

Usando o argumento opcional:

```php
<?php
$i = 0;
while (++$i) {
    switch ($i) {
        case 5:
            echo "No 5\n";
            break 1;  /* Sai apenas do switch. */
        case 10:
            echo "No 10; saindo\n";
            break 2;  /* Sai do switch e do while. */
        default:
            break;
    }
    echo "$i\n";
}
```

O exemplo acima produzirá:

```
1
2
3
4
No 5
5
6
7
8
9
No 10; saindo
```

continue

(PHP 4, PHP 5, PHP 7, PHP 8)

`continue` é usado em estruturas de repetição para pular o restante da iteração atual do laço e continuar a execução na validação da condição e depois no início da próxima iteração.

> **Nota** :
>
> No PHP, a instrução switch é considerada uma estrutura de repetição para os propósitos de `continue`. `continue` se comporta como `break` (quando nenhum argumento é passado), mas emitirá um aviso, pois é provável que seja um erro. Se um `switch` estiver dentro de um laço, `continue 2` continuará na próxima iteração do laço externo.

`continue` aceita um argumento numérico opcional que informa quantos níveis de laços aninhados ele deve pular. O valor padrão é `1`, pulando assim para o final do laço atual.

```php
<?php
$array = ['zero', 'um', 'dois', 'três', 'quatro', 'cinco', 'seis'];
foreach ($array as $chave => $valor) {
    if (!($chave % 2)) { // pula membros com chaves pares
        continue;
    }
    echo $valor . "\n";
}
```

Os exemplos acima produzirão:

```
um
três
cinco
```

```php
<?php
$i = 0;
while ($i++ < 5) {
    echo "Externo\n";
    while (1) {
        echo "Meio\n";
        while (1) {
            echo "Interno\n";
            continue 3;
        }
        echo "Isso nunca é exibido.\n";
    }
    echo "Nem isso.\n";
}
```

Os exemplos acima produzirão:

```
Externo
Meio
Interno
Externo
Meio
Interno
Externo
Meio
Interno
Externo
Meio
Interno
Externo
Meio
Interno
```

Lista de modificações de `continue`

Versão	Descrição
7.3.0	`continue` em um `switch` que está tentando agir como uma declaração `break` para o `switch` gerará um `E_WARNING`.

switch

(PHP 4, PHP 5, PHP 7, PHP 8)

A declaração `switch` é similar a uma série de declarações `if` na mesma expressão. Em situações onde é necessário comparar as mesmas variáveis (ou expressões), com muitos valores diferentes e executar uma parte diferente de código dependendo de qual valor ela iguala, a declaração `switch` é frequentemente mais conveniente e mais legível.

> **Nota** :
>
> Note que ao contrário de outras linguagens, a declaração continue aplica-se ao `switch` e funciona de forma parecida com `break`. Se possuir um `switch` dentro de um laço, e deseja continuar na próxima iteração do laço externo, utilize `continue 2`.

> **Nota** :
>
> Note que o switch/case faz comparação relaxada.

No exemplo a seguir, cada bloco de código é equivalente. Um usa uma série de declarações `if` e `elseif`, e o outra usa uma declaração `switch`. A saída é a mesm a em ambos os casos.

Exemplo #1 Estrutura do `switch`

```php
<?php
$i = 1;

// Esta declaração switch:

switch ($i) {
    case 0:
        echo "i é igual a 0\n";
        break;
    case 1:
        echo "i é igual a 1\n";
        break;
    case 2:
        echo "i é igual a 2\n";
        break;
}

// É equivalente a:
if ($i == 0) {
    echo "i é igual a 0\n";
} elseif ($i == 1) {
    echo "i é igual a 1\n";
} elseif ($i == 2) {
    echo "i é igual a 2\n";
}
```

O exemplo acima produzirá:

```
i é igual a 1
i é igual a 1
```

É importante entender como a declaração `switch` é executada a fim de evitar enganos. A declaração `switch` executa linha por linha (na verdade, declaração por declaração). No início, nenhum código é executado. Somente quando uma declaração `case` é encontrada, cuja expressão avalia um valor que corresponde ao valor do `switch`, o PHP começará a executar a declaração. O PHP continuará a executar a declaração até o fim do bloco `switch`, ou até a primeira declaração `break` encontrada. Se não for escrita uma declaração `break` ao final da lista de declarações do case, o PHP continuará executando as declarações dos próximos cases. Por exemplo:

```php
<?php
$i = 1;

switch ($i) {
    case 0:
        echo "i é igual a 0\n";
    case 1:
        echo "i é igual a 1\n";
    case 2:
        echo "i é igual a 2\n";
}
```

O exemplo acima produzirá:

```
i é igual a 1
i é igual a 2
```

Neste exemplo, se a variável *$i* for igual a 0, o PHP executará todas as declarações echo! Se a variável *$i* for igual a 1, o PHP executará as duas últimas declarações echo. Somente se obterá o comportamento esperado ('i equals 2' será exibido), se *$i* for igual a 2. Sendo assim, é importante não esquecer as declarações `break` (mesmo quando se quer evitar, de propósito, sua omissão em certas circunstâncias).

Em uma declaração `switch`, a condição é avaliada somente uma vez, e o resultado é comparado a cada declaração `case`. Em uma declaração `elseif`. a condição é avaliada outra vez. Se a condição for mais complicada que uma simples comparação e/ou está em um laço pequeno, a declaração `switch` pode ser mais rápida.

A lista de declarações de um case também pode estar vazia, passando o controle da lista de declarações para o próximo case.

```php
<?php
$i = 1;

switch ($i) {
    case 0:
    case 1:
    case 2:
        echo "i é menor que 3 mas não é negativo";
        break;
    case 3:
        echo "i é 3";
}
```

O exemplo acima produzirá:

```
i é menor que 3 mas não é negativo
```

Um case especial é o `default`. Este case corresponde a tudo que não foi correspondido pelos outros cases. Por exemplo:

```php
<?php
$i = 4;

switch ($i) {
    case 0:
        echo "i é igual a 0";
        break;
```

```
    case 1:
        echo "i é igual a 1";
        break;
    case 2:
        echo "i é igual a 2";
        break;
    default:
       echo "i não é igual a 0, 1 ou 2";
}
```

O exemplo acima produzirá:

```
i não é igual a 0, 1 ou 2
```

> **Nota** :
>
> Múltiplos casos padrões irão emitir um erro `E_COMPILE_ERROR`.

> **Nota** :
>
> Tecnicamente o caso `default` pode ser listado em qualquer ordem. Ele será usado apenas se nenhum outro caso corresponder. No entanto, por conversão, é melhor inseri-lo no fim como a última ramificação.

Se nenhuma ramificação `case` corresponder, e não existe nenhuma ramificação `default`, então nenhum código será executado, assim como se nenhuma declaração `if` fosse verdadeira.

Um valor de caso pode ser dado como uma expressão. No entanto, essa expressão será avaliada de forma isolada, e então será frouxamente comparada com o valor do switch. Isso significa que ela não pode ser usada para avaliações complexas do valor do switch. Por exemplo:

```
<?php
$alvo = 1;
$inicio = 3;

switch ($alvo) {
    case $inicio - 1:
        print "A";
        break;
    case $inicio - 2:
        print "B";
        break;
    case $inicio - 3:
        print "C";
        break;
    case $inicio - 4:
        print "D";
```

```
        break;
}
```

O exemplo acima produzirá:

```
B
```

Para comparações mais complexas, o valor `true` pode ser usado como o valor do switch. Alternativamente, blocos`if-else` podem ser usado no lugar de `switch`.

```php
<?php
$deslocamento = 1;
$inicio = 3;

switch (true) {
    case $inicio - $deslocamento === 1:
        print "A";
        break;
    case $inicio - $deslocamento === 2:
        print "B";
        break;
    case $inicio - $deslocamento === 3:
        print "C";
        break;
    case $inicio - $deslocamento === 4:
        print "D";
        break;
}
```

O exemplo acima produzirá:

```
B
```

A sintaxe alternativa das estruturas de controle é suportada por switches. Para mais informações, veja Sintaxe alternativa para estruturas de controle.

```php
<?php
$i = 1;

switch ($i):
    case 0:
        echo "i é igual a 0";
        break;
    case 1:
        echo "i é igual a 1";
        break;
```

```
    case 2:
        echo "i é igual a 2";
        break;
    default:
        echo "i não é igual a 0, 1 ou 2";
endswitch;
```

O exemplo acima produzirá:

```
i é igual a 1
```

É possível utilizar o ponto e vírgula ao invés dos dois pontos depois de um 'case'. A partir do PHP 8.5.0, esta sintaxe foi descontinuada.

```
<?php
$cerveja = 'guaraná';

switch($cerveja)
{
    case 'heineken';
    case 'budweiser';
    case 'stella';
    case 'artesanal';
        echo 'Boa escolha';
        break;
    default;
        echo 'Por favor faça uma nova seleção...';
        break;
}
```

O exemplo acima produzirá:

```
Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 6

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 7

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 8

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 9

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 12
Por favor faça uma nova seleção...
```

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.5.0. O uso deste recurso é fortemente desencorajado.

Registro de Alterações

Versão	Descrição
8.5.0	A sintaxe alternativa que usa um ponto-e-vírgula depois de `case` foi descontinuada.

Veja também

- match

match

(PHP 8)

A expressão `match` ramifica a avaliação baseada em uma verificação de identidade de um valor. Semelhante a uma declaração `switch`, uma expressão `match` possui uma expressão sujeito que é comparada com múltiplas alternativas. Ao contrário da `switch`, ela irá avaliar para um valor assim como as expressões ternárias. Diferente da `switch`, a comparação é uma verificação de identidade (`===`) em vez de uma comparação de igualdade fraca (`==`). Expressões match estão disponíveis a partir do PHP 8.0.0.

Exemplo #1 Estrutura de uma expressão `match`

```php
<?php
$valor_de_retorno = match (expressao_sujeito) {
    expressao_condicional_unica => expressao_de_retorno,
    expressao_condicional1, expressao_condicional2 => expressao_de_retorno,
};
```

Exemplo #2 Uso básico de `match`

```php
<?php
$comida = 'bolo';

$valor_de_retorno = match ($comida) {
    'apple' => 'Essa comida é uma maçã',
    'bar' => 'Essa comida é um bar',
    'bolo' => 'Essa comida é um bolo',
};

var_dump($valor_de_retorno);
```

O exemplo acima produzirá:

```
string(19) "Essa comida é um bolo"
```

Exemplo #3 Exemplo de uso de `match` com operadores de comparação

```php
<?php
$age = 18;

$output = match (true) {
    $age < 2 => "Bebê",
    $age < 13 => "Criança",
    $age <= 19 => "Adolescente",
    $age >= 40 => "Adulto"
    $age > 19 => "Adulto Jovem"
};

var_dump($output);
```

O exemplo acima produzirá:

```
string(11) "Adolescente"
```

> **Nota** :
>
> O resultado de uma expressão `match` não precisa ser usado.

> **Nota** :
>
> Quando uma expressão `match` é usada como uma expressão única, ela *precisa* ser terminada com um ponto-e-vírgula `;`.

A expressão `match` é similar a uma declaração `switch` mas tem algumas diferenças importantes:

- Um braço `match` compara valores estritamente (`===`) em vez de frouxamente como a declaração switch faz.
- Uma expressão `match` retorna um valor.
- Braços `match` não seguem para casos posteriores da maneira que declarações `switch` seguem.
- Uma expressão `match` deve ser exaustiva.

Assim como as declarações `switch`, expressões `match` são executadas a cada ramo de correspondência. No início, nenhum código é executado. As expressões condicionais só são avaliadas se todas as expressões condicionais anteriores não corresponderem à expressão sujeito. Apenas a expressão de retorno pertencente à expressão condicional de correspondência será avaliada. Por exemplo:

```php
<?php
$resultado = match ($x) {
    foo() => 'value',
    $this->bar() => 'value', // $this->bar() não é chamado se foo() === $x
    $this->baz => beep(), // beep() não é chamado a não ser que $x === $this->baz
    // etc.
};
```

Braços da expressão `match` podem conter múltiplas expressões separadas por uma vírgula. Isso é um OR lógico, e é uma forma abreviada para múltiplos braços de correspondência com o mesmo lado direito.

```php
<?php
$resultado = match ($x) {
    // Este braço de correspondência:
    $a, $b, $c => 5,
    // É equivalente a estes três braços de correspondência:
    $a => 5,
    $b => 5,
    $c => 5,
};
```

Um caso especial é o padrão `default`. Esse padrão atende a qualquer coisa que não tenha sido correspondida anteriormente. Por exemplo:

```php
<?php
$resultadoDaExpressao = match ($condicao) {
    1, 2 => foo(),
    3, 4 => bar(),
    default => baz(),
};
```

> **Nota** :
>
> Múltiplos padrões 'default' irão emitir um erro **`E_FATAL_ERROR`**.

Uma expressão `match` deve ser exaustiva. Se a expressão sujeito não for tratada por nenhum braço de correspondência um UnhandledMatchError é lançado.

Exemplo #4 Exemplo de uma expressão match não tratada

```php
<?php
$condicao = 5;

try {
    match ($condicao) {
        1, 2 => foo(),
        3, 4 => bar(),
    };
} catch (\UnhandledMatchError $e) {
    var_dump($e);
}
```

O exemplo acima produzirá:

```
object(UnhandledMatchError)#1 (7) {
  ["message":protected]=>
  string(22) "Unhandled match case 5"
  ["string":"Error":private]=>
  string(0) ""
  ["code":protected]=>
  int(0)
  ["file":protected]=>
  string(6) "script"
  ["line":protected]=>
  int(5)
  ["trace":"Error":private]=>
  array(0) {
  }
  ["previous":"Error":private]=>
  NULL
}
```

Usando expressões match para tratar verificações sem identidade

É possível usar uma expressão `match` para tratar casos condicionais sem identidade usando `true` como a expressão sujeito.

Exemplo #5 Usando uma expressão match generalizada para ramificar com base em intervalos de inteiros

```php
<?php
$idade = 23;

$resultado = match (true) {
    $idade >= 65 => 'senhor',
    $idade >= 25 => 'adulto',
    $idade >= 18 => 'adolescente',
    default => 'criança',
};

var_dump($resultado);
```

O exemplo acima produzirá:

```
string(11) "adolescente"
```

Exemplo #6 Usando uma expressão match generalizada para ramificar com base no conteúdo de strings

```php
<?php

$texto = 'Bienvenue chez nous';

$resultado = match (true) {
    str_contains($text, 'Welcome'), str_contains($text, 'Hello') => 'en',
    str_contains($text, 'Bienvenue'), str_contains($text, 'Bonjour') => 'fr',
    // ...
};

var_dump($resultado);
```

O exemplo acima produzirá:

```
string(2) "fr"
```

declare

(PHP 4, PHP 5, PHP 7, PHP 8)

A instrução `declare` é usada para definir diretivas de execução para um bloco de código. A sintaxe do `declare` é similar à sintaxe de outros construtos de controle de fluxo:

```
declare (directive)
    statement
```

A seção `directive` permite que o comportamento do bloco do `declare` seja definido. Atualmente somente três diretivas são reconhecidas:

- `ticks`
- `encoding`
- `strict_types`

As diretivas são manipuladas na compilação do arquivo, somente literais podem ser usadas como valores da diretiva. Variáveis e constantes não podem ser usadas. Para ilustrar:

```php
<?php
// Isto é válido:
declare(ticks=1);

// isto é inválido:
const TICK_VALUE = 1;
declare(ticks=TICK_VALUE);
```

A parte `statement` do bloco do `declare` será executada - como é executada e quais os efeitos colaterais que ocorrem durante a execução depende da diretiva definida no bloco `directive`.

O construtor do `declare` também pode ser usado no escopo global, afetando todo o código que o segue (entretanto se o arquivo com o `declare` for incluído, não afetará o arquivo principal).

```php
<?php
// Os exemplos abaixo tem o mesmo efeito

// Pode ser usado assim:
declare(ticks=1) {
    // script aqui
}

// ou assim:
declare(ticks=1);
// script inteiro aqui
```

Tiques

Um tique é um evento que ocorre a cada *N* declarações de baixo nível "ticáveis" executadas pelo analisador dentro do bloco `declare`. O valor de *N* é especificado usando `ticks=N` dentro do bloco `declare` da seção `directive`.

Nem todas declarações são "ticáveis". Tipicamente, expressões de condições e expressões de argumentos não são.

Os eventos que ocorrem em cada tique são especificados usando a função register_tick_function(). Veja o exemplo abaixo para mais detalhes. Note que mais de um evento pode ocorrer para cada tique.

Exemplo #1 Exemplo de uso do tique

```php
<?php

declare(ticks=1);

// A função a ser chamada em cada evento de tique
function gerenciador_de_tiques()
{
    echo "função gerenciador_de_tiques() chamada\n";
}

register_tick_function('gerenciador_de_tiques'); // Causa um evento de tique

$a = 1; // Causa um evento de tique

if ($a > 0) {
    $a += 2; // Causa um evento de tique
    print $a . "\n"; // Causa um evento de tique
}
```

O exemplo acima produzirá:

```
função gerenciador_de_tiques() chamada
função gerenciador_de_tiques() chamada
função gerenciador_de_tiques() chamada
3
função gerenciador_de_tiques() chamada
```

Consulte também register_tick_function() e unregister_tick_function().

Codificação

A codificação de um script pode ser especificada em cada arquivo com uma diretiva `encoding`.

Exemplo #2 Declarando a codificação do script

```
<?php
declare(encoding='ISO-8859-1');
// código aqui
```

> **Cuidado**
>
> Quando usada com namespaces, a única sintaxe válida para declarar é `declare(encoding='...');` onde `...` é o valor da codificação. `declare(encoding='...') {}` irá resultar em um erro de sintaxe quando usada com namespaces.

Veja mais zend.script_encoding.

return

(PHP 4, PHP 5, PHP 7, PHP 8)

A declaração `return` retorna o controle do programa para o módulo que o chamou. A execução continuará na expressão seguinte à invocação do módulo.

Se chamada dentro de uma função, a declaração `return` terminará imediatamente sua execução, e retornará seus argumentos como valor à chamada da função. A declaração `return` também termina a execução de uma declaração eval() ou um arquivo de script.

Se chamada no escopo global, a execução do script atual é finalizada. Se o arquivo de script atual for incluído ou requerido com as funções include ou require, o controle é passado de volta ao script que está chamando. Além disso, se o script atual foi incluído com a função include, o valor informado ao `return` será retornado como o valor da chamada de include. Se um `return` for chamado de dentro do script principal, sua execução será finalizada. Se o script atual for mencionado nas opções de configuração auto_prepend_file ou auto_append_file *php.ini*, a execução do script será finalizada.

Para mais informações, consulte a seção sobre Retornando Valores.

> **Nota** :
>
> Observe que, como `return` é um construtor de linguagem e não uma função, os parênteses que envolvem seus argumentos não são exigidos e seu uso é desencorajado.

> **Nota** :

> Se nenhum parâmetro for informado, e os parênteses omitidos, `null` será retornado. Chamar um `return` com parênteses mas sem argumentos resultará em um erro de interpretação.

A partir do PHP 7.1.0, declarações 'return' sem um argumento em funções que declaram um tipo de retorno acionam `E_COMPILE_ERROR`, a não ser que o tipo de retorno seja void, nesse caso declarações 'return' com um argumento acionam esse erro.

require

(PHP 4, PHP 5, PHP 7, PHP 8)

A declaração `require` é idêntica à include exceto que em caso de falha também produzirá uma exceção Error (erro de nível `E_COMPILE_ERROR` antes do PHP 8.0.0) enquanto que include apenas emitirá um alerta (nível de erro `E_WARNING`).

Veja a documentação include para saber como isso funciona.

include

(PHP 4, PHP 5, PHP 7, PHP 8)

A expressão `include` inclui e avalia o arquivo informado.

A documentação a seguir também se aplica à declaração require.

Os arquivos são incluídos baseando-se no caminho do arquivo informado ou, se não informado, o include_path especificado. Se o arquivo não for encontrado no include_path, `include` verificará no diretório do script que o executa e no diretório de trabalho corrente antes de falhar. O construtor `include` emitirá um `E_WARNING` se não localizar o arquivo; possui um comportamento diferente do require, que emitirá um `E_ERROR`.

Note que ambas `include` e `require` emitem `E_WARNING`s adicionais, se o arquivo não puder ser acessado, antes de emitir o `E_WARNING` ou `E_ERROR` final, respectivamente.

Se um caminho for definido — seja absoluto (iniciando com a letra da unidade ou `\` no Windows, ou `/` no Unix/Linux), ou relativo ao diretório atual (começando com `.` ou `..`) — o include_path será completamente ignorado. Por exemplo, se o nome do arquivo iniciar com `../`, o interpretador irá procurar pelo arquivo no diretório pai.

Para mais informações de como o PHP trabalha ao incluir arquivos e o caminho de inclusão, veja a documentação de include_path.

Quando um arquivo é incluído, o código herda o escopo de variáveis da da linha que a inclusão ocorrer. Qualquer variável disponível no arquivo que o incluiu estará disponível no arquivo incluído, daquela linha em diante. Entretanto, todas as funções e classes definidas no arquivo incluído estarão no escopo global.

Exemplo #1 Exemplo básico do construtor `include`

```
vars.php
<?php

$color = 'green';
$fruit = 'apple';
```

```
?>

test.php
<?php

echo "A $color $fruit"; // A

include 'vars.php';

echo "A $color $fruit"; // A green apple

?>
```

Se a inclusão ocorrer dentro de uma função, todo o código contido no arquivo incluído irá se comportar como se tivesse sido definido dentro da função. Portanto seguirá o escopo de variáveis da função. Uma exceção para essa regra são as constantes mágicas que são interpretadas antes da inclusão ocorrer.

Exemplo #2 Inclusão em funções

```
<?php

function foo()
{
    global $color;

    include 'vars.php';

    echo "A $color $fruit";
}

/* vars.php está no escopo de foo() por isso *
* $fruit NÃO está disponível fora deste      *
* escopo. $color está, porque foi declarada  *
* como global.                               */

foo();                    // A green apple
echo "A $color $fruit";   // A green
```

Quando um arquivo é incluído, o interpretador sai do modo PHP e entra no modo HTML no começo do arquivo incluído, e volta novamente no final. Por esta razão, qualquer código dentro do arquivo incluído que necessite ser executado como PHP deve estar entre tags válidas de início e fim do PHP.

Se os "empacotadores de inclusão de URL" estiverem habilitados no PHP, pode-se definir um arquivo a ser incluído usando uma URL (via HTTP ou outro protocolo suportado - veja Protocolos e empacotadores suportados para uma lista de protocolos) ao invés de um caminho local. Se o servidor de destino interpretar o arquivo incluído como PHP, variáveis podem ser passadas para o arquivo incluído usando uma URL de requisição com HTTP GET. Isso não significa a mesma coisa que incluir e compartilhar o escopo de variáveis com o arquivo incluído; o script é executado no servidor remoto e o resultado é incluído no script local.

Exemplo #3 Usando `include` via HTTP

```
<?php

/* Este exemplo assume que www.exemplo.com.br está configurado para processar
* arquivos .php e não arquivos .txt. Além disso, 'Funciona' aqui significa
```

```
* que as variáveis $foo e $bar estão disponíveis dentro do arquivo incluído. */

// Não vai funcionar; arquivo.txt não foi tratado por www.exemplo.com.br como PHP
include 'http://www.exemplo.com.br/arquivo.txt?foo=1&bar=2';

// Não vai funcionar; procura por um arquivo chamado 'arquivo.php?foo=1&bar=2' no
// sistema de arquivos local.
include 'arquivo.php?foo=1&bar=2';

// Funciona.
include 'http://www.exemplo.com.br/arquivo.php?foo=1&bar=2';
```

> **Aviso**
>
> ### Avisos de Segurança
>
> Um arquivo remoto pode ser processado pelo servidor remoto (dependendo da extensão do arquivo e se o servidor remoto executa, ou não,arquivos PHP), mas ainda precisa produzir um código PHP válido pois será processado pelo servidor local. Se o arquivo do servidor remoto deve ser processado como um texto simples, a função readfile() é uma opção muito melhor a ser usada. Caso contrário, deve-se ter um cuidado especial para garantir que o servidor remoto produza um código PHP válido.

Veja também Arquivos Remotos, fopen() e file() para informações relacionadas.

Manipulando retornos: `include` retorna **`false`** ao falhar e emite um aviso. Inclusões bem sucedidas, ao menos que seja sobrescritas pelo arquivo incluído, retornam `1`. É possível utilizar a declaração return dentro do arquivo incluído para finalizar o processamento e retornar para o arquivo que o incluiu. Além disso, é possível retornar valor a partir do arquivo incluído. Pode-se usar o valor do arquivo incluído como em uma função normal. Isso não é, entretanto, possível quando incluímos arquivos remotos a menos que a saída do arquivo remoto tenha tags válidas de início e fim do PHP (como em um arquivo local). Pode-se declarar as variáveis necessárias dentro dessas tags e estarão disponíveis em qualquer ponto do arquivo que o incluiu.

Pelo fato do `include` ser um construtor especial da linguagem, parênteses não são necessários ao redor do argumento. Tome cuidado ao comparar com o valor de retorno.

Exemplo #4 Comparando valores de retorno do include

```
<?php
// não vai funcionar, avaliado como include(('vars.php') == TRUE), isto é: include('1')
if (include('vars.php') == TRUE) {
    echo 'OK';
}

// funciona
if ((include 'vars.php') == TRUE) {
    echo 'OK';
}
```

Exemplo #5 `include` e a instrução return

```
return.php
<?php

$var = 'PHP';
```

```php
return $var;

?>
```

noreturn.php

```php
<?php

$var = 'PHP';

?>
```

testreturns.php

```php
<?php

$foo = include 'return.php';

echo $foo; // imprime 'PHP'

$bar = include 'noreturn.php';

echo $bar; // imprime 1

?>
```

`$bar` tem o valor `1` porque a inclusão foi bem sucedida. Note a diferença entre os dois exemplos. O primeiro usa return, enquanto o outro não. Se o arquivo não puder ser incluído, **`false`** é retornado e um **`E_WARNING`** emitido.

Se funções forem definidas no arquivo incluído, elas podem ser utilizadas no arquivo principal independente se estão antes ou depois do return. Se o arquivo for incluído duas vezes, o PHP emitirá um erro fatal, pois as funções já foram declaradas. É recomendado usar o include_once ao invés de verificar se o arquivo já foi incluído manualmente dentro do arquivo incluído.

Outra forma de "incluir" um arquivo PHP em uma variável é capturar a saída usando as Funções de Controle de Saída com o `include`. Por exemplo:

Exemplo #6 Usando buffer de saída para incluir um arquivo PHP em uma string

```php
<?php
$string = get_include_contents('somefile.php');

function get_include_contents($filename) {
    if (is_file($filename)) {
        ob_start();
        include $filename;
        return ob_get_clean();
    }
    return false;
}
```

Para automatizar a inclusão de arquivos nos scripts, veja também as configurações auto_prepend_file e auto_append_file contidas no *php.ini*.

> **Nota** :

> Como esta é uma construção da linguagem e não uma função, ela não pode ser chamada usando funções variáveis ou argumentos nomeados.

Veja também require, require_once, include_once, get_included_files(), readfile(), virtual(), e include_path.

require_once

(PHP 4, PHP 5, PHP 7, PHP 8)

A expressão `require_once` é idêntica à require, exceto que o PHP verificará se o arquivo já foi incluído, e em caso afirmativo, não o incluirá (exigirá) novamente.

Veja a documentação include_once para informação sobre o comportamento `_once`, e como ele difere de seus irmãos `_once`.

include_once

(PHP 4, PHP 5, PHP 7, PHP 8)

A expressão `include_once` inclui e avalia o arquivo informado durante a execução do script. Este comportamento é similar à expressão include, com a única diferença que, se o código do arquivo já foi incluído, não o fará novamente, e a include_once retornará **`true`**. Como o nome sugere, o arquivo será incluído somente uma vez.

A `include_once` pode ser utilizada em casos em que o mesmo arquivo pode ser incluído e validado mais de uma vez durante uma execução de um script em particular, neste caso, ajudará a evitar problemas como redefinição de funções, reatribuição de valores de variáveis, etc.

Consulte a documentação da expressão include para informações sobre como ela funciona.

goto

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

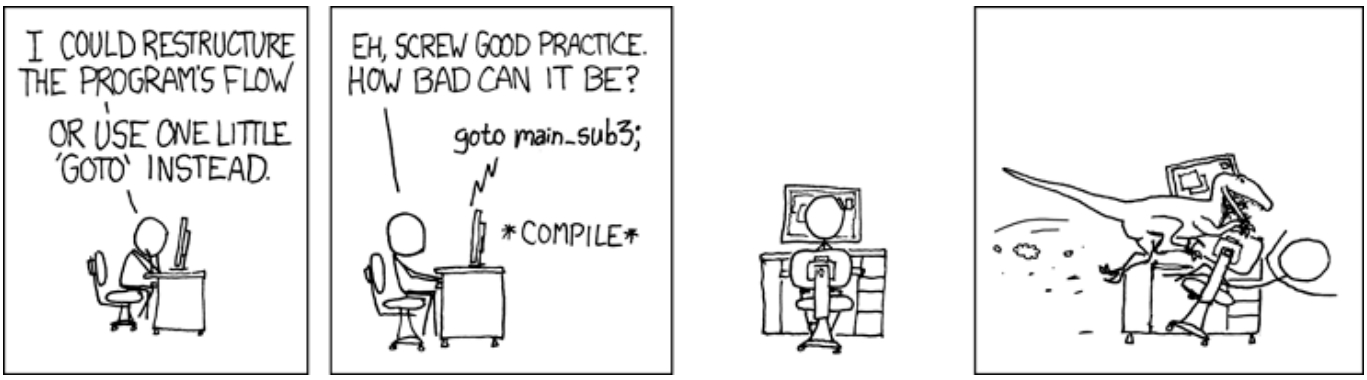


Imagem cortesia do » xkcd

O operador `goto` pode ser usado para pular para outra seção do programa. O ponto de destino é definido por um rótulo *sensível a maiúsculas e minúsculas* seguido de dois pontos, e a instrução é usada como `goto` seguida do rótulo de destino desejado. O uso do `goto` não é completamente irrestrito. O rótulo de destino deve estar no mesmo arquivo e contexto, significando que não se pode pular para fora ou para dentro de

uma função ou método. Também não pode-se saltar para dentro de um laço ou estrutura switch. Pode-se saltar para fora deles, e um uso comum é usar o `goto` no lugar de um `break` multinível.

Exemplo #1 Exemplo da estrutura de controle `goto`

```php
<?php
goto a;
echo 'Foo';

a:
echo 'Bar';
```

O exemplo acima produzirá:

```
Bar
```

Exemplo #2 Exemplo da estrutura de controle `goto` em um laço

```php
<?php
for ($i = 0, $j = 50; $i < 100; $i++) {
    while ($j--) {
        if ($j == 17) {
            goto end;
        }
    }
}
echo "i = $i";
end:
echo 'j chegou em 17';
```

O exemplo acima produzirá:

```
j chegou em 17
```

Exemplo #3 Isto não irá funcionar

```php
<?php
goto loop;
for ($i = 0, $j = 50; $i < 100; $i++) {
    while ($j--) {
        loop:
    }
}
echo "$i = $i";
```

O exemplo acima produzirá algo semelhante a:

```
Fatal error: 'goto' into loop or switch statement is disallowed in script on line 2
```

Funções

Índice

Funções definidas pelo usuário

Uma função é definida pelo uso da palavra-chave `function`, um nome, uma lista de parâmetros (que pode estar vazia) separados por vírgulas (`,`) entre parênteses, seguidos pelo corpo da função entre chaves, como a seguir:

Exemplo #1 Declarando uma nova função chamada `foo`

```php
<?php
function foo($arg_1, $arg_2, /* ..., */ $arg_n)
{
    echo "Exemplo de função.\n";
    return $valor_retornado;
}
```

> **Nota** :
>
> A partir do PHP 8.0.0, a lista de parâmetros pode ter uma vírgula no final:
>
> ```php
> <?php
> function foo($arg_1, $arg_2,) { }
> ```

Qualquer código PHP válido pode aparecer dentro do corpo de uma função, mesmo outras funções e definições de classes.

Nomes de funções seguem as mesmas regras que outros nomes no PHP. Um nome de função válido começa com uma letra ou um sublinhado, seguido por qualquer número de letras, números ou sublinhados. Com uma expressão regular, seria expressado com: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`.

> **Dica**
>
> Veja também o Guia de nomenclatura em espaço de usuário.

As funções não precisam ser criadas antes de serem referenciadas, *exceto* quando uma função é condicionalmente definida como mostrado nos dois exemplos abaixo.

Quando uma função é definida condicionalmente como nos dois exemplos abaixo, sua definição precisa ser processada *antes* de ser chamada.

Exemplo #2 Funções definidas condicionalmente

```php
<?php
$makefoo = true;

/* Não podemos chamar foo() daqui
   porque ela ainda não existe,
   mas podemos chamar bar() */
if (function_exists('foo')) {
    foo();
} else {
    echo "Função foo não existe (ainda).\n";
}

bar();

if ($makefoo) {
  function foo()
  {
    echo "Eu não existo até que o programa passe por aqui.\n";
  }
}

/* Agora podemos chamar foo()
   porque $makefoo foi avaliado como true */

if ($makefoo) foo();

function bar()
{
  echo "Eu existo imediatamente desde o programa começar.\n";
}
```

Exemplo #3 Funções dentro de funções

```php
<?php
function foo()
{
  function bar()
  {
    echo "Eu não existo até foo() ser chamada.\n";
  }
}

/* Nós não podemos chamar bar() ainda
   porque ela ainda não foi definida. */
if (function_exists('bar')) {
    bar();
}

foo();

/* Agora nós podemos chamar bar(),
   porque o processamento de foo()
   tornou a primeira acessível */
```

```
bar();
```

Todas as funções e classes no PHP tem escopo global - elas podem ser chamadas fora de uma função mesmo que tenham sido definidas dentro e vice-versa.

O PHP não suporta sobrecarga de funções, e também não é possível cancelar ou alterar a definição de funções previamente declaradas.

> **Nota** :
>
> Nomes de funções são case-insensitive para caracteres ASCII A até Z, mas é melhor chamar as funções da mesma forma que elas aparecem nas declarações.

Ambos número variável de argumentos e argumentos padrões são suportados em funções. veja também as referências das funções func_num_args(), func_get_arg() e func_get_args() para mais informações.

É possível chamar funções recursivas no PHP.

Exemplo #4 Funções Recursivas

```
<?php
function recursion($a)
{
    if ($a < 20) {
        echo "$a\n";
        recursion($a + 1);
    }
}

recursion(17);
```

> **Nota** :
>
> Chamadas recursivas a funções/métodos acima de 100-200 níveis podem exaurir a pilha e causar o término do script. Especificamente, recursão infinita é considerada um erro de programação.

Parâmetros e argumentos de funções

Os parâmetros de uma função são declarados em sua assinatura. Informações podem ser passadas para funções através da lista de argumentos, que é uma lista de expressões delimitadas por vírgulas. Os argumentos são avaliados da esquerda para a direita, e o resultado é atribuído aos parâmetros da função, antes que a função seja efetivamente chamada (avaliação *ansiosa*).

O PHP suporta a passagem de argumentos por valor (o padrão), passagem por referência e valores padrões de argumentos. Lista de argumentos de tamanho variável e argumentos nomeados também são suportados.

> **Nota** :
>
> A partir do PHP 7.3.0, é possível ter uma vírgula no final da lista de argumentos para chamadas de função:

```php
<?php
$v = foo(
    $arg_1,
    $arg_2,
);
```

A partir do PHP 8.0.0, a lista de parâmetros de uma função pode incluir uma vírgula final, que será ignorada. Isto é particularmente útil nos casos em que a lista de parâmetros é longa ou contém nomes de variáveis longos, tornando conveniente listar os parâmetros verticalmente.

Exemplo #1 Parâmetros de função com uma vírgula final

```php
<?php
function takes_many_args(
    $primeiro,
    $segundo,
    $uma_variavel_com_nome_longo,
    $parametro_com_default = 5,
    $de_novo = 'argumento padrão', // Essa vírgula final não era permitida antes do PHP 8.0.0.
)
{
    // ...
}
```

Passando argumentos por referência

Por padrão, argumentos de função são passados por valor (de forma que se você mudar o valor do parâmetro dentro da função, ele não é alterado fora da função). Para permitir que uma função modifique os seus argumentos, eles devem ser passados por referência.

Para ter um argumento para uma função sempre passado por referência, adicione antes dele um "e comercial" (&) ao nome do parâmetro na definição da função:

Exemplo #2 Passando argumentos de função por referência

```php
<?php
function add_some_extra(&$string)
{
    $string .= ' e alguma coisa mais.';
}
$str = 'Isto é uma string,';
add_some_extra($str);
echo $str;    // exibe 'Isto é uma string, e alguma coisa mais.'
```

É um erro passar uma expressão constante como um argumento para um parâmetro que espera ser passado por referência.

Valores padrão de parâmetros

Uma função pode definir valores padrão para parâmetros usando sintaxe similar à atribuição de uma variável. O padrão é usado apenas quando o argumento do parâmetro não é passado. Observe que passar `null` *não* atribui o valor padrão.

Exemplo #3 Utilizando parâmetros padrão em funções

```php
<?php
function makecoffee($type = "cappuccino")
{
    return "Fazendo uma xícara de café $type.\n";
}
echo makecoffee();
echo makecoffee(null);
echo makecoffee("espresso");
```

O exemplo acima produzirá:

```
Fazendo uma xícara de café cappuccino.
Fazendo uma xícara de café.
Fazendo uma xícara de café espresso.
```

Valores padrões de parâmetro podem ser valores escalares, arrays, o tipo especial `null` e, a partir do PHP 8.1.0, objetos usando a sintaxe new ClassName().

Exemplo #4 Usando tipos não escalares como valores padrões

```php
<?php
function fazercafe($tipos = array("cappuccino"), $cafeteira = NULL)
{
    $dispositivo = is_null($cafeteira) ? "as mãos" : $cafeteira;
    return "Fazendo uma xícara de ".join(", ", $tipos)." com $dispositivo.\n";
}
echo fazercafe();
echo fazercafe(array("cappuccino", "lavazza"), "chaleira");
```

O exemplo acima produzirá:

```
Fazendo uma xícara de cappuccino com as mãos.
Fazendo uma xícara de cappuccino, lavazza com chaleira.
```

Exemplo #5 Usando objetos como valores padrão (a partir do PHP 8.1.0)

```php
<?php
class CafeteiraPadrao {
    public function preparar() {
        return 'Fazendo café.\n';
    }
}
class CafeteiraChique {
    public function preparar() {
        return 'Preparando um belo café só para você.\n';
    }
```

```
}
function fazercafe($cafeteira = new CafeteiraPadrao)
{
    return $cafeteira->preparar();
}
echo fazercafe();
echo fazercafe(new CafeteiraChique);
```

O exemplo acima produzirá:

```
Fazendo café.
Preparando um belo café só para você.
```

O valor padrão precisa ser uma expressão constante, não (por exemplo) uma variável, um membro de classe ou uma chamada de função.

Observe que quaisquer parâmetros opcionais devem ser especificados após os parâmetros obrigatórios, caso contrário, eles não podem ser omitidos das chamadas. Considere o seguinte exemplo:

Exemplo #6 Uso incorreto de parâmetros padrão de função

```
<?php
function faziogurte($recipiente = "tigela", $sabor)
{
    return "Fazendo um(a) $recipiente de iogurte de $sabor.\n";
}

echo faziogurte("framboesa"); // "framboesa" é $recipiente, não $sabor
```

O exemplo acima produzirá:

```
Deprecated: faziogurte(): Optional parameter $recipiente declared before required parameter
$sabor is implicitly treated as a required parameter in script on line 2

Fatal error: Uncaught ArgumentCountError: Too few arguments to function faziogurte(), 1 passed
in script on line 7 and exactly 2 expected in script:2
Stack trace:
#0 script(7): faziogurte('framboesa')
#1 {main}
  thrown in script on line 2
```

Agora, compare o que está acima com este:

Exemplo #7 Uso correto de parâmetros padrão de função

```php
<?php
function fazeriogurte($sabor, $recipiente = "tigela")
{
    return "Fazendo um(a) $recipiente de iogurte de $sabor.\n";
}

echo fazeriogurte("framboesa"); // "framboesa" é $sabor
```

O exemplo acima produzirá:

```
Fazendo um(a) tigela de iogurte de framboesa.
```

A partir do PHP 8.0.0, argumentos nomeados podem ser usados para pular vários parâmetros opcionais.

Exemplo #8 Uso correto de parâmetros padrão de função

```php
<?php
function fazeriogurte($recipiente = "tigela", $sabor = "framboesa", $estilo = "Grego")
{
    return "Fazendo um(a) $recipiente de iogurte $estilo de $sabor.\n";
}

echo fazeriogurte(estilo: "natural");
```

O exemplo acima produzirá:

```
Fazendo um(a) tigela de iogurte natural de framboesa.
```

A partir do PHP 8.0.0, declarar parâmetros obrigatórios após parâmetros opcionais está *descontinuado*. Isso geralmente pode ser resolvido descartando o valor padrão, pois nunca será usado. Uma exceção a essa regra são parâmetros no formato `Type $param = null`, onde o padrão `null` torna o tipo implicitamente anulável. Este uso foi descontinuado a partir do PHP 8.4.0, e um tipo anulável explícito deve ser usado.

Exemplo #9 Declarando parâmetros opcionais após parâmetros obrigatórios

```php
<?php
function foo($a = [], $b) {}     // Padrão não utilizado; descontinuado a partir do PHP 8.0.0
function foo($a, $b) {}          // Funcionalmente equivalente, sem aviso de descontinuação

function bar(A $a = null, $b) {} // A partir do PHP 8.1.0, $a é implicitamente requerido
                                 // (porque vem antes do parâmetro requerido),
                                 // porém implicitamente anulável (descontinuado a partir do PHP
8.4.0),
                                 // porque o valor padrão do parâmetro é null
function bar(?A $a, $b) {}       // Recomendado
```

> **Nota** :
>
> A partir do PHP 7.1.0, omitir um parâmetro que não especifica um padrão lança um ArgumentCountError; em versões anteriores isso emitia um Aviso.

> **Nota** :
>
> Parâmetros que esperam o argumento por referência podem ter um valor padrão.

Número variável de argumentos

O PHP suporta argumentos em quantidade variável em funções definidas pelo usuário, utilizando o token `...`.

Listas de parâmetros podem conter o token `...` para indicar que a função aceita um número variável de argumentos. Os argumentos serão passados na variável informada como um array:

Exemplo #10 Utilizando `...` para acessar argumentos variáveis

```php
<?php
function sum(...$numbers) {
    $acc = 0;
    foreach ($numbers as $n) {
        $acc += $n;
    }
    return $acc;
}

echo sum(1, 2, 3, 4);
```

O exemplo acima produzirá:

```
10
```

Você também pode utilizar `...` quando chamando funções para transformar uma variável array, Traversable ou literal em uma lista de argumentos.

Exemplo #11 Utilizando `...` para fornecer argumentos

```php
<?php
function add($a, $b) {
    return $a + $b;
}

echo add(...[1, 2])."\n";

$a = [1, 2];
echo add(...$a);
```

O exemplo acima produzirá:

```
3
3
```

Podem ser especificados parâmetros posicionais antes do indicador. `...`. Nesse caso comente os argumentos finais, que não pareiam com um argumento posicional, serão adicionados ao array gerado por `...`.

É também possível adicionar uma dica de tipo antes do indicador `...`. Se presente, todos os argumentos capturados por `...` deverão conformar com o tipo do parâmetro.

Exemplo #12 Argumentos variáveis com dica de tipo

```php
<?php
function total_intervals($unit, DateInterval ...$intervals) {
    $time = 0;
    foreach ($intervals as $interval) {
        $time += $interval->$unit;
    }
    return $time;
}

$a = new DateInterval('P1D');
$b = new DateInterval('P2D');
echo total_intervals('d', $a, $b).' days';

// Isto falhará, já que null não é um objeto DateInterval.
echo total_intervals('d', null);
```

O exemplo acima produzirá:

```
3 days
Fatal error: Uncaught TypeError: total_intervals(): Argument #2 must be of type DateInterval,
null given, called in script on line 15 and defined in script:2
Stack trace:
#0 script(15): total_intervals('d', NULL)
#1 {main}
  thrown in script on line 2
```

Finalmente, também podem ser passados argumentos variáveis por referência ao prefixar `...` com um `&`.

Argumentos nomeados

O PHP 8.0.0 introduziu os argumentos nomeados como uma extensão aos parâmetros posicionais. Argumentos nomeados permitem a passagem de argumentos para uma função utilizando-se os nomes dos parâmetros ao invés das posições deles. Isto torna o significado do argumento documentado

automaticamente, e torna os argumentos independentes da ordem, além de permitir pular argumentos opcionais.

Argumentos nomeados são passados ao prefixar o valor com o nome do parâmetro e dois pontos. Usar palavras reservadas como nomes de parâmetros é permitido. O nome do parâmetro precisa ser um identificador e uma resolução dinâmica não é permitida.

Exemplo #13 Sintaxe dos argumentos nomeados

```php
<?php
minhaFuncao(nomeParametro: $valor);
array_foobar(array: $value);

// NÃO suportado
nome_funcao($variavelQueGuardaNomeDoParametro: $valor);
```

Exemplo #14 Comparando argumentos posicionais e nomeados

```php
<?php
// Utilizando argumentos posicionais:
array_fill(0, 100, 50);

// Utilizando argumetos nomeados:
array_fill(start_index: 0, count: 100, value: 50);
```

A ordem em que argumentos nomeados são passados não importa.

Exemplo #15 Mesmo exemplo com argumentos em ordem diferente

```php
<?php
array_fill(value: 50, count: 100, start_index: 0);
```

Argumentos nomeados podem ser combinados com argumentos posicionais. Nesse caso, os argumentos nomeados precisam estar depois dos argumentos posicionais. Também é possível especificar somente alguns dos argumentos opcionais de uma função, independente da ordem.

Exemplo #16 Combinando argumentos posicionais e nomeados

```php
<?php
htmlspecialchars($string, double_encode: false);
// Mesmo que
htmlspecialchars($string, ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401, 'UTF-8', false);
```

Passar o mesmo argumento mais de uma vez resulta em uma exceção Error.

Exemplo #17 Erro lançado ao passar um argumento ao mesmo parâmetro nomeado múltiplas vezes

```php
<?php
function foo($param) { ... }

foo(param: 1, param: 2);
// Error: Named parameter $param overwrites previous argument
```

```php
foo(1, param: 2);
// Error: Named parameter $param overwrites previous argument
```

A partir do PHP 8.1.0, é possível usar argumentos nomeados após desempacotar os argumentos. Um argumento nomeado *não deve* sobrepor argumentos já desempacotados.

Exemplo #18 Usando argumentos nomeados após desempacotar

```php
<?php
function foo($a, $b, $c = 3, $d = 4) {
  return $a + $b + $c + $d;
}

var_dump(foo(...[1, 2], d: 40)); // 46
var_dump(foo(...['b' => 2, 'a' => 1], d: 40)); // 46

var_dump(foo(...[1, 2], b: 20)); // Fatal error. Named parameter $b overwrites previous argument
```

Retornando valores

Os valores podem ser retornados utilizando a instrução opcional return. Qualquer tipo pode ser retornado, incluindo arrays e objetos. Isto faz com que as função termine sua execução imediatamente e passa o controle de volta para a linha de onde ela foi chamada. Veja a documentação da função return para maiores informações.

> **Nota** :
>
> Se return for omitido então o valor `null` será retornado.

Sintaxe do return

Exemplo #1 O uso de return

```php
<?php
function quadrado($num)
{
    return $num * $num;
}
echo quadrado(4); // exibe '16'.
```

Você não pode retornar múltiplos valores a partir de uma função, mas resultados similares podem ser obtidos retornando um array.

Exemplo #2 Retornando um array para devolver vários valores

```php
<?php
function primeiros_numeros()
{
    return [0, 1, 2];
}
// Desconstruir o array coleta cada item individualmente
[$zero, $one, $two] = primeiros_numeros();
var_dump($zero, $one, $two);
```

```php
// Anteriormente ao 7.1.0 a única alternativa era usar list()
list($zero, $one, $two) = primeiros_numeros();
```

Para retornar uma referência de uma função, use o operador de referência & em ambas a declaração da função e quando definindo o valor para a variável.

Exemplo #3 Retornando uma referência de uma função

```php
<?php
function &retorna_referencia()
{
    return $alguma_referencia;
}

$nova_referencia =& retorna_referencia();
```

Para mais detalhes sobre referências, leia a seção Referências.

Funções variáveis

O PHP suporta o conceito de funções variáveis. Isto significa que se um nome de variável tem parênteses no final dela, o PHP procurará uma função com o mesmo nome, qualquer que seja a avaliação da variável, e tentará executá-la. Entre outras coisas, isto pode ser usado para implementar callbacks, tabelas de função e assim por diante.

Funções variáveis não funcionam com construtores de linguagem como echo, print, unset(), isset(), empty(), include, require e outras assim. Utilize uma função de encapsulamento para usar quaisquer um destes construtores como uma função variável.

Exemplo #1 Exemplo de funções variáveis

```php
<?php
function foo() {
    echo "Chamou foo()\n";
}

function bar($arg = '')
{
    echo "Chamou bar(); com argumento '$arg'.\n";
}

// Essa é uma função encapsuladora para echo()
function echoit($string)
{
    echo $string;
}

$func = 'foo';
$func();        // Chama foo()

$func = 'bar';
$func('test');  // Chama bar()
```

```php
$func = 'echoit';
$func('test');  // Chama echoit()
```

Um método de um objeto também pode ser chamado com a sintaxe de funções variáveis.

Exemplo #2 Exemplo de chamada de um método variável

```php
<?php
class Foo
{
    function MetodoVariavel()
    {
        $name = 'Bar';
        $this->$name(); // Isto chama o método Bar()
    }

    function Bar()
    {
        echo "Bar foi chamada!";
    }
}

$foo = new Foo();
$funcname = "MetodoVariavel";
$foo->$funcname();  // Isto chama $foo->MetodoVariavel()
```

Quando chamando métodos estáticos, uma chamada de função tem preferência que uma propriedade estática.

Exemplo #3 Método variávei com propriedades estáticas

```php
<?php
class Foo
{
    static $variable = 'propriedade estática';
    static function Variable()
    {
        echo "Método Variable chamado\n";
    }
}

echo Foo::$variable . "\n"; // Isto exibe 'static property'. Ele busca $variable no escopo da
classe.
$variable = "Variable";
Foo::$variable();  // Isto chama $foo->Variable(), lendo a $variable no escopo da chamada.
```

O exemplo acima produzirá:

```
static property
Método Variable chamado
```

Exemplo #4 Chamáveis complexos

```php
<?php
class Foo
{
    static function bar()
    {
        echo "bar\n";
    }
    function baz()
    {
        echo "baz\n";
    }
}

$func = array("Foo", "bar");
$func(); // exibe "bar"
$func = array(new Foo, "baz");
$func(); // exibe "baz"
$func = "Foo::bar";
$func(); // exibe "bar"
```

Veja também

- is_callable()
- call_user_func()
- function_exists()
- variáveis variáveis

Funções internas

O PHP vem por padrão com muitas funções e construtores. Também existem funções que necessitam que uma extensão específica esteja compilada, se não erros fatais "undefined function" (função não definida) irão aparecer. Por exemplo, para usar as funções image como imagecreatetruecolor(), o PHP deve ser compilado com suporte GD. Ou, para usar mysqli_connect(), PHP deve ser compilado com suporte MySQLi. Existem muitas funções do núcleo que são incluídas em cada versão do PHP, como string e variable. Uma chamada a phpinfo() ou get_loaded_extensions() irá mostrar quais extensões estão carregadas no PHP. Note também que várias extensões são ativadas por padrão e que o manual do PHP é dividido por extensão. Veja configuração, instalação, e os capítulos individuais das extensões para como configurar o PHP.

A leitura e entendimento de um protótipo de função é explicado na seção do manual intitulada como ler uma definição de função. É importante perceber o que a função retorna ou se a função trabalha diretamente no valor passado como argumento. Por exemplo, str_replace() irá retornar a string modificada enquanto usort() trabalha na própria variável passada no argumento. Cada página do manual também contém informações específicas de cada função, como: parâmetros da função, mudanças de comportamento, valores de retorno tanto para sucesso e erros, como outras informações disponíveis. O conhecimento destas diferenças importantes é crucial para escrever corretamente seu código PHP.

> **Nota** :
>
> Se os parâmetros dados a uma função não forem o que ela espera, como passar um array onde uma string é esperada, o valor de retorno da função é indefinido. Neste caso provavelmente irá retornar `null`

mas isto é apenas uma convenção, e você não pode depender disso. A partir do PHP 8.0.0, uma exceção TypeError é lançada nesses casos.

> **Nota** :
>
> Tipos escalares para funções embutidas são anuláveis por padrão no modo coercivo. A partir do PHP 8.1.0, passar `null` para um parâmetro de uma função interna que não é declarado anulável é desencorajado e emite um aviso de descontinuação no modo coercitivo para alinhar com o comportamento de funções definidas pelo usuário, onde tipos escalares precisam ser marcados como anuláveis explícitamente.
>
> Por exemplo, a função strlen() espera que o parâmetro `$string` seja uma string não anulável. Por razões históricas, o PHP permite a passagem de `null` para esse parâmetro no modo coercivo, e o parâmetro é implicitamente convertido para string, resultando em um valor `""`. Em contrapartida, um TypeError é emitido no modo estrito.

```php
<?php
var_dump(strlen(null));
// "Deprecated: Passing null to parameter #1 ($string) of type string is deprecated" as of PHP
8.1.0
// int(0)

var_dump(str_contains("foobar", null));
// "Deprecated: Passing null to parameter #2 ($needle) of type string is deprecated" as of PHP
8.1.0
// bool(true)
```

Veja também

- function_exists()
- A referência das funções
- get_extension_funcs()
- dl()

Funções anônimas

Funções anônimas, também conhecidas como `closures`, permitem a criação de funções que não têm o nome especificado. Elas são mais úteis como o valor de parâmetros do tipo callable, mas podem ter vários outros usos.

Funções anônimas são implementadas utilizando a classe Closure

Exemplo #1 Exemplo de função anônima

```php
<?php
echo preg_replace_callback('~-([a-z])~', function ($match) {
    return strtoupper($match[1]);
}, 'hello-world');
// exibe helloWorld
```

Closures também podem ser utilizadas como valores de variáveis; o PHP automaticamente converte expressões assim em instâncias da classe interna Closure. Definindo um closure a uma variável usa a mesma

sintaxe que qualquer outra definição, incluindo o ponto-e-vírgula:

Exemplo #2 Exemplo de como definir uma função anônima para uma variável

```php
<?php
$greet = function($name) {
    printf("Hello %s\r\n", $name);
};

$greet('World');
$greet('PHP');
```

Closures também podem herdar variáveis do escopo pai. Essas variáveis precisam ser referenciadas utilizando a instrução `use`. A partir do PHP 7.1, essas variáveis não devem incluir superglobals, *$this*, ou variáveis com o mesmo nome como um parâmetro. A declaração de tipo de retorno de uma função deve ser posicionada *depois* de uma instrução `use`.

Exemplo #3 Herdando variáveis do escopo anterior

```php
<?php
$message = 'hello';

// Sem "use"
$example = function () {
    var_dump($message);
};
$example();

// Inherit $message
$example = function () use ($message) {
    var_dump($message);
};
$example();

// Herdando valor da variável quando a função é definida,
// não quando é chamada
$message = 'world';
$example();

// Altera a mensagem
$message = 'hello';

// Herdando por referência
$example = function () use (&$message) {
    var_dump($message);
};
$example();

// O valor modificado no escopo pai
// reflete quando a função é chamada
$message = 'world';
$example();

// Closures também aceitam argumentos normais
$example = function ($arg) use ($message) {
    var_dump($arg . ' ' . $message);
```

```php
};
$example("hello");

// Declaração de tipo de retorno após a instrução 'use'
$example = function () use ($message): string {
    return "hello $message";
};
var_dump($example());
```

O exemplo acima produzirá algo semelhante a:

```
Notice: Undefined variable: message in /example.php on line 6
NULL
string(5) "hello"
string(5) "hello"
string(5) "hello"
string(5) "world"
string(11) "hello world"
string(11) "hello world"
```

A partir do PHP 8.0.0 a lista de variáveis herdadas no escopo podem incluir uma vírgula final, que será ignorada.

Herdar variáveis do escopo pai *não* é o mesmo que usar variáveis globais. Variáveis globais existem no escopo global, o qual é o mesmo não importa a função sendo executada. O escopo pai de um closure é a função no qual o closure foi declarado (não necessariamente a função a partir do qual ele foi chamado). Veja o exemplo a seguir:

Exemplo #4 Closures e escopo

```php
<?php
// Um exemplo básico de carrinho de compras que contém uma lista de produtos
// e a quantidade de cada produto. Inclui um método que
// calcula o preço total dos itens no carrinho utilizando uma
// closure como função de retorno.
class Cart
{
    const PRICE_BUTTER  = 1.00;
    const PRICE_MILK    = 3.00;
    const PRICE_EGGS    = 6.95;

    protected $products = array();

    public function add($product, $quantity)
    {
        $this->products[$product] = $quantity;
    }

    public function getQuantity($product)
    {
        return isset($this->products[$product]) ? $this->products[$product] :
```

```
                FALSE;
    }

    public function getTotal($tax)
    {
        $total = 0.00;

        $callback =
            function ($quantity, $product) use ($tax, &$total)
            {
                $pricePerItem = constant(__CLASS__ . "::PRICE_" .
                    strtoupper($product));
                $total += ($pricePerItem * $quantity) * ($tax + 1.0);
            };

        array_walk($this->products, $callback);
        return round($total, 2);
    }
}

$my_cart = new Cart;

// Adiciona alguns itens no carrinho
$my_cart->add('butter', 1);
$my_cart->add('milk', 3);
$my_cart->add('eggs', 6);

// Exibe o total com uma taxa de venda de 5%.
print $my_cart->getTotal(0.05) . "\n";
// O resultado é 54.29
```

Exemplo #5 Vinculação automática do `$this`

```
<?php
class Test
{
    public function testing()
    {
        return function() {
            var_dump($this);
        };
    }
}

$object = new Test;
$function = $object->testing();
$function();
```

O exemplo acima produzirá:

```
object(Test)#1 (0) {
}
```

Quando declarada no contexto de uma classe, a classe corrente é automaticamente vinculada a ela, tornando `$this` disponível dentro do escopo da função. Se essa vinculação automática da classe corrente não for o comportamento esperado, funções anônimas estáticas podem ser utilizadas.

Funções anônimas estáticas

Funções anônimas podem ser declaradas estaticamente. Evita que a classe corrente seja automaticamente vinculada a ela. Objetos também podem não ser vinculados a ela em tempo de execução.

Exemplo #6 Tentando utilizar o `$this` de dentro de uma função anônima estática

```php
<?php
class Foo
{
    function __construct()
    {
        $func = static function() {
            var_dump($this);
        };
        $func();
    }
};
new Foo();
```

O exemplo acima produzirá:

```
Fatal error: Uncaught Error: Using $this when not in object context in script:7
Stack trace:
#0 script(9): Foo::{closure:Foo::__construct():6}()
#1 script(12): Foo->__construct()
#2 {main}
  thrown in script on line 7
```

Exemplo #7 Tentando vincular um objeto a uma função anônima estática

```php
<?php
$func = static function() {
    // corpo da função
};
$func = $func->bindTo(new stdClass);
$func();
```

O exemplo acima produzirá:

```
Warning: Cannot bind an instance to a static closure, this will be an error in PHP 9 in script on
line 5
```

```
Fatal error: Uncaught Error: Value of type null is not callable in script:6
Stack trace:
#0 {main}
  thrown in script on line 6
```

Registro de Alterações

Versão	Descrição
8.3.0	Closures criadas a partir de métodos mágicos podem aceitar parâmetros nomeados.
7.1.0	Funções anônimas podem não incluir superglobals, *$this*, ou qualquer variável com o mesmo nome como um parâmetro.

Notas

> **Nota** :
>
> É possível utilizar func_num_args(), func_get_arg() e func_get_args() dentro de closures.

Funções de Seta

Funções de seta foram introduzidas a partir do PHP 7.4 como uma sintaxe mais concisa para funções anônimas.

Funções anônimas e funções de seta são implementadas usando a classe Closure.

Funções de seta têm a forma básica `fn (argument_list) => expr`.

Funções de seta possuem as mesmas características das funções anônimas, exceto que o uso de variáveis do escopo pai é sempre automático.

Quando uma variável usada na expressão é definida no escopo pai, ela será implicitamente capturada por valor. No exemplo a seguir, as funções *$fn1* e *$fn2*comportam-se da mesma maneira.

Exemplo #1 Funções de seta capturam variáveis por valor automaticamente

```php
<?php
$y = 1;

$fn1 = fn($x) => $x + $y;
// Equivalente ao usar $y por valor:
$fn2 = function ($x) use ($y) {
    return $x + $y;
};

var_export($fn1(3));
```

O exemplo acima produzirá:

```
4
```

Isso também funciona se as funções de seta estiverem aninhadas:

Exemplo #2 Funções de seta capturam variáveis por valor automaticamente, mesmo quando aninhadas

```php
<?php
$z = 1;
$fn = fn($x) => fn($y) => $x * $y + $z;
// Outputs 51
var_export($fn(5)(10));
```

Da mesma forma que funções anônimas, a sintaxe das funções de seta permitem assinaturas de função arbitrária, incluindo parâmetros e tipos de retorno, valores padrão, variadics, bem como por referência passando e retornando. Todos os exemplos válidos de funções de seta:

Exemplo #3 Exemplos de funções de seta

```php
<?php
fn(array $x) => $x;
static fn($x): int => $x;
fn($x = 42) => $x;
fn(&$x) => $x;
fn&($x) => $x;
fn($x, ...$rest) => $rest;
```

Funções de seta usam vinculação de variável por valor. Isso é aproximadamente equivalente a realizar um `use($x)` para cada variável *$x* usada dentro da funções de seta. Uma passagem de variável por valor, significa que não é possível modificar quaisquer valores do escopo externo. Funções anônimas podem ser usadas em vez disso para passagem de variável por referência.

Exemplo #4 Valores do escopo externo não podem ser modificados por funções de seta

```php
<?php
$x = 1;
$fn = fn() => $x++; // Não tem efeito
$fn();
var_export($x);  // Exibe 1
```

Registro de Alterações

Versão	Descrição
7.4.0	Funções de seta tornaram-se disponíveis.

Notas

> **Nota** :
>
> É possível usar func_num_args(), func_get_arg(), e func_get_args() de dentro de uma função de seta.

Sintaxe de Chamáveis de Primeira Classe

A sintaxe de chamáveis de primeira classe é introduzida a partir do PHP 8.1.0, como uma maneira de criar funções anônimas a partir de um chamável. Ela substitui a sintaxe de chamáveis existente usando strings e arrays. A vantagem dessa sintaxe é que ela é acessível à análise estática, e usa o escopo no ponto onde o chamável é adquirido.

A sintaxe `CallableExpr(...)` é usada para criar um objeto Closure a partir de um chamável. `CallableExpr` aceita qualquer expressão que possa ser diretamente chamada na gramática do PHP:

Exemplo #1 Sintaxe de chamável de primeira classe simples

```php
<?php
class Foo {
   public function metodo() {}
   public static function metodoestatico() {}
   public function __invoke() {}
}

$obj = new Foo();
$strClasse = 'Foo';
$strMetodo = 'metodo';
$strMetodoestatico = 'metodoestatico';


$f1 = strlen(...);
$f2 = $obj(...);  // objeto chamável
$f3 = $obj->metodo(...);
$f4 = $obj->$strMetodo(...);
$f5 = Foo::metodoestatico(...);
$f6 = $strClasse::$strMetodoestatico(...);

// Chamável tradicional usando string, array
$f7 = 'strlen'(...);
$f8 = [$obj, 'metodo'](...);
$f9 = [Foo::class, 'metodoestatico'](...);
```

> **Nota** :
>
> As reticências `...` são parte da sintaxe e não uma omissão.

`CallableExpr(...)` tem a mesma semântica que Closure::fromCallable(). Isto é, ao contrário de um chamável usando string e array, `CallableExpr(...)` respeita o escopo no ponto onde ela é criada:

Exemplo #2 Comparação de escopo de `CallableExpr(...)` e chamável tradicional

```php
<?php
class Foo {
    public function obterMetodoPrivado() {
        return [$this, 'metodoPrivado'];
    }

    private function metodoPrivado() {
        echo __METHOD__, "\n";
```

```
    }
}

$foo = new Foo;
$metodoPrivado = $foo->obterMetodoPrivado();
$metodoPrivado();
// Fatal error: Call to private method Foo::metodoPrivado() from global scope
// Isso acontece porque a chamada é realizada fora de Foo e a visibilidade será verificada a partir
desse ponto.
```

```
<?php
class Foo1 {
    public function obterMetodoPrivado() {
        // Usa o escopo onde o chamável é adquirido.
        return $this->metodoPrivado(...); // Idêntico a Closure::fromCallable([$this,
'metodoPrivado']);
    }

    private function metodoPrivado() {
        echo __METHOD__, "\n";
    }
}

$foo1 = new Foo1;
$metodoPrivado = $foo1->obterMetodoPrivado();
$metodoPrivado();  // Foo1::metodoPrivado
```

> **Nota** :
>
> Criação de objetos por essa sintaxe (por exemplo `new Foo(...)`) não é suportada porque a sintaxe `new Foo()` não é considerada uma chamada.

> **Nota** :
>
> A sintaxe de chamável de primeira classe não pode ser combinada com o operador nullsafe. Ambos os resultados a seguir resultam em um erro de tempo de compilação:
>
> ```
> <?php
> $obj?->metodo(...);
> $obj?->prop->metodo(...);
> ```

Classes e Objetos

Índice

Introdução

O PHP inclui uma modelagem a objetos completa. Entre os recursos disponíveis há: visibilidade, classes e métodos abstratos e finais, métodos mágicos adicionais, interfaces e clonagem.

O PHP trata objetos da mesma maneira que referências ou manipuladores, significando que cada variável contém uma referência a um objeto ao invés de uma cópia de todo o objeto. Veja Objetos e Referências

> **Dica**
>
> Veja também o Guia de nomenclatura em espaço de usuário.

O Básico

Classe

A definição de uma classe começa com a palavra chave `class`, seguida do nome da classe, seguido de um par de chaves que englobam as definições de propriedades e métodos pertencentes à classe.

O nome de uma classe pode ser qualquer identificador válido, que não seja uma palavra reservada do PHP. A partir do PHP 8.4.0, usar um único sublinhado _ como nome de classe foi descontinuado. Um nome de classe válido começa com uma letra ou sublinhado, seguido de qualquer sequência de letras, números e sublinhados. Como uma expressão regular, pode ser expressada assim: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`.

Uma classe pode conter suas próprias constantes, variáveis (chamadas de "propriedades") e funções (chamadas de "métodos").

Exemplo #1 Definição Simples de uma Classe

```php
<?php
class SimpleClass
{
    // declaração de propriedade
    public $var = 'um valor padrão';

    // declaração de método
    public function displayVar() {
        echo $this->var;
    }
}
?>
```

A pseudo-variável *$this* está disponível quando um método é chamado a partir de um contexto de um objeto. *$this* é o valor do objeto chamado.

> **Aviso**
>
> Chamar um método não estático de maneira estática lança um Error. Anteriormente ao PHP 8.0.0, isto iria gerar um aviso de descontinuação, e *$this* estaria indefinido.
>
> **Exemplo #2 Alguns exemplos da pseudo variável *$this***
>
> ```php
> <?php
> class A
> {
>     function foo()
>     {
>         if (isset($this)) {
>             echo '$this está definida (';
>             echo get_class($this);
> ```

```
            echo ")\n";
        } else {
            echo "\$this não está definida.\n";
        }
    }
}

class B
{
    function bar()
    {
        A::foo();
    }
}

$a = new A();
$a->foo();

A::foo();

$b = new B();
$b->bar();

B::bar();
?>
```

Saída do exemplo acima no PHP 7:

```
$this está definida (A)

Deprecated: Non-static method A::foo() should not be called statically in %s  on line 27
$this não está definida.

Deprecated: Non-static method A::foo() should not be called statically in %s  on line 20
$this não está definida.

Deprecated: Non-static method B::bar() should not be called statically in %s  on line 32

Deprecated: Non-static method A::foo() should not be called statically in %s  on line 20
$this não está definida.
```

Saída do exemplo acima no PHP 8:

```
$this está definida (A)

Fatal error: Uncaught Error: Non-static method A::foo() cannot be called statically in %s :27
Stack trace:
#0 {main}
  thrown in %s  on line 27
```

Classes somente leitura

A partir do PHP 8.2.0, uma classe pode ser marcada com o modificador readonly. Marcar uma classe como readonly irá acrescentar o modificador readonly em cada propriedade declarada, e prevenir a criação de propriedades dinâmicas. Além disso, será impossível de acrescentar suportes a propriedades dinâmicas utilizando o atributo AllowDynamicProperties. Acrescentar o atributo em classes somente leitura irá disparar um erro de compilação.

```
<?php
#[\AllowDynamicProperties]
readonly class Foo {
}

// Fatal error: Cannot apply #[AllowDynamicProperties] to readonly class Foo
?>
```

Como as propriedades estáticas ou sem tipo não podem ser marcadas com o modificador `readonly`, as classes somente leitura também não podem declará-las:

```
<?php
readonly class Foo
{
    public $bar;
}

// Fatal error: Readonly property Foo::$bar must have type
?>
```

```
<?php
readonly class Foo
{
    public static int $bar;
}

// Fatal error: Readonly class Foo cannot declare static properties
?>
```

Uma classe readonly pode ser estendida se, e somente se, a classe filha também ser uma classe readonly.

Nova instância

Para criar uma instância de uma classe, a instrução `new` deve ser utilizada. Um objeto sempre será criado a não ser que a classe tenha um construtor definido que dispare uma exceção em caso de erro. Classes devem ser definidas antes de instanciadas (e em alguns casos isso é obrigatório).

Se uma variável contendo uma string com o nome da classe for utilizado com `new`, uma nova instância da classe será criada. Se a classe estiver dentro de um namespace, seu nome qualificado completo deve ser utilizado ao fazer isto.

> **Nota** :

> Se não há argumentos a serem passados para o construtor da classe, os parênteses após o nome da classe podem ser omitidos.

Exemplo #3 Criando uma instância

```php
<?php
class SimpleClass {
}

$instance = new SimpleClass();
var_dump($instance);

// Também pode ser feito com uma variável:
$className = 'SimpleClass';
$instance = new $className(); // new SimpleClass()
var_dump($instance);
?>
```

A partir do PHP 8.0.0, usar `new` com expressões arbitrárias é suportado. Isso permite instanciação mais complexa se a expressão produzir uma string. As expressões devem ser envolvidas em parênteses.

Exemplo #4 Criando uma instância usando uma expressão arbitrária

No exemplo fornecido mostramos múltiplos exemplos de expressões arbitrárias válidas que produzem um nome de classe. Isto mostra uma chamada para uma função, concatenação de string, e a constante **`::class`**.

```php
<?php

class ClasseA extends \stdClass {}
class ClasseB extends \stdClass {}
class ClasseC extends ClasseB {}
class ClasseD extends ClasseA {}

function obterAlgumaClasse(): string
{
    return 'ClasseA';
}

var_dump(new (obterAlgumaClasse()));
var_dump(new ('Classe' . 'B'));
var_dump(new ('Classe' . 'C'));
var_dump(new (ClasseD::class));
?>
```

Saída do exemplo acima no PHP 8:

```
object(ClasseA)#1 (0) {
}
object(ClasseB)#1 (0) {
}
object(ClasseC)#1 (0) {
}
```

```
object(ClasseD)#1 (0) {
}
```

No contexto da classe, é possível criar um novo objeto com `new self` e `new parent`.

Ao atribuir uma instância de uma classe já criada, a uma variável nova, a variável nova irá acessar a mesma instância do objeto que foi atribuído. Este comportamento se mantém ao se passar instâncias a uma função. Uma cópia de um objeto criado pode ser feita clonando o mesmo.

Exemplo #5 Atribuição de Objetos

```php
<?php
class SimpleClass {
    public string $var;
}

$instance = new SimpleClass();

$assigned   =  $instance;
$reference  =& $instance;

$instance->var = '$assigned terá esse valor';

$instance = null; // $instance e $reference tornam-se nulos

var_dump($instance);
var_dump($reference);
var_dump($assigned);
?>
```

O exemplo acima produzirá:

```
NULL
NULL
object(SimpleClass)#1 (1) {
  ["var"]=>
  string(25) "$assigned terá esse valor"
}
```

É possível usar algumas formas de criar instâncias de um objeto:

Exemplo #6 Criando novos objetos

```php
<?php

class Test
{
    public static function getNew()
```

```
    {
        return new static();
    }
}

class Child extends Test {}

$obj1 = new Test();  // Usando o nome da classe
$obj2 = new $obj1(); // Usando a variável que contém o objeto
var_dump($obj1 !== $obj2);

$obj3 = Test::getNew();   // Usando o método da classe
var_dump($obj3 instanceof Test);

$obj4 = Child::getNew();  // Usando o método através da classe filha
var_dump($obj4 instanceof Child);

?>
```

O exemplo acima produzirá:

```
bool(true)
bool(true)
bool(true)
```

É possível acessar um membro do objeto recém criado em uma expressão simples:

Exemplo #7 Acessando um membro de um novo objeto criado

```
<?php
echo (new DateTime())->format('Y'), PHP_EOL;

// parênteses envolvendo o objeto são opcionais a partir do PHP 8.4.0
echo new DateTime()->format('Y'), PHP_EOL;
?>
```

O exemplo acima produzirá algo semelhante a:

```
2025
2025
```

> **Nota** :
>
> Anteriormente ao PHP 7.1, os argumentos não são avaliados se não houver um construtor definido.

Propriedades e métodos

Propriedades e métodos de classe vivem em "namespaces" separados, de forma que é possível ter uma propriedade e método com mesmos nomes. A referência a propriedades e métodos tem a mesma notação, e a decisão de se uma propriedade será acessada ou uma chamada a um método feita, depende somente do contexto, ou seja, se está tentando acessar uma variável ou chamar um método.

Exemplo #8 Acesso a propriedade vs. chamar um método

```php
<?php
class Foo
{
    public $bar = 'propriedade';

    public function bar() {
        return 'método';
    }
}

$obj = new Foo();
echo $obj->bar, PHP_EOL, $obj->bar(), PHP_EOL;
```

O exemplo acima produzirá:

```
propriedade
método
```

Isto significa que chamar diretamente uma função anônima atribuída a uma propriedade não é possível. Em vez disso, por exemplo, a propriedade deve primeiro ser atribuída a uma variável. É possível chamar uma propriedade diretamente colocando-a entre parênteses.

Exemplo #9 Chamando uma função anônima armazenada em uma propriedade

```php
<?php
class Foo
{
    public $bar;

    public function __construct() {
        $this->bar = function() {
            return 42;
        };
    }
}

$obj = new Foo();

echo ($obj->bar)(), PHP_EOL;
```

O exemplo acima produzirá:

```
42
```

Herança

Uma classe pode herdar constantes, métodos e propriedades de outra classe usando a palavra-chave `extends` na declaração da classe. Não é possível herdar múltiplas classes; uma classe só pode herdar uma classe base.

Os métodos e propriedades herdados podem ser sobrescritos declarando-os com o mesmo nome definido na classe base. Entretanto, se a classe mãe definiu um método ou constante como final, eles não poderão ser sobrescritos. É possível acessar os métodos sobrescritos ou propriedades estáticas referenciado-os com parent::.

> **Nota** :
>
> A partir do PHP 8.1.0, constantes podem ser declaradas como finais.

Exemplo #10 Herança simples de classe

```php
<?php
class SimpleClass
{
    function displayVar()
    {
        echo "Classe Pai\n";
    }
}

class ExtendClass extends SimpleClass
{
    // Redefine o método pai
    function displayVar()
    {
        echo "Classe Herdeira\n";
        parent::displayVar();
    }
}

$extended = new ExtendClass();
$extended->displayVar();
?>
```

O exemplo acima produzirá:

```
Classe Herdeira
Classe Pai
```

Regras de compatibilidade de assinaturas

Ao sobrescrever um método, sua assinatura precisa ser compatível com a do método original. Caso contrário um erro fatal é emitido, ou, antes do PHP 8.0.0, um alerta `E_WARNING` seria gerado. Uma assinatura é compatível se ela respeita as regras de variância, se ela transforma um parâmetro obrigatório em opcional, se ela adiciona novos parâmetros opcionais, e se ela não restringe ou apenas aumenta a visibilidade. Isso é conhecido como Princípio de Substituição de Liskov, ou LSP. O construtor, e membros `private` não precisam seguir essas regras de compatibilidade, e portanto não há emissão de erros fatais no caso de assinaturas incompatíveis.

Exemplo #11 Métodos compatíveis

```php
<?php

class Base
{
    public function foo(int $a) {
        echo "Valid\n";
    }
}

class Extend1 extends Base
{
    function foo(int $a = 5)
    {
        parent::foo($a);
    }
}

class Extend2 extends Base
{
    function foo(int $a, $b = 5)
    {
        parent::foo($a);
    }
}

$extended1 = new Extend1();
$extended1->foo();
$extended2 = new Extend2();
$extended2->foo(1);
```

O exemplo acima produzirá:

```
Valid
Valid
```

Os exemplos a seguir demonstram que um método derivado, que remove um parâmetro ou transforma um parâmetro opcional em mandatório, não é compatível com o método original.

Exemplo #12 Erro fatal quando um método derivado remove um parâmetro

```php
<?php

class Base
{
    public function foo(int $a = 5) {
        echo "Válido?\n";
    }
}

class Extend extends Base
{
    function foo()
    {
        parent::foo(1);
    }
}
```

A saída do exemplo acima no PHP 8 é semelhante a:

```
Fatal error: Declaration of Extend::foo() must be compatible with Base::foo(int $a = 5) in
/in/evtlq on line 13
```

Exemplo #13 Erro fatal quando um método derivado transforma um parâmetro opcional em mandatório

```php
<?php

class Base
{
    public function foo(int $a = 5) {
        echo "Válido?\n";
    }
}

class Extend extends Base
{
    function foo(int $a)
    {
        parent::foo($a);
    }
}
```

A saída do exemplo acima no PHP 8 é semelhante a:

```
Fatal error: Declaration of Extend::foo(int $a) must be compatible with Base::foo(int $a = 5) in
/in/qJXVC on line 13
```

> **Aviso**
>
> Renomear um parâmetro em uma classe derivada não é uma quebra de assinatura. Entretanto isso é desencorajado porque ocasionará um Error se argumentos nomeados forem utilizados na chamada.
>
> **Exemplo #14 Erro ao utilizar argumentos renomeados em uma classe derivada**
>
> ```php
> <?php
>
> class A {
>     public function test($foo, $bar) {}
> }
>
> class B extends A {
>     public function test($a, $b) {}
> }
>
> $obj = new B;
>
> // Passando parâmetros de acordo com o contrato de A::test()
> $obj->test(foo: "foo", bar: "bar"); // ERROR!
> ```
>
> O exemplo acima produzirá algo semelhante a:
>
> ```
> Fatal error: Uncaught Error: Unknown named parameter $foo in /in/XaaeN:14
> Stack trace:
> #0 {main}
>   thrown in /in/XaaeN on line 14
> ```

::class

A palavra-chave `class` também pode ser utilizada para resolução de nome de classes. Pode-se obter o nome completo e qualificado da classe `ClassName` utilizando `ClassName::class`. Isso é particularmente útil em classes com namespaces.

Exemplo #15 Resolução de nome da classe

```php
<?php
namespace NS {
    class ClassName {
    }

    echo ClassName::class;
}
?>
```

O exemplo acima produzirá:

```
NS\ClassName
```

> **Nota** :
>
> A resolução do nome de classe através de `::class` é uma transformação em tempo de compilação. Isso significa que no momento em que o texto do nome da classe é criado, o auto carregamento ainda não ocorreu. Como consequência, nomes de classe são expandidos mesmo se a classe não existir. Não é emitido erro nestes casos.
>
> **Exemplo #16 Resolução de um nome de classe ausente**
>
> ```php
> <?php
> print Does\Not\Exist::class;
> ?>
> ```
>
> O exemplo acima produzirá:
>
> ```
> Does\Not\Exist
> ```

A partir do PHP 8.0.0, a constante `::class` também pode ser utilizada em objetos. Essa resolução acontece em tempo de execução, e não em tempo de compilação. O efeito é o mesmo de chamar get_class() em um objeto.

Exemplo #17 Resolução de nome de objeto

```php
<?php
namespace NS {
    class ClassName {
    }

    $c = new ClassName();
    print $c::class;
}
?>
```

O exemplo acima produzirá:

```
NS\ClassName
```

Métodos e propriedades nullsafe

A partir do PHP 8.0.0, propriedades e métodos também podem ser acessados com o operador "nullsafe": `?->`. O operador nullsafe funciona da mesma forma que os acessos de métodos e variáveis, exceto que se o objeto referenciado é `null`, então `null` será retornado ao invés de uma exceção ser lançada. Se uma desreferência for parte de uma cadeia de chamadas, o resto da cadeia é ignorado.

O resultado é similar a encapsular cada acesso dentro de um teste is_null(), mas mais compacto.

Exemplo #18 Operador nullsafe

```php
<?php

// A partir do PHP 8.0.0, esta linha:
$result = $repository?->getUser(5)?->name;

// É o equivalente ao seguinte código:
if (is_null($repository)) {
    $result = null;
} else {
    $user = $repository->getUser(5);
    if (is_null($user)) {
        $result = null;
    } else {
        $result = $user->name;
    }
}
?>
```

> **Nota** :
>
> O operador nullsafe é melhor utilizado quando null é considerado um valor válido e esperado de uma propriedade ou retorno de método. Para indicar um erro, uma exceção lançada é preferível.

Propriedades

Variáveis membros de classes são chamados de *propriedades*. Eles também são conhecidos por outros termos, como *campos* (field), mas para propósitos deste manual será utilizado o termo *propriedade*. Elas podem ser definidas com ao menos um modificador opcional (ver Visibilidade, Palavra-Chave 'static', ou, a partir do PHP 8.1.0, readonly), (exceto para propriedades `readonly` properties), a partir do PHP 7.4, seguido também de uma declaração de tipo, finalizando com um nome de variável comum. Essa declaração pode ainda incluir uma inicialização, desde que essa inicialização seja um valor constante.

> **Nota** :
>
> Uma forma obsoleta de declarar propriedades de classe é através da instrução `var` ao invés de um modificador.

> **Nota** :
>
> Uma propriedade declarada sem um modificador de Visibilidade funcionará como uma propriedade `public`.

Internamente às classes, métodos não estáticos podem ser acessados utilizando-se `->` (operador de objeto): *$this->property* (onde `property` é o nome da propriedade). Propriedades estáticas são acessadas utilizando-se o operador `::` (duplo dois pontos), assim: *self::$property*. Veja Palavra-Chave 'static' para mais informações sobre a diferença de propriedades estáticas e não estáticas.

A pseudo-variável *$this* está disponível dentro de qualquer classe de método chamada no contexto do objeto. *$this* é uma referência ao objeto de contexto do qual o método foi chamado.

Exemplo #1 Declaração de propriedades

```php
<?php
class SimpleClass
{
   public $var1 = 'Olá ' . 'mundo';
   public $var2 = <<<EOD
Olá mundo
EOD;
   public $var3 = 1+2;
   // Declaração de propriedade inválida
   public $var4 = self::myStaticMethod();
   public $var5 = $myVar;

   // Declaração de propriedade válida
   public $var6 = myConstant;
   public $var7 = [true, false];

   public $var8 = <<<'EOD'
Olá mundo
EOD;

   // Sem um modificador de visibilidade:
   static $var9;
   readonly int $var10;
}
?>
```

> **Nota** :
>
> Existem várias funções para manipular classes e objetos. Veja a seção sobre funções sobre classes/objetos para mais detalhes.

Declaração de propriedades tipadas

A partir do PHP 7.4.0, definições de propriedades podem incluir uma Declarações de tipo, com exceção do tipo callable.

Exemplo #2 Exemplo de propriedades tipadas

```php
<?php

class User
{
    public int $id;
    public ?string $name;

    public function __construct(int $id, ?string $name)
    {
        $this->id = $id;
        $this->name = $name;
    }
}

$user = new User(1234, null);

var_dump($user->id);
var_dump($user->name);

?>
```

O exemplo acima produzirá:

```
int(1234)
NULL
```

Propriedades tipadas precisam ser inicializadas antes de serem acessadas, senão um Error é lançado.

Exemplo #3 Acessando propriedades

```php
<?php

class Forma
{
    public int $numeroDeLados;
    public string $nome;

    public function setNumeroDeLados(int $numeroDeLados): void
    {
        $this->numeroDeLados = $numeroDeLados;
    }

    public function setNome(string $nome): void
    {
        $this->nome = $nome;
    }

    public function getNumeroDeLados(): int
    {
        return $this->numeroDeLados;
    }
```

```
    public function getNome(): string
    {
        return $this->nome;
    }
}

$triangle = new Forma();
$triangle->setNome("triângulo");
$triangle->setNumeroDeLados(3);
var_dump($triangle->getNome());
var_dump($triangle->getNumeroDeLados());

$circle = new Forma();
$circle->setNome("círculo");
var_dump($circle->getNome());
var_dump($circle->getNumeroDeLados());
?>
```

O exemplo acima produzirá:

```
string(9) "triângulo"
int(3)
string(7) "círculo"

Fatal error: Uncaught Error: Typed property Forma::$numeroDeLados must not be accessed before
initialization
```

Propriedades somente leitura

A partir do PHP 8.1.0, uma propriedade pode ser declarada com o modificador `readonly`, o que evita modificação da propriedade após a inicialização. Antes do PHP 8.4.0, uma propriedade `readonly` era implicitamente private-set e só poderia ser escrita na mesma classe. A partir do PHP 8.4.0, as propriedades `readonly` são implicitamente `protected(set)`, por isso podem ser definidas a partir de classes filhas. Isso pode ser substituído explicitamente, se desejado.

Exemplo #4 Exemplo de propriedade somente leitura

```
<?php

class Test {
   public readonly string $prop;

   public function __construct(string $prop) {
       // Inicialização permitida.
       $this->prop = $prop;
   }
}

$test = new Test("foobar");
// Legal read.
var_dump($test->prop); // string(6) "foobar"
```

```
// Modificação ilegal. Não importa que o valor seja o mesmo.
$test->prop = "foobar";
// Error: Cannot modify readonly property Test::$prop
?>
```

> **Nota** :
>
> O modificador readonly somente pode ser aplicado a propriedades tipadas. Uma propriedade somente leitura sem restrições de tipo pode ser declarada utilizando o tipo Misto.

> **Nota** :
>
> Propriedades estáticas não são suportadas.

Uma propriedade somente leitura somente pode ser inicializada uma única vez, e somente do escopo onde ela foi declarada. Quaisquer outras modificações da propriedade resultam em um Error.

Exemplo #5 Inicilização ilegal de propriedades somente leitura.

```
<?php
class Test1 {
    public readonly string $prop;
}

$test1 = new Test1;
// Inicialização ilegal fora do escopo privado.
$test1->prop = "foobar";
// Error: Cannot initialize readonly property Test1::$prop from global scope
?>
```

> **Nota** :
>
> Especificar um valor default para uma propriedade somente leitura não é permitido, porque uma propriedade com um valor default é essencialmente uma constante, e portanto não tem outro propósito.
>
> ```
> <?php
>
> class Test {
>     // Fatal error: Readonly property Test::$prop cannot have default value
>     public readonly int $prop = 42;
> }
> ?>
> ```

> **Nota** :
>
> Propriedades somente leitura não podem ser unset() depois de inicializadas. Entretanto é possível realizar essa operação antes da inicialização, dentro do escopo onde a variável foi declarada.

Nenhum tipo de modificação é permitido, e o seguintes casos resultarão em um Error:

```php
<?php

class Test {
    public function __construct(
        public readonly int $i = 0,
        public readonly array $ary = [],
    ) {}
}

$test = new Test;
$test->i += 1;
$test->i++;
++$test->i;
$test->ary[] = 1;
$test->ary[0][] = 1;
unset($test->ary[0]);
$ref =& $test->i;
$test->i =& $ref;
byRef($test->i);
foreach ($test as &$prop);
?>
```

Entretanto, propriedades somente leitura não proíbem mutação. Objetos ou recursos armazenados em propriedades somente leitura ainda podem ser modificados internamente:

```php
<?php

class Test {
    public function __construct(public readonly object $obj) {}
}

$test = new Test(new stdClass);
// Modificação interna permitida
$test->obj->foo = 1;
// Reinicialização proibida.
$test->obj = new stdClass;
?>
```

A partir do PHP 8.3.0, propriedades somente-leitura podem ser reinicializadas ao clonar um objeto usando o método __clone().

Exemplo #6 Propriedades somente-leitura e clonagem

```php
<?php
class Test1 {
    public readonly ?string $prop;

    public function __clone() {
        $this->prop = null;
    }

    public function setProp(string $prop): void {
        $this->prop = $prop;
    }
```

```php
}

$test1 = new Test1;
$test1->setProp('foobar');

$test2 = clone $test1;
var_dump($test2->prop); // NULL
?>
```

> **Nota** :
>
> A partir do PHP 8.4.0, modificação indireta de uma propriedade somente-leitura dentro de __clone(), tal como a aquisição de uma referência a ela (`$ref = &$this->prop`), não é mais permitida. Isto já era proibido durante a inicialização somente-leitura.

Propriedades dinâmicas

Ao tentar atribuir uma propriedade inexistente em um object, o PHP irá automaticamente criar a propriedade. Essa propriedade criada dinamicamente será disponível *apenas* naquela instância específica.

> **Aviso**
>
> Propriedades dinâmicas foram descontinuadas a partir do PHP 8.2.0. É recomendado declarar todas as propriedades previamente. Para suportar nomes de propriedades arbitrárias, a classe deve implementar os métodos mágicos __get() e __set(). Como último recurso, a classe pode ser marcada com o atributo `#[\AllowDynamicProperties]`.

Ganchos de Propriedade

Ganchos de propriedade, também conhecidos como "acessadores de propriedade" em algumas outras linguagens, são uma forma de interceptar e substituir o comportamento de leitura e gravação de uma propriedade. Esta funcionalidade tem dois propósitos:

1. Permite que as propriedades sejam usadas diretamente, sem os métodos 'get' e 'set', deixando aberta a opção de incluir comportamento adicional no futuro. Isso torna desnecessários a maioria dos métodos 'get'/'set' padronizados, mesmo sem usar ganchos.

2. Permite propriedades que descrevem um objeto sem a necessidade de armazenar um valor diretamente.

Existem dois ganchos disponíveis em propriedades não estáticas: `get` e `set`. Eles permitem substituir o comportamento de leitura e gravação de uma propriedade, respectivamente. Os ganchos estão disponíveis para propriedades tipadas e não tipadas.

Uma propriedade pode ser "apoiada" ou "virtual". Uma propriedade apoiada é aquela que realmente armazena um valor. Qualquer propriedade que não tenha ganchos será apoiada. Uma propriedade virtual é aquela que possui ganchos e esses ganchos não interagem com a propriedade em si. Nesse caso, os ganchos são efetivamente iguais aos métodos, e o objeto não usa nenhum espaço para armazenar um valor para aquela propriedade.

Ganchos de propriedade são incompatíveis com propriedades `readonly` (somente-leitura). Se houver necessidade de restringir acesso a uma operação `get` ou `set` além de alterar seu comportamento, deve ser usada a visibilidade de propriedade assimétrica.

> **Nota** : **Informação de Versão**
>
> Ganchos de Propriedade foram introduzidos no PHP 8.4.

Sintaxe Básica de Gancho

A sintaxe geral para declarar um gancho é a seguinte.

Exemplo #1 Ganchos de propriedade (versão completa)

```php
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
        get {
            if ($this->modified) {
                return $this->foo . ' (modified)';
            }
            return $this->foo;
        }
        set(string $value) {
            $this->foo = strtolower($value);
            $this->modified = true;
        }
    }
}

$example = new Example();
$example->foo = 'changed';
print $example->foo;
?>
```

A propriedade *$foo* termina com `{}`, em vez de ponto e vírgula. Isso indica a presença de ganchos. Ambos os ganchos `get` e `set` são definidos, embora seja permitido definir apenas um ou outro. Ambos os ganchos possuem um corpo, indicado por `{}`, que pode conter código arbitrário.

O gancho `set` permite adicionalmente especificar o tipo e o nome de um valor recebido, usando a mesma sintaxe de um método. O tipo deve ser igual ao tipo da propriedade ou uma contravariante (mais amplo) a ela. Por exemplo, uma propriedade do tipo string poderia ter um gancho `set` que aceita string|Stringable, mas não aquele que aceita apenas array.

Pelo menos um dos ganchos faz referência a `$this->foo`, a própria propriedade. Isso significa que a propriedade será "apoiada". Ao chamar `$example->foo = 'changed'`, a string fornecida será primeiro convertida em minúsculas e depois salva no valor de apoio. Ao ler a propriedade, o valor salvo anteriormente pode ser anexado condicionalmente com texto adicional.

Existem também várias variantes de sintaxe abreviada para lidar com casos comuns.

Se o gancho `get` for uma expressão única, `{}` poderá ser omitido e substituído por uma expressão de seta.

Exemplo #2 Expressão para leitura de propriedade

Este exemplo é equivalente ao anterior.

```
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
        get => $this->foo . ($this->modified ? ' (modified)' : '');

        set(string $value) {
            $this->foo = strtolower($value);
            $this->modified = true;
        }
    }
}
?>
```

Se o tipo de parâmetro do gancho `set` for igual ao tipo de propriedade (o que é típico), ele poderá ser omitido. Nesse caso, o valor a ser definido recebe automaticamente o nome *$value*.

Exemplo #3 Padrão de definição de propriedade

Este exemplo é equivalente ao anterior.

```
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
        get => $this->foo . ($this->modified ? ' (modified)' : '');

        set {
            $this->foo = strtolower($value);
            $this->modified = true;
        }
    }
}
?>
```

Se o gancho `set` estiver apenas definindo uma versão modificada do valor passado, então ele também poderá ser simplificado para uma expressão de seta. O valor avaliado pela expressão será definido no valor de apoio.

Exemplo #4 Expressão para definição de propriedade

```
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
```

```
        get => $this->foo . ($this->modified ? ' (modified)' : '');
        set => strtolower($value);
    }
}
?>
```

Este exemplo não é exatamente equivalente ao anterior, pois também não modifica `$this->modified`. Se várias instruções forem necessárias no corpo do gancho definido, use a versão entre chaves.

Uma propriedade pode implementar zero, um ou ambos os ganchos conforme a situação exigir. Todas as versões abreviadas são mutuamente independentes. Ou seja, usar um 'get' curto com um 'set' longo, ou um 'set' curto com um tipo explícito, ou assim por diante, é válido.

Em uma propriedade apoiada, omitir um gancho `get` ou `set` significa que o comportamento padrão de leitura ou gravação será usado.

> **Nota** :
>
> Ganchos podem ser definidos ao usar promoção de propriedade de construtor. No entanto, quando isso é feito, os valores fornecidos ao construtor precisam ter o tipo associado à propriedade correspondido, independentemente do que o gancho `set` possa permitir.
>
> Considere o seguinte:
>
> ```
> <?php
> class Example
> {
>     public function __construct(
>         public private(set) DateTimeInterface $created {
>             set (string|DateTimeInterface $value) {
>                 if (is_string($value)) {
>                     $value = new DateTimeImmutable($value);
>                 }
>                 $this->created = $value;
>             }
>         },
>     ) {
>     }
> }
> ```
>
> Internamente, o mecanismo decompõe para o seguinte:
>
> ```
> <?php
> class Example
> {
>     public private(set) DateTimeInterface $created {
>         set (string|DateTimeInterface $value) {
>             if (is_string($value)) {
>                 $value = new DateTimeImmutable($value);
>             }
>             $this->created = $value;
>         }
>     }
> ```

```
    public function __construct(
        DateTimeInterface $created,
    ) {
        $this->created = $created;
    }
}
```

Quaisquer tentativas de definir a propriedade fora do construtor permitirão valores do tipo string ou DateTimeInterface mas o construtor permitirá apenas DateTimeInterface. Isto acontece porque o tipo definido para a propriedade (DateTimeInterface) é usado como o tipo do parâmetro dentro da assinatura do construtor, não importando o que o gancho `set` permite.

Se este tipo de comportamento do construtor for necessário, a promoção de propriedade no construtor não pode ser usada.

Propriedades virtuais

Propriedades virtuais são propriedades que não possuem valor de apoio. Uma propriedade é virtual se nem seu gancho `get` nem seu gancho `set` fizerem referência à própria propriedade usando sintaxe exata. Ou seja, uma propriedade chamada `$foo` cujo gancho contém `$this->foo` será apoiada. Mas a seguinte não é uma propriedade apoiada e causará erro:

Exemplo #5 Propriedade virtual inválida

```php
<?php
class Example
{
    public string $foo {
        get {
            $temp = __PROPERTY__;
            return $this->$temp; // Não faz referência a $this->foo, por isso não é válida.
        }
    }
}
?>
```

Para propriedades virtuais, se um gancho for omitido, essa operação não existe e tentar usá-la produzirá um erro. As propriedades virtuais não ocupam espaço de memória em um objeto. As propriedades virtuais são adequadas para propriedades "derivadas", como aquelas que são a combinação de duas outras propriedades.

Exemplo #6 Propriedade virtual

```php
<?php
class Rectangle
{
    // Uma propriedade virtual.
    public int $area {
        get => $this->h * $this->w;
    }

    public function __construct(public int $h, public int $w) {}
}

$s = new Rectangle(4, 5);
```

```
print $s->area; // exibe 20
$s->area = 30; // Erro, pois não há operação 'set' definida.
?>
```

Definir ambos os ganchos `get` e `set` em uma propriedade virtual também é permitido.

Escopo

Todos os ganchos operam no escopo do objeto que está sendo modificado. Isso significa que eles têm acesso a todos os métodos públicos, privados ou protegidos do objeto, bem como a quaisquer propriedades públicas, privadas ou protegidas, incluindo propriedades que possam ter seus próprios ganchos de propriedade. Acessar outra propriedade de dentro de um gancho não ignora os ganchos definidos nesta propriedade.

A implicação mais notável disso é que ganchos não triviais podem fazer subchamadas a um método arbitrariamente complexo, se desejarem.

Exemplo #7 Chamando um método a partir de um gancho

```
<?php
class Person {
    public string $phone {
        set => $this->sanitizePhone($value);
    }

    private function sanitizePhone(string $value): string {
        $value = ltrim($value, '+');
        $value = ltrim($value, '1');

        if (!preg_match('/\d\d\d\-\d\d\d\-\d\d\d\d/', $value)) {
            throw new \InvalidArgumentException();
        }
        return $value;
    }
}
?>
```

Referências

Como a presença de ganchos intercepta o processo de leitura e gravação de propriedades, eles causam problemas ao adquirir uma referência a uma propriedade ou com modificação indireta, como `$this->arrayProp['chave'] = 'valor';`. Isso ocorre porque qualquer tentativa de modificação do valor por referência ignoraria um gancho 'set', se algum estiver definido.

No caso raro de ser necessário obter uma referência a uma propriedade que tenha ganchos definidos, o gancho `get` pode ser prefixado com `&` para fazer com que ele retorne por referência. Definir `get` e `&get` na mesma propriedade é um erro de sintaxe.

Não é permitido definir ganchos `&get` e `set` em uma propriedade apoiada. Conforme observado acima, escrever no valor retornado por referência ignoraria o gancho `set`. Nas propriedades virtuais, não há nenhum valor comum necessário compartilhado entre os dois ganchos, portanto, é permitido definir ambos.

Escrever em um índice de uma propriedade de array também envolve uma referência implícita. Por esse motivo, gravar em uma propriedade de array apoiada com ganchos definidos é permitido se e somente se ela

definir apenas um gancho `&get`. Em uma propriedade virtual, escrever no array retornado de `get` ou `&get` é legal, mas se isso terá algum impacto no objeto depende da implementação do gancho.

Substituir toda a propriedade do array é aceitável e se comporta da mesma forma que qualquer outra propriedade. Somente trabalhar com elementos do array é que requer cuidados especiais.

Herança

Ganchos finais

Um gancho pode também ser declarado como final, neste caso ele não pode ser substituído.

Exemplo #8 Ganchos finais

```php
<?php
class User
{
    public string $username {
        final set => strtolower($value);
    }
}

class Manager extends User
{
    public string $username {
        // Isto é permitido
        get => strtoupper($this->username);

        // Mas isto NÃO é permitido, porque 'set' é final na classe superior.
        set => strtoupper($value);
    }
}
?>
```

Uma propriedade também pode ser declarada como final. Uma propriedade final não pode ser declarada novamente por uma classe filha de forma alguma, o que impede a alteração de ganchos ou a ampliação de seu acesso.

Declarar ganchos como finais em uma propriedade declarada como final é redundante e será silenciosamente ignorado. Este é o mesmo comportamento dos métodos finais.

Uma classe filha pode definir ou redefinir ganchos individuais em uma propriedade redefinindo a propriedade e apenas os ganchos que deseja substituir. Uma classe filha também pode adicionar ganchos a uma propriedade que não possui nenhum. Isto é, essencialmente, como se os ganchos fossem métodos.

Exemplo #9 Herança de gancho

```php
<?php
class Point
{
    public int $x;
    public int $y;
}

class PositivePoint extends Point
```

```php
{
    public int $x {
        set {
            if ($value < 0) {
                throw new \InvalidArgumentException('Muito pequeno');
            }
            $this->x = $value;
        }
    }
}
?>
```

Cada gancho substitui as implementações pai independentemente umas das outras. Se uma classe filha adicionar ganchos, qualquer valor padrão definido na propriedade será removido e deverá ser declarado novamente. Isso é consistente com o modo como a herança funciona em propriedades sem gancho.

Acessando ganchos pais

Um gancho em uma classe filha pode acessar a propriedade da classe pai usando a palavra-chave `parent::$prop`, seguida pelo gancho desejado. Por exemplo, `parent::$propName::get()`. Pode ser lido como "acessar a *prop* definida na classe pai e então executar sua operação 'get'" (ou operação 'set', conforme apropriado).

Se não for acessado dessa forma, o gancho da classe pai será ignorado. Esse comportamento é consistente com o funcionamento de todos os métodos. Isso também oferece uma maneira de acessar o armazenamento da classe pai, se houver. Se não houver gancho na propriedade pai, seu comportamento 'get'/'set' padrão será usado. Os ganchos não podem acessar nenhum outro gancho, exceto seu próprio pai em sua própria propriedade.

O exemplo acima poderia ser reescrito da seguinte maneira, que permitiria que a classe `Point` adicionasse seu próprio grancho `set` no futuro sem problemas (no exemplo anterior, um gancho adicionado à classe pai seria ignorado na classe filha).

Exemplo #10 Acessando gancho pai ('set')

```php
<?php
class Point
{
    public int $x;
    public int $y;
}

class PositivePoint extends Point
{
    public int $x {
        set {
            if ($value < 0) {
                throw new \InvalidArgumentException('Muito pequeno');
            }
            parent::$x::set($value);
        }
    }
}
?>
```

Um exemplo de substituição apenas de um gancho 'get' poderia ser:

Exemplo #11 Acessando gancho pai ('get')

```php
<?php
class Strings
{
    public string $val;
}

class CaseFoldingStrings extends Strings
{
    public bool $uppercase = true;

    public string $val {
        get => $this->uppercase
            ? strtoupper(parent::$val::get())
            : strtolower(parent::$val::get());
    }
}
?>
```

Serialização

O PHP possui diversas maneiras diferentes pelas quais um objeto pode ser serializado, seja para consumo público ou para fins de depuração. O comportamento dos ganchos varia dependendo do caso de uso. Em alguns casos, o valor bruto de apoio de uma propriedade será usado, ignorando quaisquer ganchos. Em outros, a propriedade será lida ou escrita "através" do gancho, assim como qualquer outra ação normal de leitura/gravação.

- var_dump(): Usa valor bruto
- serialize(): Usa valor bruto
- unserialize(): Usa valor bruto
- __serialize()/__unserialize(): Lógica customizada, usa gancho 'get'/'set'
- Conversão de array: Usa valor bruto
- var_export(): Usa gancho 'get'
- json_encode(): Usa gancho 'get'
- JsonSerializable: Lógica customizada, usa gancho 'get'
- get_object_vars(): Usa gancho 'get'
- get_mangled_object_vars(): Usa valor bruto

Constantes de Classe

É possível definir constantes em cada classe separadamente. A visibilidade padrão de constantes de classe é `public`.

> **Nota** :
>
> Constantes de classe podem ser redefinidas em classes filhas. A partir do PHP 8.1.0, constantes de classe não podem ser redefinidas numa classe filha se ela for qualificada como final.

Também é possível que interfaces tenham constantes. Veja na documentação de interface os exemplos.

É possível referenciar a classe usando uma variável. O valor da variável não pode ser uma palavra-chave (e.g. `self`, `parent` e `static`).

Constantes de classe são alocadas por classe, e não em cada instância da classe.

A partir do PHP 8.3.0, constantes de classe podem ter um tipo escalar tal como `bool`, `int`, `float`, `string` ou mesmo `array`. Ao usar `array`, seu conteúdo só pode ser de outros tipos escalares.

Exemplo #1 Definindo e usando uma constante

```php
<?php
class MinhaClasse
{
    const CONSTANTE = 'valor constante';

    function mostrarConstante() {
        echo  self::CONSTANTE . "\n";
    }
}

echo MinhaClasse::CONSTANTE . "\n";

$classname = "MinhaClasse";
echo $classname::CONSTANTE;

$classe = new MinhaClasse();
$classe->mostrarConstante();

echo $classe::CONSTANTE;
?>
```

A constante especial **`::class`** permite a resolução de classes pelo seu nome completo em tempo de compilação, e isso é útil em classes com namespaces:

Exemplo #2 Exemplo com informação estática

```php
<?php
namespace foo {
    class bar {
    }

    echo bar::class; // foo\bar
}
?>
```

Exemplo #3 Exemplo de expressão constante

```php
<?php
const UM = 1;
class foo {
    const DOIS = UM * 2;
    const TRES = UM + self::DOIS;
    const FRASE = 'O valor de TRES é '.self::TRES;
}
?>
```

Exemplo #4 Modificadores de visibilidade em constantes de classe

```php
<?php
class Foo {
    public const BAR = 'bar';
    private const BAZ = 'baz';
}
echo Foo::BAR, PHP_EOL;
echo Foo::BAZ, PHP_EOL;
?>
```

Saída do exemplo acima no PHP 7.1:

```
bar

Fatal error: Uncaught Error: Cannot access private const Foo::BAZ in …
```

> **Nota** :
>
> Modificadores de visibilidade estão disponíveis desde o PHP 7.1.0.

Exemplo #5 Verificando variância de visibilidade de constantes de classe, a partir do PHP 8.3.0

```php
<?php

interface MyInterface
{
    public const VALUE = 42;
}

class MyClass implements MyInterface
{
    protected const VALUE = 42;
}
?>
```

Saída do exemplo acima no PHP 8.3:

```
Fatal error: Access level to MyClass::VALUE must be public (as in interface MyInterface) …
```

> **Nota** :

> A partir do PHP 8.3.0, a variância de visibilidade é verificada mais estritamente. Antes desta versão, a visibilidade de uma constante de classe poderia ser diferente da visibilidade da constante na interface implementada.

Exemplo #6 Sintaxe de busca de constante de classe, a partir do PHP 8.3.0

```php
<?php
class Foo {
    public const BAR = 'bar';
    private const BAZ = 'baz';
}

$name = 'BAR';
echo Foo::{$name}, PHP_EOL; // bar
?>
```

> **Nota** :
>
> A partir do PHP 8.3.0, constantes de classe podem ser buscadas dinamicamente usando uma variável.

Exemplo #7 Atribuindo tipos a constantes de classe, a partir do PHP 8.3.0

```php
<?php

class MyClass {
    public const bool MY_BOOL = true;
    public const int MY_INT = 1;
    public const float MY_FLOAT = 1.01;
    public const string MY_STRING = 'one';
    public const array MY_ARRAY = [self::MY_BOOL, self::MY_INT, self::MY_FLOAT, self::MY_STRING];
}

var_dump(MyClass::MY_BOOL);
var_dump(MyClass::MY_INT);
var_dump(MyClass::MY_FLOAT);
var_dump(MyClass::MY_STRING);
var_dump(MyClass::MY_ARRAY);
?>
```

Saída do exemplo acima no PHP 8.3:

```
bool(true)
int(1)
float(1.01)
string(3) "one"
array(4) {
  [0]=>
  bool(true)
  [1]=>
  int(1)
  [2]=>
```

```
  float(1.01)
  [3]=>
  string(3) "one"
}
```

Autocarregamento de Classes

Muitos programadores, ao desenvolver aplicações orientadas a objeto, criam um arquivo PHP para cada definição de classe. Um dos maiores contratempos é ter de escrever uma longa lista de "includes" no início de cada script (um para cada classe).

A função spl_autoload_register() registra qualquer número de autocarregadores, permitindo que classes e interfaces sejam automaticamente carregadas se elas não ainda não tiverem sido definidas. Ao registrar autocarregadores, o PHP tem uma última chance de carregar classes e interfaces antes que falhem com um erro.

Qualquer construção semelhante a uma classe pode ser carregada automaticamente da mesma maneira. Isso inclui classes, interfaces, traits e enumerações.

> **Cuidado**
>
> Antes do PHP 8.0.0, era possível usar __autoload() para fazer autocarregamento de classes e interfaces. No entanto, ela é uma alternativa menos flexível a spl_autoload_register() e __autoload() está descontinuada a partir do PHP 7.2.0, e removida a partir do PHP 8.0.0.

> **Nota** :
>
> spl_autoload_register() pode ser chamada várias vezes para registrar múltiplos autocarregadores. No entanto, lançar uma exceção em uma função de autocarregamento interrompe esse processo e não permitirá que outras funções de autocarregamento sejam executadas. Por esse motivo, o lançamento de exceções em funções de autocarregamento é altamente desencorajado.

Exemplo #1 Exemplo de autocarregamento

Esse exemplo tenta carregar as classes `MyClass1` e `MyClass2` dos arquivos *MyClass1.php* e *MyClass2.php*, respectivamente.

```php
<?php
spl_autoload_register(function ($class_name) {
    include $class_name . '.php';
});

$obj  = new MyClass1();
$obj2 = new MyClass2();
?>
```

Exemplo #2 Outro exemplo de autocarregamento

Este exemplo tenta carregar a interface `ITest`.

```php
<?php

spl_autoload_register(function ($name) {
    var_dump($name);
});

class Foo implements ITest {
}

/*
string(5) "ITest"

Fatal error: Interface 'ITest' not found in ...
*/
?>
```

Exemplo #3 Usando o autocarregador do Composer

» Composer gera um `vendor/autoload.php` que é configurado para carregar automaticamente pacotes gerenciados pelo Composer. Ao incluir este arquivo, esses pacotes podem ser usados sem nenhum trabalho adicional.

```php
<?php
require __DIR__ . '/vendor/autoload.php';

$uuid = Ramsey\Uuid\Uuid::uuid7();

echo "Gerado novo UUID -> ", $uuid->toString(), "\n";
?>
```

Veja também

- unserialize()
- unserialize_callback_func
- unserialize_max_depth
- spl_autoload_register()
- spl_autoload()
- __autoload()

Construtores e Destrutores

Construtores

function **__construct**(mixed `...$values = ""`): void

O PHP permite aos desenvolvedores declararem métodos construtores para as classes. Classes que têm um método construtor chamam o método a cada objeto recém criado, sendo apropriado para qualquer inicialização que o objeto necessite antes de ser utilizado.

> **Nota** :
>
> Construtores pais não são chamados implicitamente se a classe filha define um construtor. Para executar o construtor da classe pai, uma chamada a **parent::__construct()** dentro do construtor da classe filha é

> necessária. Se a classe filha não definir um construtor, será herdado da classe pai como um método normal (se não foi declarado como privado).

Exemplo #1 Construtoras em herança

```php
<?php
class BaseClass {
    function __construct() {
        print "In BaseClass constructor\n";
    }
}

class SubClass extends BaseClass {
    function __construct() {
        parent::__construct();
        print "In SubClass constructor\n";
    }
}

class OtherSubClass extends BaseClass {
    // inherits BaseClass's constructor
}

// In BaseClass constructor
$obj = new BaseClass();

// In BaseClass constructor
// In SubClass constructor
$obj = new SubClass();

// In BaseClass constructor
$obj = new OtherSubClass();
?>
```

Diferente de outros métodos, __construct() não precisa seguir as regras usuais de compatibilidade de assinatura em objetos derivados.

Construtores são métodos ordinários que são chamados durante a criação do objeto correspondente. Eles podem definir um número arbitrário de argumentos, quais podem ser obrigatórios, podem ter um tipo, e podem ter valores padrão. Argumentos de construtores são informados dentro de parênteses depois do nome da classe.

Exemplo #2 Utilizando argumentos de construtor

```php
<?php
class Point {
    protected int $x;
    protected int $y;

    public function __construct(int $x, int $y = 0) {
        $this->x = $x;
        $this->y = $y;
    }
}
```

```php
// Passagem de ambos os argumentos.
$p1 = new Point(4, 5);
// Passar somente o argumento obrigatório, $y terá o valor padrão zero.
$p2 = new Point(4);
// Com parâmetros nomeados (a partir do PHP 8.0):
$p3 = new Point(y: 5, x: 4);
?>
```

Se a classe não tem construtor, ou o construtor não tem argumentos obrigatórios, o parêntesis pode ser omitido.

Construtores em estilo antigo

Anteriormente ao PHP 8.0.0 as classes no namespace global interpretam um método com o mesmo nome da classe como sendo um construtor válido. Essa sintaxe foi descontinuada, e gerará um erro `E_DEPRECATED` embora ainda continue funcionando como um construtor. Se ambos o __construct() e um método homônimo da classe estiverem definidos, __construct() que será chamado.

Em classes dentro de namespaces, ou quaisquer classes a partir do PHP 8, um método homônimo ao nome da classe não tem um significado especial.

Sempre utilize __construct() em novos códigos.

Promoção de propriedades no construtor

A partir do PHP 8, parâmetros de construtores podem ser promovidos a uma propriedade do objeto. Isto é bastante comum quando parâmetros do construtor são informados em uma propriedade no construtor, sem nenhum processamento. Promoção de construtor fornece uma sintaxe reduzida para esse caso. O exemplo acima pode ser reescrito assim:

Exemplo #3 Utilizando promoção de propriedades no construtor

```php
<?php
class Point {
    public function __construct(protected int $x, protected int $y = 0) {
    }
}
```

Quanto um argumento de construtor inclui um modificador, o PHP interpreta como sendo uma propriedade de objeto e como um argumento do construtor, e assimilará o valor do argumento a uma propriedade de mesmo nome. O corpo do construtor pode estar vazio ou conter outras operações. Quaisquer operações serão executadas após os valores dos argumentos serem definidos nas respectivas propriedades.

Nem todos os argumentos são promovidos. É possível misturar os argumentos promovidos e não promovidos em qualquer ordem. Argumentos promovidos não têm impacto no código chamador do construtor.

> **Nota** :
>
> Utilizar um modificador de visibilidade (`public`, `protected` ou `private`) é o modo mais expressivo de gerar a promoção de propriedades, mas qualquer outro modificador (como por exemplo `readonly`) terá o mesmo efeito.

> **Nota** :
>
> Propriedades de objeto não podem ser do tipo callable dado uma ambiguidade que isto poderia introduzir. Argumentos promovidos, portanto, não podem ser do tipo callable. Quaisquer outra declaração de tipo é permitida.

> **Nota** :
>
> Propriedades promovidas são convertidas em uma propriedade normal assim como um parâmetro de função, portanto todas as restrições de nomeação de propriedades e parâmetros se aplicam.

> **Nota** :
>
> Atributos informados em um argumento promovido será replicado tanto na propriedade como no no argumento. Valores padrão de argumentos promovidos no construtor serão replicados apenas no argumento, mas não na propriedade.

New em inicializadores de parâmetros

A partir do PHP 8.1.0, objetos podem ser utilizados como valores padrão de parâmetros, variáveis estáticas e constantes globais, assim como argumentos de atributos. Novos objetos também podem ser passados na instrução define().

> **Nota** :
>
> Não é permitido nomes de classe não-string ou classes anônimas. Não é permitido o espalhamento de argumentos. Não é permitido o uso de expressões.

Exemplo #4 New em inicializações

```
<?php

// Permitido:
static $x = new Foo;

const C = new Foo;

function test($param = new Foo) {}

#[AnAttribute(new Foo)]
class Test {
    public function __construct(
        public $prop = new Foo,
    ) {}
}

// Não permitido, resulta em erro de compilação:
function test(
```

```
    $a = new (CLASS_NAME_CONSTANT)(), // Nome dinâmico de classe
    $b = new class {}, // Classe anônima
    $c = new A(...[]), // Espalhamento de argumento
    $d = new B($abc), // Expressão
) {}
?>
```

Métodos de criação estáticos

O PHP suporta apenas um único construtor por classe. Em alguns casos pode ser desejável de permitir a um objeto ser construído de maneiras diferentes, a partir de argumentos diferentes. O método recomendado para realizar isso é através de métodos estáticos, utilizados como empacotadores do construtor.

Exemplo #5 Utilizando métodos estáticos para construção

```
<?php
$some_json_string = '{ "id": 1004, "name": "Elephpant" }';
$some_xml_string = "<animal><id>1005</id><name>Elephpant</name></animal>";

class Product {

    private ?int $id;
    private ?string $name;

    private function __construct(?int $id = null, ?string $name = null) {
        $this->id = $id;
        $this->name = $name;
    }

    public static function fromBasicData(int $id, string $name): static {
        $new = new static($id, $name);
        return $new;
    }

    public static function fromJson(string $json): static {
        $data = json_decode($json, true);
        return new static($data['id'], $data['name']);
    }

    public static function fromXml(string $xml): static {
        $data = simplexml_load_string($xml);
        $new = new static();
        $new->id = (int) $data->id;
        $new->name = $data->name;
        return $new;
    }
}

$p1 = Product::fromBasicData(5, 'Widget');
$p2 = Product::fromJson($some_json_string);
$p3 = Product::fromXml($some_xml_string);

var_dump($p1, $p2, $p3);
```

O construtor pode ser privado ou protegido para evitar que ele seja chamado externamente. Nesses casos, apenas um construtor estático será capaz de instanciar a classe. Por estarem na mesma definição de classe, os

métodos estáticos são capazes de instanciar o objeto, mesmo em uma instância diferente. O construtor privado é opcional e pode não fazer sentido em todos os casos.

Os três métodos estáticos a seguir demonstram as maneiras diferentes de instanciar um objeto.

- `fromBasicData()` obtém os exatos parâmetros que são necessários, e então cria o objeto através da chamada do construtor e retorna o resultado
- `fromJson()` aceita uma string JSON, realiza algum pré-processamento para converter os dados no formato necessário ao construtor. Só então retorna o novo objeto.
- `fromXml()` aceita uma string XML, pré-processa, e então cria um objeto limpo. O construtor é chamado, mas com todos os parâmetros opcionais o método os ignora. Por fim, os valores nas propriedades do objeto são associados antes de retornar o resultado.

Nos três casos, a palavra chave `static` é convertida no nome da classe onde o código reside. Nesse caso a classe `Product`.

Destrutor

function **__destruct**(): void

O PHP introduz um conceito de destrutor similar ao de outras linguagens orientadas a objeto, como C++. O método destrutor será chamado assim que todas as referências a um objeto particular forem removidas ou quando o objeto for explicitamente destruído ou qualquer ordem na sequência de encerramento.

Exemplo #6 Exemplo de Destrutor

```php
<?php

class MyDestructableClass
{
    function __construct() {
        print "In constructor\n";
    }

    function __destruct() {
        print "Destroying " . __CLASS__ . "\n";
    }
}

$obj = new MyDestructableClass();
```

Assim como os construtores, os destrutores da classe pai não serão chamados implicitamente pelo PHP. Para executar o destrutor pai, deve-se fazer uma chamada explícita a **parent::__destruct()** no corpo do destrutor. Assim como construtores, uma classe filha pode herdar o destrutor caso não implemente um.

O destrutor será chamado mesmo se o script for terminado utilizando-se exit(). Chamar exit() em um destrutor irá impedir que as demais rotinas de encerramento executem.

Se um destrutor criar novas referências ao seu objeto, ele não será chamado uma segunda vez quando a contagem de referências atingir zero novamente ou durante a sequência de desligamento.

A partir do PHP 8.4.0, quando ciclos de coleta ocorrem durante a execução de uma Fiber, os destrutores de objetos programados para coleta são executados em uma Fiber separada, chamada `gc_destructor_fiber`. Se esta Fiber for suspensa, uma nova será criada para executar quaisquer destrutores restantes. O `gc_destructor_fiber` anterior não será mais referenciado pelo coletor de lixo e poderá ser coletado se não for

referenciado em outro lugar. Objetos cujo destrutor está suspenso não serão coletados até que o destrutor retorne ou a própria Fiber seja coletada.

> **Nota** :
>
> Destrutores chamados durante o encerramento da execução do script já enviaram os cabeçalhos HTTP. O diretório atual na fase de encerramento do script pode ser diferente em alguns SAPIs (e.g. Apache).

> **Nota** :
>
> Tentar disparar uma exceção em um destrutor (chamado no término do script), lançará um erro fatal.

Visibilidade

A visibilidade de uma propriedade ou método ou constante pode ser definida prefixando a declaração com as palavras-chave `public`, `protected` ou `private`. Membros de classes declarados como públicos podem ser acessados de qualquer lugar. Membros declarados como protegidos podem ser acessados na própria classe ou de classes referenciadas por herança acima. Membros declarados como privados só podem ser acessados por classes que definem esse membro.

Visibilidade de propriedades

Propriedades de classes podem ser declaradas como públicas, privadas ou protegidas. Propriedades declaradas sem visibilidade são definidas como públicas.

Exemplo #1 Declaração de propriedade

```php
<?php
/**
 * Define MinhaClasse
 */
class MinhaClasse
{
    public $publica = 'Public';
    protected $protegida = 'Protected';
    private $privada = 'Private';

    function imprimeAlo()
    {
        echo $this->publica;
        echo $this->protegida;
        echo $this->privada;
    }
}

$obj = new MinhaClasse();
echo $obj->publica; // Funciona
echo $obj->protegida; // Erro Fatal
echo $obj->privada; // Erro Fatal
$obj->imprimeAlo(); // Mostra Public, Protected e Private
```

```php
/**
 * Define MinhaClasse2
 */
class MinhaClasse2 extends MinhaClasse
{
    // É possível redeclarar as propriedades públicas e protegidas, mas não as privadas
    public $publica = 'Public2';
    protected $protegida = 'Protected2';

    function imprimeAlo()
    {
        echo $this->publica;
        echo $this->protegida;
        echo $this->privada;
    }
}

$obj2 = new MinhaClasse2();
echo $obj2->publica;   // Funciona
echo $obj2->protegida; // Erro fatal
echo $obj2->privada;   // Não definida
$obj2->imprimeAlo(); // Mostra Public, Protected2, Undefined

?>
```

Visibilidade de propriedade assimétrica

A partir do PHP 8.4, as propriedades de objeto também podem ter sua visibilidade definida de forma assimétrica, com escopo diferente para leitura (`get`) e escrita (`set`). Especificamente, a visibilidade `set` pode ser especificada separadamente, desde que não seja mais permissiva que a visibilidade padrão.

Exemplo #2 Visibilidade de propriedade assimétrica

```php
<?php
class Book
{
    public function __construct(
        public private(set) string $title,
        public protected(set) string $author,
        protected private(set) int $pubYear,
    ) {}
}

class SpecialBook extends Book
{
    public function update(string $author, int $year): void
    {
        $this->author = $author; // OK
        $this->pubYear = $year; // Erro fatal
    }
}

$b = new Book('How to PHP', 'Peter H. Peterson', 2024);

echo $b->title; // Funciona
echo $b->author; // Funciona
echo $b->pubYear; // Erro fatal
```

```php
$b->title = 'How not to PHP'; // Erro fatal
$b->author = 'Pedro H. Peterson'; // Erro fatal
$b->pubYear = 2023; // Erro fatal
?>
```

A partir do PHP 8.5, a visibilidade `set` também pode ser aplicada a propriedades estáticas de classes.

Exemplo #3 Visibilidade assimétrica de propriedade estática

```php
<?php
class Manager
{
    public private(set) static int $calls = 0;

    public function doAThing(): string
    {
        self::$calls++;
        // Faz outras coisas.
        return "uma string";
    }
}

$m = new Manager();

$m->doAThing(); // Funciona
echo Manager::$calls; // Funciona
Manager::$calls = 5; // Erro fatal
?>
```

O exemplo acima produzirá:

```
1
Fatal error: Uncaught Error: Cannot modify private(set) property Manager::$calls from global
scope in /algum/arquivo.php
```

Existem algumas advertências em relação à visibilidade assimétrica:

- Somente propriedades tipadas podem ter uma visibilidade `set` separada.
- A visibilidade `set` deve ser a mesma de `get` ou mais restritiva. Ou seja, `public protected(set)` e `protected protected(set)` são permitidos, mas `protected public(set)` causará um erro de sintaxe.
- Se uma propriedade for `public`, a visibilidade principal poderá ser omitida. Ou seja, `public private(set)` e `private(set)` terão o mesmo resultado.
- Uma propriedade com visibilidade `private(set)` é automaticamente `final` e não pode ser declarada novamente em uma classe filha.
- A obtenção de uma referência para uma propriedade segue a visibilidade `set`, não `get`. Isso ocorre porque uma referência pode ser usada para modificar o valor da propriedade.

- Da mesma forma, tentar escrever em uma propriedade de array envolve uma operação `get` e `set` internamente e, portanto, seguirá a visibilidade `set`, como é sempre a mais restritiva.

> **Nota** :
>
> Espaços não são permitidos na declaração de visibilidade "set". `private(set)` está correto. `private( set )` não está correto e resultará em erro de análise.

Quando uma classe estende outra, a classe filha pode redefinir qualquer propriedade que não seja `final`. Ao fazer isso, pode ampliar a visibilidade principal ou a visibilidade `set`, desde que a nova visibilidade seja igual ou mais ampla que a classe pai. Entretanto, esteja ciente de que se uma propriedade `private` for substituída, ela na verdade não altera a propriedade da classe pai, mas cria uma nova propriedade com um nome interno diferente.

Exemplo #4 Herança de propriedade assimétrica

```php
<?php
class Book
{
    protected string $title;
    public protected(set) string $author;
    protected private(set) int $pubYear;
}

class SpecialBook extends Book
{
    public protected(set) string $title; // OK, pois a leitura é mais ampla e a escrita igual.
    public string $author; // OK, pois a leitura é a mesma e a escrita é mais ampla.
    public protected(set) int $pubYear; // Erro fatal. propriedades private(set) são finais.
}
?>
```

Visibilidades de métodos

Métodos de classe podem ser definidos como público, privado ou protegido. Métodos sem qualquer declaração explícita serão definidos como público.

Exemplo #5 Declaração de método

```php
<?php
/**
 * Define MinhaClasse
 */
class MinhaClasse
{
    // Declara um construtor público
    public function __construct() { }

    // Declara um método público
    public function MeuPublico() { }

    // Declara um método protegido
    protected function MeuProtegido() { }
```

```php
    // Declara um método privado
    private function MeuPrivado() { }

    // Esse é público
    function Foo()
    {
        $this->MeuPublico();
        $this->MeuProtegido();
        $this->MeuPrivado();
    }
}

$minhaclasse = new MinhaClasse;
$minhaclasse->MeuPublico(); // Funciona
$minhaclasse->MeuProtegido(); // Erro Fatal
$minhaclasse->MeuPrivado(); // Erro Fatal
$minhaclasse->Foo(); // Public, Protected e Private funcionam


/**
 * Define MinhaClasse2
 */
class MinhaClasse2 extends MinhaClasse
{
    // Esse é public
    function Foo2()
    {
        $this->MeuPublico();
        $this->MeuProtegido();
        $this->MeuPrivado(); // Erro Fatal
    }
}

$minhaclasse2 = new MinhaClasse2;
$minhaclasse2->MeuPublico(); // Funciona
$minhaclasse2->Foo2(); // Public e Protected funcionam, Private não

class Bar
{
    public function test() {
        $this->testPrivate();
        $this->testPublic();
    }

    public function testPublic() {
        echo "Bar::testPublic\n";
    }

    private function testPrivate() {
        echo "Bar::testPrivate\n";
    }
}

class Foo extends Bar
{
    public function testPublic() {
        echo "Foo::testPublic\n";
```

```
    }

    private function testPrivate() {
        echo "Foo::testPrivate\n";
    }
}

$myFoo = new foo();
$myFoo->test(); // Bar::testPrivate
                // Foo::testPublic
?>
```

Visibilidade de constantes

A partir do PHP 7.1.0, constantes de classe podem ser definidas como públicas, protegidas ou privadas. Constantes declaradas sem visibilidade explícita são definidas como públicas.

Exemplo #6 Declaração de constantes (desde o PHP 7.1.0)

```
<?php
/**
 * Define MyClass
 */
class MyClass
{
    // Declara uma constante pública
    public const MY_PUBLIC = 'public';

    // Declara uma constante protegida
    protected const MY_PROTECTED = 'protected';

    // Declara uma constante privada
    private const MY_PRIVATE = 'private';

    public function foo()
    {
        echo self::MY_PUBLIC;
        echo self::MY_PROTECTED;
        echo self::MY_PRIVATE;
    }
}

$myclass = new MyClass();
MyClass::MY_PUBLIC;    // Funciona
MyClass::MY_PROTECTED; // Erro fatal
MyClass::MY_PRIVATE;   // Erro fatal
$myclass->foo();       // Todos os acessos funcionam dentro do método


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // Isto é público
    function foo2()
    {
```

```
        echo self::MY_PUBLIC;
        echo self::MY_PROTECTED;
        echo self::MY_PRIVATE; // Fatal Error
    }
}

$myclass2 = new MyClass2;
echo MyClass2::MY_PUBLIC; // Funciona
$myclass2->foo2(); // Public e Protected funcionam, mas não Private
?>
```

Visibilidade de outros objetos

Objetos do mesmo tipo terão acesso a outros mementos privados e protegidos mesmo que não sejam da mesma instância. Isso acontece por que os detalhes específicos de implementação já são conhecidos dentro destes objetos.

Exemplo #7 Acessando membros privados entre objetos do mesmo tipo

```
<?php
class Test
{
    private $foo;

    public function __construct($foo)
    {
        $this->foo = $foo;
    }

    private function bar()
    {
        echo 'Acessou o método privado.';
    }

    public function baz(Test $other)
    {
        // Pode-se alterar a propriedade privada:
        $other->foo = 'hello';
        var_dump($other->foo);

        // Pode-se chamar método privado:
        $other->bar();
    }
}

$test = new Test('test');

$test->baz(new Test('other'));
?>
```

O exemplo acima produzirá:

```
string(5) "hello"
```

```
Acessou o método privado.
```

Herança de Objetos

Herança é um conceito de programação estabelecido, e o PHP faz uso deste em seu modelo de objetos. Este princípio afeta a forma com que classes e objetos se relacionam com outras.

Por exemplo, ao estender uma classe, a subclasse herda todos os métodos públicos e protegidos, propriedades e constantes da classe pai. A não que uma classe sobrescreva estes métodos, eles manterão sua funcionalidade original.

Isto é útil para definir uma funcionalidade abstrata, e permitir a implementação de uma funcionalidade adicional em objetos similares sem a necessidade de reimplementar todas as funcionalidades compartilhadas.

Os métodos privados de uma classe pai não são acessíveis a uma classe filha. Como resultado, as classes filhas podem reimplementar um método privado sem levar em conta as regras normais de herança. Antes do PHP 8.0.0, entretanto, as restrições `final` e `static` eram aplicadas aos métodos privados. Desde o PHP 8.0.0, a única restrição de método privado que é aplicada é `private final` para construtores, já que essa é uma maneira comum de "desabilitar" o construtor ao usar métodos de fábrica estáticos.

A visibilidade de métodos, propriedades e constantes pode ser relaxada, por exemplo, um método com `protected` pode ser marcado como `public`, mas não pode ser restrito, por exemplo, marcar uma propriedade que tenha `public` como `private`. Uma exceção são os construtores, cuja visibilidade pode ser restrita, ou seja, um construtor `public` pode ser declarado como `private` numa classe filha.

> **Nota** :
>
> A não ser que o autoload seja usado, as classes devem ser definidas antes de utilizadas. Se uma classe estende outra, a classe pai deve ser declarada antes da estrutura da classe filha. Esta regra se aplica a classes que herdam outras classes e interfaces.

> **Nota** :
>
> Não é permitido sobrescrever uma propriedade de leitura/gravação com um propriedade somente de leitura ou vice-versa.
>
> ```php
> <?php
>
> class A {
>     public int $prop;
> }
> class B extends A {
>     // Ilegal: leitura-gravação -> somente-leitura
>     public readonly int $prop;
> }
> ?>
> ```

Exemplo #1 Exemplo de Herança

```php
<?php

class Foo
{
    public function printItem($string)
    {
        echo 'Foo: ' . $string . PHP_EOL;
    }

    public function printPHP()
    {
        echo 'PHP é ótimo' . PHP_EOL;
    }
}

class Bar extends Foo
{
    public function printItem($string)
    {
        echo 'Bar: ' . $string . PHP_EOL;
    }
}

$foo = new Foo();
$bar = new Bar();
$foo->printItem('baz'); // Saída: 'Foo: baz'
$foo->printPHP();       // Saída: 'PHP é ótimo'
$bar->printItem('baz'); // Saída: 'Bar: baz'
$bar->printPHP();       // Saída: 'PHP é ótimo'

?>
```

Compatibilidade de Tipo de Retorno com Classes Internas

Antes do PHP 8.1, a maioria das classes ou métodos internos não declaravam seus tipos de retorno, e qualquer tipo de retorno era permitido ao estendê-los.

A partir do PHP 8.1.0, a maioria dos métodos internos começaram a declarar seu tipo de retorno de forma "experimental", nesse caso o tipo de retorno dos métodos deve ser compatível com o pai sendo estendido, do contrário, um aviso de descontinuação é emitido. Note que a ausência de uma declaração de retorno explícita também é considerada uma incompatibilidade de assinatura e, portanto, resulta no aviso de descontinuação.

Se o tipo de retorno não puder ser declarado para um método que sobrepõe devido à preocupações com compatibilidade entre versões PHP, um atributo ReturnTypeWillChange pode ser adicionado para silenciar o aviso de descontinuação.

Exemplo #2 O método que sobrepõe não declara nenhum tipo de retorno

```php
<?php
class MeuDateTime extends DateTime
{
    public function modify(string $modifier) { return false; }
}
```

```
// "Deprecated: Return type of MeuDateTime::modify(string $modifier) should either be compatible
with DateTime::modify(string $modifier): DateTime|false, or the #[\ReturnTypeWillChange] attribute
should be used to temporarily suppress the notice" a partir do PHP 8.1.0
?>
```

Exemplo #3 O método que sobrepõe declara um tipo de retorno errado

```
<?php
class MeuDateTime extends DateTime
{
    public function modify(string $modifier): ?DateTime { return null; }
}

// "Deprecated: Return type of MeuDateTime::modify(string $modifier): ?DateTime should either be
compatible with DateTime::modify(string $modifier): DateTime|false, or the #[\ReturnTypeWillChange]
attribute should be used to temporarily suppress the notice" a partir do PHP 8.1.0
?>
```

Exemplo #4 O método que sobrepõe declara um tipo de retorno errado sem um aviso de descontinuação

```
<?php
class MeuDateTime extends DateTime
{
    /**
     * @return DateTime|false
     */
    #[\ReturnTypeWillChange]
    public function modify(string $modifier) { return false; }
}

// Nenhum aviso é emitido
?>
```

Operador de Resolução de Escopo (::)

O Operador de Resolução de Escopo (também chamado de Paamayim Nekudotayim), ou em termos mais simples, dois pontos duplos, é um símbolo que permite acesso a uma constante, a uma propriedade estática, ou a um método estático de uma classe ou a um dos pais dessa classe. Além disso, propriedades ou métodos estáticos podem ser substituídos através de vinculação estática tardia.

Ao referenciar estes itens do lado de fora da definição da classe, use o nome da classe.

É possível referenciar a classe usando uma variável. O valor da variável não pode ser uma palavra-chave (por exemplo, `self`, `parent` e `static`).

Paamayim Nekudotayim pode parecer, de início, uma escolha estranha para chamar dois pontos duplos. No entanto, na hora de escrever o Motor Zend 0.5 (que equipa o PHP 3), foi como a equipe da Zend decidiu chamá-lo. Na verdade, vem do hebraico e significa literalmente "duplo dois pontos".

Exemplo #1 :: de fora da definição da classe

```
<?php
class MyClass {
```

```
    const CONST_VALUE = 'Um valor constante';
}

$classname = 'MyClass';
echo $classname::CONST_VALUE;

echo MyClass::CONST_VALUE;
?>
```

Três palavras-chave especiais, *self*, *parent* e *static*, são utilizadas para acessar propriedades e métodos de dentro de uma definição de classe.

Exemplo #2 :: de dentro da definição da classe

```
<?php
class MyClass {
    const CONST_VALUE = 'Um valor constante';
}

class OtherClass extends MyClass
{
    public static $my_static = 'variável estática';

    public static function doubleColon() {
        echo parent::CONST_VALUE . "\n";
        echo self::$my_static . "\n";
    }
}

$classname = 'OtherClass';
echo $classname::doubleColon();

OtherClass::doubleColon();
?>
```

Quando uma subclasse substitui a definição de um método do pai, o PHP não chamará o método pai. Fica a cargo da subclasse chamar o método pai ou não. Isso também se aplica a definições de métodos Construtores e Destrutores, de Sobrecarga e Mágicos.

Exemplo #3 Chamando um método pai

```
<?php
class MyClass
{
    protected function myFunc() {
        echo "MyClass::myFunc()\n";
    }
}

class OtherClass extends MyClass
{
    // Substitui a definição do método pai
    public function myFunc()
    {
        // Mas ainda consegue chamar o método da classe herdada
        parent::myFunc();
```

```
        echo "OtherClass::myFunc()\n";
    }
}

$class = new OtherClass();
$class->myFunc();
?>
```

Veja também alguns exemplos de chamadas estáticas mais complexas.

Palavra-Chave 'static'

> **Dica**
>
> Esta página descreve o uso da palavra-chave `static` na definição de métodos e propriedades estáticas. A palavra-chave `static` também pode ser utilizada para definir variáveis estáticas, definir funções anônimas estáticas e em late static bindings. Veja essas páginas para informações desses outros usos de `static`.

Declarar propriedades ou métodos de uma classe como estáticos faz deles acessíveis sem a necessidade de instanciar a classe. Um membro declarado como estático não pode ser acessado através de uma instância da classe.

Métodos estáticos

Como métodos estáticos podem ser chamados sem uma instância do objeto criada, a pseudo-variável *$this* não está disponível dentro de um método declarado como estático.

> **Aviso**
>
> Chamar métodos não estáticos, estaticamente, lança um Error.
>
> Anteriormente ao PHP 8.0.0, chamar um método não estático estaticamente estava descontinuado, e gerava um aviso **E_DEPRECATED**.

Exemplo #1 Exemplo de método estático

```
<?php
class Foo {
    public static function aStaticMethod() {
        // ...
    }
}

Foo::aStaticMethod();
$classname = 'Foo';
$classname::aStaticMethod();
?>
```

Propriedades estáticas

Propriedades estáticas são acessíveis utilizando o Operador de Resolução de Escopo (`::`) e não podem ser acessados através do operador de objeto (`->`).

É possível referenciar a classe usando uma variável. O valor da variável não pode ser uma palavra chave (por exemplo, `self`, `parent` e `static`).

Exemplo #2 Exemplo de propriedade estática

```php
<?php
class Foo
{
    public static $my_static = 'foo';

    public function staticValue() {
        return self::$my_static;
    }
}

class Bar extends Foo
{
    public function fooStatic() {
        return parent::$my_static;
    }
}


print Foo::$my_static . "\n";

$foo = new Foo();
print $foo->staticValue() . "\n";
print $foo->my_static . "\n"; // "Propriedade" não definida, my_static

print $foo::$my_static . "\n";
$classname = 'Foo';
print $classname::$my_static . "\n";

print Bar::$my_static . "\n";
$bar = new Bar();
print $bar->fooStatic() . "\n";
?>
```

A saída do exemplo acima no PHP 8 é semelhante a:

```
foo
foo

Notice: Accessing static property Foo::$my_static as non static in /in/V0Rvv on line 23

Warning: Undefined property: Foo::$my_static in /in/V0Rvv on line 23

foo
foo
foo
foo
```

Abstração de Classes

O PHP possui métodos, classes e propriedades abstratas. Classes definidas como abstratas não podem ser instanciadas, e qualquer classe que contenha ao menos um método abstrato ou uma propriedade abstrata também deve ser abstrata. Métodos definidos como abstratos simplesmente declaram a assinatura e se ele é público ou protegido; eles não podem definir a implementação. Propriedades definidas como abstratas podem declarar um requisito para o comportamento de `get` ou `set`, e podem fornecer uma implementação para uma das operações, mas não para ambas.

Quando herdando de uma classe abstrata, todos os métodos marcados como abstratos da classe herdada precisam ser definidos na classe implementante. além de seguir as regras usuais de herança e compatibilidade de assinatura.

A partir do PHP 8.4, uma classe abstrata pode declarar uma propriedade abstrata, seja pública ou protegida. Uma propriedade abstrata protegida pode ser satisfeita por uma propriedade que pode ser lida/gravada no escopo protegido ou público.

Uma propriedade abstrata pode ser satisfeita por uma propriedade padrão ou por uma propriedade com ganchos definidos, correspondentes à operação necessária.

Exemplo #1 Exemplo de método abstrato

```php
<?php

abstract class ClasseAbstrata
{
    // Força a classe que estende ClasseAbstrata a definir esse método
    abstract protected function pegarValor();
    abstract protected function valorComPrefixo( $prefixo );

    // Método comum
    public function imprimir()
    {
        print $this->pegarValor();
    }
}

class ClasseConcreta1 extends ClasseAbstrata
{
    protected function pegarValor()
    {
        return "ClasseConcreta1";
    }

    public function valorComPrefixo( $prefixo )
    {
        return "{$prefixo}ClasseConcreta1";
    }
}

class ClasseConcreta2 extends ClasseAbstrata
{
    protected function pegarValor()
    {
        return "ClasseConcreta2";
    }
```

```
    public function valorComPrefixo( $prefixo )
    {
        return "{$prefixo}ClasseConcreta2";
    }
}

$classe1 = new ClasseConcreta1();
$classe1->imprimir();
echo $classe1->valorComPrefixo('FOO_'), "\n";

$classe2 = new ClasseConcreta2();
$classe2->imprimir();
echo $classe2->valorComPrefixo('FOO_'), "\n";

?>
```

O exemplo acima produzirá:

```
ConcreteClass1
FOO_ConcreteClass1
ConcreteClass2
FOO_ConcreteClass2
```

Exemplo #2 Exemplo de método abstrato

```
<?php

abstract class ClasseAbstrata
{
    // Um método abstrato precisa definir apenas os parâmetros requeridos
    abstract protected function prefixName($name);
}

class ClasseConcreta extends ClasseAbstrata
{
    // Uma classe filha pode definir parâmetros opcionais que não estão presentes na assinatura da
classe superior
    public function prefixName($name, $separator = ".")
    {
        if ($name == "Pacman") {
            $prefix = "Mr";
        } elseif ($name == "Pacwoman") {
            $prefix = "Mrs";
        } else {
            $prefix = "";
        }

        return "{$prefix}{$separator} {$name}";
    }
}
```

```php
$class = new ClasseConcreta();
echo $class->prefixName("Pacman"), "\n";
echo $class->prefixName("Pacwoman"), "\n";

?>
```

O exemplo acima produzirá:

```
Mr. Pacman
Mrs. Pacwoman
```

Exemplo #3 Exemplo de propriedade abstrata

```php
<?php

abstract class A
{
    // A extensão de classes deve ter uma propriedade que pode ser obtida publicamente.
    abstract public string $readable {
        get;
    }

    // A extensão de classes deve ter uma propriedade gravável protegida ou pública.
    abstract protected string $writeable {
        set;
    }

    // A extensão de classes deve ter uma propriedade simétrica protegida ou pública.
    abstract protected string $both {
        get;
        set;
    }
}

class C extends A
{
    // Isto satisfaz o requisito e também o torna configurável, o que é válido.
    public string $readable;

    // Isso NÃO satisfaria o requisito, pois não é legível publicamente.
    protected string $readable;

    // Isso satisfaz exatamente o requisito, portanto é suficiente.
    // Pode ser alterado, somente a partir do escopo protegido.
    protected string $writeable {
        set => $value;
    }

    // Isso expande a visibilidade de protegida para pública, o que é aceitável.
    public string $both;
}
```

```
?>
```

Uma propriedade abstrata em uma classe abstrata pode fornecer implementações para qualquer gancho, mas deve ter `get` ou `set` declarado, mas não definido (como no exemplo acima).

Exemplo #4 Exemplo de propriedade abstrata com ganchos

```php
<?php

abstract class A
{
    // Isso fornece uma implementação de conjunto padrão (mas substituível)
    // e requer que classes filhas forneçam uma implementação get.
    abstract public string $foo {
        get;

        set {
            $this->foo = $value
        }
    }
}

?>
```

Interfaces de Objetos

Interfaces de objetos permitem a criação de códigos que especificam quais métodos e propriedades uma classe deve implementar, sem definir como esses métodos ou propriedades serão implementados. Interfaces compartilham um namespace com classes, traits e enumerações, por isso eles não podem usar os mesmos nomes.

Interfaces são definidas da mesma forma que classes, mas com a palavra-chave `interface` substituindo `class` e com nenhum dos métodos tendo seu conteúdo definido.

Todos os métodos declarados em uma interface devem ser públicos, essa é a natureza de uma interface.

Na prática, interfaces servem a dois propósitos distintos:

- Elas permitem os desenvolvedores de criar objetos de classes diferentes que podem ser substituídos dado que eles implementam a mesma ou as mesmas interfaces. Um exemplo seriam os serviços variados de acesso a banco de dados, vários sistemas de pagamentos, ou estratégias de cache. Implementações diferentes podem ser substituídas sem requerer modificações nos códigos que as usam.
- Para permitir que uma função ou método aceite e opere em um parâmetro que se molda a uma interface, ao mesmo tempo que não se importa como a funcionalidade é implementada. Estas interfaces são conhecidas como `Iterable`, `Cacheable`, `Renderable`, e assim por diante, e descrevem o comportamento significativo.

Interfaces podem definir métodos mágicos para exigir que as classes implementantes também implementem esses métodos.

> **Nota** :

> Apesar de possível, incluir construtores em interfaces é altamente desencorajado. Fazer isso reduz significativamente a flexibilidade do objeto implementante da interface. Além disso, construtores não são verificados pelas regras de herança, o que pode causar comportamentos inconsistentes ou inesperados.

`implements`

Para implementar uma interface, o operador `implements` é utilizado. Todos os métodos na interface devem ser implementados na classe; não fazê-lo resultará em um erro fatal. Classes podem implementar mais de uma interface se assim for desejado, separando cada interface com uma vírgula.

> **Aviso**
>
> Uma classe que implemente uma interface pode utilizar um nome diferente para seus parâmetros, em relação à interface. Entretanto, o PHP 8.0 suporta argumentos nomeados, ou seja, chamadores podem usar os nomes de parâmetros conforme definidos na interface implementada. Por essa razão, é altamente recomendado que desenvolvedores utilizem os mesmos nomes de parâmetros que da interface implementada.

> **Nota** :
>
> Interfaces podem ser estendidas como as classes, usando o operador extends.

> **Nota** :
>
> A classe que implementa a interface precisa declarar todos os métodos da interface com uma assinatura compatível. Uma classe pode implementar várias interfaces que declarem um método com o mesmo nome. Neste caso, a implementação precisa seguir as regras de compatibilidade de assinaturas de todas as interfaces. É possível assim aplicar covariância e contravariância.

Constantes

É possível ter constantes em interfaces. Constantes de interfaces funcionam exatamente como constantes de classes, com exceção de não podem ser sobrescritas por uma classe/interface herdeira.

Propriedades

A partir do PHP 8.4.0, as interfaces também podem declarar propriedades. Se o fizerem, a declaração deverá especificar se a propriedade deve ser legível, gravável ou ambas. A declaração da interface se aplica apenas ao acesso público de leitura e gravação.

Uma classe pode satisfazer uma propriedade de interface de diversas maneiras. Pode definir uma propriedade pública. Pode definir uma propriedade virtual pública que implementa apenas o gancho correspondente. Ou uma propriedade de leitura pode ser satisfeita por uma propriedade `somente leitura`. Entretanto, uma propriedade de interface configurável não pode ser `somente leitura`.

Exemplo #1 Exemplo de propriedades de interface

```
<?php
interface I
```

```php
{
    // Uma classe de implementação DEVE ter uma propriedade legível publicamente,
    // mas ser ou não configurável publicamente é irrestrito.
    public string $readable { get; }

    // Uma classe de implementação DEVE ter uma propriedade gravável publicamente,
    // mas ser ou não legível publicamente é irrestrito.
    public string $writeable { set; }

    // Uma classe de implementação DEVE ter uma propriedade que seja
    // tanto legível quanto gravável publicamente.
    public string $both { get; set; }
}

    // Esta classe implementa todas as três propriedades como propriedades tradicionais
    // e sem ganchos; Isso é totalmente válido.
class C1 implements I
{
    public string $readable;

    public string $writeable;

    public string $both;
}

// Esta classe implementa todas as três propriedades usando apenas os
// ganchos solicitados. Isto também é inteiramente válido.
class C2 implements I
{
    private string $written = '';
    private string $all = '';

    // Usa apenas um gancho get para criar uma propriedade virtual.
    // Isso satisfaz o requisito "public get".
    // Não é gravável, mas não é exigido pela interface.
    public string $readable { get => strtoupper($this->writeable); }

    // A interface requer apenas que a propriedade seja configurável,
    // mas incluir operações get também é totalmente válido.
    // Este exemplo cria uma propriedade virtual, o que é aceitável.
    public string $writeable {
        get => $this->written;
        set {
            $this->written = $value;
        }
    }

    // Esta propriedade exige que tanto a leitura quanto a gravação
    // sejam possíveis, portanto, precisamos implementar ambas ou
    // permitir que ela tenha o comportamento padrão.
    public string $both {
        get => $this->all;
        set {
            $this->all = strtoupper($value);
        }
    }
}
?>
```

Examples

Exemplo #2 Exemplo de Interface

```php
<?php

// Declara a interface 'Template'
interface Template
{
    public function setVariable($name, $var);
    public function getHtml($template);
}

// Implementa a interface
// Exemplo correto
class WorkingTemplate implements Template
{
    private $vars = [];

    public function setVariable($name, $var)
    {
        $this->vars[$name] = $var;
    }

    public function getHtml($template)
    {
        foreach($this->vars as $name => $value) {
            $template = str_replace('{' . $name . '}', $value, $template);
        }

        return $template;
    }
}

// Exemplo incorreto
// Fatal error: Class BadTemplate contains 1 abstract methods
// and must therefore be declared abstract (Template::getHtml)
class BadTemplate implements Template
{
    private $vars = [];

    public function setVariable($name, $var)
    {
        $this->vars[$name] = $var;
    }
}
?>
```

Exemplo #3 Interfaces extensíveis

```php
<?php
interface A
{
    public function foo();
}
```

```php
interface B extends A
{
    public function baz(Baz $baz);
}

// Isto funciona
class C implements B
{
    public function foo()
    {
    }

    public function baz(Baz $baz)
    {
    }
}

// Isso não funcionará, e gerará um erro fatal
class D implements B
{
    public function foo()
    {
    }

    public function baz(Foo $foo)
    {
    }
}
?>
```

Exemplo #4 Compatibilidade de variância com interfaces múltiplas

```php
<?php
class Foo {}
class Bar extends Foo {}

interface A {
    public function myfunc(Foo $arg): Foo;
}

interface B {
    public function myfunc(Bar $arg): Bar;
}

class MyClass implements A, B
{
    public function myfunc(Foo $arg): Bar
    {
        return new Bar();
    }
}
?>
```

Exemplo #5 Herança de várias interfaces

```php
<?php
interface A
{
    public function foo();
}

interface B
{
    public function bar();
}

interface C extends A, B
{
    public function baz();
}

class D implements C
{
    public function foo()
    {
    }

    public function bar()
    {
    }

    public function baz()
    {
    }
}
?>
```

Exemplo #6 Interfaces com constantes

```php
<?php
interface A
{
    const B = 'Constante de interface';
}

// Imprime: Constante de interface
echo A::B;


class B implements A
{
    const B = 'Constante de classe';
}

// Imprime: Constante de classe
// Anteriormente ao PHP 8.1.0 isto não funcionaria porque não era possível
// sobrescrever constantes.
echo B::B;
?>
```

Exemplo #7 Interfaces com classes abstratas

```php
<?php
interface A
{
    public function foo(string $s): string;

    public function bar(int $i): int;
}

// Uma classe abstrata pode implementar apenas uma parte da interface.
// Classes que estendem a classe abstrata precisam implementar o resto.
abstract class B implements A
{
    public function foo(string $s): string
    {
        return $s . PHP_EOL;
    }
}

class C extends B
{
    public function bar(int $i): int
    {
        return $i * 2;
    }
}
?>
```

Exemplo #8 Estendendo e implementando ao mesmo tempo

```php
<?php

class One
{
    /* ... */
}

interface Usable
{
    /* ... */
}

interface Updatable
{
    /* ... */
}

// A ordem de palavras chave aqui é importante. 'extends' precisa aparecer primeiro.
class Two extends One implements Usable, Updatable
{
    /* ... */
}
?>
```

Uma interface, juntamente com a declaração de tipo, fornecem uma boa maneira de garantir que um objeto em particular possua determinados métodos. Veja o operador instanceof e declaração de tipo.

Traits

O PHP implementa uma maneira de reúso de código chamada Traits.

Traits (Traços, Características) são um mecanismo para reúso de código em linguagens de herança simples como o PHP. A intenção de uma Trait é reduzir algumas limitações de herança simples permitindo que um desenvolvedor reutilize livremente conjuntos de métodos em várias classes independentes habitando em diferentes hierarquias de classe. A semântica da combinação de Traits e classes é definida de uma maneira que reduz a complexidade, e evita os problemas típicos associados com herança múltipla e Mixins.

Uma Trait é similar a uma classe, mas destina-se apenas a agrupar funcionalidade de uma forma refinada e consistente. Não é possível instanciar uma Trait por si só. Ela é uma adição à herança tradicional e permite composição e comportamento horizontais; isto é, a aplicação de membros de classe sem exigir herança.

Exemplo #1 Exemplo de Trait

```php
<?php

trait TraitA {
    public function sayHello() {
        echo 'Olá,';
    }
}

trait TraitB {
    public function sayWorld() {
        echo 'mundo';
    }
}

class MyHelloWorld
{
    use TraitA, TraitB; // Uma classe pode usar vários Traits

    public function sayHelloWorld() {
        $this->sayHello();
        echo ' ';
        $this->sayWorld();
        echo "!\n";
    }
}

$myHelloWorld = new MyHelloWorld();
$myHelloWorld->sayHelloWorld();

?>
```

O exemplo acima produzirá:

```
Olá, mundo!
```

Precedência

Um membro herdado de uma classe base é substituído por um membro inserido por uma Trait. Na ordem de precedência, os membros da classe atual substituem os métodos da Trait, que por sua vez substituem os métodos herdados.

Exemplo #2 Exemplo de Ordem de Precedência

Um membro herdado de uma classe base é substituído pelo método inserido em MyHelloWorld da Trait SayWorld. O comportamento é o mesmo para métodos definidos na classe MyHelloWorld. Na ordem de precedência, os métodos da classe atual sobrescrevem os métodos da Trait, que por sua vez substituem os métodos da classe base.

```php
<?php
class Base {
    public function sayHello() {
        echo 'Hello ';
    }
}

trait SayWorld {
    public function sayHello() {
        parent::sayHello();
        echo 'World!';
    }
}

class MyHelloWorld extends Base {
    use SayWorld;
}

$o = new MyHelloWorld();
$o->sayHello();
?>
```

O exemplo acima produzirá:

```
Hello World!
```

Exemplo #3 Exemplo de Ordem de Precedência Alternativa

```php
<?php
trait HelloWorld {
    public function sayHello() {
        echo 'Hello World!';
    }
}

class TheWorldIsNotEnough {
    use HelloWorld;
    public function sayHello() {
```

```
        echo 'Hello Universe!';
    }
}

$o = new TheWorldIsNotEnough();
$o->sayHello();
?>
```

O exemplo acima produzirá:

```
Hello Universe!
```

Traits Múltiplas

Traits múltiplas podem ser inseridas em uma classe, listando-as na declaração `use`, separadas por vírgulas.

Exemplo #4 Uso de Traits Múltiplas

```
<?php
trait Hello {
    public function sayHello() {
        echo 'Hello ';
    }
}

trait World {
    public function sayWorld() {
        echo 'World';
    }
}

class MyHelloWorld {
    use Hello, World;
    public function sayExclamationMark() {
        echo '!';
    }
}

$o = new MyHelloWorld();
$o->sayHello();
$o->sayWorld();
$o->sayExclamationMark();
?>
```

O exemplo acima produzirá:

```
Hello World!
```

Resolução de Conflitos

Se duas Trais inserem dois métodos com o mesmo nome, um erro fatal é gerado, se o conflito não for explicitamente resolvido.

Para resolver conflitos de nomes entre Traits usados na mesma classe, o operador `insteadof` deve ser usado para escolher exatamente um dos métodos conflitantes.

Como isto permite apenas excluir métodos, o operador `as` pode ser usado para adicionar um apelido a um dos métodos. Note que o operador `as` não renomeia o método e também não afeta nenhum outro método.

Exemplo #5 Exemplo de Resolução de Conflito

Neste exemplo, Talker usa as Traits A e B. Como A e B possuem métodos conflitantes, ela especifica que usa a variante de smallTalk da Trait B, e a variante bigTalk da Trait A.

A classe Aliased_Talker usa o operador `as` para perrmitir o uso da implementação bigTalk da Trait B sob um apelido adicional `talk`.

```
<?php
trait A {
    public function smallTalk() {
        echo 'a';
    }
    public function bigTalk() {
        echo 'A';
    }
}

trait B {
    public function smallTalk() {
        echo 'b';
    }
    public function bigTalk() {
        echo 'B';
    }
}

class Talker {
    use A, B {
        B::smallTalk insteadof A;
        A::bigTalk insteadof B;
    }
}

class Aliased_Talker {
    use A, B {
        B::smallTalk insteadof A;
        A::bigTalk insteadof B;
        B::bigTalk as talk;
    }
}
?>
```

Mudando a Visibilidade de Método

Ao utilizar a sintaxe `as`, pode-se também ajustar a visibilidade de métodos na classe.

Exemplo #6 Exemplo de Mudança de Visibilidade de Método

```php
<?php
trait HelloWorld {
    public function sayHello() {
        echo 'Hello World!';
    }
}

// Modifica a visibilidade de sayHello
class MyClass1 {
    use HelloWorld { sayHello as protected; }
}

// Apelido de método com visibilidade modificada
// Visibilidade de sayHello não é modificada
class MyClass2 {
    use HelloWorld { sayHello as private myPrivateHello; }
}
?>
```

Traits Compostas de Traits

Assim como as classes podem usar Trais, outros Traits também podem. Ao utilizar uma ou mais Traits em uma definição de Trait, ela pode ser composta parcialmente ou integralmente dos membros definidos nessas outras Traits.

Exemplo #7 Exemplo de Traits Compostas de Traits

```php
<?php
trait Hello {
    public function sayHello() {
        echo 'Hello ';
    }
}

trait World {
    public function sayWorld() {
        echo 'World!';
    }
}

trait HelloWorld {
    use Hello, World;
}

class MyHelloWorld {
    use HelloWorld;
}

$o = new MyHelloWorld();
$o->sayHello();
```

```php
$o->sayWorld();
?>
```

O exemplo acima produzirá:

```
Hello World!
```

Membros abstratos de Traits

Traits suportam o uso de métodos abstratos para impor requisitos sobre uma classe expositora. Métodos públicos, protegidos e privados são suportados. Antes do PHP 8.0.0, apenas métodos abstratos públicos e protegidos eram suportados.

> **Cuidado**
>
> A partir do PHP 8.0.0, a assinatura de um método concreto deverá seguir as regras de compatibilidade de assinaturas. Anteriormente, sua assinatura poderia ser diferente.

Exemplo #8 Requisitos Expressos por Métodos Abstratos

```php
<?php
trait Hello {
    public function sayHelloWorld() {
        echo 'Hello'.$this->getWorld();
    }
    abstract public function getWorld();
}

class MyHelloWorld {
    private $world;
    use Hello;
    public function getWorld() {
        return $this->world;
    }
    public function setWorld($val) {
        $this->world = $val;
    }
}
?>
```

Membros Estáticos de Traits

Traits podem definir variáveis estáticas, métodos estáticos e propriedades estáticas.

> **Nota** :
>
> A partir do PHP 8.1.0, chamar um método estático, ou acessar uma propriedade estática diretamente em uma Trait foi descontinuado. Métodos estáticos e propriedades somente devem ser acessados em uma

> classe utilizando a Trait.

Exemplo #9 Variáveis Estáticas

```php
<?php

trait Counter
{
    public function inc()
    {
        static $c = 0;
        $c = $c + 1;
        echo "$c\n";
    }
}

class C1
{
    use Counter;
}

class C2
{
    use Counter;
}

$o = new C1();
$o->inc();
$p = new C2();
$p->inc();

?>
```

O exemplo acima produzirá:

```
1
1
```

Exemplo #10 Métodos Estáticos

```php
<?php

trait StaticExample
{
    public static function doSomething()
    {
        return 'Fazendo algo';
    }
}

class Example
```

```
{
    use StaticExample;
}

echo Example::doSomething();

?>
```

O exemplo acima produzirá:

```
Fazendo algo
```

Exemplo #11 Propriedades Estáticas

> **Cuidado**
>
> Antes do PHP 8.3.0, propriedades estáticas definidas em um trait eram compartilhadas por todas as classes na mesma herança hierárquica que usavam o trait. A partir do PHP 8.3.0, se uma classe filha usa um trait com uma propriedade estática, ela será considerada distinta daquela definida na classe pai.

```
<?php

trait T
{
    public static $counter = 1;
}

class A
{
    use T;

    public static function incrementCounter()
    {
        static::$counter++;
    }
}

class B extends A
{
    use T;
}

A::incrementCounter();

echo A::$counter, "\n";
echo B::$counter, "\n";

?>
```

Saída do exemplo acima no PHP 8.3:

```
2
1
```

Propriedades

Traits podem também definir propriedades.

Exemplo #12 Definindo Propriedades

```php
<?php

trait PropertiesTrait
{
    public $x = 1;
}

class PropertiesExample
{
    use PropertiesTrait;
}

$example = new PropertiesExample();
$example->x;

?>
```

Se uma Trait define uma propriedade, uma classe não pode definir uma propriedade com o mesmo nome a menos que seja compatível (mesma visibilidade, tipo, modificador de somente-leitura e valor inicial), caso contrário um erro fatal é emitido.

Exemplo #13 Resolução de Conflito

```php
<?php
trait PropertiesTrait {
    public $same = true;
    public $different1 = false;
    public bool $different2;
    public bool $different3;
}

class PropertiesExample {
    use PropertiesTrait;
    public $same = true;
    public $different1 = true; // Fatal error
    public string $different2; // Fatal error
    readonly protected bool $different3; // Fatal error
}
?>
```

Constantes

Traits podem, a partir do PHP 8.2.0, também definir constantes.

Exemplo #14 Definindo Constantes

```php
<?php
trait ConstantsTrait {
    public const FLAG_MUTABLE = 1;
    final public const FLAG_IMMUTABLE = 5;
}

class ConstantsExample {
    use ConstantsTrait;
}

$example = new ConstantsExample;
echo $example::FLAG_MUTABLE;
?>
```

O exemplo acima produzirá:

```
1
```

Se uma Trait define uma constante, uma classe não pode definir uma constante com o mesmo nome a não ser que seja compatível (mesma visibilidade, valor inicial, e finalidade), caso contrário um erro fatal é emitido.

Exemplo #15 Resolução de Conflito

```php
<?php
trait ConstantsTrait {
    public const FLAG_MUTABLE = 1;
    final public const FLAG_IMMUTABLE = 5;
}

class ConstantsExample {
    use ConstantsTrait;
    public const FLAG_IMMUTABLE = 5; // Fatal error
}
?>
```

Métodos finais

A partir do PHP 8.3.0, o modificador final pode ser aplicado, com o uso do operador `as`, a métodos importados de traits. Isto pode ser usado para evitar que classes filhas substituam o método. Entretanto, a classe que usa o trait ainda poderá substituir o método.

Exemplo #16 Definindo um método originário de um trait como `final`

```php
<?php

trait ConstantsTrait
```

```php
{
    public function method()
    {
        echo 'Olá';
    }
}

class FinalExampleA
{
    use CommonTrait {
        CommonTrait::method as final; // O 'final' evita que classes filhas substituam o método
    }
}

class FinalExampleB extends FinalExampleA
{
     public function method() {}
}

?>
```

O exemplo acima produzirá algo semelhante a:

```
Fatal error: Cannot override final method FinalExampleA::method() in ...
```

Classes Anônimas

Classes anônimas são úteis quando objetos simples e descartáveis precisarem ser criados.

```php
<?php

// Usando uma classe explícita
class Logger
{
    public function log($msg)
    {
        echo $msg;
    }
}

$util->setLogger(new Logger());

// Usando uma classe anônima
$util->setLogger(new class {
    public function log($msg)
    {
        echo $msg;
    }
});
```

Podem passar argumentos através de seus construtores, estender outras classes, implementar interfaces, e usar traits como uma classe normal faz:

```php
<?php

class SomeClass {}
interface SomeInterface {}
trait SomeTrait {}

var_dump(new class(10) extends SomeClass implements SomeInterface {
    private $num;

    public function __construct($num)
    {
        $this->num = $num;
    }

    use SomeTrait;
});
```

O exemplo acima produzirá:

```
object(class@anonymous)#1 (1) {
  ["Command line code0x104c5b612":"class@anonymous":private]=>
  int(10)
}
```

Aninhar uma classe anônima dentro de outra classe não dá acesso a nenhum método privado ou protegido, ou propriedades da classe externa. Para utilizar os métodos e propriedades protegidas da classe externa, a classe anônima pode estender a classe externa. Para utilizar as propriedades privadas da classe externa na classe anônima, elas devem ser passadas através do construtor:

```php
<?php

class Outer
{
    private $prop = 1;
    protected $prop2 = 2;

    protected function func1()
    {
        return 3;
    }

    public function func2()
    {
        return new class($this->prop) extends Outer {
            private $prop3;

            public function __construct($prop)
            {
```

```
                $this->prop3 = $prop;
            }

            public function func3()
            {
                return $this->prop2 + $this->prop3 + $this->func1();
            }
        };
    }
}

echo (new Outer)->func2()->func3();
```

O exemplo acima produzirá:

```
6
```

Todos os objetos criados pela mesma declaração de classe anônima são instâncias desta mesma classe.

```
<?php
function anonymous_class()
{
    return new class {};
}

if (get_class(anonymous_class()) === get_class(anonymous_class())) {
    echo 'same class';
} else {
    echo 'different class';
}
```

O exemplo acima produzirá:

```
same class
```

> **Nota** :
>
> Classes anônimas têm nomes atribuídos automaticamente como demonstrado no exemplo a seguir. Este nome deve ser considerado como um detalhe de implementação, o qual não deve ser confiável.
>
> ```
> <?php
> echo get_class(new class {});
> ```
>
> O exemplo acima produzirá algo semelhante a:

```
class@anonymous/in/oNi1A0x7f8636ad2021
```

Classes anônimas somente-leitura

A partir do PHP 8.3.0, o modificador `readonly` pode ser aplicado a classes anônimas.

Exemplo #1 Definindo uma classe anônima somente-leitura

```php
<?php
// Usando uma classe anônima
var_dump(new readonly class('[DEBUG]') {
    public function __construct(private string $prefix)
    {
    }

    public function log($msg)
    {
        echo $this->prefix . ' ' . $msg;
    }
});
```

Sobrecarga

Sobrecarga em PHP provê recursos para "criar" dinamicamente propriedades e métodos. Estas entidades dinâmicas são processadas por métodos mágicos fornecendo a uma classe vários tipos de ações.

Os métodos de sobrecarga são invocados ao interagir com propriedades ou métodos que não foram declarados ou não são visíveis no escopo corrente. O resto desta seção usará os termos "propriedades inacessíveis" e "métodos inacessíveis" para referir-se a esta combinação de declaração e visibilidade.

Todos os métodos de sobrecarga devem ser definidos como `public`.

> **Nota** :
>
> Nenhum dos argumentos dos métodos mágicos podem ser passados por referência.

> **Nota** :
>
> A interpretação do PHP de "sobrecarga" é diferente da maioria das linguagens orientadas a objetos. Sobrecarga, tradicionalmente, provê a habilidade de ter múltiplos métodos com o mesmo nome, mas com quantidades e tipos de argumentos diferentes.

Sobrecarga de propriedades

public function **__set**(string `$name`, mixed `$value`): void
public function **__get**(string `$name`): mixed
public function **__isset**(string `$name`): bool

public function **__unset**(string `$name`): void

__set() é executado ao escrever dados em propriedades inacessíveis (private ou protected) ou propriedades inexistentes.

__get() é utilizado para ler dados de propriedades inacessíveis (private ou protected) ou propriedades inexistentes.

__isset() é disparado ao chamar a função isset() ou empty() em propriedades inacessíveis (private ou protected) ou propriedades inexistentes.

__unset() é invocado ao usar o construtor unset() em propriedades inacessíveis (private ou protected) ou propriedades inexistentes.

O argumento *$name* é o nome da propriedade com o qual se está interagindo. O argumento *$value* do método __set() especifica o valor para o qual a propriedade *$name* deveria ser definida.

A sobrecarga de propriedades funciona somente no contexto de objeto. Estes métodos mágicos não são disparados em contexto estático. Portanto estes métodos não podem ser declarados como static. Um aviso é emitido se algum método mágico de sobrecarga é declarado como `static`.

> **Nota** :
>
> O valor de retorno de __set() é ignorado por causa da forma que o PHP processa o operador de atribuição. Similarmente, o método __get() nunca é chamado em atribuições encadeadas como essa:
>
> ```
> $a = $obj->b = 8;
> ```

> **Nota** :
>
> O PHP não irá chamar métodos sobrecarregados a partir do próprio método sobrecarregado. Isso significa, por exemplo, que escrever `return $this->foo` dentro de um __get() irá retornar **`null`** e lançar um **`E_WARNING`** se não houver uma `foo` propriedade definida, ao invés de chamar __get() uma segunda vez. Entretanto, métodos sobrecarregados podem invocar outros métodos sobrecarregados implicitamente (como um __set() chamando um __get()).

Exemplo #1 Sobrecarregando propriedades com __get(), __set(), __isset() e __unset()

```
<?php
class PropertyTest
{
    /** Local para dados sobrecarregados. */
    private $data = array();

    /** Sobrecarga não usada em propriedades declaradas. */
    public $declared = 1;

    /** Sobrecarga usada somente neste quando acessada fora da classe. */
    private $hidden = 2;

    public function __set($name, $value)
```

```
    {
        echo "Definindo '$name' para '$value'\n";
        $this->data[$name] = $value;
    }

    public function __get($name)
    {
        echo "Obtendo '$name'\n";
        if (array_key_exists($name, $this->data)) {
            return $this->data[$name];
        }

        $trace = debug_backtrace();
        trigger_error(
            'Propriedade indefinida via __get(): ' . $name .
            ' em ' . $trace[0]['file'] .
            ' na linha ' . $trace[0]['line'],
            E_USER_NOTICE);
        return null;
    }

    public function __isset($name)
    {
        echo "'$name' está definido?\n";
        return isset($this->data[$name]);
    }

    public function __unset($name)
    {
        echo "Removendo a definição de '$name'\n";
        unset($this->data[$name]);
    }

    /** Não é um método mágico, está aqui somente para exemplo. */
    public function getHidden()
    {
        return $this->hidden;
    }
}


$obj = new PropertyTest;

$obj->a = 1;
echo $obj->a . "\n\n";

var_dump(isset($obj->a));
unset($obj->a);
var_dump(isset($obj->a));
echo "\n";

echo $obj->declared . "\n\n";

echo "Vamos experimentar com a propriedade privada chamada 'hidden':\n";
echo "Propriedades privadas são visíveis dentro da classe, por isso __get() não é usado...\n";
echo $obj->getHidden() . "\n";
echo "Propriedades privadas não são visíveis fora da classe, por isso __get() é usado...\n";
```

```php
echo $obj->hidden . "\n";
?>
```

O exemplo acima produzirá:

```
Definindo 'a' to '1'
Obtendo 'a'
1

'a' está definido?
bool(true)
Removendo a definição de 'a'
'a' está definido?
bool(false)

1

Vamos experimentar com a propriedade privada chamada 'hidden':
Propriedades privadas são visíveis dentro da classe, por isso __get() não é usado...
2
Propriedades privadas não são visíveis fora da classe, por isso __get() é usado...
Obtendo 'hidden'

Notice: Undefined property via __get(): hidden in script on line 71 in script on line 27
```

Sobrecarga de método

public function **__call**(string `$name`, array `$arguments`): mixed
public static function **__callStatic**(string `$name`, array `$arguments`): mixed

__call() é disparado ao invocar métodos inacessíveis em um contexto de objeto.

__callStatic() é disparado quando invocando métodos inacessíveis em um contexto estático.

O argumento *$name* é o nome do método sendo chamado. O argumento *$arguments* é um array enumerado contendo os parâmetros passados para o método *$name*.

Exemplo #2 Sobrecarga de métodos instanciados com __call() e __callStatic()

```php
<?php
class MethodTest
{
    public function __call($name, $arguments)
    {
        // Observação: valor de $name é sensível a maiúsculas/minúsculas.
        echo "Chamando método '$name' do objeto "
             . implode(', ', $arguments). "\n";
    }

    public static function __callStatic($name, $arguments)
    {
        // Observação: valor de $name é sensível a maiúsculas/minúsculas.
```

```
        echo "Chamando método '$name' estático "
             . implode(', ', $arguments). "\n";
    }
}

$obj = new MethodTest;
$obj->runTest('no contexto do objeto');

MethodTest::runTest('no contexto estático');
?>
```

O exemplo acima produzirá:

```
Chamando método 'runTest' do objeto no contexto do objeto
Chamando método 'runTest' estático no contexto estático
```

Iteração de Objetos

O PHP fornece uma maneira de definir objetos para que seja possível iterar por uma lista de itens como, por exemplo, a instrução `foreach`. Por padrão, todas as propriedades visíveis serão utilizadas para a iteração.

Exemplo #1 Iteração simples de objetos

```
<?php
class MyClass
{
    public $var1 = 'valor 1';
    public $var2 = 'valor 2';
    public $var3 = 'valor 3';

    protected $protected = 'protected var';
    private   $private   = 'private var';

    function iterateVisible() {
       echo "MyClass::iterateVisible:\n";
       foreach ($this as $key => $value) {
           print "$key => $value\n";
       }
    }
}

$class = new MyClass();

foreach($class as $key => $value) {
    print "$key => $value\n";
}
echo "\n";


$class->iterateVisible();
```

```
?>
```

O exemplo acima produzirá:

```
var1 => valor 1
var2 => valor 2
var3 => valor 3

MyClass::iterateVisible:
var1 => valor 1
var2 => valor 2
var3 => valor 3
protected => protected var
private => private var
```

Como a saída mostra, o `foreach` passou por cada uma das variáveis visíveis que podem ser acessadas.

Veja também

- Generators
- Iterator
- IteratorAggregate
- SPL Iterators

Métodos Mágicos

Métodos mágicos são métodos especiais que sobrescrevem o comportamento padrão do PHP quando certas operações são realizadas em um objeto.

> **Cuidado**
>
> Todos os nomes de métodos prefixados com __ são reservados pelo PHP. Portanto, não é recomendado utilizar nomes de métodos com esse prefixo a não ser para sobrescrever o comportamento do PHP.

Os seguintes nomes de métodos são considerados mágicos: __construct(), __destruct(), __call(), __callStatic(), __get(), __set(), __isset(), __unset(), __serialize(), __unserialize(), __sleep(), __wakeup(), __toString(), __invoke(), __set_state(), __clone() e __debugInfo()

> **Aviso**
>
> Os métodos acima, com exceção de __construct(), __destruct() e __clone(), *precisam* ser declarados como `public`, de outra forma um **`E_WARNING`** é emitido. Anteriormente ao PHP 8.0.0, nenhum diagnóstico era emitido para os métodos mágicos __sleep(), __wakeup(), __serialize(), __unserialize() e __set_state().

> **Aviso**

> Se tipos forem utilizados na declaração de métodos mágicos, eles precisam ser idênticos às assinaturas previstas aqui. Senão, um erro fatal é lançado. Anteriormente ao PHP 8.0.0, nenhum diagnóstico era emitido. Entretanto, __construct() e __destruct() não devem declarar um tipo de retorno, senão um erro fatal é lançado.

__serialize() e __unserialize()

public function **__serialize**(): array
public function **__unserialize**(array `$data`): void

serialize() verifica se a classe contém uma função com o nome mágico __serialize(). Se sim, essa função é executada antes de qualquer serialização. Ela precisa construir e retornar um array associativo de chaves-valores que representam a forma serializada do objeto. Se o array não for retornado então um erro TypeError será lançado.

> **Nota** :
>
> Se ambos __serialize() e __sleep() estiverem definidos no mesmo objeto, somente __serialize() será chamado. __sleep() será ignorado. Se o objeto implementa a interface Serializable, o método `serialize()` da interface será ignorado e o método mágico __serialize() será utilizado.

O uso pretendido de __serialize() é definir uma representação arbitrária, amigável, da representação do objeto. Elementos do array podem corresponder a propriedades do objeto diretamente, mas isso não é obrigatório.

Inversamente, unserialize() verifica a presença da função mágica __unserialize(). Se presente, essa função será chamada com o array retornado de __serialize(). Ela poderá, então, restaurar as propriedades do objeto a partir do array.

> **Nota** :
>
> Se ambos __unserialize() e __wakeup() estiverem definidos, somente __unserialize() será chamado, e __wakeup() será ignorado.

> **Nota** :
>
> Esse recurso está disponível desde o PHP 7.4.0.

Exemplo #1 Serialize e unserialize

```php
<?php
class Connection
{
    protected $link;
    private $dsn, $username, $password;

    public function __construct($dsn, $username, $password)
    {
        $this->dsn = $dsn;
```

```
        $this->username = $username;
        $this->password = $password;
        $this->connect();
    }

    private function connect()
    {
        $this->link = new PDO($this->dsn, $this->username, $this->password);
    }

    public function __serialize(): array
    {
        return [
          'dsn' => $this->dsn,
          'user' => $this->username,
          'pass' => $this->password,
        ];
    }

    public function __unserialize(array $data): void
    {
        $this->dsn = $data['dsn'];
        $this->username = $data['user'];
        $this->password = $data['pass'];

        $this->connect();
    }
}?>
```

__sleep() e __wakeup()

> **Aviso**
>
> Este mecanismo de serialização está praticamente descontinuado a partir do PHP 8.5.0. Foi mantido para compatibilidade com versões anteriores. Entretanto, código novo ou existente deve ser atualizado para uso dos métodos mágicos __serialize() e __unserialize() em seu lugar.

public function **__sleep**(): array
public function **__wakeup**(): void

serialize() verifica se a classe tem uma função com o nome mágico __sleep(). Se tiver, a função é executada antes de qualquer serialização. Ela pode limpar o objeto e deve retornar um array com os nomes de todas as variáveis do objeto que devem ser serializadas. Se o método não retornar nada, então `null` é serializado e um `E_NOTICE` é emitido.

> **Nota** :
>
> Não é possível que __sleep() retorne nomes de propriedades privadas da classe superior. Fazer isso causará um erro de nível `E_NOTICE`. Como alternativa, utilize __serialize().

> **Nota** :

> A partir do PHP 8.0.0, retornar um valor que não seja um array originado de __sleep() gera um alerta. Anteriormente gerava um aviso.

O intuito do método __sleep() é enviar dados pendentes ou realizar tarefas de limpeza similares. Além disso, a função é útil se um objeto muito grande não precisar ser completamente salvo.

Ao mesmo tempo, unserialize() verifica a presença da função com o nome mágico __wakeup(). Se presente, essa função pode reconstruir quaisquer recursos que o objeto possa ter.

O intuito do método __wakeup() é restabelecer quaisquer conexões com bancos de dados que possam ter sido perdidas durante a serialização e realizar outras tarefas de reinicialização.

Exemplo #2 Sleep e wakeup

```php
<?php
class Connection
{
    protected $link;
    private $dsn, $username, $password;

    public function __construct($dsn, $username, $password)
    {
        $this->dsn = $dsn;
        $this->username = $username;
        $this->password = $password;
        $this->connect();
    }

    private function connect()
    {
        $this->link = new PDO($this->dsn, $this->username, $this->password);
    }

    public function __sleep()
    {
        return array('dsn', 'username', 'password');
    }

    public function __wakeup()
    {
        $this->connect();
    }
}?>
```

__toString()

public function **__toString**(): string

O método __toString() permite que uma classe decida como se comportar quando convertida para uma string. Por exemplo, o que `echo $obj;` irá imprimir.

> **Aviso**
>
> A partir do PHP 8.0.0, o valor de retorno segue a mesma semântica de tipo do PHP, significando que o valor será convertido para string se possível e se strict typing estiver desligado.

> Um objeto Stringable *não* é aceito para uma declaração string se strict typing estiver ativo. Caso esse comportamento seja desejado, a declaração precisa indicar Stringable e string através de um tipo união.
>
> A partir do PHP 8.0.0, quaisquer classe que contenha o método __toString() também implementa implicitamente a interface Stringable, e portanto passa os testes para essa interface. Implementar explicitamente essa interface é o recomendado.
>
> No PHP 7.4, o valor retornado *precisa* ser uma string, senão um erro Error é lançado.
>
> Anteriormente ao PHP 7.4.0, o valor retornado *precisa* ser uma string, senão um erro fatal `E_RECOVERABLE_ERROR` é emitido.

> **Aviso**
>
> Não era possível lançar uma exception de dentro de um método __toString() antes do PHP 7.4.0. Isso gera um erro fatal.

Exemplo #3 Exemplo Simples

```php
<?php
// Declara uma classe simples
class TestClass
{
    public $foo;

    public function __construct($foo)
    {
        $this->foo = $foo;
    }

    public function __toString()
    {
        return $this->foo;
    }
}

$class = new TestClass('Hello');
echo $class;
?>
```

O exemplo acima produzirá:

```
Hello
```

__invoke()

function **__invoke**(`...$values`): mixed

O método __invoke() é chamado quando um script tenta chamar um objeto como uma função.

Exemplo #4 Usando __invoke()

```php
<?php
class CallableClass
{
    public function __invoke($x)
    {
        var_dump($x);
    }
}
$obj = new CallableClass;
$obj(5);
var_dump(is_callable($obj));
?>
```

O exemplo acima produzirá:

```
int(5)
bool(true)
```

Exemplo #5 Exemplo de __invoke()

```php
<?php
class Sort
{
    private $key;

    public function __construct(string $key)
    {
        $this->key = $key;
    }

    public function __invoke(array $a, array $b): int
    {
        return $a[$this->key] <=> $b[$this->key];
    }
}

$customers = [
    ['id' => 1, 'first_name' => 'John', 'last_name' => 'Do'],
    ['id' => 3, 'first_name' => 'Alice', 'last_name' => 'Gustav'],
    ['id' => 2, 'first_name' => 'Bob', 'last_name' => 'Filipe']
];

// Ordena os clientes pelo primeiro nome
usort($customers, new Sort('first_name'));
print_r($customers);

// Ordena os clientes pelo último nome
usort($customers, new Sort('last_name'));
```

```php
print_r($customers);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [id] => 3
            [first_name] => Alice
            [last_name] => Gustav
        )

    [1] => Array
        (
            [id] => 2
            [first_name] => Bob
            [last_name] => Filipe
        )

    [2] => Array
        (
            [id] => 1
            [first_name] => John
            [last_name] => Do
        )

)
Array
(
    [0] => Array
        (
            [id] => 1
            [first_name] => John
            [last_name] => Do
        )

    [1] => Array
        (
            [id] => 2
            [first_name] => Bob
            [last_name] => Filipe
        )

    [2] => Array
        (
            [id] => 3
            [first_name] => Alice
            [last_name] => Gustav
        )

)
```

__set_state()

static function **__set_state**(array `$properties`): object

Esse método estático é chamado em classes exportadas por var_export().

O único parâmetro deste método é um array contendo propriedades exportadas no formato `['propriedade' => valor, ...]`.

Exemplo #6 Usando __set_state()

```php
<?php

class A
{
    public $var1;
    public $var2;

    public static function __set_state($an_array)
    {
        $obj = new A;
        $obj->var1 = $an_array['var1'];
        $obj->var2 = $an_array['var2'];
        return $obj;
    }
}

$a = new A;
$a->var1 = 5;
$a->var2 = 'foo';

$b = var_export($a, true);
var_dump($b);
eval('$c = ' . $b . ';');
var_dump($c);
?>
```

O exemplo acima produzirá:

```
string(60) "A::__set_state(array(
   'var1' => 5,
   'var2' => 'foo',
))"
object(A)#2 (2) {
  ["var1"]=>
  int(5)
  ["var2"]=>
  string(3) "foo"
}
```

> **Nota** :
>
> Quando exportando um objeto, var_export() não verifica se __set_state() está implementado na classe do objeto, de forma que re-importar esses objetos falham com um erro Error na ausência de __set_state(). Isto afeta particularmente algumas classes internas.
>
> É responsabilidade do programador verificar que somente os objetos cuja classe implementa **__set_state()** serão re-importados.

__debugInfo()

function **__debugInfo**(): array

Este método é chamado pela função var_dump() ao despejar um objeto para obter as propriedades que devem ser exibidas. Se este método não for definido em um objeto, todas as propriedades públicas, protegidas e privadas serão exibidas.

Exemplo #7 Utilizando o __debugInfo()

```
<?php
class C {
    private $prop;

    public function __construct($val) {
        $this->prop = $val;
    }

    public function __debugInfo() {
        return [
            'propSquared' => $this->prop ** 2,
        ];
    }
}

var_dump(new C(42));
?>
```

O exemplo acima produzirá:

```
object(C)#1 (1) {
  ["propSquared"]=>
  int(1764)
}
```

final

A palavra-chave final evita que classes filhas sobrescrevam um método, propriedade ou constante prefixando a definição com `final`. Se a própria classe estiver definida como final, ela não pode ser estendida.

Exemplo #1 Exemplo de método final

```php
<?php
class ClasseBase {
   public function teste() {
       echo "ClasseBase::teste() chamado\n";
   }

   final public function maisTeste() {
       echo "ClasseBase::maisTeste() chamado\n";
   }
}

class ClasseFilha extends ClasseBase {
   public function maisTeste() {
       echo "ClasseFilha::maisTeste() called\n";
   }
}
// Resulta em erro fatal: Não é possível substituir o método final ClasseBase::maisTeste()
?>
```

Exemplo #2 Exemplo de classe final

```php
<?php
final class ClasseBase {
   public function teste() {
       echo "Método ClasseBase::teste() chamado\n";
   }

   // Como a classe já é final, a palavra-chave final é rendundante.
   final public function maisTeste() {
       echo "Método ClasseBase::maisTeste() chamado\n";
   }
}

class ClasseFilha extends ClasseBase {
}
// Resulta em erro fatal: Classe ClasseFilha não pode herdar da classe final (ClasseBase)
?>
```

Exemplo #3 Exemplo de propriedade final a partir do PHP 8.4.0

```php
<?php
class ClasseBase {
   final protected string $teste;
}

class ClasseFilha extends ClasseBase {
    public string $teste;
}
// Resulta em erro fatal: Não é possível substituir a propriedade final ClasseBase::$teste
?>
```

Exemplo #4 Exemplo de constante final a partir do PHP 8.1.0

```php
<?php
class Foo
{
    final public const X = "foo";
}

class Bar extends Foo
{
    public const X = "bar";
}

// Erro fatal: Bar::X não pode substituir a constante final Foo::X
?>
```

> **Nota** :
>
> A partir do PHP 8.0.0, métodos privados não podem ser declarados finais, com exceção do construtor.

> **Nota** :
>
> Uma propriedade que é declarada `private(set)` é, implicitamente, `final`.

Clonando Objetos

Criar uma cópia de um objeto com propriedades totalmente replicadas nem sempre é o comportamento desejado. Um bom exemplo da necessidade de construtores de cópia é quando se tem um objeto que representa uma janela do GTK e ele guarda o recurso dessa janela GTK. Ao criar uma duplicata, pode-se desejar a criação de uma nova janela com as mesmas propriedades e fazer o novo objeto guardar o recurso da nova janela. Outro exemplo é se seu objeto guarda uma referência a outro objeto que usa e ao replicar o objeto pai, deseja-se que seja criada uma nova instância desse outro objeto para que a réplica tenha sua própria cópia separada.

Uma cópia de objeto é criada usando a palavra-chave `clone` (que chama o método __clone() do objeto, se possível).

```php
$copia_do_objeto = clone $objeto;
```

Ao clonar um objeto, o PHP fará uma cópia superficial de todas as propriedades do objeto. Qualquer propriedade que seja referência a outra variável, permanecerá como referência.

function **__clone**(): void

Depois que a clonagem se completa, se um método __clone() estiver definido, o objeto recém criado terá seu método __clone() chamado, permitindo que as prorpriedades necessárias sejam alteradas.

Exemplo #1 Clonando um objeto

```php
<?php
class SubObject
{
    static $instances = 0;
```

```
    public $instance;

    public function __construct() {
        $this->instance = ++self::$instances;
    }

    public function __clone() {
        $this->instance = ++self::$instances;
    }
}

class MyCloneable
{
    public $object1;
    public $object2;

    function __clone()
    {
        // Força uma cópia de this->object, caso contrário
        // aponta para o mesmo objeto.
        $this->object1 = clone $this->object1;
    }
}

$obj = new MyCloneable();

$obj->object1 = new SubObject();
$obj->object2 = new SubObject();

$obj2 = clone $obj;


print "Objeto original:\n";
print_r($obj);

print "Objeto clonado:\n";
print_r($obj2);

?>
```

O exemplo acima produzirá:

```
Objeto original:
MyCloneable Object
(
    [object1] => SubObject Object
        (
            [instance] => 1
        )

    [object2] => SubObject Object
        (
            [instance] => 2
        )
```

```
)
Objeto clonado:
MyCloneable Object
(
    [object1] => SubObject Object
        (
            [instance] => 3
        )

    [object2] => SubObject Object
        (
            [instance] => 2
        )

)
```

É possível acessar um membro de um objeto recém clonado em uma expressão única:

Exemplo #2 Acessar um membro de um objeto recém clonado

```php
<?php
$dateTime = new DateTime();
echo (clone $dateTime)->format('Y');
?>
```

O exemplo acima produzirá algo semelhante a:

```
2016
```

Comparando Objetos

Ao usar o operador de comparação (==), as variáveis dos objetos são comparadas de maneira simples, nominalmente: Duas instâncias de objetos são iguais se possuem os mesmos atributos e valores (valores são comparados com ==), e são instâncias da mesma classe.

Ao usar o operador de identidade (===), as variáveis dos objetos serão idênticas, se e somente se, referirem a mesma instância da mesma classe.

Um exemplo evidenciará essas regras.

Exemplo #1 Exemplo de comparação de objetos

```php
<?php
function bool2str($bool)
{
    if ($bool === false) {
        return 'FALSO';
    } else {
```

```
        return 'VERDADEIRO';
    }
}

function compareObjects(&$o1, &$o2)
{
    echo 'o1 == o2 : ' . bool2str($o1 == $o2) . "\n";
    echo 'o1 != o2 : ' . bool2str($o1 != $o2) . "\n";
    echo 'o1 === o2 : ' . bool2str($o1 === $o2) . "\n";
    echo 'o1 !== o2 : ' . bool2str($o1 !== $o2) . "\n";
}

class Flag
{
    public $flag;

    function __construct($flag = true) {
        $this->flag = $flag;
    }
}

class OtherFlag
{
    public $flag;

    function __construct($flag = true) {
        $this->flag = $flag;
    }
}

$o = new Flag();
$p = new Flag();
$q = $o;
$r = new OtherFlag();

echo "Duas instâncias da mesma classe\n";
compareObjects($o, $p);

echo "\nDuas referências para a mesma instância\n";
compareObjects($o, $q);

echo "\nInstâncias de duas classes diferentes\n";
compareObjects($o, $r);
?>
```

O exemplo acima produzirá:

```
Duas instâncias da mesma classe
o1 == o2 : VERDADEIRO
o1 != o2 : FALSO
o1 === o2 : FALSO
o1 !== o2 : VERDADEIRO

Duas referências para a mesma instância
o1 == o2 : VERDADEIRO
```

```
o1 != o2 : FALSO
o1 === o2 : VERDADEIRO
o1 !== o2 : FALSO

Instâncias de duas classes diferentes
o1 == o2 : FALSO
o1 != o2 : VERDADEIRO
o1 === o2 : FALSO
o1 !== o2 : VERDADEIRO
```

> **Nota** :
>
> Extensões podem definir suas próprias regras para comparação de objetos (==).

Vinculações Estáticas Tardias

O PHP implementa um recurso chamado Vinculações Estáticas Tardias que pode ser usado para referenciar a classe chamada no contexto de herança estática.

Mais precisamente, as Vinculações Estáticas Tardias funcionam através do armazenamento do nome da classe na última "chamada não encaminhada". No caso de chamadas a métodos estáticos, é a classe explicitamente chamada (normalmente o nome à esquerda do operador `::`); no caso de chamadas a métodos não estáticos, é o nome da classe do objeto. Uma "chamada encaminhada" é àquela estática, realizada pelos prefixos `self::`, `parent::`, `static::`, ou, se subindo na hierarquia de classes, forward_static_call(). A função get_called_class() pode ser utilizada para recuperar uma string com o nome da classe chamada e `static::` introduz seu escopo.

Esse recurso foi chamado de "Vinculações Estáticas Tardias" de uma perspectiva interna em mente. "Vinculação tardia" vem do fato que `static::` não será resolvido usando a classe onde o método foi definido, mas computada utilizando informações em tempo de execução. É também chamado "vinculação estática" pois pode ser utilizado em (mas não limitado a) chamadas de métodos estáticos.

Limitações do `self::`

Referências estáticas para a atual classe como `self::` ou `__CLASS__` são resolvidas usando a classe na qual a função pertence, como onde ele foi definido:

Exemplo #1 Uso do `self::`

```php
<?php

class A
{
    public static function who()
    {
        echo __CLASS__;
    }

    public static function test()
    {
        self::who();
    }
```

```php
}

class B extends A
{
    public static function who()
    {
        echo __CLASS__;
    }
}

B::test();

?>
```

O exemplo acima produzirá:

```
A
```

Uso de Vinculações Estáticas Tardias

Vinculações Estáticas Tardias tentam resolver a limitação introduzindo uma palavra-chave que referencia a classe que foi inicialmente chamada em tempo de execução. Basicamente, é uma palavra-chave que permite referenciar `B` em `test()`, no exemplo anterior. Foi decidido não introduzir uma nova palavra-chave, mas usar `static`, já reservada.

Exemplo #2 Uso simples de `static::`

```php
<?php

class A
{
    public static function who()
    {
        echo __CLASS__;
    }

    public static function test()
    {
        static::who(); // Aqui acontecem as Vinculações Estáticas Tardias
    }
}

class B extends A
{
    public static function who()
    {
        echo __CLASS__;
    }
}

B::test();
```

```
?>
```

O exemplo acima produzirá:

```
B
```

> **Nota** :
>
> Em contextos não estáticos, a classe chamada será a classe da instância do objeto. Assim como `$this->` chamará métodos privados do mesmo escopo, utilizar `static::` pode ter resultados diferentes. Outra diferença é que `static::` só pode referenciar propriedades estáticas.

Exemplo #3 Uso de `static::` em um contexto não-estático

```php
<?php

class A
{
    private function foo()
    {
        echo "successo!\n";
    }

    public function test()
    {
        $this->foo();
        static::foo();
    }
}

class B extends A
{
   /* foo() será copiado para B, assim seu escopo ainda será A e
    * e a chamada funcionará */
}

class C extends A
{
    private function foo()
    {
        /* método original foi substituído, escopo agora é C */
    }
}

$b = new B();
$b->test();

$c = new C();
try {
```

```
    $c->test();
} catch (Error $e) {
    echo $e->getMessage();
}

?>
```

O exemplo acima produzirá:

```
successo!
successo!
successo!
Call to private method C::foo() from scope A
```

> **Nota** :
>
> As resoluções de Vinculações Estáticas Tardias terminarão quando a chamada for realizada sem retorno. Por outro lado, chamadas estáticas utilizando instruções como `parent::` ou `self::` irão repassar a informação do chamador.
>
> **Exemplo #4 Chamadas repassadas e não repassadas**
>
> ```
> <?php
>
> class A
> {
>     public static function foo()
>     {
>         static::who();
>     }
>
>     public static function who()
>     {
>         echo __CLASS__."\n";
>     }
> }
>
> class B extends A
> {
>     public static function test()
>     {
>         A::foo();
>         parent::foo();
>         self::foo();
>     }
>
>     public static function who()
>     {
>         echo __CLASS__."\n";
>     }
> }
> ```

```php
class C extends B
{
    public static function who()
    {
        echo __CLASS__."\n";
    }
}

C::test();

?>
```

O exemplo acima produzirá:

```
A
C
C
```

Objetos e Referências

Um dos pontos fortes frequentemente mencionado da POO no PHP, é que "objetos são passados por referência por padrão". Isto não é completamente verdade. Esta seção retifica esse pensamento comum usando alguns exemplos.

Uma referência do PHP é um sinônimo, que permite duas variáveis diferentes escreverem para o mesmo valor. Em PHP, uma variável objeto não contém o próprio objeto como valor. Ela contém um identificador do objeto que permite que os acessadores do objeto encontrem o objeto real. Quando um objeto é enviado por argumento, retornado ou atribuído a outra variável, as variáveis diferentes não são sinônimos: elas armazenam uma cópia do identificador, que aponta para o mesmo objeto.

Exemplo #1 Referências e Objetos

```php
<?php
class A {
    public $foo = 1;
}

$a = new A;
$b = $a;     // $a e $b são cópias do mesmo identificador
             // ($a) = ($b) = <id>
$b->foo = 2;
echo $a->foo."\n";


$c = new A;
$d = &$c;    // $c e $d são referências
             // ($c,$d) = <id>

$d->foo = 2;
```

```php
echo $c->foo."\n";


$e = new A;

function foo($obj) {
    // ($obj) = ($e) = <id>
    $obj->foo = 2;
}

foo($e);
echo $e->foo."\n";

?>
```

O exemplo acima produzirá:

```
2
2
2
```

Serialização de Objetos

A função serialize() retorna uma string contendo uma representação em fluxo de bytes de qualquer valor que pode ser armazenado pelo PHP. A função unserialize() pode utilizar essa string para recriar os valores originais da variável. Utilizar a serialização para salvar um objeto, salvará todas as variáveis de um objeto. Os métodos de um objeto não são salvos, apenas o nome da classe.

Para desserializar um objeto com a função unserialize(), a classe desse objeto precisa estar definida. Isto é, se possuir um objeto da classe A e serializá-lo, será obtida uma string que referencia a classe A e contém todos os valores de variáveis contidas nele. Se for necessário desserializar um objeto da classe A em outro arquivo, a definição da classe A deve estar presente neste arquivo. Isso pode ser feito, por exemplo, armazenando a definição da classe A em um arquivo, e incluí-lo, fazendo uso da função spl_autoload_register().

```php
<?php
// A.php:

  class A {
      public $one = 1;

      public function show_one() {
          echo $this->one;
      }
  }

// page1.php:

  include "A.php";

  $a = new A;
```

```php
  $s = serialize($a);
  // armazena $s em algum lugar onde page2.php possa encontrar.
  file_put_contents('caixa', $s);

// page2.php:

  // isto é necessário para que a desserialização funcione adequadamente.
  include "A.php";

  $s = file_get_contents('caixa');
  $a = unserialize($s);

  // agora usa a função show_one() do objeto $a.
  $a->show_one();
?>
```

É fortemente recomendado que, quando uma aplicação serializa seus objetos, para uso posterior na aplicação, esta inclua a definição da classe do objeto em questão ao longo de toda aplicação. Não fazer isso pode resultar em objetos serem desserializados sem a definição da classe, que acarretará no PHP entregar um objeto da classe **__PHP_Incomplete_Class_Name**, que não possui métodos, tornando o objeto inútil.

Então, se, no exemplo anterior, *$a* se tornou parte da sessão adicionando uma nova chave no array superglobal *$_SESSION*, você deve incluir o arquivo `A.php` em todas suas páginas, e não somente em *page1.php* e *page2.php*.

Além dos conselhos acima, note que pode-se capturar os eventos de serialização e desserialização de um objeto usando os métodos __sleep() e __wakeup(). Usar o método __sleep() também lhe permitirá serializar somente subconjuntos de propriedades de um objeto.

Covariância e Contravariância

No 7.2.0, a contravariância parcial foi introduzida removendo as restrições de tipo nos parâmetros em um método filho. A partir do PHP 7.4.0, foi adicionado suporte a covariância e contravariância completas.

A covariância permite que um método de uma classe filha retorne um tipo mais específico que o tipo de retorno do método da classe pai. A contravariância permite a um tipo de parâmetro ser menos específico em um método de classe filha, em relação à classe pai.

Uma declaração de tipo é considerada mais específica nos seguintes casos:

- Um tipo é removido de um tipo união
- Um tipo é adicionado a um tipo interseção
- Um tipo de classe é alterado para um tipo de classe filha
- iterable é alterado para array ou Traversable

Uma declaração de tipo é considerada menos específica se o oposto for verdadeiro.

Covariância

Para ilustrar como uma variância funciona, uma classe pai abstrata simples, *Animal* é criada. *Animal* será estendida por classes filhas, *Cat* e *Dog*.

```php
<?php

abstract class Animal
{
    protected string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    abstract public function speak();
}

class Dog extends Animal
{
    public function speak()
    {
        echo $this->name . " barks";
    }
}

class Cat extends Animal
{
    public function speak()
    {
        echo $this->name . " meows";
    }
}
```

Observe que não há nenhum método que retorne valores neste exemplo. Serão adicionados alguns métodos que retornam um novo objeto do tipo de classe *Animal*, *Cat* ou *Dog*.

```php
<?php

interface AnimalShelter
{
    public function adopt(string $name): Animal;
}

class CatShelter implements AnimalShelter
{
    public function adopt(string $name): Cat // em vez de retornar o tipo Animal, pode retornar o tipo Cat
    {
        return new Cat($name);
    }
}

class DogShelter implements AnimalShelter
{
    public function adopt(string $name): Dog // em vez de retornar o tipo Animal, pode retornar o tipo Dog
    {
        return new Dog($name);
    }
```

```php
}

$kitty = (new CatShelter)->adopt("Ricky");
$kitty->speak();
echo "\n";

$doggy = (new DogShelter)->adopt("Mavrick");
$doggy->speak();
```

O exemplo acima produzirá:

```
Ricky meows
Mavrick barks
```

Contravariância

Continuando com o exemplo anterior com as classes *Animal*, *Cat* e *Dog*, duas classes chamadas *Food* e *AnimalFood* serão incluídas, e um método *eat(AnimalFood $food)* é adicionado à classe abstrata *Animal*.

```php
<?php

class Food {}

class AnimalFood extends Food {}

abstract class Animal
{
    protected string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function eat(AnimalFood $food)
    {
        echo $this->name . " eats " . get_class($food);
    }
}
```

Para ver o comportamento da contravariância, o método *eat* é substituído na classe *Dog* para permitir qualquer objeto do tipo *Food*. A classe *Cat* permanece inalterada.

```php
<?php

class Dog extends Animal
{
    public function eat(Food $food) {
        echo $this->name . " eats " . get_class($food);
    }
}
```

O próximo exemplo irá mostrar o comportamento da contravariância.

```php
<?php

$kitty = (new CatShelter)->adopt("Ricky");
$catFood = new AnimalFood();
$kitty->eat($catFood);
echo "\n";

$doggy = (new DogShelter)->adopt("Mavrick");
$banana = new Food();
$doggy->eat($banana);
```

O exemplo acima produzirá:

```
Ricky eats AnimalFood
Mavrick eats Food
```

Mas o que acontece se *$kitty* tentar comer (**eat()**) a *$banana*?

```php
$kitty->eat($banana);
```

O exemplo acima produzirá:

```
Fatal error: Uncaught TypeError: Argument 1 passed to Animal::eat() must be an instance of
AnimalFood, instance of Food given
```

Variância de propriedade

Por padrão, as propriedades não são covariantes nem contravariantes e, portanto, invariantes. Ou seja, o tipo delas não pode mudar em nenhuma classe filha. A razão para isso é que as operações "get" devem ser covariantes e as operações "set" devem ser contravariantes. A única maneira de uma propriedade satisfazer ambos os requisitos é ser invariante.

A partir do PHP 8.4.0, com a adição de propriedades abstratas (em uma interface ou classe abstrata) e propriedades virtuais, é possível declarar uma propriedade que possui apenas uma operação get ou set. Como resultado, propriedades abstratas ou propriedades virtuais que exigem apenas uma operação "get" podem ser covariantes. Da mesma forma, uma propriedade abstrata ou propriedade virtual que requerem apenas uma operação "set" pode ser contravariante.

Uma vez que uma propriedade tenha uma operação get e set, entretanto, ela não será mais covariante ou contravariante para extensão adicional. Ou seja, agora é invariante.

Exemplo #1 Variância do tipo de propriedade

```php
<?php
class Animal {}
class Dog extends Animal {}
class Poodle extends Dog {}

interface PetOwner
{
    // Apenas uma operação get é necessária, portanto isso pode ser covariante.
    public Animal $pet { get; }
}

class DogOwner implements PetOwner
{
    // Este pode ser um tipo mais restritivo, já que o lado "get"
    // ainda retorna um Animal.  Porém, como propriedade nativa,
    // os filhos desta classe não podem mais alterar o tipo.
    public Dog $pet;
}

class PoodleOwner extends DogOwner
{
    // Isso NÃO É PERMITIDO, porque DogOwner::$pet tem operações
    // get e set definidas e obrigatórias.
    public Poodle $pet;
}
?>
```

Objetos Lentos

Um objeto lento é um objeto cuja inicialização é adiada até que seu estado seja observado ou modificado. Alguns exemplos de casos de uso incluem componentes de injeção de dependência que fornecem serviços lentos totalmente inicializados somente se necessário, ORMs fornecendo entidades lentas que se hidratam do banco de dados somente quando acessados, ou um analisador JSON que atrasa a análise até que os elementos sejam acessados.

Duas estratégias de objetos lentos são suportadas: Objetos Fantasmas e Proxies Virtuais, doravante denominados "fantasmas lentos" e "proxies lentos". Em ambas as estratégias, o objeto lento é anexado a um inicializador ou fábrica que é chamado automaticamente quando seu estado é observado ou modificado pela primeira vez. Do ponto de vista da abstração, os objetos fantasmas lentos são indistinguíveis dos não-lentos: eles podem ser usados sem que se saiba que são lentos, permitindo que sejam passados e usados por códigos que desconhecem a lentidão. Os proxies lentos são igualmente transparentes, mas deve-se ter cuidado ao usar sua identidade, pois o proxy e sua instância real possuem identidades diferentes.

> **Nota** : **Informação de Versão**
>
> Objetos Lentos foram introduzidos no PHP 8.4.

Criando Objetos Lentos

É possível criar uma instância lenta de qualquer classe definida pelo usuário ou da classe stdClass (outras classes internas não são suportadas), ou redefinir uma instância dessas classes para torná-la lenta. Os pontos

de entrada para criar um objeto lento são os métodos ReflectionClass::newLazyGhost() e ReflectionClass::newLazyProxy().

Ambos os métodos aceitam uma função que é chamada quando o objeto requer inicialização. O comportamento esperado da função varia dependendo da estratégia em uso, conforme descrito na documentação de referência de cada método.

Exemplo #1 Criando um Fantasma Lento

```php
<?php
class Example
{
    public function __construct(public int $prop)
    {
        echo __METHOD__, "\n";
    }
}

$reflector = new ReflectionClass(Example::class);
$lazyObject = $reflector->newLazyGhost(function (Example $object) {
    // Inicializa o objeto no local
    $object->__construct(1);
});

var_dump($lazyObject);
var_dump(get_class($lazyObject));

// Dispara a inicialização
var_dump($lazyObject->prop);
?>
```

O exemplo acima produzirá:

```
lazy ghost object(Example)#3 (0) {
  ["prop"]=>
  uninitialized(int)
}
string(7) "Example"
Example::__construct
int(1)
```

Exemplo #2 Criando um Proxy Lento

```php
<?php
class Example
{
    public function __construct(public int $prop)
    {
        echo __METHOD__, "\n";
    }
}
```

```php
$reflector = new ReflectionClass(Example::class);
$lazyObject = $reflector->newLazyProxy(function (Example $object) {
    // Cria e retorna a instância real
    return new Example(1);
});

var_dump($lazyObject);
var_dump(get_class($lazyObject));

// Dispara a inicialização
var_dump($lazyObject->prop);
?>
```

O exemplo acima produzirá:

```
lazy proxy object(Example)#3 (0) {
  ["prop"]=>
  uninitialized(int)
}
string(7) "Example"
Example::__construct
int(1)
```

Qualquer acesso às propriedades de um objeto lento aciona sua inicialização (inclusive via ReflectionProperty). No entanto, certas propriedades podem ser conhecidas antecipadamente e não devem acionar a inicialização quando acessadas:

Exemplo #3 Inicializando Propriedades Ansiosamente

```php
<?php
class BlogPost
{
    public function __construct(
        public int $id,
        public string $title,
        public string $content,
    ) { }
}

$reflector = new ReflectionClass(BlogPost::class);

$post = $reflector->newLazyGhost(function ($post) {
    $data = fetch_from_store($post->id);
    $post->__construct($data['id'], $data['title'], $data['content']);
});

// Sem esta linha, a seguinte chamada a ReflectionProperty::setValue()
// acionaria a inicialização.
$reflector->getProperty('id')->skipLazyInitialization($post);
$reflector->getProperty('id')->setValue($post, 123);
```

```php
// Alternativamente, pode-se usar isso diretamente:
$reflector->getProperty('id')->setRawValueWithoutLazyInitialization($post, 123);

// A propriedade id pode ser acessada sem acionar a inicialização
var_dump($post->id);
?>
```

Os métodos ReflectionProperty::skipLazyInitialization() e ReflectionProperty::setRawValueWithoutLazyInitialization() oferecem maneiras de contornar a inicialização lenta ao acessar uma propriedade.

Sobre Estratégias de Objetos Lentos

Fantasmas lentos são objetos que se inicializam no local e, uma vez inicializados, são indistinguíveis de um objeto que nunca foi lento. Esta estratégia é adequada quando são controladas tanto a instanciação quanto a inicialização do objeto, tornando-a inadequada se qualquer um deles for gerenciado por outra parte.

Proxies lentos, uma vez inicializados, atuam como proxies para uma instância real: qualquer operação em um proxy lento inicializado é encaminhada para a instância real. A criação da instância real pode ser delegada a outra parte, tornando esta estratégia útil nos casos em que fantasmas lentos são inadequados. Embora os proxies lentos sejam quase tão transparentes quanto os fantasmas lentos, é necessário cautela quando sua identidade é usada, pois o proxy e sua instância real possuem identidades distintas.

Ciclo de Vida de Objetos Lentos

Os objetos podem se tornar lentos no momento da instanciação usando ReflectionClass::newLazyGhost() ou ReflectionClass::newLazyProxy(), ou após a instanciação usando ReflectionClass::resetAsLazyGhost() ou ReflectionClass::resetAsLazyProxy(). Depois disso, um objeto lento pode ser inicializado por meio de uma das seguintes operações:

- Interagir com o objeto de uma forma que acione a inicialização automática. Consulte Gatilhos de inicialização.
- Marcar todas as suas propriedades como não-lentas usando ReflectionProperty::skipLazyInitialization() ou ReflectionProperty::setRawValueWithoutLazyInitialization().
- Chamar explicitamente ReflectionClass::initializeLazyObject() ou ReflectionClass::markLazyObjectAsInitialized().

Como os objetos lentos são inicializados quando todas as suas propriedades são marcadas como não-lentas, os métodos acima não marcarão um objeto como lento se nenhuma propriedade puder ser marcada como lenta.

Gatilhos de Inicialização

Objetos lentos são projetados para serem totalmente transparentes para seus consumidores, portanto, operações normais que observam ou modificam o estado do objeto acionará automaticamente a inicialização antes que a operação seja executada. Isto inclui, mas não está limitado às seguintes operações:

- Ler ou escrever uma propriedade.
- Testar se uma propriedade está definida ou desativada.
- Acessar ou modificar uma propriedade via ReflectionProperty::getValue(), ReflectionProperty::getRawValue(), ReflectionProperty::setValue() ou ReflectionProperty::setRawValue().
- Listae propriedades com **ReflectionObject::getProperties()**, **ReflectionObject::getProperty()** ou get_object_vars().

- Iterar sobre propriedades de um objeto que não implementa Iterator ou IteratorAggregate usando foreach.
- Serializar o objeto com serialize(), json_encode() etc.
- Clonar o objeto.

Chamadas de método que não acessam o estado do objeto não acionarão a inicialização. Da mesma forma, as interações com o objeto que invocam métodos mágicos ou funções de gancho não acionarão a inicialização se esses métodos ou funções não acessarem o estado do objeto.

Operações sem Acionamento

Os métodos específicos ou operações de baixo nível a seguir permitem acesso ou modificação de objetos lentos sem acionar a inicialização:

- Marcar propriedades como não-lentas com ReflectionProperty::skipLazyInitialization() ou ReflectionProperty::setRawValueWithoutLazyInitialization().
- Recuperar a representação interna de propriedades usando get_mangled_object_vars() ou através da conversão de um objeto para um array.
- Usar serialize() quando `ReflectionClass::SKIP_INITIALIZATION_ON_SERIALIZE` está definida, a menos que __serialize() ou __sleep() acionem a inicialização.
- Chamar **ReflectionObject::__toString()**.
- Usar var_dump() ou debug_zval_dump(), a menos que __debugInfo() acione a inicialização.

Sequência de Inicialização

Esta seção descreve a sequência de operações executadas quando a inicialização é acionada, com base na estratégia em uso.

Objetos Fantasmas

- O objeto é marcado como não-lento.
- Propriedades não inicializadas com ReflectionProperty::skipLazyInitialization() ou ReflectionProperty::setRawValueWithoutLazyInitialization() são definidas com seus valores padrão, se houver. Neste estágio, o objeto se assemelha a um criado com ReflectionClass::newInstanceWithoutConstructor(), exceto pelas propriedades já inicializadas.
- A função inicializadora é então chamada com o objeto como seu primeiro parâmetro. A função é esperada, mas não obrigatória, para inicializar o estado do objeto e deve retornar `null` ou nenhum valor. O objeto não é mais lento neste ponto, então a função pode acessar suas propriedades diretamente.

Após a inicialização, o objeto é indistinguível de um objeto que nunca foi lento.

Objetos Proxy

- O objeto é marcado como não-lento.
- Ao contrário dos objetos fantasmas, as propriedades do objeto não são modificadas neste estágio.
- A função de fábrica é chamada com o objeto como seu primeiro parâmetro e deve retornar uma instância não-lenta de uma classe compatível (veja ReflectionClass::newLazyProxy()).
- A instância retornada é chamada de *instância real* e é anexada ao proxy.
- Os valores das propriedades do proxy são descartados como se unset() tivesse sido chamada.

Após a inicialização, acessar qualquer propriedade no proxy produzirá o mesmo resultado que acessar a propriedade correspondente na instância real; todos os acessos de propriedade no proxy são encaminhados

para a instância real, incluindo propriedades declaradas, dinâmicas, inexistentes ou marcadas com ReflectionProperty::skipLazyInitialization() ou ReflectionProperty::setRawValueWithoutLazyInitialization().

O objeto proxy em si *não* é substituído pela instância real.

Embora a fábrica receba o proxy como primeiro parâmetro, não se espera modificá-lo (modificações são permitidas, mas serão perdidas durante a etapa final de inicialização). Entretanto, o proxy pode ser usado para decisões baseadas nos valores das propriedades inicializadas, na classe, no próprio objeto ou em sua identidade. Por exemplo, o inicializador pode usar o valor de uma propriedade inicializada ao criar a instância real.

Comportamento Comum

O escopo e o contexto *$this* do inicializador ou da função de fábrica permanecem inalterados e as restrições de visibilidade usuais se aplicam.

Após a inicialização bem-sucedida, o inicializador ou função de fábrica não é mais referenciado pelo objeto e pode ser liberado se não tiver outras referências.

Se o inicializador lançar uma exceção, o estado do objeto será revertido para seu estado de pré-inicialização e o objeto será marcado como lento novamente. Em outras palavras, todos os efeitos no próprio objeto são revertidos. Outros efeitos colaterais, como efeitos em outros objetos, não são revertidos. Isso evita a exposição de uma instância parcialmente inicializada em caso de falha.

Clonagem

Clonar um objeto lento aciona sua inicialização antes do clone ser criado, resultando em um objeto inicializado.

Para objetos proxy, tanto o proxy quanto sua instância real são clonados e o clone do proxy é retornado. O método `__clone` é chamado na instância real, não no proxy. O proxy clonado e a instância real são vinculados como estão durante a inicialização, portanto, os acessos ao clone do proxy são encaminhados para o clone da instância real.

Esse comportamento garante que o clone e o objeto original mantenham estados separados. As alterações no objeto original ou no estado de seu inicializador após a clonagem não afetam o clone. Clonar o proxy e sua instância real, em vez de retornar apenas um clone da instância real, garante que a operação de clonagem retorne consistentemente um objeto da mesma classe.

Destrutores

Para fantasmas lentos, o destrutor só é chamado se o objeto tiver sido inicializado. Para proxies, o destrutor só é chamado na instância real, se existir.

Os métodos ReflectionClass::resetAsLazyGhost() e ReflectionClass::resetAsLazyProxy() podem invocar o destrutor do objeto que está sendo redefinido.

Lista de alterações da POO

As alterações no modelo de POO do PHP são registradas aqui. Descrições e outras notas relacionadas a estes recursos estão documentadas na documentação do modelo POO.

Versão	Descrição
8.4.0	Adicionado: suporte para Ganchos de Propriedade.
8.4.0	Adicionado: suporte para Objetos Lentos.
8.1.0	Adicionado: suporte ao modificador final para constantes de classe. Além disso, as constantes de interface agora são substituíveis por padrão.
8.0.0	Adicionado: suporte ao operador nullsafe *?->* para acessar propriedades e métodos em objetos que podem ser nulos.
7.4.0	Alterado: agora é possível lançar uma exceção dentro de **__toString()**.
7.4.0	Adicionado: suporte a covariância limitada de tipo de retorno e contravariância de tipo de argumento. O suporte completo à variância só estará disponível se o carregamento automático for usado. Dentro de um único arquivo, apenas referências de tipo não cíclicas são possíveis.
7.4.0	Adicionado: agora é possível usar tipos nas propriedades da classe.
7.3.0	Incompatibilidade: o desempacotamento de argumentos com instâncias de Traversable com chaves não-int não é mais suportado. Esse comportamento não era intencional e, portanto, foi removido.
7.3.0	Incompatibilidade: nas versões anteriores era possível separar as propriedades estáticas atribuindo uma referência. Isso foi removido.
7.3.0	Alterado: o operador instanceof agora permite literais como o primeiro operando, caso em que o resultado é sempre **`false`**.
7.2.0	Descontinuado: a função __autoload() foi descontinuada em favor da função spl_autoload_register().
7.2.0	Alterado: o seguinte nome não pode ser usado para nomear classes, interfaces ou traits: `object`.
7.2.0	Alterado: uma vírgula à direita agora pode ser adicionada à sintaxe de agrupamento de use para namespaces.
7.2.0	Alterado: extensão do tipo de parâmetro. Os tipos de parâmetro de métodos sobrescritos e de implementações de interface agora podem ser omitidos.
7.2.0	Alterado: métodos abstratos agora podem ser sobrescritos quando uma classe abstrata estende outra classe abstrata.
7.1.0	Alterado: os nomes a seguir não podem ser usados para nomear classes, interfaces ou traits: `void` e `iterable`.
7.1.0	Adicionado: agora é possível especificar a visibilidade das constantes de classe.
7.0.0	Descontinuado: chamadas estáticas a métodos que não são declarados estáticos.
7.0.0	Descontinuado: construtor no estilo do PHP 4. Ou seja, métodos com o mesmo nome da classe em que eles são definidos.
7.0.0	Adicionado: a declaração *use* de grupo: classes, funções e constantes importadas do mesmo namespace agora podem ser agrupadas em uma única declaração use.
7.0.0	Adicionado: o suporte a classes anônimas foi adicionado via `new class`.
7.0.0	Incompatibilidade: iterar sobre um object que não seja uma instância de Traversable agora terá o mesmo comportamento que iterar sobre arrays por referência.
7.0.0	Alterado: definir propriedades (compatíveis) em duas traits não dispara mais um erro.
5.6.0	Adicionado: o método __debugInfo().

Versão	Descrição
5.5.0	Adicionado: a constante mágica ::class.
5.5.0	Adicionado: finally para manipular exceções.
5.4.0	Adicionado: traits.
5.4.0	Alterado: se uma classe abstrata definir uma assinatura para o construtor, agora ela será imposta.
5.3.3	Alterado: métodos com o mesmo nome do último elemento de uma classe com namespace não serão mais tratados como construtores. Esta alteração não afeta classes sem namespace.
5.3.0	Alterado: classes que implementam interfaces com métodos que possuem valores padrão no protótipo não precisam mais corresponder ao valor padrão da interface.
5.3.0	Alterado: agora é possível referenciar a classe usando uma variável (por exemplo, `echo $nome_da_classe::CONSTANTE;`). O valor da variável não pode ser uma palavra-chave (por exemplo, `self`, `parent` ou `static`).
5.3.0	Alterado: um erro de nível **`E_WARNING`** será emitido se os métodos mágicos de sobrecarga forem declarados estáticos. Também impõe o requisito de visibilidade pública.
5.3.0	Alterado: antes do PHP 5.3.0, as exceções lançadas na função __autoload() não podiam ser capturadas no bloco catch e resultariam em um erro fatal. Agora as exceções lançadas na função __autoload podem ser capturadas no bloco catch, com uma condição. Se uma exceção personalizada for lançada, a classe de exceção personalizada deve estar disponível. A função __autoload pode ser usada recursivamente para carregar automaticamente a classe de exceção personalizada.
5.3.0	Adicionado: o método __callStatic.
5.3.0	Adicionado: suporte a heredoc e nowdoc nas definições de *constantes* e propriedades de classe. Nota: os valores heredoc devem seguir as mesmas regras que strings com aspas duplas, (por exemplo, sem variáveis dentro).
5.3.0	Adicionado: Late Static Bindings.
5.3.0	Adicionado: o método __invoke().
5.2.0	Alterado: o método __toString() só era chamado quando combinado diretamente com echo ou print. Mas agora, ele é chamado em qualquer contexto de string (por exemplo, em printf() com o modificador `%s`), mas não em outros contextos de tipo (por exemplo, com o modificador `%d`). A partir do PHP 5.2.0, a conversão de objects sem um método __toString para string emitirá um erro de nível **`E_RECOVERABLE_ERROR`**.
5.1.3	Alterado: nas versões anteriores do PHP 5, o uso de `var` era considerado descontinuado e emitiria um erro de nível **`E_STRICT`**. Agora não é mais descontinuado, portanto não emite o erro.
5.1.0	Alterado: o método estático __set_state() agora é chamado por classes exportadas com a função var_export().
5.1.0	Adicionado: os métodos __isset() e __unset().

Namespaces

Índice

Visão Geral

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

O que são namespaces? Na definição mais ampla, os namespaces (na tradução literal, "espaços de nomes") são uma forma de encapsular itens. Isso é visto como um conceito abstrato em muitos lugares. Por exemplo, em qualquer sistema operacional, os diretórios servem para agrupar arquivos relacionados, e agem como um namespace para os arquivos dentro deles. Como um exemplo concreto, o arquivo `foo.txt` pode existir em ambos os diretórios `/home/greg` e `/home/other`, mas duas cópias de `foo.txt` não podem coexistir em um mesmo diretório. Além disso, para acessar o arquivo `foo.txt` fora do diretório `/home/greg`, o nome do diretório deve ser prefixado ao nome do arquivo usando o separador de diretórios para obter `/home/greg/foo.txt`. Esse mesmo princípio se estende a namespaces no mundo da programação.

No mundo PHP, os namespaces são projetados para resolver dois problemas que os autores de bibliotecas e aplicações encontram ao criar elementos de código reutilizáveis, como classes ou funções:

1. Colisões de nomes entre o código criado e classes/funções/constantes internas do PHP ou classes/funções/constantes de terceiros.
2. Capacidade de criar apelidos (ou encurtar) Nomes_Extralongos projetados para aliviar o primeiro problema, melhorando a legibilidade do código-fonte.

Os Namespaces do PHP fornecem uma maneira de agrupar classes, interfaces, funções e constantes relacionadas. Aqui está um exemplo da sintaxe de namespace em PHP:

Exemplo #1 Exemplo da sintaxe de namespace

```php
<?php
namespace meu\nome; // veja a seção "Definindo Namespaces"

class MinhaClasse {}
function minhafuncao() {}
const MINHACONST = 1;

$a = new MinhaClasse;
print $a::class . "\n";
$c = new \meu\nome\MinhaClasse; // veja a seção "Espaço Global"
print $c::class . "\n";

$a = strlen('oi'); // veja a seção "Usando namespaces: recorrendo a
                   // funções/constantes globais"
print $a . "\n";

$d = namespace\MINHACONST; // veja a seção "operador de namespace
                           // e constante __NAMESPACE__"
print $d . "\n";
$d = __NAMESPACE__ . '\MINHACONST';
echo constant($d); // veja a seção "Namespaces e recursos de linguagem dinâmica"
```

> **Nota** :
>
> Nomes de namespace não diferenciam maiúsculas de minúsculas.

> **Nota** :

> O nome do namespace `PHP` e os nomes compostos que começam com esse nome (como `PHP\Classes`) são reservados para uso interno da linguagem e não devem ser usados no código criado pelo usuário.

Namespaces

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Embora qualquer código PHP válido possa estar contido em um namespace, apenas os seguintes tipos de código são afetados por namespaces: classes (incluindo abstratas, traits e enumerações), interfaces, funções e constantes.

Namespaces são declarados usando a palavra-chave `namespace`. Um arquivo contendo um namespace deve declarar o namespace no início do arquivo antes de qualquer outro código - com uma exceção: a palavra-chave declare.

Exemplo #1 Declarando um único namespace

```php
<?php
namespace MeuProjeto;

const CONEXAO_OK = 1;
class Conexao { /* ... */ }
function conectar() { /* ... */ }
```

> **Nota** :
>
> Nomes totalmente qualificados (ou seja, nomes que começam com uma barra invertida) não são permitidos em declarações de namespace, por serem construções interpretadas como expressões de namespace relativos.

A única construção de código permitida antes de uma declaração de namespace é a declaração `declare`, para definir a codificação de um arquivo-fonte. Além disso, nenhum código não-PHP pode preceder uma declaração de namespace, incluindo espaço em branco extra:

Exemplo #2 Declarando um único namespace

```php
<html>
<?php
namespace MeuProjeto; // erro fatal - namespace precisa ser a primeira declaração no script
```

Além disso, ao contrário de qualquer outra construção do PHP, o mesmo namespace pode ser definido em vários arquivos, permitindo a divisão do conteúdo de um namespace no sistema de arquivos.

Sub-namespaces

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Assim como diretórios e arquivos, os namespaces do PHP podem especificar uma hierarquia de namespaces. Assim, um namespace pode ser definido com subníveis.

Exemplo #1 Declarando um único namespace com hierarquia

```php
<?php
namespace MeuProjeto\Nivel\SubNivel;

const CONEXAO_OK = 1;
class Conexao { /* ... */ }
function conectar() { /* ... */  }
```

O exemplo acima cria a constante `MeuProjeto\Nivel\SubNivel\CONEXAO_OK`, a classe `MeuProjeto\Nivel\SubNivel\Conexao` e a função `MeuProjeto\Nivel\SubNivel\conectar`.

Definindo vários namespaces no mesmo arquivo

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Vários namespaces também podem ser declarados no mesmo arquivo. Existem duas sintaxes permitidas.

Exemplo #1 Declarando vários namespaces, sintaxe de combinação simples

```php
<?php
namespace MeuProjeto;

const CONEXAO_OK = 1;
class Conexao { /* ... */ }
function conectar() { /* ... */  }

namespace OutroProjeto;

const CONEXAO_OK = 1;
class Conexao { /* ... */ }
function conectar() { /* ... */  }
```

Essa sintaxe não é recomendada para combinar namespaces em um único arquivo. Em vez disso, é recomendado usar a sintaxe alternativa de colchetes.

Exemplo #2 Declarando vários namespaces, sintaxe de colchetes

```php
<?php
namespace MeuProjeto {

const CONEXAO_OK = 1;
class Conexao { /* ... */ }
function conectar() { /* ... */  }
}

namespace OutroProjeto {

const CONEXAO_OK = 1;
class Conexao { /* ... */ }
function conectar() { /* ... */  }
}
```

É altamente desaconselhado como prática de programação combinar vários namespaces no mesmo arquivo. O principal caso de uso é combinar vários scripts PHP no mesmo arquivo.

Para combinar código global sem namespace com código com namespace, apenas a sintaxe de colchetes é suportada. O código global deve ser encapsulado em uma declaração de namespace sem nome de namespace, como em:

Exemplo #3 Declarando vários namespaces e código sem namespace

```
<?php
namespace MeuProjeto {

const CONEXAO_OK = 1;
class Conexao {
    public static function iniciar() {
        print __METHOD__ . "\n";
    }
}

function conectar() {
    print __FUNCTION__ . "\n";
}
}

namespace { // código global
print strlen("olá") . "\n";
MeuProjeto\conectar();
MeuProjeto\Conexao::iniciar();
}
```

Nenhum código PHP pode existir fora dos colchetes do namespace, exceto uma declaração declare de abertura.

Exemplo #4 Declarando vários namespaces e código sem namespace

```
<?php
declare(strict_types=1);
namespace MeuProjeto {

const CONEXAO_OK = 1;
class Conexao {
    public static function iniciar() {
        print __METHOD__ . "\n";
    }
}

function conectar() {
    print __FUNCTION__ . "\n";
}
}

namespace { // código global
print strlen("olá") . "\n";
MeuProjeto\conectar();
MeuProjeto\Conexao::iniciar();
}
```

Fundamentos

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Antes de discutir o uso de namespaces, é importante entender como o PHP sabe qual elemento de namespace seu código está solicitando. Uma analogia simples pode ser feita entre os namespaces do PHP e um sistema de arquivos. Existem três maneiras de acessar um arquivo em um sistema de arquivos:

1. Nome de arquivo relativo como `foo.txt`. Será resolvido para `diretorioatual/foo.txt` onde `diretorioatual` é o diretório ocupado atualmente. Então, se o diretório atual for `/home/foo`, o nome será resolvido para `/home/foo/foo.txt`.

2. Nome de caminho relativo como `subdiretorio/foo.txt`. Será resolvido para `diretorioatual/subdiretorio/foo.txt`.

3. Nome de caminho absoluto como `/main/foo.txt`. Será resolvido para `/main/foo.txt`.

O mesmo princípio pode ser aplicado a elementos com namespace no PHP. Por exemplo, um nome de classe pode ser referenciado de três maneiras:

1. Nome não qualificado ou nome de classe sem prefixo como `$a = new foo();` ou `foo::metodoestatico();`. Se o namespace atual for `namespaceatual`, ele será resolvido para `namespaceatual\foo`. Se o código for global, sem namespace, ele será resolvido para `foo`.

   Uma ressalva: nomes não qualificados para funções e constantes serão resolvidos para funções e constantes globais se a função ou constante com namespace não for definida. Veja Usando namespaces: recorrendo a funções/constantes globais para obter detalhes.

2. Nome qualificado ou um nome de classe prefixado como `$a = new subnamespace\foo();` ou `subnamespace\foo::metodoestatico();`. Se o namespace atual for `namespaceatual`, ele será resolvido para `namespaceatual\subnamespace\foo`. Se o código for global, sem namespace, ele será resolvido para `subnamespace\foo`.

3. Nome totalmente qualificado ou um nome prefixado com um operador de prefixo global como `$a = new \namespaceatual\foo();` ou `\namespaceatual\foo::metodoestatico();`. Ele sempre será resolvido para o nome literal especificado no código, `namespaceatual\foo`.

Aqui está um exemplo dos três tipos de sintaxe em código real:

arquivo1.php

```php
<?php
namespace Foo\Bar\subnamespace;

const FOO = 1;
function foo() {}
class foo
{
    static function metodoestatico() {}
}
```

arquivo2.php

```php
<?php
namespace Foo\Bar;
include 'arquivo1.php';
```

```php
const FOO = 2;
function foo() {}
class foo
{
    static function metodoestatico() {}
}

/* Nome não qualificado */
foo(); // será resolvido para a função Foo\Bar\foo
foo::metodoestatico(); // será resolvido para a classe Foo\Bar\foo, método metodoestatico
echo FOO; // será resolvido para a constante Foo\Bar\FOO

/* Nome qualificado */
subnamespace\foo(); // será resolvido para a função Foo\Bar\subnamespace\foo
subnamespace\foo::metodoestatico(); // será resolvido para a classe Foo\Bar\subnamespace\foo,
                                    // método metodoestatico
echo subnamespace\FOO; // será resolvido para a constante constant Foo\Bar\subnamespace\FOO

/* Nome totalmente qualificado */
\Foo\Bar\foo(); // será resolvido para a função Foo\Bar\foo
\Foo\Bar\foo::metodoestatico(); // será resolvido para a classe Foo\Bar\foo, método metodoestatico
echo \Foo\Bar\FOO; // será resolvido para a constante Foo\Bar\FOO
```

Note que para acessar qualquer classe, função ou constante global, um nome totalmente qualificado pode ser usado, como \strlen() ou \Exception ou \INI_ALL.

Exemplo #1 Acessando classes, funções e constantes globais em um namespace

```php
<?php
namespace Foo;

function strlen() {}
const INI_ALL = 3;
class Exception {}

print \strlen('olá') . "\n"; // chama a função global strlen
print \INI_ALL . "\n"; // acessa a constante global INI_ALL
$c = new \Exception('erro'); // instancia um objeto da classe global Exception
print $c::class . "\n";
```

Namespaces e recursos de linguagem dinâmica

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

A implementação de namespaces do PHP é influenciada por sua natureza dinâmica como linguagem de programação. Assim, para converter código como o exemplo a seguir em código com namespace:

Exemplo #1 Acessando elementos dinamicamente

exemplo1.php:

```php
<?php
class nomedaclasse
{
    function __construct()
```

```
    {
        echo __METHOD__,"\n";
    }
}
function nomedafuncao()
{
    echo __FUNCTION__,"\n";
}
const nomedaconstante = "global";

$a = 'nomedaclasse';
$obj = new $a; // imprime nomedaclasse::__construct
$b = 'nomedafuncao';
$b(); // imprime nomedafuncao
echo constant('nomedaconstante'), "\n"; // imprime global
```

Deve-se usar o nome totalmente qualificado (nome da classe prefixado com o namespace). Observe que, como não há diferença entre um nome qualificado e um nome totalmente qualificado em um nome de classe, nome de função ou nome de constante dinâmico, a barra invertida inicial não é necessária.

Exemplo #2 Acessando dinamicamente elementos com namespace

```
<?php
namespace nomedonamespace;
class nomedaclasse
{
    function __construct()
    {
        echo __METHOD__,"\n";
    }
}
function nomedafuncao()
{
    echo __FUNCTION__,"\n";
}
const nomedaconstante = "comnamespace";

/* note que, ao usar aspas duplas, "\\nomedonamespace\\nomedaclasse" deve ser usado */
$a = '\nomedonamespace\nomedaclasse';
$obj = new $a; // imprime nomedonamespace\nomedaclasse::__construct
$a = 'nomedonamespace\nomedaclasse';
$obj = new $a; // também imprime nomedonamespace\nomedaclasse::__construct
$b = 'nomedonamespace\nomedafuncao';
$b(); // imprime nomedonamespace\nomedafuncao
$b = '\nomedonamespace\nomedafuncao';
$b(); // também imprime nomedonamespace\nomedafuncao
echo constant('\nomedonamespace\nomedaconstante'), "\n"; // imprime comnamespace
echo constant('nomedonamespace\nomedaconstante'), "\n"; // também imprime comnamespace
```

Certifique-se de ler a nota sobre escapamento de namespaces em strings.

Palavra-chave namespace e __NAMESPACE__

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

O PHP suporta duas maneiras de acessar elementos de forma abstrata dentro do namespace atual, a constante mágica **__NAMESPACE__** e a palavra-chave `namespace`.

O valor de __NAMESPACE__ é uma string que contém o nome do namespace atual. Em código global, sem namespace, ela contém uma string vazia.

Exemplo #1 Exemplo de __NAMESPACE__, código com namespace

```php
<?php
namespace MeuProjeto;

echo '"', __NAMESPACE__, '"'; // produz "MeuProjeto"
```

Exemplo #2 Exemplo de __NAMESPACE__, código global

```php
<?php
echo '"', __NAMESPACE__, '"'; // produz ""
```

A constante __NAMESPACE__ é útil para construir nomes de forma dinâmica, por exemplo:

Exemplo #3 Usando __NAMESPACE__ para construção dinâmica de nomes

```php
<?php
namespace MeuProjeto;

function get($nomedaclasse)
{
    $a = __NAMESPACE__ . '\\' . $nomedaclasse;
    return new $a;
}
```

A palavra-chave `namespace` pode ser usada para solicitar explicitamente um elemento do namespace atual ou um subnamespace. Ela é o equivalente para namespaces do operador `self` para classes.

Exemplo #4 O operador namespace, em um namespace

```php
<?php
namespace MeuProjeto;

use bla\bla as meu; // Veja "Usando namespaces: Apelidando/Importando"

bla\meu(); // chama a função MeuProjeto\bla\meu()
namespace\bla\meu(); // chama a função MeuProjeto\bla\meu()

namespace\funcao(); // chama a função MeuProjeto\funcao()
namespace\sub\funcao(); // chama a função MeuProjeto\sub\funcao()
namespace\nomedaclasse::metodo(); // chama o método estático "metodo" da classe
MeuProjeto\nomedaclasse
$a = new namespace\sub\nomedaclasse(); // instancia um objeto da classe MeuProjeto\sub\nomedaclasse
$b = namespace\CONSTANTE; // atribui o valor da constante MeuProjeto\CONSTANTE a $b
```

Exemplo #5 O operador namespace, em código global

```php
<?php

namespace\funcao(); // chama a função funcao()
namespace\sub\funcao(); // chama a função sub\funcao()
namespace\nomedaclasse::metodo(); // chama o método estático "metodo" da classe nomedaclasse
```

```php
$a = new namespace\sub\nomedaclasse(); // instancia um objeto da classe sub\nomedaclasse
$b = namespace\CONSTANTE; // atribui o valor da constante CONSTANTE a $b
```

Apelidando e Importando

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

A capacidade de se referir a um nome totalmente qualificado externo com um apelido, ou importação, é um recurso importante dos namespaces. Isso é semelhante à capacidade dos sistemas de arquivos baseados em Unix de criar links simbólicos para um arquivo ou diretório.

O PHP pode apelidar ou importar constantes, funções, classes, interfaces, traits, enums e namespaces.

A criação de apelidos é realizada com o operador `use`. Aqui está um exemplo mostrando todos os 5 tipos de importação:

Exemplo #1 Importando ou apelidando com o operador use

```php
<?php
namespace foo;
use Caminho\Completo\Classe as Outra;

// Isso é o mesmo que use Caminho\Completo\NomeNamespace as NomeNamespace
use Caminho\Completo\NomeNamespace;

// Importando uma classe global
use ArrayObject;

// Importando uma função
use function Caminho\Completo\nomeFuncao;

// Apelidando uma função
use function Caminho\Completo\nomeFuncao as funcao;

// Importando uma constante
use const Caminho\Completo\CONSTANTE;

$obj = new namespace\Outra; // Instancia um objeto da classe foo\Outra
$obj = new Outra; // Instancia um objeto da classe Caminho\Completo\Classe
NomeNamespace\SubNamespace\funcao(); // Chama a função
Caminho\Completo\NomeNamespace\SubNamespace\funcao
$a = new ArrayObject(array(1)); // Instancia um objeto da classe ArrayObject
// Sem a declaração "use ArrayObject" teria sido instanciado um objeto da classe foo\ArrayObject
funcao(); // chama a função Caminho\Completo\nomeFuncao
echo CONSTANTE; // imprime o valor de Caminho\Completo\CONSTANTE
```

Note que para nomes com namespaces (nomes de namespaces totalmente qualificados contendo o separador de namespace, como `Foo\Bar`, em oposição a nomes globais sem separadores, como `FooBar`), a barra invertida inicial não é necessária e não é recomendada, pois nomes importados precisam ser totalmente qualificados e não são processados em relação ao namespace atual.

O PHP também suporta um atalho de conveniência para colocar várias instruções use na mesma linha.

Exemplo #2 Importando ou criando apelidos com várias declarações use combinadas

```php
<?php
use Caminho\Completo\Classe as Outra, Caminho\Completo\NomeNamespace;

$obj = new Outra;    // Instancia um objeto da classe Caminho\Completo\Classe
NomeNamespace\SubNamespace\funcao(); // Chama a função
Caminho\Completo\NomeNamespace\SubNamespace\funcao
```

A importação é realizada em tempo de compilação e por isso, não afeta classes, funções ou nomes de constantes gerados dinamicamente.

Exemplo #3 Importação e nomes dinâmicos

```php
<?php
use Caminho\Completo\Classe as Outra, Caminho\Completo\NomeNamespace;

$obj = new Outra; // Instancia um objeto da classe Caminho\Completo\Classe
$a = 'Outra';
$obj = new $a;    // Instancia um objeto da classe Outra
```

Além disso, a importação afeta apenas nomes não qualificados e qualificados. Nomes totalmente qualificados são absolutos e não são afetados pelas importações.

Exemplo #4 Importação e nomes completamente qualificados

```php
<?php
use Caminho\Completo\Classe as Outra, Caminho\Completo\NomeNamespace;

$obj = new Outra;        // Instancia um objeto da classe Caminho\Completo\Classe
$obj = new \Outra;       // Instancia um objeto da classe Outra
$obj = new Outra\Coisa;  // Instancia um objeto da classe Caminho\Completo\Classe\Coisa
$obj = new \Outra\Coisa; // Instancia um objeto da classe Outra\Coisa
```

Regras de escopo para importação

A palavra-chave `use` precisa ser declarada no escopo mais externo do arquivo (o escopo global) ou dentro de declarações de namespaces. Isso ocorre por a importação ser realizada em tempo de compilação e não em tempo de execução, portanto, não pode ter escopo de bloco. O exemplo a seguir mostra um uso inválido da palavra-chave `use`:

Exemplo #5 Regra de importação inválida

```php
<?php
namespace Linguas;

function paraGroenlandes()
{
    use Linguas\Dinamarques;

    // ...
}
```

> **Nota** :

> As regras de importação são por arquivo, o que significa que os arquivos incluídos *NÃO* herdarão as regras de importação do arquivo pai.

Declarações `use` em grupo

Classes, funções e constantes importadas do mesmo `namespace` podem ser agrupadas numa única declaração `use`.

```
<?php

use algum\namespace\ClasseA;
use algum\namespace\ClasseB;
use algum\namespace\ClasseC as C;

use function algum\namespace\fn_a;
use function algum\namespace\fn_b;
use function algum\namespace\fn_c;

use const algum\namespace\ConstA;
use const algum\namespace\ConstB;
use const algum\namespace\ConstC;

// É equivalente à seguinte declaração "use" agrupada
use algum\namespace\{ClasseA, ClasseB, ClasseC as C};
use function algum\namespace\{fn_a, fn_b, fn_c};
use const algum\namespace\{ConstA, ConstB, ConstC};
```

Espaço Global

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Quando um namespace não é definido, todas as definições de classes e funções são colocadas no espaço global, como era no PHP antes de os namespaces serem suportados. Prefixar um nome com `\` especificará que o nome é necessário no espaço global, mesmo no contexto do namespace.

Exemplo #1 Usando a especificação do espaço global

```
<?php
namespace A\B\C;

/* Esta função é A\B\C\fopen */
function fopen() {
     /* ... */
     $f = \fopen(...); // chama a função fopen global
     return $f;
}
```

Recorrendo ao espaço global

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Dentro de um namespace, quando o PHP encontra um nome não qualificado em um nome de classe, função ou contexto de constante, eles são resolvidos com diferentes prioridades. Nomes de classe sempre são

resolvidos para o nome do namespace atual. Portanto, para acessar classes internas ou classes de usuário sem namespace, deve-se referir a elas com o nome totalmente qualificado como em:

Exemplo #1 Acessando classes globais em um namespace

```php
<?php
namespace A\B\C;
class Exception extends \Exception {}

$a = new Exception('oi'); // $a é um objeto da classe A\B\C\Exception
var_dump($a::class);
$b = new \Exception('oi'); // $b é um objeto da classe Exception
var_dump($b::class);

$c = new ArrayObject; // erro fatal, classe A\B\C\ArrayObject não encontrada
```

Para funções e constantes, o PHP recorrerá a funções ou constantes globais se a função ou constante com namespace não existir.

Exemplo #2 Recorrendo a funções/constantes globais em um namespace

```php
<?php
namespace A\B\C;

const E_ERROR = 45;
function strlen($str)
{
    return \strlen($str) - 1;
}

echo E_ERROR, "\n"; // imprime "45"
echo INI_ALL, "\n"; // imprime "7" - recorre à constante INI_ALL global

echo strlen('oi'), "\n"; // imprime "1"
if (is_array('oi')) { // imprime "não é array"
    echo "é array\n";
} else {
    echo "não é array\n";
}
```

Regras de resolução de nomes

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Para efeitos destas regras de resolução, aqui estão algumas definições importantes:

Definições de nomes de namespace
Nome não qualificado

Este é um identificador sem separador de namespace, como `Foo`.

Nome qualificado

Este é um identificador com separador de namespace, como `Foo\Bar`.

Nome totalmente qualificado

Este é um identificador com separador de namespace que começa com um separador de namespace, como `\Foo\Bar`. O namespace `\Foo` também é um nome totalmente qualificado.

Nome relativo

Este é um identificador que começa com `namespace`, como `namespace\Foo\Bar`.

Os nomes são resolvidos seguindo estas regras de resolução:

1. Nomes totalmente qualificados sempre são resolvidos para o nome sem separador de namespace à esquerda. Por exemplo, `\A\B` será resolvido para `A\B`.
2. Nomes relativos sempre são resolvidos para o nome com `namespace` substituído pelo namespace atual. Se o nome ocorrer no namespace global, o prefixo `namespace\` será removido. Por exemplo, `namespace\A` dentro do namespace `X\Y` será resolvido para `X\Y\A`. O mesmo nome dentro do namespace global será resolvido para `A`.
3. Para nomes qualificados, o primeiro segmento do nome será traduzido conforme a tabela de importação de classe/namespace atual. Por exemplo, se o namespace `A\B\C` for importado como `C`, o nome `C\D\E` será traduzido para `A\B\C\D\E`.
4. Para nomes qualificados, se nenhuma regra de importação se aplicar, o namespace atual será prefixo ao nome. Por exemplo, o nome `C\D\E` dentro do namespace `A\B`, será resolvido para `A\B\C\D\E`.
5. Para nomes não qualificados, o nome será traduzido conforme a tabela de importação atual para o respectivo tipo de símbolo. Isso significa que nomes semelhantes a classes serão traduzidos conforme a tabela de importação de classe/namespace, nomes de funções conforme a tabela de importação de funções e constantes conforme a tabela de importação de constantes. Por exemplo, após a declaração `use A\B\C;` um uso como `new C()` será resolvido para o nome `A\B\C()`. Da mesma forma, após a declaração `use function A\B\foo;` um uso como `foo()` será resolvido para o nome `A\B\foo`.
6. Para nomes não qualificados, se nenhuma regra de importação se aplicar e o nome se referir a um símbolo de classe, o namespace atual será prefixo. Por exemplo `new C()` dentro do namespace `A\B` será resolvido para o nome `A\B\C`.
7. Para nomes não qualificados, se nenhuma regra de importação se aplicar e o nome se referir a uma função ou constante e o código estiver fora do namespace global, o nome será resolvido em tempo de execução. Supondo que o código esteja no namespace `A\B`, é assim que uma chamada para a função `foo()` será resolvida:
    1. Procura pela função no namespace atual: `A\B\foo()`.
    2. Tenta encontrar e chamar a função *global* `foo()`.

Exemplo #1 Resoluções de nomes ilustradas

```php
<?php
namespace A;
use B\D, C\E as F;

// chamadas de funções

foo();      // primeiro tenta chamar "foo" definida no namespace "A"
            // então chama a função global "foo"
```

```
\foo();      // chama a função "foo" definida no escopo global

minha\foo(); // chama a função "foo" definida no namespace "A\minha"

F();         // primeiro tenta chamar "F" definida no namespace "A"
             // e então chama a função global "F"

// referências de classe

new B();    // cria um objeto da classe "B" definida no namespace "A"
            // se não for encontrada, tenta carregar automaticamente a classe  "A\B"

new D();    // usando regras de importação, cria um objeto da classe "D" definida no namespace "B"
            // se não for encontrada, tenta carregar automaticamente a classe "B\D"

new F();    // usando regras de importação, cria um objeto da classe "E" definida no namespace "C"
            // se não for encontrada, tenta carregar automaticamente a classe "C\E"

new \B();   // cria um objeto da classe "B" definida no escopo global
            // se não for encontrada, tenta carregar automaticamente a classe "B"

new \D();   // cria um objeto da classe "D" definida no escopo global
            // se não for encontrado, tenta carregar automaticamente a classe "D"

new \F();   // cria um objeto da classe "F" definida no escopo global
            // se não for encontrada, tenta carregar automaticamente a classe "F"

// métodos estáticos/funções de namespace de outro namespace

B\foo();    // chama a função "foo" do namespace "A\B"

B::foo();   // chama o método "foo" da classe "B" definida no namespace "A"
            // se a classe "A\B" não for encontrada, tenta carregar automaticamente a classe "A\B"

D::foo();   // usando regras de importação, chama o método "foo" da classe "D" definida no namespace
"B"
            // se a classe "B\D" não for encontrada, tenta carregar automaticamente a classe "B\D"

\B\foo();   // chama a função "foo" do namespace "B"

\B::foo();  // chama o método "foo" da classe "B" do escopo global
            // se a classe "B" não for encontrada, tenta carregar automaticamente a classe "B"

// métodos estáticos/funções de namespace do namespace atual

A\B::foo();   // chama o método "foo" da classe "B" do namespace "A\A"
              // se a classe "A\A\B" não for encontrada, tenta carregar automaticamente a class
"A\A\B"

\A\B::foo();  // chama o método "foo" da classe "B" do namespace "A"
              // se a classe "A\B" não for encontrada, tenta carregar automaticamente a classe "A\B"
```

Perguntas Frequentes

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Estas perguntas frequentes são divididas em duas seções: perguntas comuns e alguns detalhes de implementação úteis para entender os pormenores.

Primeiro, as perguntas comuns.

1. Se não usar namespaces, devo me importar com esta documentação?
2. Como usar classes internas ou globais em um namespace?
3. Como usar classes, funções ou constantes de namespace em seu próprio namespace?
4. Como nomes como `\meu\nome` ou `\nome` são resolvidos?
5. Como um nome como `meu\nome` é resolvido?
6. Como um nome de classe não qualificado como `nome` é resolvido?
7. Como um nome de função não qualificado ou nome de constante não qualificado como `nome` é resolvido?

Há alguns detalhes de implementação dos namespaces úteis para entendê-los melhor.

1. Nomes de importação não devem entrar em conflito com classes definidas no mesmo arquivo
2. Namespaces aninhados não são permitidos.
3. Nomes de namespace dinâmicos (identificadores entre aspas) devem escapar a barra invertida
4. Constantes indefinidas referenciadas usando qualquer barra invertida geram erro fatal
5. Não é possível substituir as constantes especiais **`null`**, **`true`** ou **`false`**

Se não usar namespaces, devo me importar com esta documentação?

Não. Namespaces não afetam nenhum código existente de forma alguma, ou qualquer código ainda a ser escrito que não contenha namespaces. Você pode escrever este código se desejar:

Exemplo #1 Acessando classes globais fora de um namespace

```php
<?php
$a = new \stdClass;
print $a::class;
```

O exemplo acima é equivalente ao seguinte código:

Exemplo #2 Acessando classes globais fora de um namespace

```php
<?php
$a = new stdClass;
print $a::class;
```

Como usar classes internas ou globais em um namespace?

Exemplo #3 Acessando classes internas em namespaces

```php
<?php
namespace foo;
$a = new \stdClass;
print $a::class;

function test(?\ArrayObject $parameter_type_example = null) {}

$a = \FilesystemIterator::CURRENT_AS_FILEINFO;
print $a . "\n";

// estendendo uma classe interna ou global
class MinhaExcecao extends \Exception {}
print MinhaExcecao::class . "\n";
```

Como usar classes, funções ou constantes de namespace em seu próprio namespace?

Exemplo #4 Acessando classes, funções ou constantes internas em namespaces

```php
<?php
namespace foo;

class MinhaClasse {}

// usando uma classe do namespace atual como tipo de parâmetro
function test(?MinhaClasse $exemplo_com_tipo_de_parametro = null) {}
// outra forma de usar a classe do namespace atual como tipo de parâmetro
function test(?\foo\MinhaClasse $exemplo_com_tipo_de_parametro = null) {}

// estendendo uma classe do namespace atual
class Estendida extends MinhaClasse {}
print Estendida::class . "\n";

// acessando uma função global
$a = \strlen("teste");
print $a . "\n";

// acessando uma constante global
$b = \INI_ALL;
print $b . "\n";
```

Como nomes como `\meu\nome` ou `\nome` são resolvidos?

Nomes que começam com uma barra invertida (\) sempre são resolvidos para seu conteúdo exato, então `\meu\nome` é o mesmo que `meu\nome` e `\Exception` é o mesmo que `Exception`.

Exemplo #5 Nomes totalmente qualificados

```php
<?php
namespace foo;
$a = new \meu\nome(); // instancia um objeto da classe "meu\nome"
echo \strlen('olá'); // invoca a função "strlen"
$a = \INI_ALL; // define $a com o valor da constante "INI_ALL"
```

Como um nome como `meu\nome` é resolvido?

Nomes que contêm barra invertida, mas não começam com uma barra invertida, como `meu\nome` podem ser resolvidos de duas maneiras diferentes.

Se houver uma instrução de importação que cria o apelido `meu` para outro nome, então o apelido da importação será aplicado ao `meu` em `meu\nome`.

Caso contrário, o nome do namespace atual será prefixado a `meu\nome`.

Exemplo #6 Nomes qualificados

```php
<?php
namespace foo;
use bla\bla as foo;

$a = new meu\nome(); // instancia um objeto da classe "foo\meu\nome"
foo\bar::nome(); // chama o método estático "nome" na classe "bla\bla\bar"
minha\bar(); // chama a função "foo\minha\bar"
$a = minha\BAR; // define $a com o valor da constante "foo\minha\BAR"
```

Como um nome de classe não qualificado como `nome` é resolvido?

Nomes de classes que não contêm barra invertida como `nome` podem ser resolvidos de duas maneiras diferentes.

Se houver uma instrução de importação que cria o apelido `nome` para outro nome, então o apelido da importação será aplicado.

Caso contrário, o nome do namespace atual será prefixado a `nome`.

Exemplo #7 Nomes de classe não qualificados

```php
<?php
namespace foo;
use bla\bla as foo;

$a = new nome(); // instancia um objeto da classe "foo\nome"
foo::nome(); // chama o método estático "nome" na classe "bla\bla"
```

Como um nome de função não qualificado ou nome de constante não qualificado como `nome` é resolvido?

Nomes de funções ou constantes que não contêm barra invertida como `nome` podem ser resolvidos de duas maneiras diferentes.

Primeiro, o nome do namespace atual será prefixado a `nome`.

Finalmente, se a constante ou função `nome` não existir no namespace atual, uma constante ou função global `nome` será usada se existir.

Exemplo #8 Nomes de funções ou constantes não qualificados

```php
<?php
namespace foo;
use bla\bla as foo;
```

```php
const FOO = 1;

function minha() {
    print __FUNCTION__ . "\n";
}
function foo() {
    print __FUNCTION__ . "\n";
}
function sort(&$a)
{
    \sort($a); // chama a função global "sort"
    $a = array_flip($a);
    return $a;
}

minha(); // chama "foo\minha"
print strlen('olá') . "\n"; // chama a função global "strlen" porque não existe "foo\strlen"
$arr = array(1,3,2);
$b = sort($arr); // chama a função "foo\sort"
var_dump($b);
foo(); // chama a função "foo\foo" - a importação não é aplicada

$a = FOO; // define $a com o valor da constante "foo\FOO" - a importação não é aplicada
print $a;
$b = INI_ALL; // define $b com o valor da constante global "INI_ALL"
print $b;
```

Nomes de importação não devem entrar em conflito com classes definidas no mesmo arquivo

As seguintes combinações de script são válidas:

arquivo1.php

```php
<?php
namespace minha\coisa;
class MinhaClasse {}
```

outra.php

```php
<?php
namespace outra;
class coisa {}
```

arquivo2.php

```php
<?php
namespace minha\coisa;
include 'arquivo1.php';
include 'outra.php';

use outra\coisa as MinhaClasse;
$a = new MinhaClasse; // instancia um objeto da classe coisa do namespace outra
```

Não há conflito de nomes, mesmo que a classe `MinhaClasse` exista dentro do namespace `minha\coisa`, porque a definição de MinhaClasse está em um arquivo separado. No entanto, o próximo exemplo causa um erro

fatal de conflito de nomes porque MinhaClasse está definida no mesmo arquivo que a declaração use.

```php
<?php
namespace minha\coisa;
use outra\coisa as MinhaClasse;
class MinhaClasse {} // erro fatal: MinhaClasse está em conflito com a declaração de importação
$a = new MinhaClasse;
```

Namespaces aninhados não são permitidos

O PHP não permite aninhar namespaces.

```php
<?php
namespace minha\coisa {
    namespace aninhada {
        class foo {}
    }
}
```

No entanto, é fácil simular namespaces aninhados da seguinte forma:

```php
<?php
namespace minha\coisa\aninhada {
    class foo {}
}
```

Nomes de namespace dinâmicos (identificadores entre aspas) devem escapar a barra invertida

É muito importante perceber que, como a barra invertida é usada como um caractere de escape dentro de strings, ela sempre deve ser duplicada quando usada em uma string. Caso contrário, existe o risco de consequências não desejadas:

Exemplo #9 Perigos do uso de nomes com namespace em uma string com aspas duplas

```php
<?php
$a = "\nome\perigoso"; // \n é uma nova linha dentro de strings com aspas duplas!
$obj = new $a;

$a = 'nada\perigoso'; // não há problemas aqui.
$obj = new $a;
```

Dentro de uma string com aspas simples, a sequência de escape da barra invertida é muito mais segura de usar, mas ainda é uma prática recomendada escapar as barras invertidas em todas as strings.

Constantes indefinidas referenciadas usando qualquer barra invertida geram erro fatal

Qualquer constante indefinida não qualificada como `FOO` gerará um alerta explicando que o PHP assumiu que `FOO` era o valor da constante. Qualquer constante, qualificada ou totalmente qualificada, que contenha uma barra invertida produzirá um erro fatal se não for encontrada.

Exemplo #10 Constantes não definidas

```php
<?php
namespace bar;
$a = FOO; // Produz um alerta - constante indefinida FOO assumiu "FOO"
$a = \FOO; // Erro fatal, constante de namespace indefinida FOO
```

```php
$a = Bar\FOO; // Erro fatal, constante de namespace indefinida bar\Bar\FOO
$a = \Bar\FOO; // Erro fatal, constante de namespace indefinida Bar\FOO
```

Não é possível substituir as constantes especiais `null`, `true` ou `false`

Qualquer tentativa de definir uma constante de namespace que seja uma constante especial nativa resultará em um erro fatal.

Exemplo #11 Constantes não definidas

```php
<?php
namespace bar;
const NULL = 0; // Erro fatal
const true = 'verdadeiro'; // Também um erro fatal
// etc.
```

Enumerações

Índice

Visão Geral das Enumerações

(PHP 8 >= 8.1.0)

Enumerações, ou "Enums", permitem que um desenvolvedor defina um tipo personalizado que está limitado a um número discreto de valores possíveis. Isso pode ser especialmente útil ao definir um modelo de domínio, pois permite "tornar estados inválidos irrepresentáveis."

Enumerações aparecem em muitas linguagens com uma variedade de recursos diferentes. No PHP, Enumerações são um tipo especial de objeto. A enumeração em si é uma classe, e seus possíveis casos são todos objetos de instância única dessa classe. Isso significa que casos de enumeração são objetos válidos e podem ser usados em qualquer lugar em que um objeto pode ser usado, incluindo verificações de tipo.

O exemplo mais popular de enumerações é o tipo embutido booleano, que é um tipo enumerado com valores válidos `true` e `false`. Enumerações permitem que os desenvolvedores definam suas próprias enumerações robustas.

Enumerações Básicas

Enumerações são similares às classes e compartilham o mesmo namespaces que classes, interfaces, e traits. Elas também podem ser carregadas automaticamente da mesma maneira. Uma Enumeração define um novo tipo, que possui um número fixo e limitado de valores legais possíveis.

```php
<?php

enum Naipe
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;
}
```

Essa declaração cria um novo tipo enumerado chamado `Naipe`, que possui quatro e apenas quatro valores permitidos: `Naipe::Copas`, `Naipe::Ouros`, `Naipe::Paus`, e `Naipe::Espadas`. Variáveis podem ser atribuídas a um desses valores permitidos. Uma função pode ser tipada contra um tipo enumerado, caso em que apenas os valores desse tipo podem ser passados.

```php
<?php

enum Naipe
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;
}

function pegar_uma_carta(Naipe $naipe)
{
    var_dump($naipe);
}

$val = Naipe::Ouros;

// OK
pegar_uma_carta($val);

// OK
pegar_uma_carta(Naipe::Paus);

// TypeError: pegar_uma_carta(): Argument #1 ($naipe) must be of type Naipe, string given
pegar_uma_carta('Espadas');
```

Uma enumeração pode ter zero ou mais definições `case`, sem limite máximo. Uma enumeração sem nenhum caso é sintaticamente válida, embora inútil.

Os nomes dos casos seguem as mesmas regras de sintaxe de qualquer nome do PHP, mais detalhados na seção sobre Constantes.

Por padrão, os casos não são internamente associados por um valor escalar. Ou seja, `Naipe::Copas` não é igual a `"0"`. Ao invés disso, cada caso é apoiado por um objeto único com esse nome. Isso significa que:

```php
<?php

enum Naipe
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;
}

$a = Naipe::Espadas;
$b = Naipe::Espadas;

if ($a === $b) {
    print "Naipes correspondem usando ===\n";
}
```

```php
if ($a instanceof Suit) {
    print "Naipes correspondem instanceof\n";
}

if ($a !== 'Spades') {
    print "O naipe não corresponde à string\n";
}
```

Isso também significa que os valores de enumeração nunca são < ou > do que o outro, uma vez que essas comparações não fazem sentido em objetos. Essas comparações sempre retornarão `false` ao trabalhar com valores de enumeração.

Esse tipo de caso, sem dados relacionados, é chamado de "Caso Puro". Uma enumeração que contém apenas Casos Puros é chamada de Enumeração Pura.

Todos os Casos Puros são implementados como instâncias de seus tipos 'enum'. O tipo 'enum' é representado internamente como uma classe.

Todos os Casos têm uma propriedade somente leitura, `name`, que é o nome (sensível a maiúsculas e minúsculas) do próprio caso.

```php
<?php

enum Naipe
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;
}

print Naipe::Espadas->name;
// exibe "Espadas"
```

É possível também utilizar as funções defined() e constant() para verificar a existência ou ler um item de enumeração se o nome for obtido dinamicamente. Isto é desencorajado, já que utilizar Enumerações Apoiadas funciona para a maioria dos casos.

Enumerações Apoiadas

Por padrão, os Casos não possuem um equivalente escalar. Eles são apenas objetos singleton. No entanto, existem muitos casos em que um Caso Enumerado precisa ser capaz de fazer fazer uma conversão de ida e volta para um banco de dados ou um armazenamento de dados semelhante, então ter um escalar embutido (e, portanto, trivialmente serializável) equivalente definido intrinsecamente é útil.

Para definir um equivalente escalar para uma Enumeração, a sintaxe é a seguinte:

```php
<?php

enum Naipe: string
{
    case Copas = 'C';
    case Ouros = 'O';
```

```
    case Paus = 'P';
    case Espadas = 'E';
}
```

Um caso que possui um equivalente escalar é chamado de Caso Apoiado, sendo "Apoiado" no sentido de lastreado, suportado, por um valor mais simples. Uma enumeração que possui todos Casos Apoiados é chamada Enumeração Apoiada. Uma Enumeração Apoiada pode conter apenas Casos Apoiados. Uma Enumeração Pura pode conter apenas Casos Puros.

Uma Enumeração Apoiada pode associar valores de tipos `int` ou `string`, e uma determinada enumeração suporta apenas um único tipo de cada vez (isto é, sem união de `int|string`). Se uma enumeração for marcada como tendo um equivalente escalar, então todos os casos devem possuir um escalar equivalente único definido explicitamente. Não existem equivalentes escalares gerados automaticamente (p. ex.: inteiros sequenciais). Casos apoiados devem ser únicos; dois casos apoiados não podem ter o mesmo equivalente escalar. No entanto, uma constante pode se referir a um caso, efetivamente criando um apelido. Veja as constantes de enumeração.

Valores equivalentes podem ser uma expressão escalar constante. Antes do PHP 8.2.0, os valores equivalentes deveriam ser literais ou expressões literais. Isto significa que constantes e expressões constantes não eram suportadas. Isto é, `1 + 1` era permitido, mas `1 + SOME_CONST` não era.

Casos Apoiados possuem uma propriedade somente leitura adicional, `value`, que é o valor especificado na definição.

```
<?php

enum Naipe: string
{
    case Copas = 'C';
    case Ouros = 'O';
    case Paus = 'P';
    case Espadas = 'E';
}

print Naipe::Paus->value;
// Exibe "P"
```

Para impor a propriedade `value` como somente leitura, uma variável não pode ser atribuída como uma referência para ela. Isto é, o seguinte lança um erro:

```
<?php

enum Naipe: string
{
    case Copas = 'C';
    case Ouros = 'O';
    case Paus = 'P';
    case Espadas = 'E';
}

$naipe = Naipe::Paus;
$ref = &$naipe->value;
// Erro fatal: Não é possível modificar indiretamente a propriedade somente-leitura Naipe::$value
```

Enumerações Apoiadas implementam uma interface interna BackedEnum, que expõe dois métodos adicionais:

- `from(int|string): self` recebe um escalar e retornará o caso correspondente da enumeração. Se não for encontrado, ela lançará um ValueError. Isso é útil principalmente nas situações somente valores previstos são permitidos e um valor de enumeração ausente deve ser considerado um erro de interrupção de aplicação.
- `tryFrom(int|string): ?self` recebe uma escalar e retornará o caso de enumeração correspondente. Se um não for encontrado, ela retornará `null`. Isso é útil principalmente em casos onde o escalar de entrada não é confiável e o chamador quer implementar sua própria lógica de manipulação de erros ou de valor padrão.

Os métodos `from()` e `tryFrom()` seguem as regras padrão de tipagem fraca/forte. No modo de tipagem fraca, passar um inteiro ou string é aceitável e o sistema irá coagir o valor de acordo. Passar um float também irá funcionar e será convertido. No modo de tipagem estrita, passar um inteiro para `from()` em uma enumeração associada a strings (ou vice-versa) irá resultar em um TypeError, assim como um float irá lançar um erro em todas as circunstâncias. Todos os outros tipos de parâmetro lançarão um TypeError em ambos os modos.

```php
<?php

enum Naipe: string
{
    case Copas = 'C';
    case Ouros = 'O';
    case Paus = 'P';
    case Espadas = 'E';
}

function obter_dados_do_banco($id) {
    return [
        'naipe' => 'E',
    ];
}

$registro = obter_dados_do_banco(42);
print $registro['naipe'] . "\n";

$naipe = Naipe::tryFrom('A') ?? Naipe::Espadas;
// Dados inválidos retornam null, então Naipe::Espadas em seu lugar.
print $naipe->value . "\n";

$naipe =  Naipe::from('X');
// Dados inválidos lançam um ValueError: "X" não é um valor escalar de apoio válido para a
enumeração Naipe
print $naipe->value . "\n";
```

Definir manualmente um método `from()` ou `tryFrom()` em uma Enumeração Apoiada irá resultar em um erro fatal.

Métodos de enumerações

Enumerações (tanto Puras quanto Apoiadas) podem conter métodos e podem implementar interfaces. Se uma enumeração implementa uma interface, qualquer verificação de tipo para aquela interface também aceitará todos os casos daquela enumeração.

```php
<?php

interface Colorido
{
    public function cor(): string;
}

enum Naipe implements Colorido
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;

    // Cumpre o contrato da interface.
    public function cor(): string
    {
        return match ($this) {
            Naipe::Copas, Naipe::Ouros => 'Vermelho',
            Naipe::Paus, Naipe::Espadas => 'Preto',
        };
    }

    // Não faz parte de uma interface; tudo bem.
    public function forma(): string
    {
        return "Retângulo";
    }
}

function pintar(Colorido $c)
{
   print $c->cor() . "\n";
}

pintar(Naipe::Paus);  // Funciona

print Naipe::Ouros->forma(); // imprime "Retângulo"
```

Nesse exemplo, todas as quatro instâncias de `Naipe` possuem dois métodos, `cor()` e `forma()`. Até onde o código chamador e as checagens de tipo sabem, elas se comportam exatamente da mesma forma que qualquer outra instância de objeto.

Em uma Enumeração Apoiada, a declaração de interface vai após a declaração do tipo de lastro.

```php
<?php

interface Colorido
{
    public function cor(): string;
}

enum Naipe: string implements Colorido
{
    case Copas = 'C';
    case Ouros = 'O';
```

```
    case Paus = 'P';
    case Espadas = 'E';

    // Cumpre o contrato da interface.
    public function cor(): string
    {
        return match ($this) {
            Naipe::Copas, Naipe::Ouros => 'Vermelho',
            Naipe::Paus, Naipe::Espadas => 'Preto',
        };
    }
}
```

Dentro de um método, a variável `$this` é definida e se refere à instância do Caso.

Métodos podem ser arbitrariamente complexos, mas na prática geralmente retornam um valor estático ou match no `$this` para fornecer resultados diferentes para casos diferentes.

Note que nesse caso, uma prática melhor de modelagem de dados seria definir também um Tipo 'enum' `CorDeNaipe` com valores Preto e Vermelho e retornar isso no seu lugar. No entanto, isso complicaria esse exemplo.

A hierarquia acima é logicamente similar a seguinte estrutura de classes (embora esse não seja o código real que é executado):

```
<?php

interface Colorido
{
    public function cor(): string;
}

final class Naipe implements UnitEnum, Colorido
{
    public const Copas = new self('Copas');
    public const Ouros = new self('Ouros');
    public const Paus = new self('Paus');
    public const Espadas = new self('Espadas');

    private function __construct(public readonly string $nome) {}

    public function cor(): string
    {
        return match ($this) {
            Naipe::Copas, Naipe::Ouros => 'Vermelho',
            Naipe::Paus, Naipe::Espadas => 'Preto',
        };
    }

    public function forma(): string
    {
        return "Retângulo";
    }

    public static function cases(): array
    {
```

```
        // Método ilegal, porque definir manualmente um método cases() em uma enumeração não é
permitido.
        // Veja também a seção "Listagem de valores".
    }
}
```

Métodos podem ser públicos, privados ou protegidos, apesar dos privados e protegidos serem equivalentes na prática, pois herança não é permitida.

Métodos estáticos de enumerações

Enumerações também podem ter métodos estáticos. O uso para métodos estáticos na própria enumeração é primariamente para construtores alternativos. Por exemplo:

```php
<?php

enum Tamanho
{
    case Pequeno;
    case Medio;
    case Grande;

    public static function deComprimento(int $cm): self
    {
        return match (true) {
            $cm < 50 => self::Pequeno,
            $cm < 100 => self::Medio,
            default => self::Grande,
        };
    }
}

var_dump(Tamanho::fromLength(50));
```

Métodos estáticos podem ser públicos, privados ou protegidos apesar dos privados e protegidos serem equivalentes na prática, pois herança não é permitida.

Constantes de enumeração

Enumerações podem incluir constantes, que podem ser públicas, privadas ou protegidas, apesar das privadas e protegidas serem equivalentes na prática, pois herança não é permitida.

Uma constante de enumeração pode se referir a um caso de enumeração:

```php
<?php

enum Tamanho
{
    case Pequeno;
    case Medio;
    case Grande;

    public const Enorme = self::Grande;
}
```

```php
var_dump(Tamanho::Enorme);
```

Traits

Enumerações podem utilizar Traits, que se comportam da mesma maneira que nas classes. A limitação é que usados com `use` em uma enumeração não podem conter propriedades. Eles podem incluir apenas métodos, métodos estáticos e constantes. Um Trait com propriedades resultará em um erro fatal.

```php
<?php

interface Colorido
{
    public function cor(): string;
}

trait Retangulo
{
    public function forma(): string
    {
        return "Retângulo";
    }
}

enum Naipe implements Colorido
{
    use Retangulo;

    case Copas;
    case Ouros;
    case Paus;
    case Espadas;

    public function cor(): string
    {
        return match ($this) {
            Naipe::Copas, Naipe::Ouros => 'Vermelho',
            Naipe::Paus, Naipe::Espadas => 'Preto',
        };
    }
}

$naipe = Naipe::Espadas;
var_dump($naipe->cor());
var_dump($naipe->forma());
```

Valores de enumeração em expressões constantes

Como os casos são representados como constantes na própria enumeração, eles podem ser usados como valores estáticos na maioria das expressões constantes: padrões de propriedades, padrões de variáveis estáticas, padrões de parâmetros, valores constantes globais e de classes. Eles não podem ser usados em outros valores de caso de enumeração, mas constantes normais podem se referir a um caso.

No entanto, chamadas implícitas de métodos mágicos como ArrayAccess em enumerações não são permitidas em definições estáticas ou de constantes já que não podemos garantir absolutamente que o valor

resultante é determinístico ou que a invocação do método é livre de efeitos colaterais. Chamadas de funções, chamadas de métodos e acesso às propriedades continuam a ser operações inválidas em expressões constantes.

```php
<?php

// Esta é uma definição de enumeração perfeitamente legal.
enum Direcao implements ArrayAccess
{
    case Cima;
    case Baixo;

    public function offsetExists($offset): bool
    {
        return false;
    }

    public function offsetGet($offset): mixed
    {
        return null;
    }

    public function offsetSet($offset, $value): void
    {
        throw new Exception();
    }

    public function offsetUnset($offset): void
    {
        throw new Exception();
    }
}

class Foo
{
    // Isto é permitido.
    const BAIXO = Direcao::Baixo;

    // Isto não é permitido, pois pode não ser determinístico.
    const CIMA = Direcao::Cima['curta'];
    // Fatal error: Cannot use [] on enums in constant expression
}

// Isto é perfeitamente legal, porque não é uma expressão constante.
$x = Direcao::Cima['curta'];
var_dump("\$x é " . var_export($x, true));

$foo = new Foo();
```

Diferenças de objetos

Apesar de enumerações serem construídas sobre classes e objetos, elas não suportam todas as funcionalidades relacionadas a objetos. Em particular, casos de enumeração são proibidos de ter estado.

- Construtores e Destrutores são proibidos.
- Herança não é suportada. Enumerações não podem estender ou ser estendidas.

- Propriedades estáticas ou de objeto não são permitidas.
- Clonar um caso de enumeração não é suportado, pois os casos devem ser instâncias únicas.
- Métodos mágicos, exceto para aqueles listados abaixo, não são permitidos.
- Enumerações sempre precisam ser declaradas antes de serem utilizadas.

As seguintes funcionalidades de objeto estão disponíveis, e se comportam exatamente como em qualquer outro objeto:

- Métodos públicos, privados e protegidos.
- Métodos estáticos públicos, privados e protegidos.
- Constantes públicas, privadas e protegidas.
- Enumerações podem implementar qualquer número de interfaces.
- Enumerações e casos podem ter atributos anexados a eles. O filtro de alvo **`TARGET_CLASS`** inclui as próprias enumerações. O filtro de alvo **`TARGET_CLASS_CONST`** inclui Casos de Enumeração.
- Os métodos mágicos __call, __callStatic, e __invoke
- Constantes **`__CLASS__`** e **`__FUNCTION__`** se comportam normalmente

A constante mágica `::class` em um tipo 'enum' avalia ao nome do tipo incluindo qualquer namespace, exatamente o mesmo que um objeto. A constante mágica `::class` em uma instância de Caso também avalia para o tipo 'enum', pois ele é uma instância daquele tipo.

Adicionalmente, casos não podem ser instanciados diretamente com `new`, nem com ReflectionClass::newInstanceWithoutConstructor() em reflexão. Ambos resultarão em um erro.

```php
<?php

$trevos = new Naipe();
// Erro: Não é possível instanciar a enumeração Naipe

$ferraduras = (new ReflectionClass(Naipe::class))->newInstanceWithoutConstructor()
// Erro: Não é possível instanciar a enumeração Naipe
```

Listagem de valores

Tanto Enumerações Puras quanto Enumerações Apoiadas implementam uma interface interna chamada UnitEnum. `UnitEnum` inclui um método estático `cases()`. `cases()` retorna um array embalado com todos os Casos definidos na ordem de declaração.

```php
<?php

enum Naipe
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;
}

var_dump(Naipe::cases());

enum NaipeApoiado: string
{
    case Copas = 'C';
    case Ouros = 'O';
```

```
    case Paus = 'P';
    case Espadas = 'E';
}

var_dump(NaipeApoiado::cases());
```

Definir manualmente um método `cases()` em uma enumeração resultará em um erro fatal.

Serialização

As enumerações são serializadas de maneira diferente dos objetos. Especificamente, elas têm um novo código de serialização, `"E"`, que especifica o nome do caso de enumeração. A rotina de desserialização pode então usar isso para definir uma variável para o valor singleton existente. Isso garante que:

```
<?php

enum Naipe: string
{
    case Copas = 'C';
    case Ouros = 'O';
    case Paus = 'P';
    case Espadas = 'E';
}

Naipe::Copas === unserialize(serialize(Naipe::Copas));

print serialize(Naipe::Copas);
// E:11:"Naipe:Copas";
```

Na desserialização, se uma enumeração de um caso não pode ser encontrada para combinar com um valor serializado, um aviso será emitido e **`false`** retornado.

A opção `allowed_classes` de unserialize() não afeta Enumerações.

Se uma Enumeração Pura for serializada para JSON, um erro será lançado. Se uma Enumeração Apoiada for serializada para JSON, ela será representada apenas por seu valor escalar, no tipo apropriado. O comportamento de ambas pode ser sobrescrito ao implementar JsonSerializable.

Para print_r(), a saída de um caso de enumeração é ligeiramente diferente dos objetos para minimizar confusões.

```
<?php

enum Foo
{
    case Bar;
}

enum Baz: int
{
    case Beep = 5;
}

print_r(Foo::Bar);
print_r(Baz::Beep);
```

```
/* Produz

Foo Enum (
    [name] => Bar
)
Baz Enum:int {
    [name] => Beep
    [value] => 5
}
*/
```

Porque enumerações não são extensíveis

Classes possuem contratos em seus métodos:

```php
<?php

class A {}
class B extends A {}

function foo(A $a) {}

function bar(B $b)
{
    foo($b);
}
```

Esse código é correto para tipos, pois B segue o contrato de A, e através da co e contra variância, quaisquer expectativas a respeito dos métodos será atendida.

Enumerações possuem contratos em seus itens, não em seus métodos:

```php
<?php

enum ErrorCode
{
    case SOMETHING_BROKE;
}

function quux(ErrorCode $errorCode)
{
    // Nesse primeiro momento, esse código prevê todos os casos
    match ($errorCode) {
        ErrorCode::SOMETHING_BROKE => true,
    };
}
```

A instrução match na função `quux` pode ser analisada estaticamente para verificar se cobre todos os casos possíveis em ErrorCode.

Mas imagine que seja possível estender enumerações, assim:

```php
<?php
```

```php
// Apenas para o exemplo onde enumerações não são finais.
// Isto *não* funciona no PHP.
enum MoreErrorCode extends ErrorCode
{
    case PEBKAC;
}

function fot(MoreErrorCode $errorCode)
{
    quux($errorCode);
}

fot(MoreErrorCode::PEBKAC);
```

Em condições normais de herança, uma classe que estende outra passará na checagem de tipos.

O problema ocorreria na instrução match da função `quux()` que agora não cobre todos os casos. Porque a função não prevê o item `MoreErrorCode::PEBKAC` o match irá falhar com uma exceção.

Por conta disso, as enumerações são finais e não podem ser estendidas.

Examples

Exemplo #1 Valores limitados básicos

```php
<?php

enum Ordem
{
    case Asc;
    case Desc;
}

function consulta($campos, $filtros, Ordem $ordem = Ordem::Asc)
{
     /* ... */
}
```

A função `consulta()` agora pode prosseguir segura, sabendo que a `$ordem` tem garantia de ser `Ordem::Asc` ou `Ordem::Desc`. Qualquer outro valor resultaria em um TypeError, então nenhuma verificação ou teste de erro é necessária.

Exemplo #2 Valores exclusivos avançados

```php
<?php

enum EstadoDeUsuario: string
{
    case Pendente = 'P';
    case Ativo = 'A';
    case Suspenso = 'S';
    case CanceladoPeloUsuario = 'C';

    public function rotulo(): string
    {
        return match ($this) {
```

```
            self::Pendente => 'Pendente',
            self::Ativo => 'Ativo',
            self::Suspenso => 'Suspenso',
            self::CanceladoPeloUsuario => 'Cancelado pelo usuário',
        };
    }
}

$status = EstadoDeUsuario::Suspenso;
var_dump($status->rotulo());
```

Neste exemplo, o estado de um usuário pode ser, exclusivamente, `EstadoDeUsuario::Pendente`, `EstadoDeUsuario::Ativo`, `EstadoDeUsuario::Suspenso`, ou `EstadoDeUsuario::CanceladoPeloUsuario`. Uma função pode tipar um parâmetro contra `EstadoDeUsuario` e aceitar então apenas aqueles quatro valores, ponto final.

Todos os quatros valores possuem um método `rotulo()`, que retorna uma string legível para humanos. Essa string é independente da string equivalente escalar do "nome de máquina", que pode ser usado em, por exemplo, um campo de banco de dados um uma caixa de seleção HTML.

```
<?php

enum EstadoDeUsuario: string
{
    case Pendente = 'P';
    case Ativo = 'A';
    case Suspenso = 'S';
    case CanceladoPeloUsuario = 'C';

    public function rotulo(): string
    {
        return match($this) {
            self::Pendente => 'Pendente',
            self::Ativo => 'Ativo',
            self::Suspenso => 'Suspenso',
            self::CanceladoPeloUsuario => 'Cancelado pelo usuário',
        };
    }
}

foreach (EstadoDeUsuario::cases() as $caso) {
    printf(
        "<option value=\"%s\">%s</option>\n",
        htmlentities($caso->value),
        htmlentities($caso->rotulo())
    );
}
```

Error Handling

Índice

> **Nota** :

> See also the Exception class

Errors are a normal part of the development and life cycle of an application. As such PHP provides multiple ways to inform and deal with errors.

PHP has two main mechanisms for reporting errors:

- Throwable errors, which are further split into *programming* errors via the Error class hierarchy and *recoverable* errors via the Exception class hierarchy.
- Diagnostic errors which are classified into different severities.

Throwable errors

A Throwable error is an execution event which disrupts the normal flow of instructions. When a Throwable error is `throw`n, be that by the engine or manually, it will unwind the call stack until it is caught by a `catch` block or the global scope is reached, in which case the program is terminated with a fatal error. All `finally` blocks it encounters along the way will be executed.

To throw a Throwable error from within PHP code, one must use the `throw` keyword followed by an instance of the Error or Exception class or a subclass of one of them. Trying to throw an object that is not will result in an Error being thrown with the following message: `Cannot throw objects that do not implement Throwable`.

> **Dica**
>
> The Standard PHP Library (SPL) provides various built-in exceptions. And it's possible to create custom exceptions by extending the Exception class.

It is possible to handle a Throwable error within PHP by surrounding the code susceptible to throw an error in a `try` block. A `try` block must have at least one corresponding `catch` or `finally` block.

> **Nota** :
>
> It is *not* recommended to catch Error objects as those signal a programming error.

> **Aviso**
>
> Throwable objects cannot be cloned. Attempting to clone such an object throws an Error, even if a subclass declares its own __clone() method, with the following message: `Trying to clone an uncloneable object of class Exception`.

`Global exception handler`

It's possible to add a handler for any uncaught Throwable with set_exception_handler(). This is similar to wrapping the entire code in a `try-catch` block, except that execution does not resume afterwards: the script terminates once the handler returns.

Extending Exceptions

A user-defined exception can be defined by extending the built-in Exception class. To see which methods and properties are accessible within a child class derived from Exception visit its page.

If a class extends the built-in Exception class and re-defines the constructor, it is highly recommended that it also call `parent::__construct()` to ensure all available data has been properly assigned. The __toString() method can be overridden to provide a custom output when the object is used as a string.

Exemplo #1 Extending Exception

```php
<?php
class CustomException extends Exception
{
    // Redefine the constructor so message isn't optional
    public function __construct($message, $code = 0, ?Throwable $previous = null) {
        // some code

        // Ensure everything is assigned properly
        parent::__construct($message, $code, $previous);
    }

    // Custom string representation of object
    public function __toString() {
        return __CLASS__ . ": [{$this->code}]: {$this->message}\n";
    }

    public function customFunction() {
        echo "A custom function for this type of exception\n";
    }
}

try {
    throw new CustomException("Something happened", 5);
} catch (CustomException $e) {
    echo "Caught custom exception\n", $e;
    $e->customFunction();
}
```

O exemplo acima produzirá:

```
Caught custom exception
CustomException: [5]: Something happened
A custom function for this type of exception
```

Throwable hierarchy

- Throwable
    - Error
        - ArithmeticError

            - DivisionByZeroError
        - AssertionError
        - CompileError
            - ParseError
        - TypeError
            - ArgumentCountError
        - ValueError
        - UnhandledMatchError
        - FiberError
    - Exception

The Exception class has many subclasses, among them the SPL exceptions and those defined by extensions.

Diagnostic errors

Diagnostic errors, also known as traditional errors, are used to signal a number of different conditions, and can be displayed and/or logged as required. By default PHP will report all diagnostic errors. (Prior to PHP 8.0, `E_NOTICE` and `E_DEPRECATED` diagnostics were not reported by default.)

In contrast to Throwable errors, most diagnostic errors do *not* disrupt the normal flow of execution. Fatal diagnostics, such as `E_ERROR`, terminate the script.

Which diagnostics are reported and which are ignored is controlled by the `error_reporting` *php.ini* directive, or at runtime by calling error_reporting(). It is strongly recommended that the configuration directive be set, as some errors can occur before execution of your script begins.

> **Nota** :
>
> In a development environment, `error_reporting` should always be set to `E_ALL` to be aware of and fix issues raised by PHP. However, in production the level of verbosity could be reduced to `E_ALL & ~E_NOTICE & ~E_DEPRECATED`, but in many cases `E_ALL` is still appropriate, as it may provide early warning of potential issues.

What happens to the diagnostics depends on two further *php.ini* directives. `display_errors` controls whether the diagnostic is shown as part of the script's output. This should always be disabled in a production environment, as it can include confidential information such as database passwords, but is often useful to enable in development, as it ensures immediate reporting of issues.

In addition to displaying errors, PHP can log diagnostics when the `log_errors` directive is enabled. This will log any diagnostic to the file or syslog defined by `error_log`. This can be extremely useful in a production environment, as diagnostics are logged when they occur and can then generate reports based on those logs.

Individual diagnostics can be suppressed using the `@` operator.

> **Aviso**
>
> Prior to PHP 8.0.0 it was possible to suppress critical diagnostics that would terminate the execution of the PHP script.

> **Nota** :
>
> Diagnostics can be added, removed, have their severity altered or be elevated to a Throwable error in between PHP versions.

User defined diagnostic handler

If PHP's default diagnostic handling is inadequate, it is possible to override the default diagnostic handler with a custom one which is set using set_error_handler(). Some diagnostics, such as **E_ERROR** and **E_PARSE**, cannot be handled this way (see set_error_handler() for the full list). The others can be handled in various ways. For example, this can be used to show a custom error page to the user and then report more directly than via a log, such as by sending an e-mail.

Another typical usage is to elevate diagnostic errors to exceptions with ErrorException.

Exemplo #1 Elevating diagnostic errors to exceptions

```
<?php
function exceptions_error_handler($severity, $message, $filename, $lineno) {
    if (error_reporting() & $severity) {
       throw new ErrorException($message, 0, $severity, $filename, $lineno);
    }
}

set_error_handler('exceptions_error_handler');
```

> **Nota** :
>
> It is important to verify that the severity is included in the current list of reportable diagnostics, otherwise the @ operator will be broken, as an exception will be thrown for a suppressed diagnostic.

Registro de Alterações

Versão	Descrição
8.5.0	Fatal errors (such as an exceeded maximum execution time) now include a stack trace in their error message.

Fibers

Visão geral de Fibers

(PHP 8 >= 8.1.0)

Fibers representam funções interruptíveis de pilha completa. Fibers podem ser suspensas de qualquer lugar na pilha de chamadas, pausando a execução dentro de Fiber até que Fiber seja retomada posteriormente.

Fibers pausam toda a pilha de execução, portanto, o chamador direto da função não precisa alterar a forma como invoca a função.

A execução pode ser interrompida em qualquer lugar na pilha de chamadas usando Fiber::suspend() (ou seja, a chamada para Fiber::suspend() pode estar em uma função profundamente aninhada ou nem mesmo existir).

Ao contrário dos Generators, cada Fiber tem sua própria pilha de chamadas, permitindo que sejam pausadas em chamadas de função profundamente aninhadas. Uma função que declara um ponto de interrupção (isto é, chamando Fiber::suspend()) não precisa alterar seu tipo de retorno, ao contrário de uma função que usa `yield`, que deve retornar uma instância de Generator.

Fibers podem ser suspensas em qualquer chamada de função, incluindo aquelas chamadas de dentro da VM PHP, como funções fornecidas para array_map() ou métodos chamados por `foreach` em um objeto Iterator.

Uma vez suspensa, a execução de Fiber pode ser retomada com qualquer valor usando Fiber::resume() ou lançando uma exceção na Fiber usando Fiber::throw(). O valor é retornado (ou exceção lançada) de Fiber::suspend().

> **Nota** :
>
> Antes do PHP 8.4.0, trocas de Fibers durante a execução de um destrutor de objeto não eram permitidas.

Exemplo #1 Uso básico

```php
<?php
$fiber = new Fiber(function (): void {
   $value = Fiber::suspend('fiber');
   echo "Value used to resume fiber: ", $value, PHP_EOL;
});

$value = $fiber->start();

echo "Value from fiber suspending: ", $value, PHP_EOL;

$fiber->resume('test');
?>
```

O exemplo acima produzirá:

```
Value from fiber suspending: fiber
Value used to resume fiber: test
```

Geradores

Índice

Geradores, visão geral

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Os geradores fornecem uma maneira fácil de implementar iteradores simples sem a sobrecarga ou complexidade de criar uma classe que implemente a interface Iterator.

Um gerador oferece uma maneira conveniente de fornecer dados a loops `foreach` sem ter que construir um array na memória antecipadamente, o que pode fazer com que o programa exceda um limite de memória ou exija uma quantidade considerável de tempo de processamento para essa geração. Em vez disso, uma função geradora pode ser usada, que é o mesmo que uma função normal, exceto que ao invés do retorno ocorrer uma única vez, um gerador pode executar `yield` quantas vezes forem necessárias para fornecer os valores a serem iterados. Tal como acontece com os iteradores, o acesso aleatório aos dados não é possível.

Um exemplo simples para isso é reimplementar a função range(). A função range() padrão tem que gerar um array com cada valor dentro dele e retorná-lo, o que pode resultar em grandes arrays: por exemplo, chamar **range(0, 1000000)** irá resultar numa utilização de memória de mais de 10 MB.

Como alternativa, nós podemos implementar um gerador `xrange()`, que só precisará de memória suficiente para criar um objeto Iterator e acompanhar o estado atual do gerador internamente, que utiliza menos de 1 kilobyte de memória.

Exemplo #1 Implementando range() como um gerador

```php
<?php
function xrange($start, $limit, $step = 1) {
    if ($start <= $limit) {
        if ($step <= 0) {
            throw new LogicException('O passo precisa ser positivo');
        }

        for ($i = $start; $i <= $limit; $i += $step) {
            yield $i;
        }
    } else {
        if ($step >= 0) {
            throw new LogicException('O passo precisa ser negativo');
        }

        for ($i = $start; $i >= $limit; $i += $step) {
            yield $i;
        }
    }
}

/*
 * Observe abaixo que ambos range() e xrange() tem a mesma
 * saída.
 */

echo 'Números ímpares a partir de range():  ';
foreach (range(1, 9, 2) as $number) {
    echo "$number ";
```

```php
}
echo "\n";

echo 'Números ímpares a partir de xrange(): ';
foreach (xrange(1, 9, 2) as $number) {
    echo "$number ";
}
```

O exemplo acima produzirá:

```
Números ímpares a partir de range():  1 3 5 7 9
Números ímpares a partir de xrange(): 1 3 5 7 9
```

Objetos Generator

Quando uma função geradora é chamada, um objeto interno de tipo Generator é retornado. Este objeto implementa a interface Iterator que um objeto somente leitura de uma direção implementaria, provendo também métodos que podem ser utilizados para manipular o estado do gerador, incluindo enviar e retornar valores.

Sintaxe do Gerador

Uma função geradora se parece com uma função normal, exceto que ao invés de retornar um valor, um gerador pode executar `yield` quantas vezes forem necessárias. Qualquer função que contenha `yield` é uma função geradora.

Quando uma função geradora é chamada, ela retorna um objeto que pode ser iterado. Quando você itera através desse objeto (por exemplo, por um loop `foreach`), o PHP irá chamar os métodos de iteração do objeto toda vez que precisar de um valor, em seguida salva o estado do gerador quando o valor é produzido, de modo que possa ser retomado quando o próximo valor for necessário.

Quando não houver mais valores a serem produzidos, o gerador pode simplesmente retornar, e o código que chamou continua como se um array tivesse ficado sem valores.

> **Nota** :
>
> Um gerador pode retornar valores, que podem ser recuperados usando Generator::getReturn().

A palavra-chave **yield**

O coração de uma função geradora é a palavra-chave **yield**. Na sua forma mais simples, uma declaração yield se parece muito com um retorno, exceto que em vez de parar a execução da função e retornar, o yield fornece um valor para o código de loop sobre o gerador e pausa a execução da função do gerador.

Exemplo #1 Um exemplo simples de valores yield

```php
<?php
function gen_one_to_three() {
```

```
    for ($i = 1; $i <= 3; $i++) {
        // Observe que $i é preservado entre chamadas.
        yield $i;
    }
}

$generator = gen_one_to_three();
foreach ($generator as $value) {
    echo "$value\n";
}
```

O exemplo acima produzirá:

```
1
2
3
```

> **Nota** :
>
> Internamente, chaves inteiras sequenciais serão pareadas com os valores entregues, assim como um array não associativo.

Produzindo valores com chaves

O PHP suporta arrays associativos e geradores não são diferentes. Além de produzir valores simples, como mostrado acima, você também pode produzir uma chave ao mesmo tempo.

A sintaxe para preparar um par de chave/valor é muito semelhante ao utilizado para definir um array associativo, como mostrado abaixo.

Exemplo #2 Produzindo um par de chave/valor

```
<?php
/*
 * A saída são valores separados por ponto e vírgula, com o primeiro
 * valor sendo usado como um id ou chave.
 */

$input = <<<'EOF'
1;PHP;Likes dollar signs
2;Python;Likes whitespace
3;Ruby;Likes blocks
EOF;

function input_parser($input) {
    foreach (explode("\n", $input) as $line) {
        $fields = explode(';', $line);
        $id = array_shift($fields);

        yield $id => $fields;
```

```
    }
}

foreach (input_parser($input) as $id => $fields) {
    echo "$id:\n";
    echo "    $fields[0]\n";
    echo "    $fields[1]\n";
}
```

O exemplo acima produzirá:

```
1:
    PHP
    Likes dollar signs
2:
    Python
    Likes whitespace
3:
    Ruby
    Likes blocks
```

Produzindo valores nulos

O yield pode ser chamado sem um argumento para produzir um valor `null` com uma chave automática.

Exemplo #3 Produzindo valores `null`

```
<?php
function gen_three_nulls() {
    foreach (range(1, 3) as $i) {
        yield;
    }
}

var_dump(iterator_to_array(gen_three_nulls()));
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  NULL
  [1]=>
  NULL
  [2]=>
  NULL
}
```

Produzindo valores por referência

Funções geradoras são capazes de produzir valores por referência bem como por valor. Isso é feito da mesma forma que retornar referências de funções: incluindo um & no início do nome da função.

Exemplo #4 Produzindo valores por referência

```php
<?php
function &gen_reference() {
    $value = 3;

    while ($value > 0) {
        yield $value;
    }
}

/*
 * Observe que podemos mudar $number dentro do loop, e
 * porque o gerador retorna referências, $value
 * dentro de gen_reference() é modificado.
 */
foreach (gen_reference() as &$number) {
    echo (--$number).'... ';
}
```

O exemplo acima produzirá:

```
2... 1... 0...
```

Delegação de gerador via **yield from**

A delegação de gerador permite retornar valores de outro gerador, objeto Traversable ou um array utilizando para isso a instrução **yield from**. O gerador externo retornará todos os valores do gerador interno, objeto ou array até que o mesmo não seja mais válido, a partir de onde a execução continuará no gerador externo.

Se um gerador é utilizado com **yield from**, a expressão **yield from** também retornará qualquer valor retornado pelo gerador interno.

> **Cuidado**
>
> #### Armazenando em um array (por exemplo, com iterator_to_array())
>
> **yield from** não redefine as chaves. Ele preserva as chaves retornadas pelo objeto Traversable, ou array. Deste modo, alguns valores podem compartilhar uma chave em comum com outro **yield** ou **yield from**, o qual, após inserido em um array, irá sobrescrever os valores anteriores com essa chave.
>
> Um caso onde isso importa é a função iterator_to_array() retornando um array com chaves por padrão, levando a resultados possivelmente inesperados. iterator_to_array() possui um segundo parâmetro `preserve_keys` que pode ser definido como `false` para coletar todos os valores enquanto ignora as chaves retornadas pelo Generator.

Exemplo #5 yield from com iterator_to_array()

```php
<?php
function inner() {
    yield 1; // chave 0
    yield 2; // chave 1
    yield 3; // chave 2
}
function gen() {
    yield 0; // chave 0
    yield from inner(); // chaves 0-2
    yield 4; // chave 1
}
// Passe false no segundo parâmetro para obter um array [0, 1, 2, 3, 4]
var_dump(iterator_to_array(gen()));
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(4)
  [2]=>
  int(3)
}
```

Exemplo #6 Uso básico de yield from

```php
<?php
function count_to_ten() {
    yield 1;
    yield 2;
    yield from [3, 4];
    yield from new ArrayIterator([5, 6]);
    yield from seven_eight();
    yield 9;
    yield 10;
}

function seven_eight() {
    yield 7;
    yield from eight();
}

function eight() {
    yield 8;
}

foreach (count_to_ten() as $num) {
```

```
    echo "$num ";
}
```

O exemplo acima produzirá:

```
1 2 3 4 5 6 7 8 9 10
```

Exemplo #7 yield from e valores retornados

```
<?php
function count_to_ten() {
    yield 1;
    yield 2;
    yield from [3, 4];
    yield from new ArrayIterator([5, 6]);
    yield from seven_eight();
    return yield from nine_ten();
}

function seven_eight() {
    yield 7;
    yield from eight();
}

function eight() {
    yield 8;
}

function nine_ten() {
    yield 9;
    return 10;
}

$gen = count_to_ten();
foreach ($gen as $num) {
    echo "$num ";
}
echo $gen->getReturn();
```

O exemplo acima produzirá:

```
1 2 3 4 5 6 7 8 9 10
```

Comparando geradores com objetos Iterator

A principal vantagem dos geradores é a sua simplicidade. Muito menos código clichê tem de ser reescrito em comparação com a implementação de uma classe Iterator, e o código geralmente é muito mais legível. Por exemplo, a seguinte função e classe são equivalentes:

```php
<?php
function getLinesFromFile($fileName) {
    if (!$fileHandle = fopen($fileName, 'r')) {
        return;
    }

    while (false !== $line = fgets($fileHandle)) {
        yield $line;
    }

    fclose($fileHandle);
}

// ao invés de:

class LineIterator implements Iterator {
    protected $fileHandle;

    protected $line;
    protected $i;

    public function __construct($fileName) {
        if (!$this->fileHandle = fopen($fileName, 'r')) {
            throw new RuntimeException('Não foi possível abrir o arquivo "' . $fileName . '"');
        }
    }

    public function rewind() {
        fseek($this->fileHandle, 0);
        $this->line = fgets($this->fileHandle);
        $this->i = 0;
    }

    public function valid() {
        return false !== $this->line;
    }

    public function current() {
        return $this->line;
    }

    public function key() {
        return $this->i;
    }

    public function next() {
        if (false !== $this->line) {
            $this->line = fgets($this->fileHandle);
            $this->i++;
        }
    }

    public function __destruct() {
```

```
        fclose($this->fileHandle);
    }
}
```

No entanto, essa flexibilidade tem um custo: os geradores somente movem-se para frente e não podem ser rebobinados após a iteração ter sido iniciada. Isso também significa que o geradores não pode ser iterado múltiplas vezes: o gerador precisará ser reconstruído chamando a função geradora novamente.

Veja também

- Iteração de Objetos

Atributos

Índice

Visão geral dos atributos

(PHP 8)

Os atributos PHP fornecem metadados estruturados e legíveis por máquina para classes, métodos, funções, parâmetros, propriedades e constantes. Eles podem ser inspecionados no momento da execução por meio da API Reflection, permitindo comportamento dinâmico sem modificar o código. Os atributos fornecem uma maneira declarativa de anotar código com metadados.

Atributos permitem o desacoplamento da implementação de um recurso de seu uso. Enquanto interfaces definem estrutura ao impor métodos, atributos fornecem metadados em vários elementos, incluindo métodos, funções, propriedades e constantes. Diferentemente de interfaces, que impõem implementações de métodos, atributos anotam código sem alterar sua estrutura.

Atributos podem complementar ou substituir métodos de interface opcionais fornecendo metadados em vez de estrutura imposta. Considere uma interface `ActionHandler` que representa uma operação em um aplicativo. Algumas implementações podem exigir uma etapa de configuração, enquanto outras não. Em vez de forçar todas as classes que implementam `ActionHandler` a definir um método `setUp()`, um atributo pode indicar requisitos de configuração. Essa abordagem aumenta a flexibilidade, permitindo que os atributos sejam aplicados várias vezes quando necessário.

Exemplo #1 Implementando métodos opcionais de uma interface com Atributos

```
<?php
interface ActionHandler
{
    public function execute();
}

#[Attribute]
class SetUp {}

class CopyFile implements ActionHandler
{
    public string $fileName;
    public string $targetDirectory;
```

```
    #[SetUp]
    public function fileExists()
    {
        if (!file_exists($this->fileName)) {
            throw new RuntimeException("Arquivo não existe");
        }
    }

    #[SetUp]
    public function targetDirectoryExists()
    {
        if (!file_exists($this->targetDirectory)) {
            mkdir($this->targetDirectory);
        } elseif (!is_dir($this->targetDirectory)) {
            throw new RuntimeException("Diretório de destino $this->targetDirectory não é um
diretório");
        }
    }

    public function execute()
    {
        copy($this->fileName, $this->targetDirectory . '/' . basename($this->fileName));
    }
}

function executeAction(ActionHandler $actionHandler)
{
    $reflection = new ReflectionObject($actionHandler);

    foreach ($reflection->getMethods() as $method) {
        $attributes = $method->getAttributes(SetUp::class);

        if (count($attributes) > 0) {
            $methodName = $method->getName();

            $actionHandler->$methodName();
        }
    }

    $actionHandler->execute();
}

$copyAction = new CopyFile();
$copyAction->fileName = "/tmp/foo.jpg";
$copyAction->targetDirectory = "/home/user";

executeAction($copyAction);
```

Sintaxe de Atributo

A sintaxe de atributo consiste em vários componentes-chave. Uma declaração de atributo começa com `#[` e termina com `]`. Dentro, um ou mais atributos podem ser listados, separados por vírgulas. O nome do atributo pode ser não qualificado, qualificado, ou totalmente qualificado, conforme descrito em Usando Namespaces Básicos. Argumentos para o atributo são opcionais e colocados entre parênteses `()`. Argumentos podem ser apenas valores literais ou expressões constantes. Tanto a sintaxe de argumento posicional quanto a de argumento nomeado são suportadas.

Nomes de atributos e seus argumentos são resolvidos para uma classe, e os argumentos são passados para seu construtor quando uma instância do atributo é solicitada por meio da API Reflection. Portanto, é recomendado introduzir uma classe para cada atributo.

Exemplo #1 Sintaxe de Atributo

```php
<?php
// a.php
namespace MyExample;

use Attribute;

#[Attribute]
class MyAttribute
{
    const VALUE = 'value';

    private $value;

    public function __construct($value = null)
    {
        $this->value = $value;
    }
}

// b.php

namespace Another;

use MyExample\MyAttribute;

#[MyAttribute]
#[\MyExample\MyAttribute]
#[MyAttribute(1234)]
#[MyAttribute(value: 1234)]
#[MyAttribute(MyAttribute::VALUE)]
#[MyAttribute(array("key" => "value"))]
#[MyAttribute(100 + 200)]
class Thing
{
}

#[MyAttribute(1234), MyAttribute(5678)]
class AnotherThing
{
}
```

Lendo Atributos com a API Reflection

Para acessar atributos de classes, métodos, funções, parâmetros, propriedades e constantes de classe, use o método **getAttributes()** fornecido pela API Reflection. Este método retorna um array de instâncias de ReflectionAttribute. Essas instâncias podem ser consultadas para o nome do atributo, argumentos e podem ser usadas para instanciar o atributo representado.

Separar a representação do atributo refletido de sua instância real fornece mais controle sobre o tratamento de erros, como classes de atributos ausentes, argumentos digitados incorretamente ou valores ausentes. Objetos da classe de atributo são instanciados somente após chamar ReflectionAttribute::newInstance(), garantindo que a validação do argumento ocorra naquele ponto.

Exemplo #1 Lendo Atributos Usando a API Reflection

```php
<?php

#[Attribute]
class MyAttribute
{
    public $value;

    public function __construct($value)
    {
        $this->value = $value;
    }
}

#[MyAttribute(value: 1234)]
class Thing
{
}

function dumpAttributeData($reflection) {
    $attributes = $reflection->getAttributes();

    foreach ($attributes as $attribute) {
       var_dump($attribute->getName());
       var_dump($attribute->getArguments());
       var_dump($attribute->newInstance());
    }
}

dumpAttributeData(new ReflectionClass(Thing::class));
```

O exemplo acima produzirá:

```
string(11) "MyAttribute"
array(1) {
  ["value"]=>
  int(1234)
}
object(MyAttribute)#3 (1) {
  ["value"]=>
  int(1234)
}
```

Em vez de iterar sobre todos os atributos na instância de reflexão, você pode recuperar apenas aqueles de uma classe de atributo específica passando o nome da classe de atributo como um argumento.

Exemplo #2 Lendo Atributos Específicos Usando a API Reflection

```php
<?php
#[Attribute]
class MyAttribute
{
    public $value;

    public function __construct($value)
    {
        $this->value = $value;
    }
}

#[MyAttribute(value: 1234)]
class Thing
{
}

function dumpMyAttributeData($reflection) {
    $attributes = $reflection->getAttributes(MyAttribute::class);

    foreach ($attributes as $attribute) {
       var_dump($attribute->getName());
       var_dump($attribute->getArguments());
       var_dump($attribute->newInstance());
    }
}

dumpMyAttributeData(new ReflectionClass(Thing::class));
```

Declarando Classes de Atributos

É recomendado definir uma classe separada para cada atributo. No caso mais simples, uma classe vazia com a declaração `#[Attribute]` é suficiente. O atributo pode ser importado do namespace global usando uma declaração `use`.

Exemplo #1 Classe de Atributo Simples

```php
<?php

namespace Example;

use Attribute;

#[Attribute]
class MyAttribute
{
}
```

Para restringir os tipos de declarações às quais um atributo pode ser aplicado, passe uma máscara de bits como o primeiro argumento para a declaração `#[Attribute]`.

Exemplo #2 Usando a especificação de destino para restringir onde os atributos podem ser usados

```php
<?php

namespace Example;

use Attribute;

#[Attribute(Attribute::TARGET_METHOD | Attribute::TARGET_FUNCTION)]
class MyAttribute
{
}
```

Declarar **MyAttribute** em outro tipo agora lançará uma exceção durante a chamada para ReflectionAttribute::newInstance()

Os seguintes alvos podem ser especificados:

- `Attribute::TARGET_CLASS`
- `Attribute::TARGET_FUNCTION`
- `Attribute::TARGET_METHOD`
- `Attribute::TARGET_PROPERTY`
- `Attribute::TARGET_CLASS_CONSTANT`
- `Attribute::TARGET_PARAMETER`
- `Attribute::TARGET_ALL`

Por padrão, um atributo só pode ser usado uma vez por declaração. Para permitir que um atributo seja repetível, especifique-o na máscara de bits da declaração `#[Attribute]` usando o sinalizador `Attribute::IS_REPEATABLE`.

Exemplo #3 Usando IS_REPEATABLE para permitir o atributo em uma declaração várias vezes

```php
<?php

namespace Example;

use Attribute;

#[Attribute(Attribute::TARGET_METHOD | Attribute::TARGET_FUNCTION | Attribute::IS_REPEATABLE)]
class MyAttribute
{
}
```

Referências

Índice

O que são referências

Referência em PHP significa acessar o mesmo conteúdo de variável através de vários nomes. Isto não é parecido com os ponteiros em C. Por exemplo, não é possível efetuar aritmética de ponteiros com referências, já que elas não são endereços de memória. Consulte O que referências não são para mais informações. Ao invés disso, elas são apelidos para tabela de símbolos. Observe que no PHP, um nome de variável e um conteúdo de variável são coisas diferentes, portanto o mesmo conteúdo pode ter nomes diferentes. A

analogia mais próxima é com arquivos e nomes de arquivos Unix - os nomes de variáveis são como entradas de diretórios, enquanto que o conteúdo de variável é com o conteúdo do arquivo em si. Referências são parecidas com ligações concretas no sistema de arquivo Unix.

O que as referências fazem

Há três operações básicas ao se utilizar referências: atribuição por referência, passagem por referência, e retorno por referência. Esta seção fará uma introdução dessas operações, com atalhos para leituras posteriores.

Atribuição por referência

Referências no PHP permitem criar duas variáveis que se referem ao mesmo conteúdo. Ou seja, quando se faz:

```
<?php
$a =& $b;
```

$a e *$b* apontam para o mesmo conteúdo.

> **Nota** :
>
> *$a* e *$b* são completamente iguais aqui, mas não porque *$a* está apontando para *$b* ou vice-versa, mas sim que *$a* e *$b* apontam para o mesmo lugar.

> **Nota** :
>
> Se uma variável indefinida for atribuída, passada ou retornada por referência, ela será criada.
>
> **Exemplo #1 Usando referência com variáveis indefinidas**
>
> ```
> <?php
>
> function foo(&$var) {}
>
> foo($a); // $a é "criada" e definida como null
>
> $b = array();
> foo($b['b']);
> var_dump(array_key_exists('b', $b)); // bool(true)
>
> $c = new stdClass();
> foo($c->d);
> var_dump(property_exists($c, 'd')); // bool(true)
> ```

A mesma sintaxe pode ser utilizada com funções que retornem referências:

```
<?php
$foo =& find_var($bar);
```

Usar essa mesma sintaxe com uma função que *não* retorna por referência irá gerar um erro, da mesma forma no caso do operador new. Ainda que objetos sejam internamente passados como ponteiros, isso não é o

mesmo que uma referência, como explicado em Objetos e Referências.

> **Aviso**
>
> Se for atribuída uma referência para uma variável declarada `global` dentro da função, a referência será visível somente dentro da função. Pode-se evitar isso usando o array *$GLOBALS*.
>
> **Exemplo #2 Referenciando variáveis globais de dentro de funções**

```php
<?php
$var1 = "Variável de exemplo";
$var2 = "";

function global_references($use_globals)
{
    global $var1, $var2;

    if (!$use_globals) {
        $var2 =& $var1; // visível somente dentro da função
    } else {
        $GLOBALS["var2"] =& $var1; // visível também no contexto global
    }
}

global_references(false);
echo "var2 é definida como '$var2'\n"; // var2 é definida como ''

global_references(true);
echo "var2 é definida como '$var2'\n"; // var2 é definida como 'Variável de exemplo'
```

> Veja `global $var;` como atalho para `$var =& $GLOBALS['var'];`. Assim, atribuir outra referência para `$var` somente modifica a variável de referência local.

> **Nota** :
>
> Se um valor for atribuído por referência a uma variável numa instrução `foreach`, a referência também é modificada.
>
> **Exemplo #3 Referências e o comando foreach**

```php
<?php
$ref = 0;
$row =& $ref;

foreach (array(1, 2, 3) as $row) {
    // Faz alguma coisa
}

echo $ref; // 3 - último elemento do array iterado
```

Ainda que não seja uma atribuição por referência explícita, expressões criadas com o construtor `array()` também podem se comportar como tais com o prefixo `&` no elemento de array a ser acrescentado. Exemplo:

```php
<?php
$a = 1;
```

```php
$b = array(2, 3);

$arr = array(&$a, &$b[0], &$b[1]);
$arr[0]++;
$arr[1]++;
$arr[2]++;
/* $a == 2, $b == array(3, 4); */
var_dump($a);
var_dump($b);
```

Note que referências dentro de arrays são potencialmente perigosas. Fazer uma atribuição normal (sem referência) com uma referência à direita não transforma a expressão a esquerda numa referência, mas referências dentro de arrays são preservadas nessas atribuições normais. Isso também se aplica a chamadas de função onde arrays são passados por valor. Exemplo:

```php
<?php

/* Atribuição de variáveis escalares */
$a = 1;
$b =& $a;
$c = $b;
$c = 7; // $c não é referência; não modifica $a ou $b
print "a = {$a}; b = {$b}; c = {$c}\n\n";

/* Atribuição de variáveis do array */
$arr = array(1);
$a =& $arr[0]; // $a e $arr[0] estão no mesmo conjunto de referências
$arr2 = $arr; // Não atribui por referência!
$arr2[0]++;
/* $a == 2, $arr == array(2) */
/* O conteúdo de $arr é modificado embora ele não seja uma referência! */
print "a = {$a}\n";
var_dump($arr);
var_dump($arr2);
```

Em outras palavras, o comportamento de referências em arrays é definido elemento por elemento. O comportamento de referências dos elementos individuais é dissociado do status da referência do array que os contém.

Passagem por referência

A segunda coisa que referências fazem é passar variáveis por referência. Isso é feito com a criação de uma variável local em uma função e uma variável no escopo chamador que referenciam o mesmo conteúdo. Assim:

```php
<?php
function foo(&$var)
{
    $var++;
}

$a = 5;
foo($a);
print $a;
```

a variável *$a* será 6 no final. Isto ocorre porque na função *foo* a variável *$var* se referem ao mesmo conteúdo de *$a*. Para mais informações disso veja a seção de passagem por referência.

Retorno por referência

A terceira coisa que as referências fazem é retorno por referência.

O que referências não são

Como dito anteriormente, referências não são ponteiros. Isso significa que a construção a seguir não funciona como se possa esperar:

```php
<?php
function foo(&$var)
{
    $var =& $GLOBALS["baz"];
}

foo($bar);
```

O que acontece é que *$var* em *foo* será ligada com *$bar* pelo chamador, mas depois será re-ligada com *$GLOBALS["baz"]*. Não há como ligar *$bar* no escopo chamador para outra coisa utilizando-se o mecanismo de referência, pois *$bar* não está disponível na função *foo* (ela é representada por *$var*, mas *$var* somente tem o conteúdo da variável e não a vinculação nome-valor na tabela de símbolos do chamador). Pode-se utilizar retorno por referência para referenciar variáveis selecionadas pela função.

Passagem por referência

Uma variável pode ser passada por referência a uma função de forma que a função possa modificar a variável. A sintaxe é a seguinte:

```php
<?php
function foo(&$var)
{
    $var++;
}

$a = 5;
foo($a);
print $a; // $a é 6 aqui
```

> **Nota** :
>
> Não há o sinal de referência na chamada da função - apenas nas definições de função. As definições de função, sozinhas, são suficientes para passar o argumento por referência.

Os itens seguintes podem ser passados por referência:

- Variáveis, como `foo($a)`
- Referências retornadas de funções, como:

```php
<?php

function foo(&$var)
{
    $var++;
    print $var;
}

function &bar()
{
    $a = 5;
    return $a;
}

foo(bar());
```

Veja mais sobre isso em retorno por referência.

Nenhuma outra expressão pode ser passada por referência pois não tem comportamento definido. Os seguintes exemplos de passagem por referência são inválidos:

```php
<?php

function foo(&$var)
{
    $var++;
}

function bar() // Repare que falta &
{
    $a = 5;
    return $a;
}

foo(bar()); // Gera um aviso

foo($a = 5); // Expressão, não uma variável
foo(5); // Gera um erro fatal

class Foobar {}

foo(new Foobar()) // Produz um aviso no PHP 7.0.7
                  // Somente variáveis devem ser passadas por referência
```

Retornando referências

Retorno por referência é útil quando é necessário utilizar uma função para localizar variáveis cuja referência precisa ser obtida. *Não* deve ser usado retorno por referência para aumentar desempenho, o mecanismo é esperto o bastante para otimizar isso. Referências somente devem ser retornadas quando existir uma razão técnica para isso! Para retornar referências, a sintaxe abaixo deve ser usada:

```php
<?php

class Foo
{
    public $value = 42;
```

```
    public function &getValue()
    {
        return $this->value;
    }
}

$obj = new Foo();
$myValue = &$obj->getValue(); // $myValue é uma referência a $obj->value, que é 42
$obj->value = 2;
echo $myValue;                // exibe o valor valor de $obj->value, ou seja, 2
```

Neste exemplo, a propriedade do objeto retornado pela função *getValue* precisa ser assimilada, não copiada, como acontecerá se não utilizar a sintaxe de referências.

> **Nota** :
>
> Diferentemente da passagem de parâmetros por referência, aqui é necessário utilizar `&` em ambos os lugares - primeiro para indicar o retorno por referência (e não a cópia), e depois para indicar a ligação da referência (em vez da assimilação convencional) que precisa ser explícita para *$myValue*.

> **Nota** :
>
> Se houver tentativa de retornar uma referência de uma função com a sintaxe `return ($this->value);`, isto *não* funcionará como esperado para retornar o resultado de uma *expressão*, e não uma variável, por referência. Deve-se retornar variáveis por referência somente para uma função - nada além disso.

Para utilizar a referência retornada, é necessário utilizar a atribuição por referência:

```
<?php

function &collector()
{
    static $collection = array();
    return $collection;
}

$collection = &collector();
// Agora $collection é uma variável por referência que referencia o array estático dentro da função

$collection[] = 'foo';

print_r(collector());
```

O exemplo acima produzirá:

```
Array
(
    [0] => foo
)
```

> **Nota** :
>
> Se a atribuição for feita sem o símbolo & symbol, por exemplo, `$collection = collector();`, a variável *$collection* receberá uma cópia do valor, e não a referência retornada pela função.

Para passar a referência retornada para outra função que espera uma referência, a seguinte sintaxe pode ser utilizada:

```
<?php

function &collector()
{
    static $collection = array();
    return $collection;
}

array_push(collector(), 'foo');
```

> **Nota** :
>
> Note que `array_push(&collector(), 'foo');` *não* funcionará, e retornará um erro fatal.

Destruindo referências

Quando uma referência é quebrada, ela apenas para de fazer o apontamento entre o nome da variável e o conteúdo. Mas isto não significa que o conteúdo da variável é destruído. Por exemplo:

```
<?php

$a = 1;
$b =& $a;
unset ($a);
var_dump($a);
var_dump($b);
```

não apaga *$b*, apenas *$a*.

Novamente, é mais fácil pensar em analogia ao comando UNIX **unlink**.

Demonstrando referências

Vários construtores sintáticos do PHP são implementados através de mecanismos de referência, assim, tudo o explicado até aqui sobre referências também se aplica a esses construtores. Alguns desses construtores, como a passagem e o retorno de referências foram mencionados acima. Outros construtores que também utilizam referências são:

Referências globais

Quando uma variável é declarada como **global $var**, de fato uma referência está sendo criada para a variável global. Isto significa que isto é o mesmo que:

```php
<?php
$var =& $GLOBALS["var"];
```

O que significa também que destruir *$var* não apaga a variável global.

Variáveis predefinidas

O PHP fornece variáveis predefinidas que representam variáveis externas, variáveis nativas de ambiente e outras informações sobre o ambiente de execução, como o número e os valores dos argumentos passados para o script no ambiente CLI.

Índice

Superglobais

Superglobais

Superglobais — Variáveis nativas que estão sempre disponíveis em todos escopos

Descrição

Várias variáveis predefinidas no PHP são "superglobais", que significa que elas estão disponíveis em todos os escopos para todo o script. Não há necessidade de fazer **global $variable;** para acessá-lo dentro de funções ou métodos.

Estas variáveis superglobais são:

- *$GLOBALS*
- *$_SERVER*
- *$_GET*
- *$_POST*
- *$_FILES*
- *$_COOKIE*
- *$_SESSION*
- *$_REQUEST*
- *$_ENV*

Notas

> **Nota** : **Disponibilidade da variável**
>
> Por padrão, todas as superglobais estão disponíveis, mas há diretivas que afetam esta disponibilidade. Para mais informações, consulte na documentação sobre variables_order.

> **Nota** : **Variável variáveis**
>
> Superglobais não podem ser usadas como variável variáveis dentro de função ou métodos de classes.

Veja também

- escopo de variável
- A diretiva variables_order
- A extensão filter

$GLOBALS

$GLOBALS

(PHP 4, PHP 5, PHP 7, PHP 8)

$GLOBALS — Referencia todas variáveis disponíveis no escopo global

Descrição

Um array associativo contendo referências para todas as variáveis que estão atualmente definidas no escopo global do script. O nome das variáveis são chaves do array.

Examples

Exemplo #1 Exemplo da *$GLOBALS*

```php
<?php

function teste()
{
    $foo = "variável local";

    echo '$foo no escopo global: ' . $GLOBALS["foo"] . "\n";
    echo '$foo no escopo atual: ' . $foo . "\n";
}

$foo = "Conteúdo de exemplo";
teste();

?>
```

O exemplo acima produzirá algo semelhante a:

```
$foo no escopo global: Conteúdo de exemplo
$foo no escopo atual: variável local
```

> **Aviso**
>
> A partir do 8.1.0, acesso de escrita a todo o array *$GLOBALS* não é mais suportado:
>
> **Exemplo #2 Escrever em todo o *$GLOBALS* irá resultar em um erro.**
>
> ```php
> <?php
> ```

```php
// Gera um erro em tempo de compilação:
$GLOBALS = [];
$GLOBALS += [];
$GLOBALS =& $x;
$x =& $GLOBALS;
unset($GLOBALS);
array_pop($GLOBALS);
// ...e qualquer outra operação de escrita/leitura em $GLOBALS

?>
```

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

> **Nota** : **Disponibilidade da variável**
>
> Diferente de todas as outras superglobais, *$GLOBALS* tem essencialmente sempre estado disponível no PHP.

> **Nota** :
>
> A partir do PHP 8.1.0, *$GLOBALS* agora é uma cópia somente-leitura da tabela de símbolos globais. Isto é, variáveis globais não podem ser modificadas por meio de suas cópias. Anteriormente, o array *$GLOBALS* era excluído do comportamento usual "por valor" dos arrays PHP e variáveis globais podiam ser modificadas por meio de suas cópias.

```php
<?php

// Antes do PHP 8.1.0
$a = 1;

$globals = $GLOBALS; // Cópia ostensiva "por valor"
$globals['a'] = 2;
var_dump($a); // int(2)

// A partir do PHP 8.1.0,
// isto não mais modifica $a. O comportamento anterior violava a semântica "por valor".
$globals = $GLOBALS;
$globals['a'] = 1;

// Para restaurar o comportamento anterior, itera sua cópia e atribui cada propriedade de volta
ao $GLOBALS.
foreach ($globals as $key => $value) {
    $GLOBALS[$key] = $value;
}
```

```
?>
```

$_SERVER

$_SERVER

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_SERVER — Informação do servidor e ambiente de execução

Descrição

$_SERVER é um array contendo informação como cabeçalhos, caminhos, e outras informações do script. As informações nesse array são criadas pelo servidor web específico, e portanto não há garantias que sempre existirão as mesmas informações, podendo algumas serem omitidas ou adicionadas em relação à lista a seguir. Entretanto, a maioria dessas variáveis são documentadas na » especificação CGI/1.1, e estão normalmente definidas.

> **Nota** :
>
> Quando PHP é executado na linha de comando a maioria dessas informações não estará disponível ou não terá qualquer sentido.

Além dos valores listados a seguir, o PHP irá criar itens adicionais com valores dos cabeçalhos de requisição. Esses itens serão prefixados com `HTTP_` seguido do nome do cabeçalho, capitalizados e com sublinhados ao invés de hifens. Por exemplo, o cabeçalho `Accept-Language` estará disponível como `$_SERVER['HTTP_ACCEPT_LANGUAGE']`.

Índices

'*PHP_SELF*'

O nome do arquivo do script que está sendo executado, relativo à raiz do documento. Por exemplo, *$_SERVER['PHP_SELF']* em um script no endereço *http://example.com/foo/bar.php* seria */foo/bar.php*. A constante __FILE__ contém o caminho completo e nome do arquivo atual (possivelmente incluído).

Se o PHP estiver sendo executado em linha de comando, esta variável contém o nome do script.

'argv'

Array de argumentos passado para o script. Quando o script é executado na linha de comando, isto permite um acesso aos parâmetros de linha de comando no estilo da linguagem C. Quando chamado via método GET, ele conterá a string da consulta.

'argc'

Contém o número de parâmetros da linha de comando passados para o script (se executando da linha de comando).

'*GATEWAY_INTERFACE*'

O número de revisão da especificação CGI que o servidor está utilizando, por exemplo: `'CGI/1.1'`.

'*SERVER_ADDR*'

O endereço IP do servidor onde está o script em execução.

'*SERVER_NAME*'

O nome host do servidor onde o script atual é executado. Se o script estiver sendo executado em um host virtual, este será o valor definido para esse host.

> **Nota** :
>
> No Apache 2, você deve definir `UseCanonicalName = On` e `ServerName`. Caso contrário, este valor refletirá o nome do servidor fornecido pelo cliente, que pode ser forjado. Não é seguro confiar neste valor em contextos que dependem de segurança.

'*SERVER_SOFTWARE*'

A string de identificação do servidor, fornecida nos cabeçalhos ao responder a requisições.

'*SERVER_PROTOCOL*'

Nome e número da revisão do protocolo de informação pelo qual a página foi requerida, por exemplo`'HTTP/1.0'`;

'*REQUEST_METHOD*'

Contém o método de requisição utilizado para acessar a página. Geralmente `''GET''`, `''HEAD''`, `''POST''` ou `''PUT''`.

> **Nota** :
>
> O script PHP é terminado depois dos cabeçalhos enviados (ou seja, depois de produzir alguma saída sem saída do buffer) se o método da requisição for `HEAD`.

'*REQUEST_TIME*'

O timestamp de quando o PHP começou a processar a requisição.

'*REQUEST_TIME_FLOAT*'

O timestamp de quando o PHP começou a processar a requisição, com precisão em microssegundos.

'*QUERY_STRING*'

A string de consulta, se houver, pela qual a página foi acessada.

'*DOCUMENT_ROOT*'

O diretório raiz sob o qual o script atual é executado, como definido no arquivo de configuração do servidor.

'*HTTPS*'

Definido para um valor não vazio se o script foi requisitado através do protocolo HTTPS.

'*REMOTE_ADDR*'

O endereço IP de onde o usuário está visualizando a página atual.

'*REMOTE_HOST*'

O nome do host que o usuário utilizou para chamar a página atual. O DNS reverso do *REMOTE_ADDR* do usuário.

> **Nota** :
>
> O servidor web precisa estar configurado para criar essa variável. Por exemplo, no Apache é necessário inserir um `HostnameLookups On` dentro do *httpd.conf*. Veja também gethostbyaddr().

'*REMOTE_PORT*'

A porta TCP na máquina do usuário utilizada para comunicação com o servidor web.

'*REMOTE_USER*'

O usuário autenticado.

'*REDIRECT_REMOTE_USER*'

O usuário autenticado utilizado se a requisição foi redirecionada internamente.

'*SCRIPT_FILENAME*'

O caminho absoluto do script atualmente em execução.

> **Nota** :
>
> Se o script for executado pela CLI com um caminho relativo, como *file.php* ou *../file.php*, *$_SERVER['SCRIPT_FILENAME']* irá conter o caminho relativo especificado pelo usuário.

'*SERVER_ADMIN*'

O valor fornecido pela diretiva SERVER_ADMIN (do Apache) no arquivo de configuração do servidor. Se o script está sendo executado em um host virtual, este será o valor definido para esse host virtual.

'*SERVER_PORT*'

A porta na máquina servidora utilizada pelo servidor web para comunicação. Como padrão, este valor é `''80''`. Utilizando SSL, entretanto, esse valor será alterado para a porta HTTP de comunicação segura.

> **Nota** :

> Utilizando o Apache 2, deve-se configurar `UseCanonicalName = On`, assim como `UseCanonicalPhysicalPort = On` para se obter a porta física (real), caso contrário, este valor pode ser forjado e pode, ou não, retornar como valor a porta física. Não é seguro confiar neste valor em contextos que dependem de segurança.

'*SERVER_SIGNATURE*'

String contendo a versão do servidor e nome do host virtual que é adicionado às páginas geradas no servidor, se ativo.

'*PATH_TRANSLATED*'

O caminho real do script relativo ao sistema de arquivos (não a raiz dos documentos), depois de realizados todos os mapeamentos de caminhos (virtuais para reais).

> **Nota** :
>
> Usuários do Apache 2 podem usar `AcceptPathInfo = On` dentro do *httpd.conf* para definir o *PATH_INFO*.

'*SCRIPT_NAME*'

Contém o caminho completo do script atual. Útil para páginas que precisam apontar para elas mesmas (dinamicamente). A constante __FILE__ contém o caminho completo e nome do arquivo atual (mesmo se tiver sido incluído).

'*REQUEST_URI*'

O URI fornecido para acessar a página atual, por exemplo, '`/index.html`'.

'*PHP_AUTH_DIGEST*'

Quando executando no Apache como módulo fazendo autenticação HTTP, esta variável é definida para o cabeçalho 'Authorization' enviado pelo cliente (que então pode ser usado para fazer a validação apropriada).

'*PHP_AUTH_USER*'

Ao efetuar autenticação HTTP, esta variável estará definida com o nome de usuário fornecido pelo usuário.

'*PHP_AUTH_PW*'

Ao efetuar autenticação HTTP, esta variável estará definida com a senha fornecida pelo usuário.

'*AUTH_TYPE*'

Ao efetuar autenticação HTTP, esta variável estará definida com o tipo de autenticação utilizado.

'*PATH_INFO*'

Contém qualquer informação fornecida pelo usuário, se disponível, que leve ao nome do script atual, precedendo a string de consulta. Por exemplo, se o script atual fosse acessado via a URL

http://www.example.com/php/path_info.php/some/stuff?foo=bar, então *$_SERVER['PATH_INFO']* teria como valor `/some/stuff`

'*ORIG_PATH_INFO*'

Versão original do '*PATH_INFO*' antes de ser processada pelo PHP.

Examples

Exemplo #1 Exemplo de *$_SERVER*

```php
<?php
echo $_SERVER['SERVER_NAME'];
?>
```

O exemplo acima produzirá algo semelhante a:

```
www.example.com
```

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

Veja também

- A extensão filter

$_GET

$_GET

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_GET — Variáveis de string de consulta

Descrição

Um array associativo de variáveis passadas para o script atual através dos parâmetros da URL (também conhecidos como string de consulta). Observe que este array é preenchido sempre que uma string de consulta estiver presente, independentemente do método de requisição HTTP.

Examples

Exemplo #1 Exemplo da *$_GET*

```php
<?php
echo 'Olá ' . htmlspecialchars($_GET["nome"]) . '!';
?>
```

Assumindo que o usuário digitou `http://example.com/?name=Hannes`.

O exemplo acima produzirá algo semelhante a:

```
Olá Hannes!
```

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

> **Nota** :
>
> Os valores em *$_GET* são passados automaticamente através de urldecode().

Veja também

- Manuseamento de variáveis externas
- A extensão filter

$_POST

$_POST

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_POST — Dados de formulário de requisições POST HTTP

Descrição

Um array associativo de variáveis passados para o script atual via método HTTP POST quando utilizado `application/x-www-form-urlencoded` ou `multipart/form-data` como valor do cabeçalho HTTP Content-Type na requisição.

Examples

Exemplo #1 Exemplo da *$_POST*

```php
<?php
echo 'Hello ' . htmlspecialchars($_POST["name"]) . '!';
?>
```

Assumindo que o usuário enviou uma requisição POST com `name=Hannes` no corpo.

O exemplo acima produzirá algo semelhante a:

```
Hello Hannes!
```

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

> **Nota** :
>
> Para ler dados POST enviados com outros tipos de conteúdo (por exemplo `application/json` ou `application/xml`), deve-se usar *php://input*. Ao contrário de *$_POST* que só funciona com `application/x-www-form-urlencoded` e `multipart/form-data`, o uso de *php://input* fornece acesso direto ao dado bruto do corpo da requisição.

Veja também

- Manipulando variáveis externas
- A extensão filter

$_FILES

$_FILES

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_FILES — Variáveis de Upload de Arquivos HTTP

Descrição

Um array associativo de items enviado através do script atual pelo método HTTP POST. A estrutura deste array é detalhada na seção Uploads com o método POST.

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

Veja também

- move_uploaded_file() - Move um arquivo enviado para uma nova localização
- Manuseamento de upload de arquivos

$_REQUEST

$_REQUEST

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_REQUEST — Variáveis de requisição HTTP

Descrição

Um array associativo que por padrão contém informações de *$_GET*, *$_POST* e *$_COOKIE*.

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

> **Nota** :
>
> Quando executando em linha de comando , esta *não* incluirá as entradas argv e argc; estas estão presentes no array *$_SERVER*.

> **Nota** :
>
> As variáveis em *$_REQUEST* são providas para o script via mecanismos de entradas GET, POST, e COOKIE e portanto poderia ser modificadas por um usuário remoto e não podem ser confiadas. A presença e ordem das variáveis listadas neste array é definido de acordo com o PHP request_order, e variables_order diretivas de configuração

Veja também

- Manuseamento de variáveis externas
- A extensão filter

$_SESSION

$_SESSION

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_SESSION — Variáveis de sessão

Descrição

Um array associativo contendo variáveis de sessão disponíveis para o atual script. Veja a documentação das funções de Sessão para mais informação em como usar isto.

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

Veja também

- session_start() - Inicia uma nova sessão ou continua uma sessão existente

$_ENV

$_ENV

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_ENV — Variáveis de ambiente

Descrição

Um array associativo de variáveis passadas para o script atual via o método do ambiente.

Estas variáveis são importadas para o PHP do ambiente sob o qual o parser do PHP é executado. Muitas são providas pelo shell sob o qual o PHP é executado e diferentes sistemas são prováveis executar diferentes tipos de shells, uma lista definitiva é impossível. Veja a documentação de shell para saber a lista de variáveis de ambiente definidas.

Outras variáveis de ambiente incluem variáveis CGI, elas aparecem desconsiderando se o PHP é executado como um módulo do servidor ou processador CGI.

Examples

Exemplo #1 Exemplo da *$_ENV*

```php
<?php
echo 'Meu nome de usuário é ' .$_ENV["USER"] . '!';
?>
```

Assumindo que "bjori" executou este script

O exemplo acima produzirá algo semelhante a:

```
Meu nome de usuário é bjori!
```

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

Veja também

- getenv() - Obtém o valor de uma única ou de todas as variáveis de ambiente
- A extensão filter

$_COOKIE

$_COOKIE

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_COOKIE — Cookies HTTP

Descrição

Um array associativo de variáveis passadas para o atual script via HTTP Cookies.

Examples

Exemplo #1 Exemplo da *$_COOKIE*

```php
<?php
echo 'Hello ' . htmlspecialchars($_COOKIE["name"]) . '!';
?>
```

Assumindo que o "nome" do cookie tenha sido definido antes

O exemplo acima produzirá algo semelhante a:

```
Hello Hannes!
```

Notas

> **Nota** :
>
> Esta é uma variável 'superglobal' ou variável global automática. Isso significa simplesmente que ela está disponível em todos os escopos de um script. Não há necessidade de usar **global $variable;** para acessá-la dentro de funções ou métodos.

Veja também

- setcookie() - Envia um cookie
- Manuseando variáveis externas
- A extensão filter

$php_errormsg

$php_errormsg

(PHP 4, PHP 5, PHP 7)

$php_errormsg — A mensagem de erro anterior

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 7.2.0 e foi *REMOVIDO* a partir do PHP 8.0.0. O uso deste recurso é fortemente desencorajado.

Use error_get_last() em seu lugar.

Descrição

$php_errormsg é uma variável contendo o texto da última mensagem de erro gerada pelo PHP. Esta variável somente estará disponível dentro do escopo no qual o erro ocorreu e somente se a opção de configuração track_errors estiver "on" (o padrão é "off").

> **Aviso**
>
> Se um manipulador de erro definido pelo usuário (set_error_handler()) estiver configurado, *$php_errormsg* somente será definida se a função retornar `false`.

Registro de Alterações

Versão	Descrição
8.0.0	A diretiva track_errors que criava a variável *$php_errormsg* foi removida.
7.2.0	A diretiva track_errors que criava a variável *$php_errormsg* foi descontinuada.

Examples

Exemplo #1 Exemplo de *$php_errormsg*

```
<?php
@strpos();
echo $php_errormsg;
?>
```

O exemplo acima produzirá algo semelhante a:

```
Wrong parameter count for strpos()
```

Veja também

- error_get_last() - Obtém o último erro ocorrido

$http_response_header

$http_response_header

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

$http_response_header — Cabeçalhos de resposta HTTP

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.5.0. O uso deste recurso é fortemente desencorajado.

Use http_get_last_response_headers() em seu lugar.

Descrição

O array *$http_response_header* é similar à função get_headers(). Ao usar o empacotador HTTP, a variável *$http_response_header* será populada com os cabeçalhos da resposta HTTP. *$http_response_header* será criada no escopo local.

Examples

Exemplo #1 Exemplo de *$http_response_header*

```
<?php
function get_contents() {
  file_get_contents("http://example.com");
  var_dump($http_response_header); // variável é populada no escopo local
}
get_contents();
var_dump($http_response_header); // uma chamada a get_contents() não popula a variável fora do
```

```
escopo da função
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(9) {
  [0]=>
  string(15) "HTTP/1.1 200 OK"
  [1]=>
  string(35) "Date: Sat, 12 Apr 2008 17:30:38 GMT"
  [2]=>
  string(29) "Server: Apache/2.2.3 (CentOS)"
  [3]=>
  string(44) "Last-Modified: Tue, 15 Nov 2005 13:24:10 GMT"
  [4]=>
  string(27) "ETag: "280100-1b6-80bfd280""
  [5]=>
  string(20) "Accept-Ranges: bytes"
  [6]=>
  string(19) "Content-Length: 438"
  [7]=>
  string(17) "Connection: close"
  [8]=>
  string(38) "Content-Type: text/html; charset=UTF-8"
}

Warning: Undefined variable $http_response_header
NULL
```

Veja também

- http_get_last_response_headers() - Recupera os últimos cabeçalhos de reposta HTTP
- http_clear_last_response_headers() - Limpa os cabeçalhos de resposta HTTP armazenados

$argc

$argc

(PHP 4, PHP 5, PHP 7, PHP 8)

$argc — O número de argumentos passados para o script

Descrição

Contém o número de argumentos passados para o atual script quando executando através da linha de comando.

> **Nota** :

> O nome do arquivo do script é sempre passado como um argumento para o script, portanto o valor mínimo de *$argc* é 1.

> **Nota** :
>
> Esta variável não está disponível quando register_argc_argv está desabilitada.

Examples

Exemplo #1 Exemplo da *$argc*

```
<?php
var_dump($argc);
?>
```

Quando executando o exemplo com: php script.php arg1 arg2 arg3

O exemplo acima produzirá algo semelhante a:

```
int(4)
```

Notas

> **Nota** :
>
> Isto também está disponível como *$_SERVER['argc']*.

Veja também

- getopt() - Obtêm opções da linha de comando
- $argv

$argv

$argv

(PHP 4, PHP 5, PHP 7, PHP 8)

$argv — Array de argumentos passados para o script

Descrição

Contém um array de todos argumentos passados para o script quando executando através da linha de comando.

> **Nota** :
>
> O primeiro argumento é sempre o nome do arquivo do script atual, portanto *$argv[0]* é o nome do script.

> **Nota** :
>
> Esta variável não está disponível quando register_argc_argv está desabilitada.

> **Aviso**
>
> Para testar se um script está sendo executado a partir da linha de comando, php_sapi_name() deve ser usada em vez de verificar se *$argv* ou *$_SERVER['argv']* estão definidas.

Examples

Exemplo #1 Exemplo de *$argv*

```
<?php
var_dump($argv);
?>
```

Quando executando o exemplo com: php script.php arg1 arg2 arg3

O exemplo acima produzirá algo semelhante a:

```
array(4) {
  [0]=>
  string(10) "script.php"
  [1]=>
  string(4) "arg1"
  [2]=>
  string(4) "arg2"
  [3]=>
  string(4) "arg3"
}
```

Notas

> **Nota** :
>
> Isto também está disponível como *$_SERVER['argv']*.

Veja também

- getopt() - Obtêm opções da linha de comando
- $argc

Exceções predefinidas

Índice

Veja também as Exceções da SPL.

Exception

(PHP 5, PHP 7, PHP 8)

Introdução

Exception é a classe base para todas as exceções.

Resumo da classe

```
class Exception implements Throwable {
/* Propriedades */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos */
public function __construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function getMessage(): string
final public function getPrevious(): ?Throwable
final public function getCode(): int
final public function getFile(): string
final public function getLine(): int
final public function getTrace(): array
final public function getTraceAsString(): string
public function __toString(): string
private function __clone(): void
}
```

Propriedades

message

A mensagem da exceção

code

O código da exceção

file

O nome do arquivo onde a exceção foi criada

line

A linha onde a exceção foi criada

previous

A exceção previamente lançada, se informada

string

O rastreamento de pilha na forma string

trace

O rastreamento de pilha na forma de array

Índice

Exception::__construct

Exception::__construct

(PHP 5, PHP 7, PHP 8)

Exception::__construct — Construtor da exceção

Descrição

public function **Exception::__construct**(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)

Construtor de Exception.

Parâmetros

`message`

A mensagem da exceção.

`code`

O código da exceção.

`previous`

A exceção anterior, para o encadeamento de exceções.

> **Nota** :
>
> Chamar o construtor da classe Exception de uma classe herdada faz ignorar os argumentos padrão, caso as propriedades $code e $message já estiverem configuradas.

Notas

> **Nota** :
>
> A `message` *NÃO* é binary safe.

Exception::getMessage

Exception::getMessage

(PHP 5, PHP 7, PHP 8)

Exception::getMessage — Obtém a mensagem da exceção

Descrição

final public function **Exception::getMessage**(): string

Retorna a mensagem da exceção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a mensagem da exceção como uma string.

Examples

Exemplo #1 Exemplo da **Exception::getMessage()**

```php
<?php
try {
    throw new Exception("Alguma mensagem de erro.");
} catch(Exception $e) {
    echo $e->getMessage();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Alguma mensagem de erro.
```

Veja também

- Throwable::getMessage() - Obtém a mensagem

Exception::getPrevious

Exception::getPrevious

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Exception::getPrevious — Retorna o Throwable anterior

Descrição

final public function **Exception::getPrevious**(): ?Throwable

Retorna o Throwable anterior (o terceiro valor passado no construtor Exception::__construct()).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a Throwable anterior se disponível, ou senão `null`.

Examples

Exemplo #1 Exemplo de **Exception::getPrevious()**

Iterando e imprimindo a pilha de erro.

```
<?php
class MyCustomException extends Exception {}

function doStuff() {
    try {
        throw new InvalidArgumentException("Primeiro erro lançado.", 112);
    } catch(Exception $e) {
        throw new MyCustomException("Segundo erro lançado.", 911, $e);
    }
}


try {
    doStuff();
} catch(Exception $e) {
    do {
        printf("%s:%d %s (%d) [%s]\n", $e->getFile(), $e->getLine(), $e->getMessage(), $e-
>getCode(), get_class($e));
    } while($e = $e->getPrevious());
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
/home/bjori/ex.php:8 Segundo erro lançado. (911) [MyCustomException]
/home/bjori/ex.php:6 Primeiro erro lançado. (112) [InvalidArgumentException]
```

Veja também

- Throwable::getPrevious() - Retorna o Throwable anterior

Exception::getCode

Exception::getCode

(PHP 5, PHP 7, PHP 8)

Exception::getCode — Obtém o código da exceção

Descrição

final public function **Exception::getCode**(): int

Retorna o código da exceção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o código da Exceção como um int da Exception mas possivelmente outros tipos descendentes de Exception (PDOException, por exemplo, retorna como string).

Examples

Exemplo #1 Exemplo da **Exception::getCode()**

```php
<?php
try {
    throw new Exception("Alguma mensagem de erro.", 30);
} catch(Exception $e) {
    echo "O código da exceção é: " . $e->getCode();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
O código da exceção é: 30
```

Veja também

- Throwable::getCode() - Obtém o código da exceção

Exception::getFile

Exception::getFile

(PHP 5, PHP 7, PHP 8)

Exception::getFile — Obtém o arquivo no qual a exceção foi criada

Descrição

final public function **Exception::getFile**(): string

Obtém o nome do arquivo de onde a exceção foi criada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome do arquivo no qual a exceção foi criada.

Examples

Exemplo #1 Exemplo da Exception::getFile()

```php
<?php
try {
    throw new Exception;
} catch(Exception $e) {
    echo $e->getFile();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
/home/bjori/tmp/ex.php
```

Veja também

- Throwable::getFile() - Obtém o arquivo no qual o objeto foi criado

Exception::getLine

Exception::getLine

(PHP 5, PHP 7, PHP 8)

Exception::getLine — Obtém a linha na qual a exceção foi criada

Descrição

final public function **Exception::getLine**(): int

Retorna o número da linha onde a exceção foi disparada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número da linha onde a exceção foi disparada.

Examples

Exemplo #1 Exemplo da **Exception::getLine()**

```php
<?php
try {
    throw new Exception("Alguma mensagem de erro");
} catch(Exception $e) {
    echo "A exceção foi criada na linha: " . $e->getLine();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
A exceção foi criada na linha: 3
```

Veja também

- Throwable::getLine() - Obtém a linha na qual o objeto foi instanciado

Exception::getTrace

Exception::getTrace

(PHP 5, PHP 7, PHP 8)

Exception::getTrace — Obtém a stack trace

Descrição

final public function **Exception::getTrace**(): array

Retorna a stack trace da exceção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a stack trace da exceção como um array.

Examples

Exemplo #1 Exemplo da Exception::getTrace()

```php
<?php
function test() {
 throw new Exception;
}

try {
 test();
} catch(Exception $e) {
 var_dump($e->getTrace());
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["file"]=>
    string(22) "/home/bjori/tmp/ex.php"
    ["line"]=>
    int(7)
    ["function"]=>
    string(4) "test"
    ["args"]=>
    array(0) {
    }
  }
}
```

Veja também

- Throwable::getTrace() - Obtém o rastreamento de pilha

Exception::getTraceAsString

Exception::getTraceAsString

(PHP 5, PHP 7, PHP 8)

Exception::getTraceAsString — Obtém a stack trace como uma string

Descrição

final public function **Exception::getTraceAsString**(): string

Retorna a stack trace da exceção como uma string.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a stack trace da exceção como uma string.

Examples

Exemplo #1 Exemplo da **Exception::getTraceAsString()**

```php
<?php
function test() {
    throw new Exception;
}

try {
    test();
} catch(Exception $e) {
    echo $e->getTraceAsString();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
#0 /home/bjori/tmp/ex.php(7): test()
#1 {main}
```

Veja também

- Throwable::getTraceAsString() - Obtém o rastreamento de pilha como uma string

Exception::__toString

Exception::__toString

(PHP 5, PHP 7, PHP 8)

Exception::__toString — Representação em string da exceção

Descrição

public function **Exception::__toString**(): string

Retorna a representação string da exceção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a representação string da exceção.

Examples

Exemplo #1 Exemplo da Exception::__toString()

```php
<?php
try {
    throw new Exception("Alguma mensagem de erro.");
} catch(Exception $e) {
    echo $e;
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
exception 'Exception' with message 'Alguma mensagem de erro.' in /home/bjori/tmp/ex.php:3
Stack trace:
#0 {main}
```

Veja também

- Throwable::__toString() - Obtém uma representação em string do objeto disparado

Exception::__clone

Exception::__clone

(PHP 5, PHP 7, PHP 8)

Exception::__clone — Clona a exceção

Descrição

private function **Exception::__clone**(): void

Exceptions não podem ser clonadas, e isso resultará no lançamento de um Error.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Exceções *não* são clonáveis.

Registro de Alterações

Versão	Descrição
8.1.0	**Exception::__clone()** não é mais final.

ErrorException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Uma exceção Error.

Resumo da classe

```
class ErrorException extends Exception {
/* Propriedades */
protected int $severity = E_ERROR;
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos */
public function __construct(
    string $message = "",
    int $code = 0,
    int $severity = E_ERROR,
    ?string $filename = null,
    ?int $line = null,
    ?Throwable $previous = null
)
final public function getSeverity(): int
/* Métodos herdados */
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Propriedades

severity

O nível da exceção

Examples

Exemplo #1 Uso da função set_error_handler() para transformar mensagens de erro em ErrorException

```php
<?php

set_error_handler(function (int $errno, string $errstr, string $errfile, int $errline) {
    if (!(error_reporting() & $errno)) {
        // Este código de erro não é incluído em error_reporting.
        return;
    }

    if ($errno === E_DEPRECATED || $errno === E_USER_DEPRECATED) {
        // Não lança uma exceção para avisos de obsolescência porque avisos
        // inesperados podem quebrar a aplicação.
        return;
    }

    throw new \ErrorException($errstr, 0, $errno, $errfile, $errline);
});

// Desserializar dados errados emite um alerta que será convertido em um
// ErrorException pelo manipulador.
unserialize('broken data');

?>
```

O exemplo acima produzirá algo semelhante a:

```
Fatal error: Uncaught ErrorException: unserialize(): Error at offset 0 of 11 bytes in test.php:16
Stack trace:
#0 [internal function]: {closure}(2, 'unserialize(): ...', 'test.php', 16)
#1 test.php(16): unserialize('broken data')
#2 {main}
  thrown in test.php on line 16
```

Índice

ErrorException::__construct

ErrorException::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ErrorException::__construct — Constrói a exceção

Descrição

```
public function ErrorException::__construct(
    string $message = "",
    int $code = 0,
    int $severity = E_ERROR,
    ?string $filename = null,
    ?int $line = null,
    ?Throwable $previous = null
)
```

Construtor da Exception.

Parâmetros

`message`

A mensagem da exceção.

`code`

O código da exceção.

`severity`

A severidade da exceção.

> **Nota** :
>
> Como a severidade da seção pode ser qualquer valor int, é desejado que as constantes de erro sejam utilizadas.

`filename`

O nome do arquivo onde a exceção é disparada.

`line`

A número da linha onde a exceção é disparada.

`previous`

A exceção anterior, usada para encadeamento de exceções.

Registro de Alterações

Versão	Descrição
8.0.0	`filename` e `line` agora podem ser null. Anteriormente seus padrões eram `__FILE__` e `__LINE__`, respectivamente.

ErrorException::getSeverity

ErrorException::getSeverity

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ErrorException::getSeverity — Obtém o nível da exceção

Descrição

final public function **ErrorException::getSeverity**(): int

Retorna o nível da exceção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nível da exceção.

Examples

Exemplo #1 Exemplo do método ErrorException::getSeverity()

```php
<?php
try {
    throw new ErrorException("Exception message", 0, E_USER_ERROR);
} catch(ErrorException $e) {
    echo "A severidade desta exceção é: " . $e->getSeverity();
    var_dump($e->getSeverity() === E_USER_ERROR);
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
A severidade desta exceção é: 256
bool(true)
```

ClosedGeneratorException

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introdução

Uma exceção **ClosedGeneratorException** é lançada ao se tentar obter um valor de um Generator fechado.

Resumo da classe

class **ClosedGeneratorException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";

```
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Error

(PHP 7, PHP 8)

Introdução

Error é a classe base para todas as exceções de erros internos do PHP.

Resumo da classe

```
class Error implements Throwable {
/* Propriedades */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos */
public function __construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function getMessage(): string
final public function getPrevious(): ?Throwable
final public function getCode(): int
final public function getFile(): string
final public function getLine(): int
final public function getTrace(): array
final public function getTraceAsString(): string
public function __toString(): string
private function __clone(): void
}
```

Propriedades

message

A mensagem de erro

code

O código do erro

file

O nome do arquivo onde o erro aconteceu

line

A linha onde o erro aconteceu

previous

A exceção anteriormente lançada

string

Uma representação string do stack trace

trace

O stack trace no formato array

Índice

Error::__construct

Error::__construct

(PHP 7, PHP 8)

Error::__construct — Constrói o objeto error

Descrição

public function **Error::__construct**(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)

Constrói um erro.

Parâmetros

`message`

A mensagem de erro.

`code`

O código do erro.

`previous`

O throwable anterior, utilizado para o encadeamento de exceção.

Notas

> **Nota** :
>
> O parâmetro `message` *NÃO* é binary safe.

Error::getMessage

Error::getMessage

(PHP 7, PHP 8)

Error::getMessage — Recupera a mensagem de erro

Descrição

final public function **Error::getMessage**(): string

Recupera a mensagem de erro.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Recupera a mensagem de erro como uma string.

Examples

Exemplo #1 Exemplo do método Error::getMessage()

```php
<?php
try {
    throw new Error("Mensagem de erro");
} catch(Error $e) {
    echo $e->getMessage();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Mensagem de erro
```

Veja também

- Throwable::getMessage() - Obtém a mensagem

Error::getPrevious

Error::getPrevious

(PHP 7, PHP 8)

Error::getPrevious — Retorna o último Throwable

Descrição

final public function **Error::getPrevious**(): ?Throwable

Retorna Throwable anterior (o terceiro parâmetro do Error::__construct()).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna Throwable anterior, se disponível, ou `null`.

Examples

Exemplo #1 Exemplo do método Error::getPrevious()

Iterando e imprimindo a pilha de erro.

```
<?php
class MeuErroCustomizado extends Error {}

function fazAlgumaCoisa() {
    try {
        throw new TypeError("Você está fazendo errado!", 112);
    } catch(Error $e) {
        throw new MeuErroCustomizado("Algo aconteceu.", 911, $e);
    }
}


try {
    fazAlgumaCoisa();
} catch(Error $e) {
    do {
        printf("%s:%d %s (%d) [%s]\n", $e->getFile(), $e->getLine(), $e->getMessage(), $e-
>getCode(), get_class($e));
    } while($e = $e->getPrevious());
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
/home/bjori/ex.php:8 Algo aconteceu. (911) [MeuErroCustomizado]
/home/bjori/ex.php:6 Você está fazendo errado! (112) [TypeError]
```

Veja também

- Throwable::getPrevious() - Retorna o Throwable anterior

Error::getCode

Error::getCode

(PHP 7, PHP 8)

Error::getCode — Recupera o código do erro

Descrição

final public function **Error::getCode**(): int

Retorna o código do erro.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o código do erro como um int

Examples

Exemplo #1 Exemplo do método Error::getCode()

```php
<?php
try {
    throw new Error("Mensagem de erro", 30);
} catch(Error $e) {
    echo "O código do erro é: " . $e->getCode();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
O código do erro é: 30
```

Veja também

- Throwable::getCode() - Obtém o código da exceção

Error::getFile

Error::getFile

(PHP 7, PHP 8)

Error::getFile — Recupera o nome do arquivo onde o erro ocorreu

Descrição

final public function **Error::getFile**(): string

Recupera o nome do arquivo onde o erro ocorreu.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o nome do arquivo onde o erro ocorreu.

Examples

Exemplo #1 Exemplo do método Error::getFile()

```php
<?php
try {
    throw new Error;
} catch(Error $e) {
    echo $e->getFile();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
/home/bjori/tmp/ex.php
```

Veja também

- Throwable::getFile() - Obtém o arquivo no qual o objeto foi criado

Error::getLine

Error::getLine

(PHP 7, PHP 8)

Error::getLine — Recupera a linha onde o erro ocorreu

Descrição

final public function **Error::getLine**(): int

Recupera o número da linha onde o erro ocorreu.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número da linha onde o erro ocorreu.

Examples

Exemplo #1 Exemplo do método **Error::getLine()**

```php
<?php
try {
    throw new Error("Mensagem de erro");
} catch(Error $e) {
    echo "O erro foi criado na linha: " . $e->getLine();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
O erro foi criado na linha: 3
```

Veja também

- Throwable::getLine() - Obtém a linha na qual o objeto foi instanciado

Error::getTrace

Error::getTrace

(PHP 7, PHP 8)

Error::getTrace — Recupera a pilha de rastreio

Descrição

final public function **Error::getTrace**(): array

Recupera a pilha de rastreio.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Recupera a pilha de rastreio como um array.

Examples

Exemplo #1 Exemplo do método Error::getTrace()

```php
<?php
function test() {
 throw new Error;
}

try {
 test();
} catch(Error $e) {
 var_dump($e->getTrace());
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["file"]=>
    string(22) "/home/bjori/tmp/ex.php"
    ["line"]=>
    int(7)
    ["function"]=>
    string(4) "test"
    ["args"]=>
    array(0) {
    }
  }
}
```

Veja também

- Throwable::getTrace() - Obtém o rastreamento de pilha

Error::getTraceAsString

Error::getTraceAsString

(PHP 7, PHP 8)

Error::getTraceAsString — Recupera a pilha de rastreio como uma string

Descrição

final public function **Error::getTraceAsString**(): string

Recupera a pilha de rastreio como uma string.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Recupera a pilha de rastreio como uma string.

Examples

Exemplo #1 Exemplo do método Error::getTraceAsString()

```php
<?php
function test() {
    throw new Error;
}

try {
    test();
} catch(Error $e) {
    echo $e->getTraceAsString();
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
#0 /home/bjori/tmp/ex.php(7): test()
#1 {main}
```

Veja também

- Throwable::getTraceAsString() - Obtém o rastreamento de pilha como uma string

Error::__toString

Error::__toString

(PHP 7, PHP 8)

Error::__toString — Representação em string do erro

Descrição

public function **Error::__toString**(): string

Retorna a representação em string do erro.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a representação do erro em string.

Examples

Exemplo #1 Exemplo do método Error::__toString()

```php
<?php
try {
    throw new Error("Mensagem de erro.");
} catch(Error $e) {
    echo $e;
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Error: Mensagem de erro. in /home/bjori/tmp/ex.php:3
Stack trace:
#0 {main}
```

Veja também

- Throwable::__toString() - Obtém uma representação em string do objeto disparado

Error::__clone

Error::__clone

(PHP 7, PHP 8)

Error::__clone — Clona o erro

Descrição

private function **Error::__clone**(): void

Error não pode ser clonada, então o método resulta em um erro fatal.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Erros *não* podem ser clonados.

Registro de Alterações

Versão	Descrição
8.1.0	**Error::__clone()** não é mais final.

ArgumentCountError

(PHP 7 >= PHP 7.1.0, PHP 8)

Introdução

ArgumentCountError é disparado quando parâmetros insuficientes são passados para uma função ou método definidos pelo usuário.

Este erro também é disparado quando argumentos demais são passados para uma função interna não variável.

Resumo da classe

class **ArgumentCountError** extends TypeError {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

ArithmeticError

(PHP 7, PHP 8)

Introdução

ArithmeticError é lançado quando um erro ocorre ao executar operações matemáticas. Esses erros incluem a tentativa de executar um deslocamento de bits por uma quantidade negativa, e qualquer chamada para a função intdiv() que resultaria num valor fora dos possíveis limites para um tipo int.

Resumo da classe

```
class ArithmeticError extends Error {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

AssertionError

(PHP 7, PHP 8)

Introdução

AssertionError é lançada quando uma afirmação feita por meio da função assert() falha.

Resumo da classe

```
class AssertionError extends Error {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
```

public function Error::__toString(): string
private function Error::__clone(): void
}

DivisionByZeroError

(PHP 7, PHP 8)

Introdução

DivisionByZeroError é lançada quando uma tentativa de dividir um número por zero é feita.

Resumo da classe

class **DivisionByZeroError** extends ArithmeticError {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

CompileError

(PHP 7 > 7.3.0, PHP 8)

Introdução

CompileError é disparado para alguns erros de compilação, que anteriormente emitiam um erro fatal.

Resumo da classe

class **CompileError** extends Error {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;

```
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

ParseError

(PHP 7, PHP 8)

Introdução

ParseError é lançado quando um erro ocorre na interpretação do código PHP, como quando a função eval() é chamada.

> **Nota** :
>
> **ParseError** estende CompileError a partir do PHP 7.3.0. Anteriormente estendia de Error.

Resumo da classe

```
class ParseError extends CompileError {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
```

}

TypeError

(PHP 7, PHP 8)

Introdução

Um **TypeError** pode ser lançado quando:

- Um valor sendo informado em uma propriedade de classe que não corresponde com o tipo declarado.
- O tipo do argumento que é passado para uma função que não coincide com o tipo declarado para parâmetro.
- Um valor sendo retornado de uma função que não coincide com o tipo declarado da função.

Resumo da classe

class **TypeError** extends Error {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Registro de Alterações

Versão	Descrição
7.1.0	Em modo estrito, passar um número incorreto de argumentos para uma função interna do PHP não resulta mais em um **TypeError** genérico. Em vez disso, é lançado um ArgumentCountError mais específico, que estende **TypeError**.

ValueError

(PHP 8)

Introdução

ValueError é disparado quando o tipo de um argumento está correto mas o seu valor está incorreto. Por exemplo, ao passar um inteiro negativo quando a função espera um positivo, ou ao passar uma string ou array vazio quando a função espera que não seja vazio.

Resumo da classe

```
class ValueError extends Error {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

UnhandledMatchError

(PHP 8)

Introdução

Um **UnhandledMatchError** é lançado quando o sujeito passado para uma expressão match não é tratado por nenhum braço da expressão match.

Resumo da classe

```
class UnhandledMatchError extends Error {
/* Propriedades herdadas */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Métodos herdados */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
```

final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

FiberError

(PHP 8 >= 8.1.0)

Introdução

FiberError é disparada quando uma operação inválida é realizada em uma Fiber.

Resumo da classe

final class **FiberError** extends Error {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos */
public function __construct()
/* Métodos herdados */
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Índice

FiberError::__construct

FiberError::__construct

(PHP 8 >= 8.1.0)

FiberError::__construct — Construtor para não permitir instanciação direta

Descrição

public function **FiberError::__construct**()

Parâmetros

Esta função não possui parâmetros.

Erros/Exceções

Dispara uma exceção Error quando chamada.

RequestParseBodyException

(PHP 8 >= 8.4.0)

Introdução

Uma exceção **RequestParseBodyException** é lançada na função request_parse_body() quando o corpo da requisição é inválido, de acordo com o cabeçalho `Content-Type`.

Resumo da classe

class **RequestParseBodyException** extends Exception {
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Interfaces e Classes predefinidas

Índice

Veja também as Interfaces SPL e classes reservadas.

Traversable

(PHP 5, PHP 7, PHP 8)

Introdução

Interface para detectar se uma classe é percorrível utilizando o `foreach`.

Interface básica abstrata que não pode ser implementada sozinha. Ao invés, deve ser implementada as interfaces IteratorAggregate ou Iterator.

Resumo da Interface

interface **Traversable** {
}

Esta interface não possui métodos, seu único propósito é servir como interface base para todas as classes percorríveis.

Registro de Alterações

Versão	Descrição
8.0.0	A interface **Traversable** agora pode ser implementada em classes abstratas. Classes estendidas precisam implementar Iterator ou IteratorAggregate.

Notas

> **Nota** :
>
> Classes internas (built-in) que implementam essa interface podem ser utilizadas em um construtor de linguagem `foreach` e não precisam implementar as interfaces IteratorAggregate ou Iterator.

> **Nota** :
>
> Anteriormente ao PHP 8.0.0, esta interface interna do mecanismo não podia ser implementada diretamente em scripts PHP — nem mesmo por classes abstratas. IteratorAggregate ou Iterator precisavam ser utilizadas em seu lugar.

Iterator

(PHP 5, PHP 7, PHP 8)

Introdução

Interface para iteradores externos ou objetos poderem ser iterados por eles mesmos internamente.

Resumo da Interface

interface **Iterator** extends Traversable {
/* Métodos */
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void

public function valid(): bool
}

Iterators predefinidos

O PHP já possui diversos iteradores para muitas das tarefas do dia a dia. Veja iteradores SPL para uma lista.

Examples

Exemplo #1 Utilização básica

Este exemplo demonstra em qual ordem os métodos são chamados quando estiver usando um `foreach` em um iterador.

```php
<?php
class myIterator implements Iterator {
    private $position = 0;
    private $array = array(
        "firstelement",
        "secondelement",
        "lastelement",
    );

    public function __construct() {
        $this->position = 0;
    }

    public function rewind(): void {
        var_dump(__METHOD__);
        $this->position = 0;
    }

    #[\ReturnTypeWillChange]
    public function current() {
        var_dump(__METHOD__);
        return $this->array[$this->position];
    }

    #[\ReturnTypeWillChange]
    public function key() {
        var_dump(__METHOD__);
        return $this->position;
    }

    public function next(): void {
        var_dump(__METHOD__);
        ++$this->position;
    }

    public function valid(): bool {
        var_dump(__METHOD__);
        return isset($this->array[$this->position]);
    }
}

$it = new myIterator;
```

```php
foreach($it as $key => $value) {
    var_dump($key, $value);
    echo "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(18) "myIterator::rewind"
string(17) "myIterator::valid"
string(19) "myIterator::current"
string(15) "myIterator::key"
int(0)
string(12) "firstelement"

string(16) "myIterator::next"
string(17) "myIterator::valid"
string(19) "myIterator::current"
string(15) "myIterator::key"
int(1)
string(13) "secondelement"

string(16) "myIterator::next"
string(17) "myIterator::valid"
string(19) "myIterator::current"
string(15) "myIterator::key"
int(2)
string(11) "lastelement"

string(16) "myIterator::next"
string(17) "myIterator::valid"
```

Veja também

Veja também a iteração em objetos.

Índice

Iterator::current

Iterator::current

(PHP 5, PHP 7, PHP 8)

Iterator::current — Retorna o elemento atual

Descrição

public function **Iterator::current**(): mixed

Retorna o elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Pode retornar qualquer tipo.

Iterator::key

Iterator::key

(PHP 5, PHP 7, PHP 8)

Iterator::key — Retorna a chave do elemento atual

Descrição

public function **Iterator::key**(): mixed

Retorna a chave do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorta um scalar em caso de sucesso, ou `null` em caso de falha.

Erros/Exceções

Emite `E_NOTICE` em caso de falha.

Iterator::next

Iterator::next

(PHP 5, PHP 7, PHP 8)

Iterator::next — Move para o próximo elemento

Descrição

public function **Iterator::next**(): void

Move a posição atual para o próximo elemento.

> **Nota** :
>
> Este método é chamado *após* cada loop `foreach`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Qualquer valor retornado é ignorado.

Iterator::rewind

Iterator::rewind

(PHP 5, PHP 7, PHP 8)

Iterator::rewind — Retrocede o Iterator para o primeiro elemento

Descrição

public function **Iterator::rewind**(): void

Retrocede para o primeiro elemento do Iterator.

> **Nota** :
>
> Este é o *primeiro* método chamado quando se inicia um loop `foreach`. Ele *não será* executado *depois* de laços `foreach`.
>
> Como o `foreach` sempre chama este método antes de iniciar uma iteração, o avanço manual (por exemplo através de SplFileObject::seek()) irá reiniciar o iterador.
>
> Para iterar sem retroceder o iterador, encapsule-o em um NoRewindIterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Qualquer valor retornado é ignorado.

Iterator::valid

Iterator::valid

(PHP 5, PHP 7, PHP 8)

Iterator::valid — Verifica se a posição atual é válida

Descrição

public function **Iterator::valid**(): bool

Este método é chamado após Iterator::rewind() e Iterator::next() para verificar se a posição atual é válida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor retornado será convertido para bool e então avaliado. Retorna `true` em caso de sucesso ou `false` em caso de falha.

IteratorAggregate

(PHP 5, PHP 7, PHP 8)

Introdução

Interface para criar um iterador externo.

Resumo da Interface

interface **IteratorAggregate** extends Traversable {
/* Métodos */
public function getIterator(): Traversable
}

Examples

Exemplo #1 Uso básico

```php
<?php

class myData implements IteratorAggregate
{
    public function getIterator(): Traversable
    {
        return new ArrayIterator([
            "key one" => "item one",
            "key two" => "item two",
            "key three" => "item three"
        ]);
    }
}

$obj = new myData();

foreach ($obj as $key => $value) {
    var_dump($key, $value);
    echo "\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
string(7) "key one"
string(8) "item one"

string(7) "key two"
string(8) "item two"

string(9) "key three"
```

```
string(10) "item three"
```

Índice

IteratorAggregate::getIterator

IteratorAggregate::getIterator

(PHP 5, PHP 7, PHP 8)

IteratorAggregate::getIterator — Recupera um iterador ou Traversable externo

Descrição

public function **IteratorAggregate::getIterator**(): Traversable

Retorna um iterador ou Traversable externo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma instância de um objeto que implementa Iterator ou Traversable

Erros/Exceções

Lança uma Exception em caso de falha.

InternalIterator

(PHP 8)

Introdução

Classe para facilitar a implementação de IteratorAggregate para classes *internas*.

Resumo da classe

final class **InternalIterator** implements Iterator {
/* Métodos */
private function __construct()
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
}

Índice

InternalIterator::__construct

InternalIterator::__construct

(PHP 8)

InternalIterator::__construct — Construtor privado para não permitir instanciação direta

Descrição

private function **InternalIterator::__construct**()

Parâmetros

Esta função não possui parâmetros.

InternalIterator::current

InternalIterator::current

(PHP 8)

InternalIterator::current — Retorna o elemento atual

Descrição

public function **InternalIterator::current**(): mixed

Retorna o elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o elemento atual.

InternalIterator::key

InternalIterator::key

(PHP 8)

InternalIterator::key — Retorna a chave do elemento atual

Descrição

public function **InternalIterator::key**(): mixed

Retorna a chave do elemento atual.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a chave do elemento atual.

InternalIterator::next

InternalIterator::next

(PHP 8)

InternalIterator::next — Move para o próximo elemento

Descrição

public function **InternalIterator::next**(): void

Move a posição atual para o próximo elemento.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

InternalIterator::rewind

InternalIterator::rewind

(PHP 8)

InternalIterator::rewind — Retrocede o Iterator para o primeiro elemento

Descrição

public function **InternalIterator::rewind**(): void

Retrocede para o primeiro elemento do Iterator.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

InternalIterator::valid

InternalIterator::valid

(PHP 8)

InternalIterator::valid — Verifica se a posição atual é válida

Descrição

public function **InternalIterator::valid**(): bool

Verifica se a posição atual é válida.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna verdadeiro se a posição atual é válida, falso se inválida.

Throwable

(PHP 7, PHP 8)

Introdução

Throwable é a interface base para qualquer objeto lançável usando a declaração `throw`, incluindo as classes Error e Exception.

> **Nota** :
>
> As classes do PHP não podem implementar a interface **Throwable** diretamente, e devem, ao invés, estender a classe Exception.

Resumo da Interface

interface **Throwable** extends Stringable {
/* Métodos */
public function getMessage(): string
public function getCode(): int
public function getFile(): string
public function getLine(): int
public function getTrace(): array
public function getTraceAsString(): string
public function getPrevious(): ?Throwable
public function __toString(): string
/* Métodos herdados */
public function Stringable::__toString(): string
}

Registro de Alterações

Versão	Descrição
8.0.0	**Throwable** implementa a interface Stringable.

Índice

Throwable::getMessage

Throwable::getMessage

(PHP 7, PHP 8)

Throwable::getMessage — Obtém a mensagem

Descrição

public function **Throwable::getMessage**(): string

Retorna a mensagem associada com o objeto disparado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a mensagem associada com o objeto disparado.

Veja também

- Exception::getMessage() - Obtém a mensagem da exceção

Throwable::getCode

Throwable::getCode

(PHP 7, PHP 8)

Throwable::getCode — Obtém o código da exceção

Descrição

public function **Throwable::getCode**(): int

Retorna o código de erro associado com o objeto de exceção disparado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o código de exceção como int na classe Exception mas possivelmente como outro tipo em descendentes de Exception (por exemplo como string em PDOException).

Veja também

- Exception::getCode() - Obtém o código da exceção

Throwable::getFile

Throwable::getFile

(PHP 7, PHP 8)

Throwable::getFile — Obtém o arquivo no qual o objeto foi criado

Descrição

public function **Throwable::getFile**(): string

Obtém o nome do arquivo no qual o objeto disparado foi criado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Obtém o nome do arquivo no qual o objeto disparado foi criado.

Veja também

- Exception::getFile() - Obtém o arquivo no qual a exceção foi criada

Throwable::getLine

Throwable::getLine

(PHP 7, PHP 8)

Throwable::getLine — Obtém a linha na qual o objeto foi instanciado

Descrição

public function **Throwable::getLine**(): int

Retorna o número da linha onde o objeto disparado foi instanciado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número da linha onde o objeto disparado foi instanciado.

Veja também

- Exception::getLine() - Obtém a linha na qual a exceção foi criada

Throwable::getTrace

Throwable::getTrace

(PHP 7, PHP 8)

Throwable::getTrace — Obtém o rastreamento de pilha

Descrição

public function **Throwable::getTrace**(): array

Obtém o rastreamento de pilha como um array.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o rastreamento de pilha como um array no mesmo formato de debug_backtrace().

Veja também

- Exception::getTrace() - Obtém a stack trace

Throwable::getTraceAsString

Throwable::getTraceAsString

(PHP 7, PHP 8)

Throwable::getTraceAsString — Obtém o rastreamento de pilha como uma string

Descrição

public function **Throwable::getTraceAsString**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o rastreamento de pilha como uma string.

Veja também

- Exception::getTraceAsString() - Obtém a stack trace como uma string

Throwable::getPrevious

Throwable::getPrevious

(PHP 7, PHP 8)

Throwable::getPrevious — Retorna o Throwable anterior

Descrição

public function **Throwable::getPrevious**(): ?Throwable

Retorna qualquer Throwable anterior (por exemplo, aquele fornecido como o terceiro parâmetro para Exception::__construct()).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o Throwable anterior se disponível, ou `null` caso contrário.

Veja também

- Exception::getPrevious() - Retorna o Throwable anterior

Throwable::__toString

Throwable::__toString

(PHP 7, PHP 8)

Throwable::__toString — Obtém uma representação em string do objeto disparado

Descrição

public function **Throwable::__toString**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a representação em string do objeto disparado.

Veja também

- Exception::__toString() - Representação em string da exceção

Countable

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Classes que implementam **Countable** podem ser usadas com a função count().

Resumo da Interface

interface **Countable** {
/* Métodos */
public function count(): int
}

Índice

Countable::count

Countable::count

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Countable::count — Conta elementos de um objeto

Descrição

public function **Countable::count**(): int

Este método é executado quando o parâmetro `value` para a função count() é um objeto que implementa Countable.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número personalizado como um int.

Examples

Exemplo #1 Exemplo de Countable::count()

```php
<?php

class Counter implements Countable
{
    private $count = 0;

    public function count(): int
    {
        return ++$this->count;
    }
}

$counter = new Counter();

for ($i = 0; $i < 10; ++$i) {
    echo "Eu fui contado " . count($counter) . " vezes\n";
}

?>
```

O exemplo acima produzirá algo semelhante a:

```
Eu fui contado 1 vezes
Eu fui contado 2 vezes
Eu fui contado 3 vezes
Eu fui contado 4 vezes
Eu fui contado 5 vezes
Eu fui contado 6 vezes
Eu fui contado 7 vezes
Eu fui contado 8 vezes
Eu fui contado 9 vezes
Eu fui contado 10 vezes
```

ArrayAccess

(PHP 5, PHP 7, PHP 8)

Introdução

Interface que provê o acesso a objetos como se fossem arrays.

Resumo da Interface

interface **ArrayAccess** {
/* Métodos */
public function offsetExists(mixed `$offset`): bool
public function offsetGet(mixed `$offset`): mixed
public function offsetSet(mixed `$offset`, mixed `$value`): void
public function offsetUnset(mixed `$offset`): void
}

Examples

Exemplo #1 Utilização básica

```php
<?php
class Obj implements ArrayAccess {
    public $container = [
        "one"   => 1,
        "two"   => 2,
        "three" => 3,
    ];

    public function offsetSet($offset, $value): void {
        if (is_null($offset)) {
            $this->container[] = $value;
        } else {
            $this->container[$offset] = $value;
        }
    }

    public function offsetExists($offset): bool {
        return isset($this->container[$offset]);
    }

    public function offsetUnset($offset): void {
        unset($this->container[$offset]);
    }

    public function offsetGet($offset): mixed {
        return isset($this->container[$offset]) ? $this->container[$offset] : null;
    }
}

$obj = new Obj;

var_dump(isset($obj["two"]));
var_dump($obj["two"]);
unset($obj["two"]);
var_dump(isset($obj["two"]));
$obj["two"] = "A value";
var_dump($obj["two"]);
$obj[] = 'Append 1';
$obj[] = 'Append 2';
```

```php
$obj[] = 'Append 3';
print_r($obj);
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
int(2)
bool(false)
string(7) "A value"
Obj Object
(
    [container] => Array
        (
            [one] => 1
            [three] => 3
            [two] => A value
            [0] => Append 1
            [1] => Append 2
            [2] => Append 3
        )

)
```

Índice

ArrayAccess::offsetExists

ArrayAccess::offsetExists

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetExists — Verifica se uma posição existe

Descrição

public function **ArrayAccess::offsetExists**(mixed `$offset`): bool

Verifica se uma posição existe ou não.

Este método é executado ao utilizar-se a função isset() ou empty() em objetos que implementem ArrayAccess.

> **Nota** :
>
> Ao utilizar a função empty(), o método ArrayAccess::offsetGet() será chamado e checado por vazio somente se o método **ArrayAccess::offsetExists()** retornar `true`

Parâmetros

`offset`

Uma posição a ser checada.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

> **Nota** :
>
> O valor de retorno será convertido para bool se um não booleano for retornado.

Examples

Exemplo #1 Exemplo do método ArrayAccess::offsetExists()

```php
<?php
class obj implements ArrayAccess {
    public function offsetSet($offset, $value): void {
        var_dump(__METHOD__);
    }
    public function offsetExists($var): bool {
        var_dump(__METHOD__);
        if ($var == "foobar") {
            return true;
        }
        return false;
    }
    public function offsetUnset($var): void {
        var_dump(__METHOD__);
    }
    #[\ReturnTypeWillChange]
    public function offsetGet($var) {
        var_dump(__METHOD__);
        return "value";
    }
}

$obj = new obj;

echo "Executa obj::offsetExists()\n";
var_dump(isset($obj["foobar"]));

echo "\nExecuta obj::offsetExists() e obj::offsetGet()\n";
var_dump(empty($obj["foobar"]));

echo "\nExecuta obj::offsetExists(), *não executa* obj:offsetGet()\n";
var_dump(empty($obj["foobaz"]));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Executa obj::offsetExists()
string(17) "obj::offsetExists"
```

```
bool(true)

Executa obj::offsetExists() e obj::offsetGet()
string(17) "obj::offsetExists"
string(14) "obj::offsetGet"
bool(false)

Executa obj::offsetExists(), *não executa* obj:offsetGet()
string(17) "obj::offsetExists"
bool(true)
```

ArrayAccess::offsetGet

ArrayAccess::offsetGet

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetGet — Posição a ser obtida

Descrição

public function **ArrayAccess::offsetGet**(mixed `$offset`): mixed

Retorna o valor de uma posição específica.

Este método é executado ao checar se uma posição está vazia, com a função empty().

Parâmetros

`offset`

A posição a ser obtida.

Valor Retornado

Pode retornar valores de qualquer tipo.

Notas

> **Nota** :
>
> É possível para implementações deste método serem retornadas por referência. Isso modifica indiretamente a sobrecarga de dimensões do array de objetos ArrayAccess possíveis.
>
> Uma modificação direta é aquela que substitui completamente o valor de uma dimensão do array, como em `$obj[6] = 7`. Uma modificação indireta, por outro lado, modifica somente parte da dimensão, ou tenta atribuir a dimensão por referência a outra variável, como em `$obj[6][7] = 7` ou `$var =& $obj[6]`. Incrementos com `++` e decrementos com `--` também são implementados de forma a exigirem modificações indiretas.
>
> Enquanto uma modificação direta engatilha uma chamada ao método ArrayAccess::offsetSet(), uma modificação indireta engatilha uma chamada ao **ArrayAccess::offsetGet()**. Neste caso, a implementação

> do método **ArrayAccess::offsetGet()** deve permitir o retorno por referência, caso contrário uma mensagem `E_NOTICE` será disparada.

Veja também

- ArrayAccess::offsetExists() - Verifica se uma posição existe

ArrayAccess::offsetSet

ArrayAccess::offsetSet

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetSet — Atribui um valor a uma posição específica

Descrição

public function **ArrayAccess::offsetSet**(mixed `$offset`, mixed `$value`): void

Atribui um valor a uma posição específica.

Parâmetros

`offset`

A posição na qual se deseja atribuir um valor.

`value`

O valor a ser atribuído.

Valor Retornado

Nenhum valor é retornado.

Notas

> **Nota** :
>
> O parâmetro `offset` será definido como `null` se outro valor não for informado, como no exemplo a seguir.

```php
<?php
$arrayaccess[] = "first value";
$arrayaccess[] = "second value";
print_r($arrayaccess);
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => first value
```

```
    [1] => second value
)
```

> **Nota** :
>
> Esta função não é chamada em atribuições por referência e outras modificações indiretas as dimensões de arrays sobrecarregados com ArrayAccess (indiretas no sentido de e não serem feitas para modificar a dimensão diretamente, mas para modificar uma subdimensão ou uma subpropriedade, ou atribuir a dimensão do array por referência a outra variável). Em vez disso, o método ArrayAccess::offsetGet() será chamado. A operação será bem sucedida somente se este método retornar por referência.

ArrayAccess::offsetUnset

ArrayAccess::offsetUnset

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetUnset — Remove uma posição

Descrição

public function **ArrayAccess::offsetUnset**(mixed `$offset`): void

Destrói uma posição.

> **Nota** :
>
> Este método *não* será chamado ao se fazer conversão de tipo com (unset)

Parâmetros

`offset`

A posição a ser removida.

Valor Retornado

Nenhum valor é retornado.

Serializable

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Interface para serialização personalizada.

Classes que implementam esta interface não possuem mais suporte aos métodos __sleep() e __wakeup(). O método serialize() é chamado não importando se uma instância possui a necessidade de ser serializada. Não

invoca o método **__destruct()**, ou possui qualquer outro efeito colateral a não ser que programado no método. Quando os dados são desserializados, a classe toma conhecimento e o método unserialize() apropriado é chamado como um construtor, em vez de chamar o método **__construct()**. Se for necessário executar o construtor padrão, pode-se fazer isso no método unserialize().

> **Aviso**
>
> A partir do PHP 8.1.0, uma classe que implementa **Serializable** sem implementar __serialize() e __unserialize() gerará um aviso de descontinuação.

Resumo da Interface

interface **Serializable** {
/* Métodos */
public function serialize(): ?string
public function unserialize(string `$data`): void
}

Examples

Exemplo #1 Uso básico

```php
<?php
class obj implements Serializable {
    private $data;
    public function __construct() {
        $this->data = "My private data";
    }
    public function serialize() {
        return serialize($this->data);
    }
    public function unserialize($data) {
        $this->data = unserialize($data);
    }
    public function getData() {
        return $this->data;
    }
}

$obj = new obj;
$ser = serialize($obj);

var_dump($ser);

$newobj = unserialize($ser);

var_dump($newobj->getData());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Deprecated: obj implements the Serializable interface, which is deprecated. Implement
__serialize() and __unserialize() instead (or in addition, if support for old PHP versions is
necessary) in script on line 2
string(38) "C:3:"obj":23:{s:15:"My private data";}"
string(15) "My private data"
```

Índice

Serializable::serialize

Serializable::serialize

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Serializable::serialize — Representação em string de um objeto

Descrição

public function **Serializable::serialize**(): ?string

Deve retornar uma representação em string do objeto.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma representação em string do objeto ou **`null`**.

Erros/Exceções

Dispara uma Exception quando retorna outros tipos diferentes de string ou **`null`**.

Veja também

- __sleep()
- __serialize()

Serializable::unserialize

Serializable::unserialize

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Serializable::unserialize — Constrói o objeto

Descrição

public function **Serializable::unserialize**(string `$data`): void

Chamado durante a desserialização do objeto.

> **Nota** :

> Este método age como o construtor do objeto. O método __construct() *não será* chamado depois deste método.

Parâmetros

`data`

A representação em string do objeto.

Valor Retornado

O valor de retorno deste método é ignorado.

Veja também

- __wakeup()
- __unserialize()

Closure

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introdução

Classe usada para representar funções anônimas.

Funções anônimas, produzem objetos desse tipo. Esta classe possui métodos que permitem ainda mais controle de funções anônimas após sua criação.

Além dos métodos listados aqui, a classe também possui o método `__invoke`. Para consistência com outras classes que implementam o invoque mágico, já que esse método não é utilizado para chamar a função.

Resumo da classe

final class **Closure** {
/* Métodos */
private function __construct()
public static function bind(Closure `$closure`, ?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure
public function bindTo(?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure
public function call(object `$newThis`, mixed `...$args`): mixed
public static function fromCallable(callable `$callback`): Closure
public static function getCurrent(): Closure
}

Registro de Alterações

Versão	Descrição
8.4.0	O resultado de **Closure::__debugInfo()** agora inclui o nome, a linha e o arquivo do closure.

Notas

> **Nota** :

> Objetos **Closure** não podem ser serializados porque podem conter variáveis vinculadas e um contexto de execução específico. A tentativa de serializar lançará uma Exception.

Índice

Closure::__construct

Closure::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Closure::__construct — Construtor que desabilita a instanciação

Descrição

private function **Closure::__construct**()

Este método existe somente para desabilitar a instanciação da classe Closure. Objetos desta classe são criados como descrito na página de funções anônimas.

Parâmetros

Esta função não possui parâmetros.

Veja também

- Funções anônimas

Closure::bind

Closure::bind

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Closure::bind — Duplica uma closure com um objeto vinculado e um escopo de classe

Descrição

public static function **Closure::bind**(Closure `$closure`, ?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure

Este método é uma versão estática do Closure::bindTo(). Veja a documentação do método para mais informações.

Parâmetros

`closure`

A função anônima a ser vincular.

`newThis`

O objeto que a função anônima fornecida deve vincular, ou `null` para a closure não vincular.

`newScope`

O escopo da classe ao qual a closure deve ser informado, ou 'static' para manter o atual. Se um objeto é fornecido, o tipo do objeto será usado. Isso determina a visibilidade dos métodos protegidos e privados do objeto vinculado. Não é permitido passar uma (objeto da) classe interna como parâmetro this.

Valor Retornado

Retorna um novo objeto Closure ou `null` em caso de falha.

Examples

Exemplo #1 Exemplo do método Closure::bind()

```php
<?php
class A {
    private static $sfoo = 1;
    private $ifoo = 2;
}
$cl1 = static function() {
    return A::$sfoo;
};
$cl2 = function() {
    return $this->ifoo;
};

$bcl1 = Closure::bind($cl1, null, 'A');
$bcl2 = Closure::bind($cl2, new A(), 'A');
echo $bcl1(), "\n";
echo $bcl2(), "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
1
2
```

Veja também

- Funções anônimas
- Closure::bindTo() - Duplica a closure com um novo objeto vinculado e escopo de classe.

Closure::bindTo

Closure::bindTo

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Closure::bindTo — Duplica a closure com um novo objeto vinculado e escopo de classe.

Descrição

public function **Closure::bindTo**(?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure

Cria e retorna uma nova função anônima com o mesmo corpo e mesmas variáveis ligadas como a original, mas possivelmente com um diferente objeto ligado e um novo escopo de classe.

O "obeto ligado" determina o valor que `$this` terá no corpo da função e o "escopo de classe" representa uma classe que determina quais membros privados e protegidos a função anônima será capaz de acessar. Nomeadamente, os membros que estarão visíveis serão os mesmos que estariam, se a função anônima fosse um método da classe informada como valor do parâmetro `newScope`.

Closures estáticas não podem ter nenhum objeto ligado (o valor do parâmetro `newThis` deveria ser **`null`**), mas este método pode entretanto ser utilizada para alterar seus escopos de classe.

Este método vai assegurar que, para uma closure não estática, possuir uma instância ligada irá implicar em ter escopo definido e vice-versa. Para este fim, closures não estáticas que recebem um escopo com instância **`null`** são tornadas estáticas; e closures não estáticas sem escopo que recebem uma instância não nula têm o escopo definido para uma classe não especificada.

> **Nota** :
>
> Se for requerido duplicar as funções anônimas, pode ser utilizada clonagem alternativamente.

Parâmetros

`newThis`

O objeto ao qual a função anônima informada deve ser ligada, ou **`null`** para que a closure seja não ligada.

`newScope`

O escopo de classe ao qual a closure deve ser associada, ou 'static' para manter a atual. Se um objeto for informado, o tipo do objeto será usado no lugar. Isto determina a visibilidade de métodos protegidos e privados do objeto ligado. Não é permitido passar (um objeto de) uma classe interna neste parâmetro.

Valor Retornado

Retorna o novo objeto Closure criado ou **`null`** em caso de falha.

Examples

Exemplo #1 Exemplo de **Closure::bindTo()**

```php
<?php

class A
{
    private $val;

    public function __construct($val)
    {
```

```
        $this->val = $val;
    }

    public function getClosure()
    {
        // Retorna a closure ligada a este objeto e escopo
        return function() {
            return $this->val;
        };
    }
}

$ob1 = new A(1);
$ob2 = new A(2);

$cl = $ob1->getClosure();
echo $cl(), "\n";

$cl = $cl->bindTo($ob2);
echo $cl(), "\n";

?>
```

O exemplo acima produzirá algo semelhante a:

```
1
2
```

Veja também

- Funções Anônimas
- Closure::bind() - Duplica uma closure com um objeto vinculado e um escopo de classe

Closure::call

Closure::call

(PHP 7, PHP 8)

Closure::call — Vincula e chama a closure

Descrição

public function **Closure::call**(object `$newThis`, mixed `...$args`): mixed

Vincula temporáriamente a closure ao `newThis`, e a chama com qualquer parâmetro fornecido.

Parâmetros

`newThis`

O object a ser vinculado a closure enquanto durar a chamada.

`args`

Zero ou mais parâmetros, que serão fornecidos como parâmetros para a closure.

Valor Retornado

Retorna o valor de retorno da closure.

Examples

Exemplo #1 Exemplo do método **Closure::call()**

```
<?php
class Value {
    protected $value;

    public function __construct($value) {
        $this->value = $value;
    }

    public function getValue() {
        return $this->value;
    }
}

$three = new Value(3);
$four = new Value(4);

$closure = function ($delta) { var_dump($this->getValue() + $delta); };
$closure->call($three, 4);
$closure->call($four, 4);
?>
```

O exemplo acima produzirá:

```
int(7)
int(8)
```

Closure::fromCallable

Closure::fromCallable

(PHP 7 >= 7.1.0)

Closure::fromCallable — Converte um chamável em uma closure

Descrição

public static function **Closure::fromCallable**(callable `$callback`): Closure

Cria e retorna uma nova função anônima a partir do `callback` fornecido usando o escopo atual. Esse método verifica se o `callback` é chamável no escopo atual e lança um TypeError se ele não for.

> **Nota** :
>
> A partir do PHP 8.1.0, Sintaxe de chamáveis de primeira classe possui a mesma semântica que este método.

Parâmetros

`callback`

O chamável a ser convertido.

Valor Retornado

Retorna a recém-criada Closure ou lança um TypeError se o `callback` não é chamável no escopo atual.

Veja também

- Funções anônimas
- Sintaxe de chamável de primeira classe

Closure::getCurrent

Closure::getCurrent

(PHP 8 >= 8.5.0)

Closure::getCurrent — Retorna o closure em execução no momento

Descrição

public static function **Closure::getCurrent**(): Closure

Retorna o closure em execução no momento. Este método é útil principalmente para implementar closures recursivos sem a necessidade de capturar uma referência à variável do closure com a palavra-chave `use`.

Este método precisa ser chamado de dentro de um closure; chamá-lo de fora de um contexto de closure resultará em um erro `Error: Current function is not a closure.`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a instância do Closure em execução.

Erros/Exceções

Lança um Error se chamado de fora de um contexto de closure.

Examples

Exemplo #1 Exemplo de Closure::getCurrent()

Usando **Closure::getCurrent()** para implementar uma função de Fibonacci recursiva:

```php
<?php
$fibonacci = function (int $n) {
    if ($n === 0 || $n === 1) {
        return $n;
    }

    $fn = Closure::getCurrent();
    return $fn($n - 1) + $fn($n - 2);
};

echo $fibonacci(10); // Exibe: 55
?>
```

Exemplo #2 Comparação com método tradicional

Antes do PHP 8.5, para implementar closures recursivos era necessário capturar uma referência à variável do closure usando a palavra-chave `use`:

```php
<?php
// Método tradicional (ainda funciona no PHP 8.5)
$fibonacci = function (int $n) use (&$fibonacci) {
    if ($n === 0) return 0;
    if ($n === 1) return 1;
    return $fibonacci($n - 1) + $fibonacci($n - 2);
};

echo $fibonacci(10); // Exibe: 55
?>
```

O método **Closure::getCurrent()** elimita a necessidade de declarar a variável com uma referência na cláusula `use`, tornando o código mais limpo e menos suscetível a erros.

stdClass

(PHP 4, PHP 5, PHP 7, PHP 8)

Introdução

Uma classe vazia genérica com propriedades dinâmicas.

Objetos desta classe podem ser instaciados com o operador new ou criados por conversores de tipo para o objeto. Muitas funções do PHP também criam instâncias desta classe, ex.: json_decode(), mysqli_fetch_object() ou PDOStatement::fetchObject().

Apesar de não implementar os métodos mágicos __get()/__set(), esta classe permite propriedades dinâmicas e não requer o atributo `#[\AllowDynamicProperties]`.

Esta não é uma classe base já que o PHP não possui um conceito de uma classe base universal. Entretanto, é possível criar uma classe customizada que se estende a partir de **stdClass** e como resultado herda a funcionalidade de propriedades dinâmicas.

Resumo da classe

#[\AllowDynamicProperties]
class **stdClass** {
}

Esta classe não tem métodos ou propriedades padrão.

Examples

Exemplo #1 Criado como resultado de uma conversão de tipo em um objeto

```php
<?php
$obj = (object) array('foo' => 'bar');
var_dump($obj);
```

O exemplo acima produzirá:

```
object(stdClass)#1 (1) {
  ["foo"]=>
  string(3) "bar"
}
```

Exemplo #2 Criado como resultado da função json_decode()

```php
<?php
$json = '{"foo":"bar"}';
var_dump(json_decode($json));
```

O exemplo acima produzirá:

```
object(stdClass)#1 (1) {
  ["foo"]=>
  string(3) "bar"
}
```

Exemplo #3 Declarando propriedades dinâmicas

```php
<?php
$obj = new stdClass();
$obj->foo = 42;
$obj->{1} = 42;
var_dump($obj);
```

O exemplo acima produzirá:

```
object(stdClass)#1 (2) {
  ["foo"]=>
  int(42)
  ["1"]=>
  int(42)
}
```

Generator

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introdução

Os objetos **Generator** são retornados por geradores.

> **Cuidado**
>
> Os objetos **Generator** não podem ser instanciados através do construtor new.

Resumo da classe

final class **Generator** implements Iterator {
/* Métodos */
public function current(): mixed
public function getReturn(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function send(mixed `$value`): mixed
public function throw(Throwable `$exception`): mixed
public function valid(): bool
public function __wakeup(): void
}

Veja também

See also object iteration.

Índice

Generator::current

Generator::current

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::current — Obtém o valor gerado

Descrição

public function **Generator::current**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o valor gerado.

Generator::getReturn

Generator::getReturn

(PHP 7, PHP 8)

Generator::getReturn — Obtém o valor de retorno de um gerador

Descrição

public function **Generator::getReturn**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o valor de retorno de um gerador, assim que finalizada sua execução.

Examples

Exemplo #1 Exemplo do método Generator::getReturn()

```php
<?php

$gen = (function() {
    yield 1;
    yield 2;

    return 3;
})();

foreach ($gen as $val) {
    echo $val, PHP_EOL;
}

echo $gen->getReturn(), PHP_EOL;
```

O exemplo acima produzirá:

```
1
2
3
```

Generator::key

Generator::key

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::key — Obtém o chave gerada

Descrição

public function **Generator::key**(): mixed

Obtém a chave do valor gerado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a chave gerada.

Examples

Exemplo #1 Exemplo do método Generator::key()

```php
<?php

function Gen()
{
    yield 'key' => 'value';
}

$gen = Gen();

echo "{$gen->key()} => {$gen->current()}";
```

O exemplo acima produzirá:

```
key => value
```

Generator::next

Generator::next

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::next — Continua a execução do gerador

Descrição

public function **Generator::next**(): void

Chamar **Generator::next()** tem o mesmo efeito que chamar Generator::send() com o argumento `null`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Generator::rewind

Generator::rewind

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::rewind — Executa o gerador até o primeiro "yield" inclusive

Descrição

public function **Generator::rewind**(): void

Executa o gerador até o *primeiro* `yield`, inclusive. Se o gerador já estiver no *primeiro* `yield`, nenhuma ação será tomada. Se o gerador já tiver avançado além de uma expressão `yield`, este método lançará uma Exception.

> **Nota** :
>
> Este é o *primeiro* método chamado ao iniciar um laço `foreach`. Ele *não* será executado *após* os laços `foreach`.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de Generator::rewind()

```
<?php

function generator(): Generator
{
    echo "Eu sou um gerador!\n";
```

```php
    for ($i = 1; $i <= 3; $i++) {
        yield $i;
    }
}

// Inicializa o gerador
$generator = generator();

// Retrocede o gerador para o início da primeira expressão yield,
// se já não estava lá
$generator->rewind(); // Eu sou um gerador!

// Nada acontece aqui. O gerador já está retrocedido.
$generator->rewind(); // Sem saída (NULL)

// Isto retrocede o gerador para a primeira expressão yield,
// se já não estava lá, e itera sobre o gerador.
foreach ($generator as $value) {
    // Após gerar o primeiro valor, o gerador permanece na
    // primeira expressão yield até que continue a execução e avance ao próximo yield
    echo $value, PHP_EOL; // 1

    break;
}

// Continua e retrocede novamente. Nenhum erro ocorre porque o gerador não avançou além do primeiro
yield
$generator->rewind();

echo $generator->current(), PHP_EOL; // 1

// Nenhum erro ocorre, o gerador ainda está no primeiro yield
$generator->rewind();

// Isto avança o gerador para a segunda expressão yield
$generator->next();

try {
    // Aqui uma Exception é lançada
    // porque o gerador já avançou para o segundo yield
    $generator->rewind(); // Fatal error: Uncaught Exception: Cannot rewind a generator that was
already run
} catch (Exception $e) {
    echo $e->getMessage();
}

?>
```

O exemplo acima produzirá:

```
Eu sou um gerador!
1
1
Cannot rewind a generator that was already run
```

Generator::send

Generator::send

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::send — Envia um valor ao gerador

Descrição

public function **Generator::send**(mixed `$value`): mixed

Envia o valor informado ao gerador como resultado da expressão `yield` corrente e retorna a execução do gerador.

Se o gerador não estiver na expressão `yield` quando o método for chamado, primeiro será deixado avançar para a primeira expressão `yield` antes de enviar o valor. Assim como não é necessário "forçar" os geradores do PHP com uma chamada ao método Generator::next() (como é feito em Python).

Parâmetros

`value`

> Valor a ser enviado ao gerador. Este valor será o retornado pela expressão `yield` em que o gerador estiver.

Valor Retornado

Retorna o valor gerado.

Examples

Exemplo #1 Usando o método Generator::send() para injetar valores

```php
<?php
function printer() {
    echo "Valor inicial.".PHP_EOL;
    while (true) {
        $string = yield;
        echo $string.PHP_EOL;
    }
}

$printer = printer();
$printer->send('Envio 1.');
$printer->send('Envio 2.');
?>
```

O exemplo acima produzirá:

```
Valor inicial.
Envio 1.
Envio 2.
```

Generator::throw

Generator::throw

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::throw — Lança uma exceção no gerador

Descrição

public function **Generator::throw**(Throwable `$exception`): mixed

Lança uma exceção no gerador e retorna a execução do gerador. O comportamento será o mesmo como se a expressão `yield` corrente fosse substituída por uma declaração `throw $exception`.

Se o gerador já tiver sido fechado ao invocar este método, a exceção será lançada no contexto daquele que o chamou.

Parâmetros

`exception`

Exceção a ser lançada no gerador.

Valor Retornado

Retorna o valor gerado.

Examples

Exemplo #1 Lança uma exceção no gerador

```php
<?php
function gen() {
    echo "Foo\n";
    try {
        yield;
    } catch (Exception $e) {
        echo "Exception: {$e->getMessage()}\n";
    }
    echo "Bar\n";
}

$gen = gen();
$gen->rewind();
$gen->throw(new Exception('Test'));
?>
```

O exemplo acima produzirá:

```
Foo
Exception: Test
Bar
```

Generator::valid

Generator::valid

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::valid — Verifica se o gerador está ativo

Descrição

public function **Generator::valid**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `false` se o gerador foi fechado. Caso contrário retornará `true`.

Generator::__wakeup

Generator::__wakeup

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::__wakeup — Serializa o gerador (sempre falha)

Descrição

public function **Generator::__wakeup**(): void

Lança uma exceção, já que geradores não podem ser serializados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nenhum valor é retornado.

Fiber

(PHP 8 >= 8.1.0)

Introdução

Fibers representam funções interruptíveis de pilha completa. Fibers podem ser suspensas de qualquer lugar na pilha de chamadas, pausando a execução dentro de Fiber até que ela seja retomada posteriormente.

Resumo da classe

final class **Fiber** {
/* Métodos */
public function __construct(callable `$callback`)
public function start(mixed `...$args`): mixed
public function resume(mixed `$value = null`): mixed
public function throw(Throwable `$exception`): mixed
public function getReturn(): mixed
public function isStarted(): bool
public function isSuspended(): bool
public function isRunning(): bool
public function isTerminated(): bool
public static function suspend(mixed `$value = null`): mixed
public static function getCurrent(): ?Fiber
}

Veja também

Visão geral de Fibers

Índice

Fiber::__construct

Fiber::__construct

(PHP 8 >= 8.1.0)

Fiber::__construct — Cria uma nova instância de Fiber

Descrição

public function **Fiber::__construct**(callable `$callback`)

Parâmetros

`callback`

> O callable a ser invocado ao iniciar a Fiber. Argumentos informados a Fiber::start() serão fornecidos como argumentos ao callable informado.

Fiber::start

Fiber::start

(PHP 8 >= 8.1.0)

Fiber::start — Inicia execução de uma Fibra

Descrição

public function **Fiber::start**(mixed `...$args`): mixed

Uma lista variável de argumentos a ser fornecida para o callable usado na construção da Fiber.

Se a Fiber já tiver sido iniciada quando este método for chamado, uma exceção FiberError será disparada.

Parâmetros

`args`

Os argumentos que serão utilizados na chamada ao callable fornecido ao construtor da Fiber.

Valor Retornado

O valor fornecido à primeira chamada a Fiber::suspend() ou **`null`** se a Fiber retornar. Se a Fiber disparar uma exceção antes de ser suspensa, ela será disparada a partir da chamada a este método.

Fiber::resume

Fiber::resume

(PHP 8 >= 8.1.0)

Fiber::resume — Retoma a execução de uma Fiber com um valor

Descrição

public function **Fiber::resume**(mixed `$value = null`): mixed

Retoma a execução da Fiber usando o valor fornecido como resultado da chamada Fiber::suspend() atual.

Se a Fiber não estiver suspensa quando este método for chamado, uma exceção FiberError será disparada.

Parâmetros

`value`

O valor para retomar a Fiber. Este valor será o valor de retorno da chamada atual de Fiber::suspend().

Valor Retornado

O valor fornecido para a próxima chamada de Fiber::suspend() ou **`null`** se a Fiber retornar. Se a Fiber disparar uma exceção antes de ser suspensa, ela será disparada a partir da chamada a este método.

Fiber::throw

Fiber::throw

(PHP 8 >= 8.1.0)

Fiber::throw — Retoma a execução de uma Fiber com uma exceção

Descrição

public function **Fiber::throw**(Throwable `$exception`): mixed

Retoma a Fiber disparando a exceção informada a partir da chamada Fiber::suspend() atual.

Se a Fiber não estiver suspensa quando este método é chamado, uma exceção FiberError será disparada.

Parâmetros

`exception`

A exceção a ser disparada na Fiber a partir da chamada Fiber::suspend() atual.

Valor Retornado

O valor fornecido à próxima chamada a Fiber::suspend() ou **`null`** se a Fiber retornar. Se a Fiber disparar uma exceção antes de ser suspensa, ela será disparada a partir da chamada a este método.

Examples

```php
<?php

$fiber = new Fiber(function () {
   try {
       // Suspende execução do fiber declarando um ponto de interrupção
       Fiber::suspend();
   } catch (Throwable $e) {
       echo $e->getMessage();
   }
});

$fiber->start();

// Continua a execução do fiber passando
// a exceção a ser lançada no ponto de interrupção
$fiber->throw(new Exception('Mensagem de uma exceção lançada no ponto de interrupção atual'));

?>
```

O exemplo acima produzirá algo semelhante a:

```
Mensagem de uma exceção lançada no ponto de interrupção atual
```

Fiber::getReturn

Fiber::getReturn

(PHP 8 >= 8.1.0)

Fiber::getReturn — Obtém o valor retornado pela Fiber

Descrição

public function **Fiber::getReturn**(): mixed

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o valor retornado pela callable fornecida a Fiber::__construct(). Se a Fiber não retornar um valor, por não ter iniciado ou terminado, ou por ter disparado uma exceção, uma FiberError será disparada.

Fiber::isStarted

Fiber::isStarted

(PHP 8 >= 8.1.0)

Fiber::isStarted — Determina se uma Fiber foi iniciada

Descrição

public function **Fiber::isStarted**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` apenas se a Fiber foi iniciada; caso contrário retorna `false`.

Fiber::isSuspended

Fiber::isSuspended

(PHP 8 >= 8.1.0)

Fiber::isSuspended — Determina se uma Fiber está suspensa

Descrição

public function **Fiber::isSuspended**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se uma Fiber está atualmente suspensa; caso contrário retorna `false`.

Fiber::isRunning

Fiber::isRunning

(PHP 8 >= 8.1.0)

Fiber::isRunning — Determina se a Fiber está em execução

Descrição

public function **Fiber::isRunning**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` apenas se a Fiber estiver em execução. Um Fiber é considerado em execução após uma chamada a Fiber::start(), Fiber::resume() ou Fiber::throw() que ainda não retornaram. Retorna `false` se a Fiber não está em execução.

Fiber::isTerminated

Fiber::isTerminated

(PHP 8 >= 8.1.0)

Fiber::isTerminated — Determina se a Fiber terminou

Descrição

public function **Fiber::isTerminated**(): bool

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` apenas após a Fiber ter terminado, ou retornando da execução ou disparando uma exceção; caso contrário retorna `false`.

Fiber::suspend

Fiber::suspend

(PHP 8 >= 8.1.0)

Fiber::suspend — Suspende a execução da Fiber atual

Descrição

public static function **Fiber::suspend**(mixed `$value = null`): mixed

Suspende a execução da Fiber atual. O valor fornecido a este método será retornado da chamada a Fiber::start(), Fiber::resume() ou Fiber::throw() que mudou a execução para a Fiber atual.

Quando a Fiber é retomada, este método retorna o valor fornecido a Fiber::resume(). Se a Fiber é retomada usando Fiber::throw(), a exceção fornecida àquele método será disparada a partir da chamada a este método.

Se este método for chamado de fora de uma Fiber, uma exceção FiberError será disparada.

Parâmetros

`value`

> O valor a retornar da chamada a Fiber::start(), Fiber::resume() ou Fiber::throw() que mudou a execução para a Fiber atual.

Valor Retornado

O valor fornecido a Fiber::resume().

Fiber::getCurrent

Fiber::getCurrent

(PHP 8 >= 8.1.0)

Fiber::getCurrent — Obtém a instância de Fiber sendo executada no momento

Descrição

public static function **Fiber::getCurrent**(): ?Fiber

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a instância de Fiber sendo executada no momento ou `null` se este método é chamado do lado de fora de uma Fiber.

WeakReference

(PHP 7 >= 7.4.0, PHP 8)

Introdução

Referências fracas permitem que o programador mantenha uma referência a um objeto que não impede que o objeto seja destruído. Eles são úteis para implementar estruturas como cache. Se o objeto original tiver sido destruído, `null` será retornado ao chamar o método WeakReference::get(). O objeto original será destruído quando o refcount dele cair para zero; criar referências fracas não aumenta o `refcount` do objeto que está sendo referenciado.

WeakReferences não podem ser serializadas.

Resumo da classe

final class **WeakReference** {
/* Métodos */
public function __construct()
public static function create(object `$object`): WeakReference
public function get(): ?object
}

Exemplos de referência fraca

Exemplo #1 Uso básico de referência fraca

```php
<?php

$obj = new stdClass();
$weakref = WeakReference::create($obj);

var_dump($weakref->get());
```

```php
unset($obj);

var_dump($weakref->get());

?>
```

O exemplo acima produzirá algo semelhante a:

```
object(stdClass)#1 (0) {
}
NULL
```

Registro de Alterações

Versão	Descrição
8.4.0	O resultado de **WeakReference::__debugInfo()** agora inclui o objeto referenciado ou `null` se a referência não for mais válida.

Índice

WeakReference::__construct

WeakReference::__construct

(PHP 7 >= 7.4.0, PHP 8)

WeakReference::__construct — Construtor que desabilita a instanciação

Descrição

public function **WeakReference::__construct**()

Este método existe apenas para desabilitar a instanciação da classe WeakReference. Referências fracas devem ser instanciadas com o método de fábrica WeakReference::create().

Parâmetros

Esta função não possui parâmetros.

Veja também

- WeakReference::create() - Cria uma nova referência fraca

WeakReference::create

WeakReference::create

(PHP 7 >= 7.4.0, PHP 8)

WeakReference::create — Cria uma nova referência fraca

Descrição

public static function **WeakReference::create**(object `$object`): WeakReference

Cria uma nova WeakReference.

Parâmetros

`object`

O objeto a ser referenciado fracamente.

Valor Retornado

Retorna um novo objeto WeakReference, ou a instância existente se já houver uma WeakReference para o mesmo objeto.

WeakReference::get

WeakReference::get

(PHP 7 >= 7.4.0, PHP 8)

WeakReference::get — Obtém um objeto referenciado fracamente

Descrição

public function **WeakReference::get**(): ?object

Obtém um objeto referenciado fracamente. Se o objeto já tiver sido destruído, retorna **`null`**.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o object referenciado ou **`null`** se o objeto tiver sido destruído.

WeakMap

(PHP 8)

Introdução

Um **WeakMap** é um mapa (ou dicionário) que aceita objetos como chaves. Entretanto, diferentemente do objeto similar SplObjectStorage, um objeto em uma chave de um **WeakMap** não contribui com a contagem de referência do objeto. Isto é, se em algum ponto a única referência restante a um objeto é a chave de um **WeakMap**, o objeto sofrerá uma coleta de lixo e será removido do **WeakMap**. Seu caso de uso primário é construir caches de dados derivados de um objeto que não precisam durar mais que o objeto.

WeakMap implementa ArrayAccess, Traversable (via IteratorAggregate) e Countable, por isso na maioria dos casos pode ser usada da mesma forma que um array associativo.

Resumo da classe

final class **WeakMap** implements ArrayAccess, Countable, IteratorAggregate {

/* Métodos */
public function count(): int
public function getIterator(): Iterator
public function offsetExists(object $object): bool
public function offsetGet(object $object): mixed
public function offsetSet(object $object, mixed $value): void
public function offsetUnset(object $object): void
}

Examples

Exemplo #1 Exemplo de uso de **WeakMap**

```
<?php
$wm = new WeakMap();

$o = new stdClass;

class A {
    public function __destruct() {
        echo "Dead!\n";
    }
}

$wm[$o] = new A;

var_dump(count($wm));
echo "Unsetting...\n";
unset($o);
echo "Done\n";
var_dump(count($wm));
```

O exemplo acima produzirá:

```
int(1)
Unsetting...
Dead!
Done
int(0)
```

Índice

WeakMap::count

WeakMap::count

(PHP 8)

WeakMap::count — Conta o número de entradas ativas no mapa

Descrição

public function **WeakMap::count**(): int

Conta o número de entradas ativas no mapa.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de entradas ativas no mapa.

WeakMap::getIterator

WeakMap::getIterator

(PHP 8)

WeakMap::getIterator — Obtém um iterador externo

Descrição

public function **WeakMap::getIterator**(): Iterator

Retorna um iterador externo.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma instância de um objeto que implementa a classe Iterator ou Traversable

Erros/Exceções

Dispara uma Exception em caso de falha.

WeakMap::offsetExists

WeakMap::offsetExists

(PHP 8)

WeakMap::offsetExists — Verifica se um certo objeto está no mapa

Descrição

public function **WeakMap::offsetExists**(object `$object`): bool

Verifica se o objeto informado está referenciado no mapa.

Parâmetros

`object`

Objeto a ser verificado.

Valor Retornado

Retorna `true` se o objeto estiver contido no mapa, `false` caso contrário.

WeakMap::offsetGet

WeakMap::offsetGet

(PHP 8)

WeakMap::offsetGet — Retorna o valor apontado por um certo objeto

Descrição

public function **WeakMap::offsetGet**(object `$object`): mixed

Retorna o valor apontado por um certo objeto.

Parâmetros

`object`

Algum objeto contido como chave em um mapa.

Valor Retornado

Retorna o valor associado ao objeto passado como argumento.

Erros/Exceções

Lança uma exceção da classe Error em caso de falha.

WeakMap::offsetSet

WeakMap::offsetSet

(PHP 8)

WeakMap::offsetSet — Atualiza o mapa com um novo par chave-valor

Descrição

public function **WeakMap::offsetSet**(object `$object`, mixed `$value`): void

Atualiza o mapa com um novo par chave-valor. Se a chave já existia no mapa, o valor antigo é substituído pelo novo.

Parâmetros

`object`

O objeto que serve como chave no par de chave-valor.

`value`

O dado arbitrário que serve como valor no par chave-valor..

Valor Retornado

Nenhum valor é retornado.

WeakMap::offsetUnset

WeakMap::offsetUnset

(PHP 8)

WeakMap::offsetUnset — Remove uma entrada do mapa

Descrição

public function **WeakMap::offsetUnset**(object `$object`): void

Remove uma entrada do mapa.

Parâmetros

`object`

O objeto chave a ser removido do mapa.

Valor Retornado

Nenhum valor é retornado.

Stringable

(PHP 8)

Introdução

A interface **Stringable** denota uma classe que tenha um método __toString(). Diferentemente da maioria das interfaces, **Stringable** está implicitamente presente em qualquer classe que possua o método mágico __toString() definido, embora ela possa e deveria ser declarada explicitamente.

Seu principal valor é permitir que funções façam verificações de tipo em uniões `string|Stringable` para aceitar tanto uma string primitiva quanto um objeto que possa ser convertido para uma string.

Resumo da Interface

interface **Stringable** {
/* Métodos */
public function __toString(): string
}

Exemplos de Stringable

Exemplo #1 Uso Básico de Stringable

Este exemplo usa a promoção de propriedade do construtor.

```
<?php
class IPv4Address implements Stringable {
```

```
    public function __construct(
        private string $oct1,
        private string $oct2,
        private string $oct3,
        private string $oct4,
    ) {}

    public function __toString(): string {
        return "$this->oct1.$this->oct2.$this->oct3.$this->oct4";
    }
}

function showStuff(string|Stringable $value) {
    // Para um Stringable, isto irá chamar __toString() implicitamente.
    print $value;
}

$ip = new IPv4Address('123', '234', '42', '9');

showStuff($ip);
?>
```

O exemplo acima produzirá algo semelhante a:

```
123.234.42.9
```

Índice

Stringable::__toString

Stringable::__toString

(PHP 8)

Stringable::__toString — Obtém a representação em string do objeto

Descrição

public function **Stringable::__toString**(): string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a representação em string do objeto.

Veja também

- __toString()

UnitEnum

(PHP 8 >= 8.1.0)

Introdução

A interface **UnitEnum** é aplicada automaticamente a todas as enumerações pelo motor do PHP. Não pode ser implementada por classes definidas pelo usuário. Enumerações não podem substituir seus métodos, já que implementações padrão são fornecidas pelo motor. Está disponível apenas para verificações de tipo.

Resumo da Interface

interface **UnitEnum** {
/* Métodos */
public static function cases(): array
}

Índice

UnitEnum::cases

UnitEnum::cases

(PHP 8 >= 8.1.0)

UnitEnum::cases — Gera uma lista de casos de uma enumeração

Descrição

public static function **UnitEnum::cases**(): array

Este método irá retornar um array de todos os casos de uma enumeração, em ordem de declaração.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de todos os casos definidos desta enumeração, em ordem de declaração.

Examples

Exemplo #1 Uso básico

O exemplo a seguir ilustra como os casos de enumeração são retornados.

```php
<?php
enum Naipe
{
    case Copas;
    case Ouros;
    case Paus;
    case Espadas;
}
```

```
var_dump(Naipe::cases());
```

O exemplo acima produzirá:

```
array(4) {
  [0]=>
  enum(Naipe::Copas)
  [1]=>
  enum(Naipe::Ouros)
  [2]=>
  enum(Naipe::Paus)
  [3]=>
  enum(Naipe::Espadas)
}
```

BackedEnum

(PHP 8 >= 8.1.0)

Introdução

A interface **BackedEnum** é aplicada automaticamente pelo motor do PHP a enumerações lastreadas. Não pode ser implementada por classes definidas pelo usuário. Enumerações não podem substituir seus métodos, já que as implementações padrões são fornecidas pelo motor. Está disponível apenas para verificações de tipo.

Resumo da Interface

interface **BackedEnum** extends UnitEnum {
/* Métodos */
public static function from(int|string `$value`): static
public static function tryFrom(int|string `$value`): ?static
/* Métodos herdados */
public static function UnitEnum::cases(): array
}

Índice

BackedEnum::from

BackedEnum::from

(PHP 8 >= 8.1.0)

BackedEnum::from — Mapeia um escalar para uma instância de enumeração

Descrição

public static function **BackedEnum::from**(int|string `$value`): static

O método **from()** traduz uma string ou um int para o caso correspondente do Enum, se houver. Se não houver caso correspondente definido, ele irá emitir um ValueError.

Parâmetros

`value`

O valor escalar a ser mapeado para um caso de enumeração.

Valor Retornado

Uma instância de caso para esta enumeração.

Examples

Exemplo #1 Uso Básico

O exemplo a seguir ilustra como os casos de enumeração são retornados.

```php
<?php
enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

$h = Suit::from('H');

var_dump($h);

$b = Suit::from('B');
?>
```

O exemplo acima produzirá:

```
enum(Suit::Hearts)

Fatal error: Uncaught ValueError: "B" is not a valid backing value for enum "Suit" in
/file.php:15
```

Veja também

- UnitEnum::cases() - Gera uma lista de casos de uma enumeração
- BackedEnum::tryFrom() - Mapeia um escalar para uma instância de enumeração ou nulo

BackedEnum::tryFrom

BackedEnum::tryFrom

(PHP 8 >= 8.1.0)

BackedEnum::tryFrom — Mapeia um escalar para uma instância de enumeração ou nulo

Descrição

public static function **BackedEnum::tryFrom**(int|string `$value`): ?static

O método **tryFrom()** traduz uma string ou um int para o caso correspondente do Enum, se houver. Se não houver caso correspondente definido, ele irá retornar nulo.

Parâmetros

`value`

O valor escalar a ser mapeado para um caso de enumeração.

Valor Retornado

Uma instância de caso para esta enumeração, ou null se não encontrado.

Examples

Exemplo #1 Uso básico

O exemplo a seguir ilustra como os casos de enumeração são retornados.

```php
<?php
enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

$h = Suit::tryFrom('H');

var_dump($h);

$b = Suit::tryFrom('B') ?? Suit::Spades;

var_dump($b);
?>
```

O exemplo acima produzirá:

```
enum(Suit::Hearts)
enum(Suit::Spades)
```

Veja também

- UnitEnum::cases() - Gera uma lista de casos de uma enumeração
- BackedEnum::from() - Mapeia um escalar para uma instância de enumeração

SensitiveParameterValue

(PHP 8 >= 8.2.0)

Introdução

A classe **SensitiveParameterValue** permite empacotar valores sensíveis para protegê-los de exposição acidental.

Valores de parâmetros que possuem o atributo SensitiveParameter serão automaticamente empacotados dentro de um objeto **SensitiveParameterValue** entre os rastreamentos de pilha.

Resumo da classe

final class **SensitiveParameterValue** {
/* Propriedades */
private readonly mixed *$value*;
/* Métodos */
public function __construct(mixed `$value`)
public function __debugInfo(): array
public function getValue(): mixed
}

Propriedades

value

O valor sensível a ser protegido de exposição acidental.

Índice

SensitiveParameterValue::__construct

SensitiveParameterValue::__construct

(PHP 8 >= 8.2.0)

SensitiveParameterValue::__construct — Constrói um novo objeto SensitiveParameterValue

Descrição

public function **SensitiveParameterValue::__construct**(mixed `$value`)

Constrói um objeto SensitiveParameterValue que contém um valor sensível.

Parâmetros

`value`

Um valor arbitrário que deve ser armazenado no objeto SensitiveParameterValue.

SensitiveParameterValue::__debugInfo

SensitiveParameterValue::__debugInfo

(PHP 8 >= 8.2.0)

SensitiveParameterValue::__debugInfo — Protege os valores sensíveis de exposição acidental

Descrição

public function **SensitiveParameterValue::__debugInfo**(): array

Retorna um array vazio para proteger o valor sensível de exposição acidental quando var_dump() for usado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array vazio.

Examples

Exemplo #1 Passando um objeto SensitiveParameterValue para var_dump()

```php
<?php
$s = new \SensitiveParameterValue('secret');

var_dump($s);
?>
```

O exemplo acima produzirá:

```
object(SensitiveParameterValue)#1 (0) {
}
```

SensitiveParameterValue::getValue

SensitiveParameterValue::getValue

(PHP 8 >= 8.2.0)

SensitiveParameterValue::getValue — Retorna o valor sensível

Descrição

public function **SensitiveParameterValue::getValue**(): mixed

Obtém o valor sensível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O valor sensível.

Examples

Exemplo #1 Exemplo de **SensitiveParameterValue::getValue()**

```php
<?php
$s = new \SensitiveParameterValue('secret');

echo "O valor protegido é: ", $s->getValue(), "\n";
?>
```

O exemplo acima produzirá:

```
O valor protegido é: secret
```

__PHP_Incomplete_Class

(PHP 4 >=4.0.1, PHP 5, PHP 7, PHP 8)

Introdução

Criada por unserialize() ao tentar desserializar uma classe indefinida ou uma classe não listada nas classes permitidas na chave `allowed_classes` do array do parâmetro `options` da função unserialize().

Antes do PHP 7.2.0, usar is_object() na classe **__PHP_Incomplete_Class** retornaria `false`. A partir do PHP 7.2.0, será retornado `true`.

Resumo da classe

#[\AllowDynamicProperties]
final class **__PHP_Incomplete_Class** {
}

Esta classe não tem propriedades ou métodos padrões. Quando criada por unserialize(), além de todas as propriedades e valores desserializados, o objeto terá uma propriedade `__PHP_Incomplete_Class_Name` que conterá o nome da classe desserializada.

Registro de Alterações

Versão	Descrição
8.0.0	Esta classe agora é `final`.

Examples

Exemplo #1 Criada por unserialize()

```php
<?php

class MyClass
{
    public string $property = "myValue";
}

$myObject = new MyClass;

$foo = serialize($myObject);

// desserializa todos os objetos em objetos __PHP_Incomplete_Class
$disallowed = unserialize($foo, ["allowed_classes" => false]);

var_dump($disallowed);

// desserializa todos os objetos em objetos __PHP_Incomplete_Class exceto os de MyClass2 e MyClass3
$disallowed2 = unserialize($foo, ["allowed_classes" => ["MyClass2", "MyClass3"]]);

var_dump($disallowed2);

// desserializa uma classe indefinida em objeto __PHP_Incomplete_Class
$undefinedClass = unserialize('O:16:"MyUndefinedClass":0:{}');

var_dump($undefinedClass);
```

O exemplo acima produzirá:

```
object(__PHP_Incomplete_Class)#2 (2) {
  ["__PHP_Incomplete_Class_Name"]=>
  string(7) "MyClass"
  ["property"]=>
  string(7) "myValue"
}
object(__PHP_Incomplete_Class)#3 (2) {
  ["__PHP_Incomplete_Class_Name"]=>
  string(7) "MyClass"
  ["property"]=>
  string(7) "myValue"
}
object(__PHP_Incomplete_Class)#4 (1) {
  ["__PHP_Incomplete_Class_Name"]=>
  string(16) "MyUndefinedClass"
}
```

Atributos Predefinidos

Índice

O PHP fornece alguns atributos predefinidos que podem ser usados.

Attribute

(PHP 8)

Introdução

Atributos oferecem a capacidade de adicionar informações estruturadas de metadados e legíveis por máquina em declarações no código: classes, métodos, funções, parâmetros, propriedades e constantes de classes podem ser alvos de um atributo. Os metadados definidos pelos atributos podem ser inspecionados em tempo de execução usando as APIs de Reflexão. Dessa forma, os atributos podem ser pensados como uma linguagem de configuração incorporada diretamente no código.

Resumo da classe

```
#[\Attribute]
final class Attribute {
/* Constantes */
const int TARGET_CLASS;
const int TARGET_FUNCTION;
const int TARGET_METHOD;
const int TARGET_PROPERTY;
const int TARGET_CLASS_CONSTANT;
const int TARGET_PARAMETER;
const int TARGET_CONSTANT;
const int TARGET_ALL;
const int IS_REPEATABLE;
/* Propriedades */
public int $flags;
/* Métodos */
public function __construct(int $flags = Attribute::TARGET_ALL)
}
```

Constantes predefinidas

Attribute::TARGET_CLASS

Attribute::TARGET_FUNCTION

Attribute::TARGET_METHOD

Attribute::TARGET_PROPERTY

Attribute::TARGET_CLASS_CONSTANT

Attribute::TARGET_PARAMETER

Attribute::TARGET_CONSTANT

Attribute::TARGET_ALL

Attribute::IS_REPEATABLE

Propriedades

flags

Registro de Alterações

Versão	Descrição
8.5.0	Adicionada a constante `Attribute::TARGET_CONSTANT`.

Veja também

Visão geral dos atributos

Índice

Attribute::__construct

Attribute::__construct

(PHP 8)

Attribute::__construct — Constrói uma nova instância da classe Attribute

Descrição

public function **Attribute::__construct**(int `$flags = Attribute::TARGET_ALL`)

Constrói uma nova instância da classe Attribute.

Parâmetros

`flags`

AllowDynamicProperties

(PHP 8 >= 8.2.0)

Introdução

Este atributo é usado para marcar classes que permitem propriedades dinâmicas.

> **Nota** :
>
> Embora os atributos em si não sejam herdados, o efeito do atributo `AllowDynamicProperties` *é* herdado. Classes filhas de uma classe marcada com este atributo também permitirão propriedades dinâmicas, mesmo se elas não declararem explicitamente o atributo.

Resumo da classe

#[\Attribute]
final class **AllowDynamicProperties** {
/* Métodos */

public function __construct()
}

Examples

As propriedades dinâmicas foram descontinuadas a partir do PHP 8.2.0, portanto, usá-las sem marcar a classe com este atributo emitirá um aviso de descontinuação.

Exemplo #1 AllowDynamicProperties com propriedade não existente

```php
<?php
class DefaultBehaviour { }

#[\AllowDynamicProperties]
class ClassAllowsDynamicProperties { }

$o1 = new DefaultBehaviour();
$o2 = new ClassAllowsDynamicProperties();

$o1->nonExistingProp = true;
$o2->nonExistingProp = true;
?>
```

Saída do exemplo acima no PHP 8.2:

```
Deprecated: Creation of dynamic property DefaultBehaviour::$nonExistingProp is deprecated in file
on line 10
```

Exemplo #2 AllowDynamicProperties com propriedade não existente em classe herdada

```php
<?php
class DefaultBehaviour { }

#[\AllowDynamicProperties]
class ClassAllowsDynamicProperties { }

class InheritedClassAllowsDynamicProperties extends ClassAllowsDynamicProperties { }

$o1 = new DefaultBehaviour();
$o2 = new InheritedClassAllowsDynamicProperties();

$o1->nonExistingProp = true;
$o2->nonExistingProp = true;
?>
```

Saída do exemplo acima no PHP 8.2:

```
Deprecated: Creation of dynamic property DefaultBehaviour::$nonExistingProp is deprecated in file
on line 12
```

Veja também

Visão geral dos atributos

Índice

AllowDynamicProperties::__construct

AllowDynamicProperties::__construct

(PHP 8 >= 8.2.0)

AllowDynamicProperties::__construct — Constrói uma nova instância do atributo AllowDynamicProperties

Descrição

public function **AllowDynamicProperties::__construct**()

Constrói uma nova instância de AllowDynamicProperties.

Parâmetros

Esta função não possui parâmetros.

DelayedTargetValidation

(PHP 8 >= 8.5.0)

Introdução

Este atributo atrasa os erros de validação de destino para atributos internos do momento da compilação até o momento em que o atributo é instanciado por meio da API de Reflexão.

Quando aplicado a uma declaração, qualquer uso inválido de atributos internos no mesmo destino não acionará um erro de tempo de compilação. Em vez disso, a validação é adiada e realizada quando o atributo é instanciado por meio de ReflectionAttribute::newInstance().

Isso visa principalmente a compatibilidade futura, permitindo que o código use atributos que podem obter destinos válidos adicionais em versões futuras do PHP sem quebrar em versões mais antigas.

Resumo da classe

```
#[\Attribute]
final class DelayedTargetValidation {
}
```

Examples

Exemplo #1 Atrasar a validação de um destino inválido

```php
<?php

class Base {
    protected function foo(): void {}
}

class Child extends Base {

    #[\DelayedTargetValidation]
    #[\Override]
    public const NAME = 'child';

    #[\Override]
    protected function foo(): void {}
}
```

Em versões do PHP onde Override não é permitido em constantes de classe, isso não produz um erro de tempo de compilação.

Exemplo #2 A validação ocorre durante a reflexão

```php
<?php

$reflection = new ReflectionClassConstant(Child::class, 'NAME');

foreach ($reflection->getAttributes() as $attribute) {
    $attribute->newInstance(); // Pode lançar exceção se for inválido
}
```

Quando qualquer atributo aplicado ao mesmo destino (exceto o próprio DelayedTargetValidation) é instanciado via reflexão usando ReflectionAttribute::newInstance(), a validação do destino é realizada e uma exceção pode ser lançada se o atributo for usado em um destino não suportado.

Notas

Este atributo afeta apenas a validação de destino dos atributos internos.

Não suprime a validação funcional realizada por esses atributos. Por exemplo, Override ainda emitirá um erro se um método não sobrescrever de fato um método pai.

Veja também

- Visão geral de atributos
- Override

Deprecated

(PHP 8 >= 8.4.0)

Introdução

Este atributo é usado para marcar uma funcionalidade como descontinuada. Usar uma funcionalidade descontinuada causará a emissão de um erro `E_USER_DEPRECATED`.

Resumo da classe

```
#[\Attribute]
final class Deprecated {
/* Propriedades */
public readonly ?string $message;
public readonly ?string $since;
/* Métodos */
public function __construct(?string $message = null, ?string $since = null)
}
```

Propriedades

message

> Uma mensagem opcional explicando o motivo da descontinuação e a possível funcionalidade de substituição. Será incluída na mensagem de descontinuação emitida.

since

> Uma string opcional que indica desde quando a funcionalidade foi descontinuada. Os conteúdos não são validados pelo PHP e podem conter um número de versão, uma data ou qualquer outro valor que seja considerado apropriado. Será incluída na mensagem de descontinuação emitida.
>
> A funcionalidade que fizer parte do PHP usará a versão principal.secundária como o valor de *since*, por exemplo `'8.4'`

Examples

```php
<?php

#[\Deprecated(message: "use safe_replacement() instead", since: "1.5")]
function unsafe_function()
{
   echo "Isto não é seguro", PHP_EOL;
}

unsafe_function();

?>
```

A saída do exemplo acima no PHP 8.4 é semelhante a:

```
Deprecated: Function unsafe_function() is deprecated since 1.5, use safe_replacement() instead in
example.php on line 9
Isto não é seguro
```

Veja também

- Visão geral de atributos
- ReflectionFunctionAbstract::isDeprecated()
- ReflectionClassConstant::isDeprecated()

- `E_USER_DEPRECATED`

Índice

Deprecated::__construct

Deprecated::__construct

(PHP 8 >= 8.4.0)

Deprecated::__construct — Constrói uma nova instância do atributo Deprecated

Descrição

public function **Deprecated::__construct**(?string `$message` = `null`, ?string `$since` = `null`)

Constrói uma nova instância de Deprecated.

Parâmetros

`message`

O valor da propriedade message.

`since`

O valor da propriedade since.

NoDiscard

(PHP 8 >= 8.5.0)

Introdução

Este atributo pode ser usado para indicar que o valor de retorno de uma função ou método não deve ser descartado. Se o valor de retorno não for usado de nenhuma forma, um aviso será emitido.

Isso é útil para funções onde não verificar o valor de retorno provavelmente é um bug.

Para descartar intencionalmente o valor de retorno de tal função, use a conversão (void) para suprimir o aviso.

> **Nota** :
>
> Como os atributos são projetados para serem retrocompatíveis, `#[\NoDiscard]` pode ser adicionado a funções e métodos mesmo quando o PHP 8.4 ou versões anteriores são suportados, ele simplesmente não fará nada. No PHP 8.5 e versões posteriores, um aviso será emitido se o resultado não for usado. Para suprimir o aviso sem usar `(void)`, que não é suportado antes do PHP 8.5, considere usar uma variável como `$_`.

> **Nota** :
>
> `#[\NoDiscard]` aplica-se à função ou método específico na declaração em que é escrito, e o aviso é emitido com base na declaração que é realmente chamada. Como resultado, adicionar `#[\NoDiscard]` a

> um método de interface ou a um método abstrato não emite um aviso, porque o método invocado é o método que implementa ou substitui. Da mesma forma, um método que substitui um método `#[\NoDiscard]` não emite o aviso, a menos que ele próprio esteja marcado com o atributo. Em contraste, um método importado de uma trait mantém o atributo, porque o método da trait é copiado para a classe que o utiliza como se tivesse sido declarado lá.

Resumo da classe

#[\Attribute]
final class **NoDiscard** {
/* Propriedades */
public readonly ?string *$message*;
/* Métodos */
public function __construct(?string `$message` = null)
}

Propriedades

message

Uma mensagem opcional explicando por que o valor retornado não deve ser descartado.

Examples

Exemplo #1 Uso básico

```php
<?php

/**
 * Processa todos os itens fornecidos e retorna um array com os resultados da
 * operação para cada item. `null` indica sucesso e uma exceção indica
 * um erro. As chaves do array de resultados correspondem às chaves do array $items.
 *
 * @param array<string> $items
 * @return array<null|Exception>
 */
#[\NoDiscard("pois o processamento pode falhar para itens individuais")]
function bulk_process(array $items): array {
    $results = [];

    foreach ($items as $key => $item) {
        if (\random_int(0, 9999) < 9999) {
   // Finge fazer algo útil com $item,
   // o que terá sucesso em 99,99% dos casos.
            echo "Processando {$item}", PHP_EOL;
            $error = null;
        } else {
            $error = new \Exception("Falha ao processar {$item}.");
        }

        $results[$key] = $error;
    }

    return $results;
}
```

```
bulk_process($items);

?>
```

A saída do exemplo acima no PHP 8.5 é semelhante a:

```
Warning: The return value of function bulk_process() should either be used or intentionally
ignored by casting it as (void), pois o processamento pode falhar para itens individuais
```

Exemplo #2 Descartar intencionalmente o valor de retorno

```
<?php

#[\NoDiscard]
function some_command(): int {
    return 1;
}

// Suprimir o aviso usando (void) - PHP 8.5+
(void) some_command();

// Para compatibilidade com versões do PHP anteriores à 8.5, utilize uma variável temporária
$_ = some_command();

?>
```

Veja também

- Visão geral dos atributos

Índice

NoDiscard::__construct

NoDiscard::__construct

(PHP 8 >= 8.5.0)

NoDiscard::__construct — Constrói uma nova instância do atributo NoDiscard.

Descrição

public function **NoDiscard::__construct**(?string `$message` = `null`)

Constrói uma nova instância de NoDiscard.

Parâmetros

`message`

O valor da propriedade message.

Override

(PHP 8 >= 8.3.0)

Introdução

Este atributo é usado para indicar que um método ou uma propriedade se destina a substituir um método ou uma propriedade de uma classe superior ou que ele/ela implementa um método ou propriedade definido(a) em uma interface.

Se nenhum método ou propriedade com o mesmo nome existir em uma classe superior ou em uma interface implementada, um erro será emitido no momento da compilação.

O atributo não pode ser usado no método __construct(), que é isento de verificação de assinaturas.

Resumo da classe

#[\Attribute]
final class **Override** {
/* Métodos */
public function __construct()
}

Registro de Alterações

Versão	Descrição
8.5.0	**Override** pode ser aplicada a propriedades.

Examples

Exemplo #1 Uso com métodos

```php
<?php

class Base {
    protected function foo(): void {}
}

final class Extended extends Base {
    #[\Override]
    protected function boo(): void {}
}

?>
```

A saída do exemplo acima no PHP 8.3 é semelhante a:

```
Fatal error: Extended::boo() has #[\Override] attribute, but no matching parent method exists
```

Exemplo #2 Uso com propriedades

```php
<?php

class Base {
    protected string $foo;
}

final class Extended extends Base {
    #[\Override]
    protected string $boo;
}

?>
```

A saída do exemplo acima no PHP 8.5 é semelhante a:

```
Fatal error: Extended::$boo has #[\Override] attribute, but no matching parent property exists
```

Veja também

- Visão geral de atributos

Índice

Override::__construct

Override::__construct

(PHP 8 >= 8.3.0)

Override::__construct — Constrói uma nova instância de atributo Override

Descrição

public function **Override::__construct**()

Constrói uma nova instância de Override.

Parâmetros

Esta função não possui parâmetros.

ReturnTypeWillChange

(PHP 8 >= 8.1.0)

Introdução

A partir do PHP 8.1.0, foi iniciada uma fase provisória de transição para declarações de tipo de retorno para métodos de classe internos.

A maioria dos métodos internos não finais agora requerem que os métodos substitutos declarem um tipo de retorno compatível. Caso contrário, um aviso de descontinuação é emitido durante a validação da herança, alertando que a assinatura viola as regras de covariância.

Em uma futura versão do PHP, a verificação da assinatura do método se tornará rigorosa, e incompatibilidades causarão um erro fatal.

Caso o tipo de retorno não possa ser declarado para um método substituto devido a preocupação com compatibilidade entre versões do PHP, ou se o método substituto declara um tipo de retorno incompatível, um atributo `#[\ReturnTypeWillChange]` pode ser adicionado para silenciar o aviso de descontinuação.

> **Aviso**
>
> Ao substituir métodos definidos em classes definidas pelo usuário, os tipos de retorno são verificados rigorosamente. Um erro fatal é emitido em caso de incompatibilidade, mesmo que o método substituto esteja marcado com o atributo.
>
> O atributo **ReturnTypeWillChange** suprime os avisos de descontinuação *somente* durante a fase de tipo de retorno provisório. Assim que a verificação de tipo estrita for aplicada, o atributo deixará de funcionar e uma incompatibilidade na assinatura do método substituto de uma classe interna causará um erro fatal.

Resumo da classe

```
#[\Attribute]
final class ReturnTypeWillChange {
/* Métodos */
public function __construct()
}
```

Veja também

Visão geral dos atributos

Índice

ReturnTypeWillChange::__construct

ReturnTypeWillChange::__construct

(PHP 8 >= 8.1.0)

ReturnTypeWillChange::__construct — Constrói uma nova instância do atributo ReturnTypeWillChange

Descrição

public function **ReturnTypeWillChange::__construct**()

Constrói uma nova instância de ReturnTypeWillChange.

Parâmetros

Esta função não possui parâmetros.

SensitiveParameter

(PHP 8 >= 8.2.0)

Introdução

Esse atributo é usado para marcar um parâmetro que é confidencial e deve ter seu valor ocultado se estiver presente em um stack trace.

Resumo da classe

#[\Attribute]
final class **SensitiveParameter** {
/* Métodos */
public function __construct()
}

Examples

```php
<?php

function defaultBehavior(
    string $secret,
    string $normal
) {
    throw new Exception('Error!');
}

function sensitiveParametersWithAttribute(
    #[\SensitiveParameter]
    string $secret,
    string $normal
) {
    throw new Exception('Error!');
}

try {
    defaultBehavior('password', 'normal');
} catch (Exception $e) {
    echo $e, PHP_EOL, PHP_EOL;
}

try {
    sensitiveParametersWithAttribute('password', 'normal');
} catch (Exception $e) {
    echo $e, PHP_EOL, PHP_EOL;
}

?>
```

A saída do exemplo acima no PHP 8.2 é semelhante a:

```
Exception: Error! in example.php:7
Stack trace:
#0 example.php(19): defaultBehavior('password', 'normal')
#1 {main}
```

```
Exception: Error! in example.php:15
Stack trace:
#0 example.php(25): sensitiveParametersWithAttribute(Object(SensitiveParameterValue), 'normal')
#1 {main}
```

Veja também

- Visão geral dos atributos
- SensitiveParameterValue

Índice

SensitiveParameter::__construct

SensitiveParameter::__construct

(PHP 8 >= 8.2.0)

SensitiveParameter::__construct — Constrói uma nova instância do atributo SensitiveParameter

Descrição

public function **SensitiveParameter::__construct**()

Constrói uma nova instância de SensitiveParameter.

Parâmetros

Esta função não possui parâmetros.

Opções e parâmetros de contexto

O PHP oferece várias opções e parâmetros de contexto que podem ser usados com todos os sistemas de arquivos e empacotadores de fluxo. O contexto é criado com a função stream_context_create(). As opções são definidas com a função stream_context_set_option() e os parâmetros com a função stream_context_set_params().

Índice

Opções de contexto de Socket

Opções de contexto de Socket

Opções de contexto de Socket — Lista de opções de contexto de Socket

Descrição

As opções de contextos para Socket estão disponíveis em todos os empacotadores que trabalham em sockets, como `tcp`, `http` e `ftp`.

Opções

`bindto`

Utilizado para especificar o endereço IP (tanto IPv4 quanto IPv6) e/ou o número da porta que o PHP usará para acessar a rede. A sintaxe é `ip:port` para endereços IPv4, e `[ip]:port` para endereços IPv6. Definir o IP ou a porta com `0` fará com que o sistema escolha o IP e/ou a porta.

> **Nota** :
>
> Como o FTP cria duas conexões socket durante sua operação normal, o número da porta não poderá ser especificado ao se usar esta opção.

`backlog`

Utilizado para limitar o número de conexões excedentes na fila de escuta do socket.

> **Nota** :
>
> Este parâmetro só é aplicável na função stream_socket_server().

`ipv6_v6only`

Sobrescreve a diretiva do sistema operacional no que se refere ao mapeamento IPv4 em IPv6.

> **Nota** :
>
> Isto é importante quando tentando ouvir em um endereço IPv4 separadamente da existência de um binding em `[::]`.
>
> Somente aplicável em stream_socket_server().

`so_reuseport`

Permite várias ligações no mesmo IP:porta, mesmo de processos separados.

> **Nota** :
>
> Somente aplicável em stream_socket_server().

`so_broadcast`

Permite enviar e receber dados de/para endereços de transmissão ampla.

> **Nota** :
>
> Somente aplicável a stream_socket_server().

`tcp_nodelay`

Ao ativar essa opção com `true` será configurado `SOL_TCP,NO_DELAY=1`, de forma a desabilitar o algoritmo Nagle TCP.

Registro de Alterações

Versão	Descrição
7.1.0	Adicionado o parâmetro `tcp_nodelay`.
7.0.1	Adicionado o parâmetro `ipv6_v6only`.

Examples

Exemplo #1 Exemplo básico de uso do parâmetro `bindto`

```php
<?php
// conecta à internet usando o IP '192.168.0.100'
$opts = array(
    'socket' => array(
        'bindto' => '192.168.0.100:0',
    ),
);


// conecta à internet usando o IP '192.168.0.100' e a porta '7000'
$opts = array(
    'socket' => array(
        'bindto' => '192.168.0.100:7000',
    ),
);


// conecta à internet usando o endereço IPv6 '2001:db8::1'
// e a porta '7000'
$opts = array(
    'socket' => array(
        'bindto' => '[2001:db8::1]:7000',
    ),
);


// conecta à internet usando a porta '7000'
$opts = array(
    'socket' => array(
        'bindto' => '0:7000',
    ),
);


// cria o contexto...
$context = stream_context_create($opts);

// ...que será utilizado para buscar os dados
echo file_get_contents('http://www.example.com', false, $context);

?>
```

Opções de contexto do HTTP

Opções de contexto do HTTP

Opções de contexto do HTTP — Lista de opções de contexto do HTTP

Descrição

Opções de contexto dos transportadores `http://` e `https://`.

Opções

`method` string

> **`GET`**, **`POST`** ou qualquer outro método HTTP suportado pelo servidor remoto.
>
> **`GET`** é o padrão.

`header` array ou string

> Cabeçalhos adicionais que devem ser enviados durante a requisição. Valores nesta opção sobrescrevem outros valores (como `User-agent:`, `Host:`, e `Authentication:`), mesmo ao seguir redirecionamentos `Location:`. Portanto, não é recomendado definir um cabeçalho `Host:`, se `follow_location` estiver habilitado.
>
> Os valores em string devem ser pares de `Chave: valor` delimitados por `\r\n`, por exemplo,. `"Content-Type: application/json\r\nConnection: close"`. Os valores em array devem ser uma lista de pares de `Chave: valor`, por exemplo, `["Content-Type: application/json", "Connection: close"]`.

`user_agent` string

> Valor a ser enviado no cabeçalho `User-Agent:`. Este valor só é utilizado se o user-agent *não* for especificado na opção de contexto `header` acima.
>
> Por padrão, a configuração user_agent do *php.ini* é utilizada.

`content` string

> Dados adicionais a serem enviados depois dos cabeçalhos. Tipicamente utilizado em requisições POST ou PUT.

`proxy` string

> URI que especifica o endereço do servidor de proxy (ex.: `tcp://proxy.example.com:5100`).

`request_fulluri` bool

> Quando definido como **`true`**, todo o URI será utilizado na construção da requisição (ex.: `GET http://www.example.com/path/to/file.html HTTP/1.0`). Mesmo não sendo um formato padronizado de requisição, alguns servidores de proxy o exigem.
>
> **`false`** é o padrão.

`follow_location` int

> Segue os redirecionamentos feitos com o cabeçalho `Location`. Defina com `0` para desabilitar.

`1` é o padrão.

`max_redirects` int

O número máximo de redirecionamentos a serem feitos. O valor `1` ou inferior significam que os redirecionamentos não serão feitos.

`20` é o padrão.

`protocol_version` float

Versão do protocolo HTTP.

O padrão é `1.1` a partir do PHP 8.0.0; antes dessa versão o padrão era `1.0`.

`timeout` float

Limite de tempo de leitura em segundos, especificado com um float (por exemplo, `10.5`).

Por padrão, a configuração default_socket_timeout do *php.ini* é utilizada.

`ignore_errors` bool

Retorna o conteúdo mesmo que códigos de status de falha sejam retornados.

`false`.

Examples

Exemplo #1 Buscando uma página e enviando dados pelo método POST

```php
<?php

$postdata = http_build_query(
    [
        'var1' => 'some content',
        'var2' => 'doh'
    ]
);

$opts = [
    'http' => [
        'method'  => 'POST',
        'header'  => 'Content-type: application/x-www-form-urlencoded',
        'content' => $postdata
    ]
];

$context = stream_context_create($opts);

$result = file_get_contents('http://example.com/submit.php', false, $context);

?>
```

Exemplo #2 Ignorando redirecionamentos mas trazendo os cabeçalhos e conteúdo

```php
<?php

$url = "http://www.example.org/header.php";

$opts = [
    'http' => [
        'method' => 'GET',
        'max_redirects' => '0',
        'ignore_errors' => '1'
    ]
];

$context = stream_context_create($opts);
$stream = fopen($url, 'r', false, $context);

// informação de cabeçalho e metadados
// sobre o fluxo
var_dump(stream_get_meta_data($stream));

// dados reais no $url
var_dump(stream_get_contents($stream));
fclose($stream);
?>
```

Notas

> **Nota** : **Opções de contextos de soquetes de fluxo básicos**
>
> Opções de contextos adicionais podem ser suportadas em transportes básicos Em fluxos `http://`, use as opções de contexto do transporte `tcp://`. Em fluxos `https://`, use as opções de contexto do transporte `ssl://`.

> **Nota** : **Linha de status HTTP**
>
> Quando este empacotador de fluxo faz um redirecionamento, o `wrapper_data` retornado pela função stream_get_meta_data() pode não conter necessariamente a linha de status HTTP, que atualmente se aplica ao conteúdo no índice `0`.

```
array (
  'wrapper_data' =>
  array (
    0 => 'HTTP/1.0 301 Moved Permanently',
    1 => 'Cache-Control: no-cache',
    2 => 'Connection: close',
    3 => 'Location: http://example.com/foo.jpg',
    4 => 'HTTP/1.1 200 OK',
    ...
```

> A primeira requisição retornou `301` (redirecionamento permanente), com isso o empacotador de fluxo fará o redirecionamento automaticamente para conseguir uma resposta `200` (índice = `4`).

Veja também

- http://
- Opções de contexto de Socket
- Opções de contexto de SSL

Opções de contexto do FTP

Opções de contexto do FTP

Opções de contexto do FTP — Lista de opções de contexto do FTP

Descrição

Opções de contexto para os transportadores `ftp://` e `ftps://`.

Opções

`overwrite` bool

> Permite a sobrescrita de arquivos existentes no servidor remoto. Aplicável somente no modo de escrita (uploading).
>
> `false` como padrão.

`resume_pos` int

> Deslocamento do arquivo a se começar a transferência. Aplicável somente no modo de leitura (downloading).
>
> `0` (Início do arquivo), como padrão.

`proxy` string

> Requisição Proxy FTP pelo servidor proxy http. Aplicável em arquivos somente em operações de leitura. Ex: `tcp://squid.example.com:8000`.

Notas

> **Nota** : **Opções de contexto de fluxos de socket subjacentes**
>
> Opções de contextos adicionais podem ser suportadas pelo transporte subjacente. Para fluxos `ftp://`, refira-se às opções para o transporte `tcp://`. Para `ftps://`, refira-se às opções de contexto do transporte `ssl://`.

Veja também

- ftp://
- Opções de contexto de Socket

- Opções de contexto de SSL

Opções de contexto de SSL

Opções de contexto de SSL

Opções de contexto de SSL — Lista de opções de contexto de SSL

Descrição

Opções de contexto dos transportadores `ssl://` e `tls://`.

Opções

`peer_name` string

> Nome do peer a ser utilizado. Se este valor não for definido, o nome será adivinhado com base no nome do servidor utilizado na abertura do fluxo.

`verify_peer` bool

> Exige a verificação do certificado SSL utilizado.
>
> **`true`** como padrão.

`verify_peer_name` bool

> Exige a verificação do nome do peer.
>
> **`true`** como padrão.

`allow_self_signed` bool

> Permite certificados auto assinados. Precisa do parâmetro `verify_peer`.
>
> **`false`** como padrão.

`cafile` string

> Localização do arquivo de Autoridade Certificadora no sistema de arquivos que deve ser usado pela opção de contexto `verify_peer` para autenticar a identidade do peer remoto.

`capath` string

> Se a opção `cafile` não for especificada, ou o certificado não for encontrado, o diretório apontado pela opção `capath` será utilizado na busca de um certificado adequado. A opção `capath` deve ser um diretório de certificado com hash correto.

`local_cert` string

> Caminho para o certificado local no sistema de arquivos. Deve ser um arquivo PEM codificado que contém seu certificado e chave privada. Pode opcionalmente conter a cadeia de certificados dos emitentes. A chave privada também pode estar em um arquivo separado especificado pela opção `local_pk`.

`local_pk` string

Caminho para a chave privada local no sistema de arquivos, em caso de se usar arquivos separados para o certificado (`local_cert`) e para a chave privada.

`passphrase` string

Senha usada para codificar o arquivo `local_cert`

`verify_depth` int

Abortará se a cadeia de certificados for muito profunda.

Não verificar por padrão.

`ciphers` string

Define a lista de cifras disponíveis. O formato desta string é descrito em » ciphers(1).

`DEFAULT` como padrão.

`capture_peer_cert` bool

Se definida como **`true`**, a opção de contexto `peer_certificate` será criada contendo o certificado peer.

`capture_peer_cert_chain` bool

Se definida como **`true`**, a opção de contexto `peer_certificate_chain` será criada contendo a cadeia de certificados.

`SNI_enabled` bool

Se definida como **`true`**, o indicador do nome do servidor será habilitado. Habilitar a SNI permite múltiplos certificados no mesmo endereço IP.

`disable_compression` bool

Se definida, desabilita a compressão TLS. Pode ajudar a mitigar o vetor de ataque CRIME.

`peer_fingerprint` string | array

Abortará quando o certificado resumido remoto não corresponder a um hash especificado.

Quando uma string for utilizada, o tamanho determinará qual algoritmo de hash será aplicado, "md5" (32) ou "sha1" (40).

Quando um array for utilizado, as chaves indicarão o nome do hash e os valores correspondentes ao resumo esperado.

`security_level` int

Define o nível de segurança. Se não especificado, o padrão da biblioteca é usado. Os níveis de segurança são descritos em » SSL_CTX_get_security_level(3).

Disponível a partir do PHP 7.2.0 e OpenSSL 1.1.0.

Registro de Alterações

Versão	Descrição
7.2.0	Adicionado o parâmetro `security_level`. Requer OpenSSL >= 1.1.0.

Notas

> **Nota** :
>
> Por `ssl://` ser um transporte básico para os empacotadores `https://` e `ftps://`, quaisquer opções de contexto aplicáveis a `ssl://`, também se aplicam a `https://` e `ftps://`.

> **Nota** :
>
> Para SNI (Server Name Indication - Indicador de Nome de Servidor) estar disponível, o PHP deve ser compilado com o OpenSSL 0.9.8j ou superior. Utilize a constante **`OPENSSL_TLSEXT_SERVER_NAME`** para determinar o suporte ao SNI.

Veja também

- Opções de contexto de Socket

Opções de contexto do Phar

Opções de contexto do Phar

Opções de contexto do Phar — Lista de opções de contexto do Phar

Descrição

Opções de contexto do empacotador `phar://`.

Opções

`compress` int

Uma das constantes de compressão do Phar.

`metadata` mixed

Metadados do Phar. Consulte o método Phar::setMetadata().

Veja também

- phar://
- Empacotador de fluxo do Phar

Parâmetros de contexto

Parâmetros de contexto

Parâmetros de contexto — Listagem de parâmetros de contexto

Descrição

Estes `parâmetros` podem ser definidos em um `contexto` usando a função stream_context_set_params().

Parâmetros

`notification` callable

Um callable pode ser chamado quando um evento ocorre em um fluxo.

Veja stream_notification_callback para mais detalhes.

Opções de contexto para Zip

Opções de contexto para Zip

Opções de contexto para Zip — Lista de opções de contexto de Zip

Descrição

Opções de contexto para Zip estão disponíveis para empacotadores `zip`.

Opções

`password`

Utilizado para especificar a senha definida para o arquivo criptografado.

Registro de Alterações

Versão	Descrição
7.2.0, PECL zip 1.14.0	Incluído o parâmetro `password`.

Examples

Exemplo #1 Exemplo básico de uso de `password`

```php
<?php
// Ler o arquivo criptografado
$opts = array(
    'zip' => array(
        'password' => 'secret',
    ),
);
// criar o contexto...
$context = stream_context_create($opts);

// ...e usá-lo para obter os dados
echo file_get_contents('zip://test.zip#test.txt', false, $context);

?>
```

Opções de contexto Zlib

Opções de contexto Zlib

Opções de contexto Zlib — Lista de opções de contexto de Zlib

Descrição

Opções de contexto Zlib estão disponíveis para empacotadores `zlib`.

Opções

`level`

Usado para especificar o nível de compressão (`0` - `9`).

Registro de Alterações

Versão	Descrição
7.3.0	O parâmetro `level` foi incluído.

Protocolos e empacotadores suportados

O PHP vem com muitos empacotadores para vários protocolos no estilo URL para uso com as funções do sistema de arquivos, tais como fopen(), copy(), file_exists() e filesize(). Além desses empacotadores, é possível registrar empacotadores customizados usando a função stream_wrapper_register().

> **Nota** :
>
> A sintaxe de URL usada para descrever um empacotadores suporta apenas a sintaxe `esquema://...`. As sintaxes `esquema:/` e `esquema:` não são suportadas.

Índice

file://

file://

file:// — Acessando o sistema de arquivos local

Descrição

`file://` é o empacotador padrão usado com o PHP e representa o sistema de arquivos local. Quando um caminho relativo é especificado (um caminho que não começa com `/`, `\`, `\\`, ou uma letra de unidade do Windows), o caminho fornecido será aplicado ao diretório atual de trabalho. Em muitos casos este é o diretório no qual o script reside a menos que tenha sido alterado. Usando o SAPI CLI, o padrão é o diretório do qual o script foi chamado.

Com algumas funções, tais como fopen() e file_get_contents(), `include_path` pode opcionalmente ser pesquisado com caminho relativo também.

Uso

- */caminho/para/arquivo.ext*
- *caminho/relativo/para/arquivo.ext*
- *arquivoNoDiretorioAtual.ext*
- *C:/caminho/para/arquivoWindows.ext*
- *C:\caminho\para\arquivoWindows.ext*
- *\\smbserver\share\caminho\para\arquivoWindows.ext*
- *file:///caminho/para/arquivo.ext*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Não
Permite Leitura	Sim
Permite Escrita	Sim
Permite Adição	Sim
Permite Leitura e Escrita Simultâneas	Sim
Suporta stat()	Sim
Suporta unlink()	Sim
Suporta rename()	Sim
Suporta mkdir()	Sim
Suporta rmdir()	Sim

http://

http://

https://

http:// -- https:// — Acessando URLs HTTP(s)

Descrição

Permite acesso somente leitura a arquivos/recursos via HTTP. Por padrão um GET HTTP 1.0 é usado. Um cabeçalho `Host:` é enviado com a requisição para lidar com hosts virtuais baseados em nomes. Se estiver configurado uma string user_agent no arquivo *php.ini* ou no contexto do fluxo, ele também será incluído na requisição.

O fluxo permite acesso ao *corpo* do recurso; os cabeçalhos são armazenados na variável *$http_response_header*.

Se for importante conhecer o URL do recurso de onde o documento se originou (depois de todos os redirecionamentos processados), será necessário processar a série de cabeçalhos de resposta retornados pelo fluxo.

A diretiva from será usada para o cabeçalho `From:` se definida e não sobrescrita pelo Opções e parâmetros de contexto.

Uso

- *http://example.com*
- *http://example.com/arquivo.php?var1=val1&var2=val2*
- *http://usuario:senha@example.com*
- *https://example.com*
- *https://example.com/arquivo.php?var1=val1&var2=val2*
- *https://usuario:senha@example.com*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Sim
Permite Leitura	Sim
Permite Escrita	Não
Permite Adição	Não
Permite Leitura e Escrita Simultâneas	N/A
Suporta stat()	Não
Suporta unlink()	Não
Suporta rename()	Não
Suporta mkdir()	Não
Suporta rmdir()	Não

Examples

Exemplo #1 Detectando a qual se chegou depois de todos os redirecionamentos

```php
<?php
$url = 'http://www.example.com/redirecting_page.php';

$fp = fopen($url, 'r');

$meta_data = stream_get_meta_data($fp);
foreach ($meta_data['wrapper_data'] as $response) {

    /* Fomos redirecionados? */
    if (strtolower(substr($response, 0, 10)) == 'location: ') {

        /* atualiza $url para onde fomos redirecionados */
        $url = substr($response, 10);
    }
```

```
}

?>
```

Notas

> **Nota** :
>
> HTTPS só é suportado quando a extensão openssl estiver habilitada.

Conexões HTTP são somente leitura; escrever dados ou copiar arquivos para um recurso HTTP não é suportado.

Enviar requisições *POST* e *PUT*, por exemplo, pode ser feito com a ajuda de Contextos HTTP.

Veja também

- Opções de contexto do HTTP
- *$http_response_header*
- stream_get_meta_data() - Obtém cabeçalhos/metadados de ponteiros de arquivos/fluxos

ftp://

ftp://

ftps://

ftp:// -- ftps:// — Acessando URLs FTP(s)

Descrição

Permite acesso de leitura a arquivos existentes e criação de novos arquivos via FTP. Se o servidor não suportar modo ftp passivo, a conexão irá falhar.

Pode-se abrir arquivos tanto para leitura quanto para escrita, mas não para ambos simultaneamente. Se o arquivo remoto já existir no servidor ftp e houver tentativa de abri-lo para escrita, mas não estiver especificada a opção de contexto `overwrite`, a conexão irá falhar. Se for necessário sobrescrever os arquivos existentes via ftp, deve-se especificar a opção `overwrite` no contexto e abrir o arquivo para escrita. Alternativamente, pode ser usada a extensão FTP.

Se estiver configurada a diretiva from no *php.ini*, então este valor será enviado como a senha do FTP anônimo.

Uso

- *ftp://example.com/pub/arquivo.txt*
- *ftp://usuario:senha@example.com/pub/arquivo.txt*
- *ftps://example.com/pub/arquivo.txt*
- *ftps://usuario:senha@example.com/pub/arquivo.txt*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Sim
Permite Leitura	Sim
Permite Escrita	Sim (arquivos novos/existentes com `overwrite`)
Permite Adição	Sim
Permite Leitura e Escrita Simultâneas	Não
Suporta stat()	Somente os elementos filesize(), filemtime(), filetype(), file_exists(), is_file() e is_dir().
Suporta unlink()	Sim
Suporta rename()	Sim
Suporta mkdir()	Sim
Suporta rmdir()	Sim

Notas

> **Nota** :
>
> FTPS somente é suportado quando a extensão openssl estiver habilitada.
>
> Se o servidor não suportar SSL, a conexão cairá para um FTP comum não criptografado.

> **Nota** : **Adição**
>
> Arquivos podem ter conteúdo acrescentado pelo empacotador de URL `ftp://`.

Veja também

- Opções de contexto do FTP

php://

php://

php:// — Acessando vários fluxos de E/S

Descrição

O PHP fornece um número de fluxos de E/S miscelâneos que permitem acesso aos fluxos de entrada e saída do próprio PHP, a entrada padrão, descritores de arquivo de saída e de erro, fluxos de arquivos temporário em memória e em disco, e filtros que podem manipular outros recursos de arquivo enquanto eles são lidos ou escritos.

php://stdin, php://stdout e php://stderr

php://stdin, *php://stdout* e *php://stderr* permitem acesso direto ao fluxo correspondente de entrada ou saída do processo PHP. O fluxo referencia um descritor de arquivo duplicado, para que quando um *php://stdin* for aberto e depois fechado, apenas a cópia do descritor será fechada - o fluxo real referenciado por **`STDIN`** não é afetado. Recomenda-se que as constantes **`STDIN`**, **`STDOUT`** e **`STDERR`** sejam usadas no lugar de abrir manualmente os fluxos utilizando estes empacotadores.

php://stdin é somente leitura, enquanto que *php://stdout* e *php://stderr* são somente escrita.

php://input

php://input é um fluxo somente leitura que permite a leitura de dados brutos do corpo da requisição. *php://input* não está disponível em requisições POST com `enctype="multipart/form-data"` se a opção enable_post_data_reading estiver habilitada.

php://output

php://output é um fluxo somente escrita que permite escrita ao mecanismo de buffer de saída da mesma maneira que print e echo.

php://fd

php://fd permite acesso direto ao descritor de arquivo fornecido. Por exemplo, *php://fd/3* se refere ao descritor de arquivo 3.

php://memory e php://temp

php://memory e *php://temp* são fluxos somente leitura que permitem que dados temporários sejam armazenados em um empacotador parecido com um arquivo. Uma diferença entre os dois é que *php://memory* irá sempre armazenar seus dados na memória, enquanto que *php://temp* irá usar um arquivo temporário assim que a quantidade de dado armazenado atingir um limite predefinido (o padrão é 2 MB). A localização deste arquivo temporário é determinada da mesma forma que a função sys_get_temp_dir().

O limite de memória de *php://temp* pode ser controlado concatenando-se `/maxmemory:NN`, onde `NN` é a quantidade máxima de dados a manter na memória antes de usar um arquivo temporário, em bytes.

> **Cuidado**
>
> Algumas extensões PHP podem requerer um fluxo padrão de E/S, e podem tentar converter um fluxo fornecido para um fluxo de E/S padrão. Esta conversão pode falhar para fluxos de memória já que ela requer que a função C **fopencookie()** esteja dispnível. Esta função C *não está* disponível no Windows.

php://filter

php://filter é um tipo de meta-empacotador projetado para permitir a aplicação de filtros a um fluxo no momento da abertura. Isto é útil para funções de arquivo completas como readfile(), file() e file_get_contents() pois não haveria oportunidade de aplicar um filtro ao fluxo antes que os conteúdo fosse lido.

O alvo do *php://filter* utiliza os parâmetros a seguir como parte de seu caminho. Cadeias múltiplas de filtro podem ser especificadas em um caminho. Favor referir-se aos exemplos para especificidades no uso destes parâmetros.

Parâmetros de php://filter

Nome	Descrição
`resource=<fluxo a ser filtrado>`	Este parâmetro é requerido. Ele especifica o fluxo que deseja-se filtrar.
`read=<lista de filtros a aplicar à cadeia de leitura>`	Este parâmetro é opcional. Um ou mais nomes de filtros podem ser fornecidos, separados pelo caractere de barra vertical (`|`).
`write=<lista de filtros a aplicar à cadeia de escrita>`	Este parâmetro é opcional. Um ou mais nomes de filtros podem ser fornecidos, separados pelo caractere de barra vertical (`|`).
`<lista de filtros a aplicar a ambas as cadeias>`	Quaisquer listas de filtros que não são prefixadas por `read=` ou `write=` serão aplicadas tanto às cadeias de leitura quanto às de escrita, conforme apropriado.

Opções

Sumário do Empacotador (para `php://filter`, refira-se ao sumário do empacotador sendo filtrado)

Atributo	Suportado
Restrito por allow_url_fopen	Não
Restrito por allow_url_include	`php://input`, `php://stdin`, `php://memory` e `php://temp` somente.
Permite Leitura	`php://stdin`, `php://input`, `php://fd`, `php://memory` e `php://temp` somente.
Permite Escrita	`php://stdout`, `php://stderr`, `php://output`, `php://fd`, `php://memory` e `php://temp` somente.
Permite Adição	`php://stdout`, `php://stderr`, `php://output`, `php://fd`, `php://memory` e `php://temp` somente. (Equivalente a escrita)
Permite Leitura e Escrita Simultâneas	`php://fd`, `php://memory` e `php://temp` somente.
Suporta stat()	Não. Entretanto, `php://memory` e `php://temp` suportam fstat().
Suporta unlink()	Não
Suporta rename()	Não
Suporta mkdir()	Não
Suporta rmdir()	Não
Suporta stream_select()	`php://stdin`, `php://stdout`, `php://stderr`, `php://fd` e `php://temp` somente.

Examples

Exemplo #1 php://temp/maxmemory

Este parâmetro opcional permite configurar o limite de memória antes que *php://temp* comece a usar um arquivo temporário.

```php
<?php
// Define o limite para 5 MB.
$fiveMBs = 5 * 1024 * 1024;
$fp = fopen("php://temp/maxmemory:$fiveMBs", 'r+');

fputs($fp, "hello\n");
```

```php
// Lê o que foi escrito.
rewind($fp);
echo stream_get_contents($fp);
?>
```

Exemplo #2 php://filter/resource=<fluxo a ser filtrado>

Este parâmetro deve ser localizado no final da especificação *php://filter* e deve ser apontado ao fluxo que deseja-se filtrar.

```php
<?php
/* Isto é equivalente a simplesmente:
  readfile("http://www.example.com");
  já que nenhum filtro é realmente especificado */

readfile("php://filter/resource=http://www.example.com");
?>
```

Exemplo #3 php://filter/read=<lista de filtros a aplicar à cadeia de leitura>

Este parâmetro leva um ou mais nomes de filtros separados pelo caractere de barra vertical `|`.

```php
<?php
/* Isto irá mostrar o conteúdo de
  www.example.com inteiramente em letras maiúsculas */
readfile("php://filter/read=string.toupper/resource=http://www.example.com");

/* Isto fará o mesmo que o comando acima
  mas também irá codificar com ROT13 */
readfile("php://filter/read=string.toupper|string.rot13/resource=http://www.example.com");
?>
```

Exemplo #4 php://filter/write=<lista de filtros a aplicar à cadeia de escrita>

Este parâmetro leva um ou mais nomes de filtros separados pelo caractere de barra vertical `|`.

```php
<?php
/* Isto irá filtrar a string "Olá Mundo"
  através do filtro rot13 filter, e então escrever no
  exemplo.txt no diretório atual */
file_put_contents("php://filter/write=string.rot13/resource=exemplo.txt","Olá Mundo");
?>
```

Exemplo #5 php://memory e php://temp não são reutilizáveis

php://memory e *php://temp* não são reutilizáveis, isto é, depois que os fluxos foram fechados não há como referir-se a eles novamente.

```php
<?php
file_put_contents('php://memory', 'PHP');
echo file_get_contents('php://memory'); // não imprime nada
```

Exemplo #6 php://input para leitura de dados JSON do corpo da requisição

Este exemplo demonstra como ler dados JSON brutos de requisições POST, PUT e PATCH usando *php://input*.

```
<?php
$input = file_get_contents("php://input");
$json_array = json_decode(
  json: $input,
  associative: true,
  flags: JSON_THROW_ON_ERROR
);

echo "Dados JSON recebidos: ";
print_r($json_array);
?>
```

zlib://

zlib://

bzip2://

zip://

zlib:// -- bzip2:// -- zip:// — Fluxos de compactação

Descrição

compress.zlib:// e *compress.bzip2://*

zlib: funciona como gzopen(), exceto que o fluxo pode ser usado com fread() e outras funções do sistema de arquivos. Isso foi descontinuado devido a ambiguidades com nomes de arquivos contendo caracteres ':'; use *compress.zlib://* em vez disso.

compress.zlib:// e *compress.bzip2://* são equivalentes para gzopen() e bzopen() respectivamente, e funcionam mesmo em sistemas que não suportam fopencookie.

A extensão ZIP registra o empacotador *zip:*. a partir do PHP 7.2.0 e libzip 1.2.0+, foi adicionado suporte para as senhas para arquivos criptografados, permitindo que as senhas sejam fornecidas por contextos de fluxo. As senhas podem ser definidas usando `'password'`, uma opção de contexto de fluxo.

Uso

- *compress.zlib://file.gz*
- *compress.bzip2://file.bz2*
- *zip://archive.zip#dir/file.txt*

Opções

Resumo do Empacotador

Atributo	Suportado
Restringido por allow_url_fopen	Não
Permite leitura	Sim

Atributo	Suportado
Permite escrita	Sim (exceto `zip://`)
Permite adição	Sim (exceto `zip://`)
Permite leitura e escrita simultâneas	Não
Suporta stat()	Não, use o empacotador `file://` para verificar o "stat" de arquivos compactados.
Suporta unlink()	Não, use o empacotador `file://` para excluir arquivos compactados.
Suporta rename()	Não
Suporta mkdir()	Não
Suporta rmdir()	Não

Veja também

- Opções de contexto Zlib

data://

data://

data:// — Dados (RFC 2397)

Descrição

O empacotador de fluxo *data:* (» RFC 2397).

Uso

- *data://text/plain; base64,*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Sim
Restrito por allow_url_include	Sim
Permite Leitura	Sim
Permite Escrita	Não
Permite Adição	Não
Permite Leitura e Escrita Simultâneas	Não
Suporta stat()	Não
Suporta unlink()	Não
Suporta rename()	Não
Suporta mkdir()	Não

Atributo	Suportado
Suporta rmdir()	Não

Examples

Exemplo #1 Imprimir conteúdo de data://

```php
<?php
// prints "I love PHP"
echo file_get_contents('data://text/plain; base64,SSBsb3ZlIFBIUAo=');
?>
```

Exemplo #2 Obtém o tipo de mídia

```php
<?php
$fp   = fopen('data://text/plain; base64,', 'r');
$meta = stream_get_meta_data($fp);

// prints "text/plain"
echo $meta['mediatype'];
?>
```

glob://

glob://

glob:// — Encontra caminhos que correspondem a um padrão

Descrição

O empacotador de fluxo *glob:*.

Uso

- *glob://*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Não
Restrito por allow_url_include	Não
Permite Leitura	Não
Permite Escrita	Não
Permite Adição	Não
Permite Leitura e Escrita Simultâneas	Não
Suporta stat()	Não
Suporta unlink()	Não
Suporta rename()	Não

Atributo	Suportado
Suporta mkdir()	Não
Suporta rmdir()	Não

Examples

Exemplo #1 Uso básico

```php
<?php
// Percorre todos os arquivos *.php no diretório ext/spl/examples/
// e mostra o nome do arquivo e seu tamanho
$it = new DirectoryIterator("glob://ext/spl/examples/*.php");
foreach($it as $f) {
    printf("%s: %.1FK\n", $f->getFilename(), $f->getSize()/1024);
}
?>
```

```
tree.php: 1.0K
findregex.php: 0.6K
findfile.php: 0.7K
dba_dump.php: 0.9K
nocvsdir.php: 1.1K
phar_from_dir.php: 1.0K
ini_groups.php: 0.9K
directorytree.php: 0.9K
dba_array.php: 1.1K
class_tree.php: 1.8K
```

phar://

phar://

phar:// — Arquivo PHP

Descrição

O empacotador de fluxo *phar://*. Consulte Empacotador de fluxo Phar para uma descrição detalhada.

Uso

- *phar://*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Não

Atributo	Suportado
Restrito por allow_url_include	Não
Permite Leitura	Sim
Permite Escrita	Sim
Permite Adição	Não
Permite Leitura e Escrita Simultâneas	Sim
Suporta stat()	Sim
Suporta unlink()	Sim
Suporta rename()	Sim
Suporta mkdir()	Sim
Suporta rmdir()	Sim

Veja também

- Opções de contexto do Phar

ssh2://

ssh2://

ssh2:// — Shell Seguro 2

Descrição

ssh2.shell:// ssh2.exec:// ssh2.tunnel:// ssh2.sftp:// ssh2.scp:// (PECL)

> **Nota** : **Este empacotador não é habilitado por padrão.**
>
> Para utilizar os empacotadores *ssh2.*://*, a extensão » SSH2 disponível em » PECL deve estar instalada.

Além de aceitar detalhes de login URI tradicionais, os empacotadores ssh2 também farão reúso de conexões abertas através da passagem do recurso da conexão na parte do host da URL.

Uso

- *ssh2.shell://usuario:senha@example.com:22/xterm*
- *ssh2.exec://usuario:senha@example.com:22/usr/local/bin/algumcomando*
- *ssh2.tunnel://usuario:senha@example.com:22/192.168.0.1:14*
- *ssh2.sftp://usuario:senha@example.com:22/caminho/para/arquivo*

Opções

Sumário do Empacotador

Atributo	ssh2.shell	ssh2.exec	ssh2.tunnel	ssh2.sftp	ssh2.scp

Atributo	ssh2.shell	ssh2.exec	ssh2.tunnel	ssh2.sftp	ssh2.scp
Restrito por allow_url_fopen	Sim	Sim	Sim	Sim	Sim
Permite Leitura	Sim	Sim	Sim	Sim	Sim
Permite Escrita	Sim	Sim	Sim	Sim	Não
Permite Adição	Não	Não	Não	Sim (quando suportado pelo servidor)	Não
Permite Leitura e Escrita Simultâneas	Sim	Sim	Sim	Sim	Não
Suporta stat()	Não	Não	Não	Sim	Não
Suporta unlink()	Não	Não	Não	Sim	Não
Suporta rename()	Não	Não	Não	Sim	Não
Suporta mkdir()	Não	Não	Não	Sim	Não
Suporta rmdir()	Não	Não	Não	Sim	Não

Opções de contexto

Nome	Uso	Padrão
`session`	Recurso ssh2 pré-conectado a ser reutilizado	
`sftp`	Recurso sftp pré-alocado a ser reutilizado	
`methods`	Métodos de troca de chaves, de chave de host, de cifras, de compressão e de MAC a serem usados	
`callbacks`		
`username`	Nome de usuário a ser conectado	
`password`	Senha a ser usada com a autenticação	
`pubkey_file`	Nome do arquivo de chave pública a ser usado com a autenticação	
`privkey_file`	Nome do arquivo de chave privada a ser usado com a autenticação	
`env`	Array associativo de variáveis de ambiente a ser configurado	
`term`	Tipo de emulação de terminal a ser requisitado quando da alocação de um pty	
`term_width`	Largura do terminal requisitado quando da alocação de um pty	
`term_height`	Altura do terminal requisitado quando da alocação de um pty	
`term_units`	Unidadem usadas em term_width e term_height	`SSH2_TERM_UNIT_CHARS`

Examples

Exemplo #1 Abrindo um fluxo para conexão ativa

```php
<?php
$session = ssh2_connect('example.com', 22);
ssh2_auth_pubkey_file($session, 'username', '/home/username/.ssh/id_rsa.pub',
                                            '/home/username/.ssh/id_rsa', 'secret');
$stream = fopen("ssh2.tunnel://$session/remote.example.com:1234", 'r');
?>
```

Exemplo #2 Esta variável *$session* deve ser mantida disponível!

Para usar os empacotadores *ssh2.*://$session*, a variável de recurso *$session* deve ser mantida. O código abaixo não terá o efeito desejado:

```php
<?php
$session = ssh2_connect('example.com', 22);
ssh2_auth_pubkey_file($session, 'username', '/home/username/.ssh/id_rsa.pub',
                                            '/home/username/.ssh/id_rsa', 'secret');
$connection_string = "ssh2.sftp://$session/";
unset($session);
$stream = fopen($connection_string . "path/to/file", 'r');
?>
```

unset() fecha a sessão, porque *$connection_string* não contém uma referência à variável *$session*, somente uma string convertida derivada dela. Isto também acontece quando a função unset() está implícita devido a uma saída de escopo (como em uma função).

rar://

rar://

rar:// — RAR

Descrição

Este empacotador usa o caminho para o arquivo RAR (relativo ou absoluto) codificado em url, um asterisco opcional (*), um símbolo de número opcional (#) e uma entrada de nome opcional codificada em url, como armazenado no arquivo. Especificar uma entrada de nome requer o símbolo de número; Uma barra (/) no início da entrada de nome é opcional.

Este empacotador pode abrir tanto arquivos quanto diretórios. Quando abre diretórios, o asterisco força o retorno não codificado dos nomes das entradas do diretório. Se não houver asterisco, eles serão retornados com codificação url – a razão para isto é permitir que o empacotador seja usado corretamente com funcionalidades internas como RecursiveDirectoryIterator na presença de nomes de arquivo que se parecem com dados codificados de url.

Se o símbolo de número (#) e a parte de nome de entrada não forem incluídos, a raiz dos arquivos será mostrada. Isto difere de diretórios comuns pelo fato que o fluxo resultante não conterá informações como horário de modificação, já que o diretório raiz não é armazenado em uma entrada individual no arquivo. O uso do empacotador com RecursiveDirectoryIterator requer que o símbolo de número seja incluído na URL ao acessar a raiz, para que as URLs dos ramos filhos sejam construídos corretamente.

> **Nota** : **Este empacotador não é habilitado por padrão**
>
> Para usar o empacotador *rar://*, a extensão » rar disponível em » PECL deve ser instalada.

rar:// Disponível desde o PECL rar 3.0.0

Uso

- *rar://<nome de arquivo codificado como url>[*][#[<nome de entrada codificado como url>]]*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Não
Restrito por allow_url_include	Não
Permite Leitura	Sim
Permite Escrita	Não
Permite Adição	Não
Permite Leitura e Escrita Simultâneas	Não
Suporta stat()	Sim
Suporta unlink()	Não
Suporta rename()	Não
Suporta mkdir()	Não
Suporta rmdir()	Não

Opções de contexto

Nome	Uso	Padrão
`open_password`	A senha utilizada para criptografar os cabeçalhos do arquivo, se houver. WinRAR irá criptografar todos os arquivos com a mesma senha do cabeçalho quando ela estiver presente, ou seja, para arquivos com cabeçalhos criptografados, `file_password` será ignorada.	
`file_password`	A senha utilizada para criptografar um arquivo, se houver. Se os cabeçalhos também estiverem criptografados, esta opção será ignorada em favor de `open_password`. A razão pela qual existem duas opções é para cobrir a possibilidade de suportar arquivos com senhas diferentes para cabeçalhos e arquivos, se este tipo de arquivo porventura vier a existir. Note que se o arquivo não tiver seus cabeçalhos criptografados, `open_password` será ignorada e esta opção deve ser usada no lugar.	
`volume_callback`	Uma função de retorno para determinar o caminho de volumes faltantes. Consulte RarArchive::open() para mais informação.	

Examples

Exemplo #1 Explorando um arquivo RAR

```
<?php

class MyRecDirIt extends RecursiveDirectoryIterator {
    function current() {
        return rawurldecode($this->getSubPathName()) .
            (is_dir(parent::current())?" [DIR]":"");
    }
}

$f = "rar://" . rawurlencode(dirname(__FILE__)) .
```

```
    DIRECTORY_SEPARATOR . 'dirs_and_extra_headers.rar#';

$it = new RecursiveTreeIterator(new MyRecDirIt($f));

foreach ($it as $s) {
    echo $s, "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
|-allow_everyone_ni [DIR]
|-file1.txt
|-file2_ß.txt
|-with_streams.txt
\-ß [DIR]
  |-%2\ßempty%2E [DIR]
  | \-%2\ßempty%2E\file7.txt
  |-ß\empty [DIR]
  |-ß\file3.txt
  |-ß\file4_ß.txt
  \-2_ß\ß [DIR]
    |-2_ß\ß\file5.txt
    \-2_ß\ß\file6_ß.txt
```

Exemplo #2 Abrindo um arquivo criptografado (criptografia de cabeçalho)

```
<?php
$stream = fopen("rar://" .
    rawurlencode(dirname(__FILE__)) . DIRECTORY_SEPARATOR .
    'encrypted_headers.rar' . '#encfile1.txt', "r", false,
    stream_context_create(
        array(
            'rar' =>
                array(
                    'open_password' => 'samplepassword'
                )
            )
        )
    );
var_dump(stream_get_contents($stream));
/* datas de criação e de último acesso são opcionais no WinRAR, por isso a maioria
 * dos arquivos não as possui */
var_dump(fstat($stream));
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(26) "Encrypted file 1 contents."
Array
(
    [0] => 0
    [1] => 0
    [2] => 33206
    [3] => 1
    [4] => 0
    [5] => 0
    [6] => 0
    [7] => 26
    [8] => 0
    [9] => 1259550052
    [10] => 0
    [11] => -1
    [12] => -1
    [dev] => 0
    [ino] => 0
    [mode] => 33206
    [nlink] => 1
    [uid] => 0
    [gid] => 0
    [rdev] => 0
    [size] => 26
    [atime] => 0
    [mtime] => 1259550052
    [ctime] => 0
    [blksize] => -1
    [blocks] => -1
)
```

ogg://

ogg://

ogg:// — Fluxos de áudio

Descrição

Os arquivos abertos para leitura por meio do empacotador *ogg://* são tratados como áudio compactado codificado usando o codec `OGG/Vorbis`. Da mesma forma, os arquivos abertos para gravação ou adição por meio do empacotador *ogg://* são gravados como dados de áudio compactados. stream_get_meta_data(), quando usado em um arquivo `OGG/Vorbis` aberto para leitura, retornará vários detalhes sobre o fluxo, incluído a etiqueta `vendor`, quaisquer comentários incluídos em `comments`, o número de canais em `channels`, a taxa no parâmetro `rate`, e o intervalo da taxa de codificação descrito por: `bitrate_lower`, `bitrate_upper`, `bitrate_nominal`, e `bitrate_window`.

ogg:// (PECL)

> **Nota** : **Este empacotador não é habilitado por padrão**

> Para poder utilizar o empacotador *ogg://*, a extensão » OGG/Vorbis disponível em » PECL precisa estar instalada.

Uso

- *ogg://soundfile.ogg*
- *ogg:///path/to/soundfile.ogg*
- *ogg://http://www.example.com/path/to/soundstream.ogg*

Opções

Resumo do Empacotador

Atributo	Suportado
Restringido por allow_url_fopen	Não
Permite leitura	Sim
Permite escrita	Sim
Permite adição	Sim
Permite leitura e ecrita simultâneas	Não
Suporta stat()	Não
Suporta unlink()	Não
Suporta rename()	Não
Suporta mkdir()	Não
Suporta rmdir()	Não

Opções de contexto

Nome	Uso	Padrão	Modo
`pcm_mode`	Codificação PCM a ser aplicada durante a leitura, uma das seguintes constantes: **`OGGVORBIS_PCM_U8`**, **`OGGVORBIS_PCM_S8`**, **`OGGVORBIS_PCM_U16_BE`**, **`OGGVORBIS_PCM_S16_BE`**, **`OGGVORBIS_PCM_U16_LE`** e **`OGGVORBIS_PCM_S16_LE`**. (8 bits versus 16 bits, com sinal ou sem sinal, big ou little `endian`)	OGGVORBIS_PCM_S16_LE	Leitura
`rate`	Taxa de amostragem de dados de entrada, expressa em Hz	44100	Escrita/Adição
`bitrate`	Quando fornecido como um número inteiro, refere-se à taxa de bits fixa para codificação. (16000 to 131072) Quando fornecido como um float, refere-se à qualidade da taxa de bits variável a ser utilizada. (-1.0 to 1.0)	128000	Escrita/Adição
`channels`	O número de canais de áudio a serem codificados, geralmente 1 (mono) ou 2 (estéreo). Pode variar até 16.	2	Escrita/Adição

Nome	Uso	Padrão	Modo
`comments`	Um array de strings para codificar no cabeçalho da faixa.		Escrita/Adição

expect://

expect://

expect:// — Fluxos de Interação entre Processos

Descrição

Fluxos abertos através do empacotador *expect://* fornecem acesso ao stdin, stdout e stderr do processo via PTY.

> **Nota** : **Este empacotador não é habilitado por padrão**
>
> Para usar o empacotador *expect://*, a extensão » Expect disponível em » PECL deve ser instalada.

expect:// (PECL)

Uso

- *expect://command*

Opções

Sumário do Empacotador

Atributo	Suportado
Restrito por allow_url_fopen	Não
Permite Leitura	Sim
Permite Escrita	Sim
Permite Adição	Sim
Permite Leitura e Escrita Simultâneas	Não
Suporta stat()	Não
Suporta unlink()	Não
Suporta rename()	Não
Suporta mkdir()	Não
Suporta rmdir()	Não

Lista de Palavras Reservadas do PHP

Índice

Esta é uma lista de identificadores predefinidos do PHP. Nenhum dos identificadores listados aqui deve ser usado como identificador em qualquer um dos seus scripts a menos que seja explicitamente indicado. Esta lista inclui palavras-chave e variáveis predefinidas, constantes e nomes de classes. Estas listas ainda não são exaustivas nem completas.

Lista de Palavras-chave

Estas palavras têm um significado especial no PHP. Algumas delas representam coisas que parecem funções, algumas parecem constantes, e assim por diante - mas na verdade não o são realmente: elas são constructos da linguagem. As palavras seguintes não podem ser usadas como constantes, nomes de classes, ou nome de funções. Estas palavras-chave são permitidas em propriedades, constantes e nomes de métodos em classes, interfaces, traits, e enumerações com exceção de `class`, que não pode ser utilizada como nome de constante.

Palavras-chave do PHP

__halt_compiler()	abstract	and	array()	as
break	callable	case	catch	class
clone	const	continue	declare	default
die()	do	echo	else	elseif
empty()	enddeclare	endfor	endforeach	endif
endswitch	endwhile	eval()	exit()	extends
final	finally	fn (desde o PHP 7.4)	for	foreach
function	global	goto	if	implements
include	include_once	instanceof	insteadof	interface
isset()	list()	match (desde o PHP 8.0)	namespace	new
or	print	private	protected	public
readonly (desde o PHP 8.1.0) *	require	require_once	return	static
switch	throw	trait	try	unset()
use	var	while	xor	yield
yield from				

* `readonly` pode ser utilizado como nome de função

Constantes de tempo de compilação

__CLASS__	__DIR__	__FILE__	__FUNCTION__	__LINE__
__METHOD__	__PROPERTY__	__NAMESPACE__	__TRAIT__	

Classes Predefinidas

Esta seção lista classes predefinidas padrão. Extensões diversas definem outras classes que são descritas em suas referências.

Classes Definidas Padrão

Estas classes são definidas dentro do conjunto padrão de funções incluídas na compilação do PHP.

Directory

: Criada pela função dir().

stdClass

: Uma classe vazia criada como o resultado de coerção para objeto ou funções padrão da linguagem.

__PHP_Incomplete_Class

: Possivelmente criada pela função unserialize().

Exception

ErrorException

php_user_filter

Closure

: A classe predefinida final Closure é utilizada para representar funções anônimas.

Generator

: A classe predefinida final Generator é utilizada para representar geradores.

ArithmeticError

AssertionError

DivisionByZeroError

Error

Throwable

ParseError

TypeError

Classes especiais

Os seguintes identificadores não podem ser usados como nome de classe pois eles têm um propósito especial.

self

: Classe atual.

static

Classe atual em tempo de execução.

parent

Classe pai.

Constantes Predefinidas

Principais Constantes Predefinidas

Essas constantes são definidas pelo núcleo do PHP. Isso inclui o PHP, o motor Zend e módulos SAPI.

PHP_VERSION (string)

A versão atual do PHP como uma string no formato "maior.menor.lançamento[extra]".

PHP_MAJOR_VERSION (int)

O número de versão "maior" do PHP como um inteiro (exemplo: int(5) para a versão "5.2.7-extra").

PHP_MINOR_VERSION (int)

O número de versão "menor" do PHP como um inteiro (exemplo: int(2) para a versão "5.2.7-extra").

PHP_RELEASE_VERSION (int)

O número de versão de "lançamento" do PHP como um inteiro (exemplo: int(7) para a versão "5.2.7-extra").

PHP_VERSION_ID (int)

A versão atual do PHP como um inteiro, útil para comparações entre versões (exemplo: int(50207) na versão "5.2.7-extra").

PHP_EXTRA_VERSION (string)

A versão "extra" do PHP como uma string (exemplo: '-extra' na versão "5.2.7-extra"). Geralmente utilizado por distribuidores para indicar uma versão de empacotamento.

PHP_BUILD_DATE (string)

A data e o horário que o PHP foi compilado, no formato `"M d Y H:i:s"`. Disponível a partir do PHP 8.5.0.

PHP_BUILD_PROVIDER (string)

O fornecedor que compilou o PHP. Disponível a partir do PHP 8.5.0.

> **Nota** :
>
> Esta constante pode não estar disponível em todas as compilações do PHP. (ex: não definida ao compilar a partir do código-fonte sem especificar um valor durante a configuração).

ZEND_THREAD_SAFE (bool)

Indica se a compilação atual do PHP é segura para "threads".

ZEND_DEBUG_BUILD (bool)

Indica se a compilação atual do PHP é uma compilação para depuração.

PHP_ZTS (bool) Sinônimo de **ZEND_THREAD_SAFE**

Indica se a compilação atual do PHP é segura para "threads".

PHP_DEBUG (bool) Sinônimo de **ZEND_DEBUG_BUILD**

Indica se a compilação atual do PHP é uma compilação para depuração.

DEBUG_BACKTRACE_PROVIDE_OBJECT (int)

Preenche o índice "object".

DEBUG_BACKTRACE_IGNORE_ARGS (int)

Não inclui o argumento de funções no stack trace.

PHP_MAXPATHLEN (int)

O tamanho máximo do nome de um arquivo (incluindo seus diretórios) suportado nesta versão do PHP.

PHP_OS (string)

O sistema operacional para o qual o PHP foi compilado.

PHP_OS_FAMILY (string)

A família do sistema operacional para o qual o PHP foi compilado. Qualquer uma entre `'Windows'`, `'BSD'`, `'Darwin'`, `'Solaris'`, `'Linux'` ou `'Unknown'`. Disponível a partir do PHP 7.2.0.

PHP_SAPI (string)

O nome da Server API para esta compilação do PHP. Veja também php_sapi_name().

PHP_EOL (string)

O símbolo correto de 'Fim de linha' para esta plataforma.

PHP_INT_MAX (int)

O maior inteiro suportado nesta compilação do PHP. Geralmente int(2147483647) em sistemas de 32 bit e int(9223372036854775807) em sistemas de 64 bits.

PHP_INT_MIN (int)

O menor inteiro suportado nesta compilação do PHP. Geralmente int(-2147483648) em sistemas 32 bits e int(-9223372036854775808) em sistemas 64 bit. Normalmente PHP_INT_MIN === ~PHP_INT_MAX.

PHP_INT_SIZE (int)

O tamanho de um inteiro em bytes nesta compilação do PHP.

PHP_FLOAT_DIG (int)

Número de dígitos decimais que podem ser arredondados para um float e recuperados sem perda de precisão. Disponível a partir do PHP 7.2.0.

PHP_FLOAT_EPSILON (float)

O menor número positivo representável x, tal que `x + 1.0 != 1.0`. Disponível a partir do PHP 7.2.0.

PHP_FLOAT_MIN (float)

O menor número de ponto flutuante *positivo* representável. Se você precisar do menor número de ponto flutuante *negativo*, use `-PHP_FLOAT_MAX`. Disponível a partir do PHP 7.2.0.

PHP_FLOAT_MAX (float)

O maior número de ponto flutuante representável. Disponível a partir do PHP 7.2.0.

DEFAULT_INCLUDE_PATH (string)

PEAR_INSTALL_DIR (string)

PEAR_EXTENSION_DIR (string)

PHP_EXTENSION_DIR (string)

O diretório padrão onde localizar extensões carregáveis dinamicamente (podendo ser sobrescrito por extension_dir). Tem como o padrão **PHP_PREFIX** (ou `PHP_PREFIX . "\\ext"` no Windows).

PHP_PREFIX (string)

O valor **--prefix** foi configurado durante o comando "configure". No Windows, este é o valor de **--with-prefix** informado no comando "configure".

PHP_BINDIR (string)

O valor **--bindir** foi definido no comando "configure". No Windows, é o valor informado em **--with-prefix** durante o comando "configure".

PHP_SBINDIR (string)

O valor **--sbindir** foi definido no comando "configure". No Windows, é o valor informado em **--with-prefix** durante o comando "configure". Disponível a partir do PHP 8.4.0.

PHP_BINARY (string)

Especifica o caminho do binário do PHP utilizado na execução do script.

PHP_MANDIR (string)

Especifica onde as páginas do manual foram instaladas.

> **Nota** :
>
> Esta constante não está presente nas versões Windows do PHP.

PHP_LIBDIR (string)

PHP_DATADIR (string)

PHP_SYSCONFDIR (string)

PHP_LOCALSTATEDIR (string)

PHP_CONFIG_FILE_PATH (string)

PHP_CONFIG_FILE_SCAN_DIR (string)

PHP_SHLIB_SUFFIX (string)

O sufixo de bibliotecas compartilhadas da plataforma na qual o PHP foi compilado. Geralmente "so" (Unix) ou "dll" (Windows).

PHP_FD_SETSIZE (int)

O número máximo de descritores de arquivos para as chamadas de sistema "select". Disponível a partir do PHP 7.1.0.

E_ERROR (int)
E_WARNING (int)
E_PARSE (int)
E_NOTICE (int)
E_CORE_ERROR (int)
E_CORE_WARNING (int)
E_COMPILE_ERROR (int)
E_COMPILE_WARNING (int)
E_USER_ERROR (int)
E_USER_WARNING (int)
E_USER_NOTICE (int)
E_RECOVERABLE_ERROR (int)
E_DEPRECATED (int)
E_USER_DEPRECATED (int)
E_ALL (int)
E_STRICT (int)

Constantes para relatório de erros

__COMPILER_HALT_OFFSET__ (int)

true (bool)

Veja Booleanos.

false (bool)

Veja Booleanos.

null (null)

Veja Null.

PHP_WINDOWS_EVENT_CTRL_C (int)

Um evento `CTRL +C` do Windows. Disponível a partir do PHP 7.4.0 (somente para o Windows).

`PHP_WINDOWS_EVENT_CTRL_BREAK` (int)

Um evento `CTRL +BREAK` do Windows. Disponível a partir do PHP 7.4.0 (somente para o Windows).

`PHP_CLI_PROCESS_TITLE` (bool)

Indica se a configuração e obtenção do título do processo está disponível. Disponível apenas no CLI SAPI.

`STDERR` (resource)

Um fluxo já aberto para `stderr`. Disponível apenas no CLI SAPI.

`STDIN` (resource)

Um fluxo já aberto para `stdin`. Disponível apenas no CLI SAPI.

`STDOUT` (resource)

Um fluxo já aberto para `stdout`. Disponível apenas no CLI SAPI.

Veja também: Constantes Mágicas.

Constantes Padrão Predefinidas

Todas as constantes de extensões do núcleo são definidas no PHP por padrão.

Lista de outras palavras reservadas

As palavras a seguir não podem ser utilizadas como nome de classes, interfaces, traits ou enumerações. Anteriormente ao PHP 8.0, também eram proibidas de ser usadas em namespaces.

Palavras reservadas

parent	self	int	float
bool	string	true	false
null	void (a partir do PHP 7.1)	iterable (a partir do PHP 7.1)	object (a partir do PHP 7.2)
mixed (a partir do PHP 8.0)	never (a partir do PHP 8.1)	array (a partir do PHP 8.5)	callable (a partir do PHP 8.5)

A seguinte lista de palavras tem reservas brandas. Embora elas possam ser utilizadas como nomes de classes, interfaces, traits e enumerações, seu uso é altamente desencorajado pois elas podem se tornar palavras reservadas em versões futuras do PHP.

Palavras reservadas brandas

enum	resource	numeric	

Segurança

Introdução

PHP é uma linguagem poderosa e um interpretador, seja incluído em um servidor web como um módulo ou executado separadamente como binário CGI, é possível acessar arquivos, executar comandos e abrir conexões de rede no servidor. Essas propriedades fazem qualquer coisa executando em um servidor web inseguras por padrão. PHP é desenhado especificamente para ser uma linguagem mais segura para escrever programas CGI que Perl ou C, e com a escolha correta de opções de configuração em tempo compilação ou de execução, e práticas corretas de programação, ela pode dar a combinação exata de liberdade e segurança que você precisa.

Como existem diferentes maneiras de utilizar o PHP, existem várias opções de configuração controlando seu comportamento. Um grande leque de opções garante que você pode usar o PHP para vários propósitos, mas também significa que existem combinações dessas opções e configurações do servidor que resultam em uma instalação insegura.

A flexibilidade de configuração do PHP é comparável com a flexibilidade de código. PHP pode ser usado para montar um servidor de aplicações completo, com todo o poder de um usuário shell, ou pode ser usado para inclusão simples de arquivos no lado do servidor com pouco risco em um ambiente bem controlado. Como montar o ambiente, e o quão seguro ele é, depende muito do desenvolvedor.

Esse capítulo começa com alguns conselhos genéricos de segurança, explica as diferentes combinações de opções de configuração e as situações nas quais elas podem ser usadas com segurança e descreve diferentes considerações quanto ao desenvolvimento para diferentes níveis de segurança.

Considerações Gerais

Um sistema completamente seguro é virtualmente impossível de se conseguir, então uma abordagem freqüentemente usada em segurança é um compromisso entre risco e usabilidade. Se cada variável enviada pelo usuário precisar de duas formas de validação biométrica (como escaneamento de retina e impressão digital), você teria um nível de checagem extremamente alto. Demoraria meia hora para preencher um formulário mais ou menos complexo, o que incentiva os usuários a achar maneiras de burlar a segurança.

A melhor segurança é frequentemente aquele preenche os requerimentos sem obstruir o usuário a fazer o seu trabalho, ou sobrecarregando o programador com complexidade excessiva. De fato, alguns ataques de segurança exploram esse tipo de segurança super-produzida, que tende a degradar com o tempo.

Uma frase que vale a pena lembrar: Um sistema é tão bom quanto o elo mais fraco na corrente. Se todas as transações são maciçamente registradas baseado no tempo, localização, tipo de transação, etc. mas o usuário só é verificado baseado em um único cookie, a validade de ligar os usuários ao registro de transação torna-se muito fraca.

Quando estiver testando, tenha em mente que você não será capaz de testar todas as possibilidades nem mesmo para as páginas mais simples. A entrada que você pode esperar será totalmente diferente da entrada dada por um empregado irritado, um cracker com meses livres para tentar quebrar o sistema, ou um gato andando pelo teclado. Por isso é melhor olhar para o código da perspectiva lógica, para discernir onde dados inesperados podem ser introduzidos, e depois seguir onde o mesmo é modificado, reduzido ou amplificado.

A Internet está cheia de gente tentando fazer o próprio nome quebrando o código dos outros, derrubando sites, enviando conteúdo indevido e de outras formas fazendo seu dia interessante. Não importa se você tem um site grande ou pequeno, você é um alvo simplesmente por estar online, tendo um servidor que pode ser conectado. Muitos programas de cracking não discernem por tamanho, eles simplesmente vasculham blocos gigantes de IPs procurando por vítimas. Tente não se tornar um.

Instalando como binário CGI

Índice

Ataque Possível

Usando o PHP como binário CGI é uma opção de instalação que por alguma razão não deseja integrar o PHP como um módulo no servidor (como o Apache), ou usará o PHP com tipos diferentes de empacotadores CGI para criar ambientes **chroot** e **setuid** seguros para os scripts. Esta configuração geralmente envolve a instalação do binário **php** executável para o diretório *cgi-bin* do servidor web. A CERT Advisory » CA-96.11 recomenda não colocar nenhum interpretador em *cgi-bin*. Mesmo se o binário **php** possa ser usado como um interpretador autônomo, o PHP foi projetado para evitar os ataques que essa forma de instalação torna possível:

- Acessar arquivos de sistema: *http://my.host/cgi-bin/php?/etc/passwd*

  A informação de consulta em uma URL depois da interrogação (`?`) é passada como argumentos de linha de comando para o interpretador pela interface CGI. Normalmente os interpretadores abrem e executam o arquivo especificado como primeiro argumento na linha de comando.

  Quando invocado como binário CGI, o **php** se recusa a interpretar os argumentos de linha de comando.

- Acessar qualquer domento web no servidor: *http://my.host/cgi-bin/php/secret/doc.html*

  A parte de informação de caminho da URL depois do nome do binário do PHP, */secret/doc.html* é convencionalmente usada para especificar o nome do arquivo a ser aberto e interpretado pelo programa CGI Normalmente algumas diretivas de configuração do servidor web (Apache: `Action`) são usadas para redirecionar requisições para documentos como *http://my.host/secret/script.php* para o interpretados do PHP. Dessa maneira, o servidor web primeiro checa as permissões de acesso ao diretório */secret*, e depois cria a requisição redirecionada *http://my.host/cgi-bin/php/secret/script.php*. Infelizmente, se a requisição é dada originalmente nessa forma, a checagem de permissão não é feita para o arquivo */secret/script.php*, mas apenas para o arquivo */cgi-bin/php*. Dessa maneira qualquer usuário que pode acessar */cgi-bin/php* pode acessar quaisquer documentos protegidos no servidor web.

  No PHP, as diretivas de tempo de execução cgi.force_redirect, doc_root e user_dir podem ser usadas para prevenir esse ataque, se a árvore de documentos do servidor tiver qualquer diretório com restrições de acesso. Veja abaixo a explicação completa das diferentes combinações.

Caso 1: apenas arquivos públicos são disponibilizados

Se o seu servidor não tiver qualquer conteúdo que não seja restringido por senha ou por IP, não há necessidade para essas opções de configuração. Se seu servidor web não permitir que você faça redirecionamento, ou o servidor não tem uma maneira de comunicar ao binário PHP que a requisição é uma

redirecionada de maneira segura, você pode especificar a opção --enable-force-cgi-redirect no script configure. Você ainda tem que assegurar que os seus scripts PHP não confiem com uma ou outra maneira de chamar o script, nem diretamente *http://my.host/cgi-bin/php/dir/script.php* nem por redirecionamento *http://my.host/dir/script.php*.

Redirecionamento pode ser configurado no Apache usando `AddHandler` e diretivas `Action` (veja abaixo).

Caso 2: usando cgi.force_redirect

A diretiva de configuração cgi.force_redirect impede que **php** seja chamado diretamente com uma URL do tipo *http://my.host/cgi-bin/php/secretdir/script.php*. Ao invés disso, PHP só irá executar dessa forma se tiver passado por um servidor web com essa diretiva.

Normalmente o redirecionamento na configuração do Apache é feita com as seguintes diretivas:

```
Action php-script /cgi-bin/php
AddHandler php-script .php
```

Essa opção só foi testada com o servidor Apache, e confia no Apache para configurar a variável de ambiente não-padrão do CGI *REDIRECT_STATUS* em requisições redirecionadas. Se seu servidor web não suporta nenhuma maneira de dizer se a requisição é direta ou redirecionada, você não pode usar essa opção e você deve usar uma das outras maneiras de executar a versão do CGI documentada aqui.

Caso 3: configurando doc_root ou user_dir

Para incluir conteúdo ativo, como script e executáveis, nos diretórios de documentos do servidor é algumas vezes considerada uma prática insegura. Se, por conta de um erro de configuração, os scripts não são executados mas mostrados como documentos HTML normais, isso pode resultar em vazamento de propriedade intelectual ou informação de segurança, como senhas. Portanto, muitos administradores preferem configurar outra estrutura de diretório para scripts que são acessíveis apenas pelo CGI do PHP, e, portanto, sempre interpretados e não enviados ao navegador.

Além disso, se o método para assegurar que as requisições não são redirecionadas, como descrito na seção anterior, não estiver disponível, é necessário configurar um doc_root no script que seja diferente do diretório raiz dos documentos do servidor web.

Você pode configurar a raiz de documentos dos scripts PHP pela diretiva de configuração doc_root no arquivo de configuração, ou você pode criar a variável de ambiente *PHP_DOCUMENT_ROOT*. Se ela existir, a versão CGI do PHP sempre construirá o nome de arquivo para ser aberto com esse `doc_root` e a informação de caminho na requisição, para assegurar que nenhum script é executado fora desse diretório (exceto por `user_dir` abaixo).

Outra opção utilizável é user_dir. Quando `user_dir` for indefinido, a única coisa controlando o arquivo aberto é `doc_root`. Abrir uma URL como *http://my.host/~user/doc.php* não resulta em abrir um arquivo no diretório home de users, mas um arquivo chamado *~user/doc.php* dentro de `doc_root` (sim, um nome de diretório começando com til [~]).

Se `user_dir` estiver, por exemplo, definido para *public_php*, uma requisição como *http://my.host/~user/doc.php* abrirá o arquivo chamado *doc.php* no diretório chamado *public_php* dentro do diretório home do usuário. Se o home do usuário é */home/user*, o arquivo executado é */home/user/public_php/doc.php*.

A expansão de `user_dir` acontece independente da configuração de `doc_root`, para que você possa controlar a raiz de documentos e de acesso de diretório do usuário separadamente.

Caso 4: Interpretador do PHP fora da árvore de diretórios do servidor web

Uma maneira muito segura é colocar o interpretador do PHP em algum lugar longe dos arquivos da árvore de diretórios do servidor. Em */usr/local/bin*, por exemplo. A única desvantagem real para essa opção é que você terá que colocar uma linha similar à:

```
#!/usr/local/bin/php
```

como a primeira linha de qualquer arquivo contento tags do PHP. Você também terá que tornar o arquivo executável. Isso é, tratá-lo exatamente como você trataria qualquer script CGI escrito em Perl ou sh ou qualquer outra linguagem de script comum que usa o mecanismo de escape de shell `#!` para ser executado.

Para fazer o PHP tratar as informações de *PATH_INFO* e *PATH_TRANSLATED* corretamente com essa configuração, o interpretador do PHP deve ser compilado com a opção de configure --enable-discard-path.

Instalando como módulo do Apache

Quando o PHP é usado como módulo do Apache, ele herda as permissões do usuário do Apache (normalmente as do usuário "nobody"). Isso tem vários impactos de segurança e autorização. Por exemplo, se você estiver usando o PHP para acessar um banco de dados, a menos que o banco de dados tenha um controle de acesso interno, você terá que fazer com que o banco de dados seja acessível ao usuário "nobody". Isso significa que um script malicioso pode acessar e modificar o banco de dados, mesmo sem um usuário e senha. É possível que um web spider possa passar em uma página web de administração do banco de dados e remover todos os bancos de dados. Você pode se proteger contra isso usando autorização do Apache, ou você pode desenvolver seu modelo de acesso próprio usando LDAP, arquivos *.htaccess*, etc. e incluir esse código como parte dos seus scripts PHP.

Normalmente, uma vez que a segurança está estabelecida até esse ponto onde o usuário do PHP (no caso, o usuário apache) tem pouco risco atribuído a ele, você descobre que o PHP não tem permissão de escrita nos diretórios dos usuários. Ou talvez tenha sido proibido de acessar ou alterar bancos de dados. Também foi proibido de escrever arquivos, bons ou ruins, ou fazer transações de bancos de dados, boas ou ruins.

Um erro frequente de segurança feito até esse ponto é permitir ao Apache permissões de administrador (root), ou aumentar as habilidades do Apache de qualquer outra forma.

Aumentar as permissões do usuário do Apache para a de administrador é extremamente perigoso e pode comprometer o sistema inteiro, então executar com sudo, com chroot, ou então executar como root não deve ser considerado por aqueles que não são profissionais em segurança.

Existem algumas soluções mais simples. Usando a diretiva open_basedir pode-se controlar e restringir quais diretórios o PHP tem permissão de usar. Você também pode configurar áreas exclusivas para o Apache, restringir todas as atividades web para arquivos que não sejam de algum usuário ou do sistema.

Segurança da Sessão

É importante manter o gerenciamento de sessão HTTP seguro. A segurança relacionada a sessão está descrita na seção Segurança de Sessão da referência Módulo de Sessão.

Segurança do Sistema de Arquivos

Índice

O PHP está sujeito à segurança implementada na maioria dos sistemas de servidor com respeito às permissões de arquivos e diretórios. Isto permite controlar que arquivos no sistema podem ser lidos. É preciso ter cuidado com arquivos que são acessíveis globalmente para garantir que sejam seguros para leitura por todos os usuários que têm acesso ao sistema de arquivos.

Já que o PHP foi projetado para permitir acesso em nível de usuário ao sistema de arquivos, é totalmente possível escrever um script PHP que permitirá ler arquivos de sistema como o */etc/passwd*, modificar conexões de rede, enviar tarefas massivas de impressão, etc. Isto tem algumas implicações óbvias, já que é necessário garantir que os arquivos lidos e gravados são os apropriados.

Considere o seguinte script, onde um usuário indica que quer apagar um arquivo no seu diretório "home". Isso presume uma situação onde uma interface web PHP é usada regularmente para gerenciamento de arquivos, de forma que o usuário Apache tem permissão para apagar arquivos nos diretórios "home" dos usuários.

Exemplo #1 Verificação fraca de variáveis resulta em....

```php
<?php

// Remove um arquivo do diretório "home" do usuário
$username = $_POST['user_submitted_name'];
$userfile = $_POST['user_submitted_filename'];
$homedir  = "/home/$username";

unlink("$homedir/$userfile");

echo "O arquivo foi removido!";

?>
```

Já que o nome do usuário e do arquivo são enviados pelo formulário, um usuário pode enviar um nome de usuário e de arquivo que pertença a outra pessoa e apagá-lo, mesmo que eles não tenham permissão para fazê-lo. Neste caso, é preciso ter alguma outra forma de autenticação. Considere o que poderia acontecer se as variáveis enviadas forem `"../etc/"` e `"passwd"`. O código então leria efetivamente:

Exemplo #2 ... um ataque ao sistema de arquivos

```php
<?php

// Remove um arquivo de qualquer lugar no disco rígido que
// o usuário do PHP tenha acesso. Se o PHP tiver acesso de administrador:
$username = $_POST['user_submitted_name']; // "../etc"
$userfile = $_POST['user_submitted_filename']; // "passwd"
$homedir  = "/home/$username"; // "/home/../etc"

unlink("$homedir/$userfile"); // "/home/../etc/passwd"

echo "O arquivo foi removido!";

?>
```

Existem duas medidas importantes que devem ser tomadas para prevenir esses problemas.

- Dar permissões limitadas ao usuário web binário do PHP.
- Verificar todas as variáveis que são enviadas.

Aqui temos um script melhorado:

Exemplo #3 Verificação mais segura do nome do arquivo

```php
<?php

// Remove um arquivo do disco rígido que o
// usuário do PHP tenha acesso.
$username = $_SERVER['REMOTE_USER']; // usando um mecanismo de autenticação
$userfile = basename($_POST['user_submitted_filename']);
$homedir  = "/home/$username";

$filepath = "$homedir/$userfile";

if (file_exists($filepath) && unlink($filepath)) {
    $logstring = "Removido $filepath\n";
} else {
    $logstring = "Falha ao remover $filepath\n";
}

$fp = fopen("/home/logging/filedelete.log", "a");
fwrite($fp, $logstring);
fclose($fp);

echo htmlentities($logstring, ENT_QUOTES);

?>
```

No entanto, ele ainda possui falhas. Se o sistema de autenticação permitir que os usuários criem seus próprios logins e um usuário escolher o login `"../etc/"`, o sistema está novamente exposto. Por este motivo, é preferível escrever uma verificação mais personalizada:

Exemplo #4 Verificação mais segura do nome do arquivo

```php
<?php

$username     = $_SERVER['REMOTE_USER']; // usando um mecanismo de autenticação
$userfile     = $_POST['user_submitted_filename'];
$homedir      = "/home/$username";

$filepath     = "$homedir/$userfile";

if (!ctype_alnum($username) || !preg_match('/^(?:[a-z0-9_-]|\.(?!\.))+$/iD', $userfile)) {
    die("Usuário/arquivo inválido");
}

// etc.

?>
```

Dependendo do sistema operacional, existe uma variedade enorme de arquivos com os quais devemos nos preocupar, incluindo entradas de dispositivos (*/dev/* ou *COM1*), arquivos de configuração (*/etc/* e `.ini`), áreas de armazenamento de arquivo bem conhecidas (*/home/*, *Meus Documentos*), etc. Por este motivo, normalmente é mais fácil criar uma política onde se proíbe tudo exceto aquilo que for explicitamente permitido.

Problemas relacionados aos bytes nulos

Como o PHP usa funções do C para operações relacionadas ao sistema de arquivos, ele pode lidar com bytes nulos de maneira inesperada. Como bytes nulos denotam fim de string em C, as strings que os contêm não serão consideradas de forma inteira, mas apenas até que um byte nulo ocorra. O exemplo a seguir mostra um código vulnerável que demonstra esse problema:

Exemplo #1 Script vulnerável a bytes nulos

```php
<?php

$file = $_GET['file']; // "../../etc/passwd\0"

if (file_exists('/home/wwwrun/' . $file. '.php')) {
    // file_exists retornará true porque o arquivo /home/wwwrun/../../etc/passwd existe
    include '/home/wwwrun/' . $file . '.php';
    // o arquivo /etc/passwd será incluído

}

?>
```

Portanto, qualquer string comprometida que é usada em uma operação de sistema de arquivos deve sempre ser validada corretamente. Aqui está uma versão melhorada do exemplo anterior:

Exemplo #2 Validando a entrada corretamente

```php
<?php

$file = $_GET['file'];

// Autorizando valores possíveis
switch ($file) {
    case 'main':
    case 'foo':
    case 'bar':
        include '/home/wwwrun/include/' . $file . '.php';
        break;
    default:
        include '/home/wwwrun/include/main.php';
}

?>
```

Segurança de Bancos de Dados

Índice

Hoje em dia, bancos de dados são componentes cardinais de qualquer aplicação web permitindo que websites forneçam conteúdo dinâmico variável. Uma vez que informações muito sensíveis e/ou secretas podem ser guardadas em um banco de dados, proteger seus bancos de dados é essencial.

Para retirar ou guardar qualquer informação, você precisa conectar-se ao banco de dados, enviar uma consulta (query) legítima, pegar o resultado e fechar a conexão. Atualmente, a linguagem mais usada para consulta nessa interação é a Structured Query Language (SQL). Veja como um atacante pode manipular uma consulta SQL.

O PHP não pode proteger seu banco de dados sozinho. As seções a seguir tentam ser uma introdução básica em relação a como acessar e manipular banco de dados a partir de scripts PHP.

É preciso manter em mente essa regra simples: defesa em profundidade. Em quanto mais lugares há ações para aumentar a proteção do banco de dados, menor a probabilidade de um atacante ter sucesso em expor ou abusar de qualquer informação guardada. Uma boa modelagem do esquema (schema) do banco de dados e da aplicação para lidar com os maiores medos.

Desenhando Bancos de Dados

O primeiro passo é sempre criar o banco de dados, a não ser que você queira usar um de terceiros. Quando um banco de dados é criado, ele é atribuído a um dono, o mesmo que executou os comandos de criação. Normalmente, só o dono (ou um superusuário) pode fazer algo com os objetos naquele banco de dados, e para permitir que outros usuários usem, privilégios devem ser concedidos.

Aplicações nunca devem conectar-se ao banco de dados como seu dono ou um superusuário, porque esses usuários podem executar qualquer consulta que quiserem como, por exemplo, modificar o esquema (ex.: destruindo tabelas) ou removendo seu conteúdo completamente.

Devem ser criados usuários de bancos de dados diferentes para cada aspecto da sua aplicação com direitos bem limitados aos objetos do banco de dados. Apenas os privilégios necessários devem ser concedidos, e deve-se evitar que o mesmo usuário possa interagir com o banco de dados em casos de uso diferentes. Isso significa que se invasores conseguirem acessar seu banco de dados usando credenciais da sua aplicação, eles só podem afetar o banco tanto quanto sua aplicação poderia.

Conectando com o Banco de Dados

Deve-se estabelecer as conexões sobre SSL para criptografar as comunicações cliente/servidor para aumentar a segurança, ou você pode usar ssh para criptografar a conexão de rede entre clientes e o servidor de banco de dados. Se uma dessas opções for usada, o monitoramento do seu tráfego e obtenção de informação sobre seu banco de dados serão dificultados para um possível atacante.

Modelo de Armazenamento Criptografado

SSL/SSH protege dados transitando de um cliente para o servidor, mas não protege os dados guardados em um banco de dados. SSL é um protocolo on-the-wire.

Uma vez que o atacante ganhe acesso direto ao seu banco de dados (perpassando o servidor web), os dados armazenados podem ser expostos ou usados inadequadamente, a não ser que a informação seja protegida pelo banco em si. Criptografar os dados é uma boa maneira de diminuir essa ameaça, mas poucos bancos de dados oferecem esse tipo de criptografia de dados.

A maneira mais fácil de contornar esse problema é primeiro criar seu próprio pacote de criptografia, e então usá-lo no seus scripts PHP. O PHP pode ajudá-lo com várias extensões, tais como OpenSSL e Sodium, cobrindo uma grande variedade de algoritmos de criptografia. O script criptografa os dados antes de inseri-los no banco de dados e descriptografa quando os recupera. Veja as referências para outros exemplos de como a criptografia funciona.

Hashing

No caso de dados realmente ocultos, se sua representação bruta não for necessária (ou seja, não será exibido), o hashing deve ser levado em consideração. O exemplo conhecido de hashing é armazenar o hash criptográfico de uma senha em um banco de dados, em vez da própria senha.

As funções password fornecem uma maneira conveniente para criar o hash de dados confidenciais e trabalhar com esses hashes.

password_hash() é usado para criar o hash uma determinada string usando o algoritmo mais forte atualmente disponível e password_verify() verifica se a senha fornecida corresponde ao hash armazenado no banco de dados.

Exemplo #1 Usando campo de senha hasheado

```php
<?php

// guardando hash da senha
$query  = sprintf("INSERT INTO users(name,pwd) VALUES('%s','%s');",
            pg_escape_string($username),
            password_hash($password, PASSWORD_DEFAULT));
$result = pg_query($connection, $query);

// consultando se o usuário enviou a senha correta
$query = sprintf("SELECT pwd FROM users WHERE name='%s';",
            pg_escape_string($username));
$row = pg_fetch_assoc(pg_query($connection, $query));

if ($row && password_verify($password, $row['pwd'])) {
    echo 'Bem-vindo, ' . htmlspecialchars($username) . '!';
} else {
    echo 'Autenticação falhou para ' . htmlspecialchars($username) . '.';
}

?>
```

Injeção de SQL

A injeção de SQL é uma técnica onde o agressor explora falhas no código da aplicação responsável em criar e povoar instruções SQL. O agressor pode assim obter acesso privilegiado a partes da aplicação, extrair todos os dados do banco de dados, alterar os dados, e até mesmo executar comandos perigosos em nível do sistema onde o banco de dados roda. A falha ocorre quando desenvolvedores concatenam ou interpolam dados arbitrários em instruções SQL.

Exemplo #1 Dividindo o conjunto de resultados em páginas ... e criando super-usuários (PostgreSQL)

No exemplo a seguir, dados de usuário são diretamente interpolados na instrução SQL, permitindo ao agressor obter uma conta de superusuário no banco de dados.

```php
<?php

$offset = $_GET['offset']; // Usando os dados sem validação!
$query  = "SELECT id, name FROM products ORDER BY name LIMIT 20 OFFSET $offset;";
$result = pg_query($conn, $query);

?>
```

Usuários normais clicam nos links 'próxima' e 'anterior' onde *$offset* é codificado na URL. O script espera que o valor de *$offset* seja um número decimal. No entanto, e se alguém tentar quebrar a instrução SQL, utilizando a seguinte URL:

```
0;
insert into pg_shadow(usename,usesysid,usesuper,usecatupd,passwd)
    select 'crack', usesysid, 't','t','crack'
    from pg_shadow where usename='postgres';
--
```

Se isso acontecesse, então o script daria de presente acesso de superusuário ao atacante. Perceba que `0;` é para fornecer uma deslocamento válido para a consulta original e terminá-la.

> **Nota** :
>
> É uma técnica comum forçar o avaliador de SQL ignorar o resto da consulta escrita pelo desenvolvedor com `--`, que é o sinal de comentário no SQL.

Uma maneira de ganhar senhas é desviar suas páginas de resultado de busca. A única coisa que o atacante precisa fazer é ver se alguma variável enviada é usada em um comando SQL que não é tratado corretamente. Esses filtros podem ser configurados de forma a personalizar cláusulas `WHERE`, `ORDER BY`, `LIMIT` e `OFFSET` em cláusulas `SELECT` Se seu banco de dados suporta o construtor `UNION`, o atacante pode tentar adicionar uma consulta inteira à consulta original para listar senhas de uma tabela arbitrária. É altamente recomendável gravar apenas hashs criptográficos das senhas, ao invés de gravar a senha.

Exemplo #2 Listando artigos ... e algumas senhas (qualquer banco de dados)

```php
<?php

$query  = "SELECT id, name, inserted, size FROM products
           WHERE size = '$size'";
$result = odbc_exec($conn, $query);

?>
```

A parte estática da consulta pode ser combinada com outro comando `SELECT` que revela todas as senhas:

```
'
union select '1', concat(uname||'-'||passwd) as name, '1971-01-01', '0' from usertable;
--
```

Instruções `UPDATE` e `INSERT` também podem ser abusadas em ataques.

Exemplo #3 De recuperando uma senha ... para ganhando mais privilégios (qualquer banco de dados)

```php
<?php
$query = "UPDATE usertable SET pwd='$pwd' WHERE uid='$uid';";
?>
```

Se um usuário malicioso envia o valor `' or uid like'%admin%` para *$uid* para mudar a senha do administrador, ou simplesmente configura *$pwd* para `hehehe', trusted=100, admin='yes` (com um espaço sobrando) para ganhar mais privilégios. Então, a consulta ficará retorcida:

```php
<?php

// $uid: ' or uid like '%admin%
$query = "UPDATE usertable SET pwd='...' WHERE uid='' or uid like '%admin%';";

// $pwd: hehehe', trusted=100, admin='yes
$query = "UPDATE usertable SET pwd='hehehe', trusted=100, admin='yes' WHERE
...;";

?>
```

Pode parecer que o agressor precisa saber alguma coisa da arquitetura do banco para conduzir um ataque efetivo, mas obter esse tipo de informação é geralmente bem simples. Por exemplo, o código pode ser parte de um sistema open source e publicamente disponível. Esse tipo de informação pode também pode ser obtido em sistemas de código fechado -- mesmo no caso dele estar ofuscado ou compilado -- e mesmo através do seu próprio código, através de mensagens de erro. Outros métodos incluem o uso de nomes típicos de tabelas e colunas. Por exemplo, um formulário de login normalmente utiliza tabelas chamadas 'user' com colunas chamadas 'id', 'username', e 'password'.

Exemplo #4 Atacando o sistema de um banco de dados (MSSQL Server)

Um exemplo assustador de como comandos do sistema operacional podem ser acessados em alguns bancos de dados.

```php
<?php

$query  = "SELECT * FROM products WHERE id LIKE '%$prod%'";
$result = mssql_query($query);

?>
```

Se o atacante enviar o valor `a%' exec master..xp_cmdshell 'net user test testpass /ADD' --` para *$prod*, então *$query* terá o valor:

```php
<?php

$query  = "SELECT * FROM products
           WHERE id LIKE '%a%'
           exec master..xp_cmdshell 'net user test testpass /ADD'--";
$result = mssql_query($query);

?>
```

O MSSQL Server executa comandos SQL em um lote incluindo um comando para adicionar um novo usuário para o banco de dados de contas locais. Se essa aplicação estiver sendo executada como `sa` e o serviço MSSQLSERVER estivesse sendo executado com privilégios suficientes, o atacante teria agora uma conta com a qual poderia acessar essa máquina.

> **Nota** :
>
> A função **mssql_query()** pertencia à extensão `mssql`, que foi removida no PHP 7.0.0. Ela é usada aqui apenas para ilustrar o ataque; o código atual conectando ao MSSQL Server deve usar as extensões sqlsrv ou pdo_sqlsrv.

> **Nota** :
>
> Alguns dos exemplos acima estão ligados a bancos específicos. Isso não significa que um ataque similar é impossível contra outros produtos. Seu servidor de banco de dados pode ter uma vulnerabilidade similar de outra maneira.

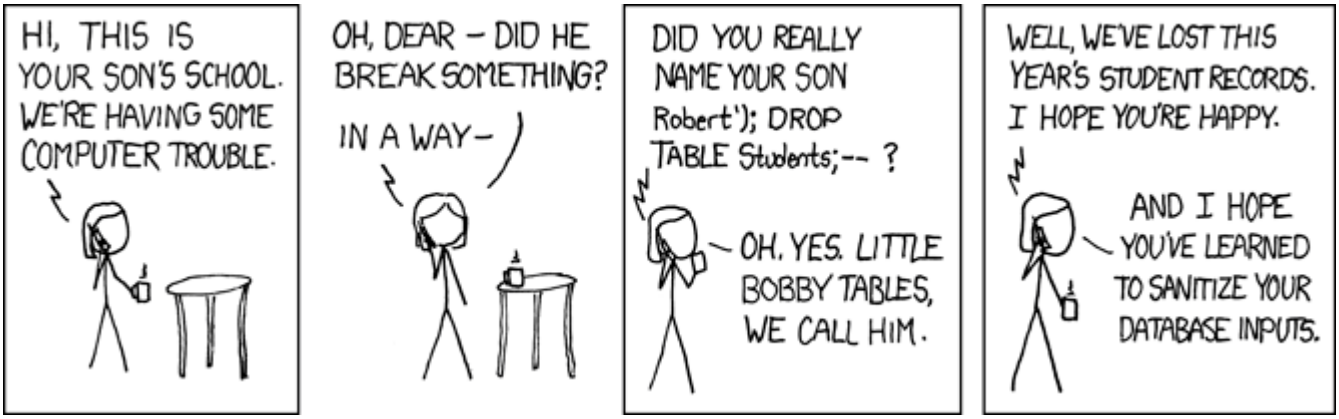


Imagem cortesia de » xkcd

Técnicas para evitar ataques

A maneira recomendada de evitar ataques de injeção de SQL é informar todos os dados em instruções preparadas. Usar instruções parametrizadas não é o suficiente para evitar SQL injection, mas é a maneira mais rápida e segura de fornecer dados a instruções SQL. Todos os dados dinâmicos em `WHERE`, `SET`, e `VALUES` precisam ser substituídos por âncoras. Os dados em si serão informados durante a execução, e serão enviados separadamente do comando SQL.

Informar dados via parâmetros deve ser utilizado apenas para dados. Outras partes dinâmicas de uma instrução SQL precisa ser filtrada por uma lista prévia e conhecida de valores válidos.

Exemplo #5 Evitando SQL injection ao utilizar instruções preparadas PDO

```php
<?php

// A parte SQL dinâmica precisa é validada a partir de dados prévios
$sortingOrder = $_GET['sortingOrder'] === 'DESC' ? 'DESC' : 'ASC';
$productId = $_GET['productId'];
// O SQL é preparado utilizando âncoras
$stmt = $pdo->prepare("SELECT * FROM products WHERE id LIKE ? ORDER BY price {$sortingOrder}");
// O valor é informado, incluindo caracteres curinga
$stmt->execute(["%{$productId}%"]);

?>
```

Instruções preparadas são fornecidas no PDO, no MySQLi, e por outras bibliotecas de bancos de dados.

Ataques de injeção de SQL são principalmente baseados na exploração de código que não é escrito pensando em segurança. Nunca confie em nenhum dado enviado pelo usuário, e menos ainda em dados enviados pelo navegador, mesmo que o dado venha de um option box ou um campo hidden, nem mesmo cookies. O primeiro exemplo mostra como uma instrução SQL muito simples pode causar um dano desastroso.

Uma estratégia de defesa envolve várias boas práticas de codificação:

- Nunca se conecte ao banco de dados utilizando um usuário administrador ou dono dos objetos do banco. Sempre utilize usuários com privilégios mínimos.
- Sempre verifique se o dado enviado tem o tipo esperado. O PHP possui várias funções de validação de dados, de coisas simples como as encontradas em funções de variável e em funções de string (por exemplo, is_numeric(), ctype_digit()) a coisas mais avançadas como suporte a expressões regulares compatíveis com Perl.
- Se a aplicação espera dados numéricos, considere verificar os dados com ctype_digit(), ou modificar os dados utilizando settype(), ou ainda reformatar o dado com sprintf().
- Se o banco de dados não suportar enviar dados por parâmetros, então é necessário escapar todos os dados de usuário não numéricos, passando o dado para funções específicas de escape do banco (por exemplo mysql_real_escape_string(), **sqlite_escape_string()**, etc). Funções genéricas como addslashes() são úteis apenas em contextos específicos (por exemplo, no MySQL é possível modificar o comportamento das aspas com *NO_BACKSLASH_ESCAPES*), então o escape específico é necessário.
- Nunca imprima nenhum dado ou erro específico do banco de dados, especialmente dados referentes a schema. Veja também exibição de erros e funções de manipulação e log de erros.

Além disso, você ganha em relatar consultas ou dentro do script ou no próprio banco de dados, se esse suportar. Obviamente, o relatório é incapaz de prevenir qualquer tentativa danosa, mas pode ser útil para ajudar a rastrear qual aplicação foi atacada. O relatório não é útil em si, mas através da informação que ele contém. Mais detalhes geralmente é melhor que menos.

Relatando Erros

Com relação a segurança, relatório de erros é uma faca de dois gumes. Pode beneficiar o aumento da segurança, ou fornecer informação ao atacante.

Uma tática padrão de ataque envolve determinar como um sistema funciona entrando dados incorretos e checando os tipos e contextos dos erros que são retornados. Isso permite que um cracker sonde por informações sobre o servidor, para determinar possíveis fraquezas. Por exemplo, se um atacantes tinha recolhido informação sobre uma página baseado em uma submissão de dados anterior, ele pode tentar sobrescrever variáveis, ou modificá-las:

Exemplo #1 Atacando variáveis com uma página HTML personalizada

```
<form method="post" action="attacktarget?username=badfoo&amp;password=badfoo">
<input type="hidden" name="username" value="badfoo" />
<input type="hidden" name="password" value="badfoo" />
</form>
```

Os erros do PHP que são retornados normalmente podem ser úteis para um desenvolvedor que está tentando depurar um script, indicando coisas como a função ou arquivo que falhou, o arquivo PHP no qual a falha ocorreu, e o número da linha de código causadora da falha. Toda essa informação pode ser explorada. Não é incomum para um desenvolvedor PHP usar show_source(), highlight_string(), ou highlight_file() como medidas de depuração, mas em um site de produção, isso pode expor variáveis ocultas, sintaxe incorreta, ou outra informações perigosas. Especialmente perigoso é rodar código de fontes conhecidas com tratadores de depuração integrados, ou usar técnicas de depuração comuns. Se o atacante pode determinar qual técnica geral você estiver usando, eles podem tentar, por força-bruta, enviar várias strings de depuração comuns para a página:

Exemplo #2 Explorando variáveis comuns de depuração

```
<form method="post" action="attacktarget?errors=Y&amp;showerrors=1&amp;debug=1">
<input type="hidden" name="errors" value="Y" />
<input type="hidden" name="showerrors" value="1" />
<input type="hidden" name="debug" value="1" />
</form>
```

Independente do método de tratamento de erros, a habilidade de sondar um sistema por erros acaba dando informações úteis a um atacante.

Por exemplo, o próprio estilo de um erro genérico do PHP indica que o sistema está rodando o PHP. Se o atacante estava procurando por uma página `.html`, e queria sondar qual o back-end (para procurar fraquezas conhecidas no sistema) enviando dados incorretos, ele pode ser capaz de determinar que um sistema foi feito com PHP.

Uma função de erro pode indicar se o sistema pode está executando um engine de banco de dados específico, ou dar dicas de como uma página foi programada ou desenhada. Isso permite uma investigação profunda sobre portas abertas de bancos de dados, ou procurar por bugs específicos ou fraquezas de uma página. Enviando diferentes pedaços de dados ruins, por exemplo, um atacante pode determinar a ordem de autenticação em um script, (a partir da linha do erro) assim como sondar por exploits que podem ser aproveitados em diferentes partes do script.

Um erro geral do PHP ou do sistema de arquivos indicam quais permissões o servidor web tem, assim como a estrutura e organização dos arquivos no servidor web. Códigos de erros escritos pelo desenvolvedor podem agravar o problema, levando à exploração fácil de informação até então "escondida".

Existem três soluções principais para esse problema. A primeira é verificar exaustivamente todas as funções, e tentar compensar pelo volume dos erros. A segunda é desabilitar completamente os relatórios de erros no código de produção. A terceira é usar as funções personalizáveis de tratamento de erro do PHP para criar seu próprio tratador de erro. Dependendo da sua política de segurança, você pode perceber que todas são aplicáveis à sua situação.

Uma maneira de perceber esse problema antes que o pior aconteça é usar a diretiva error_reporting(), para ajudar a aumentar a segurança de seu código e achar uso de variáveis que pode ser perigoso. Ao testar o seu código, antes de colocar em produção, com **`E_ALL`**, você pode rapidamente encontrar áreas onde suas variáveis podem sofrer alterações nocivas ou modificações quaisquer. Uma vez que esteja pronto para produção, você deve ou desabilitar mensagens de erro completamente configurando a diretiva error_reporting() com o valor 0, ou desligar o envio de erros usando a opção `display_errors` do arquivo *php.ini*, para evitar sondagem do seu código. Se você escolher a segunda opção, você deve também definir o caminho para o arquivo de registro usando a diretiva `error_log`, e ligar a diretiva `log_errors`.

Exemplo #3 Encontrado variáveis perigosas com E_ALL

```
<?php
if ($username) { // Não inicializado ou verificado antes do uso
    $good_login = 1;
}
if ($good_login == 1) { // Se o teste acima falhar, não inicializado ou verificado antes do uso
    readfile ("/dados/altamente/sensiveis/index.html");
}
?>
```

Dados Enviados pelo Usuário

A maior fraqueza na maioria dos programas PHP não é inerente à linguagem em si, mas meramente um problema de código escrito desconsiderando a segurança. Por essa razão, você sempre deve investir um pouco de tempo considerando as implicações de um certo pedaço de código, para ter certeza que o dano possível se uma variável não esperada for submetida ao mesmo.

Exemplo #1 Uso Perigoso de Variáveis

```
<?php
// remove um arquivo do diretório home do usuário... ou talvez
// de outra pessoa?
unlink ($evil_var);

// Escreve registro do acesso... ou talvez uma entrada em /etc/passwd?
fwrite ($fp, $evil_var);

// Executa algo trivial... ou rm -rf *?
system ($evil_var);
exec ($evil_var);

?>
```

Você sempre deve examinar cuidadosamente seu código para se assegurar que quaisquer variáveis sendo enviadas do navegador web estão sendo checadas de maneira correta, e faz a si mesmo as seguintes perguntas:

- Seu script só afetará os arquivos desejados?
- Dados incomuns ou indesejados podem ser utilizados?
- Esse script pode ser usado de maneiras não intencionadas?
- Ele pode ser usado em conjunto com outros scripts de maneira negativa?
- As transações serão registradas adequadamente?

Respondendo essas perguntas adequadamente enquanto escrevendo o script, ao invés de depois, previne a reescrita indesejada quando você precisar aumentar a segurança. Começando com essa linha de raciocínio, você não garante a segurança do seu sistema, mas pode ajudar a aumentá-la.

Melhore a segurança desabilitando configurações de conveniência que obscurecem a origem, validade ou integridade dos dados de entrada. A criação implícita de variáveise a entrada não verificada podem levar a

vulnerabilidades como ataques de injeção e manipulação de dados.

Recursos como `register_globals` e `magic_quotes` (ambos removidos no PHP 5.4.0) contribuíram para esses riscos criando variáveis automaticamente a partir da entrada do usuário e escapando dados inconsistentemente. Embora não esteja mais no PHP, riscos semelhantes persistem se o tratamento de entrada for mal gerenciado.

Habilite error_reporting(E_ALL) para ajudar a detectar variáveis não inicializadas e validar a entrada. Use tipos estritos (declare(strict_types=1), introduzido no PHP 7) para reforçar a segurança de tipos, evitar conversões de tipos não intencionais e melhorar a segurança geral.

Escondendo o PHP

Em geral, segurança por obscuridade é uma das maneiras mais fracas de segurança. Mas em alguns casos, cada pequena medida de segurança extra é desejável.

Algumas técnicas simples podem ajudar a esconder o PHP, possivelmente retardando um atacante que está tentando descobrir fraquezas no seu sistema. Configurar a diretiva expose_php para `off` no arquivo *php.ini*, reduz a quantidade de informação disponível aos atacantes.

Outra tática é configurar o servidor web, como o Apache, para executar tipos de arquivos diferentes pelo PHP, ou por uma diretiva de arquivo *.htaccess*, ou no próprio arquivo de configuração do Apache. É possível usar extensões de arquivos como disfarce:

Exemplo #1 Escondendo o PHP como outra linguagem

```
# Fazer código PHP parecer com outros tipos de códigos
AddType application/x-httpd-php .asp .py .pl
```

Ou obscurecer completamente:

Exemplo #2 Usando extensões desconhecidas para o PHP

```
# Fazer código PHP parecer com tipos desconhecidos
AddType application/x-httpd-php .bop .foo .133t
```

Ou esconder ele como código HTML, o que tem uma pequena queda de performance porque todo HTML será avaliado pelo engine do PHP:

Exemplo #3 Usando extensão HTML para arquivos PHP

```
# Fazer código PHP parecer com HTML
AddType application/x-httpd-php .htm .html
```

Para isso funcionar efetivamente, deve-se renomear os arquivos PHP com as extensões acima. Embora seja uma forma de segurança por obscuridade, é uma pequena medida preventiva e com poucos custos.

Mantendo-se Atualizado

PHP, como qualquer outro sistema grande, está sob constante revisão e melhoramento. Cada versão nova normalmente incluirá mudanças, sejam grandes ou pequenas, para melhorar a segurança e reparar falhas, erros de configuração, e outros problemas que podem afetar a segurança geral e estabilidade do seu sistema.

Como outras linguagens de script e programas de nível de sistema, a melhor política é atualizar frequentemente e manter-se atento às novas versões e suas mudanças.

Características

Autenticação HTTP com PHP

Pode-se utilizar a função header() para enviar uma mensagem de `"Authentication Required"` para o navegador cliente, causando o aparecimento de uma janela para a entrada de Nome de Usuário/Senha. Uma vez que o usuário preencha um nome de usuário e uma senha, a URL contendo o script PHP será chamada mais uma vez com as variáveis predefinidas *PHP_AUTH_USER*, *PHP_AUTH_PW*, e *AUTH_TYPE* para determinar o nome de usuário, senha e tipo da autenticação, respectivamente. Estas variáveis predefinidas são encontradas no array *$_SERVER*. *Somente* o método de autenticação "Basic" é suportado. Consulte a função header() para mais informações.

Um exemplo de fragmento de script que forçaria a autenticação do cliente em uma página:

Exemplo #1 Exemplo de Autenticação HTTP "Basic"

```php
<?php
if (!isset($_SERVER['PHP_AUTH_USER'])) {
    header('HTTP/1.1 401 Unauthorized');
    header('WWW-Authenticate: Basic realm="My Realm"');
    echo 'Texto enviado caso o usuário clique no botão Cancelar';
    exit;
} else {
    echo "<p>Olá, {$_SERVER['PHP_AUTH_USER']}.</p>";
    echo "<p>Você digitou {$_SERVER['PHP_AUTH_PW']} como sua senha.</p>";
}
?>
```

> **Nota** : **Compatibilidade**
>
> Por favor, seja cauteloso ao codificar as linhas do cabeçalho HTTP. Para garantir máxima compatibilidade com todos os clientes, a palavra-chave "Basic" deve ser escrita com um "B" maiúsculo, a string realm deve ser envolvida por aspas duplas (nunca por aspas simples), e apenas um espaço deve preceder o código *401* na linha do cabeçalho *HTTP/1.1 401*. Parâmetros de autenticação precisam ser separados por vírgula.

Ao invés de simplesmente exibir *PHP_AUTH_USER* e *PHP_AUTH_PW*, como no exemplo acima, talvez queira verificar a validade do nome de usuário e senha. Talvez enviando uma consulta a um banco de dados, ou procurando o usuário em um arquivo dbm.

> **Nota** : **Configuração do Apache**
>
> O PHP usa uma diretiva chamada `AuthType` para determinar se a autenticação externa está em efeito.

Perceba, entretanto, que a diretiva citada acima não previne que alguém que controle uma URL não autenticada roube senhas de URLs autenticadas no mesmo servidor.

> **Nota** : **Comportamento do navegador**
>
> A autenticação básica HTTP é realmente básica e não foi projetada para suportar logouts. Como o HTTP é um protocolo sem estado, a maioria dos navegadores armazena em cache as credenciais fornecidas assim que um código de status `2xx` é detectado, e as envia em todas as requisições, até que o navegador seja fechado. Não há uma maneira definida para um servidor solicitar uma nova solicitação de credenciais. Ao longo dos anos, várias soluções alternativas para isso se espalharam como conselhos na internet, mas todas dependem de como diferentes navegadores optaram por lidar com casos extremos indefinidos (ou mesmo violações do padrão HTTP). É melhor evitar essas soluções alternativas e não usar a autenticação básica para nada sério.

> **Nota** : **Configuração do IIS**
>
> Para que a autenticação HTTP funcione no servidor IIS com a versão CGI do PHP, a diretiva php.ini cgi.rfc2616_headers deve ser definida como `0` (o valor padrão) e a configuração do IIS em "`Directory Security`" (ou "Segurança do Diretório" / "Autenticação") deve ser editada. Clique em "`Edit`" (ou "Editar") e marque apenas "`Anonymous Access`" (ou "Acesso Anônimo"), deixando todos os outros campos desmarcados.

Cookies

O PHP suporta de forma transparente cookies HTTP. Cookies são um mecanismo para armazenar dados no navegador remoto e assim rastrear ou identificar usuários que retornam. Você pode criar cookies usando as funções setcookie() ou setrawcookie(). Cookies fazem parte do cabeçalho HTTP, logo, setcookie() deve ser chamada antes que qualquer saída seja enviada ao navegador. Esta é a mesma limitação que a função header() tem. Você pode usar as funções de buffering de saída para atrasar as impressões no script até que você tenha decidido se vai ou não configurar qualquer cookie ou enviar quaisquer cabeçalhos.

Qualquer cookie enviado para o servidor pelo cliente será automaticamente incluído no array auto-global *$_COOKIE* se variables_order contém "C". Se você deseja definir vários valores em um único cookie, simplesmente acrescente `[]` ao nome do cookie.

Para mais detalhes, incluindo comentários sobre problemas de browsers, veja as funções setcookie() e setrawcookie().

Sessões

Suporte a sessões no PHP consiste de uma maneira de preservar dados através de acessos subsequentes. Isso permite a criação de aplicações mais personalizadas e aumenta o apelo do seu web site. Todas as informações estão na seção Session reference.

Gerenciamento de envio de arquivos

Índice

Envio de arquivos com o método POST

Este recurso permite realizar envios de arquivos de texto e binários. Com as funções de autenticação e manipulação de arquivos do PHP, você tem o controle completo de quem pode fazer o envio e o que deve ser feito com o arquivo após o envio estar completo.

O PHP é capaz de receber envios de arquivos de qualquer navegador compatível com RFC-1867.

> **Nota** : **Nota Sobre Configurações Relacionadas**
>
> Veja também as diretivas file_uploads, upload_max_filesize, upload_tmp_dir, post_max_size, max_input_time no *php.ini*

O PHP também suporta o método PUT para envio de arquivos como o usado pelo Netscape Composer e clientes para Amaya W3C. Veja Suporte ao Método PUT para maiores detalhes.

Exemplo #1 Formulário para Envio de Arquivo

Uma interface para envio de arquivo pode ser criada com um formulário especial parecido com este:

```
<!-- O tipo de codificação de dados, enctype, DEVE ser especificado abaixo -->
<form enctype="multipart/form-data" action="__URL__" method="POST">
    <!-- MAX_FILE_SIZE deve preceder o campo input -->
    <input type="hidden" name="MAX_FILE_SIZE" value="30000" />
    <!-- O nome do elemento input determina o nome do array $_FILES -->
    Enviar esse arquivo: <input name="userfile" type="file" />
    <input type="submit" value="Enviar arquivo" />
</form>
```

O parâmetro `__URL__` no exemplo acima deve ser substituído e apontar para um arquivo PHP.

O campo escondido (input type="hidden") `MAX_FILE_SIZE` (medido em bytes) deve preceder o campo de entrada do arquivo (input type="file"), e o seu valor é o tamanho limite aceito pelo PHP. Este elemento do formulário deve sempre ser usado para evitar que usuários tenham o problema de ter que esperar pela transferência de um arquivo para só então descobrir que ele era grande demais e a transferência falhar. Fique ciente: é muito simples burlar esse parâmetro pelo lado do navegador, portanto nunca dependa exclusivamente desse recurso para bloquear arquivos com maior tamanho. Isso é um recurso meramente conveniente para usuários da aplicação no lado do cliente. Os parâmetros do PHP (no servidor) para "maximum-size", no entanto, não podem ser burlados.

> **Nota** :
>
> Tenha certeza que seu formulário de envio tenha o atributo `enctype="multipart/form-data"` do contrário o envio não irá funcionar.

A variável global *$_FILES* conterá toda a informação do arquivo enviado. O conteúdo do formulário de exemplo acima será como o exemplo descrito abaixo. Note que isso assume o nome do arquivo enviado *userfile*, como foi usado no script de exemplo acima, contudo isso pode ter qualquer nome.

$_FILES['userfile']['name']

O nome original do arquivo na máquina do cliente.

$_FILES['userfile']['type']

O tipo mime do arquivo, se o navegador fornecer essa informação. Um exemplo poderia ser `"image/gif"`. O tipo mime não é verificado pelo PHP portanto não considere que esse valor será concedido.

$_FILES['userfile']['size']

O tamanho, em bytes, do arquivo enviado.

$_FILES['userfile']['tmp_name']

O nome temporário com o qual o arquivo enviado foi armazenado no servidor.

$_FILES['userfile']['error']

O código de erro associado a esse envio de arquivo.

$_FILES['userfile']['full_path']

O caminho completo conforme enviado pelo navegador. Esse valor nem sempre contém uma estrutura de diretórios real, e não se pode confiar nele. Disponível a partir do PHP 8.1.0.

Os arquivos serão guardados por padrão no diretório temporário do servidor, a menos que outro local seja especificado através da diretiva upload_tmp_dir no *php.ini*. O diretório padrão do servidor pode ser alterado através da configuração da variável de ambiente *TMPDIR* no ambiente onde o PHP está sendo executado. Configurando isso usando putenv() em um script PHP não irá funcionar. Esta variável de ambiente também pode ser usada para certificar que outras operações também estão funcionando nos arquivos enviados.

Exemplo #2 Validando o envio de arquivos

Veja também as funções is_uploaded_file() e move_uploaded_file() para maiores informações. O exemplo a seguir irá processar o envio de um arquivo que vem de um formulário.

```php
<?php
$uploaddir = '/var/www/uploads/';
$uploadfile = $uploaddir . basename($_FILES['userfile']['name']);

echo '<pre>';
if (move_uploaded_file($_FILES['userfile']['tmp_name'], $uploadfile)) {
    echo "Arquivo válido e enviado com sucesso.\n";
} else {
    echo "Possível ataque de upload de arquivo!\n";
}

echo 'Aqui está mais informações de debug:';
print_r($_FILES);

print "</pre>";

?>
```

O script PHP que recebe o arquivo enviado deve implementar qualquer lógica que for necessária para determinar o que deve ser feito com o arquivo enviado. Você pode, por exemplo, usar a variável *$_FILES['userfile']['size']* para descartar qualquer arquivo que seja muito pequeno ou muito grande. Você pode

usar a variável *$_FILES['userfile']['type']* para descartar arquivos incompatíveis com um determinado critério, mas use isso apenas como a primeira de uma série de verificações, porque esse valor está totalmente sob o controle do cliente e não é verificado pelo PHP. Você também pode usar *$_FILES['userfile']['error']* e planejar sua lógica de acordo com os Códigos de erro. Independente da lógica, você também deve apagar o arquivo do diretório temporário ou movê-lo para outro local.

Se nenhum arquivo for selecionado em seu formulário, o PHP irá retornar *$_FILES['userfile']['size']* como 0, e *$_FILES['userfile']['tmp_name']* como "none" (nenhum).

O arquivo será excluído do diretório temporário ao fim da requisição se ele não for movido ou renomeado.

Exemplo #3 Enviando um array de arquivos

O PHP suporta array HTML mesmo com arquivos.

```
<form action="" method="post" enctype="multipart/form-data">
<p>Imagens:
<input type="file" name="pictures[]" />
<input type="file" name="pictures[]" />
<input type="file" name="pictures[]" />
<input type="submit" value="Enviar" />
</p>
</form>
```

```
<?php
foreach ($_FILES["pictures"]["error"] as $key => $error) {
    if ($error == UPLOAD_ERR_OK) {
        $tmp_name = $_FILES["pictures"]["tmp_name"][$key];
        // basename() pode prevenir ataques de percorrer o sistema de arquivos,
        // mas mais validações são necessárias
        $name = basename($_FILES["pictures"]["name"][$key]);
        move_uploaded_file($tmp_name, "data/$name");
    }
}
?>
```

O progresso do envio de arquivos pode ser implementado usando a Sessão de Progresso de Envio.

Explicando mensagens de erro

O PHP retorna um código de erro apropriado na array do arquivo. O código de erro pode ser encontrado no segmento `error` da array que é criada pelo PHP durante o envio do arquivo. Em outras palavras, o erro pode ser encontrado em *$_FILES['userfile']['error']*.

O valor desse código de erro é uma das constantes **`UPLOAD_ERR_*`**.

Problemas Comuns

O item `MAX_FILE_SIZE` não pode assumir um tamanho de arquivo maior que o tamanho de arquivo configurado em upload_max_filesize no arquivo *php.ini*. O valor padrão é de 2 megabytes.

Se um limite de memória está ativo, um valor maior para o limite de memória no parâmetro memory_limit pode ser necessário. Tenha certeza de estabelecer um limite de memória grande o suficiente através do parâmetro memory_limit.

Se o tempo máximo de execução definido através do parâmetro max_execution_time for muito pequeno, a execução do script pode ultrapassar o limite de tempo. Tenha certeza de estabelecer um tempo máximo de execução grande o suficiente através do parâmetro `max_execution_time`.

> **Nota** :
>
> O parâmetro max_execution_time somente afeta o tempo de execução do script em si. Qualquer tempo gasto com atividades que aconteçam fora da execução do script como chamadas de sistema usando system(), a função sleep(), pesquisas em banco de dados, tempo gasto pelo processo de enviar um arquivo, etc. não são considerados no momento de determinar o limite de tempo máximo de um script em execução.

> **Aviso**
>
> O parâmetro max_input_time define o tempo máximo, em segundos, que é permitido ao script a receber entradas, isto inclui envio de arquivos. Para um arquivo grande, múltiplos arquivos ou usuários com conexões lentas, o padrão de `60` segundos pode ser ultrapassado.

Se o parâmetro post_max_size for muito pequeno, arquivos grandes podem não ser carregados. Tenha certeza de definir o parâmetro `post_max_size` grande o suficiente.

O parâmetro max_file_uploads controla o número máximo de arquivos que podem ser enviados em uma única requisição. Caso sejam enviados mais arquivos que o definido neste limite, então *$_FILES* interrompe o processamento de arquivos uma vez que o limite tenha sido atingido. Por exemplo, se o parâmetro max_file_uploads está definido como `10`, então *$_FILES* nunca possuirá mais de 10 itens.

Não validar o arquivo que você está operando pode permitir que os usuários acessem informações sensíveis em outros diretórios.

Devido ao grande número de estilos de listagem de diretórios não podemos garantir que arquivos com nomes exóticos (como os que contém espaços) sejam manuseados corretamente.

Um desenvolvedor não deve misturar campos comuns de entrada `input` e campos de envio de arquivo na mesma variável de formulário (usando um nome de campo `input` como `foo[]`).

Enviando múltiplos arquivos

Múltiplos arquivos podem ser enviados usando diferentes nomes (`name`) para entradas (`input`).

Também é possível carregar vários arquivos simultaneamente e ter a informação automaticamente organizada em arrays. Para isso, é necessário usar a mesma sintaxe das arrays submetidas pelo formulário HTML que você usa para múltiplos selects e checkboxes:

Exemplo #1 Carregando múltiplos arquivos

```
<form action="file-upload.php" method="post" enctype="multipart/form-data">
  Envie esses arquivos:<br />
  <input name="userfile[]" type="file" /><br />
  <input name="userfile[]" type="file" /><br />
  <input type="submit" value="Enviar arquivos" />
</form>
```

Quando o formulário acima é submetido, os arrays *$_FILES['userfile']*, *$_FILES['userfile']['name']*, e *$_FILES['userfile']['size']* serão inicializados.

Por exemplo, suponha os nomes dos arquivos */home/test/review.html* e */home/test/xwp.out*. Neste caso, *$_FILES['userfile']['name'][0]* deve conter o valor *review.html*, e *$_FILES['userfile']['name'][1]* deve conter o valor *xwp.out*. Similarmente, *$_FILES['userfile']['size'][0]* deve conter o tamanho do arquivo *review.html*, e assim por diante.

$_FILES['userfile']['name'][0], *$_FILES['userfile']['tmp_name'][0]*, *$_FILES['userfile']['size'][0]*, e *$_FILES['userfile']['type'][0]* também são definidas.

> **Aviso**
>
> O parâmetro max_file_uploads atua como um limite no número de arquivos que podem ser enviados em uma única requisição. Você precisa ter certeza que seu formulário não tentará enviar mais arquivos que o limite definido.

Exemplo #2 Carregando um diretório inteiro

Nos campos de carregamento de arquivos HTML, é possível carregar um diretório inteiro com o atributo `webkitdirectory`. Esse recurso é suportado na maioria dos navegadores modernos.

Com a informação `full_path`, é possível armazenar os caminhos relativos, ou reconstruir o mesmo diretório no servidor.

```
<form action="file-upload.php" method="post" enctype="multipart/form-data">
  Enviar este diretório:<br />
  <input name="userfile[]" type="file" webkitdirectory multiple />
  <input type="submit" value="Enviar arquivos" />
</form>
```

> **Aviso**
>
> O atributo `webkitdirectory` não é padronizado e não está no roteiro de padronização. Não utilize-o em sites em produção voltados para a Web: ele não funcionará para todos os usuários. Também podem haver grandes incompatibilidades entre implementações e o comportamento pode mudar no futuro.
>
> O PHP analisa apenas a informação de caminho relativo enviada pelo navegador/user-agent, e passa essa informação para o array *$_FILES*. Não há garantia de que os valores no array `full_path` contenham uma estrutura de diretórios real, e a aplicação PHP não deve confiar nesta informação.

Suporte ao método PUT

O PHP oferece suporte para o método HTTP PUT utilizado por alguns clientes para armazenar arquivos em um servidor. Requisições PUT são muito mais simples que o envio de arquivos usando requisições POST, elas se parecem com algo assim:

```
PUT /path/filename.html HTTP/1.1
```

Isso normalmente significaria que o cliente remoto gostaria de salvar o seguinte conteúdo: */path/filename.html* em seu diretório web. É óbvio não se tratar de uma boa ideia que o Apache ou o PHP

permitam automaticamente que todos sobrescrevam qualquer arquivo em seu diretório web. Então, para considerar isso como uma requisição é necessário dizer ao seu servidor web que você quer que um determinado script PHP cuide dessa requisição. Para fazer isso no Apache utilize a diretiva *Script*. Ela pode ser colocada quase em qualquer local de seu arquivo de configuração do Apache. Um local comum é dentro de um bloco `<Directory>` ou talvez dentro de um bloco `<VirtualHost>`. Uma linha como a seguinte deve funcionar:

```
Script PUT /put.php
```

Isto diz ao Apache para enviar todas as requisições PUT para as URIs que combinem com o contexto dessa linha para o script *put.php*. Isto assume que o PHP esteja ativo e permite extensões *.php*. O destino para todas as requisições PUT para esse script deve ser o próprio script, não o nome do arquivo enviado.

Com o PHP pode ser feito algo como se segue no script put.php. Isso copia o conteúdo de um arquivo enviado para o arquivo *myputfile.ext* no servidor. Provavelmente é necessário a realização de algumas verificações e/ou autenticação de usuário antes de realizar essa cópia.

Exemplo #1 Salvando arquivos HTTP PUT

```php
<?php
/* PUT data vem do fluxo stdin */
$putdata = fopen("php://input", "r");

/* Abre um arquivo para escrita */
$fp = fopen("myputfile.ext", "w");

/* Lê os dados 1KB de cada vez
   e escreve no arquivo */
while ($data = fread($putdata,1024))
  fwrite($fp,$data);

/* Fecha os fluxos */
fclose($fp);
fclose($putdata);
?>
```

Veja também

- Segurança do sistema de arquivos

Usando arquivos remotos

Enquanto **allow_url_fopen** estiver disponível no arquivo *php.ini*, você pode usar URLs HTTP e FTP com a maioria das funções que recebem um nome de arquivo como parâmetro. Além disso, URLs podem ser usadas com as funções include, include_once, require e require_once (**allow_url_include** precisa estar habilitado para isto). Veja Protocolos e empacotadores suportados para mais informações sobre protocolos suportados pelo PHP.

Por exemplo, você pode usar isso para abrir um arquivo em um web server remoto, avaliar a saída para a informação que você precisa, e então usar a informação em uma query de banco de dados, ou simplesmente mostrar em um estilo que combine com o resto do seu website.

Exemplo #1 Pegando o título de uma página remota

```php
<?php
$file = fopen ("http://www.example.com/", "r");
if (!$file) {
    echo "<p>Incapaz de abrir arquivo remoto.\n";
    exit;
}
while (!feof ($file)) {
    $line = fgets ($file, 1024);
    /* Isso só funciona se o título e suas tags estiverem na mesma linha */
    if (preg_match ("@\<title\>(.*)\</title\>@i", $line, $out)) {
        $title = $out[1];
        break;
    }
}
fclose($file);
?>
```

Você também pode escrever arquivos em um servidor FTP (presumindo que você se conectou como um usuário com os direitos de acesso corretos). Você só pode criar arquivos novos usando esse método. Se você tentar sobrescrever um arquivo que já existe, a chamada para fopen() falhará.

Para conectar como um usuário diferente de 'anonymous', você precisa especificar o nome de usuário (e possivelmente a senha) dentro da URL, como '`ftp://user:password@ftp.example.com/path/to/file`'. (Você pode usar o mesmo tipo de sintaxe para acessar arquivos via HTTP quando eles requerem autenticação Basic).

Exemplo #2 Guardando informação em um servidor remoto

```php
<?php
$file = fopen ("ftp://ftp.example.com/incoming/outputfile", "w");
if (!$file) {
    echo "<p>Incapaz de abrir arquivo remoto para escrita.\n";
    exit;
}
/* Escreva informações aqui. */
fwrite ($file, $_SERVER['HTTP_USER_AGENT'] . "\n");
fclose ($file);
?>
```

> **Nota** :
>
> Você talvez tenha tido a idéia, pelo exemplo acima, de usar essa técnica para escrever para um arquivo de log remoto. Infelizmente isso não funcionaria porque a chamada a fopen() falhará se o arquivo remoto já existir. Para fazer logs distribuídos dessa maneira, você deve dar uma olhada na função syslog().

Tratamento de Conexões

O estado de uma conexão é mantido internamente no PHP. Existem 4 estados possíveis:

- 0 - NORMAL

- 1 - ABORTADO
- 2 - EXPIRADO
- 3 - ABORTADO e EXPIRADO

Quando um script PHP está sendo executado normalmente, o estado NORMAL está ativo. Se o cliente remoto desconectar, o estado ABORTADO é ativado. Uma desconexão do cliente remoto é normalmente causada pelo usuário pressionando o botão de parada. Se o tempo limite imposto pelo PHP (veja set_time_limit()) é alcançado, o estado EXPIRADO é ativado.

É possível decidir se a desconexão do cliente cause ou não uma interrupção no script. As vezes é útil deixar o script em execução até que seja concluído mesmo se não houver nenhum navegador remoto recebendo a saída. O comportamento padrão, no entanto, é a interrupção do script quando o cliente remoto se desconecta. Esse comportamento pode ser configurado através da diretiva ignore_user_abort do *php.ini*, assim como pela diretiva correspondente `php_value ignore_user_abort` no *httpd.conf* do Apache ou com a função ignore_user_abort(). Se o PHP não for instruído a ignorar o aborto por parte do usuário e ele abortar, o script será encerrado. A única exceção é se uma função de finalização tiver sido registrada usando register_shutdown_function(). Com uma função de finalização, quando um usuário remoto clica no botão de parada, a próxima vez que o script tentar gerar alguma saída, o PHP detectará que a conexão foi cancelada e a função de finalização é chamada. Essa função de finalização também será chamada no fim do script mesmo que ele termine normalmente, então para fazer algo diferente caso o cliente desconecte, a função connection_aborted() pode ser usada. Essa função retorna `true` se a conexão foi abortada.

O script também pode ser finalizado pelo temporizador interno. O tempo limite padrão é de 30 segundos. Pode ser alterado usando a diretiva **max_execution_time** do *php.ini* ou a diretiva correspondente no *httpd.conf* do Apache `php_value max_execution_time` assim como com a função set_time_limit(). Quando o temporizador chega ao limite, o script será abortado e assim como o caso acima de desconexão pelo cliente, se uma função de finalização foi registrada, ela será chamada. Dentro da função de finalização, pode ser verificado se a causa da finalização foi tempo limite excedido chamando-se a função connection_status(). Essa função retorna 2 se a causa foi tempo limite excedido.

Importante notar que ambos os estados ABORTADO e EXPIRADO podem estar ativos ao mesmo tempo. Isso é possível se o PHP for instruído a ignorar o aborto causado pelo usuário. O PHP continuará a perceber que o usuário pode ter quebrado a conexão, mas o script continuará sendo executado. Se o tempo limite for alcançado, ele será abortado e sua função de finalização, se existente, será chamada. Nesse ponto, a função connection_status() retornará 3.

Conexões Persistentes com o Banco de Dados

O que são Conexões Persistentes?

Conexões persistentes são conexões que não fecham quando a execução do script termina. Quando uma conexão persistente é requisitada, o PHP verifica se uma conexão persistente idêntica (que foi mantida aberta anteriormente) já existe; se existir, ela é reutilizada, senão, uma nova conexão é criada. Uma conexão 'idêntica' é uma aberta para o mesmo host com o mesmo nome de usuário e a mesma senha (onde for aplicável).

Não há nenhuma maneira de solicitar uma conexão específica ou de garantir que a conexão retornada sera uma existente ou uma nova (se todas as conexões existentes estiverem em uso ou se a solicitação estiver sendo atendida por um trabalhador diferente, que tem um conjunto separado de conexões).

Portanto, conexões persistentes do PHP não podem ser usadas para, por exemplo:

- atribuir uma sessão de banco de dados específica a um usuário da web específico
- criar uma grande transação em várias solicitações
- iniciar uma consulta em uma solicitação e coletar os resultados em outra

Conexões persistentes não fornecem *nenhuma* funcionalidade que não seria possível com conexões não persistentes.

Solicitações da Web

Existem duas maneiras pelas quais um servidor web pode utilizar o PHP para gerar páginas web:

O primeiro método é usar o PHP como um CGI "wrapper". Quando executado dessa maneira, uma instância do interpretador do PHP é criada e destruída para cada requisição de página (para uma página PHP) para o servidor web. Como ela é destruída após cada requisição, quaisquer recursos que ela adquirir (como uma conexão para um servidor de banco de dados SQL) são fechados quando ela é destruída. Nesse caso, não há ganho em usar conexões persistentes - elas simplesmente não persistem.

O segundo método, e o mais popular, é rodar o PHP-FPM ou o PHP como um módulo em um servidor web com multi-processos (atualmente somente o Apache). Essas configurações têm tipicamnente um processo (o pai) que coordena uma série de processos (seus filhos) que realmente fazem o trabalho de servir as páginas web. Quando uma requisição chega de um cliente, ela é entregue à um dos filhos que já não estiver servindo outro cliente. Isso significa que quando o mesmo cliente faz uma segunda requisição para o servidor, ele pode ser atendido por um processo filho diferente do que da primeira vez. Uma vez que uma conexão persistente tenha sido aberta, cada página subsequente servida pelo mesmo processo filho pode reutilizar a conexão já estabelecida ao servidor SQL.

> **Nota** :
>
> O método em uso pode ser consultado verificando o valor de "Server API" na saída de phpinfo() ou o valor de `PHP_SAPI`, executado a partir de uma solicitação web.
>
> Se a API do Servidor for "Apache 2 Handler" ou "FPM/FastCGI", as conexões persistentes serão usadas em todas as solicitações atendidas pelo mesmo trabalhador. Para qualquer outro valor, as conexões persistentes não persistirão após cada solicitação.

Command-line Processes

Como a linha de comando PHP usa um novo processo para cada script, conexões persistentes não são compartilhadas entre scripts de linha de comando, portanto, não há nenhum valor em usá-las em scripts transitórios, como crons ou comandos. No entanto, elas podem ser úteis, por exemplo, em um servidor de aplicação de longa duração que atende a muitas solicitações ou tarefas, onde cada uma pode precisar de sua própria conexão com o banco de dados.

Por que usá-las?

Conexões persistentes são benéficas quando a sobrecarga para criar uma conexão ao servidor SQL é alta. Se essa sobrecarga é realmente significativa depende de vários fatores como o tipo de banco de dados, se reside na mesma máquina do servidor web e o quão carregada está a máquina. Quando a sobrecarga de conexão é alta, conexões persistentes podem ajudar consideravelmente: cada processo filho se conecta uma vez só durante toda sua duração, e não a cada vez que ele processa uma página que requer uma conexão ao servidor SQL. Isso significa que cada filho que abre uma conexão persistente manterá sua própria conexão

persistente ao servidor. Por exemplo, se cada um de 20 processos filhos diferentes rodarem um script que faz uma conexão persistente ao servidor SQL, haverá 20 conexões separadas a esse servidor, uma de cada filho.

Possíveis desvantagens: limites de conexão

Perceba, no entanto, que isso pode ter desvantagens ao usar um banco de dados com limite de conexões que são excedidas pela conexões persistentes dos filhos. Se o banco de dados tem um limite de 16 conexões simultâneas e durante uma sessão movimentada do servidor 17 processos filhos tentarem se conectar, um deles falhará. Se houver bugs nos scripts que não permitem que as conexões se fechem (como loops infinitos), um banco de dados com apenas 16 conexões pode rapidamente ficar atolado.

Conexões persistentes geralmente aumentam o número de conexões abertas a qualquer momento, pois os trabalhadores ociosos ainda manterão as conexões para as solicitações anteriores. Se um grande número de trabalhadores for criado para lidar com um pico de solicitações, as conexões que eles abriram permanecerão até que o trabalhador seja encerrado ou que o servidor de banco de dados feche a conexão.

Certifique-se de que o número máximo de conexões permitidas pelo servidor de banco de dados seja maior que o número máximo de trabalhadores de solicitação da web (além de qualquer outro uso, como crons ou conexões administrativas).

Consulte a documentação do banco de dados para obter informações sobre como lidar com conexões abandonadas ou ociosas (tempo limite). Tempo limite longo pode aumentar significativamente o número de conexões persistentes abertas ao mesmo tempo.

Possíveis desvantagens: manter o estado de conexão

Algumas extensões de banco de dados realizam limpeza automática quando a conexão é reutilizada; outras deixam essa tarefa a critério do desenvolvedor da aplicação. Dependendo da extensão de banco de dados escolhida e do projeto da aplicação, a limpeza manual pode ser necessária antes da saída do script. Alterações que podem deixar conexões em um estado inesperado incluem:

- Banco de dados selecionado / padrão
- Bloqueio de tabelas
- Transações não confirmadas
- Tabelas temporárias
- Configurações ou recursos específicos de conexão, como perfilagem

Bloqueios de tabela e transações que não são limpos ou fechados podem fazer com que outras consultas sejam bloqueadas indefinidamente e/ou causar a reutilização subsequente da conexão, causando alterações inesperadas.

Selecionar o banco de dados errado fará com que a reutilização subsequente da conexão não consiga executar as consultas conforme o esperado (ou executá-las no banco de dados errado se os esquemas forem semelhantes o suficiente).

Se as tabelas temporárias não forem limpas, as solicitações subsequentes não conseguirão recriar a mesma tabela.

Pode-se implementar a limpeza usando destruidores de classe ou register_shutdown_function(). Podem ser considerados também proxies de grupo de conexão dedicados que incluam isso como parte de sua funcionalidade.

Palavras Finais

Considerando seu comportamento e as potenciais desvantagens descritas acima, não se deve usar conexões persistentes sem uma análise cuidadosa. Elas não devem ser usadas sem implementar alterações adicionais na aplicação ou sem um configuração cuidadosa do servidor de banco de dados e servidor web e/ou PHP-FPM.

Considere soluções alternativas, como investigar e corrigir as causas de sobrecargas na criação de conexões (por exemplo, desabilitando pesquisas reversas de DNS no servidor de banco de dados) ou proxies de grupo de conexões dedicados.

Para APIs da web de alto volume, considere usar tempos de execução alternativos ou servidores de aplicações de longa execução.

Veja também

- ibase_pconnect()
- oci_pconnect()
- odbc_pconnect()
- pfsockopen()
- pg_connect()
- MySQLi e Conexões Persistentes
- Gerenciamento de Conexões PDO

Uso da linha de Comando

Índice

Introdução

O principal foco do CLI SAPI é o desenvolvimento de aplicações shell com PHP. Existem algumas diferenças entre CLI SAPI e outras SAPIs que são explicadas neste capítulo. Vale a pena ressaltar que CLI e CGI são diferentes SAPIs embora compartilhem boa parte dos mesmos comportamentos.

O CLI SAPI é ativado por padrão usando **--enable-cli**, mas pode ser desativado usando a opção **--disable-cli** quando se executa o comando **./configure** durante o processo de compilação do PHP.

O nome, a localização e a existência dos arquivos binários do CLI/CGI podem ser diferentes dependendo de como o PHP está instalado no seu sistema. Por padrão, quando executado o **make**, ambos CGI e CLI são compilados e colocados em *sapi/cgi/php-cgi* e *sapi/cli/php*, respectivamente, no seu diretório fonte do PHP. Você vai notar que ambos são nomeados *php*. O que acontece durante o **make install** depende da sua linha de configuração. Se o módulo SAPI é escolhido durante a configuração, como apxs, ou a opção **--disable-cgi** é utilizada, o CLI é copiado para *{PREFIX}/bin/php* durante **make install**, caso contrário o CGI é colocado lá. Assim, por exemplo, se o **--with-apxs** está na sua linha de configuração, em seguida, o CLI é copiado para *{PREFIX}/bin/php* durante o **make install**. Se você quiser substituir a instalação do binário CGI, use **make install-cli** depois de **make install**. Alternativamente, você pode especificar **--disable-cgi** na sua linha de configuração.

> **Nota** :
>
> Porque ambos **--enable-cli** e **--enable-cgi** são ativadas por padrão, ter simplesmente um **--enable-cli** na sua linha de configuração não significa necessariamente que o CLI será copiado para *{PREFIX}/bin/php* durante o **make install**.

O binário CLI é distribuído na pasta principal como *php.exe* no Windows. A versão CGI é distribuída como *php-cgi.exe*. Além disso, um *php-win.exe* é distribuído se o PHP é configurado com **--enable-cli-win32**. Isso faz o mesmo que a versão CLI, exceto que ele não exibe nenhum output e também não disponibiliza nenhum console.

> **Nota** : **Qual SAPI eu tenho?**
>
> A partir da linha de comando, digitar **php -v** lhe dirá se o *php* é CGI ou CLI. Veja também a função php_sapi_name() e a constante `PHP_SAPI`.

> **Nota** :
>
> A página do `man` Unix está disponível digitando **man php** no ambiente shell.

Diferenças em relação a outros SAPIs

Diferenças notáveis do CLI SAPI em comparação com outros SAPIs:

- Diferente do CGI SAPI, nenhum cabeçalho é enviado para a saída.

  Embora o CGI SAPI forneça uma maneira de suprimir os cabeçalhos HTTP, não existe nenhuma forma equivalente de habilitá-los no CLI SAPI.

  CLI é inicializado no modo silencioso por padrão, embora as chaves **-q** e **--no-header** são mantidas por compatibilidade para que seja possível usar scripts CGI antigos.

  Ele não altera o diretório atual para aquele no qual o script está sendo executado (As opções **-C** e **--no-chdir** são mantidas por compatibilidade)

  Mensagens de erro em texto plano (sem formatação HTML).

- Existem certas diretivas do *php.ini* que são sobrescritas pelas do CLI SAPI porque elas não fazem sentido em um ambiente de linha de comando:

Diretivas do *php.ini* sobrescritas

Diretiva	Valor padrão do CLI SAPI	Comentário
html_errors	`false`	Tem como padrão `false`, já que pode ser bem difícil ler mensagens de erro na linha de comando quando elas estão misturadas com tags HTML não interpretadas.
implicit_flush	`true`	Em um ambiente de linha de comando, é geralmente desejado para a saída, como a partir de print, echo e similares, que elas sejam exibidas imediatamente, e não mantidas em um buffer. Apesar disso, ainda é possível usar output buffering para atrasar ou manipular a saída padrão.
max_execution_time	0 (unlimited)	O PHP em um ambiente de linha de comando tende a ser usado com um número de propostas muito mais diversas do

Diretiva	Valor padrão do CLI SAPI	Comentário
		que para aplicações web típicas, e como essas formas podem ter muito mais longevidade, o tempo máximo de execução é definido como sem limite.

register_argc_argv	`true`	Definir isso como `true` significa que o script executado a partir do CLI SAPI sempre terá acesso ao *argc* (número de argumentos passados para a aplicação) e *argv* (array contendo os valores dos argumentos).
		
		As variáveis *$argc* e *$argv* do PHP são automaticamente definidas com seus valores apropriados quando usando o CLI SAPI. Esses valores podem também ser encontrados no array *$_SERVER*, por exemplo: *$_SERVER['argv']*.
		
		Aviso
		
		A presença de *$argv* ou *$_SERVER['argv']* não é uma indicação confiável de que um script está sendo executado da linha de comando porque elas podem ser definidas em outros contextos quando register_argc_argv está habilitada. O valor retornado por php_sapi_name() é o que deve ser verificado.
		
		`<?php`
		
		`if (php_sapi_name() === 'cli') {`
		`echo "Este script está sendo executado da linha de comando!\n";`
		`}`
		
		Aviso
		
		Este recurso foi *DESCONTINUADO* a partir do PHP 8.5.0. O uso deste recurso é fortemente desencorajado.

output_buffering	`false`	Apesar do valor no *php.ini* ser explicitamente definido como `false`, as funções de Output buffering estão disponíveis.
max_input_time	`false`	O PHP CLI não suporta GET, POST ou upload de arquivos.

Nota :

Essas diretivas não podem ser inicializadas com outro valor no arquivo de configuração *php.ini* ou um valor personalizado (caso seja especificado). Essa limitação ocorre pois os valores são aplicados

> depois que todos os arquivos de configuração foram lidos. Entretanto, seus valores podem ser alterados em tempo de execução (porém isso não se aplica a todos eles como register_argc_argv).

> **Nota** :
>
> É recomendado definir ignore_user_abort para aplicações de linha de comando. Veja ignore_user_abort() para mais informações.

- Para facilitar o trabalho em ambientes de linha de comando, um número de constantes é definido para fluxos de E/S.
- O CLI SAPI **não** altera o diretório de trabalho para o diretório de onde o arquivo é executado.

  Exemplo #1 Exemplo demonstrando a diferença entre CGI e SAPI:

  ```php
  <?php
  // Um script de teste simples chamado test.php
  echo getcwd(), "\n";
  ?>
  ```

  Quando é utilizada a versão CGI a saída é:

  ```
  $ pwd
  /tmp

  $ php -q another_directory/test.php
  /tmp/another_directory
  ```

  Isso demonstra claramente que o PHP altera o diretório de trabalho para o diretório onde o arquivo é executado.

  Usando o CLI SAPI temos:

  ```
  $ pwd
  /tmp

  $ php -f another_directory/test.php
  /tmp
  ```

  Isso permite grande flexibilidade no desenvolvimento de ferramentas de linha de comando em PHP.

  > **Nota** :

> O CGI SAPI suporta esse comportamento do CLI SAPI através da opção **-C** utilizada quando executado a partir da linha de comando.

Opções

A lista de opções de linha de comando fornecidas pelo binário do PHP pode ser requisitada a qualquer momento bastando executar o PHP com a opção **-h**:

```
Usage: php [options] [-f] <file> [--] [args...]
   php [options] -r <code> [--] [args...]
   php [options] [-B <begin_code>] -R <code> [-E <end_code>] [--] [args...]
   php [options] [-B <begin_code>] -F <file> [-E <end_code>] [--] [args...]
   php [options] -- [args...]
   php [options] -a

  -a               Run interactively
  -c <path>|<file> Look for php.ini file in this directory
  -n               No php.ini file will be used
  -d foo[=bar]     Define INI entry foo with value 'bar'
  -e               Generate extended information for debugger/profiler
  -f <file>        Parse and execute <file>.
  -h               This help
  -i               PHP information
  -l               Syntax check only (lint)
  -m               Show compiled in modules
  -r <code>        Run PHP <code> without using script tags <?..?>
  -B <begin_code>  Run PHP <begin_code> before processing input lines
  -R <code>        Run PHP <code> for every input line
  -F <file>        Parse and execute <file> for every input line
  -E <end_code>    Run PHP <end_code> after processing all input lines
  -H               Hide any passed arguments from external tools.
  -S <addr>:<port> Run with built-in web server.
  -t <docroot>     Specify document root <docroot> for built-in web server.
  -s               Output HTML syntax highlighted source.
  -v               Version number
  -w               Output source with stripped comments and whitespace.

  args...          Arguments passed to script. Use -- args when first argument
                   starts with - or script is read from stdin

  --ini            Show configuration file names

  --rf <name>      Show information about function <name>.
  --rc <name>      Show information about class <name>.
  --re <name>      Show information about extension <name>.
  --rz <name>      Show information about Zend extension <name>.
  --ri <name>      Show configuration for extension <name>.
```

Opções de linha de comando

Opção	Opção Estendida	Descrição
-a	--interactive	Executa o PHP interativamente. Para mais informações veja a seção Shell interativo.
-b	--bindpath	Vincula o Path para o modo externo do servidor FASTCGI (apenas em CGI).
-C	--no-chdir	Não executa chdir para o diretório do script (apenas em CGI).
-q	--no-header	Modo silencioso. Suprime os cabeçalhos HTTP na saída (apenas em CGI).
-T	--timing	Calcula o tempo de execução do script repetidas *count* de vezes (apenas em CGI).

-c	--php-ini	Especifica um diretório para procurar pelo *php.ini*, ou um arquivo `INI` personalizado (que não precisa se chamar *php.ini*), exemplo:
		`$ php -c /custom/directory/ my_script.php`
		`$ php -c /custom/directory/custom-file.ini my_script.php`
		Se essa opção não for especificada, o *php.ini* será procurado no local padrão.

-n	--no-php-ini	Ignore o *php.ini* completamente.
-d	--define	Define um valor personalizado para qualquer uma das diretivas de configuração permitidas em arquivos *php.ini*. A sintaxe é: `-d configuration_directive[=value]` **Exemplo #1 Exemplo de uso de `-d` para definir uma configuração INI**

Opção	Opção Estendida	Descrição

		``
		# Omitir o valor irá configurar a diretiva para "1"
		$ php -d max_execution_time
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(1) "1"
		
		# Passar um valor vazio irá configurar a detiva para ""
		php -d max_execution_time=
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(0) ""
		
		# A diretiva de configudação será configurada para tudo o que
		estiver depois do sinal de '='
		$ php -d max_execution_time=20
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(2) "20"
		$ php
		        -d max_execution_time=doesntmakesense
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(15) "doesntmakesense"
		``

-e	--profile-info	Ativa o modo de informação estendido, para ser usado por um debugger/profiler.
-f	--file	Lê e executa o arquivo especificado. A opção **-f** é opcional e pode ser omitida - fornecer somente o nome do arquivo a ser executado é suficiente.
-h e -?	--help e --usage	Exibe uma lista de opções de linha de comando com a descrição do seu funcionamento.
-i	--info	Invoca phpinfo(), e exibe o resultado. Caso o PHP não esteja executando corretamente, é aconselhável usar o comando **php -i** e verificar se alguma mensagem de erro é informada antes ou no lugar da tabela de informações. Esteja ciente de que ao usar o modo CGI a saída será em HTML e portanto bastante longa.
-l	--syntax-check	Verifica a sintaxe mas não executa o código PHP informado. A entrada padrão será processada se nenhum arquivo for especificado, caso contrário cada arquivo será verificado. Em caso de sucesso, o texto `No syntax errors detected in <filename>` é escrito na saída padrão. Em caso de falha, o texto `Errors parsing <filename>` é escrito na saída padrão junto com o erro do analisador interno. Se alguma falha for encontrada nos arquivos especificados (ou na entrada padrão), o código de retorno do shell será `-1`, caso contrário o código de retorno será `0`. Esta opção encontrará erros fatais (como funções indefinidas) que requerem a execução do código.

Opção	Opção Estendida	Descrição
		Nota : Antes do PHP 8.3.0, só era possível especificar um nome de arquivo para ser verificado. **Nota** : Essa opção não funciona em conjunto com a opção **-r**.

-m	--modules	**Exemplo #2 Retorna os módulos PHP e Zend integrados e carregados**
		
		`$ php -m`
		`[PHP Modules]`
		`xml`
		`tokenizer`
		`standard`
		`session`
		`posix`
		`pcre`
		`overload`
		`mysql`
		`mbstring`
		`ctype`
		
		`[Zend Modules]`

-r	--run	Habilita a execução do código PHP incluído diretamente na linha de comando. As tags de abertura e fechamento do PHP (`<?php` e `?>`) **não são necessárias** e irão causar um erro de leitura caso sejam incluídas.
		
		Nota :
		Deve-se ter cuidado ao usar esta forma do PHP para não colidir com a substituição de variáveis de linha de comando realizadas pelo shell.
		
		Exemplo #3 Obtendo um erro de sintaxe ao usar aspas duplas
		
		`$ php -r "$foo = get_defined_constants();"`
		`PHP Parse error:  syntax error, unexpected '=' in Command line code on line 1`

Opção	Opção Estendida	Descrição

```
Parse error: syntax error, unexpected '=' in Command line code on
line 1
```

O problema aqui é que o sh/bash realiza substituições de variáveis mesmo usando aspas duplas ". já que a variável *$foo* provavelmente não está definida, ela será avaliada como nada, o que faz com que o código passado ao PHP para execução na verdade seja lido como:

```
$ php -r " = get_defined_constants();"
```

A maneira correta é usar aspas simples '. Variáveis em strings de aspas simples não são substituídas pelo sh/bash.

Exemplo #4 Usando aspas simples para evitar a substituição de variáveis do shell

```
$ php -r '$foo = get_defined_constants(); var_dump($foo);'
array(370) {
  ["E_ERROR"]=>
  int(1)
  ["E_WARNING"]=>
  int(2)
  ["E_PARSE"]=>
  int(4)
  ["E_NOTICE"]=>
  int(8)
  ["E_CORE_ERROR"]=>
  [...]
```

Se estiver usando um ambiente de linha de comando que não seja sh/bash, outros problemas podem ser encontrados - caso necessário, um relato de problema pode ser aberto em » https://github.com/php/php-src/issues. Ainda assim é muito fácil encontrar problemas ao tentar usar variáveis (do shell ou do PHP) em código de linha de comando, ou usando barra invertida para escapar, por isso tenha muito cuidado quando fizer isso. Você foi avisado!

Opção	Opção Estendida	Descrição
		Nota : **-r** está disponível no CLI SAPI, mas não no SAPI *CGI*. **Nota** : Essa opção é normalmente utilizada com código muito básico, portanto algumas diretivas de configuração (como auto_prepend_file e auto_append_file) são ignoradas nesse modo.
-B	--process-begin	Código PHP a ser executado antes de processar a entrada padrão.
-R	--process-code	Código PHP a ser executado para cada linha de entrada. Existem duas variáveis especiais disponíveis nesse modo: *$argn* e *$argi*. *$argn* conterá a linha que o PHP estiver processando no momento, enquanto *$argi* irá conter o número da linha.
-F	--process-file	Arquivo PHP a ser executado para cada linha de entrada.

-E	--process-end	Código PHP a ser executado após processar a entrada.
		
		Exemplo #5 Usando as opções -B, -R e -E para contar o número de linhas de um projeto.
		
		`$ find my_proj | php -B '$l=0;' -R '$l += count(@file($argn));' -E 'echo "Total Lines: $l\n";'`
		`Total Lines: 37328`

-S	--server	Inicia o servidor interno do PHP.
-t	--docroot	Específica a raiz do documento para o servidor interno do PHP.
-s	--syntax-highlight e --syntax-highlighting	Exibe o código com destaque de sintaxe em cores. Essa opção usa os mecanismos internos para interpretar os arquivos e escrever uma versão HTML colorida do código para a saída padrão. Repare que tudo que ele faz é gerar um bloco de tags HTML `<code> [...] </code>`, sem os cabeçalhos HTML.

Opção	Opção Estendida	Descrição
		Nota : Essa opção não funciona em conjunto com a opção **-r**.

-v	--version	**Exemplo #6 Usando -v para obter o nome do SAPI e a versão do PHP e do Zend**
		`$ php -v`
		`PHP 5.3.1 (cli) (built: Dec 11 2009 19:55:07)`
		`Copyright (c) 1997-2009 The PHP Group`
		`Zend Engine v2.3.0, Copyright (c) 1998-2009 Zend Technologies`

-w	--strip	Exibe o código com remoção de comentários e de espaços em branco. **Nota** : Essa opção não funciona em conjunto com a opção **-r**.

	--ini	Exibir nomes de arquivos de configuração e diretórios verificados. Opcionalmente, passe `--ini=diff` para mostrar as diferenças entre os arquivos de configuração carregados e a configuração padrão.
		Exemplo #7 Exemplo de `--ini`
		`$ php --ini`
		`Configuration File (php.ini) Path: /usr/dev/php/5.2/lib`
		`Loaded Configuration File: /usr/dev/php/5.2/lib/php.ini`
		`Scan for additional .ini files in: (none)`
		`Additional .ini files parsed: (none)`
		Nota :
		A opção `--ini=diff` não está disponivel em versões anteriores ao PHP 8.5.0.

--rf	--rfunction	Exibe informações sobre a função ou método de objeto informado (como por exemplo o número e nome dos parâmetros). Essa opção só está disponível se o PHP for compilado com suporte a Reflexão.

Opção	Opção Estendida	Descrição

		Exemplo #8 uso básico da opção `--rf`
		
		`$ php --rf var_dump`
		`Function [ <internal> public function var_dump ] {`
		
		`  - Parameters [2] {`
		`  Parameter #0 [ <required> $var ]`
		`  Parameter #1 [ <optional> $... ]`
		`  }`
		`}`

--rc	--rclass	Exibe informações sobre uma determinada classe (lista de constantes, propriedades e métodos).
		
		Essa opção só está disponível se o PHP for compilado com suporte a Reflexão.
		
		Exemplo #9 exemplo de `--rc`
		
		`$ php --rc Directory`
		`Class [ <internal:standard> class Directory ] {`
		
		`  - Constants [0] {`
		`  }`
		
		`  - Static properties [0] {`
		`  }`
		
		`  - Static methods [0] {`
		`  }`
		
		`  - Properties [0] {`
		`  }`
		
		`  - Methods [3] {`
		`  Method [ <internal> public method close ] {`
		`  }`
		
		`  Method [ <internal> public method rewind ] {`
		`  }`
		
		`  Method [ <internal> public method read ] {`
		`  }`
		`  }`
		`}`

--re	--rextension	Exibe informações sobre uma determinada extensão (lista de opções do *php.ini*, funções definidas, constantes e classes). Essa opção só está disponível se o PHP for compilado com suporte a Reflexão. **Exemplo #10 Exemplo de `--re`**

Opção	Opção Estendida	Descrição

		`$ php --re json`
		`Extension [ <persistent> extension #19 json version 1.2.1 ] {`
		
		`  - Functions {`
		`  Function [ <internal> function json_encode ] {`
		`  }`
		`  Function [ <internal> function json_decode ] {`
		`  }`
		`  }`
		`}`

--rz	--rzendextension	Exibe as informações de configuração de uma determinada extensão Zend (a mesma informação retornada pela phpinfo()).

--ri	--rextinfo	Exibe as informações de configuração de uma determinada extensão (as mesmas informações retornadas pela phpinfo()). As informações de configurações do núcleo da linguagem estão disponíveis usando "main" como nome da extensão.
		
		Exemplo #11 exemplo de `--ri`
		
		`$ php --ri date`
		
		`date`
		
		`date/time support => enabled`
		`"Olson" Timezone Database Version => 2009.20`
		`Timezone Database => internal`
		`Default timezone => Europe/Oslo`
		
		`Directive => Local Value => Master Value`
		`date.timezone => Europe/Oslo => Europe/Oslo`
		`date.default_latitude => 59.930972 => 59.930972`
		`date.default_longitude => 10.776699 => 10.776699`
		`date.sunset_zenith => 90.583333 => 90.583333`
		`date.sunrise_zenith => 90.583333 => 90.583333`

> **Nota** :
>
> As opções `-rBRFEH`, `--ini` e `--r[fcezi]` estão disponíveis somente no CLI.

Uso

Existem três formas de fornecer código PHP para ser executado pelo CLI SAPI:

1. Diga ao PHP para executar determinado arquivo.

```
$ php my_script.php

$ php -f my_script.php
```

As duas formas (usando ou não a opção **-f**) executam o arquivo *my_script.php*. Note que não existe restrição sobre quais arquivos podem ser executados; em particular, o nome do arquivo não precisa conter a extensão `.php`.

2. Passa o código PHP diretamente através da linha de comando.

```
$ php -r 'print_r(get_defined_constants());'
```

Um cuidado especial deve ser tomado em relação a substituição de variáveis e o uso de aspas.

> **Nota** :
>
> Leia o exemplo cuidadosamente: não existem tags de abertura e fechamento! A opção **-r** simplesmente não necessita delas, e usá-las irá gerar um erro de interpretação.

3. Direciona a entrada padrão (`stdin`) para o código PHP.

Isso dá a poderosa habilidade de criar código PHP dinamicamente e alimentar o binário, assim como demonstrado nesse exemplo fictício:

```
$ some_application | some_filter | php | sort -u > final_output.txt
```

Você não pode combinar essas formas de executar o código.

Assim como qualquer aplicação de linha de comando, o PHP aceita qualquer quantidade de argumentos; entretanto, os scripts PHP também podem receber outros argumentos. A quantidade de argumentos que podem ser passados para o script não é limitada pelo PHP (e apesar do shell ter um limite no número de caracteres que podem ser passados, ele não é geralmente atingido). Os argumentos passados para os scripts ficam disponíveis no array global *$argv*. O primeiro índice (zero) sempre contém o nome do script que está sendo chamado pela linha de comando. Note que, se o código é executado diretamente usando a opção de linha de comando **-r**, o valor de *$argv[0]* será `"Standard input code"`; anteriormente ao PHP 7.2.0, isso era um hífen (`"-"`). A mesma coisa ocorre se o código é executado via um pipe a partir do **STDIN**.

Uma segunda variável global, *$argc*, contém o número de elementos contidos no array *$argv* array (**não** o número de argumentos passados para o script).

Desde que os argumentos passados pelo script não comecem com o caractere `-`, não existe nada em especial com que você deva se preocupar. Passar um argumento para o script que comece com `-` irá causar problemas, porque o interpretador do PHP irá pensar que ele mesmo deve manipular esse argumento, mesmo antes de executar o script. Para prevenir isso use o separador de lista de argumentos `--`. Depois que esse separador é interpretado pelo PHP, cada argumento seguinte é passado intocado para o script.

```
# Isto não executará o código, apenas mostrará as opções do PHP
$ php -r 'var_dump($argv);' -h
Usage: php [options] [-f] <file> [args...]
[...]

# Isto passará o argumento '-h' para o script, assim evitando do PHP interpretá-lo
$ php -r 'var_dump($argv);' -- -h
array(2) {
  [0]=>
  string(1) "-"
  [1]=>
  string(2) "-h"
}
```

Entretanto, em sistemas Unix existe uma outra maneira de usar o PHP em linha de comando: fazer com que a primeira linha do script comece com `#!/usr/bin/php` (ou qualquer que seja o caminho para o binário do PHP CLI caso seja diferente. O restante do arquivo deve conter código PHP normal com as já tradicionais tags de abertura e fechamento. Uma vez que os atributos de execução do arquivo estejam apropriadamente definidos (exemplo. **chmod +x test**), o script poderá ser executado como qualquer outro script shell ou perl:

Exemplo #1 Executando scripts PHP como shell scripts

```
#!/usr/bin/php
<?php
var_dump($argv);
?>
```

Assumindo que esse arquivo tenha o nome *test* no diretório atual, é possível fazer o seguinte:

```
$ chmod +x test
$ ./test -h -- foo
array(4) {
  [0]=>
  string(6) "./test"
  [1]=>
  string(2) "-h"
  [2]=>
  string(2) "--"
  [3]=>
```

```
  string(3) "foo"
}
```

Como pode ser visto, nesse caso nenhum cuidado especial precisa ser tomado ao passar parâmetros começando com -.

O executável do PHP pode ser usado para rodar scripts absolutamente independentes do servidor web. Em sistemas Unix, a primeira linha especial `#!` (ou "shebang") deve ser adicionada a scripts PHP para que o sistema possa automaticamente dizer qual programa deve executar o script. Em plataformas Windows, é possível associar o *php.exe* com um clique duplo no arquivos com extensão `.php`, ou um arquivo batch pode ser criado para rodar scripts através do PHP. A primeira linha especial shebang para Unix não causa nenhum efeito no Windows (já que é formatada como um comentário PHP), então programas multiplataforma podem ser escritos com a inclusão disso. Um exemplo simples de programa de linha de comando em PHP é mostrado abaixo.

Exemplo #2 Script planejado para ser executado na linha de comando (script.php)

```
#!/usr/bin/php
<?php

if ($argc != 2 || in_array($argv[1], array('--help', '-help', '-h', '-?'))) {
?>

This is a command line PHP script with one option.

  Usage:
  <?php echo $argv[0]; ?> <option>

  <option> can be some word you would like
  to print out. With the --help, -help, -h,
  or -? options, you can get this help.

<?php
} else {
    echo $argv[1];
}
?>
```

O script acima inclui a shebang na primeira linha para indicar que esse arquivo deve ser executado pelo PHP. Nós estamos trabalhando com a versão CLI aqui, então nenhum cabeçalho HTTP será exibido.

O programa primeiramente verifica se o primeiro argumento obrigatório foi passado (juntamente com o nome do arquivo, que também é contabilizado). Em caso negativo, ou caso o argumento seja **--help**, **-help**, **-h** ou **-?**, uma mensagem de ajuda é disparada, usando *$argv[0]* para imprimir dinamicamente o nome do script como foi digitado na linha de comando. Do contrário, o argumento é exibido da forma como foi escrito.

Para executar o script acima em sistemas Unix, ele deve ser tornado executável, e chamado simplesmente com **script.php echothis** ou **script.php -h**. Em sistemas Windows, um arquivo batch similar ao seguinte pode ser criado para essa tarefa:

Exemplo #3 Arquivo batch para rodar um script de linha de comando em PHP (script.bat)

```
@echo OFF
"C:\php\php.exe" script.php %*
```

Assumindo que o programa acima foi nomeado como *script.php*, e que o CLI *php.exe* está em *C:\php\php.exe*, esse arquivo batch irá executá-lo, repassando todas as opções: **script.bat echothis** ou **script.bat -h**.

Veja também a documentação da extensão Readline com mais funções usadas para melhorar as aplicações de linha de comando em PHP.

No Windows, o PHP pode ser configurado para rodar sem a necessidade de informar o *C:\php\php.exe* ou a extensão `.php`, como descrito em PHP para linha de comando no Microsoft Windows.

> **Nota** :
>
> No Windows é recomendado rodar o PHP em uma conta de usuário normal. Ao rodar em uma conta de serviço certas operações podem falhar, por conta da situação "No mapping between account names and security IDs was done" (não foi feito nenhum mapeamento entre nomes de contas e IDs de segurança).

Fluxos de E/S

O CLI SAPI define algumas constantes para fluxos de E/S com o objetivo de tornar a programação em linha de comando um pouco mais fácil.

Constantes específicas da CLI

Constante	Descrição

STDIN	Um fluxo já aberto para o `stdin`. Isso evita ter que abrí-lo com
	`<?php`
	`$stdin = fopen('php://stdin', 'r');`
	`?>`
	Se você desejar ler uma única linha do `stdin`, você pode usar
	`<?php`
	`$line = trim(fgets(STDIN)); // Lê uma linha do STDIN`
	`fscanf(STDIN, "%d\n", $number); // Lê um número do STDIN`
	`?>`

STDOUT	Um fluxo já aberto para o `stdout`. Isso evitar ter que abrí-lo com
	`<?php`
	`$stdout = fopen('php://stdout', 'w');`
	`?>`

STDERR	Um fluxo já aberto com `stderr`. Isso evita ter que abri-lo com
	`<?php`
	`$stderr = fopen('php://stderr', 'w');`
	`?>`

Como dito acima, não é necessário, por exemplo, abrir um fluxo para `stderr` por si mesmo, basta usar a constante ao invés do recurso do fluxo:

```
php -r 'fwrite(STDERR, "stderr\n");'
```

Você não precisa fechar explicitamente esses fluxos, já que eles serão automaticamente fechados pelo PHP quando o script terminar.

> **Nota** :
>
> Essas constantes não estão disponíveis se estiver lendo os scripts PHP de `stdin`.

Shell interativo

O CLI SAPI fornece um shell interativo usando a opção **-a** se o PHP for compilado com a opção **--with-readline**. A partir do PHP 7.1.0 o shell interativo também está disponível no Windows, se a extensão readline estiver habilitada.

Usando o shell interativo você será capaz de escrever código PHP e executá-lo diretamente.

Exemplo #1 Executando código usando o shell interativo

```
$ php -a
Interactive shell

php > echo 5+8;
13
php > function addTwo($n)
php > {
php { return $n + 2;
php { }
php > var_dump(addtwo(2));
int(4)
php >
```

O shell interativo também possui sugestões de complemento com tab para funções, constantes, nomes de classes, variáveis, chamadas para métodos estáticos e constantes de classe.

Exemplo #2 Sugestões de complemento com tab

Pressionar a tecla tab duas vezes quando tiver mais de uma sugestão de complemento resultará numa lista desses complementos.

```
php > strp[TAB][TAB]
strpbrk   strpos    strptime
php > strp
```

Quando existir apenas um possível complemento, pressionar a tecla tab uma vez irá completar o restante da mesma linha:

```
php > strpt[TAB]ime(
```

Complementos também funcionarão para nomes que forem definidos durante a mesma sessão do shell interativo:

```
php > $fooThisIsAReallyLongVariableName = 42;
php > $foo[TAB]ThisIsAReallyLongVariableName
```

O shell interativo armazena seu histórico que pode ser acessado usando as teclas para cima e para baixo. O histórico é salvo no arquivo *~/.php_history*. A partir do PHP 8.4.0, o caminho para o arquivo de histórico pode ser definido usando a variável de ambiente *PHP_HISTFILE*.

O CLI SAPI fornece as configurações do *php.ini* `cli.pager` e `cli.prompt`. A configuração `cli.pager` permite que um programa externo (como o *less*) aja como uma alternativa para a saída ao invés dela ser exibida diretamente na tela. A configuração `cli.prompt` faz com que seja possível alterar a saída `php >`.

Também é possível definir configurações *php.ini* no shell interativo usando uma notação abreviada.

Exemplo #3 Definindo configurações *php.ini* no shell interativo

A opção `cli.prompt`:

```
php > #cli.prompt=hello world :>
hello world :>
```

Usando acento grave é possível executar o PHP diretamente no terminal:

```
php > #cli.prompt=`echo date('H:i:s');` php >
15:49:35 php > echo 'hi';
hi
15:49:43 php > sleep(2);
15:49:45 php >
```

Setando o paginador para *less*:

```
php > #cli.pager=less
php > phpinfo();
(output displayed in less)
php >
```

A configuração `cli.prompt` suporta algumas sequências de escape:

Sequências de escape do `cli.prompt`

Sequência	Descrição
`\e`	Usada para adicionar cores ao terminal. Um exemplo pode ser `\e[032m\v \e[031m\b \e[34m\> \e[0m`
`\v`	A versão do PHP.
`\b`	Indica em qual bloco o PHP está. Por exemplo `/*` irá indicar que o PHP está dentro de um comentário de múltiplas linhas. O escopo externo será denominado por `php`.
`\>`	Indica o caractere do terminal. Por padrão é `>`, mas pode alterar quando o shell estiver dentro de um bloco indeterminado ou de uma string. Possíveis caracteres são: `' " { ( >`

> **Nota** :
>
> Arquivos incluídos através de auto_prepend_file e auto_append_file são interpretados nesse modo porém com algumas restrições - Exemplo: funções devem ser definidas antes de sua chamada.

Modo interarativo

Se a extensão readline não estiver disponível, anteriormente ao PHP 8.1.0, chamar o CLI SAPI com a opção **-a** fornecia o modo interativo. Nesse modo, um script PHP completo deveria ser fornecido via STDIN, terminado por uma sequência de `CTRL +D` (POSIX) ou `CTRL +Z` seguido de `+ENTER` (Windows), para que o script seja avaliado. Isto é basicamente o mesmo que invocar CLI SAPI sem a opção **-a**.

A partir do PHP 8.1.0, chamar a CLI SAPI com a opção **-a** falhará na ausência da extensão readline.

Servidor web embutido

> **Aviso**
>
> Esse servidor web foi desenvolvido para auxiliar no desenvolvimento de aplicações. Ele também pode ser útil para testes ou para demonstrações de aplicações que forem executadas em ambientes controlados. Ele não foi desenvolvido para ser um web server completo. Ele não deve ser utilizado em uma rede pública.

O CLI SAPI fornece um servidor web embutido.

Esse servidor embutido roda apenas um processo single-thread, de forma que aplicações PHP irão congelar se uma requisição bloquear.

Requisições de URI são servidas a partir do diretório atual onde o PHP foi iniciado, a não ser que a opção -t seja usada para especificar uma raiz de documento explicitamente. Se uma requisição de URI especificar um arquivo específico então ou o index.php ou index.html no diretório em questão será retornado. Se nenhum deles existir, a procura por index.php e index.html continuará nos diretórios superiores até que algum deles seja encontrado ou o document root seja alcançado. Caso um index.php ou index.html seja encontrado então é retornado e $_SERVER['PATH_INFO'] é configurado para a parte final da URI. Caso contrário, uma resposta 404 é retornada.

Se um arquivo PHP for fornecido na linha de comando quando o servidor web for iniciado ele será tratado como roteador. O script é executado no início de cada requisição HTTP. Se o script retornar `false`, então o recurso requisitado será retornado da maneira como estiver. Do contrário, a saída do script será retornada para o navegador.

Tipos MIME padrão serão retornados para arquivos com extensões: `.3gp`, `.apk`, `.avi`, `.bmp`, `.css`, `.csv`, `.doc`, `.docx`, `.flac`, `.gif`, `.gz`, `.gzip`, `.htm`, `.html`, `.ics`, `.jpe`, `.jpeg`, `.jpg`, `.js`, `.kml`, `.kmz`, `.m4a`, `.mov`, `.mp3`, `.mp4`, `.mpeg`, `.mpg`, `.odp`, `.ods`, `.odt`, `.oga`, `.ogg`, `.ogv`, `.pdf`, `.png`, `.pps`, `.pptx`, `.qt`, `.svg`, `.swf`, `.tar`, `.text`, `.tif`, `.txt`, `.wav`, `.webm`, `.wmv`, `.xls`, `.xlsx`, `.xml`, `.xsl`, `.xsd`, `.zip` .

A partir do PHP 7.4.0, o servidor web interno pode ser configurado para se bifurcar em múltiplos processos para testar código que requer múltiplas requisições concorrentes ao servidor web interno. Configure a variável de ambiente *PHP_CLI_SERVER_WORKERS* para o número de processos desejados antes de iniciar o servidor.

> **Nota** :
>
> Trabalhadores múltiplos não são suportados no Windows.

> **Nota** :
>
> Para informações sobre o uso e as opções do PHP em linha de comando, execute **php --help** ou **man php**. Nem todas as opções se aplicam quando o PHP está sendo executado como servidor web.

> **Aviso**
>
> Este recurso *experimental não* foi projetado para uso em ambiente de produção. Geralmente, o servidor web interno *não* tem o objetivo de ser usado em ambiente de produção.

Exemplo #1 Iniciando o servidor web

```
$ cd ~/public_html
$ php -S localhost:8000
```

O terminal irá exibir:

```
PHP 5.4.0 Development Server started at Thu Jul 21 10:43:28 2011
Listening on localhost:8000
Document root is /home/me/public_html
Press Ctrl-C to quit
```

Após serem feitas requisições de URI para for http://localhost:8000/ e http://localhost:8000/myscript.html o terminal irá exibir algo similar a:

```
PHP 5.4.0 Development Server started at Thu Jul 21 10:43:28 2011
Listening on localhost:8000
Document root is /home/me/public_html
Press Ctrl-C to quit.
[Thu Jul 21 10:48:48 2011] ::1:39144 GET /favicon.ico - Request read
[Thu Jul 21 10:48:50 2011] ::1:39146 GET / - Request read
[Thu Jul 21 10:48:50 2011] ::1:39147 GET /favicon.ico - Request read
[Thu Jul 21 10:48:52 2011] ::1:39148 GET /myscript.html - Request read
[Thu Jul 21 10:48:52 2011] ::1:39149 GET /favicon.ico - Request read
```

Note que antes do PHP 7.4.0, recursos estáticos representados por symlinks não eram acessíveis pelo Windows, sendo necessário um script de roteamento nesses casos.

Exemplo #2 Iniciando o servidor web com um diretório raiz específico

```
$ cd ~/public_html
$ php -S localhost:8000 -t foo/
```

O terminal irá exibir:

```
PHP 5.4.0 Development Server started at Thu Jul 21 10:50:26 2011
Listening on localhost:8000
Document root is /home/me/public_html/foo
Press Ctrl-C to quit
```

Exemplo #3 Usando um script de roteamento

Nesse exemplo, requisições para imagens serão exibidas, mas requisições para arquivos HTML irão exibir "Welcome to PHP":

```
<?php
// router.php
if (preg_match('/\.(?:png|jpg|jpeg|gif)$/', $_SERVER["REQUEST_URI"])) {
    return false;    // serve o recurso requisitado sem modificação.
} else {
    echo "<p>Welcome to PHP</p>";
}
?>
```

```
$ php -S localhost:8000 router.php
```

Exemplo #4 Verificando o Uso do CLI Web Server

Para reutilizar um script de roteamento específico de um framework durante o desenvolvimento com o servidor web CLI e depois também em um servidor de produção:

```
<?php
// router.php
if (php_sapi_name() == 'cli-server') {
    /* roteia ativos estáticos e retorna false */
}
/* continua com operações normais do index.php */
?>
```

```
$ php -S localhost:8000 router.php
```

Exemplo #5 Tratando de tipos de arquivos não suportados

Se você precisar servir um recurso estático cujo MIME type não é tratado pelo servidor web CLI, use:

```
<?php
// router.php
```

```php
$path = pathinfo($_SERVER["SCRIPT_FILENAME"]);
if ($path["extension"] == "el") {
    header("Content-Type: text/x-script.elisp");
    readfile($_SERVER["SCRIPT_FILENAME"]);
}
else {
    return FALSE;
}
?>
```

```
$ php -S localhost:8000 router.php
```

Exemplo #6 Acessando o servidor web CLI de máquinas remotas

Você pode tornar o servidor web acessível na porta 8000 para qualquer interface com:

```
$ php -S 0.0.0.0:8000
```

> **Aviso**
>
> O servidor web embutido não deve ser exposto em redes públicas.

Configurações INI

Opções de configuração da CLI SAPI

Nome	Padrão	Modificável	Registro de Alterações
cli_server.color	"0"	INI_ALL	

Aqui está uma breve explicação das diretivas de configuração.

`cli_server.color` bool

Habilita o uso de cores ANSI na saída do terminal pelo servidor web.

Coletor de Lixo

Índice

Esta seção explica os méritos do novo mecanismo do Coletor de Lixo (também conhecido como GC - Garbage Collector) que é parte do PHP 5.3.

Básico sobre Contagem de Referência

Uma variável PHP é armazenada em um contêiner chamado "zval". Um zval contém, além do tipo e do valor da variável, dois bits adicionais de informação. O primeiro é chamado de "is_ref" e é um valor booleano que indica se a variável é parte de um "conjunto de referência" ou não. Com este bit, o motor do PHP sabe como diferenciar variáveis normais de referências. Como o PHP permite referências no nível do usuário, como as criadas pelo operador &, um contêiner zval também tem um mecanismo de contagem de referência interno para otimizar o uso de memória. Esta segunda parte de informação adicional, chamado "refcount", contém a quantidade de nomes de variáveis (também chamadas de símbolos) que apontam para este contêiner. Todos os símbolos são armazenados em uma tabela de símbolos, e existe uma por escopo. Existe um escopo para o

script principal (ou seja, aquele requisitado através do navegador), assim como um escopo para cada função ou método.

Um contêiner zval é criado quando uma nova variável é criada com um valor constante, como em:

Exemplo #1 Criando um novo contêiner zval

```
<?php
$a = "new string";
?>
```

Neste caso, o nome do símbolo, `a`, é criado no escopo atual, e um novo contêiner de variável é criado com o tipo string e o valor `new string`. O bit "is_ref" é por padrão definido para `false` porque nenhuma referência no nível do usuário foi criada. O "refcount" é definido para `1` já que existe apenas um símbolo que faz uso deste contêiner de variável. Note que referências (isto é, "is_ref" igual a `true`) com "refcount" igual a `1`, são tratadas como se elas não fossem referências (como se "is_ref" fosse `false`). Se o » Xdebug estiver instalado, esta informação pode ser mostrada chamando-se a função **xdebug_debug_zval()**.

Exemplo #2 Mostrando a informação zval

```
<?php
$a = "new string";
xdebug_debug_zval('a');
?>
```

O exemplo acima produzirá:

```
a: (refcount=1, is_ref=0)='new string'
```

Atribuir esta variável a outro nome de variável irá aumentar o "refcount".

Exemplo #3 Aumentando o "refcount" de um zval

```
<?php
$a = "new string";
$b = $a;
xdebug_debug_zval( 'a' );
?>
```

O exemplo acima produzirá:

```
a: (refcount=2, is_ref=0)='new string'
```

O refcount é 2 aqui, porque o mesmo contêiner de variável está ligado tanto com *a* quanto com *b*. O PHP é inteligente o suficiente para não copiar o contêiner real da variável quando não for necessário. Contêineres são destruídos quando o "refcount" atinge zero. O "refcount" é diminuído em uma unidade quando qualquer símbolo ligado ao contêiner da variável deixa o escopo (ex.: quando a função termina) ou quando um símbolo perde a atribuição (ex.: chamando unset()). O exemplo a seguir mostra isso:

Exemplo #4 Diminuindo o "refcount" de zval

```php
<?php
$a = "new string";
$c = $b = $a;
xdebug_debug_zval( 'a' );
$b = 42;
xdebug_debug_zval( 'a' );
unset( $c );
xdebug_debug_zval( 'a' );
?>
```

O exemplo acima produzirá:

```
a: (refcount=3, is_ref=0)='new string'
a: (refcount=2, is_ref=0)='new string'
a: (refcount=1, is_ref=0)='new string'
```

Se agora `unset($a);` for chamada, o contêiner da variável, incluindo o tipo e o valor, serão removidos da memória.

Tipos Compostos

As coisas ficam um pouco mais complexas com tipos compostos como arrays e objects. Ao contrário dos valores escalares, arrays e objects armazenam suas propriedades em uma tabela de símbolos própria. Isto significa que o exemplo a seguir cria três contêineres zval:

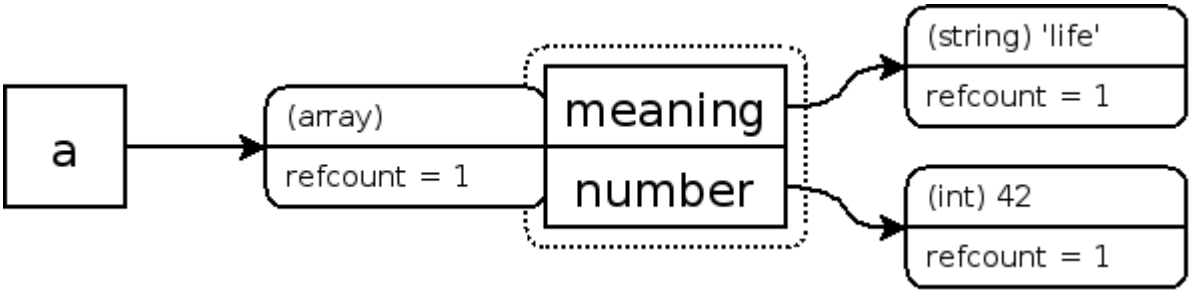
Exemplo #5 Criando um zval de array

```php
<?php
$a = array( 'meaning' => 'life', 'number' => 42 );
xdebug_debug_zval( 'a' );
?>
```

O exemplo acima produzirá algo semelhante a:

```
a: (refcount=1, is_ref=0)=array (
   'meaning' => (refcount=1, is_ref=0)='life',
   'number' => (refcount=1, is_ref=0)=42
)
```

Ou graficamente



Os três contêineres zval são: *a*, *meaning*, e *number*. Regras similares se aplicam para aumento e redução de "refcounts". Abaixo, outro elemento é adicionado ao array, e define seu valor ao conteúdo de um elemento já existente:

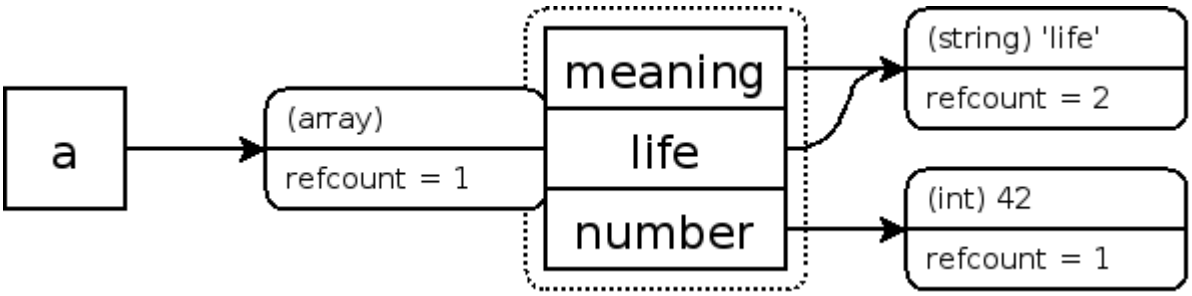
Exemplo #6 Adicionando elemento já existente a um array

```php
<?php
$a = array( 'meaning' => 'life', 'number' => 42 );
$a['life'] = $a['meaning'];
xdebug_debug_zval( 'a' );
?>
```

O exemplo acima produzirá algo semelhante a:

```
a: (refcount=1, is_ref=0)=array (
   'meaning' => (refcount=2, is_ref=0)='life',
   'number' => (refcount=1, is_ref=0)=42,
   'life' => (refcount=2, is_ref=0)='life'
)
```

Ou graficamente



Pela saída do Xdebug acima, pode-se perceber que tanto o elemento antigo do array quanto o novo agora apontam para um contêiner zval cujo "refcount" é 2. Embora a saída do Xdebug mostre dois contêineres zval com valor `'life'`, eles são o mesmo. A função **xdebug_debug_zval()** não mostra isso, mas pode-se ver isso mostrando o ponteiro de memória.

Remover o elemento de um array é como remover um símbolo de um escopo. Fazendo isso, o "refcount" de um contêiner ao qual um elemento do array aponta é reduzido. Novamente, quando o "refcount" atinge zero, o contêiner da variável é removido da memória. Um exemplo para mostrar isto:

Exemplo #7 Removendo um elemento de um array

```php
<?php
$a = array( 'meaning' => 'life', 'number' => 42 );
$a['life'] = $a['meaning'];
unset( $a['meaning'], $a['number'] );
xdebug_debug_zval( 'a' );
?>
```

O exemplo acima produzirá algo semelhante a:

```
a: (refcount=1, is_ref=0)=array (
   'life' => (refcount=1, is_ref=0)='life'
)
```

Agora, as coisas ficam interessantes se o próprio array for adicionado como um elemento do array, o que é mostrado no exemplo a seguir, onde também um operador de referência foi inserido, senão o PHP criaria uma cópia:

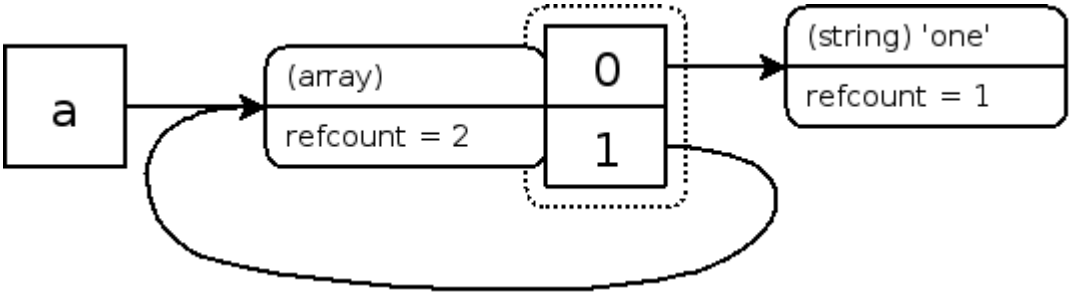
Exemplo #8 Adicionando o próprio array como um elemento de si mesmo

```php
<?php
$a = array( 'one' );
$a[] =& $a;
xdebug_debug_zval( 'a' );
?>
```

O exemplo acima produzirá algo semelhante a:

```
a: (refcount=2, is_ref=1)=array (
   0 => (refcount=1, is_ref=0)='one',
   1 => (refcount=2, is_ref=1)=...
)
```

Ou graficamente



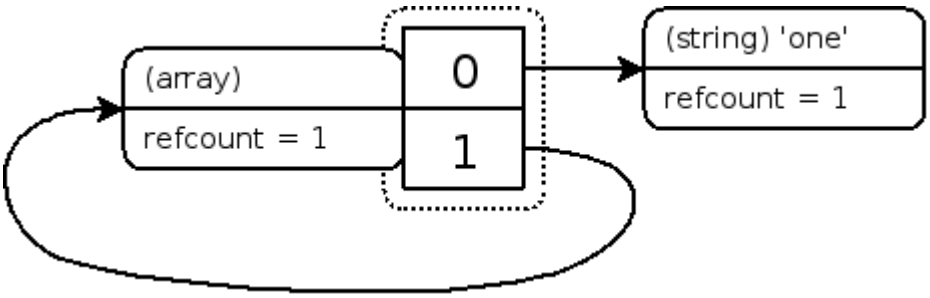
Pode-se perceber que a variável do array (*a*) assim como o segundo elemento (*1*) agora apontam para um contêiner de variável que tem um "refcount" de 2. Os "..." no exemplo acima mostram que há recursão envolvida, e que, obviamente, neste caso significa que os "..." apontam de volta ao array original.

Como antes, tirar a atribuição de uma variável remove o símbolo, e a contagem de referência do contêiner da variável à qual o símbolo aponta é reduzida em uma unidade. Então, se a variável *$a* perder a atribuição após execução do código acima, a contagem de referência do contêiner da variável à qual *$a* e o elemento "1" apontam será diminuída em uma unidade, de "2" para "1". Isto pode ser representado assim:

Exemplo #9 Removendo a atribuição de *$a*

```
(refcount=1, is_ref=1)=array (
   0 => (refcount=1, is_ref=0)='one',
   1 => (refcount=1, is_ref=1)=...
)
```

Ou graficamente



Problemas na Limpeza

Embora não haja mais um símbolo em nenhum escopo apontando para esta estrutura, ela não pode ser limpa porque o elemento "1" do array ainda aponta para este mesmo array. Como não há símbolo externo apontando para ela, não há como um usuário limpar esta estrutura; e aí acontece o vazamento de memória. Felizmente, o PHP irá limpar esta estrutura de dados no final da requisição, mas até lá, ela irá ocupar um espaço valioso na memória. Esta situação ocorre frequentemente quando se está implementando algoritmos de interpretação ou outros onde existe um elemento "filho" apontando de volta para um elemento "pai". A mesma situação também pode com certeza ocorrer com objetos, onde na verdade existe mais probabilidade de ocorrer, já que objetos são sempre implicitamente usados "por referência".

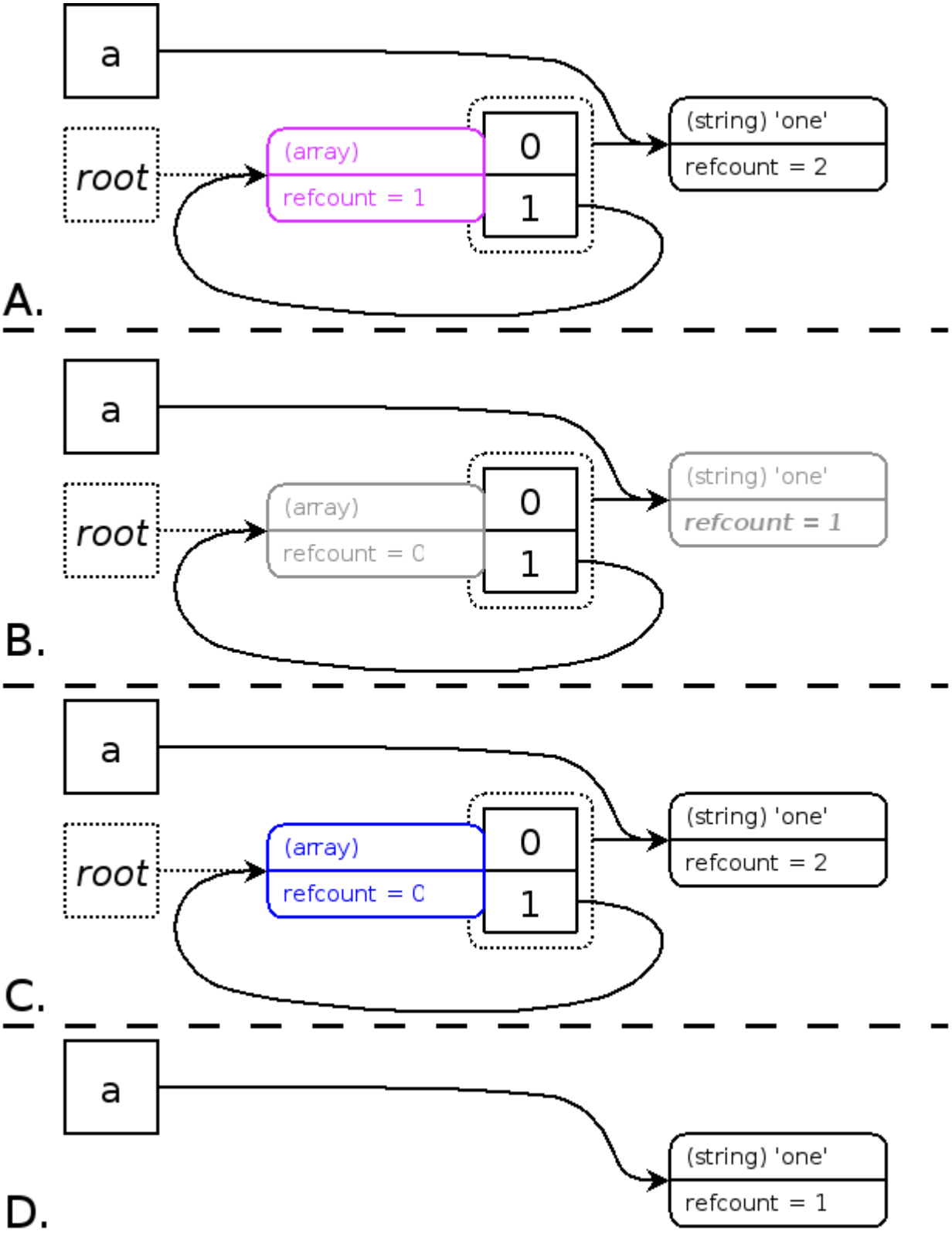
Isso pode não ser um problema quanto acontecer somente uma ou duas vezes, mas se houver milhares, ou até milhões dessas perdas de memória, obviamente começa a ser um problema. É especialmente

problemático em scripts de execução longa, como daemons onde a requisição basicamente nunca termina, ou em grandes conjuntos de testes de unidades. Este último já causou problemas durante a execução de testes de unidades para o componente Template da bilioteca eZ Components. Em alguns casos, era necessário mais de 2GB de memória, que o servidor de testes não tinha.

Ciclos de Coleta

Tradicionalmente, mecanismos de memória de contagem de referência, como os usados anteriormente pelo PHP, falham ao lidar com vazamentos de memória de referência circular; entretanto, desde a versão 5.3.0, o PHP implementa o algoritmo síncrono do artigo » Concurrent Cycle Collection in Reference Counted Systems que lida com este problema.

Uma explicação completa de como o algoritmo funciona estaria um pouco além do escopo desta seção, mas o básico é explicado aqui. Primeiramente, deve-se estabelecer algumas regras gerais. Se um "refcount" é incrementado, ele ainda está em uso e, portanto, não é lixo. Se o refcount é reduzido e alcança zero, o zval pode ser liberado. Isso significa que os ciclos de coleta somente podem ser criados quando um argumento "refcount" é reduzido para um valor diferente de zero. Adicionalmente, em um ciclo de coleta, é possível descobrir quais partes são lixo, verificando se é possível reduzir seus "refcounts" em uma unidade, e então observando quais dos zvals têm um "refcount" diferente de zero.



Para evitar chamadas de verificação de ciclos de coleta com qualquer redução possível de um refcount, o algoritmo em vez disso coloca todas as raízes (zvals) possíveis no "buffer de raízes" (tornando-os "roxos"). Ele também certifica que cada raiz possível chegue ao buffer apenas uma vez. Apenas quando o buffer de raízes está cheio é que o mecanismo de coleta se inicia para todos os diferentes zvals contidos. Veja o passo A na figura acima.

No passo B, o algoritmo executa uma pesquisa em profundidade em todas as raízes possíveis para reduzir em um os refcounts de cada zval que ele encontra, certificando-se de não reduzir um refcount no mesmo zval duas vezes (marcando-os de "cinza"). No passo C, o algoritmo novamente executa uma pesquisa em

profundidade a partir de cada nó de raiz, para verificar o refcount de cada zval de novo. Se ele encontra o valor zero, o zval é marcado de "branco" (azul na figura). Se ele for maior que zero, ele reverte a redução do refcount em uma unidade com uma pesquisa em profundidade daquele ponto em diante, e eles são marcados de "preto" novamente. No último passo (D), o algoritmo percorre o buffer de raízes removendo as raízes de zval de lá e, ao mesmo tempo, verifica quais zvals foram marcados de "branco" no passo anterior. Cada zval marcado de "branco" será liberado da memória.

Agora que há um entendimento básico de como o algoritmo funciona, vejamos como isto se integra com o PHP. Por padrão, o coletor de lixo do PHP fica habilitado. Existe, porém uma configuração do *php.ini* que permite mudar isso: zend.enable_gc.

Quando o coletor de lixo é habilitado, o algoritmo de pesquisa de ciclos como descrito acima é executado toda vez que o buffer ficar cheio. O buffer de raízes tem um tamanho fixo de 10.000 raízes possíveis (embora isso possa ser alterado mudando-se a constante **`GC_THRESHOLD_DEFAULT`** em `Zend/zend_gc.c` no código-fonte do PHP, e recompilando-o). Quando o coletor de lixo é desabilitado, o algoritmo de pesquisa de ciclos nunca será executado. Entretanto, possíveis raízes serão sempre registradas no buffer de raízes, não importando se o mecanismo de coleta de lixo tenha sido ou não habilitado com esta configuração.

Se o buffer de raízes ficar cheio de raízes possíveis enquanto o mecanismo de coleta de lixo está desabilitado, as possíveis raízes adicionais simplesmente não serão registradas. Essas raízes não registradas nunca serão analisadas pelo algoritmo. Se eles fossem parte de um ciclo de referência circular, eles nunca seriam eliminados e iriam criar um vazamento de memória.

O motivo pelo qual as raízes possíveis são registradas mesmo se o mecanismo for desabilitado é porque é mais rápido registrar raízes possíveis do que ter que verificar se o mecanismo está ligado toda vez que uma raiz possível puder ser encontrada. O próprio mecanismo de coleta e análise de lixo, no entanto, pode levar um tempo considerável.

Além de mudar a configuração zend.enable_gc, também é possível habilitar e desabilitar o mecanismo de coleta de lixo chamando-se gc_enable() ou gc_disable() respectivamente. Chamar estas funções tem o mesmo efeito de ligar ou desligar o mecanismo com a configuração. Também é possível forçar a coleta de ciclos mesmo se o buffer de raízes possíveis não estiver cheio. Para isto, pode-se usar a função gc_collect_cycles(). Esta função retorna quantos ciclos foram coletados pelo algoritmo.

A razão por trás da possibilidade do próprio usuário ligar e desligar o mecanismo, e iniciar a coleta de ciclos, é que algumas partes de aplicações podem ser altamente sensíveis a tempo de execução. Nesses casos, pode não ser desejado que o mecanismo de coleta inicie. Obviamente, desligar o coletor de lixo para certas partes de uma aplicação cria o risco de gerar vazamentos de memória porque algumas raízes possíveis podem não caber no buffer limitado. Portanto, provavelmente é mais sábio chamar a função gc_collect_cycles() logo antes de chamar a função gc_disable() para liberar a memória que poderia ser perdida através de raízes possíveis que estariam já registradas no buffer. Isso então leva a um buffer vazio para que haja mais espaço para armazenar raízes possíveis enquanto o mecanismo de ciclos de coleta está desligado.

Considerações de Desempenho

Já foi mencionado na seção anterior que simplesmente coletar as raízes possíveis tem um impacto muito pequeno em desempenho, mas isso quando compara-se o PHP 5.2 com o PHP 5.3. Embora o registro de raízes possíveis comparado ao não registro, como no PHP 5.2, seja mais lento, outras mudanças em tempo de execução no PHP 5.3 evitam que esta perda particular de desempenho apareça.

Existem duas grandes áreas onde o desempenho é afetado. A primeira é o uso reduzido de memória, e a segunda é o atraso em tempo de execução quando o mecanismo de coleta de lixo faz suas limpezas de

memória. Estes dois problemas serão mostrados a seguir.

Uso Reduzido de Memória

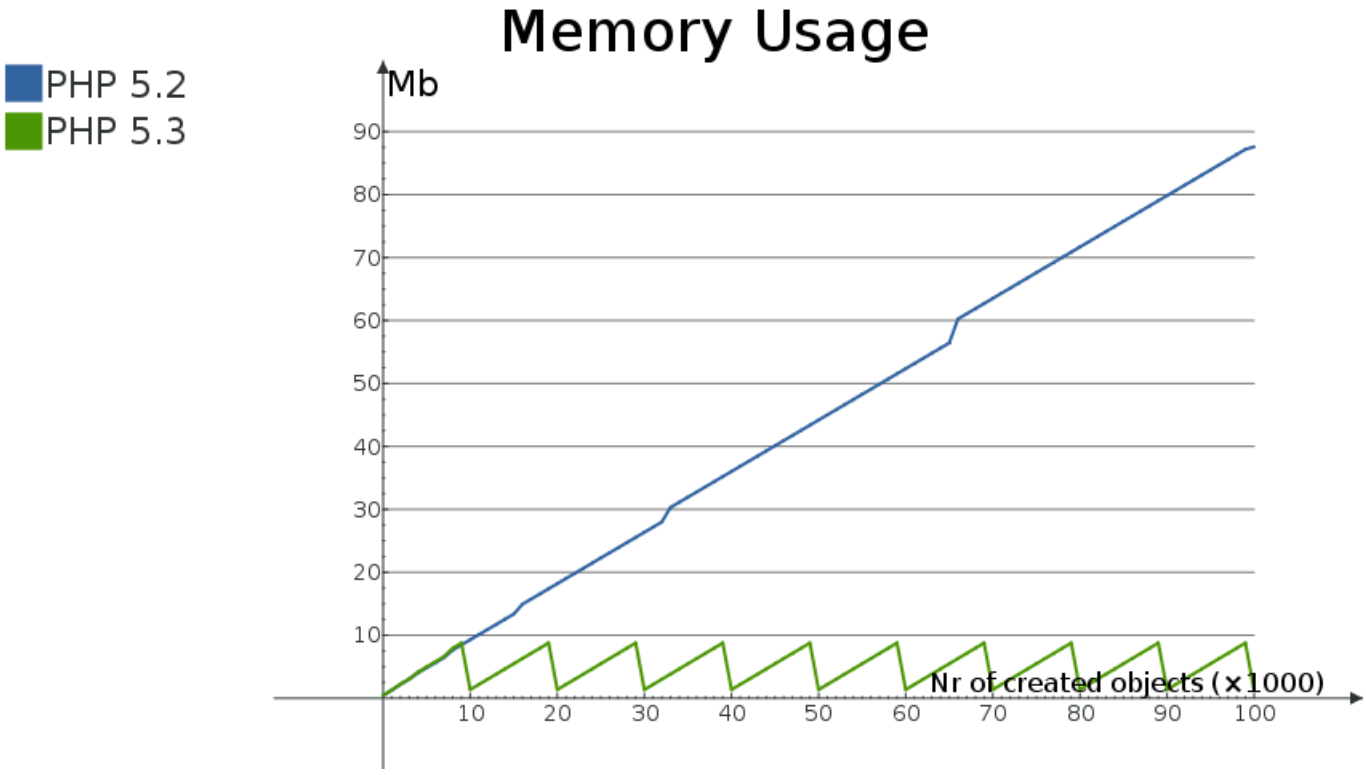
Primeiramente, o grande motivo pelo qual o mecanismo de coleta de lixo existe é para reduzir o uso de memória através de limpeza de variáveis em referência circular assim que os pré-requisitos são preenchidos. Na implementação do PHP, isso acontece assim que o buffer de raízes fica cheio, ou quando a função gc_collect_cycles() é chamada. No gráfico abaixo, é mostrado o uso de memória do script a seguir, tanto no PHP 5.2 quanto no PHP 5.3, excluindo a memória básica que o próprio PHP usa quando se inicia.

Exemplo #1 Exemplo de uso de memória

```php
<?php
class Foo
{
    public $var = '3.14159265359';
    public $self;
}

$baseMemory = memory_get_usage();

for ( $i = 0; $i <= 100000; $i++ )
{
    $a = new Foo;
    $a->self = $a;
    if ( $i % 500 === 0 )
    {
        echo sprintf( '%8d: ', $i ), memory_get_usage() - $baseMemory, "\n";
    }
}
?>
```



Neste exemplo bem acadêmico, está sendo criado um objeto no qual uma propriedade é definida para apontar para o próprio objeto. Quando a variável *$a* no script é re-atribuída na iteração seguinte do loop, um

vazamento de memória normalmente iria acontecer. Neste caso, dois contêineres são vazados (o zval objeto e o zval propriedade), mas apenas uma raiz possível é encontrada: a variável que perdeu a atribuição. Quando o buffer de raízes está cheio depois de 10.000 iterações (com um total de 10.000 raízes possíveis), o mecanismo de coleta de lixo entra e libera a memória associada com essas raízes possíveis. Isto pode ser visto claramente com o gráfico irregular de uso de memória para o PHP 5.3. Depois de 10.000 iterações, o mecanismo entre e libera a memória associada com as variáveis com referência circular.. O mecanismo em si não tem muito trabalho neste exemplo, porque a estrutura que é vazada é extremamente simples. Pelo diagrama, pode-se verificar que o uso de memória no PHP 5.3 é de aproximadamente 9Mb, enquanto que no PHP 5.2 o uso de memória continua crescendo.

Atraso em Tempo de Execução

A segunda área onde o mecanismo de coleta de lixo influencia o desempenho é o tempo gasto quando o mecanismo entra para liberar a memória "vazada". Para verificar quanto é este tempo, o script anterior foi minimamente modificado para permitir um número maior de iterações e a remoção dos números de uso de memória intermediária. O segundo script está apresentado a seguir:

Exemplo #2 Influência do Coletor de Lixo no desempenho

```
<?php
class Foo
{
    public $var = '3.14159265359';
    public $self;
}

for ( $i = 0; $i <= 1000000; $i++ )
{
    $a = new Foo;
    $a->self = $a;
}

echo memory_get_peak_usage(), "\n";
?>
```

O script será executado duas vezes, uma com a configuração zend.enable_gc habilitada, e outra desabilitada:

Exemplo #3 Executando o script acima

```
time php -dzend.enable_gc=0 -dmemory_limit=-1 -n example2.php
# e
time php -dzend.enable_gc=1 -dmemory_limit=-1 -n example2.php
```

Em uma máquina específica, o primeiro comando parece levar 10.7 segundos, enquanto que o segundo leva 11.4 segundos. Isto é um atraso de aproximadamente 7%. Entretanto, a quantidade máxima de memória usada pelo script é reduzida em 98%, de 931Mb para 10Mb. Esta referência não é muito científica, ou mesmo representativa para aplicações do mundo real, mas demonstra os benefícios de uso de memória que este mecanismo de coleta de lixo fornece. A boa notícia é que este atraso é sempre de 7% para este script particular, enquanto que as capacidades de redução de memória economizam mais e mais memória quando referências circulares adicionais são encontradas durante a execução do script.

Estatísticas de GC Internas do PHP

É possível obter um pouco mais de informação sobre como o o mecanismo de coleta de lixo é executado no PHP. Mas para fazer isto, deve-se recompilar o PHP para habilitar o benchmark e o código de coleta de dados. Deve-se definir a variável de ambiente `CFLAGS` para `-DGC_BENCH=1` antes de executar `./configure` com as opções desejadas. A sequência a seguir deve fazer este truque:

Exemplo #4 Recompilando o PHP para habilitar o benchmarking de GC

```
export CFLAGS=-DGC_BENCH=1
./config.nice
make clean
make
```

Quando o exemplo acima for executado novamente com o novo binário do PHP, o resultado abaixo será visualizado assim que o PHP terminar a execução:

Exemplo #5 Estatísticas GC

```
GC Statistics
-------------
Runs:               110
Collected:          2072204
Root buffer length: 0
Root buffer peak:   10000

      Possible            Remove from  Marked
        Root    Buffered     buffer     grey
      --------  --------  -----------  ------
ZVAL   7175487   1491291    1241690   3611871
ZOBJ  28506264   1527980     677581   1025731
```

As estatísticas mais informativas são mostradas no primeiro bloco. Pode-se ver aqui que o mecanismo de coleta de lixo foi executado 110 vezes, e no total, mais de 2 milhões de alocações de memória foram liberadas durante estas 110 execuções. Assim que o mecanismo tenha sido executado pelo menos uma vez, o "Root buffer peak" (pico do buffer de raízes) será sempre 10.000.

Conclusão

Em geral, o coletor de lixo no PHP irá causar um atraso apenas quando o algoritmo de coleta realmente for executado, enquanto que em scripts normais (menores), não deve haver nenhum prejuízo no desempenho.

Entretanto, em casos onde o mecanismo de coleta é executado em scripts normais, a redução de memória que ele vai proporcionar irá permitir que mais desses scripts possam ser executados ao mesmo tempo no servidor, já que a quantidade de memória usada no total não será muito grande.

Os benefícios são mais aparentes para scripts de longa execução, como os scripts de conjunto de testes ou daemons. Adicionalmente, para aplicações » PHP-GTK que geralmente tendem a rodar por mais tempo que scripts para a Web, o novo mecanismo deve fazer uma diferença considerável em relação a vazamentos de memória que insistem em acontecer.

Instrumentação dinâmica DTrace

Índice

Introdução ao PHP e DTrace

DTrace é um framework de instrumentação com alta disponibilidade e pequena sobrecarga, disponível em um número de plataformas incluindo Solaris, macOS, Oracle Linux e BSD. DTrace pode instrumentar comportamento do sistema operacional e execução de programa de usuário. Ele pode exibir valores de argumentos e ser usado para inferir estatísticas de desempenho. As sondas são monitoradas nos scripts criados pelo usuário escritos na linguagem própria do DTrace (D). Isto permite uma análise eficiente de pontos de dados.

Sondas PHP que não estão monitoradas ativamente por um script D do usuário não contêm código instrumentado, então não há degradação de desempenho durante a execução normal da aplicação. Sondas que estão sendo monitoradas incorrem em uma sobrecarga pequena o suficiente para permitir monitoramento pelo DTrace em sistemas ativos em fase de produção.

PHP incorpora sondas com "Instrumentação Estaticamente Definida no Nível de Usuário" (USDT em inglês) que são ativadas em tempo de execução. Por exemplo, quando um script D está monitorando a sonda de `function-entry` do PHP, a cada momento que uma função do PHP é chamada, esta sonda é disparada e o código de ação do script D associado é executado. Este código de ação poderia, por exemplo, imprimir argumentos de sonda como a localização do arquivo-fonte da função PHP. Ou a ação poderia agregar dados como o número de vezes que cada função é chamada.

Apenas as sondas USDT do PHP são descritas aqui. Refira-se à literatura externa do Dtrace, tanto geral quanto específica do sistema operacional, para entender como o DTrace pode ser usado para instrumentar funções arbitrárias, e como ele pode ser usado para instrumentar comportamento do sistema operacional. Note que nem todas as características do DTrace estão disponíveis em todas as implementações.

As sondas estáticas do Dtrace no PHP podem alternativamente ser utilizadas com a ferramenta SystemTap de algumas distribuições Linux.

Usando PHP e DTrace

O PHP pode ser configurado com sondas estáticas do DTrace em plataformas que suportam a Instrumentação Dinâmica do DTrace.

Configurando o PHP para Sondas Estáticas DTrace

Refira-se à documentação externa específica da plataforma para habilitar o suporte do DTrace no sistema operacional. Por exemplo, no Oracle Linux inicialize um kernel UEK3 e rode os seguintes comandos:

```
# modprobe fasttrap
# chmod 666 /dev/dtrace/helper
```

Ao invés de usar `chmod`, pode-se alternativamente usar uma regra de pacote ACL para limitar o acesso a um dispositivo para um usuário específico.

Construa o PHP com o parâmetro de configuração `--enable-dtrace`:

```
# ./configure --enable-dtrace ...
# make
# make install
```

Isto faz com que as sondas estáticas estejam disponíveis no núcleo do PHP. Quaisquer extensões PHP que fornecem seus próprios sondas devem ser construídas separadamente como extensões compartilhadas.

Para habilitar as sondas, defina a variável de ambiente **USE_ZEND_DTRACE=1** para os processos alvos do PHP.

Sondas Estáticos DTrace no Núcleo do PHP

As sondas estáticos a seguir estão disponíveis no PHP

Nome da Sonda	Descrição da Sonda	Argumentos da Sonda
`request-startup`	Dispara quando uma requisição inicia.	char **file*, char **request_uri*, char **request_method*
`request-shutdown`	Dispara quando uma requisição termina.	char **file*, char **request_uri*, char **request_method*
`compile-file-entry`	Dispara quando a compilação de um script inicia.	char **compile_file*, char **compile_file_translated*
`compile-file-return`	Dispara quando a compilação de um script termina.	char **compile_file*, char **compile_file_translated*
`execute-entry`	Dispara quando um array de opcode está para ser executado. Por exemplo, ele dispara em chamadas de função, de includes e em continuação de geradores.	char **request_file*, int *lineno*
`execute-return`	Dispara depois da execução de um array de opcode.	char **request_file*, int *lineno*
`function-entry`	Dispara quando o motor do PHP entra em uma chamada de função ou método.	char **function_name*, char **request_file*, int *lineno*, char **classname*, char **scope*
`function-return`	Dispara quando o motor do PHP retorna de uma chamada de função ou método.	char **function_name*, char **request_file*, int *lineno*, char **classname*, char **scope*
`exception-thrown`	Dispara quando uma exceção é disparada.	char **classname*
`exception-caught`	Dispara quando uma exceção é capturada.	char **classname*
`error`	Dispara quando um erro ocorre, independente do nível de error_reporting.	char **errormsg*, char **request_file*, int *lineno*

Extensões de PHP podem também conter sondas estáticas adicionais.

Listando As Sondas Estáticas DTrace no PHP

Para listar as sondas disponíveis, inicie um processo PHP e então execute:

```
# dtrace -l
```

A saída será similar a:

```
   ID   PROVIDER            MODULE                          FUNCTION NAME
   [ . . . ]
    4   php15271               php               dtrace_compile_file compile-file-entry
    5   php15271               php               dtrace_compile_file compile-file-return
    6   php15271               php                        zend_error error
    7   php15271               php  ZEND_CATCH_SPEC_CONST_CV_HANDLER exception-caught
    8   php15271               php     zend_throw_exception_internal exception-thrown
    9   php15271               php                  dtrace_execute_ex execute-entry
   10   php15271               php           dtrace_execute_internal execute-entry
   11   php15271               php                  dtrace_execute_ex execute-return
   12   php15271               php           dtrace_execute_internal execute-return
   13   php15271               php                  dtrace_execute_ex function-entry
   14   php15271               php                  dtrace_execute_ex function-return
   15   php15271               php              php_request_shutdown request-shutdown
   16   php15271               php               php_request_startup request-startup
```

Os valores da coluna Provider consistem de `php` mais o id do processo PHP sendo executado.

Se o servidor web Apache estiver rodando, o módulo poderia ser, por exemplo, *libphp5.so*, e haveria múltiplos blocos de listagens, um para cada processo Apache em execução.

A coluna Function refere-se aos nomes das funções da implementação interna do PHP, em C, onde cada fornecedor está localizado.

Se um processo PHP não estiver em execução, então nenhuma sonda PHP será mostrada.

Exemplo de DTrace com PHP

Este exemplo mostra o básico da linguagem D de scripts DTrace.

Exemplo #1 Arquivo *all_probes.d* para instrumentar todas as Sondas Estáticas do PHP com DTrace

```
#!/usr/sbin/dtrace -Zs

#pragma D option quiet

php*:::compile-file-entry
{
    printf("PHP compile-file-entry\n");
    printf("  compile_file              %s\n", copyinstr(arg0));
    printf("  compile_file_translated   %s\n", copyinstr(arg1));
}

php*:::compile-file-return
{
    printf("PHP compile-file-return\n");
    printf("  compile_file              %s\n", copyinstr(arg0));
    printf("  compile_file_translated   %s\n", copyinstr(arg1));
}

php*:::error
{
    printf("PHP error\n");
    printf("  errormsg                  %s\n", copyinstr(arg0));
    printf("  request_file              %s\n", copyinstr(arg1));
    printf("  lineno                    %d\n", (int)arg2);
```

```
}

php*:::exception-caught
{
    printf("PHP exception-caught\n");
    printf("  classname                %s\n", copyinstr(arg0));
}

php*:::exception-thrown
{
    printf("PHP exception-thrown\n");
    printf("  classname                %s\n", copyinstr(arg0));
}

php*:::execute-entry
{
    printf("PHP execute-entry\n");
    printf("  request_file             %s\n", copyinstr(arg0));
    printf("  lineno                   %d\n", (int)arg1);
}

php*:::execute-return
{
    printf("PHP execute-return\n");
    printf("  request_file             %s\n", copyinstr(arg0));
    printf("  lineno                   %d\n", (int)arg1);
}

php*:::function-entry
{
    printf("PHP function-entry\n");
    printf("  function_name            %s\n", copyinstr(arg0));
    printf("  request_file             %s\n", copyinstr(arg1));
    printf("  lineno                   %d\n", (int)arg2);
    printf("  classname                %s\n", copyinstr(arg3));
    printf("  scope                    %s\n", copyinstr(arg4));
}

php*:::function-return
{
    printf("PHP function-return\n");
    printf("  function_name            %s\n", copyinstr(arg0));
    printf("  request_file             %s\n", copyinstr(arg1));
    printf("  lineno                   %d\n", (int)arg2);
    printf("  classname                %s\n", copyinstr(arg3));
    printf("  scope                    %s\n", copyinstr(arg4));
}

php*:::request-shutdown
{
    printf("PHP request-shutdown\n");
    printf("  file                     %s\n", copyinstr(arg0));
    printf("  request_uri              %s\n", copyinstr(arg1));
    printf("  request_method           %s\n", copyinstr(arg2));
}

php*:::request-startup
{
```

```
    printf("PHP request-startup\n");
    printf("  file                      %s\n", copyinstr(arg0));
    printf("  request_uri               %s\n", copyinstr(arg1));
    printf("  request_method            %s\n", copyinstr(arg2));
}
```

Este script usa a opção `-z` para a aplicação *dtrace*, permitindo que ela seja executada quando não há processo PHP em execução. Se esta opção fosse omitida, o script iria terminar imediatamente porque ele saberia que nenhuma das sondas a serem monitoradas existiriam.

O script instrumenta todos os pontos de sondas estáticas do núcleo do PHP durante toda a duração de um script PHP em execução. Execute o script D:

```
# ./all_probes.d
```

Executa um script PHP ou aplicação. O script D de monitoramento terá como saída os argumentos de cada uma das sondas quando for disparado.

Quando o monitoramento estiver completo, o script D pode ser terminado com `CTRL+C`.

Em máquinas com múltiplas CPUs, pode parecer que as sondas estão em ordem não sequencial. Isto depende de qual CPU está processando as sondas, e como as threads migram através dos CPUs. Mostrar os timestamps das sondas irá ajudar a reduzir a confusão, por exemplo:

```
php*:::function-entry
{
      printf("%lld: PHP function-entry ", walltimestamp);
      [ . . .]
}
```

Veja também

- Instrumentação Dinâmica OCI8 e DTrace

Usando SystemTap com Sondas Estáticas DTrace

Em algumas distribuições Linux, o utilitário SystemTap de instrumentação pode ser usado para instrumentar sondas estáticas do PHP. Isso está disponível no PHP 5.4.20 e PHP 5.5.

Instalando PHP com SystemTap

Instale o pacote de desenvolvimento do SystemTap, SDT:

```
# yum install systemtap-sdt-devel
```

Instale o PHP com as sondas DTrace habilitadas:

```
# ./configure --enable-dtrace ...
# make
```

Listando as Sondas Estáticas com SystemTap

As sondas estáticas em PHP podem ser listados usando *stap*:

```
# stap -l 'process.provider("php").mark("*")' -c 'sapi/cli/php -i'
```

A saída será:

```
process("sapi/cli/php").provider("php").mark("compile__file__entry")
process("sapi/cli/php").provider("php").mark("compile__file__return")
process("sapi/cli/php").provider("php").mark("error")
process("sapi/cli/php").provider("php").mark("exception__caught")
process("sapi/cli/php").provider("php").mark("exception__thrown")
process("sapi/cli/php").provider("php").mark("execute__entry")
process("sapi/cli/php").provider("php").mark("execute__return")
process("sapi/cli/php").provider("php").mark("function__entry")
process("sapi/cli/php").provider("php").mark("function__return")
process("sapi/cli/php").provider("php").mark("request__shutdown")
process("sapi/cli/php").provider("php").mark("request__startup")
```

Exemplo de SystemTap com PHP

Exemplo #1 *all_probes.stp* para instrumentação de todos as Sondas Estáticas do PHP com SystemTap

```
probe process("sapi/cli/php").provider("php").mark("compile__file__entry") {
    printf("Probe compile__file__entry\n");
    printf("  compile_file %s\n", user_string($arg1));
    printf("  compile_file_translated %s\n", user_string($arg2));
}
probe process("sapi/cli/php").provider("php").mark("compile__file__return") {
    printf("Probe compile__file__return\n");
    printf("  compile_file %s\n", user_string($arg1));
    printf("  compile_file_translated %s\n", user_string($arg2));
}
probe process("sapi/cli/php").provider("php").mark("error") {
    printf("Probe error\n");
    printf("  errormsg %s\n", user_string($arg1));
    printf("  request_file %s\n", user_string($arg2));
    printf("  lineno %d\n", $arg3);
}
probe process("sapi/cli/php").provider("php").mark("exception__caught") {
    printf("Probe exception__caught\n");
    printf("  classname %s\n", user_string($arg1));
}
probe process("sapi/cli/php").provider("php").mark("exception__thrown") {
    printf("Probe exception__thrown\n");
    printf("  classname %s\n", user_string($arg1));
}
probe process("sapi/cli/php").provider("php").mark("execute__entry") {
    printf("Probe execute__entry\n");
    printf("  request_file %s\n", user_string($arg1));
    printf("  lineno %d\n", $arg2);
}
probe process("sapi/cli/php").provider("php").mark("execute__return") {
    printf("Probe execute__return\n");
    printf("  request_file %s\n", user_string($arg1));
    printf("  lineno %d\n", $arg2);
}
probe process("sapi/cli/php").provider("php").mark("function__entry") {
    printf("Probe function__entry\n");
```

```
    printf("  function_name %s\n", user_string($arg1));
    printf("  request_file %s\n", user_string($arg2));
    printf("  lineno %d\n", $arg3);
    printf("  classname %s\n", user_string($arg4));
    printf("  scope %s\n", user_string($arg5));
}
probe process("sapi/cli/php").provider("php").mark("function__return") {
    printf("Probe function__return: %s\n", user_string($arg1));
    printf(" function_name %s\n", user_string($arg1));
    printf("  request_file %s\n", user_string($arg2));
    printf("  lineno %d\n", $arg3);
    printf("  classname %s\n", user_string($arg4));
    printf("  scope %s\n", user_string($arg5));
}
probe process("sapi/cli/php").provider("php").mark("request__shutdown") {
    printf("Probe request__shutdown\n");
    printf("  file %s\n", user_string($arg1));
    printf("  request_uri %s\n", user_string($arg2));
    printf("  request_method %s\n", user_string($arg3));
}
probe process("sapi/cli/php").provider("php").mark("request__startup") {
    printf("Probe request__startup\n");
    printf("  file %s\n", user_string($arg1));
    printf("  request_uri %s\n", user_string($arg2));
    printf("  request_method %s\n", user_string($arg3));
}
```

O script acima irá instrumentar todos os pontos de sondas estáticas do núcleo do PHP durante toda a duração de um script PHP em execução:

```
# stap -c 'sapi/cli/php test.php' all_probes.stp
```

Usando bpftrace com Sondas Estáticas DTrace do PHP

Em distribuições Linux com um kernel que suporta eBPF, o utilitário bpftrace pode se conectar diretamente às sondas USDT DTrace do PHP, sem precisar do SystemTap.

Instalando o bpftrace

Instale o bpftrace usando o gerenciador de pacotes da distribuição. Por exemplo, no Oracle Linux, RHEL ou Fedora:

```
# dnf install bpftrace
```

Ou, no Debian ou Ubuntu:

```
# apt install bpftrace
```

Os exemplos abaixo assumem que o binário PHP alvo está instalado em */usr/bin/php*.

As mesmas sondas USDT também são expostas por outras SAPIs construídas a partir da mesma árvore de fontes, então o alvo da sonda pode ser, ao invés disso, o módulo do Apache (*libphp.so*) ou o binário do FastCGI Process Manager (*php-fpm*); substitua o caminho apropriado ou conecte-se pelo PID com `-p` conforme necessário.

Certifique-se de que o binário alvo foi construído com o DTrace e que a variável de ambiente está configurado corretamente. Veja Configurando o PHP para Sondas Estáticas DTrace para detalhes.

As sondas estáticas do PHP podem ser listadas usando **bpftrace**:

```
# bpftrace -l 'usdt:/usr/bin/php:php:*'
```

A saída é:

```
usdt:/usr/bin/php:php:compile__file__entry
usdt:/usr/bin/php:php:compile__file__return
usdt:/usr/bin/php:php:error
usdt:/usr/bin/php:php:exception__caught
usdt:/usr/bin/php:php:exception__thrown
usdt:/usr/bin/php:php:execute__entry
usdt:/usr/bin/php:php:execute__return
usdt:/usr/bin/php:php:function__entry
usdt:/usr/bin/php:php:function__return
usdt:/usr/bin/php:php:request__shutdown
usdt:/usr/bin/php:php:request__startup
```

Exemplo #1 *all_probes.bt* para instrumentação de todas as Sondas Estáticas do PHP com bpftrace

```
#!/usr/bin/env bpftrace

usdt:/usr/bin/php:php:compile__file__entry
{
    printf("Probe compile__file__entry\n");
    printf("  compile_file %s\n", str(arg0));
    printf("  compile_file_translated %s\n", str(arg1));
}
usdt:/usr/bin/php:php:compile__file__return
{
    printf("Probe compile__file__return\n");
    printf("  compile_file %s\n", str(arg0));
    printf("  compile_file_translated %s\n", str(arg1));
}
usdt:/usr/bin/php:php:error
{
    printf("Probe error\n");
    printf("  errormsg %s\n", str(arg0));
    printf("  request_file %s\n", str(arg1));
    printf("  lineno %d\n", (int32)arg2);
}
usdt:/usr/bin/php:php:exception__caught
{
    printf("Probe exception__caught\n");
    printf("  classname %s\n", str(arg0));
}
usdt:/usr/bin/php:php:exception__thrown
{
    printf("Probe exception__thrown\n");
    printf("  classname %s\n", str(arg0));
}
usdt:/usr/bin/php:php:execute__entry
{
```

```
    printf("Probe execute__entry\n");
    printf("  request_file %s\n", str(arg0));
    printf("  lineno %d\n", (int32)arg1);
}
usdt:/usr/bin/php:php:execute__return
{
    printf("Probe execute__return\n");
    printf("  request_file %s\n", str(arg0));
    printf("  lineno %d\n", (int32)arg1);
}
usdt:/usr/bin/php:php:function__entry
{
    printf("Probe function__entry\n");
    printf("  function_name %s\n", str(arg0));
    printf("  request_file %s\n", str(arg1));
    printf("  lineno %d\n", (int32)arg2);
    printf("  classname %s\n", str(arg3));
    printf("  scope %s\n", str(arg4));
}
usdt:/usr/bin/php:php:function__return
{
    printf("Probe function__return\n");
    printf("  function_name %s\n", str(arg0));
    printf("  request_file %s\n", str(arg1));
    printf("  lineno %d\n", (int32)arg2);
    printf("  classname %s\n", str(arg3));
    printf("  scope %s\n", str(arg4));
}
usdt:/usr/bin/php:php:request__shutdown
{
    printf("Probe request__shutdown\n");
    printf("  file %s\n", str(arg0));
    printf("  request_uri %s\n", str(arg1));
    printf("  request_method %s\n", str(arg2));
}
usdt:/usr/bin/php:php:request__startup
{
    printf("Probe request__startup\n");
    printf("  file %s\n", str(arg0));
    printf("  request_uri %s\n", str(arg1));
    printf("  request_method %s\n", str(arg2));
}
```

O script acima irá instrumentar todos os pontos de sondas estáticas do núcleo do PHP durante toda a duração de um script PHP em execução. O bpftrace requer privilégios de root:

```
# USE_ZEND_DTRACE=1 bpftrace -c '/usr/bin/php test.php' all_probes.bt
```

Para instrumentar um processo PHP já em execução (por exemplo, um worker do *php-fpm* ou um processo Apache carregando *libphp.so*), conecte-se pelo PID:

```
# bpftrace -p $PID all_probes.bt
```

O caminho alvo `usdt:` no script deve corresponder ao binário do processo em execução; ajuste `usdt:/usr/bin/php` para o binário do *php-fpm* ou para *libphp.so* conforme apropriado.

Perguntas Mais Frequentes

Informações Gerais

Essa seção trata das questões mais gerais sobre o PHP: o que é e o que faz.

1. O que é o PHP?
2. O que a sigla PHP significa?
3. Qual é a relação entre as versões?
4. Posso executar várias versões do PHP ao mesmo tempo?
5. Acho que encontrei um bug! A quem eu devo dizer?

O que é o PHP?

> Do prefácio do manual:
>
> PHP é uma linguagem de script embutida no HTML. Muito da sua sintaxe é emprestada de C, Java e Perl com algumas características específicas do PHP adicionadas. O objetivo da linguagem é permitir que desenvolvedores web escrevam páginas geradas dinamicamente de forma rápida.

O que a sigla PHP significa?

> PHP significa *PHP: Hypertext Preprocessor*. Isso confunde muitas pessoas porque a primeira palavra do acrônimo é o acrônimo. Esse tipo de acrônimo é chamado de acrônimo recursivo. Para mais informações, os curiosos podem visitar o » Dicionário On-Line Gratuito de Computação ou o » item na Wikipedia sobre acrónimos recursivos.

Qual é a relação entre as versões?

> PHP/FI 2.0 é uma versão mais antiga e não mais suportada do PHP. PHP 3 é o sucessor do PHP/FI 2.0 e é bem melhor. PHP 8 é a geração atual do PHP, que usa a Zend Engine 4, a qual, entre outras coisas, oferece muitas funcionalidades adicionais de POO.

Posso executar várias versões do PHP ao mesmo tempo?

> Sim. Veja o arquivo *INSTALL* que vem incluso na distribuição dos fontes do PHP.

Acho que encontrei um bug! A quem eu devo dizer?

> Você deve ir ao Banco de Dados de Bugs do PHP e assegurar-se de que o bug não é conhecido. Se você não encontrá-lo no banco, use o formulário para relatar o bug. É importante usar o banco de bugs ao invés de apenas enviar um e-mail para uma das listas de discussão porque o bug terá um número de rastreamento atribuído a ele e será possível voltar depois e checar o status do bug. O banco de bugs pode ser encontrado em » https://github.com/php/php-src/issues.

Listas de Discussão

Essa seção responde algumas perguntas relacionadas a como entrar em contato com a comunidade PHP. A melhor maneira é através das listas de discussão via e-mail.

1. Existe alguma lista de discussão do PHP?
2. Existem outras comunidades?
3. Posso iniciar minha própria lista de discussões de PHP?
4. Ajuda! Não consigo me inscrever ou sair de nenhuma das listas de discussão!
5. Existe um histórico das listas de discussão em algum lugar?
6. O que posso perguntar na lista de discussão?
7. Que informação devo incluir quando faço perguntas na lista?

Existe alguma lista de discussão do PHP?

É claro! Existem várias listas de discussão sobre vários assuntos. Uma listagem das principais listas de discussão pode ser encontrada na nossa » listas de discussão por email.

A lista de discussão mais geral é `php-general`. Para participar, mande um e-mail em branco para » php-general+subscribe@lists.php.net. Para sair da lista, mande um e-mail para » php-general+unsubscribe@lists.php.net.

Você pode se inscrever e sair das listas usando nossa interface web disponível em » listas de discussão, além de instruções incluídas no rodapé de cada e-mail da lista.

Existem outras comunidades?

Existem incontáveis comunidades voltadas ao PHP ao redor do mundo. Nós temos links, para algumas delas (e informação de como aprender mais) em nossa página de » Suporte

Posso iniciar minha própria lista de discussões de PHP?

Com certeza! Aliás, nós não só permitimos como encorajamos você a iniciar uma lista de discussões! Ajudar os outros dividindo seu conhecimento e experiência com o PHP amplia o crescimento da comunidade e o desenvolvimento global do PHP, além da própria linguagem.

Ajuda! Não consigo me inscrever ou sair de nenhuma das listas de discussão!

A causa mais comum para as pessoas terem dificuldades em sair de nossas listas de discussão é por causa do uso de e-mails encaminhados. Por exemplo, se seu e-mail for `elephpant@example.com`, mas você se inscreveu na lista de discussão com o e-mail `php-lists@example.com` que encaminha para `elephpant@example.com`, tentar remover da lista o e-mail `elephpant@example.com` não vai funcionar pois este e-mail não é conhecido nos nossos sistemas. Ao invés disso, você vai precisar pedir para remover da lista o e-mail para o qual as mensagens estão sendo enviadas, enviando um e-mail a partir dele - neste exemplo, `php-lists@example.com`.

Existe um histórico das listas de discussão em algum lugar?

Sim, você encontrará uma lista de sites com o histórico na página de » listas de email. Você também vai encontrar dezenas de arquivos e/ou referências para nossas listas de discussões usando seu mecanismo de busca preferido ao procurar por "php mailing list archives".

Os artigos da lista de discussão também são arquivados como mensagens de notícia. Você pode acessar o servidor de notícias em » news://news.php.net/ com um cliente de notícias. Também existe uma interface web experimental para o servidor de notícias em » http://news.php.net/

O que posso perguntar na lista de discussão?

Uma vez que o PHP está crescendo em popularidade todo dia, o tráfego na lista de discussão php-general cresceu e até agora a lista recebe entre 75 a 200 mensagens por dia. Por isso é do interesse de todos que você use a lista como último recurso quando você procurou em todos os outros lugares.

Antes de postar em uma lista verifique nessa FAQ e no manual se não consegue encontrar a informação aqui. Se não encontrar então tente os arquivos das listas (veja acima). Se está tendo problemas instalando ou configurando o PHP primeiro leia toda a documentação incluída e os README's. Se ainda não conseguir encontrar a informação que lhe ajude, você é bem vindo a utilizar a lista apropriada.

Para garantir que você receba as melhores respostas (e para reduzir a probabilidade de frustrar seus colegas desenvolvedores) envie sua pergunta para a lista apropriada. Por exemplo, se você está tendo dificuldades para instalar PHP, você deve enviar sua pergunta para a lista de discussão `php-install`. Uma ressalva: algumas listas têm nomes semelhantes mas usos completamente diferentes; uma pergunta sobre os scripts PHP no Windows deve ser direcionado para a lista de usuários do Windows PHP, *not* para a lista do Windows Internals.

Antes de perguntar, você deve ler o arquivo » Como Fazer Perguntas de Maneira Inteligente que é uma boa idéia para todo mundo.

Que informação devo incluir quando faço perguntas na lista?

Mensagens como "Não consigo colocar o PHP para funcionar! Me ajude! O que está errado?" são inúteis. Se você tiver problemas fazendo o PHP funcionar, você deve incluir que sistema operacional você está usando, qual a versão do PHP você está tentando instalar, como você obteve (pré-compilado, CVS, RPM e por aí vai), o que você fez até agora, onde ficou preso e a mensagem de erro exata.

Isso vale para qualquer outro problema também. Você deve incluir informações sobre o que você fez, o que você está tentando fazer e quaisquer mensagens de erro encontradas no processo. Se você tiver problemas com nosso código fonte, você precisa incluir a parte do código que não está funcionando, indicando a linha que apresenta o problema através do posicionamento de um comentário de código. Não inclua mais código que o necessário! Isso torna a mensagem difícil de ler e muitas pessoas podem simplesmente ignorar ela por isso. Se você não tiver certeza sobre quanta informação incluir na mensagem, é melhor incluir muito do que pouco.

Outra coisa importante a lembrar é resumir o problema no assunto da mensagem. Um assunto como "SOCCOOORRROO!!!" ou "Qual é o problema aqui" será ignorada pela maioria dos leitores.

E finalmente, você é encorajado a ler o artigo sobre » Como Fazer Perguntas de Maneira Inteligente já que irá ser de grande ajuda para todo mundo, especialmente você.

Obtendo o PHP

Essa seção tem detalhes sobre localizações de download do PHP e problemas com Sistemas Operacionais.

1. Onde posso obter o PHP?
2. Existem versões binárias pré-compiladas?
3. Onde posso pegar as bibliotecas necessárias para compilar alguma das extensões opcionais do PHP?
4. Como faço essas bibliotecas funcionarem?
5. Eu peguei a última versão do código-fonte do PHP do repositório Git na minha máquina Windows, o que eu preciso para compilá-lo?
6. Onde eu acho o Arquivo de Capacidades do Navegador (Browser Capabilities File)?

7. O que significa thread safety quando estou fazendo o download do PHP?

Onde posso obter o PHP?

O PHP pode ser obtido de qualquer um dos membros da rede de sites do PHP. Esses podem ser encontrados em » https://www.php.net/. Pode-se usar também um Git anônimo para pegar a última versão do fonte. Para mais informações, vá até » https://www.php.net/git.php.

Existem versões binárias pré-compiladas?

Os binários pré-compilados são distribuídos somente para sistemas Windows, pois isso exigiria capacidade do time do PHP de compilar o PHP para cada uma das plataformas Linux/Unix com cada combinação de extensão. Perceba também, que muitas distribuições Linux vêm com o PHP instalado atualmente. Os binários do Windows podem ser baixados da página de » Downloads, para binários do Linux, por favor visite o website de sua distribuição.

Onde posso pegar as bibliotecas necessárias para compilar alguma das extensões opcionais do PHP?

> **Nota** :
>
> Aquelas marcadas com um * são do melhor de nosso conhecimento não thread safe; elas não são recomendadas para utilização em um ambiente multi-thread.

- » LDAP (Unix).
- » LDAP (Unix/Win) : Diretório Mozilla (LDAP) SDK
- » servidor LDAP livre.
- » Berkeley DB2 (Unix/Win) : http://www.sleepycat.com/.
- » SNMP* (Unix): .
- » GD* (Unix/Win).
- » mSQL* (Unix).
- » PostgreSQL (Unix).
- » IMAP* (Win/Unix).
- » Sybase-CT* (Linux, libc5) : Disponível localmente.
- » FreeType (libttf):.
- » ZLib (Unix/Win32).
- » expat XML parser (Unix/Win32).
- » PDFLib.
- » mcrypt.
- » mhash.

- » t1lib.
- » dmalloc.
- » aspell.
- » libedit.

Como faço essas bibliotecas funcionarem?

Você precisará seguir as instruções disponíveis com a biblioteca. Algumas dessas bibliotecas são detectadas automaticamente quando você executar o script 'configure' do PHP (como a biblioteca GD), e outras você terá que habilitar usando opções '`--with-EXTENSION`' do '`configure`'. Execute '`configure --help`' para listar essas opções.

Eu peguei a última versão do código-fonte do PHP do repositório Git na minha máquina Windows, o que eu preciso para compilá-lo?

Veja a Wiki do PHP para instruções atualizadas: » Instruções passo-a-passo

Onde eu acho o Arquivo de Capacidades do Navegador (Browser Capabilities File)?

Você pode achar um arquivo *browscap.ini* em » http://browscap.org/.

O que significa thread safety quando estou fazendo o download do PHP?

"Thread Safety" (segurança para threads) significa que o binário pode trabalhar em um contexto de servidor web multithread, como o Apache 2 no Windows. Essa segurança funciona através da criação de uma cópia local em cada thread, de modo que os dados não colidam com outra thread.

Então como eu escolho? Se você escolher executar o PHP como binário CGI, não será necessária a segurança de threads, porque o binário é chamado a cada requisição. Para servidores web multithreads, como o IIS5 e IIS6, deve ser usada a versão segura para threads do PHP.

Questões sobre banco de dados

Esta seção possui questões comuns sobre a relação entre o PHP e bases de dados. Sim, o PHP pode acessar virtualmente qualquer base de dados disponível atualmente.

1. Eu ouvi que é possível acessar o Microsoft SQL Server através do PHP. Como?
2. Consigo acessar bases de dados Microsoft Access?

Eu ouvi que é possível acessar o Microsoft SQL Server através do PHP. Como?

Em máquinas Unix você pode utilizar o PDO_ODBC ou a API ODBC Unificada.

Em máquinas Windows você também pode utilizar PDO_SQLSRV ou SQLSRV.

Veja também a resposta da questão seguinte.

Consigo acessar bases de dados Microsoft Access?

Se você estiver rodando PHP em uma box Unix e quiser conversar com o Microsoft Access em uma box Windows será necessário instalar os drivers ODBC para Unix. » OpenLink Software possui drivers para sistemas baseados em Unix que podem fazer isto.

Outra alternativa é usar um servidor SQL que possua drivers Windows ODBC e utilizá-los para armazenar os dados, que você pode então acessar através do Microsoft Access (usando ODBC) e do PHP (usando os drivers já existentes), ou usando um formato de arquivo intermediário que o Microsoft Access e o PHP consigam entender, como arquivos comuns ou bases de dados dBase. Neste ponto Tim Hayes, da OpenLink Software, escreve:

> Usar outra base de dados como intermediária não é uma boa ideia, quando você pode usar o ODBC do PHP diretamente para a base de dados - por exemplo, com os drivers fornecidos pela OpenLink. Se você precisar utilizar um formato de arquivo intermediário, a OpenLink lançou o Virtuoso (uma engine de base de dados virtual) para NT, Linux e outras plataformas Unix. Por favor visite » nossa página para um download gratuito.

Uma opção que se mostrou funcional é utilizar MySQL e seus drivers MyODBC no Windows e sincronizar as bases de dados. Steve Lawrence disse:

- Instale o MySQL em sua plataforma normalmente. A ultima versão está disponível em » http://www.mysql.com/ Nenhuma configuração especial é necessária, exceto quando você configura um banco de dados, e configura uma conta de usuário, quando você deve colocar % no campo host, ou o hostname da máquina Windows com a qual você deseja acessar o MySQL. Anote o nome do seu servidor, o usuário e a senha.
- Baixe o driver MyODBC para Windows através do site oficial do MySQL. Instale-o em sua máquina Windows. Você pode testar a operação com os utilitários incluídos no programa.
- Crie um usuário ou dsn de sistema em seu ODBC Administrator, localizado no painel de controle. Crie um nome dsn, entre com seu hostname, usuário, senha, porta, etc. para sua base de dados MySQL configurada no passo 1.
- Instale o Access com uma instalação completa, isto dá certeza de que você instalou os add-ins necessários... você precisará, ao menos, ter suporte à ODBC e ao gerenciador de tabelas ligadas (linked table manager).
- Agora a parte divertida! Crie uma nova base de dados Access. Clique com o botão direito na janela de tabelas e selecione Link Tables, ou sob o menu File selecione Get External Data e depois Link Tables. Quando a caixa de navegação de arquivos aparecer, selecione arquivos do tipo: ODBC. Selecione System dsn e o nome do dsn criado no passo 3. Selecione a tabela para ligar, pressione OK, e presto! Agora você consegue abrir a tabela e adicionar/remover/editar dados em seu servidor MySQL! Você também pode construir queries, importar/exportar tabelas para MySQL, construir formulários e relatórios, etc.

Dicas e Truques:

- Você pode construir suas tabelas no Access e exportá-las para o MySQL, e então conectá-las de volta. Isto torna o processo de criação de tabelas mais rápido.
- Quando criar tabelas no Access você deve definir uma chave primária para ter acesso à escrita na tabela no Access. Certifique-se de que criou uma chave primária no MySQL antes de conectá-lo ao Access.
- Se você alterar a tabela no MySQL, você deve conectá-la novamente no Access. Vá para tools > add-ins > linked table manager, vá até seu ODBC DSN, e selecione a tabela para conectá-la

novamente a partir de lá. Você também pode mover seu fonte dsn por ali, apenas selecione o checkbox "always prompt for new location" antes de pressionar OK.

Instalação

Esta seção contém perguntas comuns sobre a maneira de instalar o PHP. O PHP está disponível para quase todos os sistemas operacionais, e praticamente qualquer servidor web.

Para instalar o PHP, siga as instruções em Instalação e Configuração.

1. Por que não devemos usar o Apache2 com um módulo MPM com threads em um ambiente de produção?
2. Unix/Windows: Onde deveria estar localizado meu arquivo php.ini?
3. Unix: Eu instalei o PHP, mas toda vez que carrego um documento, recebo a mensagem 'Document Contains No Data'! O que está acontecendo?
4. Unix: Eu instalei o PHP usando RPMS, mas o Apache não está processando as páginas PHP! O que está acontecendo aqui?
5. Unix: Eu corrigi o Apache com o patch das extensões FrontPage, e repentinamente o PHP parou de funcionar. O PHP é incompatível com as extensões FrontPage do Apache?
6. Unix/Windows: Eu tenho o PHP instalado, mas quando tento acessar um arquivo de script PHP através do meu browser, recebo uma tela em branco.
7. Unix/Windows: Eu instalei o PHP, mas quando tento acessar um arquivo de script PHP através do meu browser, recebo um erro 500 do servidor.
8. Alguns sistemas operacionais: Eu instalei o PHP sem erros, mas quando eu tento iniciar o Apache recebo erros de undefined symbol: [mybox:user /src/php5] root# apachectl configtest apachectl: /usr/local/apache/bin/httpd Undefined symbols: _compress _uncompress
9. Windows: Eu instalei o PHP, mas quando tento acessar um arquivo de script PHP através do meu navegador, eu recebo o erro: cgi error: The specified CGI application misbehaved by not returning a complete set of HTTP headers. The headers it did return are:
10. Windows: Eu segui todas as instruções, mas ainda não consegui fazer o PHP e o IIS trabalharem juntos!
11. Quando executo o PHP como CGI com o IIS, PWS, OmniHTTPD ou Xitami, recebo o seguinte erro: Security Alert! PHP CGI cannot be accessed directly..
12. Como eu sei se meu php.ini está sendo encontrado e lido? Parece que ele não está, já que as minhas alterações não estão sendo efetuadas.
13. Como eu adiciono meu diretório PHP ao PATH do Windows?
14. Como eu torno o arquivo php.ini disponível para o PHP no Windows?
15. Windows: Como verificar se o PHP pode escrever no diretório temporário com o IIS?
16. É possível usar a negociação de conteúdo do Apache (opção MultiViews) com o PHP?
17. O PHP é limitado a processar apenas os métodos de requisições GET e POST?

Por que não devemos usar o Apache2 com um módulo MPM com threads em um ambiente de produção?

> O PHP é como uma cola. É a cola usada para construir aplicações web legais unindo dezenas de bibliotecas de terceiros e fazendo tudo parecer como uma entidade coerente através de uma interface de linguagem intuitiva e fácil de aprender. A flexibilidade e poder do PHP dependem da estabilidade e robustez da plataforma subjacente. Ele precisa de um sistema operacional funcionando, um servidor web funcionando e bibliotecas de terceiros funcionando para juntar. Quando qualquer um desses elementos para de funcionar o PHP precisa de meios de identificar e remediar os problemas rapidamente. Quando tornamos a arquitetura subjacente mais complexa por não ter threads de

execução completamente separadas, segmentos de memória completamente separados e uma sandbox sólida onde cada requisição é executada, mais problemas são introduzidos ao sistema do PHP.

Se quiser utilizar um módulo MPM com threads, veja uma configuração FastCGI onde o PHP seja executado em seu próprio espaço de memória.

Unix/Windows: Onde deveria estar localizado meu arquivo *php.ini*?

Por padrão no Unix ele deve estar em */usr/local/lib* que é o *<install-path>/lib*. A maioria das pessoas vai querer mudar isso em tempo de compilação com a flag --with-config-file-path. Poderíamos, por exemplo, configurá-la com algo como:

```
--with-config-file-path=/etc
```

E então poderíamos copiar *php.ini-development* do local de origem da instalação para */etc/php.ini* e editá-lo para fazer quaisquer alterações locais que desejássemos.

```
--with-config-file-scan-dir=PATH
```

No Windows, o caminho padrão para o arquivo *php.ini* é o diretório Windows. Se estivermos usando o servidor web Apache, o *php.ini* primeiro é procurado no diretório de instalação do Apache, por exemplo, *c:\program files\apache group\apache*. Desta maneira, poderíamos ter diferentes arquivos *php.ini* para diferentes versões do Apache na mesma máquina.

Veja também o capítulo sobre o arquivo de configuração.

Unix: Eu instalei o PHP, mas toda vez que carrego um documento, recebo a mensagem 'Document Contains No Data'! O que está acontecendo?

Isso provavelmente significa que o PHP está tendo algum tipo de problema e está sofrendo um core dump (fim com despejo de memória). Olhe nos logs de erro do servidor web para ver se este é o caso, e então tente reproduzir o problema com um pequeno caso de teste. Se você souber como usar 'gdb', é muito útil quando você pode fornecer um backtrace com seu relatório de erros para ajudar os desenvolvedores a identificar o problema. Se estiver usando o PHP como um módulo do Apache tente algo como:

- Pare seus processos httpd
- gdb httpd
- Pare seus processos httpd
- > execute -X -f /caminho/para/httpd.conf
- Então acesse o URL que está causando problemas com seu browser.
- > execute -X -f /caminho/para/httpd.conf
- Se estiver ocorrendo um core dump, o gdb deverá informar sobre isso agora
- type: bt
- Você deve incluir o backtrace no seu relatório de erros. Envie o relatório para » https://github.com/php/php-src/issues

Se o seu script usa as funções de expressões regulares (preg_match() e demais), você deve ter certeza de que compilou o PHP e o Apache com a mesma versão do pacote de expressões regulares. Isso deve acontecer automaticamente com o PHP e Apache 1.3.x.

Unix: Eu instalei o PHP usando RPMS, mas o Apache não está processando as páginas PHP! O que está acontecendo aqui?

Supondo que você instalou tanto o Apache quanto o PHP a partir dos pacotes RPM, você precisa descomentar ou adicionar algumas ou todas as linhas a seguir ao seu arquivo *httpd.conf*:

```
# Módulos extra
AddModule mod_php.c
AddModule mod_perl.c

# Módulos extra
LoadModule php_module         modules/mod_php.so
LoadModule php5_module        modules/libphp5.so
LoadModule perl_module        modules/libperl.so
```

E adicionar:

```
AddType application/x-httpd-php .php
```

... às propriedades globais, ou às propriedades do VirtualDomain ao qual você deseja adicionar o suporte ao PHP.

Unix: Eu corrigi o Apache com o patch das extensões FrontPage, e repentinamente o PHP parou de funcionar. O PHP é incompatível com as extensões FrontPage do Apache?

Não, O PHP funciona bem com as extensões FrontPage. O problema é que o patch do FrontPage modifica diversas estruturas do Apache, das quais o PHP depende. Recompilar o PHP (usando 'make clean ; make') depois de aplicar o patch FP deve resolver o problema.

Unix/Windows: Eu tenho o PHP instalado, mas quando tento acessar um arquivo de script PHP através do meu browser, recebo uma tela em branco.

Faça um 'view source' no navegador web e provavelmente descobrirá que pode ver o código fonte do seu script PHP. Isto significa que o seu servidor web não enviou o script para o interpretador do PHP. Algo está errado com as configurações do seu servidor - verifique novamente as configurações de seu servidor, com base nas instruções de instalação do PHP.

Unix/Windows: Eu instalei o PHP, mas quando tento acessar um arquivo de script PHP através do meu browser, recebo um erro 500 do servidor.

Alguma coisa deu errado quando o servidor tentou executar o PHP. Para conseguir ver uma mensagem de erro razoável, na linha de comando, mude para o diretório que contém o executável do PHP (*php.exe* no Windows) e execute **php -i**. Se o PHP tiver algum problema para rodar, então uma mensagem de erro apropriada será exibida, o que lhe dará pistas do que precisa ser feito em seguida. Se você receber uma tela cheia de códigos HTML (a saída da função phpinfo()) então o PHP está funcionando, e seu problema pode ser relacionado à configuração do seu servidor, que você deve verificar novamente.

Alguns sistemas operacionais: Eu instalei o PHP sem erros, mas quando eu tento iniciar o Apache recebo erros de undefined symbol:

```
[mybox:user /src/php5] root# apachectl configtest
 apachectl: /usr/local/apache/bin/httpd Undefined symbols:
  _compress
  _uncompress
```

Isto não tem nada a ver com o PHP, mas com as bibliotecas do cliente MySQL. Algumas necessitam de **--with-zlib**, outras não. Isso também é abordado nas FAQ do MySQL.

Windows: Eu instalei o PHP, mas quando tento acessar um arquivo de script PHP através do meu navegador, eu recebo o erro:

```
cgi error:
 The specified CGI application misbehaved by not
 returning a complete set of HTTP headers.
 The headers it did return are:
```

Esta mensagem de erro significa que o PHP falhou em gerar qualquer saída. Para começar a ver uma mensagem de erro razoável, na linha de comando, mude para o diretório que contém o executável PHP (*php.exe* no Windows) e execute **php -i**. Se o PHP estiver com algum problema para rodar, então uma mensagem de erro adequada será exibida, o que lhe dará uma ideia do que precisa ser feito em seguida. Se você receber uma tela cheia de códigos HTML (a saída da função phpinfo()) então o PHP está funcionando.

Uma vez que o PHP esteja funcionando na linha do comando, tente acessar o script via navegador novamente. Se ele ainda falhar, então pode ser um dos seguintes:

- As permissões de arquivo no seu script PHP, *php.exe*, *php5ts.dll*, *php.ini* ou qualquer extensão do PHP que você está tentando carregar são tais que o usuário anônimo `ISUR_<machinename>` não pode acessá-los.
- O arquivo de script não existe (ou provavelmente não está onde você acha que ele está em relação ao seu diretório root web). Note que com o IIS você pode interceptar o erro marcando a opção 'verificar se o arquivo existe' quando configurar os mapeamentos de script no Gerenciador de Serviços de Internet. Se um arquivo de script não existe então o servidor web irá retornar um erro 404. Há também o benefício adicional que o IIS fará qualquer autenticação exigida para você com base nas permissões NTLanMan no seu arquivo de script.

Windows: Eu segui todas as instruções, mas ainda não consegui fazer o PHP e o IIS trabalharem juntos!

Certifique-se de que qualquer usuário que precise executar um script PHP tem permissão para executar o *php.exe*! O IIS usa um usuário anônimo que é adicionado no momento que o IIS é instalado. Este usuário precisa de permissões para o *php.exe*. Além disso, qualquer usuário autenticado também precisará ter permissão para executar o *php.exe*. E para o IIS4 você precisa dizer que o PHP é um engine de script. Além disso, você também vai querer ler esta FAQ.

Quando executo o PHP como CGI com o IIS, PWS, OmniHTTPD ou Xitami, recebo o seguinte erro: `Security Alert! PHP CGI cannot be accessed directly..`

Você deve definir a diretiva cgi.force_redirect para `0`. O padrão é `1`, então certifique-se que a diretiva não está comentada (com um `;`). Como todas as diretivas, esta é definida no *php.ini*

Pelo fato do padrão ser `1`, é fundamental que você tenha certeza de que o arquivo *php.ini* correto está sendo lido. Leia esta FAQ para detalhes.

Como eu sei se meu *php.ini* está sendo encontrado e lido? Parece que ele não está, já que as minhas alterações não estão sendo efetuadas.

Para ter certeza de que seu *php.ini* está sendo lido pelo PHP, faça uma chamada à função phpinfo(). Próximo ao topo, haverá uma lista chamada `Configuration File (php.ini)`. Esta irá lhe dizer onde o PHP está procurando o *php.ini* e se ele está sendo lido ou não. Se houver apenas um diretório do *PATH*, então ele não está sendo lido e você deve colocar seu *php.ini* neste diretório. Se o *php.ini* está incluso dentro do *PATH*, ele está sendo lido.

Se o *php.ini* está sendo lido e seu PHP está sendo executado como um módulo, então certifique-se de reiniciar seu servidor web depois de fazer alterações no *php.ini*.

Veja também php_ini_loaded_file().

Como eu adiciono meu diretório PHP ao *PATH* do Windows?

No Windows:

- Vá até o Painel de Controle e abra o ícone de Sistema (Iniciar → Painel de Controle)
- Vá até a aba Avançado
- Clique no botão 'Variáveis de Ambiente'
- Procure o painel 'Variáveis do Sistema'
- Encontre o item Path (talvez você precise rolar a lista para encontrá-lo)
- Dê um duplo clique no item Path
- Adicione seu diretório do PHP ao final, incluindo ';' antes (por exemplo, `;C:\php`)
- Aperte OK e reinicie seu computador

> **Nota** :
>
> Certifique-se de reiniciar depois de seguir os passos acima para garantir que as alterações ao *PATH* sejam aplicadas.

Como eu torno o arquivo *php.ini* disponível para o PHP no Windows?

Existem várias maneiras de fazer isso. Se você estiver usando o Apache, consulte a documentação do Apache, caso contrário, você deve definir a variável de ambiente *PHPRC*.

Windows: Como verificar se o PHP pode escrever no diretório temporário com o IIS?

1. Clique com o botão direito no diretório temporário (*%TEMP%*) no Explorador de Arquivos para obter as permissões. O diretório temporário está disponível na configuração ou através da função phpinfo().
2. Para o IIS, verifique que o usuário `IIS_User` tem permissão de `MODIFY` (alteração).

É possível usar a negociação de conteúdo do Apache (opção MultiViews) com o PHP?

Se os links para arquivos PHP incluem extensões, tudo funcionará perfeitamente. Esta FAQ é apenas para o caso em que links para arquivos PHP não incluem a extensão e você quer usar a negociação de conteúdo para selecionar arquivos PHP a partir de URLs sem nenhuma extensão. Neste caso, substitua a linha `AddType application/x-httpd-php .php` por:

```
AddHandler php5-script php
AddType text/html php
```

Esta solução não funciona para o Apache 1, pois o módulo PHP não captura `php-script`.

O PHP é limitado a processar apenas os métodos de requisições GET e POST?

Não, é possível lidar com qualquer método de requisição, por exemplo, CONNECT. Status de respostas adequados podem ser enviados com header(). Se somente os métodos GET e POST devem ser manipulados, isso pode ser alcançado com esta configuração do Apache:

```
<LimitExcept GET POST>
Deny from all
</LimitExcept>
```

Problemas de Compilação

Esta seção reúne os erros mais comuns que ocorrem na compilação.

1. Eu peguei a última versão do PHP usando o serviço anônimo do GIT, mas não há nenhum script de configuração!
2. Estou tendo problemas para configurar o PHP para trabalhar com Apache. Diz que não consegue encontrar httpd.h, mas está bem onde eu disse que está!
3. Se enquanto estiver configurando o (./configure) do PHP, você se deparar com um erro semelhante ao seguinte: checking lex output file root... ./configure: lex: command not found configure: error: cannot find output from lex; giving up
4. Quando tento iniciar o Apache, recebo a seguinte mensagem: fatal: relocation error: file /path/to/libphp4.so: symbol ap_block_alarms: referenced symbol not found
5. Quando eu executo configure, ele diz que não pode encontrar os arquivos de inclusão ou a biblioteca GD, gdbm, ou algum outro pacote!
6. Quando se está compilando o arquivo language-parser.tab.c, ele me devolve um erro que diz yytname undeclared.
7. Quando eu executo make, parece rodar bem mas então falha quando tenta ligar a aplicação final, queixando-se que não consegue encontrar alguns arquivos.
8. Ao linkar o PHP, ele reclama de um número de referências indefinidas.

9. Segui todos os passos da instalação do módulo da versão do Apache no Unix, e meus scripts PHP estão sendo mostrados no meu navegador ou ele me pede para salvar os arquivos.
10. Lá diz para usar: --activate-module=src/modules/php4/libphp4.a, mas esse arquivo não existe, então mudei para --activate-module=src/modules/php4/libmodphp4.a e não funciona!? O que está acontecendo?
11. Quando tento compilar o Apache com o PHP como um módulo estático usando --activate-module=src/modules/php4/libphp4.a ele me diz que o meu compilador não é compatível com ANSI.
12. Quando tento compilar o PHP usando --with-apxs recebo mensagens de erro estranhas.
13. Durante make, eu recebo erros no microtime, e um monte de RUSAGE_.
14. Ao compilar o PHP com MySQL, o configure vai bem, mas durante o make recebo um erro semelhante ao seguinte: ext/mysql/libmysqlclient/my_tempnam.o(.text+0x46): In function my_tempnam': /php4/ext/mysql/libmysqlclient/my_tempnam.c:103: the use of tempnam' is dangerous, better use mkstemp', o que está errado ?
15. Eu quero atualizar o meu PHP. Onde posso encontrar a linha ./configure que foi usada para compilar a minha instalação atual do PHP?
16. Quando compilo o PHP com a biblioteca GD retorna erros de compilação ou segfaults na execução.
17. Ao compilar o PHP eu recebo erros aparentemente aleatórios, como se ele travasse. Estou usando o Solaris, se isso importa.

Eu peguei a última versão do PHP usando o serviço anônimo do GIT, mas não há nenhum script de configuração!

> Você precisa ter o pacote GNU autoconf instalado para que possa gerar o script configure a partir do *configure.in*. Basta executar **./buildconf** no diretório raiz após a obtenção do código fonte do servidor do GIT. (Além disso, a menos que você execute com **configure** com a opção `--enable-maintainer-mode`, o script configure não será automaticamente recompilado quando o arquivo *configure.in* for atualizado, então você deve certificar-se de fazer isso manualmente quando você perceber que o *configure.in* mudou. Um sintoma disso é encontrar coisas como @VARIABLE@ em seu Makefile após configure ou *config.status*tiver sido executado.)

Estou tendo problemas para configurar o PHP para trabalhar com Apache. Diz que não consegue encontrar *httpd.h*, mas está bem onde eu disse que está!

> Você precisa informar ao script configure/setup a localização do diretório raíz do Apache. Isto significa que você precisa especificar **--with-apache=/caminho/para/apache** e *não* **--with-apache=/caminho/para/apache/src**.

Se enquanto estiver configurando o (`./configure`) do PHP, você se deparar com um erro semelhante ao seguinte:

```
checking lex output file root... ./configure: lex: command not found

configure: error: cannot find output from lex; giving up
```

> Não deixe de ler as instruções de instalação cuidadosamente e note que você precisa de ambos flex e bison instalados para compilar o PHP. Dependendo da sua configuração você irá instalar bison e flex a partir de uma fonte ou pacote, como um RPM.

Quando tento iniciar o Apache, recebo a seguinte mensagem:

```
fatal: relocation error: file /path/to/libphp4.so:
symbol ap_block_alarms: referenced symbol not found
```

Esse erro geralmente aparece quando um compila o core do Apache como uma biblioteca DSO para uso compartilhado. Tente reconfigurar o apache, certificando-se de usar pelo menos um dos seguintes parâmetros:

```
--enable-shared=max --enable-rule=SHARED_CORE
```

Para obter mais informações, leia o arquivo *INSTALL* que se encontra na pasta raíz do Apache ou a » página sobre DSO no manual do Apache.

Quando eu executo configure, ele diz que não pode encontrar os arquivos de inclusão ou a biblioteca GD, gdbm, ou algum outro pacote!

Você pode fazer o script 'configure' procurar por arquivos de cabeçalho e bibliotecas em locais diferentes, especificando parâmetros adicionais para passar para o pré-processador C e linker, tais como:

```
CPPFLAGS=-I/path/to/include LDFLAGS=-L/path/to/library ./configure
```

Se uma variante csh estiver sendo usada no seu shell de login (por quê?), o comando seria:

```
env CPPFLAGS=-I/path/to/include LDFLAGS=-L/path/to/library ./configure
```

Quando se está compilando o arquivo *language-parser.tab.c*, ele me devolve um erro que diz `yytname undeclared`.

Você precisa atualizar sua versão do Bison. Você pode encontrar a última versão em » http://www.gnu.org/software/bison/bison.html.

Quando eu executo make, parece rodar bem mas então falha quando tenta ligar a aplicação final, queixando-se que não consegue encontrar alguns arquivos.

Algumas versões antigas do make não colocam corretamente as versões compiladas dos arquivos do diretório "functions" dentro deste mesmo diretório. Tente rodar **cp *.o functions** e então execute novamente **make** para ver se isso ajuda. Se não ajudar, você realmente deve atualizar para uma versão mais recente do GNU make.

Ao linkar o PHP, ele reclama de um número de referências indefinidas.

Confira a linha do link e tenha certeza que todas as bibliotecas apropriadas estão sendo incluídas no final. É comum que talvez você tenha esquecido os '-ldl' e alguma biblioteca requerida para suporte de

banco de dados que você incluiu.

Algumas pessoas também reportaram que precisaram adicionar '-ldl' antes de *libphp4.a* ao linkar com o Apache.

Segui todos os passos da instalação do módulo da versão do Apache no Unix, e meus scripts PHP estão sendo mostrados no meu navegador ou ele me pede para salvar os arquivos.

Isso significa que o módulo do PHP não está sendo chamado por alguma razão. Três coisas para se verificar antes de pedir mais ajuda:

- Tenha certeza que o binário httpd que você está executando é o que você acabou de compilar. Para fazer isso, tente executar: `/path/to/binary/httpd -l`

  Se você não vir *mod_php4.c* na lista, então não está executando o binário correto. Encontre e instale o binário correto.

- Certifique-se de ter adicionado o Mime Type correto em um dos seus arquivos `Apache .conf`. Deveria ser: `AddType application/x-httpd-php .php`

  Também certifique-se que esta linha AddType não esteja escondida dentro de um bloco <Virtualhost> ou <Directory> que possa impedi-la de ser aplicada para a localização de seu script de teste.

- Finalmente, a localização padrão dos arquivos de configuração do Apache mudou do Apache 1.2 para o Apache 1.3. Você deve ter certeza que o arquivo de configuração que você está adicionando à linha do AddType realmente está sendo lido. Você pode colocar um erro de sintaxe óbvio em seu arquivo *httpd.conf* ou alguma outra mudança evidente que diga se o arquivo está sendo lido corretamente.

Lá diz para usar: `--activate-module=src/modules/php4/libphp4.a`, mas esse arquivo não existe, então mudei para `--activate-module=src/modules/php4/libmodphp4.a` e não funciona!? O que está acontecendo?

Note que o arquivo *libphp4.a* não é para existir. O processo apache irá criá-lo!

Quando tento compilar o Apache com o PHP como um módulo estático usando `--activate-module=src/modules/php4/libphp4.a` ele me diz que o meu compilador não é compatível com ANSI.

Esta é uma mensagem de erro enganosa do Apache que foi corrigida nas versões mais recentes.

Quando tento compilar o PHP usando `--with-apxs` recebo mensagens de erro estranhas.

Há três coisas para verificar aqui. Em primeiro lugar, por alguma razão o Apache quando compila o script Perl apxs, às vezes acaba sendo compilado sem o compilador e os parâmetros corretos. Procure pelo seu script apxs (tente o comando **which apxs**), às vezes é encontrado em */usr/local/apache/bin/apxs* ou */usr/sbin/apxs*. Abra-o e verifique se existem linhas semelhantes a estas:

```
my $CFG_CFLAGS_SHLIB  = ' ';          # substituted via Makefile.tmpl
my $CFG_LD_SHLIB      = ' ';          # substituted via Makefile.tmpl
my $CFG_LDFLAGS_SHLIB = ' ';          # substituted via Makefile.tmpl
```

Se é isso que você vê, você encontrou o seu problema. Elas podem conter apenas espaços ou outros valores incorretos, como 'q()'. Altere estas linhas de modo que fiquem assim:

```
my $CFG_CFLAGS_SHLIB  = '-fpic -DSHARED_MODULE'; # substituted via Makefile.tmpl
my $CFG_LD_SHLIB      = 'gcc';                   # substituted via Makefile.tmpl
my $CFG_LDFLAGS_SHLIB = q(-shared);              # substituted via Makefile.tmpl
```

O segundo problema possível só deve ser um problema no Red Hat 6.1 e 6.2. O script apxs do Red Hat está quebrado. Procure por esta linha:

```
my $CFG_LIBEXECDIR    = 'modules';         # substituted via APACI install
```

Se você vê a linha acima, mude para isto:

```
my $CFG_LIBEXECDIR    = '/usr/lib/apache'; # substituted via APACI install
```

Finalmente, se você reconfigurar/reinstalar o Apache, adicione **make clean** ao processo após o **./configure** e antes de **make**.

Durante make, eu recebo erros no microtime, e um monte de `RUSAGE_`.

Enquanto é executada a porção **make** da instalação, se você encontrar problemas que parecem semelhantes a estes:

```
microtime.c: In function `php_if_getrusage':
microtime.c:94: storage size of `usg' isn't known
microtime.c:97: `RUSAGE_SELF' undeclared (first use in this function)
microtime.c:97: (Each undeclared identifier is reported only once
microtime.c:97: for each function it appears in.)
microtime.c:103: `RUSAGE_CHILDREN' undeclared (first use in this function)
make[3]: *** [microtime.lo] Error 1
make[3]: Leaving directory `/home/master/php-4.0.1/ext/standard'
make[2]: *** [all-recursive] Error 1
make[2]: Leaving directory `/home/master/php-4.0.1/ext/standard'
make[1]: *** [all-recursive] Error 1
make[1]: Leaving directory `/home/master/php-4.0.1/ext'
make: *** [all-recursive] Error 1
```

Seu sistema está quebrado. É preciso corrigir os arquivos */usr/include* instalando o pacote glibc-devel que corresponde ao seu glibc. Isto não tem absolutamente nada a ver com o PHP. Para provar isso a si mesmo, tente este teste simples:

```
$ cat >test.c <<X
#include <sys/resource.h>
X
$ gcc -E test.c >/dev/null
```

Se isso mostrar erros, você saberá que os arquivos include estão bagunçados.

Ao compilar o PHP com MySQL, o configure vai bem, mas durante o `make` recebo um erro semelhante ao seguinte: *ext/mysql/libmysqlclient/my_tempnam.o(.text+0x46): In function my_tempnam': /php4/ext/mysql/libmysqlclient/my_tempnam.c:103: the use of tempnam' is dangerous, better use mkstemp'*, o que está errado ?

Primeiramente, é importante perceber que este é um `Warning` e não um erro fatal. Porque esta é muitas vezes a última saída durante o `make`, ela pode parecer um erro fatal, mas não é. Claro que, se você

definir o seu compilador para parar em warnings, ele vai parar. Além disso, tenha sempre em mente que o suporte ao MySQL está habilitado por padrão.

> **Nota** :
>
> A partir do PHP 4.3.2, você também verá o seguinte texto depois que a compilação (make) for concluída:
>
> ```
> Build complete.
>
> (It is safe to ignore warnings about tempnam and tmpnam).
> ```

Eu quero atualizar o meu PHP. Onde posso encontrar a linha *./configure* que foi usada para compilar a minha instalação atual do PHP?

Ou você olha no arquivo config.nice, na pasta raíz atual da sua instalação do PHP, ou, se este não estiver disponível, basta executar um script

```
<?php phpinfo(); ?>
```

No topo da saída a linha **./configure** que foi usada para compilar esta instalação do PHP é mostrada.

Quando compilo o PHP com a biblioteca GD retorna erros de compilação ou segfaults na execução.

Certifique-se de que sua biblioteca GD e o PHP estejam ligadas às mesmas bibliotecas dependentes (libpng por exemplo).

Ao compilar o PHP eu recebo erros aparentemente aleatórios, como se ele travasse. Estou usando o Solaris, se isso importa.

Usar utilitários que não são GNU ao compilar o PHP pode causar problemas. Certifique-se de usar as ferramentas GNU, a fim de ter certeza de que a compilação do PHP funcionará. Por exemplo, no Solaris, usando tanto o SunOS BSD-compatible quanto versões do Solaris `sed` não vai funcionar, mas usando as versões GNU ou Sun POSIX (XPG4) do `sed` irá funcionar. Links: » GNU sed, » GNU flex, e » GNU bison.

Utilizando o PHP

Esta seção reúne vários erros comuns que você pode encontrar enquanto estiver escrevendo scripts PHP.

1. Não consigo me lembrar da ordem dos parâmetros das funções PHP. Eles são aleatórios?
2. Eu gostaria de escrever um script PHP genérico que fosse capaz de manipular dados provenientes de qualquer formulário. Como eu sei quais variáveis do método POST estão disponíveis?
3. Preciso converter todas as aspas simples (') para uma barra invertida seguida por uma aspa simples (\'). Como eu posso fazer isso utilizando expressões regulares? Gostaria de converter também " para \" e \ para \\.
4. Quando eu faço o seguinte, a saída é exibida na ordem errada: <?php function myfunc($argument) { echo $argument + 10; } $variable = 10; echo "myfunc($variable) = " . myfunc($variable); ?> O que está acontecendo?

5. Ei, o que aconteceu com as minhas novas linhas? <pre> <?php echo "Esta deve ser a primeira linha."; ?> <?php echo "Esta deveria ser uma nova linha."; ?> </pre>
6. Recebi a mensagem 'Warning: Cannot send session cookie - headers already sent...' ou 'Cannot add header information - headers already sent...'.
7. Preciso acessar informações no cabeçalho de requisição diretamente. Como eu faço isso?
8. Quando eu tento utilizar autenticação com o IIS eu recebo a mensagem 'No Input file specified'.
9. Windows: Não consigo acessar arquivos compartilhados em outro computador utilizando IIS.
10. Como faço para misturar XML e PHP? Ele reclama das minhas tags <?xml!
11. Onde posso encontrar uma lista completa das variáveis que estão disponíveis para mim no PHP?
12. Como posso gerar arquivos PDF sem utilizar bibliotecas não-livres e comerciais como PDFLib? Eu gostaria de alguma que fosse livre e não precisasse de bibliotecas PDF externas.
13. Algumas diretivas PHP podem receber valores em bytes de forma abreviada (1K), ao contrário de apenas valores int. Quais são todas as opções disponíveis de bytes de forma abreviada?

Não consigo me lembrar da ordem dos parâmetros das funções PHP. Eles são aleatórios?

O PHP é uma "cola" que reúne centenas de bibliotecas externas, então às vezes isso fica desorganizado. No entanto, uma regra simples é a seguinte:

Os parâmetros de funções de Array são ordenados como "*needle, haystack*" ao passo que funções de String são o contrário, logo "*haystack, needle*".

A partir do PHP 8.0, argumentos nomeados permitem a passagem de argumentos pelo nome do parâmetro, tornando a ordem dos parâmetros menos significante.

Eu gostaria de escrever um script PHP genérico que fosse capaz de manipular dados provenientes de qualquer formulário. Como eu sei quais variáveis do método POST estão disponíveis?

O PHP disponibiliza muitas variáveis predefinidas, como a superglobal *$_POST*. É possível iterar por *$_POST*, já que ela é um array associativo com todos os valores enviados com o método POST. Por exemplo, vamos iterar por ela com `foreach`, verificar se há valores vazios com empty() e imprimi-los na tela.

```
<?php
$empty = $post = array();
foreach ($_POST as $varname => $varvalue) {
    if (empty($varvalue)) {
        $empty[$varname] = $varvalue;
    } else {
        $post[$varname] = $varvalue;
    }
}

print "<pre>";
if (empty($empty)) {
    print "Nenhum dos valores enviados com POST estão vazios, enviado:\n";
    var_dump($post);
} else {
    print "Temos " . count($empty) . " valores vazios\n";
    print "Enviado:\n"; var_dump($post);
    print "Vazio:\n"; var_dump($empty);
    exit;
}
?>
```

Preciso converter todas as aspas simples (') para uma barra invertida seguida por uma aspa simples (\'). Como eu posso fazer isso utilizando expressões regulares? Gostaria de converter também " para \" e \ para \\.

> Considerando que isso seja para um banco de dados, utilize o mecanismo de escape que vem com o banco de dados. Por exemplo, utilize mysql_real_escape_string() com MySQL e pg_escape_string() com PostgreSQL. Existem também as funções genéricas addslashes() e stripslashes(), que são mais comuns com códigos PHP antigos.
>
> Escapar valor manualmente é passível de erros e depende do contexto. Prefira o uso de APIs de banco de dados que suportem instruções preparadas e vinculação de parâmetros em vez de construir consultas por concatenação de strings escapadas.

Quando eu faço o seguinte, a saída é exibida na ordem errada:

```
<?php
function myfunc($argument)
{
    echo $argument + 10;
}
$variable = 10;
echo "myfunc($variable) = " . myfunc($variable);
?>
```

O que está acontecendo?

> Para ser capaz de utilizar os resultados de suas funções em uma expressão (como concatenar com uma string no exemplo acima), você precisa utilizar return para retornar o valor e não echo.

Ei, o que aconteceu com as minhas novas linhas?

```
<pre>
<?php echo "Esta deve ser a primeira linha."; ?>
<?php echo "Esta deveria ser uma nova linha."; ?>
</pre>
```

> No PHP, um bloco de código é fechado com "?>" ou "?>\n" (onde \n significa uma nova linha). Logo, no exemplo anterior, as sentenças serão exibidas na mesma linha porque o PHP omite o caractere de nova linha após o final do bloco de código. Isso significa que você precisa inserir uma nova linha depois de cada bloco de código PHP para exibi-las em uma nova linha.
>
> Por que o PHP faz isso? Porque quando está formatando HTML, isto normalmente torna sua vida mais fácil porque não se quer esta nova linha, senão você teria que criar linhas extremamente longas ou tornar o código fonte ilegível para alcançar este efeito.

Recebi a mensagem 'Warning: Cannot send session cookie - headers already sent...' ou 'Cannot add header information - headers already sent...'.

> As funções header(), setcookie() e as funções de sessão precisam adicionar os cabeçalhos ao fluxo de saída, mas os cabeçalhos só podem ser enviados antes de qualquer outro conteúdo. Não pode haver nenhuma saída antes de utilizar estas funções como HTML. A função headers_sent() irá verificar se o seu script já enviou cabeçalhos e veja também as funções de Controle de Saída.

Preciso acessar informações no cabeçalho de requisição diretamente. Como eu faço isso?

A função getallheaders() fará isso se você estiver rodando o PHP como um módulo do Apache. Logo, o fragmento de código abaixo sempre exibirá todos os cabeçalhos de requisição:

```
<?php
$headers = getallheaders();
foreach ($headers as $name => $content) {
    echo "headers[$name] = $content<br />\n";
}
?>
```

Veja também apache_lookup_uri(), apache_response_headers() e fsockopen()

Quando eu tento utilizar autenticação com o IIS eu recebo a mensagem 'No Input file specified'.

O modelo de segurança do IIS é o culpado aqui. Este é um problema comum a todos os programas CGI rodando no IIS. Uma maneira de contornar é criar um arquivo HTML puro (não interpretado pelo PHP) como a página de entrada de um diretório autenticado. Então utilize a tag META para redirecionar para a página PHP ou fornecer um atalho para a página PHP. O PHP irá então reconhecer a autenticação corretamente. Com o módulo ISAPI, este não é um problema. Isto não deve afetar outros servidores web NT. Para mais informações veja: » http://support.microsoft.com/kb/q160422/ e a seção do manual em Autenticação HTTP .

Windows: Não consigo acessar arquivos compartilhados em outro computador utilizando IIS.

Você terá que alterar o `Go to Internet Information Services`. Localize o arquivo PHP e vá até as suas propriedades. Vá até a aba `File Security`, `Edit -< Anonymous access and authentication control`.

Pode-se corrigir o problema desabilitando `Anonymous Access` e deixando `Integrated Window Authentication` habilitado, ou habilitando `Anonymous Access` e editando o usuário para que ele não possa ter o direito de acesso.

Como faço para misturar XML e PHP? Ele reclama das minhas tags <?xml!

Para utilizar <?xml diretamente no seu código PHP, você terá que desabilitar as short tags definindo a diretiva PHP short_open_tags como `0`. Você não pode configurar esta diretiva com a função ini_set(). Independentemente de short_open_tags estar ativada ou não, você pode fazer algo como: `<?php echo '<?xml'; ?>`. O padrão para esta diretiva é `On`.

Onde posso encontrar uma lista completa das variáveis que estão disponíveis para mim no PHP?

Leia a página do manual variáveis predefinidas dado que ele inclui uma lista parcial das variáveis predefinidas disponíveis para o seu script. Uma lista completa das variáveis disponíveis (e muito mais informação) pode ser vista chamando a função phpinfo(). Não deixe de ler a seção do manual variáveis de fontes externas uma vez que descreve cenários comuns para variáveis externas como aquelas originadas de um formulário HTML, cookie ou da URL.

Como posso gerar arquivos PDF sem utilizar bibliotecas não-livres e comerciais como PDFLib? Eu gostaria de alguma que fosse livre e não precisasse de bibliotecas PDF externas.

Existem algumas poucas alternativas escritas em PHP como » FPDF e » TCPDF.

Algumas diretivas PHP podem receber valores em bytes de forma abreviada (1K), ao contrário de apenas valores int. Quais são todas as opções disponíveis de bytes de forma abreviada?

As opções disponíveis são K (para Kilobytes), M (para Megabytes) e G (para Gigabytes), tanto em maiúsculas ou minúsculas. Qualquer outra coisa é tratada como bytes. `1M` é igual a um Megabyte ou `1048576` bytes. `1K` é igual a um Kilobyte ou `1024` bytes. Essas notações abreviadas podem ser utilizadas no *php.ini* e na função ini_set(). Importante observar que os valores são truncados para int apenas, ou seja, `0.5M` será interpretado como `0`.

> **Nota** : **kilobyte versus kibibyte**
>
> A notação do PHP descreve um kilobyte equivalente a 1024 bytes, enquanto que o padrão IEC considera que este deve ser um kibibyte. Resumindo: k e K = 1024 bytes.

Hash de senhas

Esta seção explica as razões por trás do uso de funções de hash para proteger senhas, bem como fazê-lo de forma eficaz.

1. Por que deve ser feito hash das senhas fornecidas pelos usuários?
2. Por que as funções de hash mais comuns tais como md5 e sha1 são inadequadas para senhas?
3. Como as senhas devem ter seus hashes gerados, se as funções de hash comuns não são adequadas?
4. O que é um salt?
5. Como os salts são armazenados?

Por que deve ser feito hash das senhas fornecidas pelos usuários?

Gerar hash de senha é uma das considerações mais básicas de segurança que deve ser feita ao projetar qualquer aplicação ou serviço que aceite senhas dos usuários. Sem gerar hash, todas as senhas que são armazenadas podem ser roubadas se o armazenamento de dados for comprometido, e então imediatamente usadas para comprometer não só a aplicação ou serviço, mas também as contas dos usuários em outros serviços, se eles não usarem senhas únicas.

Pela aplicação de um algoritmo de hash para as senhas dos usuários antes de armazená-las, torna-se implausível para um atacante determinar a senha original, enquanto ainda é possível comparar o hash resultante com a senha original no futuro.

É importante observar, entretanto, que hash de senhas só as protegem de serem comprometidas no armazenamento de dados, mas não necessariamente as protegem de serem interceptadas por código malicioso injetado na aplicação ou serviço em si.

Por que as funções de hash mais comuns tais como md5() e sha1() são inadequadas para senhas?

Algoritmos de hash como MD5, SHA1 e SHA256 são projetados para serem muito rápidos e eficientes. Com técnicas e equipamentos de computação modernos, tornou-se trivial usar "força bruta" na saída desses algoritmos para determinar a entrada original.

Por causa da rapidez com que um computador moderno pode "reverter" esses algoritmos de hash, muitos profissionais de segurança da informação sugerem fortemente que esses algoritmos não sejam utilizados para hash de senhas.

Como as senhas devem ter seus hashes gerados, se as funções de hash comuns não são adequadas?

Ao gerar hash de senhas, as duas considerações mais importantes são o custo do cálculo e o salt. Quanto mais custoso computacionalmente for o algoritmo de hash, maior será o tempo para usar força bruta na sua saída.

O PHP fornece uma API de hash de senha nativo que lida de forma segura tanto a geração de hash quanto a verificação de senhas de forma segura.

O algoritmo sugerido para uso na geração de hash de senhas é o Blowfish, que também é o padrão usado pela API de hash da senha, pois é significativamente mais custoso computacionalmente que o MD5 ou que o SHA1, e ainda assim é escalável.

A função crypt() também está disponível para hash de senhas, mas ela só é recomendada para interoperabilidade com outros sistemas. No lugar dela, é fortemente encorajado o uso da API de hash de senha nativa sempre que possível.

O que é um salt?

Um salt de criptografia são dados que são aplicados durante o processo de hash para eliminar a possibilidade da saída ser pesquisada em uma lista de pares de hashes pré-calculados e suas entradas, conhecida como tabela arco-íris.

Em termos mais simples, um salt é uma pequena adição de dados, que faz com que os hashes sejam significativamente mais difíceis de serem decifrados. Há uma série de serviços na internet que fornecem listas extensas de hashes pré-computados, bem como a entrada original para esses hashes. O uso de um salt faz com que seja improvável ou impossível encontrar o hash resultante em uma destas listas.

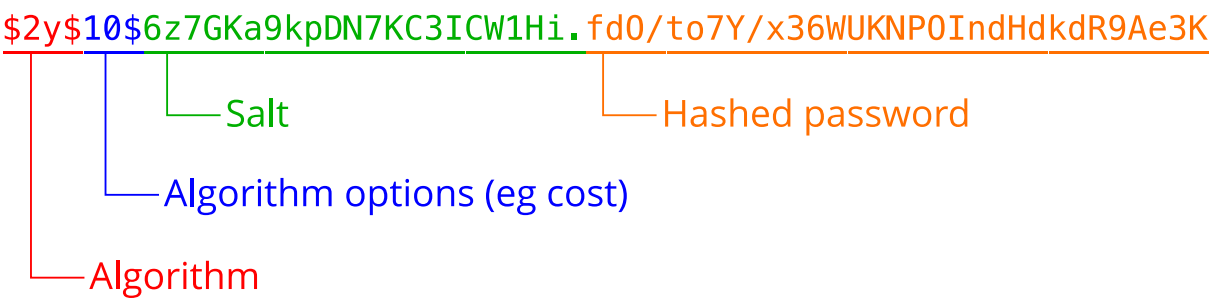
password_hash() criará um salt aleatório se um não for fornecido, e esta geralmente é a abordagem mais fácil e mais segura.

Como os salts são armazenados?

Ao usar password_hash() ou crypt(), o valor de retorno inclui o salt como parte do hash gerado. Este valor deve ser armazenado na íntegra no banco de dados, uma vez que inclui informações sobre a função de hash que foi usada e pode, então, ser passado diretamente às funções password_verify() ao verificar senhas.

> **Aviso**
>
> password_verify() deve sempre ser usada ao invés de gerar novamente o hash e comparar o resultado a um hash armazenado para evitar ataques de temporização.

O diagrama a seguir mostra o formato de um valor de retorno de crypt() ou password_hash(). Como pode ser visto, eles são auto-contidos, com todas as informações sobre o algoritmo e o salt necessárias para verificações futuras de senhas.



PHP e HTML

PHP e HTML possuem muita interação: PHP pode gerar HTML, e o HTML pode passar informações para o PHP. Antes de ler esses faqs, é importante que você aprenda como recuperar variáveis de fontes externas. A página do manual neste tópico inclui muitos exemplos.

1. Que codificação/decodificação é preciso quando eu passo um valor através de um formulário/URL?
2. Estou tentando usar uma tag <input type="image">, mas as variáveis $foo.x e $foo.y não estão disponíveis. $_GET['foo.x'] não existem também. Onde elas estão?
3. Como eu crio arrays em um HTML <form>?
4. Como eu faço para obter os resultados de uma múltipla seleção de uma tag HTML?
5. Como eu posso passar uma variável do Javascript para o PHP?

Que codificação/decodificação é preciso quando eu passo um valor através de um formulário/URL?

Existem muitos estágios para o qual a codificação é importante. Assumindo que você tenha uma string *$data*, o qual contém a string que você quer passar de um jeito não codificado, esses são os estágios relevantes:

- Interpretação HTML. A fim de especificar uma string aleatória, você *precisa* incluí-la entre aspas, e converter todo o valor dos caracteres especiais para a realidade HTML usando htmlspecialchars().
- URL: Uma URL consiste de diversas partes. Se você quer que seus dados sejam interpretados como um item, você *precisa* codificá-los com urlencode().

Exemplo #1 Um elemento oculto de formulário HTML

```
<?php
    echo "<input type='hidden' value='" . htmlspecialchars($data) . "' />\n";
?>
```

> **Nota** :
>
> É errado usar urlencode() com *$data*, porque é a responsabilidade do navegador fazer a função de urlencode() nos dados. Todos os navegadores populares fazem isso corretamente. Observe que isso acontecerá independentemente do método (ou seja, GET ou POST). Mesmo assim, isso só será notado em caso de solicitação GET, porque as requisições POST normalmente são escondidas.

Exemplo #2 Dados a serem editados pelo usuário

```php
<?php
    echo "<textarea name='mydata'>\n";
    echo htmlspecialchars($data)."\n";
    echo "</textarea>";
?>
```

> **Nota** :
>
> Os dados são mostrados no navegador como pretendido, porque o navegador irá interpretar os símbolos de escape do HTML.
>
> Em consequência do envio, tanto via GET como POST, os dados vão ser urlencoded pelo navegador para transferência, e urldecoded diretamente pelo PHP. Resumindo, você não precisa fazer qualquer urlencoded/urldecoded pelo php, tudo é feito automaticamente.

Exemplo #3 Em uma URL

```php
<?php
    echo "<a href='" . htmlspecialchars("/nextpage.php?stage=23&data=" .
        urlencode($data)) . "'>\n";
?>
```

> **Nota** :
>
> De fato você está fingindo uma requisição HTML GET, portanto é necessário urlencode() os dados manualmente.

> **Nota** :
>
> Você precisa htmlspecialchars() toda a URL, porque a URL ocorre como um valor de um atributo-HTML. Neste caso, o navegador irá primeiramente un-htmlspecialchars() o valor, e então passar a URL. O PHP vai entender a URL corretamente, porque você codificou os dados com urlencode().
>
> Você vai notar que `&` na URL é substituído por `&`. Embora a maioria dos navegadores vão se recuperar se vecê esquecer disso, isso nem sempre é possível. Então, mesmo se a URL não é dinâmica, você *precisa* htmlspecialchars() a URL.

Estou tentando usar uma tag <input type="image">, mas as variáveis *$foo.x* e *$foo.y* não estão disponíveis. *$_GET['foo.x']* não existem também. Onde elas estão?

Ao enviar um formulário, é possível usar uma imagem em vez de um botão submit padrão, como na tag:

```
<input type="image" src="image.gif" name="foo" />
```

Quando o usuário clica em qualquer lugar na imagem, o formulário que o acompanha será transmitido para o servidor com duas variáveis adicionais: *foo.x* e *foo.y*.

Porque *foo.x* e *foo.y* faria nome de variáveis inválidas no PHP, elas são automaticamente convertidas para *foo_x* e *foo_y*. Ou seja, os períodos são substituídos por sublinhados. Então, você teria acesso a essas variáveis como qualquer outro descrito na seção sobre como recuperar variáveis de fontes externas. Por exemplo, *$_GET['foo_x']*.

> **Nota** :
>
> Os espaços nos nomes das variáveis requisitadas são convertidas para sublinhado.

Como eu crio arrays em um HTML <form>?

Para obter o resultado do seu <form> enviado como um array para seu script PHP você nomeia os elementos <input>, <select> ou <textarea> como estes:

```
<input name="MyArray[]" />
<input name="MyArray[]" />
<input name="MyArray[]" />
<input name="MyArray[]" />
```

Observe os colchetes após o nome da variável, é isso que torna uma variável um array. Você pode agrupar os elementos dentro de diferentes arrays, atribuindo o mesmo nome a diferentes elementos:

```
<input name="MyArray[]" />
<input name="MyArray[]" />
<input name="MyOtherArray[]" />
<input name="MyOtherArray[]" />
```

Isso produz dois arrays, MyArray e MyOtherArray, que serão enviados para o script PHP. Também é possível atribuir chaves específicas para seus arrays:

```
<input name="AnotherArray[]" />
<input name="AnotherArray[]" />
<input name="AnotherArray[email]" />
<input name="AnotherArray[phone]" />
```

O array AnotherArray irá conter agora as chaves 0, 1, email e phone.

> **Nota** :
>
> Especificar uma chave de array é opcional no HTML. Se você não especificar as chaves, o array preenche na ordem em que os elementos aparecem no formulário. Nosso primeiro exemplo irá conter as chaves 0, 1, 2 e 3.

Veja também: Funções de Array e Variáveis de fontes Externas.

Como eu faço para obter os resultados de uma múltipla seleção de uma tag HTML?

A seleção de múltiplas tags em uma construção HTML permite aos usuários selecionar múltiplos itens de uma lista. Esses itens são então passados ao manipulador de ações para o formulário. O problema é que eles são todos passados com o mesmo nome widget. I.e.

```
<select name="var" multiple="yes">
```

Cada opção selecionada chegará ao manipulador de ações como:

```
var=option1
var=option2
var=option3
```

Cada opção irá substituir o conteúdo da variável anterior *$var*. A solução é usar a característica "array de elementos do formulário" do PHP. A seguir devem ser usados:

```
<select name="var[]" multiple="yes">
```

Isso diz ao PHP para tratar *$var* como um array e cada atribuição de um valor para [var] acrescenta um item ao array. O primeiro item se torna *$var[0]*, o próximo *$var[1]*, etc. A função count() pode ser chamado para determinar quantas opções foram selecionadas, e a função sort() pode ser usada para classificar o array de opções se necessário.

Note que se você está usando JavaScript o `[]` no nome do elemento pode lhe causar problemas quando você tenta se referir ao elemento pelo nome. Ao invés use o ID numérico do elemento do formulário, ou coloque o nome da variável entre aspas simples e use isso como um índice para os elementos do array, por exemplo:

```
variable = document.forms[0].elements['var[]'];
```

Como eu posso passar uma variável do Javascript para o PHP?

Uma vez que o Javascript é (normalmente) uma tecnologia da parte do cliente, e o PHP é (normalmente) uma tecnologia da parte do servidor, e uma vez que o HTTP é um protocolo "sem lado", as duas linguagens não podem compartilhar variáveis diretamente.

É , no entanto, possível passar variáveis entre as duas. Uma maneira de realizar isso é gerar um código Javascript com PHP, e ter um navegador que se atualize, passando variáveis específicas de volta para o script PHP. O exemplo abaixo mostra precisamente como fazer isso -- isso permite ao código PHP capturar a altura e largura da tela, algo que normalmente só é possível no lado do cliente.

Exemplo #4 Gerando Javascript com PHP

```
<?php
if (isset($_GET['width']) AND isset($_GET['height'])) {
  // exibe as variáveis da geometria
  echo "A largura da tela é: ". $_GET['width'] ."<br />\n";
  echo "A altura da tela é: ". $_GET['height'] ."<br />\n";
} else {
  // passar as variáveis de geometria
  // (preserve a string de consulta original
  //   -- variáveis POST precisam ser manipuladas de forma diferente)

  echo "<script language='javascript'>\n";
  echo "  location.href=\"{$_SERVER['SCRIPT_NAME']}?{$_SERVER['QUERY_STRING']}"
            . "&width=\" + screen.width + \"&height=\" + screen.height;\n";
  echo "</script>\n";
```

```
  exit();
}
?>
```

PHP e COM

PHP pode ser usado para acessar objetos COM e DCOM em plataformas Win32.

1. Eu construí uma DLL para calcular algo. Existe uma maneira de executar essa DLL no PHP?
2. O que 'Unsupported variant type: xxxx (0xxxxx)' significa?
3. É possível manipular objetos visuais no PHP?
4. Posso guardar um objeto COM em uma sessão?
5. Como eu capturo erros de COM?
6. Posso gerar arquivos DLL a partir de scripts PHP como eu posso no Perl?
7. O que significa 'Unable to obtain IDispatch interface for CLSID {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}'?
8. Como posso executar um objeto COM a partir de um servidor remoto?
9. Eu vejo o erro 'DCOM is disabled in C:\path...\scriptname.php on line 6', o que posso fazer?
10. É possível carregar/manipular um objeto ActiveX em uma página com PHP?
11. É possível obter uma instância em execução de um componente?
12. Existe alguma maneira de lidar com um evento enviado por um objeto COM?
13. Estou tendo problemas ao tentar invocar um método de um objeto COM que expõe mais de uma interface. O que posso fazer?
14. Então PHP funciona com COM. E com COM+?
15. Se o PHP pode manipular objetos COM, podemos imaginar o uso de MTS para manipular recursos de componentes, em conjunto com o PHP?

Eu construí uma DLL para calcular algo. Existe uma maneira de executar essa DLL no PHP?

Se for uma DLL simples, ainda não tem como executá-la no PHP. Se a DLL contiver um serviço COM, você pode acessá-lo se ele implementar a interface IDispatch.

O que 'Unsupported variant type: xxxx (0xxxxx)' significa?

Existem dezenas de tipos VARIANT e combinações dos mesmos. A maioria deles já é suportada mas alguns ainda têm que ser implementados. Arrays não são completamente suportados. Apenas arrays indexados com uma única dimensão podem ser passados entre o PHP e COM. Se você achar outros tipos que não são suportados, por favor relate-os como um problema (se não tiverem sido relatados ainda) e passe o máximo de informação disponível.

É possível manipular objetos visuais no PHP?

Geralmente é, mas como o PHP é principalmente usado como linguagem de script para web, ele é executado no contexto dos servidores web, então objetos visuais nunca aparecerão no desktop do servidor. Se você usa o PHP para script de aplicações (por exemplo, em conjunto com PHP-GTK) não há limitação no acesso e manipulação de objetos visuais através de COM.

Posso guardar um objeto COM em uma sessão?

Não, não pode. Instâncias COM são tratadas como recursos (tipo "resource") e, portanto, só estão disponíveis no contexto de um script único.

Como eu capturo erros de COM?

A extensão COM lança exceções do tipo `com_exception`, que você pode capturar e então inspecionar o membro `code` para determinar o que fazer em seguida.

Posso gerar arquivos DLL a partir de scripts PHP como eu posso no Perl?

Não, infelizmente não existe tal ferramenta disponível para o PHP.

O que significa 'Unable to obtain IDispatch interface for CLSID {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}'?

Esse erro pode ter várias razões:

- o CLSID está errado
- a DLL requisitada não existe ou não pode ser lida
- o componente requisitado não implementa a interface IDispatch

Como posso executar um objeto COM a partir de um servidor remoto?

Exatamente como você executa objetos locais. Você só tem que passar o IP da máquina remota como segundo parâmetro para o construtor COM.

Tenha certeza de marcar com.allow_dcom=**true** no seu *php.ini*.

Eu vejo o erro 'DCOM is disabled in C:\path...\scriptname.php on line 6', o que posso fazer?

Edite seu arquivo *php.ini* e marque com.allow_dcom=**true**.

É possível carregar/manipular um objeto ActiveX em uma página com PHP?

Isso não tem nada a ver com o PHP. Objetos ActiveX são carregados no lado do cliente se eles forem requisitados pelo documento HTML. Não há relação com o script PHP e, portanto, nenhuma possível interação direta com o lado do servidor.

É possível obter uma instância em execução de um componente?

É possível com a ajuda de monikers. Se você quiser obter referências múltiplas para a mesma instância do Word, você pode criar essa instância como mostrado abaixo:

```
<?php
$word = new COM("C:\docs\word.doc");
?>
```

Isso criará uma nova instância se não houver nenhuma instância em execução disponível e retornará um identificador para a instância em execução, se disponível.

Existe alguma maneira de lidar com um evento enviado por um objeto COM?

Você pode definir um interceptador de eventos e vinculá-lo usando com_event_sink(). Você pode usar com_print_typeinfo() para que o PHP gere um esqueleto para a classe interceptadora de eventos.

Estou tendo problemas ao tentar invocar um método de um objeto COM que expõe mais de uma interface. O que posso fazer?

A resposta é tão simples quanto insatisfatória. Não sabemos exatamente, mas achamos que não há nada que você possa fazer.

Então PHP funciona com COM. E com COM+?

COM+ estende COM por meio de um framework para manipular componentes através de MTS e MSMQ, mas não há nada de especial que o PHP precise suportar para usar tais componentes.

Se o PHP pode manipular objetos COM, podemos imaginar o uso de MTS para manipular recursos de componentes, em conjunto com o PHP?

O PHP em si não lida com transações ainda. Por isso, se um erro ocorrer, nenhuma reversão é iniciada. Se você usar componentes que suportam transações, você terá que implementar a manipulação de transações por conta própria.

Questões Diversas

Aqui estão algumas questões que nós não pudemos colocar em outra categoria. Aqui você encontra elas.

1. Como eu posso utilizar os manuais compactados com o bz2 no Windows?

Como eu posso utilizar os manuais compactados com o bz2 no Windows?

Se você não tiver uma ferramenta de arquivos para trabalhar com arquivos bz2 » download a ferramenta de linha de comando da Redhat (por favor veja maiores informações abaixo).

Se você não gostar de usar uma ferramenta de linha de comando, você pode tentar ferramentas gratuitas como » Stuffit Expander, » UltimateZip, » 7-Zip, ou » Quick Zip. Se você tiver ferramentas como » WinRAR ou » Power Archiver, você pode facilmente descomprimir arquivos bz2 com elas. Se você usa o Total Commander, (anteriormente Windows Commander), um plugin para o bz2 está disponível gratuitamente em » Total Commander.

A ferramenta de linha de comando bzip2 da Redhat:

Usuários Win2k Sp2 obtenha a versão 1.0.2, todos os outros usuários de windows a versão 1.00. Após o download renomeie o executável para bzip2.exe. Para sua conveniência, coloque em um diretório que esteja no seu path, ex. C:\Windows, onde C representa o drive que você instalou o Windows.

Nota: lang significa a sua língua(Português do Brasil: pt_BR) e x para o formato desejado, ex.: pdf. Para descompactar php_manual_lang.x.bz2 siga estas instruções simples:

- abra uma janela com o prompt de comando
- Use o comando cd para a pasta onde você guardou o php_manual_lang.x.bz2
- digite bzip2 -d php_manual_lang.x.bz2, descompactando php_manual_lang.x na mesma pasta

Caso você tenha feito o download de php_manual_lang.tar.bz2 com vários arquivos html nele, o procedimento é o mesmo. A única diferença é que você obtém um arquivo php_manual_lang.tar. O formato tar format é conhecido e tratado com as ferramentas de arquivos mais comuns para o Windows, por exemplo » WinZip.